



Cortex-A53

Android 底层及应用开发

FS6818 部分

(V1.1)

华清远见教育集团 • 研发中心



170-9108-5953



2306275952/2668462267



<http://dev.hqyj.com>



support@farsight.com.cn

华清远见
dev.hqvi.com



前 言

本书以 FS6818 开发平台为载体，系统介绍了 Android 底层以及应用的理论知识、环境搭建及综合应用实验，主要讲解 Android 系统底层和应用开发技术。Android 是典型的嵌入式操作系统，和传统的基于 PC 桌面的编程有很大差别，嵌入式系统的特点在于可定制性、可移植性、硬件相关性、功耗低，基于这些特点，本书针对特定硬件平台从零构建 Android 系统，包括平台移植、应用开发、内核开发及驱动开发。本书的特色如下：

(1) 重基础，适合教学。

(2) 重素质，全面讲解。本书在一般性教材的基础上，对嵌入式系统的软硬件开发环境进行了大量的讲解，可以让读者更进一步、更全面地了解嵌入式的开发过程。

(3) 重实践，与实际项目相结合。本书每个原理讲解都对应着丰富的实验内容，并且原理讲解和实验内容一一对应，使得每个知识都可以被深刻的理解运用，本书附带的光盘中给出了参考设计代码和文档。

(4) 重应用。书中的实例对时下经常使用的功能、设备、器材进行讲解和说明，力求教材所涉及的内容能紧跟行业实际应用的需要。

本书共分 29 个章节。

第 1 章：Android 5.1 底层开发环境配置实验。

第 2 章：Android 5.1 编译实验。

第 3 章：Android 5.1 镜像烧写实验。

第 4 章：Android 系统与嵌入式开发。

第 5 章：Android JNI 层解析。

第 6 章：Android HAL 层解析。

第 7 章：Android 驱动介绍。

第 8 章：Android LED 点灯实验。

第 9 章：Android UART 通讯实验。

第 10 章：Android 与传感器设计与实现

第 11 章：Android 应用基本概念。

第 12 章：Android 开发环境搭建。

第 13 章：程序设计基础。

第 14 章：Android 生命周期。

第 15 章：用户界面开发。

第 16 章：组件间通信。

第 17 章：数据存储与访问。

第 18 章：多媒体开发。

第 19 章：定位与地图。

第 20 章：Android 中的图形图像。

第 21 章：Android NDK 开发。



第 22 章：Android 网络编程。

第 23 章：eclipse 开发环境搭建实验。

第 24 章：Android 界面编程实验。

第 25 章：组件间通信实验。

第 26 章：数据存储实验。

第 27 章：多媒体实验。

第 28 章：Android 图形图像实验。

第 29 章：网络编程实验。

华清远见
dev.hqvi.com



目 录

前 言.....	I
目 录.....	i
第 1 章 Android 底层开发环境配置实验.....	- 1 -
1.1 华清远见开发环境简介	- 1 -
1.2 实验目的	- 2 -
1.3 实验原理	- 2 -
1.4 实验设备	- 2 -
1.5 实验步骤	- 2 -
1.5.1 检查主机配置.....	- 2 -
1.5.2 Intel Virtualization Technology（32 位操作系统）	- 3 -
1.5.3 安装 VMware Player.....	- 5 -
1.5.4 运行开发环境.....	- 12 -
第 2 章 Android 系统编译.....	- 27 -
2.1 源码准备	- 27 -
2.2 解压源码	- 28 -
2.3 编译源码	- 28 -
2.4 重点文件说明	- 30 -
第 3 章 Android 镜像烧写实验	- 31 -
3.1 开发板连接方法（当用户单独使用开发板时）	- 31 -
3.2 实验箱连接方法（当开发板和实验箱底板固定时）	- 32 -
3.3 设置串口调试工具.....	- 32 -
3.4 制作 SD 卡启动盘.....	- 35 -
3.4.1 SD 卡格式化分区	- 35 -
3.4.2 烧写 uboot 到 sd 卡上.....	- 38 -
3.5 环境配置	- 38 -
3.5.1 拷贝 Fastboot 工具.....	- 38 -
3.5.2 安装 Fastboot 驱动.....	- 40 -
3.5.3 安装 ADB 驱动	- 44 -
3.5.4 安装常见驱动提示错误	- 47 -
3.6 烧写系统	- 47 -
第 4 章 Android 系统与嵌入式开发.....	- 52 -
4.1 Android 与传统嵌入式 Linux 系统.....	- 52 -
4.2 嵌入式系统概述	- 52 -
4.2.1 嵌入式系统简介.....	- 52 -



4.2.2	嵌入式系统的特点	- 53 -
4.2.3	嵌入式系统的发展	- 53 -
4.3	嵌入式系统的组成	- 55 -
4.3.1	嵌入式系统的硬件组成	- 55 -
4.3.2	嵌入式系统的软件组成	- 56 -
4.4	嵌入式系统开发概述	- 57 -
4.4.1	交叉编译	- 58 -
4.4.2	交叉调试	- 59 -
4.5	Android 系统概述	- 61 -
4.6	Android 系统架构	- 61 -
4.6.1	Linux 内核 (Linux Kernel)	- 62 -
4.6.2	Android 核心库 (Libraries)	- 62 -
4.6.3	Android 运行时环境 (Android Runtime)	- 62 -
4.6.4	Android 应用程序框架 (Application Framework)	- 63 -
4.6.5	Android 应用程序和小部件	- 63 -
4.7	小结	- 63 -
第 5 章	Android JNI 层解析	- 64 -
5.1	JNI 介绍	- 64 -
5.2	JNI 原理	- 64 -
5.3	JNI 使用方法	- 65 -
5.4	小结	- 66 -
第 6 章	Android HAL 层解析	- 67 -
6.1	HAL 介绍	- 67 -
6.2	HAL 内容	- 68 -
6.2.1	HAL 主要的存储目录	- 68 -
6.2.2	两种 HAL 架构比较	- 68 -
6.3	mokid 工程代码下载与结构分析	- 69 -
6.3.1	mokid 项目概述	- 69 -
6.3.2	结构分析	- 69 -
6.3.3	第一种方法 直接调用 service 方法的实现过程	- 72 -
6.3.4	第二种方法——经过 Manager 调用 service	- 80 -
6.4	小结	- 84 -
第 7 章	Android 驱动介绍	- 85 -
7.1	Android 移植和驱动的关系	- 85 -
7.2	设备驱动程序	- 85 -



7.2.1	设备驱动概念	- 85 -
7.2.2	内核驱动程序	- 86 -
7.2.3	用户空间驱动程序	- 86 -
7.2.4	Linux 设备驱动概述	- 86 -
7.3	Linux 核心与 Android 驱动	- 88 -
7.3.1	Android 专用驱动	- 88 -
7.3.2	Android 使用的设备驱动	- 89 -
7.4	小结	- 95 -
第 8 章	Android LED 点灯实验	- 96 -
8.1	实验目的	- 96 -
8.2	实验平台	- 96 -
8.3	实验原理	- 96 -
8.4	实验步骤	- 97 -
8.4.1	底层代码编译	- 97 -
8.4.2	应用程序安装	- 98 -
8.5	实验现象	- 98 -
8.1	实验代码	- 100 -
8.1.1	led 灯实验应用层代码	- 100 -
8.1.2	LED 灯实验 jni 层代码	- 102 -
8.1.3	LED 灯实验 hal 层代码	- 106 -
第 9 章	Android UART 通讯实验	- 109 -
9.1	实验目的	- 109 -
9.2	实验原理	- 109 -
9.3	实验步骤	- 109 -
9.3.1	底层代码编译	- 109 -
9.3.2	应用程序安装	- 110 -
9.4	实验现象	- 110 -
9.5	实验代码	- 114 -
9.5.1	串口通讯实验应用层代码	- 114 -
9.5.2	串口通讯实验 jni 层代码	- 116 -
9.5.3	串口通讯实验 hal 层代码	- 120 -
第 10 章	Android 与传感器设计与实现	- 125 -
10.1	Android Sensor 实现原理概述	- 125 -
10.2	JNI 原理	- 125 -
10.3	Android HAL	- 126 -



10.4	Sensor 传感器系统	- 126 -
10.4.1	Sensor Stub 分析	- 127 -
10.5	Logcat 调试	- 134 -
10.6	小结	- 135 -
第 11 章	Android 应用基本概念	- 137 -
11.1	Android 简介	- 137 -
11.2	Android 平台特性	- 138 -
11.3	Android 开发框架	- 139 -
11.3.1	应用方面	- 139 -
11.3.2	数据存储	- 141 -
11.3.3	网络访问方面	- 141 -
11.3.4	开发流程	- 142 -
11.4	本章小结	- 143 -
第 12 章	Android 应用开发环境搭建	- 144 -
12.1	Android 开发环境的安装与配置	- 144 -
12.1.1	安装 JDK 和配置 Java 开发环境	- 144 -
12.1.2	安装 7Zip 压缩软件	- 145 -
12.1.3	解压 adt-bundle-windows	- 146 -
12.1.4	运行 Eclipse	- 147 -
12.1.5	配置 SDK	- 147 -
12.1.6	配置 ADT	- 150 -
12.2	创建第一个 Android 应用	- 153 -
12.3	在模拟器上运行程序	- 157 -
12.4	在目标开发平台上运行程序	- 159 -
12.5	导入一个已有工程	- 162 -
12.6	可能遇到的错误及解决方法	- 166 -
12.7	本章小结	- 167 -
第 13 章	程序设计基础	- 168 -
13.1	Android 程序框架	- 168 -
13.1.1	Android 项目目录结构	- 168 -
13.1.2	Android 应用解析	- 173 -
13.2	Android 程序 UI 设计	- 179 -
13.3	Java 语言在 Android 程序中的使用	- 180 -
13.3.1	Interface 的使用	- 180 -
13.3.2	abstract class 的使用	- 181 -



13.3.3	Interface 与 abstract class 的区别	- 182 -
13.3.4	for 循环的使用	- 183 -
13.3.5	Map 类的使用	- 183 -
13.3.6	Integer 与 String 之间的转换	- 185 -
13.3.7	synchronized 同步关键字	- 185 -
13.3.8	new 的使用	- 186 -
13.4	本章小结	- 187 -
第 14 章	Android 生命周期	- 188 -
14.1	程序生命周期	- 188 -
14.2	Android 组件	- 189 -
14.3	Activity 生命周期	- 190 -
14.3.1	全生命周期	- 193 -
14.3.2	可视生命周期	- 194 -
14.3.3	活动生命周期	- 194 -
14.4	Activity 启动模式	- 195 -
14.4.1	standard 标准模式	- 201 -
14.4.2	singleTop	- 202 -
14.4.3	singleTask	- 203 -
14.4.4	singleInstance	- 204 -
14.5	程序调试	- 206 -
14.5.1	LogCat 命令行工具	- 206 -
14.5.2	DevTools 开发调试工具	- 209 -
14.6	本章小结	- 212 -
第 15 章	用户界面开发	- 213 -
15.1	用户界面基础	- 213 -
15.2	界面布局	- 215 -
15.2.1	线性布局 (LinearLayout)	- 216 -
15.2.2	框架布局 (FrameLayout)	- 220 -
15.2.3	表格布局 (TableLayout)	- 223 -
15.2.4	相对布局 (RelativeLayout)	- 226 -
15.2.5	绝对布局 (AbsoluteLayout)	- 229 -
15.3	界面控件	- 231 -
15.3.1	TextView 和 EditText	- 231 -
15.3.2	Button 和 ImageButton	- 233 -
15.3.3	CheckBox 和 RadioButton	- 236 -



15.3.4	Spinner.....	- 238 -
15.3.5	ListView	- 240 -
15.3.6	TabHost.....	- 242 -
15.4	菜单.....	- 244 -
15.4.1	选项菜单.....	- 244 -
15.4.2	子菜单	- 247 -
15.4.3	上下文菜单 (Context Menu)	- 248 -
15.5	界面事件	- 252 -
15.5.1	按键事件.....	- 252 -
15.5.2	触摸事件.....	- 255 -
15.6	自定义样式和主题.....	- 260 -
15.7	9Patch.....	- 262 -
15.8	本章小结	- 263 -
第 16 章	组件间通信	- 264 -
16.1	Intent 对象及其属性	- 264 -
16.1.1	Intent 的 action 属性.....	- 264 -
16.1.2	Intent 的 data 属性.....	- 265 -
16.1.3	Intent 的 type 属性.....	- 265 -
16.1.4	Intent 的 category 属性.....	- 266 -
16.1.5	Intent 的 extras 属性	- 266 -
16.1.6	Intent 的 ComponentName 属性.....	- 266 -
16.2	系统标准 ActivityAction 应用	- 267 -
16.2.1	启动 Activity.....	- 267 -
16.2.2	获取 Activity 返回值	- 273 -
16.3	Intent 过滤器.....	- 278 -
16.3.1	注册 Intent 过滤器	- 278 -
16.3.2	Intent 解析.....	- 280 -
16.4	广播消息	- 281 -
16.5	本章小结	- 283 -
第 17 章	数据存储与访问	- 284 -
17.1	简单存储	- 284 -
17.1.1	SharedPreferences.....	- 284 -
17.1.2	示例	- 285 -
17.2	文件存储	- 289 -
17.2.1	内部存储	- 289 -



17.2.2	外部存储	- 293 -
17.2.3	资源文件	- 296 -
17.3	数据库存储	- 300 -
17.3.1	SQLite 简介	- 300 -
17.3.2	手动建立数据库	- 302 -
17.3.3	代码建库 (SQLiteOpenHelper)	- 307 -
17.3.4	数据操作 (SQLiteDatabase)	- 311 -
17.4	数据共享(ContentProviders)	- 316 -
17.4.1	创建 ContentProvider	- 318 -
17.4.2	使用数据提供者	- 322 -
17.5	本章小结	- 324 -
第 18 章	多媒体开发	- 325 -
18.1	Android 系统提供内置的媒体格式	- 325 -
18.2	在窗体布局上——VideoView	- 326 -
18.3	音频及视频播放——MediaPlayer	- 327 -
18.3.1	从源文件中播放	- 330 -
18.3.2	从文件系统中播放	- 330 -
18.3.3	从网络中播放	- 332 -
18.3.4	音频播放示例	- 335 -
18.3.5	视频播放示例	- 337 -
18.4	音/视频的录制	- 343 -
18.5	Camera 照相	- 350 -
18.6	本章小结	- 355 -
第 19 章	定位与地图	- 356 -
19.1	位置服务	- 356 -
19.1.1	Android Location API	- 356 -
19.1.2	GPS 定位步骤	- 357 -
第 20 章	Android 中的图形图像	- 364 -
20.1	在 Android 中访问图片	- 364 -
20.1.1	使用图片文件创建 Drawable 对象	- 364 -
20.1.2	使用 XML 文件定义 Drawable 属性	- 366 -
20.1.3	Bitmap 和 BitmapFactory	- 367 -
20.2	Android 中的动画	- 368 -
20.2.1	Tween 动画	- 369 -
20.2.2	Frame 动画	- 379 -



20.3	动态图形绘制	- 383 -
20.3.1	动态图形绘制类简介	- 383 -
20.3.2	动态图形绘制的基本思路	- 385 -
20.3.3	绘制几何图形	- 388 -
20.4	图形特效	- 393 -
20.4.1	使用 Matrix 实现旋转、缩放和平移	- 393 -
20.4.2	使用 Shader 类渲染图形	- 397 -
20.5	本章小结	- 401 -
第 21 章	Android NDK 开发	- 403 -
21.1	Android NDK 简介	- 403 -
21.2	NDK 编译环境	- 404 -
21.3	NDK 开发示例	- 408 -
21.4	本章小结	- 411 -
第 22 章	Android 网络编程	- 412 -
22.1	Android 网络通信基础	- 412 -
22.1.1	标准 Java 接口	- 412 -
22.1.2	Apache 接口	- 413 -
22.1.3	Android 网络接口	- 414 -
22.2	HTTP 通信	- 415 -
22.2.1	URLConnection 接口	- 415 -
22.2.2	HttpClient 接口	- 420 -
22.3	Socket 通信	- 423 -
22.3.1	Socket 传输模式	- 424 -
22.3.2	Socket 编程原理	- 424 -
22.3.3	Socket 编程实例	- 426 -
22.3.4	Socket 与 HTTP 通信的区别	- 431 -
22.3.5	本章小结	- 431 -
22.4	框架布局实验	- 433 -
22.4.1	实验目的	- 433 -
22.4.2	实验平台	- 433 -
22.4.3	实验原理	- 433 -
22.4.4	实验步骤	- 433 -
22.4.5	实验代码	- 434 -
22.5	表格布局实验	- 435 -
22.5.1	实验目的	- 435 -



22.5.2	实验平台	- 435 -
22.5.3	实验原理	- 435 -
22.5.4	实验步骤	- 435 -
22.5.5	实验代码	- 436 -
22.6	相对布局实验	- 437 -
22.6.1	实验目的	- 437 -
22.6.2	实验平台	- 437 -
22.6.3	实验原理	- 437 -
22.6.4	实验步骤	- 437 -
22.6.5	实验代码	- 438 -
22.7	TextView 实验	- 439 -
22.7.1	实验目的	- 439 -
22.7.2	实验平台	- 439 -
22.7.3	实验原理	- 439 -
22.7.4	实验步骤	- 439 -
22.7.5	实验代码	- 440 -
22.8	EditText 实验	- 441 -
22.8.1	实验目的	- 441 -
22.8.2	实验平台	- 441 -
22.8.3	实验原理	- 441 -
22.8.4	实验步骤	- 441 -
22.8.5	实验代码	- 442 -
22.9	Button 实验	- 443 -
22.9.1	实验目的	- 443 -
22.9.2	实验平台	- 443 -
22.9.3	实验原理	- 443 -
22.9.4	实验步骤	- 443 -
22.9.5	实验代码	- 444 -
22.10	ImageButton 实验	- 445 -
22.10.1	实验目的	- 445 -
22.10.2	实验平台	- 445 -
22.10.3	实验原理	- 445 -
22.10.4	实验步骤	- 445 -
22.10.5	实验代码	- 446 -
22.11	日期选择器实验	- 446 -



22.11.1	实验目的	- 446 -
22.11.2	实验平台	- 446 -
22.11.3	实验原理	- 447 -
22.11.4	实验步骤	- 447 -
22.11.5	实验代码	- 447 -
22.12	CheckBox 实验	- 449 -
22.12.1	实验目的	- 449 -
22.12.2	实验平台	- 449 -
22.12.3	实验原理	- 449 -
22.12.4	实验步骤	- 449 -
22.12.5	实验代码	- 450 -
22.13	RadioButton 实验	- 451 -
22.13.1	实验目的	- 451 -
22.13.2	实验平台	- 451 -
22.13.3	实验原理	- 451 -
22.13.4	实验步骤	- 451 -
22.13.5	实验代码	- 452 -
22.14	Spinner 实验	- 453 -
22.14.1	实验目的	- 453 -
22.14.2	实验平台	- 453 -
22.14.3	实验原理	- 453 -
22.14.4	实验步骤	- 453 -
22.14.5	实验代码	- 454 -
22.15	ListView 实验	- 455 -
22.15.1	实验目的	- 455 -
22.15.2	实验平台	- 456 -
22.15.3	实验原理	- 456 -
22.15.4	实验步骤	- 456 -
22.15.5	实验代码	- 457 -
22.16	选项菜单实验	- 459 -
22.16.1	实验目的	- 459 -
22.16.2	实验平台	- 459 -
22.16.3	实验原理	- 459 -
22.16.4	实验步骤	- 460 -
22.16.5	实验代码	- 461 -



22.17	子菜单实验	- 461 -
22.17.1	实验目的	- 461 -
22.17.2	实验平台	- 461 -
22.17.3	实验原理	- 461 -
22.17.4	实验步骤	- 462 -
22.17.5	实验代码	- 463 -
22.18	上下文菜单实验	- 464 -
22.18.1	实验目的	- 464 -
22.18.2	实验平台	- 464 -
22.18.3	实验原理	- 464 -
22.18.4	实验步骤	- 465 -
22.18.5	实验代码	- 466 -
第 23 章	组件间通讯实验	- 468 -
23.1	实验目的	- 468 -
23.2	实验平台	- 468 -
23.3	实验原理	- 468 -
23.4	实验步骤	- 468 -
23.5	实验代码	- 469 -
第 24 章	数据存储实验	- 471 -
24.1	SharedPreferences 实验	- 471 -
24.1.1	实验目的	- 471 -
24.1.2	实验平台	- 471 -
24.1.3	实验原理	- 471 -
24.1.4	实验步骤	- 471 -
24.1.5	实验代码	- 472 -
24.2	文件存储实验	- 473 -
24.2.1	实验目的	- 473 -
24.2.2	实验平台	- 473 -
24.2.3	实验原理	- 473 -
24.2.4	实验步骤	- 473 -
24.2.5	实验代码	- 474 -
24.3	SQLite 数据库实验	- 475 -
24.3.1	实验目的	- 475 -
24.3.2	实验平台	- 475 -
24.3.3	实验原理	- 475 -



24.3.4	实验步骤	- 475 -
24.3.5	实验代码	- 476 -
24.4	数据共享实验	- 479 -
24.4.1	实验目的	- 479 -
24.4.2	实验平台	- 479 -
24.4.3	实验原理	- 480 -
24.4.4	实验步骤	- 480 -
24.4.5	实验代码	- 481 -
第 25 章	多媒体实验	- 482 -
25.1	实验目的	- 482 -
25.2	实验平台	- 482 -
25.3	实验原理	- 482 -
25.4	实验步骤	- 482 -
25.5	实验代码	- 483 -
第 26 章	Android 图形图像实验	- 485 -
26.1	实验目的	- 485 -
26.2	实验平台	- 485 -
26.3	实验原理	- 485 -
26.4	实验步骤	- 485 -
26.5	实验代码	- 486 -
第 27 章	网络编程实验	- 488 -
27.1	实验目的	- 488 -
27.2	实验平台	- 488 -
27.3	实验原理	- 488 -
27.4	实验步骤	- 488 -
27.5	实验代码	- 489 -



华清远见
dev.hqvi.com



底层部分



第 1 章 Android 底层开发环境配置实验

编译 linux 内核和 android 文件系统，推荐使用华清远见开发环境。

1.1 华清远见开发环境简介

华清远见开发环境是基于 Ubuntu 12.04 LTS 64-bit 操作系统搭建的，使用 [VMware Player](#)（**免费版**）作为虚拟机工具软件（用户也可以使用 VMware 公司所提供的**付费版**虚拟机软件 [VMware Workstation](#) 代替 VMware Player）。本开发环境可用作嵌入式 Linux 和 Android 的编译与开发。

本开发环境在 Ubuntu 12.04 64-bit LTS 基础上，安装了编译调试 Bootloader、Linux、Android 系统所需要的工具和依赖的库，用户可以在无需额外操作的基础上，直接使用本开发环境进行嵌入式的学习和工作。

本开发环境在 Ubuntu 12.04 64-bit 基础上，安装配置了如下工具：

- 将 GCC、G++编译器版本从 4.6 降至 4.4；
- 安装了 Android 编译所需要的工具和库（[source.android.com](#)）；
- 安装 SUN JAVA JDK 6；
- 安装 SUN JAVA JDK 7
- 安装内核编译所依赖的工具包；
- 解决了 libncurses 32 位和 64 位不能同时安装导致编译 Android 和配置内核软件冲突的问题；
- 安装制作安卓文件系统 yaffs2 格式 mkyaffs 工具；
- 添加了常用的 arm-linux 交叉工具链，版本号为 4.3.2、4.4.6、4.5.1；
- 安装 Vim、Ctags；
- 安装 Vim 常用插件；
- 安装配置 TFTP；
- 安装配置 NFS 网络文件系统服务；
- 安装 SSH 工具网络服务程序；
- 安装 Krmit 串口调试工具；
- 安装 Sogou 输入法；
- 关闭 Ubuntu 更新提示；

说明 1：Ubuntu 用户名为“linux”，主机名为“ubuntu64”，默认密码为“1”

说明 2：以上安装过的软件和库，用户在不明确的前提下，切勿再次安装！（比如 [source.android.com](#) 要求的、tftp 服务等等），如果因为重复安装导致源码编译错误，请重新解压开发环境镜像。

下图是使用华清远见开发环境编译源码：



1.2 实验目的

熟悉华清远见开发环境，为编译 linux 内核和 android 文件系统和 android 底层开发做准备。

1.3 实验原理

Google 推荐使用 64 位的 Ubuntu12.04 作为主机编译 android，可以使用 VMware 虚拟机软件，虽然有些慢，但对于普通用户开发方便一些，内存要求至少 1G，硬盘至少 30G，编译需要 jdk 的支持，由于是采用 64 位虚拟机编译，对于 32 位系统，应该在 BIOS 中打开 VT（虚拟化技术），而且还要下载 android 编译所需的软件包。

1.4 实验设备

硬件设备：

pc 机（安装有 windows 操作系统）。

软件工具：

VMware player 虚拟机下的 ubuntu 12.04 开发环境。

1.5 实验步骤

1.5.1 检查主机配置

华清远见开发环境是基于 Ubuntu 12.04 LTS 64-bit 操作系统搭建的，使用 VMware Player 免费版作为虚拟机工具软件。用作 Linux 和 Android 的编译与开发。所以建议开发主机硬件配置越高越好，配置越高则开发效率则越高。

配置	参数
CPU	Intel/AMD CPU; 主频 2GHz 或者更高;



	双核或者更多； 支持 EM64T (INTEL)或者 X86_64(AMD)指令； 支持 Intel Virtualization Technology
内存	内存 2G 以上（Windows XP / 7 / 8 / 10 32 位）； 内存 4G 以上（Windows 7 / 8 / 10 64 位）；
硬盘	虚拟机存放硬盘分区剩余空间 80G 以上； 推荐使用虚拟机挂载物理 ext 分区（无 Linux 基础者慎用）
USB	支持 USB2.0 或者更高； 2 路或者更多；
网卡	至少一路以太网卡；
串口	RS232 串口（也可以用 USB 转串口替代）
操作系统	Windows XP 或者更新； 32 位/64 位操作系统；

1.5.2 Intel Virtualization Technology（32 位操作系统）

华清远见开发环境是基于 Ubuntu 12.04 LTS 64-bit 操作系统搭建的，使用 VMware Player 作为虚拟机工具软件。所以在如果主机为 32 位操作系统，那么系统必须支持英特尔虚拟化技术（Intel Virtualization Technology）才可以通过 VMware Player 工具使用 64 位操作系统。

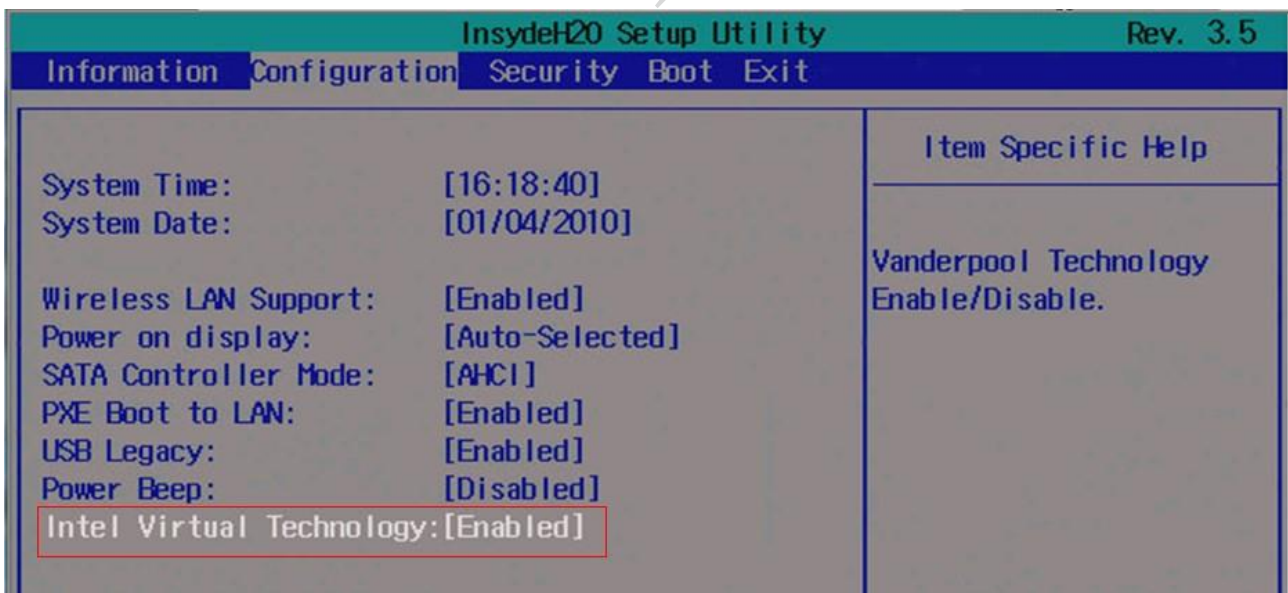
【检查主机 CPU 是否支持 Intel Virtualization Technology】

如下图所示，用户可以使用光盘目录下中的 CPU-Z 软件检查自己的 CPU 是否支持虚拟机化技术。



【在 BIOS 中打开 Intel Virtualization Technology】

如果 CPU 支持虚拟化技术，请在 BIOS 设置里面打开（不同型号的主板 BIOS 界面可能不同，如果找不到可以自行搜索“机型+BIOS+VT”的关键字）。





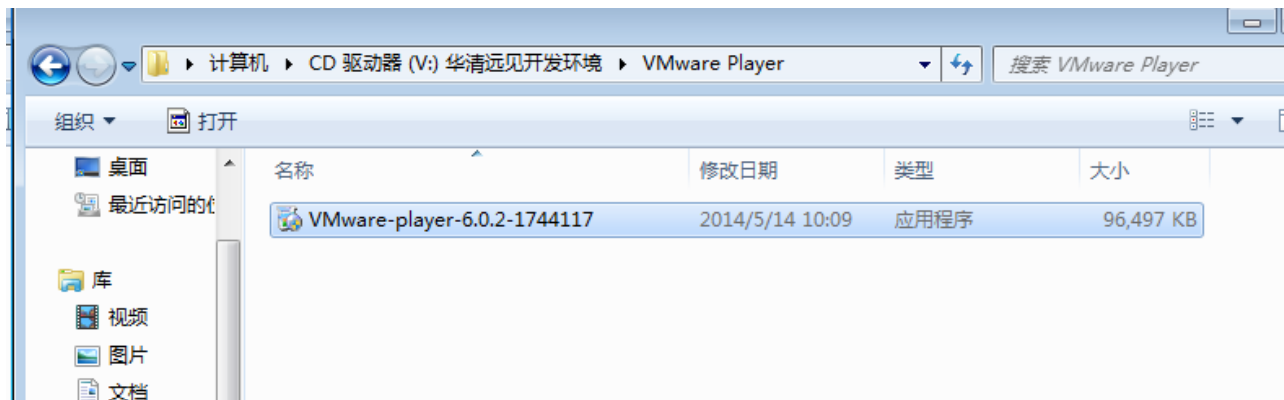
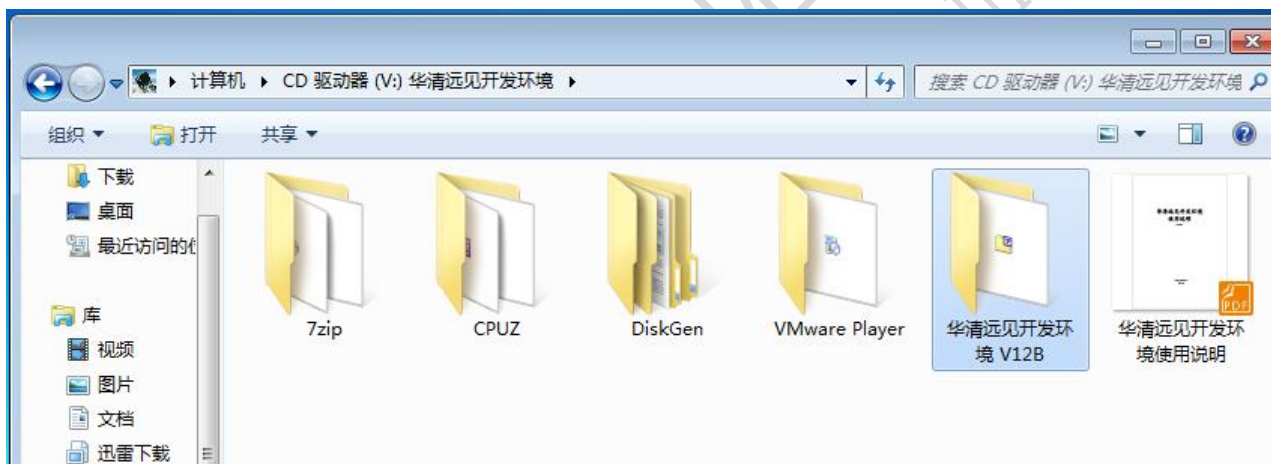
1.5.3 安装 VMware Player

VMware Player 从 6.0 版本之后默认支持中文，所以华清远见开发环境 V12B 使用当前最新版的 VMware Player（版本号为 6.0.2 build-1744117），如要正常使用此开发环境，必须保证 VMware Player 版本号大于等于当前给出的版本号，否则可能会出现因为 VMware Tools 版本过高引起虚拟机无法正常启动的情况。

（如果用户使用 VMware Workstation，版本号应该大于等于（10.0.1-1379776），否则可能会出现因为 VMware Tools 版本过高引起虚拟机无法正常启动的情况。）

【打开 VMware Player 安装程序】

打开（华清远见开发环境光盘：Linux 开发环境\VMware Player）目录下的 VMware Player 安装程序（VMware-player-6.0.2-1744117.exe）。





【VMware Player 安装程序初始化】

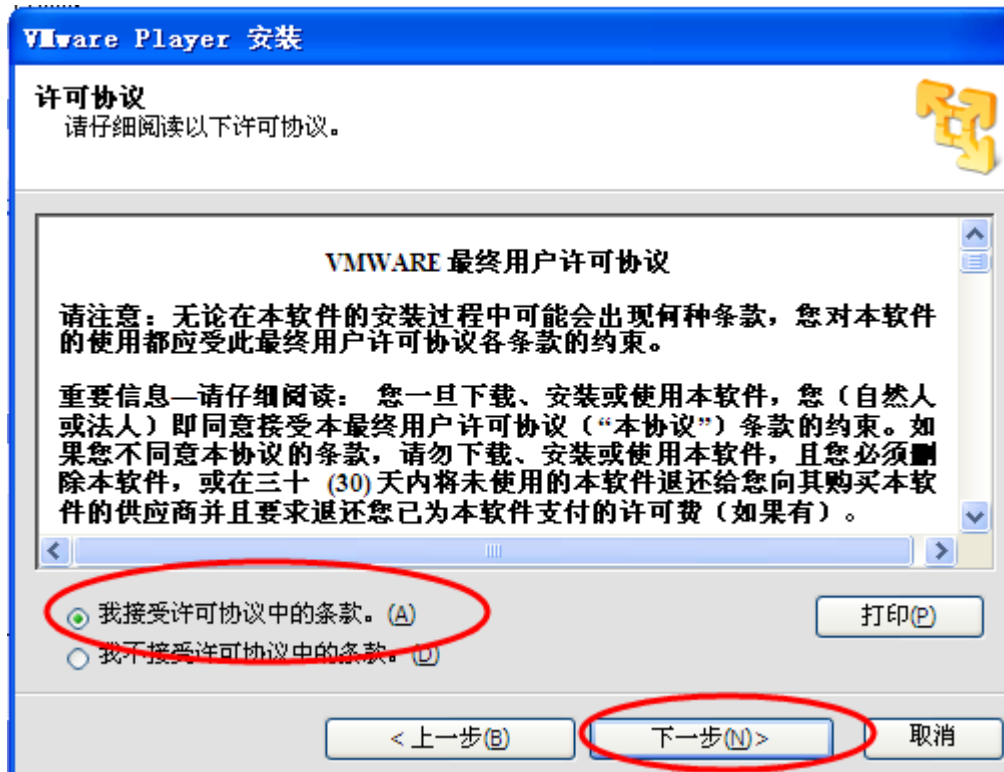


【VMware Player 安装向导】

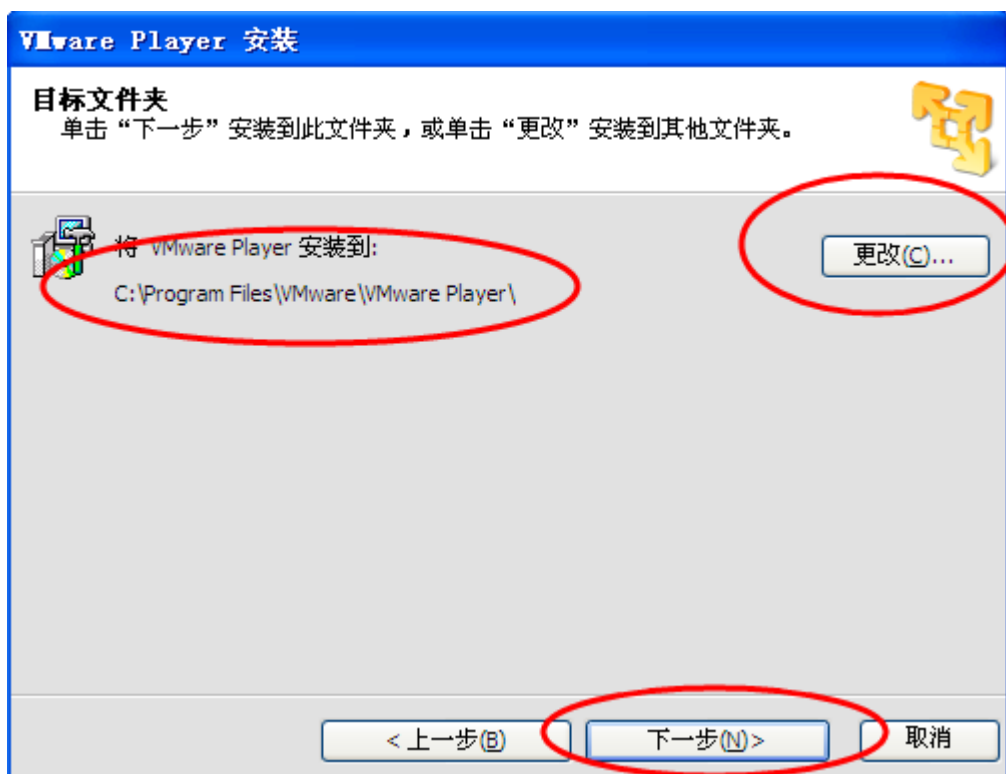




【VMware Player 许可协议】



【更改安装路径】

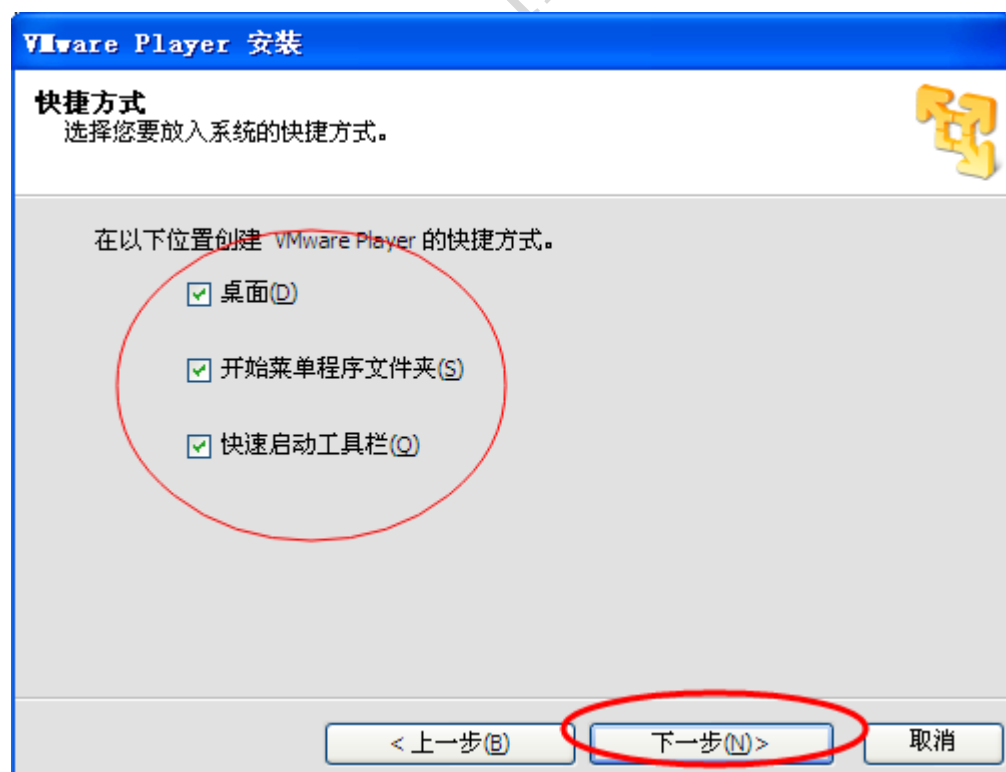


【选择是否检查更新，加入用户体验改进计划】





【创建快捷方式】

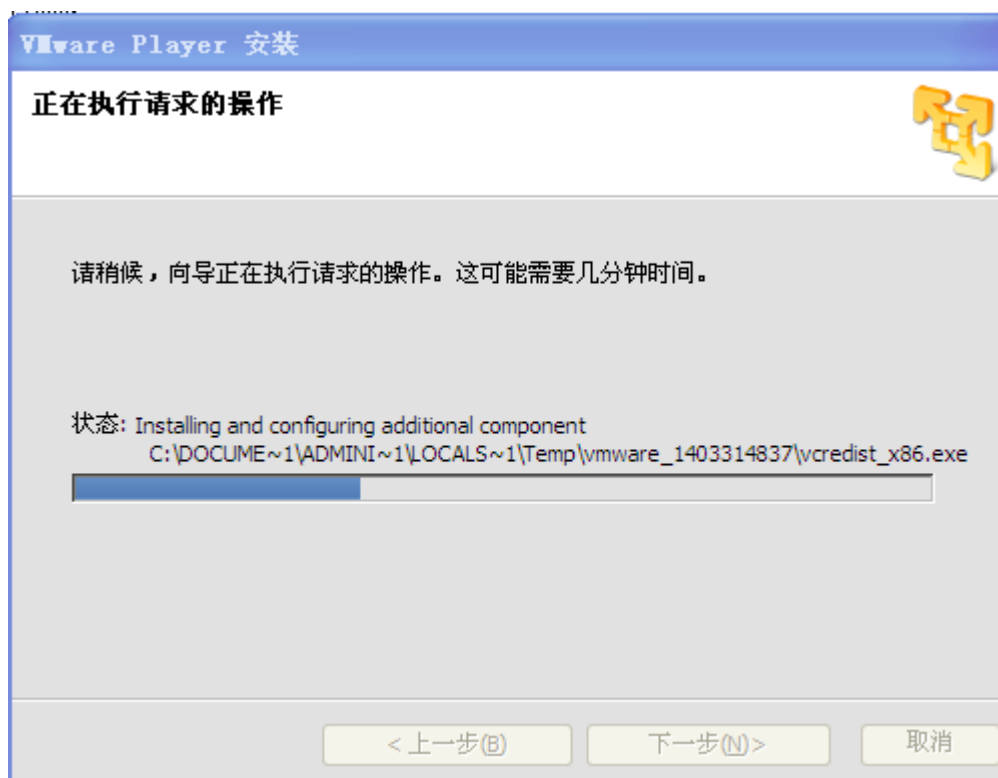




【确认安装】



【正在安装】



【安装完成】



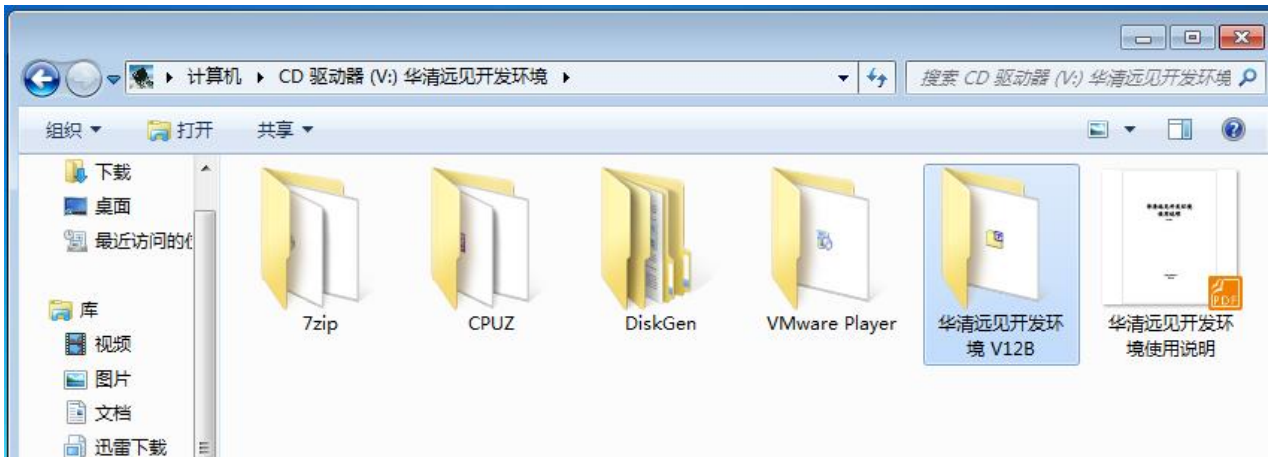


1.5.4 运行开发环境

解压虚拟机镜像

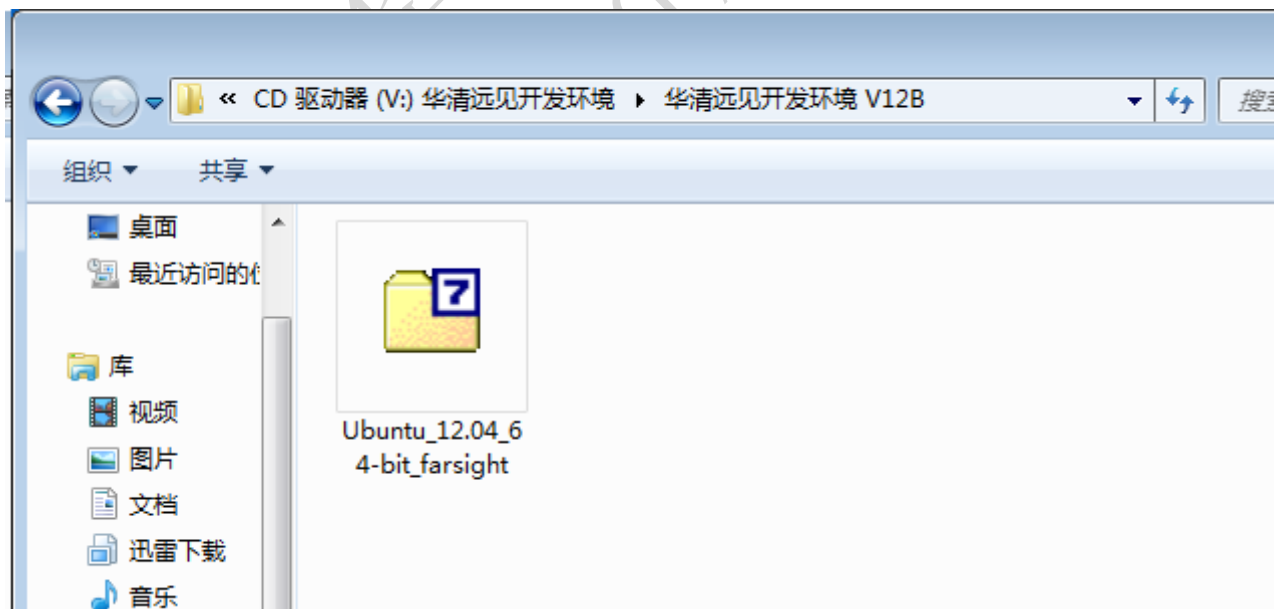
【打开华清远见开发环境】

打开华清远见开发环境光盘“Linux 开发环境\华清远见开发环境 V12B”目录。

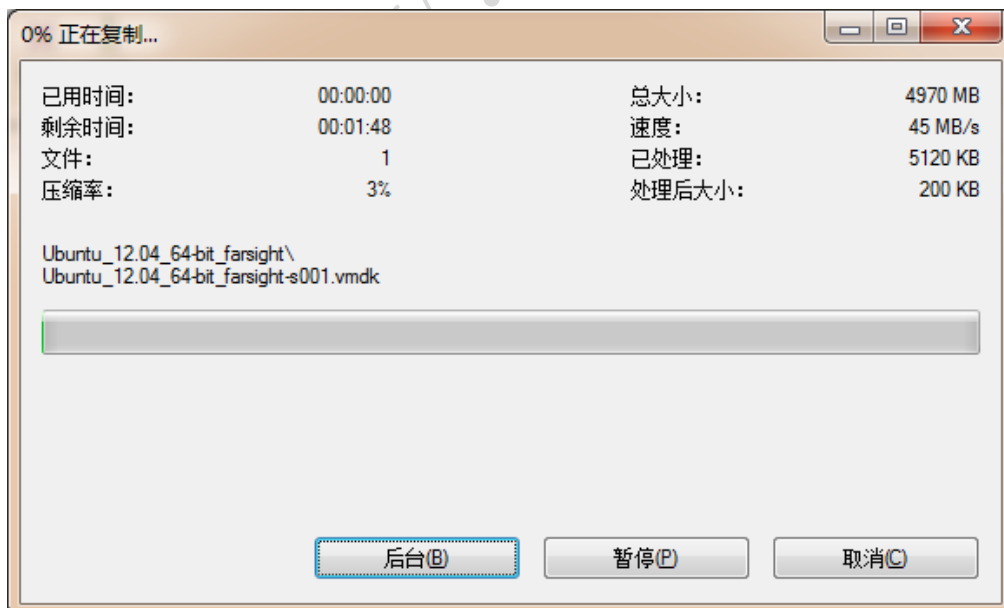
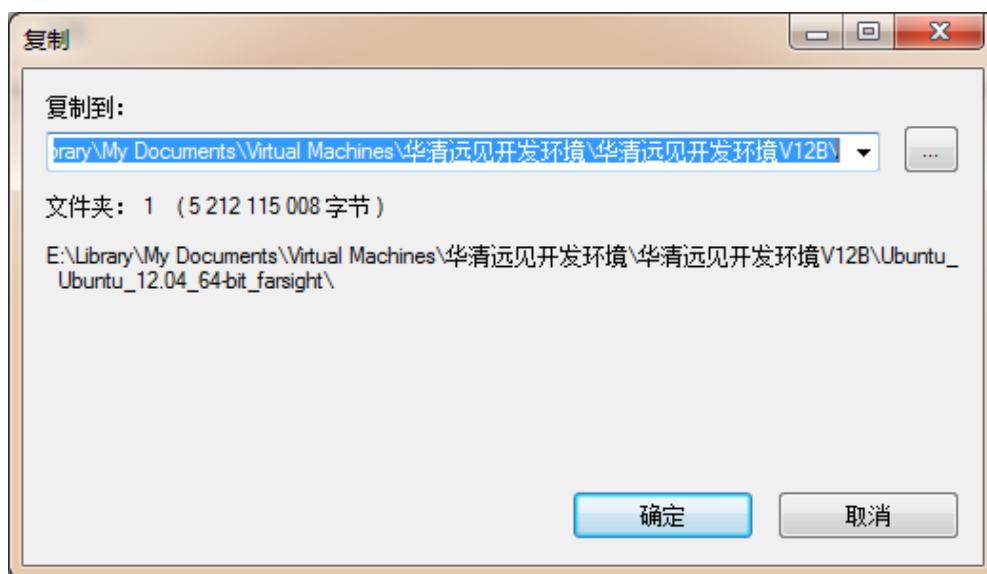
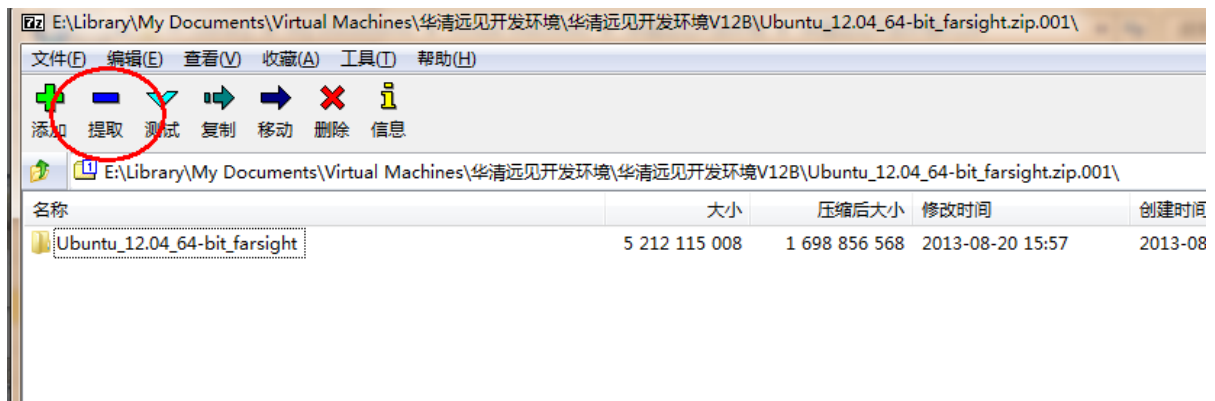


【打开华清远见开发环境压缩文件】

此处建议使用光盘下的 7zip 压缩软件解压。



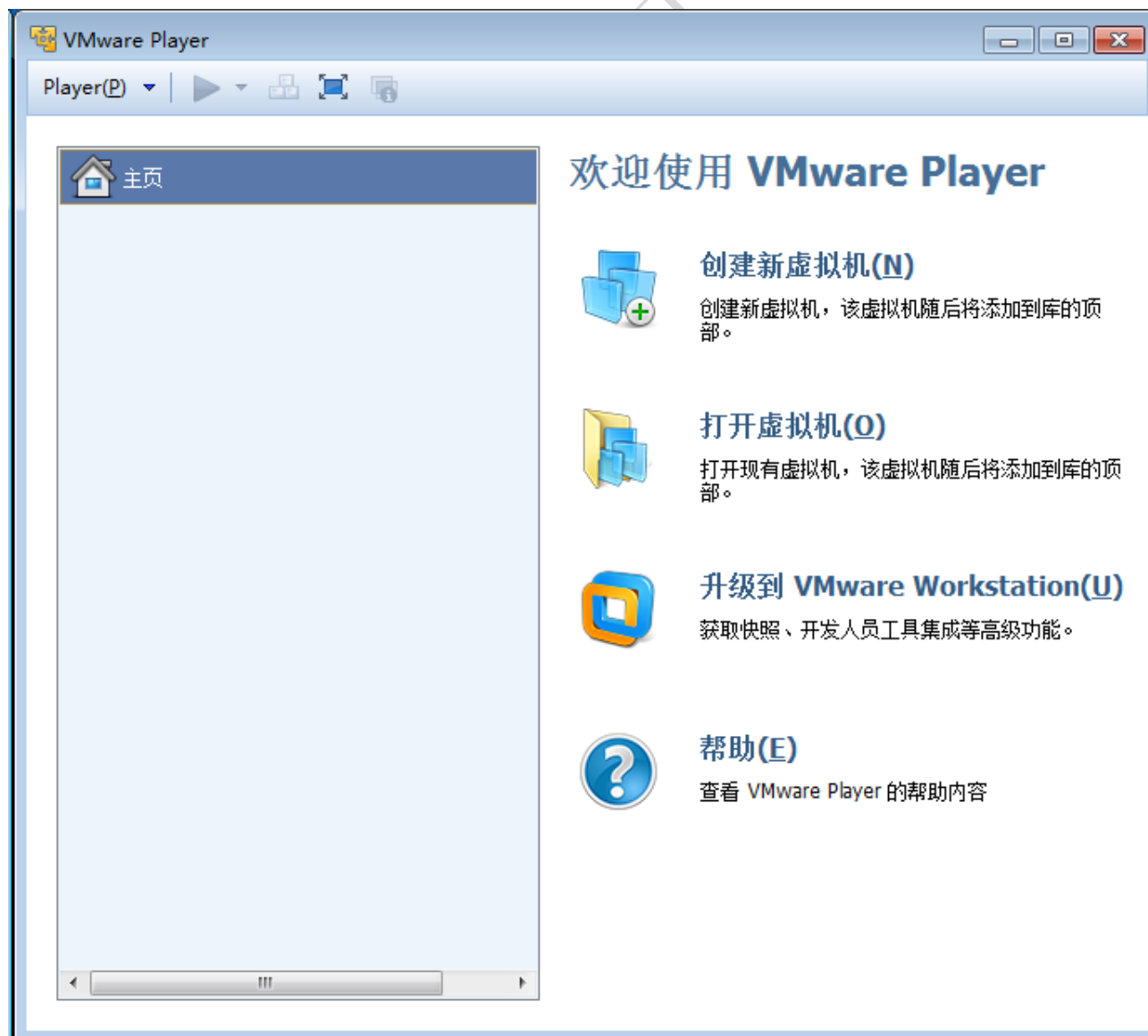
【解压华清远见开发环境】



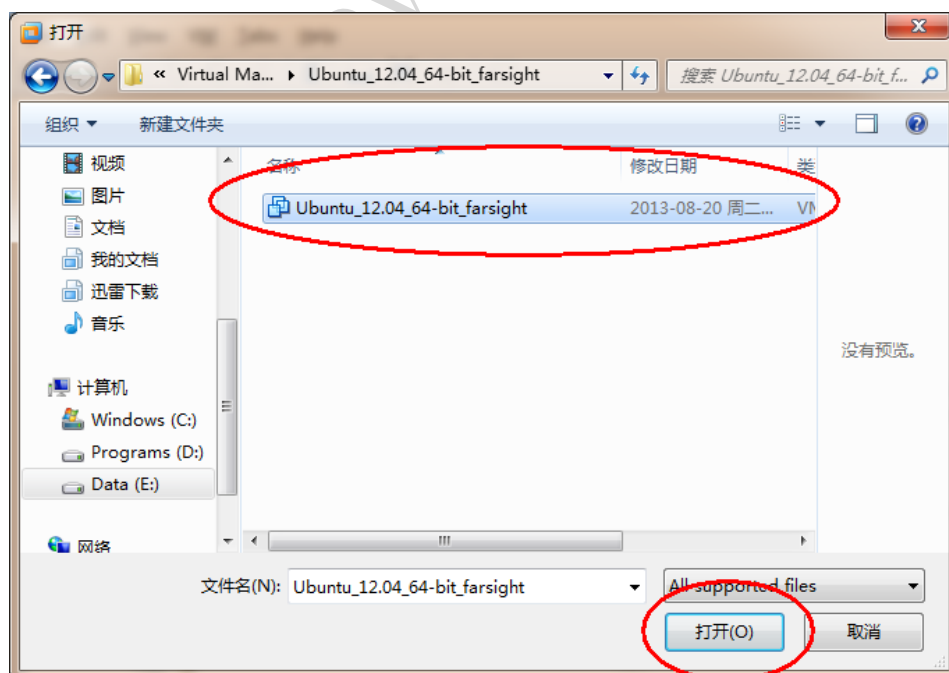
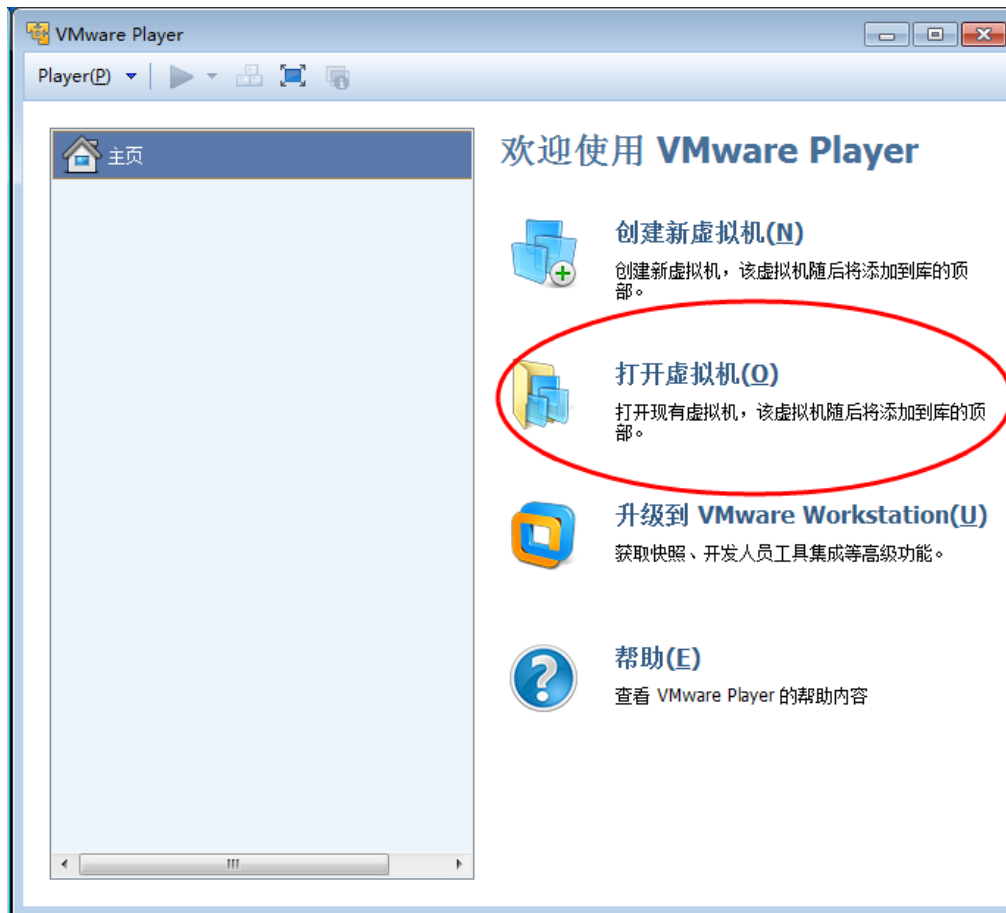


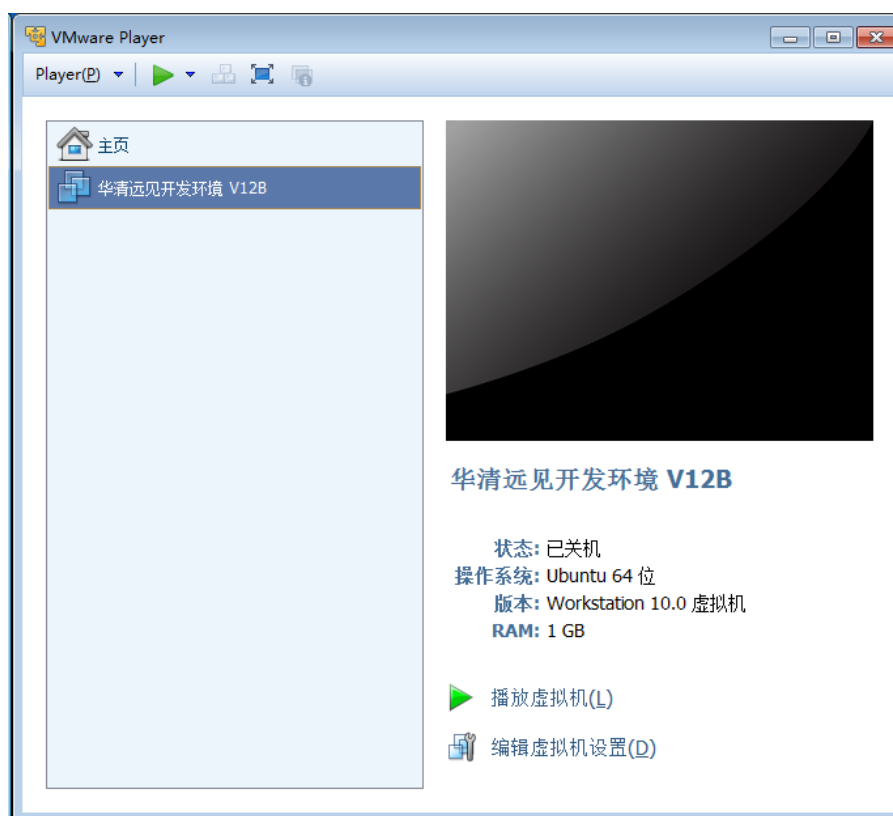
打开虚拟机

【打开 VMware Player】

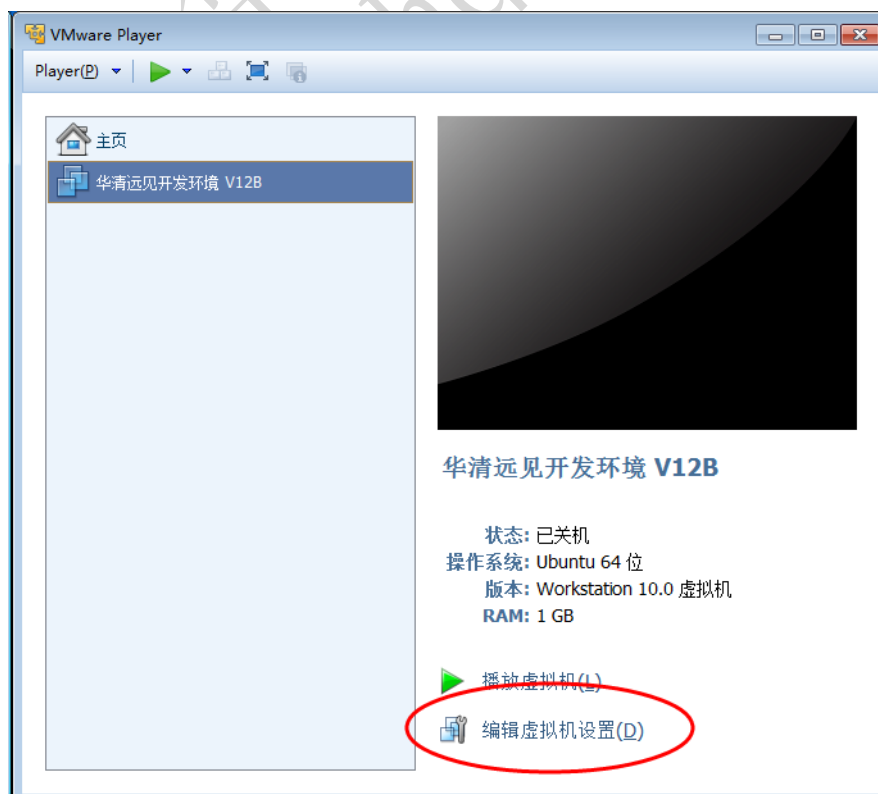


【打开虚拟机镜像】





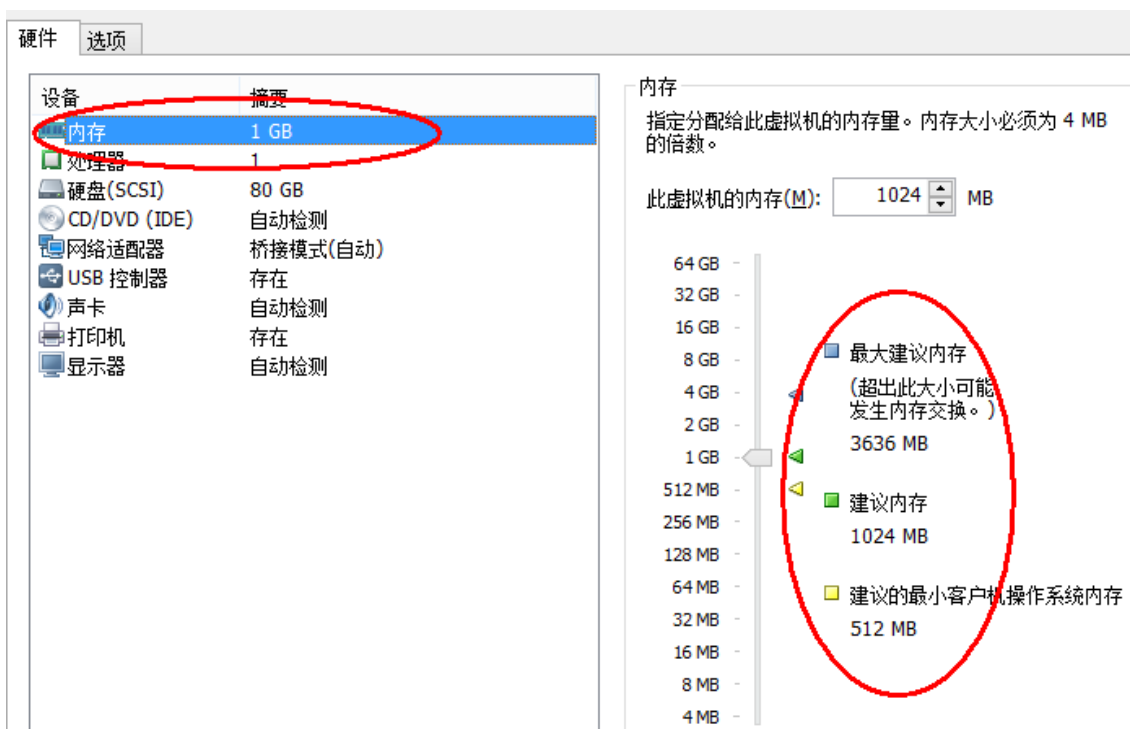
配置优化虚拟机





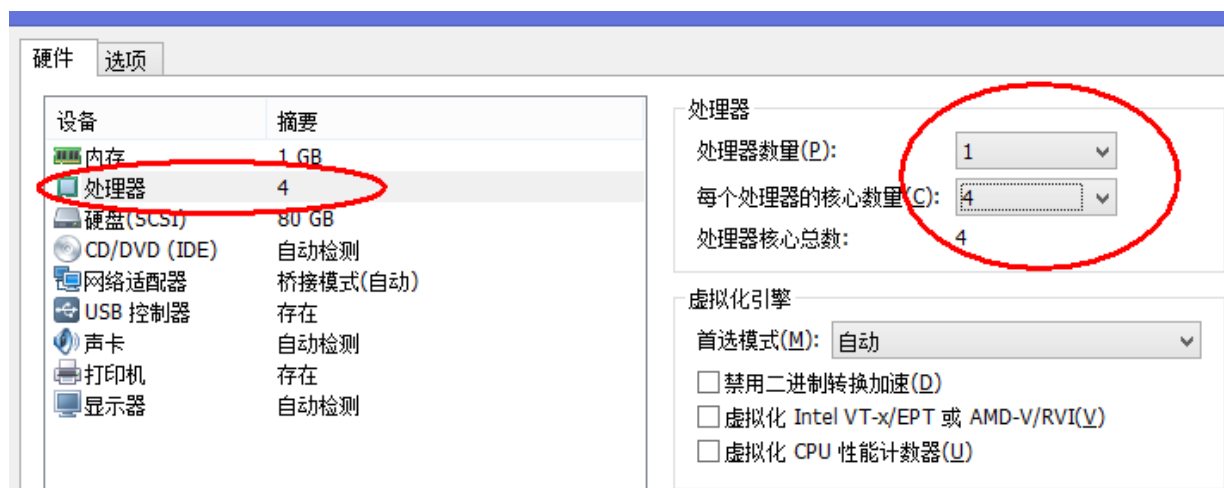
【增加内存大小】

根据主机配置修改虚拟机内存大小。例如主机内存 1G，那分配虚拟机的内存大小应该小于 512M，否则物理机操作系统运行会卡；如果主机内存大于 4G（足够大），那可以根据 VMware Player 的提示和自己的需求修改内存大小。**注意：如果需要编译 Android，那内存大小最好大于 1G。**



【修改 CPU 数量】

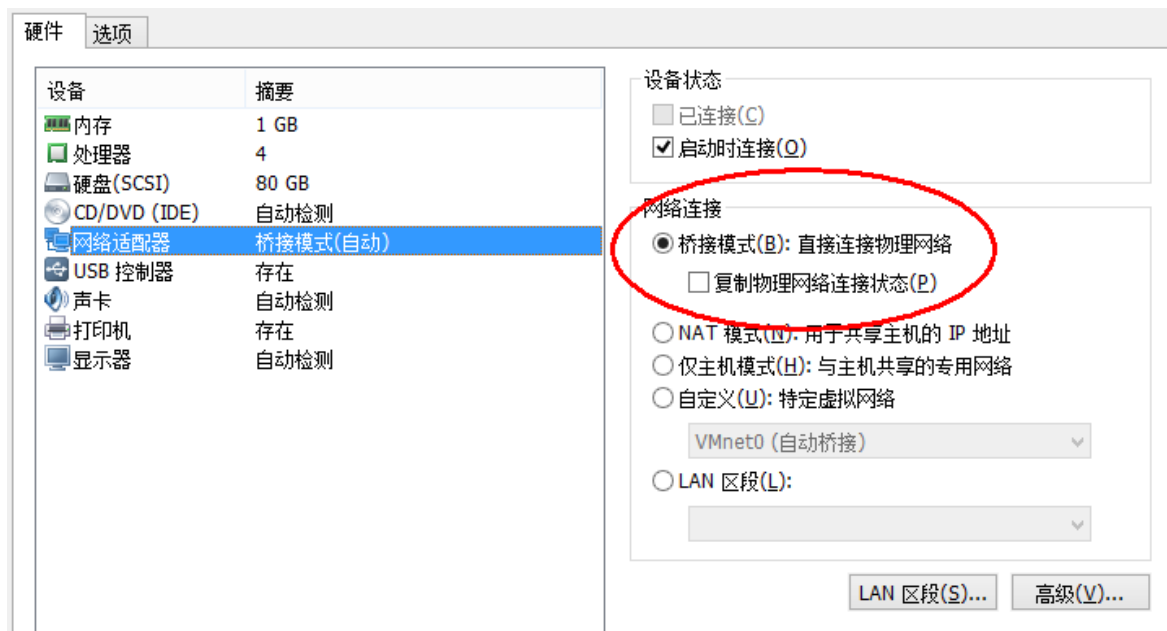
根据主机 CPU 配置修改虚拟机 CPU 数量。例如笔者 CPU 为 Intel Core-i3 M380（双核四线程），那处理器数量设置为 1,每个处理器的核心数量设置为 4。**注意：如果设置的总核心数不要超过 CPU 的核心数。**



华清远见
dev.havi.com

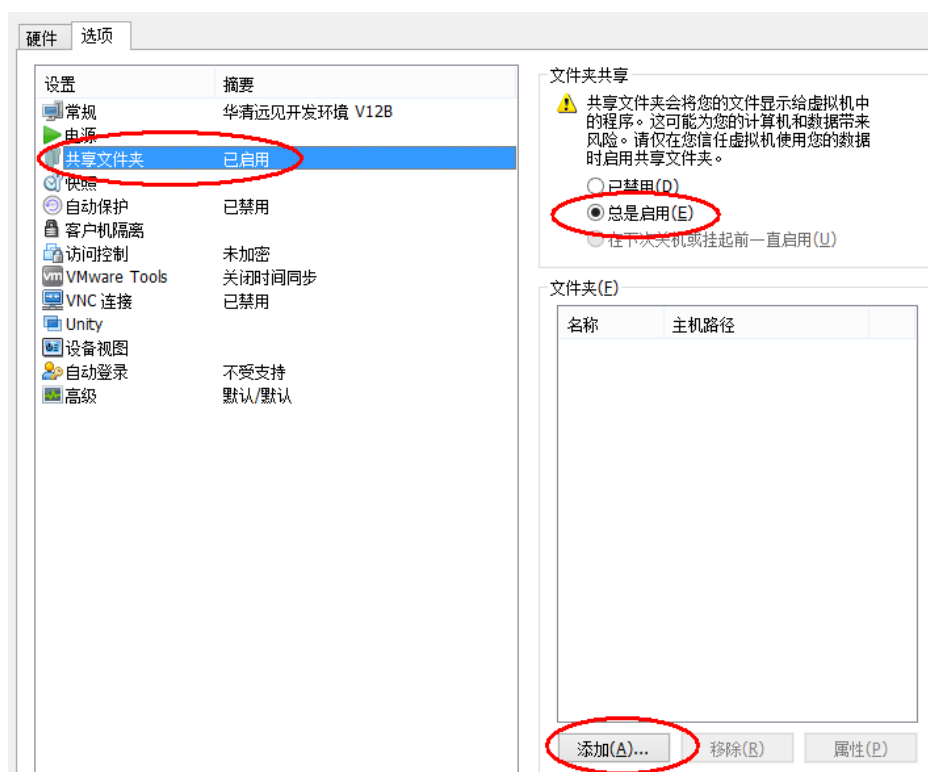


【确保网络连接为桥接模式】

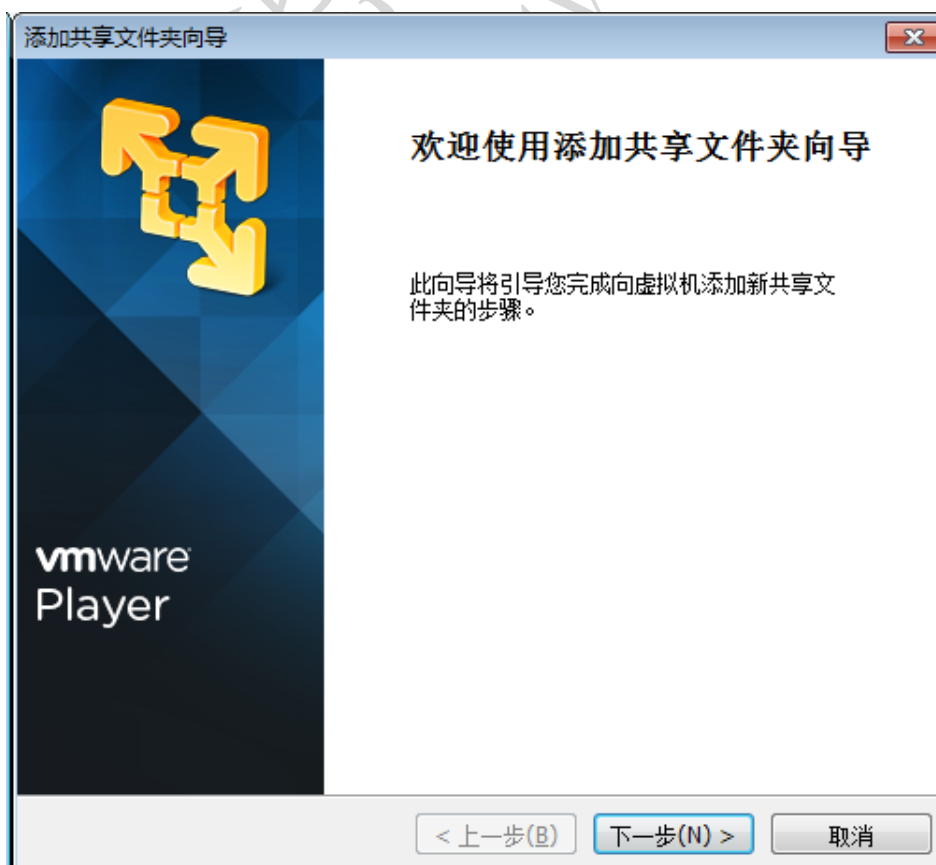


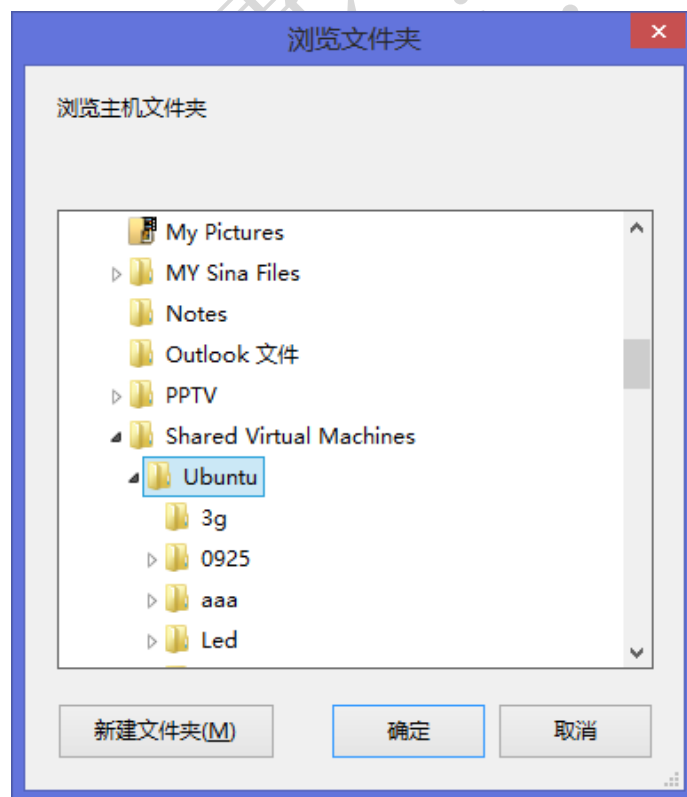
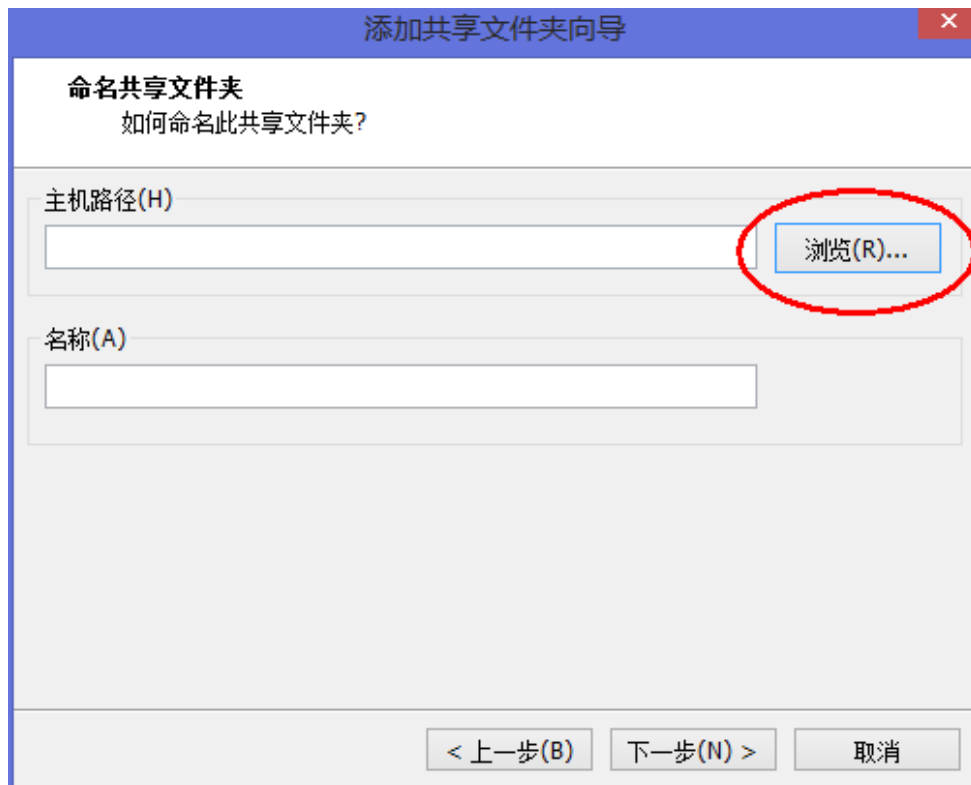
【增加共享目录】

共享目录可以在虚拟机访问物理硬盘分区的内容，也可以将虚拟机里的文件拷贝至物理机，是虚拟机和物理机很好的交流桥梁。

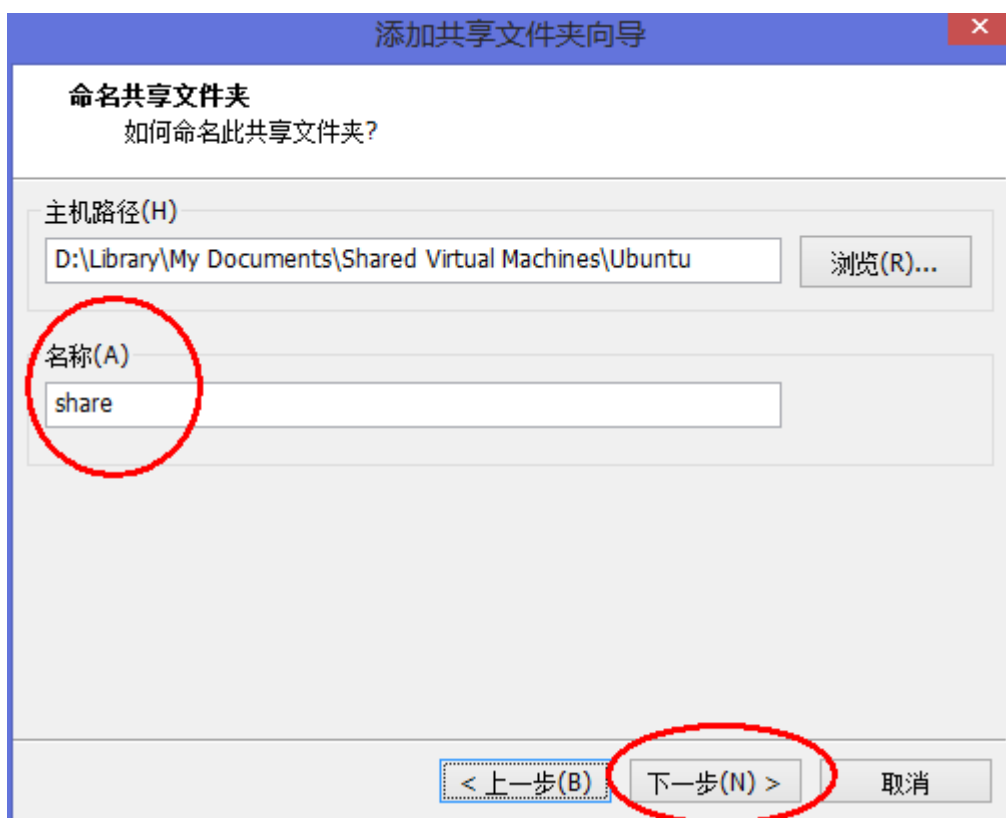


【点击上图“添加”】



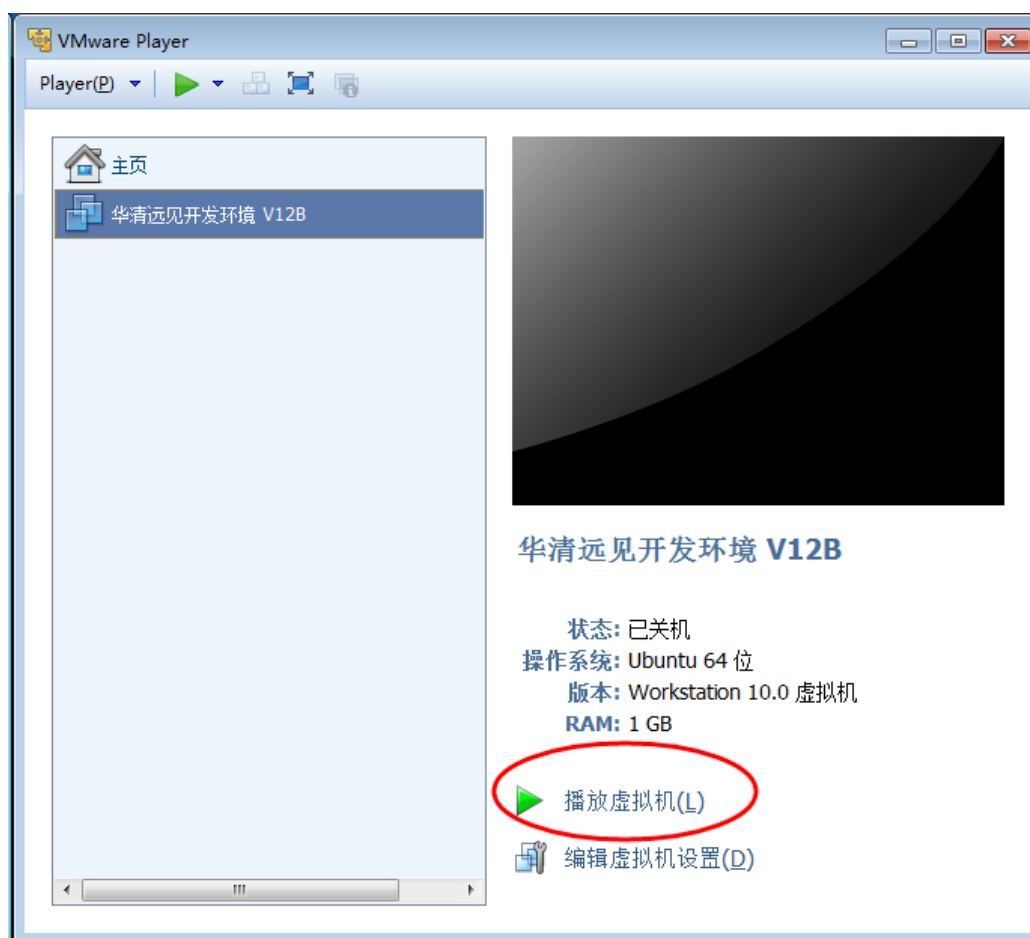


【修改在虚拟机内看到物理磁盘目录的名字】

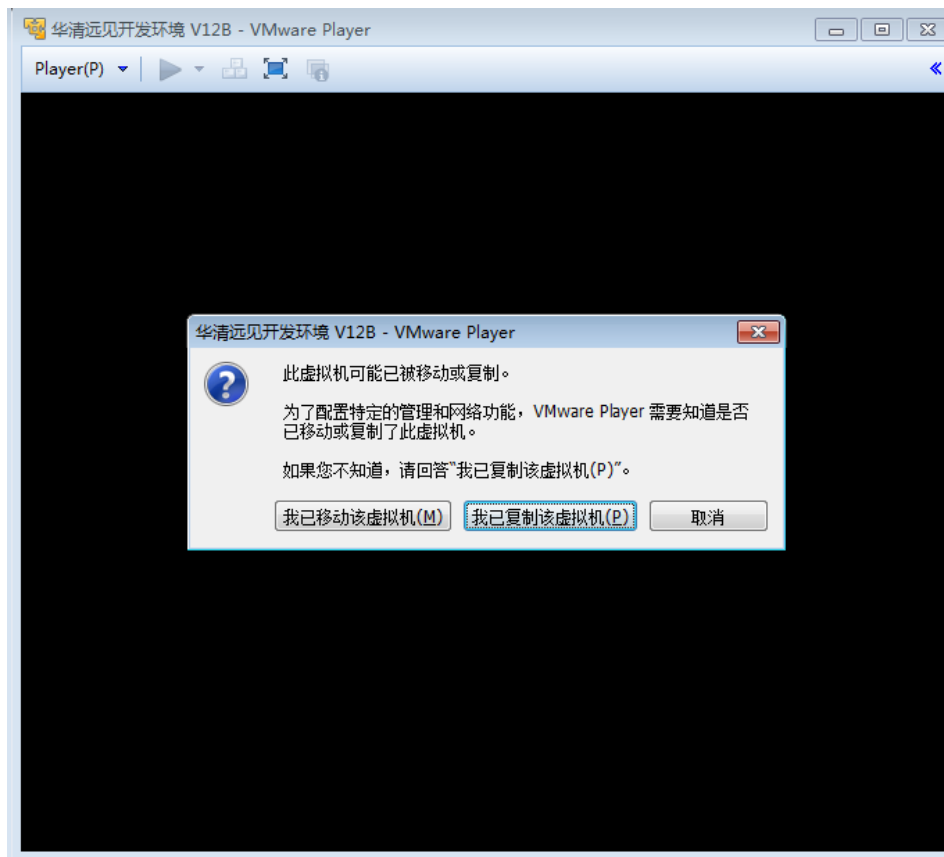




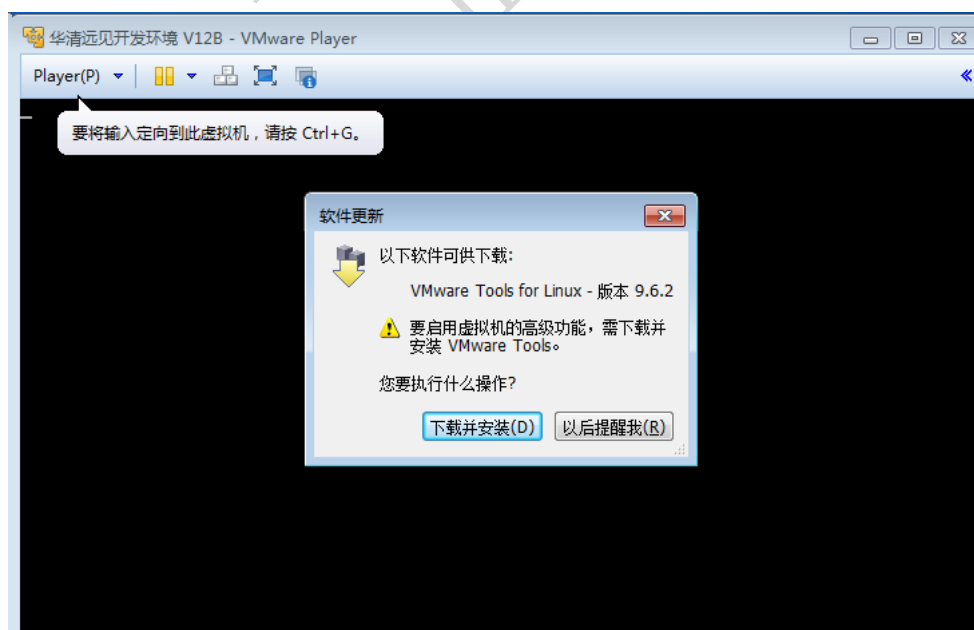
启动虚拟机

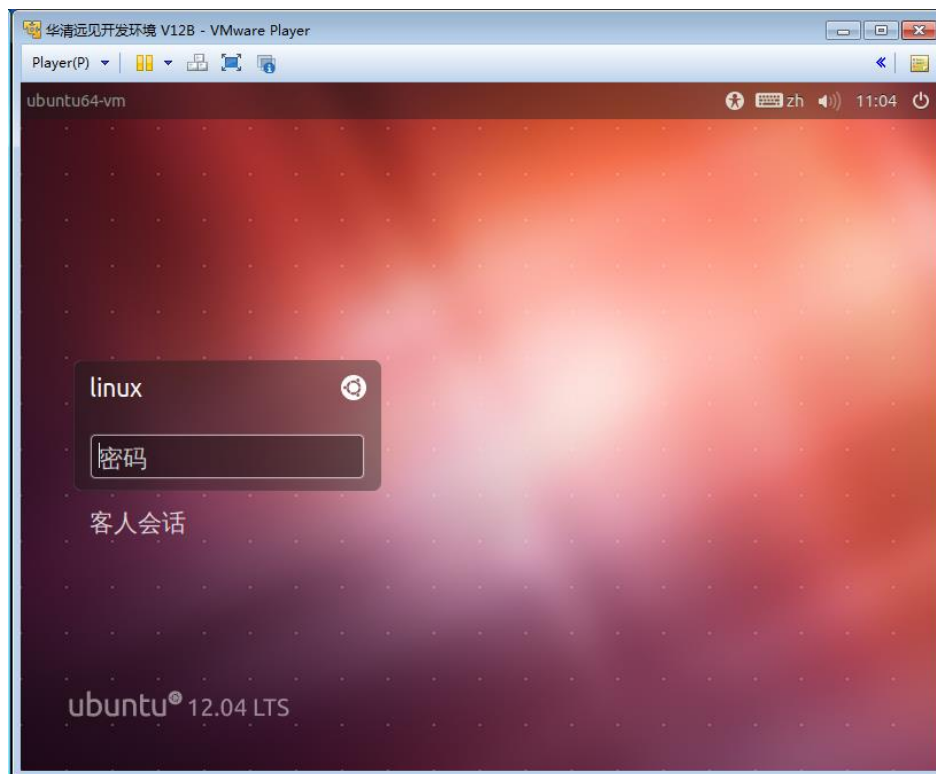


【启动虚拟机，选择“我已复制该虚拟机”】

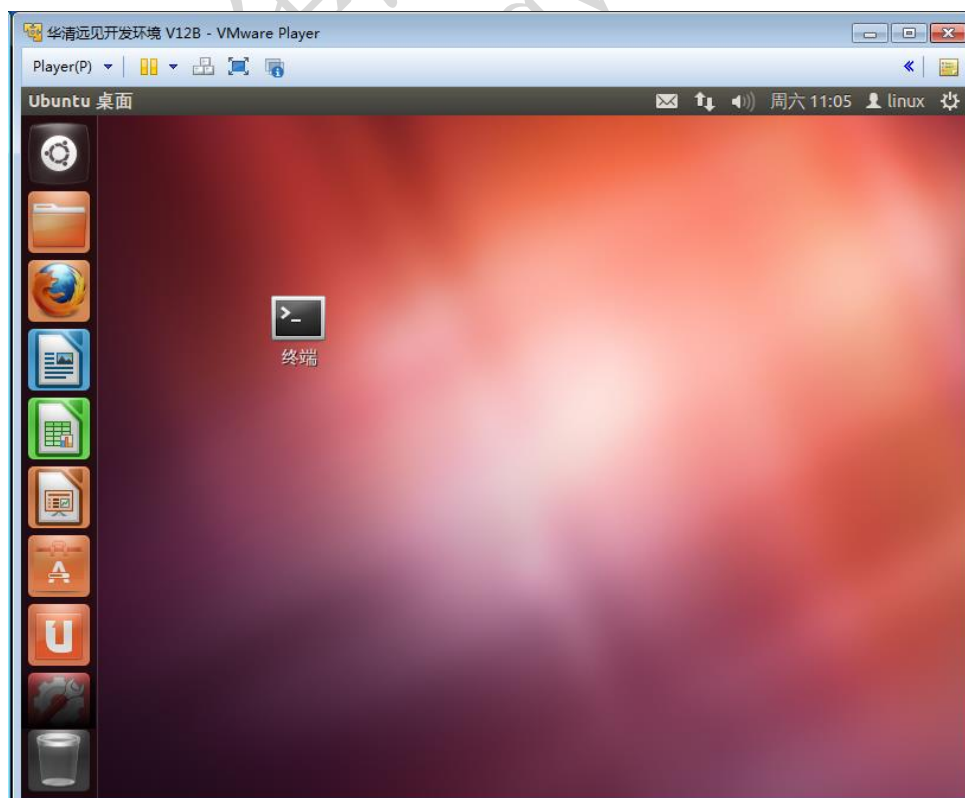


【在有网络的情况下，会提示有软件更新可以下载，可以选择下载，或者以后提醒】





【输入密码，密码为1】





JDK7 环境配置

Fs6818 需使用 jdk 1.7，系统默认为 jdk 1.6，所以需要切换至 jdk1.7，否则编译安卓会出现如下错误：

```
=====
Checking build tools versions...
*****
You are attempting to build with the incorrect version
of java.

Your version is: java version "1.6.0_29" Java(TM) SE Runtime Environment (build 1.6.0_29-b11) Java HotSpot
The required version is: "1.7.x"

Please follow the machine setup instructions at
https://source.android.com/source/initializing.html
*****
build/core/main.mk:167: *** stop. Stop.
```

执行如下指令完成切换

```
$ sudo update-alternatives --config java
```

```
$ sudo update-alternatives --config javac
```

输入指令后选择 1.7 对应编号“2”

```
linux@ubuntu64-vm:~/workdir/Fs6818$ sudo update-alternatives --config java
[sudo] password for linux:
有 2 个候选项可用于替换 java (提供 /usr/bin/java)。

 选择      路径                                     优先级  状态
-----
* 0        /usr/local/jdk1.6.0_29/bin/java             100    自动模式
 1        /usr/local/jdk1.6.0_29/bin/java             100    手动模式
 2        /usr/local/jdk1.7.0_79/bin/java             100    手动模式

要维持当前值[*]请按回车键，或者键入选择的编号：2
update-alternatives: 使用 /usr/local/jdk1.7.0_79/bin/java 来提供 /usr/bin/java (java)，于 手动模式 中。
linux@ubuntu64-vm:~/workdir/Fs6818$ sudo update-alternatives --config javac
有 2 个候选项可用于替换 javac (提供 /usr/bin/javac)。

 选择      路径                                     优先级  状态
-----
* 0        /usr/local/jdk1.6.0_29/bin/javac             100    自动模式
 1        /usr/local/jdk1.6.0_29/bin/javac             100    手动模式
 2        /usr/local/jdk1.7.0_79/bin/javac             100    手动模式

要维持当前值[*]请按回车键，或者键入选择的编号：2
update-alternatives: 使用 /usr/local/jdk1.7.0_79/bin/javac 来提供 /usr/bin/javac (javac)，于 手动模式 中。
linux@ubuntu64-vm:~/workdir/Fs6818$
```

验证 jdk 版本

```
$ java -version
```

```
linux@ubuntu64-vm:~/workdir$ java -version
java version "1.7.0_79"
Java(TM) SE Runtime Environment (build 1.7.0_79-b15)
Java HotSpot(TM) Server VM (build 24.79-b02, mixed mode)
```

出现如上现象时说明当前 JDK 版本已经成功配置为 JDK7。【更多内容请参见【华清远见开发环境】使用手册】



第 2 章 Android 系统编译

由于系统源码较为庞大，在实际操作之前请先确认前期实验时是否已经拷贝并解压过系统源码，如果已经执行过该操作用户可掠过本章“源码准备”、“解压源码”小节。

2.1 源码准备

本章节编译源码所使用的环境为上述的【华清远见开发环境】，强烈推荐用户使用此环境作为此开发板的开发环境。用户也可以使用非虚拟机的方式，或者使用其他发行版本的 Linux 操作系统，方法参见本章节，就不再详细叙述。

建议用户首先在虚拟机的工作目录下建立自己的工作目录，此处就以【/home/linux/workdir】为例。

```
$ mkdir /home/linux/workdir -p
```

首先将 FS6818 开发平台的源码拷贝到虚拟机的共享目录（D:\share\）下。按照【环境搭建】章节中【配置优化虚拟机】中【增加共享目录】的步骤，将源码所在文件夹（源码位置：【华清远见-CORTEXA53 系统源码：FS6818】）整个拷贝到下图所示的目录下。

SoftWare (D:) > share > FS6818

名称	修改日期	类型	大小
Fs6818.tar.xz00	2016/6/30 14:42	XZ00 文件	1,024,000...
Fs6818.tar.xz01	2016/6/30 14:56	XZ01 文件	1,024,000...
Fs6818.tar.xz02	2016/6/30 15:08	XZ02 文件	1,024,000...
Fs6818.tar.xz03	2016/6/30 15:45	XZ03 文件	1,024,000...
Fs6818.tar.xz04	2016/6/30 16:20	XZ04 文件	1,024,000...
Fs6818.tar.xz05	2016/6/30 16:28	XZ05 文件	618,950 KB

此时，我们就可以在虚拟机中查看到拷贝的源码了。

```
$ ls -l /mnt/hgfs/share/FS6818/
```

```
linux@ubuntu64-vm:~/workdir$ ls -l /mnt/hgfs/share/FS6818/
总用量 5738950
-rwxrwxrwx 1 root root 1048576000 6月 30 14:42 Fs6818.tar.xz00
-rwxrwxrwx 1 root root 1048576000 6月 30 14:56 Fs6818.tar.xz01
-rwxrwxrwx 1 root root 1048576000 6月 30 15:08 Fs6818.tar.xz02
-rwxrwxrwx 1 root root 1048576000 6月 30 15:45 Fs6818.tar.xz03
-rwxrwxrwx 1 root root 1048576000 6月 30 16:20 Fs6818.tar.xz04
-rwxrwxrwx 1 root root 633803932 6月 30 16:28 Fs6818.tar.xz05
linux@ubuntu64-vm:~/workdir$
```

拷贝源码到 Linux 下

```
$ cd /home/linux/workdir
```

```
$ cp /mnt/hgfs/share/FS6818/Fs6818.tar.xz0* .
```



```
linux@ubuntu64-vm:~/workdir$ ls -l
总用量 5738976
-rwxrwxr-x 1 linux linux 1048576000 7月 1 10:28 Fs6818.tar.xz00
-rwxrwxr-x 1 linux linux 1048576000 7月 1 10:29 Fs6818.tar.xz01
-rwxrwxr-x 1 linux linux 1048576000 7月 1 10:30 Fs6818.tar.xz02
-rwxrwxr-x 1 linux linux 1048576000 7月 1 10:31 Fs6818.tar.xz03
-rwxrwxr-x 1 linux linux 1048576000 7月 1 10:32 Fs6818.tar.xz04
-rwxrwxr-x 1 linux linux 633803932 7月 1 10:32 Fs6818.tar.xz05
```

2.2 解压源码

拷贝源码到 Linux 系统下后解压源码:

```
$ cat Fs6818.tar.xz0* | tar Jxvf -
```

解压后显示如下:

```
Fs6818/vendor/farsight/security/fs6818/release.pem
Fs6818/vendor/farsight/security/fs6818/shared.pem
Fs6818/vendor/farsight/security/fs6818/testkey.x509.pem
Fs6818/vendor/farsight/security/fs6818/media.pk8
Fs6818/vendor/farsight/security/fs6818/shared.x509.pem
Fs6818/vendor/farsight/security/fs6818/media.x509.pem
Fs6818/vendor/farsight/security/fs6818/platform.pem
Fs6818/vendor/farsight/security/fs6818/release.pk8
Fs6818/vendor/farsight/security/fs6818/testkey.pem
Fs6818/vendor/farsight/security/fs6818/testkey.pk8
Fs6818/vendor/farsight/security/fs6818/media.pem
Fs6818/uboot
```

注意: 如果这里提示错误, 请不要继续接下来的操作, 解压错误的原因可能是源码损坏, 请重新拷贝源码。

2.3 编译源码

由于 android 工程很大, 编译需要较长时间, 请耐心等待。进入源码顶层目录:

```
$ cd Fs6818
```

由于所有平台均使用同一套代码, 而每个平台中需要的驱动或者配置都不太一样, 所以每个平台都有自己专属的配置。

【内核配置】

用户可以直接在图形界面中选择相应的平台或者直接拷贝已有的平台配置文件到.config 中。

```
$ cd kernel
```

方式一, 图形界面配置平台, 根据自己的平台选择对应选项, 笔者这里是 FS6818M4 平台 (WSN6818C 平台请选择 FS6818)

```
$ make menuconfig
```

System Type --->

S5P6818 Board name (FS6818M4 base s5p6818) --->

() FS6818 base s5p6818

() FS6818B base s5p6818



(X) FS6818M4 base s5p6818

方式二，拷贝已有的平台配置文件到.config 文件。fs6818B_defconfig_vX 中的 X 代表的是版本号，请尽量选择高版本号!!!

```
$ cp arch/arm/configs/fs6818M4_defconfig_vX .config
```

选择平台完成后回到源码顶层目录

```
$ cd ..
```

【安卓配置】

返回安卓源码首层目录，打开如下文件：

```
$ vi device/farsight/fs6818/BoardConfig.mk
```

```
1 TARGET_BOARD_INFO_FILE := device/farsight/fs6818/board-info.txt
2
3 # Select the corresponding platform equipment
4 #FS6818      := true
5 FS6818M4     := true
6 #FS6818B     := true
7 #FS6818C     := true
8
9 TARGET_BOARD_PLATFORM      := slsiap
10 TARGET_BOOTLOADER_BOARD_NAME := fs6818
11
12 TARGET_CPU_ABI := armeabi-v7a
13 TARGET_CPU_ABI2 := armeabi
14 TARGET_CPU_SMP := true
```

根据用户所使用的平台对应的变量删除对应变量前的‘#’，其他平台前加‘#’号。

【全局编译】

（编译前请确保 java 版本为 1.7.0_79，详细操作请查看本文【环境搭建】章节中【华清远见开发环境】中【JDK7 环境配置】内容）：

```
$ ./mk -a
```

编译成功显示如下：



```
Created filesystem with 1535/41856 inodes and 110143/167424 blocks
patch 0 is 203 bytes (of 4954122)
patch 1 is 401671 bytes (of 704127)
patch 2 is 182 bytes (of 375)
chunk 0: normal ( 0, 4954122) 203
chunk 1: deflate ( 4954122, 1324256) 401671 (null)
chunk 2: normal ( 6278378, 790) 182
Install system fs image: out/target/product/fs6818/system.img
out/target/product/fs6818/system.img+out/target/product/fs6818/obj/PACKAGING/recovery_patch_intermediates/recovery_from_boot.p maxsize=700123776 b

#### make completed successfully (02:34 (mm:ss)) ####

/home/linux/workdir/Fs6818/out/release/uImage' -> '/home/linux/workdir/Fs6818/out/target/product/fs6818//boot/uImage'
/home/linux/workdir/Fs6818/out/target/product/fs6818//ramdisk.img' -> '/home/linux/workdir/Fs6818/out/target/product/fs6818//boot/root.img.gz'
/home/linux/workdir/Fs6818/out/target/product/fs6818//ramdisk-recovery.img' -> '/home/linux/workdir/Fs6818/out/target/product/fs6818//boot/ramdis
make_ext4fs -s -T -1 -l 67108864 -a boot /home/linux/workdir/Fs6818/out/target/product/fs6818//boot.img /home/linux/workdir/Fs6818/out/target/prod
Creating filesystem with parameters:
Size: 67108864
Block size: 4096
Blocks per group: 32768
Inodes per group: 4096
Inode size: 256
Journal blocks: 1024
Label:
Blocks: 16384
Block groups: 1
Reserved block group size: 7
Created filesystem with 18/4096 inodes and 4886/16384 blocks
/home/linux/workdir/Fs6818/out/target/product/fs6818//boot.img' -> '/home/linux/workdir/Fs6818/out/release/boot.img'
/home/linux/workdir/Fs6818/out/target/product/fs6818//system.img' -> '/home/linux/workdir/Fs6818/out/release/system.img'
/home/linux/workdir/Fs6818/out/target/product/fs6818//cache.img' -> '/home/linux/workdir/Fs6818/out/release/cache.img'
/home/linux/workdir/Fs6818/out/target/product/fs6818//recovery.img' -> '/home/linux/workdir/Fs6818/out/release/recovery.img'
/home/linux/workdir/Fs6818/out/target/product/fs6818//userdata.img' -> '/home/linux/workdir/Fs6818/out/release/userdata.img'
```

注意：编译过程中有时会出现错误，重新编译便可顺利通过。

全局编译完成后，会生成一个镜像目录，out/release。

```
linux@ubuntu64-vm:~/workdir/Fs6818$ ls -l out/release/
总用量 566520
-rw-r--r-- 1 linux linux 20013288 7月 1 15:10 boot.img
-rw-r--r-- 1 linux linux 8962352 7月 1 12:50 cache.img
-rw-rw-r-- 1 linux linux 134 7月 1 15:07 env.txt
-rw-r--r-- 1 linux linux 6279168 7月 1 15:10 recovery.img
-rwxrwxr-x 1 linux linux 945 7月 1 15:07 s5p6818-sdmmc.sh
-rw-r--r-- 1 linux linux 441656472 7月 1 15:10 system.img
-rw-rw-r-- 1 linux linux 303380 7月 1 15:07 ubootpak.bin
-rw-rw-r-- 1 linux linux 4950304 7月 1 15:07 uImage
-rw-r--r-- 1 linux linux 97924564 7月 1 13:51 userdata.img
```

2.4 重点文件说明

编译后生产文件中我们需要烧写到设备上有五个文件，分别是 ubootpak.bin、boot.img、system.img、cache.img 和 userdata.img。

ubootpak.bin：包括 2ndboot.bin、nsih 和 u-boot.bin 三个文件

boot.img：包括 zImage 和 ramdisk.img 和开机 LOGO 图片

system.img：android 文件系统

cache.img：缓存文件

userdata.img：用户数据文件

【编译脚本 mk 参数简介】

在以下实验中会用到 mk 这个脚本来编译源码，下面是对该脚本的参数介绍：

-u：只编译 bootloader 源码，生成 ubootpak.bin；

-k：只编译内核源码，生成 uImage；

-b：编译内核源码并将生成的 uImage 及开机 logo 打包成 boot_linux.img；

-s：编译安卓源码，并打包成安卓必须镜像，执行该参数前需确保 bootloader 及内核已经编译；

-a：执行-u、-k、-s 三个操作，便编译安卓源码使用该参数即可。



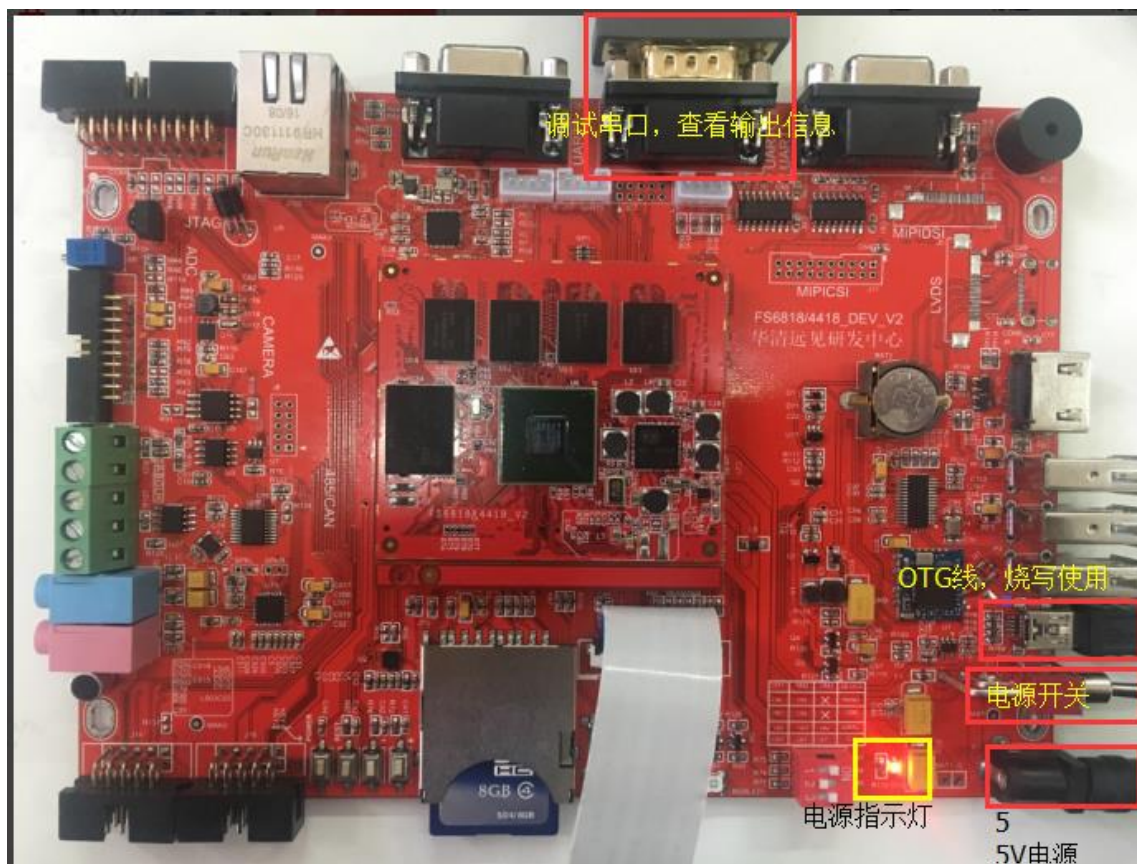
第 3 章 Android 镜像烧写实验

注意：这里的镜像烧写到 eMMC 中，而非 SD 卡中。

连接必备三要素，没电一切都是白扯；不连接调试串口，你如何使用相应命令；不连接 OTG 线，如何将 PC 机上的镜像数据传输至开发板？所以明白了这三点，连线就是轻而易举的事儿了。

- 电源线，连接后打开电源开关，观察电源指示灯是否常亮。
- 串口线，连接调试串口，并配置串口调试工具，能够正常输出启动信息。
- OTG 线，使用 fastboot 进行镜像烧写时数据传输方式。

3.1 开发板连接方法（当用户单独使用开发板时）





3.2 实验箱连接方法（当开发板和实验箱底板固定时）

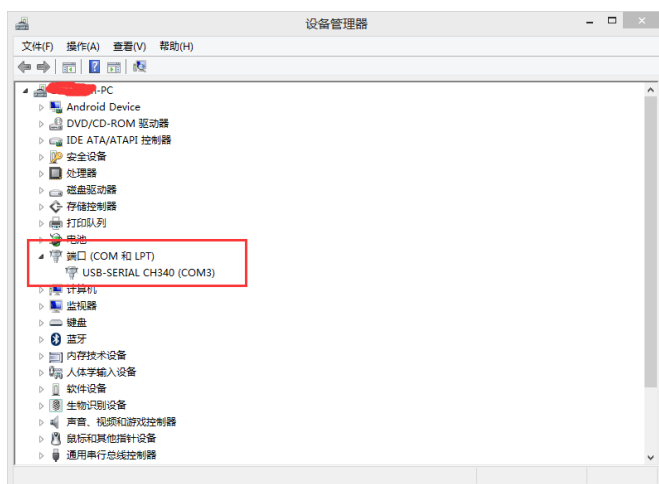


3.3 设置串口调试工具

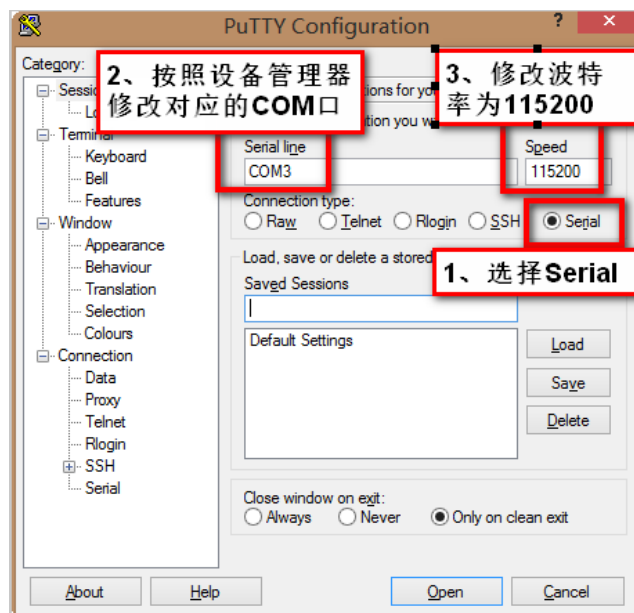
一般的开发平台都会留有系统调试的串口，目的是让用户可以通过串口和开发板进行通信，一般打开系统开关时，系统会向串口输出启动信息，此时我们接入相应的串口线（我们提供的串口线为CH340USB转串口线），让数据通过串口线传至PC机上，我们在PC上使用串口软件（putty）进行信息显示。Putty在使用时需要首先配置一下，配置过程如下。

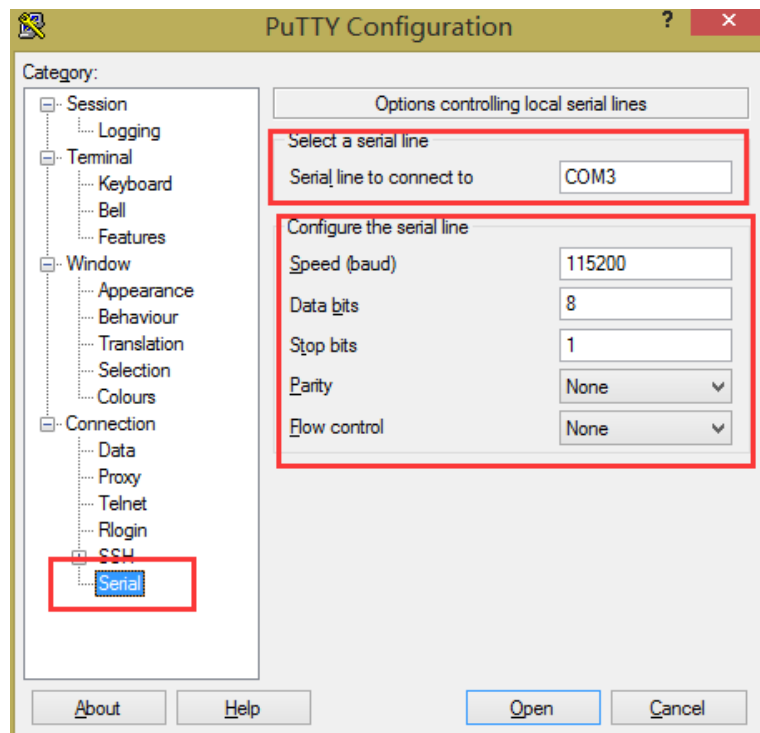
打开【华清远见-嵌入式 ARM 实验箱资料- I -FS6818\工具软件\串口调试工具\putty.EXE】文件。

打开设备管理器，确定当前PC机上的串口号。



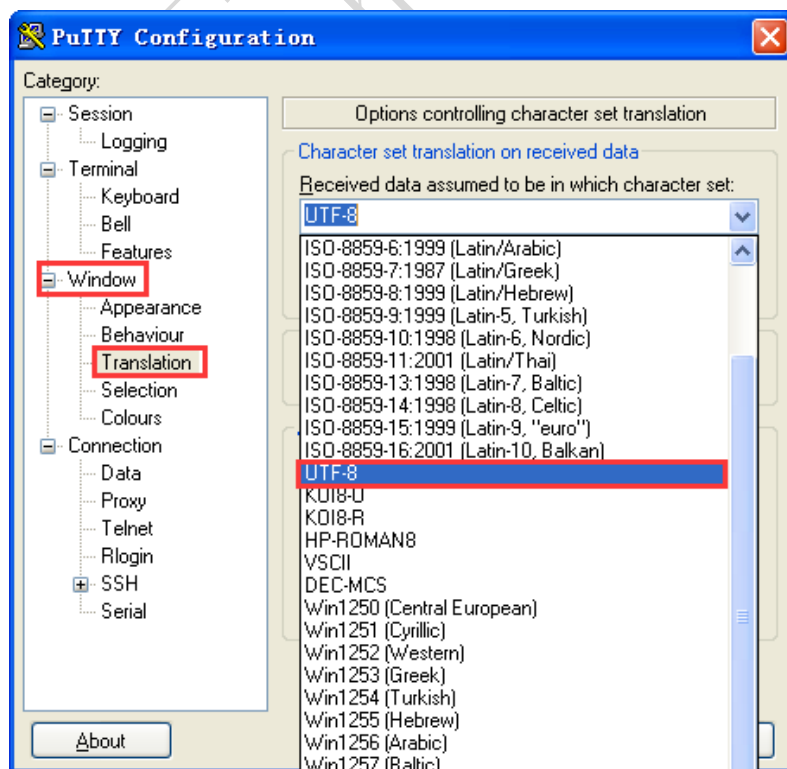
选择串口（Serial）连接方式：





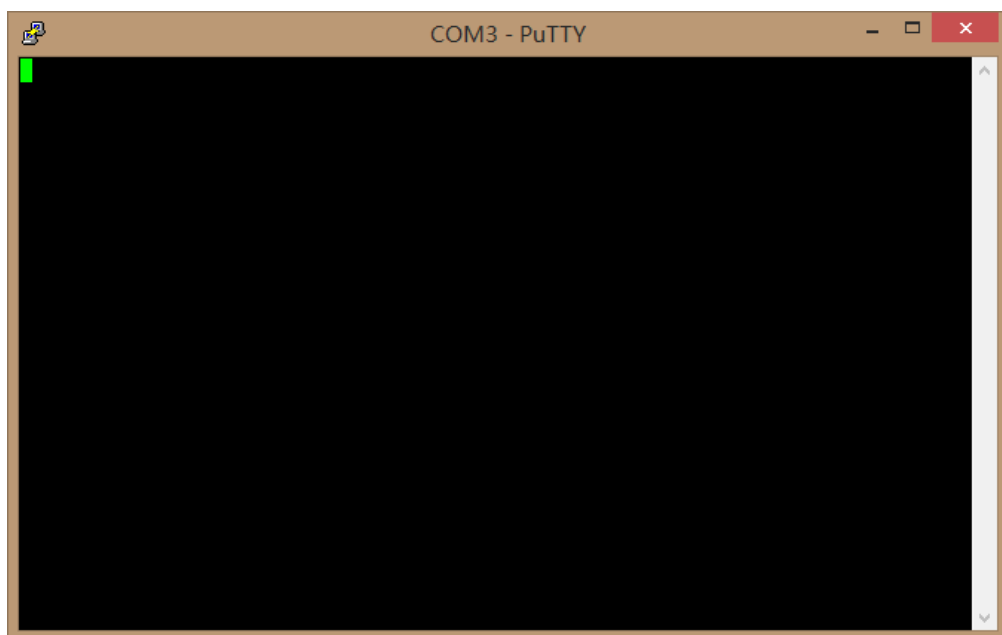
选中第一步方框内的 Serial,再点击第二步中的 Serial,进入串口设置的对话框（注意：这里如果使用台式机物理串口一般为 COM1；使用 USB 转串口线需要提前安装驱动程序，光盘中提供常用的 CH340 和 PL2303 两种驱动，路径为【华清远见-嵌入式 ARM 实验箱资料- I -FS6818\工具软件\串口调试工具\USB 转串口驱动】）。

再选中 Window 中的 Translation,在选项栏中选择文本编码格式为 UTF-8，防止出现中文乱码：





点击 open 打开串口。

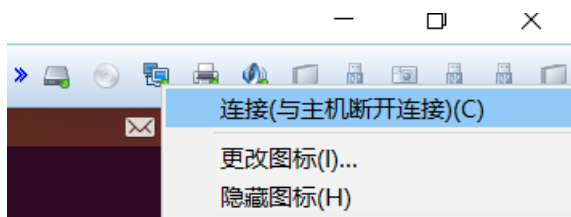


3.4 制作 SD 卡启动盘

这一章节的主要目的是当用户由于误操作将 eMMC 中的 uboot 擦除以后，无法进行任何有效操作时使用，用户可以将 uboot 映像烧写到 SD 卡，再将 SD 卡插到开发板的 SD 卡槽，开机选择 SD 卡启动方式，然后执行 fastboot 命令更新镜像。

3.4.1 SD 卡格式化分区

准备一张不小于 2GB 的 SD 卡，并插到装有 ubuntu 操作系统或装有 ubuntu 虚拟机的 PC 机本文以 ubuntu 虚拟机为例。将 SD 卡插入 PC（因多数笔记本自带 SD 卡读卡器无法被虚拟机识别，所以请使用外置 SD 卡读卡器），点击 vmware player 右上角 SD 卡对应盘符选择连接，接入虚拟机。



打开 Linux 终端，在 Linux 的终端窗口，使用 fdisk 命令删除 SD 卡上原来所有分区，一般 sdb 为系统为 SD 卡分配的设备节点。注意，具体由节点名称而定，有可能是 sdc,sde 等。使用如下指令查询



设备节点:

没有插入 SD 时, 在 Linux 终端输入查看系统磁盘分区命令:

```
$ cat /proc/partitions
```

显示为:

```
linux@ubuntu64-vm:~$ cat /proc/partitions
major minor #blocks name

 11        0   1048575 sr0
   8        0  83886080 sda
   8        1  81053696 sda1
   8        2         1 sda2
   8        5  2829312 sda5
linux@ubuntu64-vm:~$
```

插入 SD 卡在此执行命令, 再次输入查看系统分区命令, 显示中多出来的就是 SD 卡对应分区。

```
linux@ubuntu64-vm:~$ cat /proc/partitions
major minor #blocks name

 11        0   1048575 sr0
   8        0  83886080 sda
   8        1  81053696 sda1
   8        2         1 sda2
   8        5  2829312 sda5
   8       16   7802880 sdb
   8       17   7798784 sdb1
```

其中, sdb 为设备名称, sdb1 为分区名称, 分区或删除分区操作对象为设备名 sdb。

执行命令删除分区:

```
sudo fdisk /dev/sdb
```

```
linux@ubuntu64-vm:~$ sudo fdisk /dev/sdb
[sudo] password for linux:
Command (m for help):
```

提示输入命令完成相应操作, 可以输入 m 查看帮助, 有命令的详细介绍, 其中 d 为删除分区。



```
Command (m for help): m
Command action
  a  toggle a bootable flag
  b  edit bsd disklabel
  c  toggle the dos compatibility flag
  d  delete a partition
  e  test known partition types
  m  print this menu
  n  add a new partition
  o  create a new empty DOS partition table
  p  print the partition table
  q  quit without saving changes
  s  create a new empty Sun disklabel
  t  change a partition's system id
  u  change display/entry units
  v  verify the partition table
  w  write table to disk and exit
  x  extra functionality (experts only)

Command (m for help):
```

输入“d”删除分区，如果 SD 卡只有一个分区输入命令后会被直接删除，如果有多个分区根据提示删除即可：

```
Command (m for help): d
Selected partition 1
```

重建分区，输入“n”添加一个新分区，输入后会提示分区类型选择“p”主分区，接着会提示分区号输入“1”，接着会提示输入分区起始扇区位置，这里需要预留一部分给 uboot 所以输入“524288”即预留 256M 空间，接下来选择分区最后扇区编号，这个直接回车使用默认值即可，最后输入“w”写入分区表：

```
Command (m for help): n
Partition type:
  p  primary (0 primary, 0 extended, 4 free)
  e  extended
Select (default p): p
Partition number (1-4, default 1):
Using default value 1
First sector (2048-15605759, default 2048): 524288
Last sector, +sectors or +size[K,M,G] (524288-15605759, default 15605759):
Using default value 15605759

Command (m for help): w
The partition table has been altered!

Calling ioctl() to re-read partition table.
Syncing disks.
```

查看分区：

```
linux@ubuntu64-vm:~$ cat /proc/partitions
major minor #blocks name
11        0    1048575 sr0
 8        0   83886080 sda
 8        1   81053696 sda1
 8        2         1 sda2
 8        5   2829312 sda5
 8       16    7802880 sdb
 8       17    7540736 sdb1
```

格式化分区，将分区格式化为 fat 格式：

```
sudo mkfs.vfat /dev/sdb1
```

```
linux@ubuntu64-vm:~$ sudo mkfs.vfat /dev/sdb1
mkfs.vfat 3.0.12 (29 Oct 2011)
linux@ubuntu64-vm:~$
```

3.4.2 烧写 uboot 到 sd 卡上

Uboot 烧写工具需要编译安卓源码后生成，源码编译完成后进入镜像生成目录，即 out/release 目录，执行如下指令烧写 uboot 到 SD 卡（由于安卓编译时间较长，此处已将 uboot 烧写工具放在【华清远见-嵌入式 ARM 实验箱资料- I -FS6818\工具软件\SD 卡烧写工具】目录下，用户可直接将其拷贝到虚拟机进行如下操作）：

```
sudo ./s5p6818-sdmmc.sh /dev/sdb ubootpak.bin
```

```
linux@ubuntu64-vm:~/workdir/Fs6818/out/release$ sudo ./s5p6818-sdmmc.sh /dev/sdb ubootpak.bin
记录了 592+1 的读入
记录了 593+0 的写出
303616字节(304 kB)已复制, 0.176638 秒, 1.7 MB/秒
^^ The image is fused successfully
linux@ubuntu64-vm:~/workdir/Fs6818/out/release$
```

这时，该 SD 卡就可以引导开发板启动 uboot 了。

3.5 环境配置

在安卓手机中 Fastboot 是一种比 Recovery 更底层的刷机模式。Fastboot 是一种线刷，就是使用 USB 数据线连接手机的一种刷机模式。相对于某些系统卡刷来说，线刷更可靠，安全。

Fastboot 工具在【华清远见-嵌入式 ARM 实验箱资料- I -FS6818\工具软件\Fastboot\Fastboot】目录下，为了方便把他复制到 D 盘。

3.5.1 拷贝 Fastboot 工具

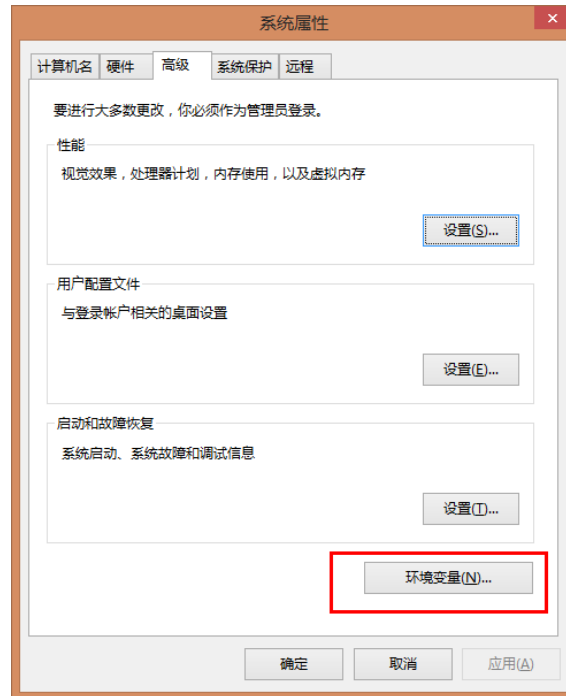
拷贝【华清远见-嵌入式 ARM 实验箱资料- I -FS6818\工具软件\Fastboot\Fastboot】目录到 D 盘下。



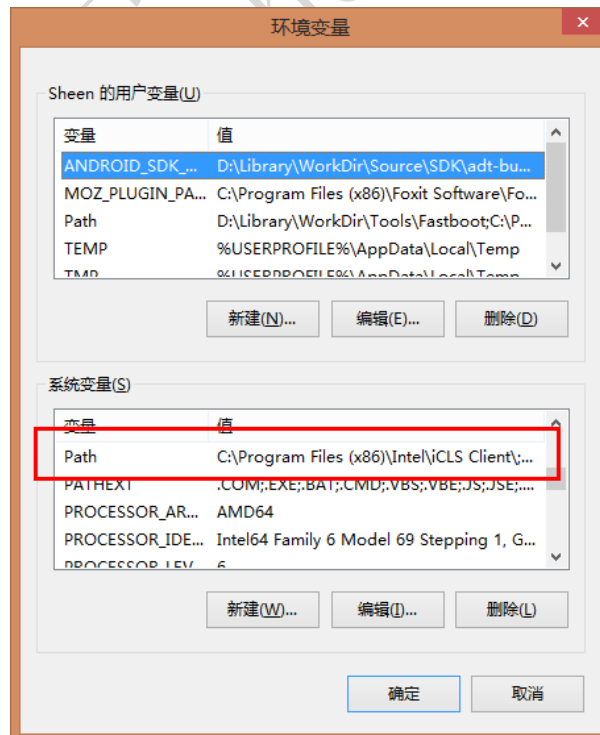
拷贝完成后，打开命令提示符，进入该目录，即可以使用 fastboot；但是用户在非该目录的路径下，无法

使用 fastboot，所以用户需要将 fastboot 的路径添加到系统的环境变量中，在系统的任何路径下都可以直接使用 fastboot 命令。

打开【计算机属性】下得而【系统属性】，查看【高级】，选择环境变量。

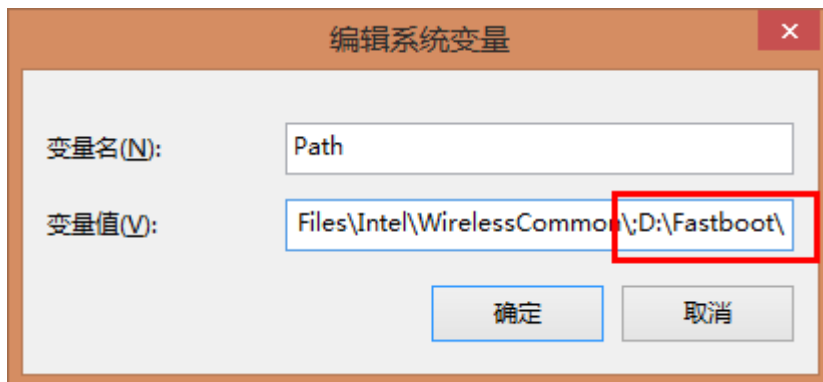


编译【系统变量】的【Path】项。

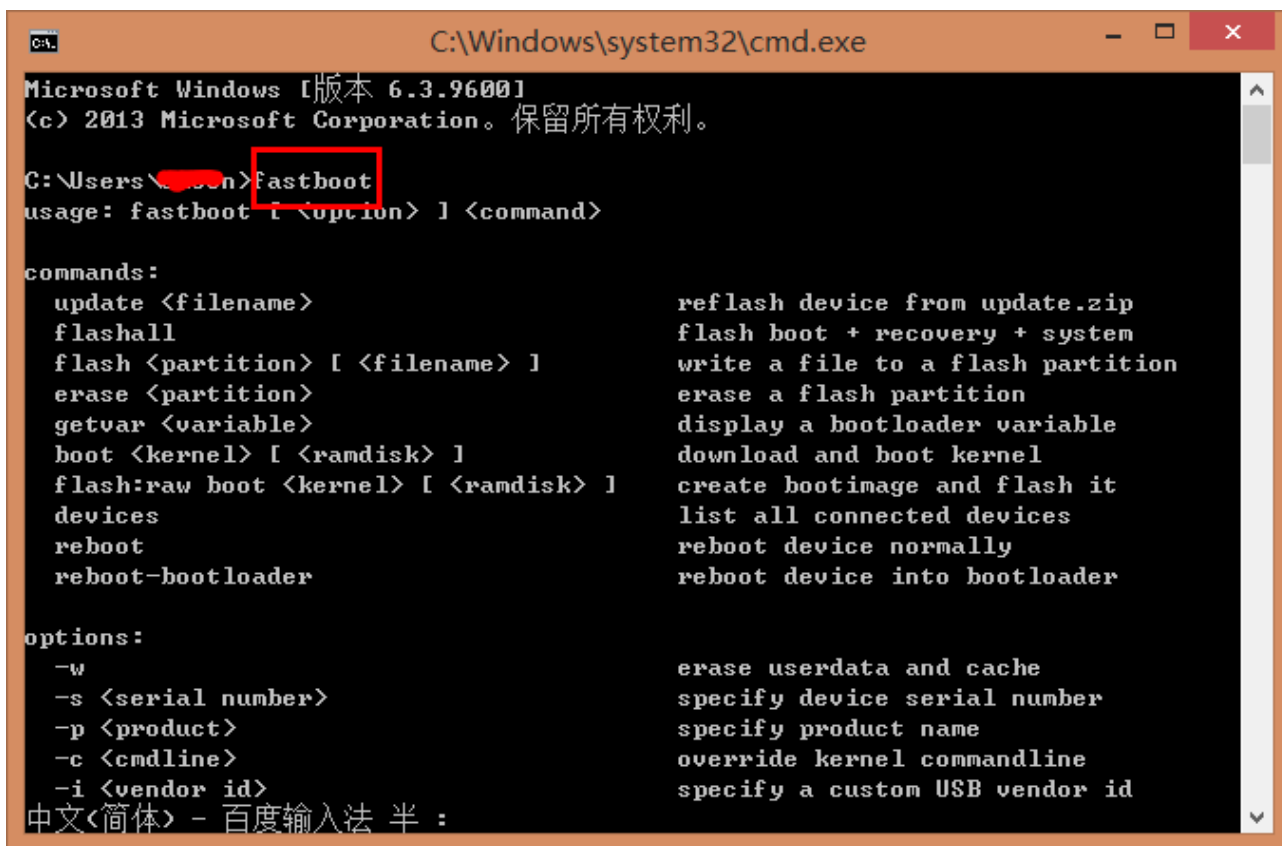




在【变量值】最后添加【;D:\Fastboot\】（注意开始的分号）。



点击【确定】，打开【cmd】，输入【fastboot】测试环境变量是否添加成功。



如上图所示即添加环境变量成功。

以上为烧写系统前的铺垫工作，这些工作只有在第一次烧写系统时需要去做，以后的学习中直接使用 就可以了，不需要重新再搭建这些环境。

3.5.2 安装 Fastboot 驱动

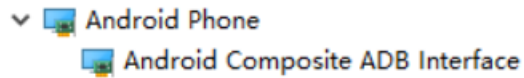
安装 fastboot 驱动时，需要先将开发板进入 fastboot 模式，在串口调试助手中倒计时阶段按任意键停止后，输入 fastboot 命令，开发板即进入 fastboot 模式。

第一次使用 Fastboot 需要安装驱动，驱动位置在【华清远见-嵌入式 ARM 实验箱资料-I-FS6818\工

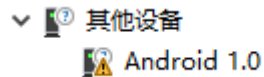


具软件\Fastboot\Fastboot 驱动】下。按前文连接开发板串口和 USB 口。

此时，如果系统安装了驱动，在 windows 系统设备管理器中应该如下显示：

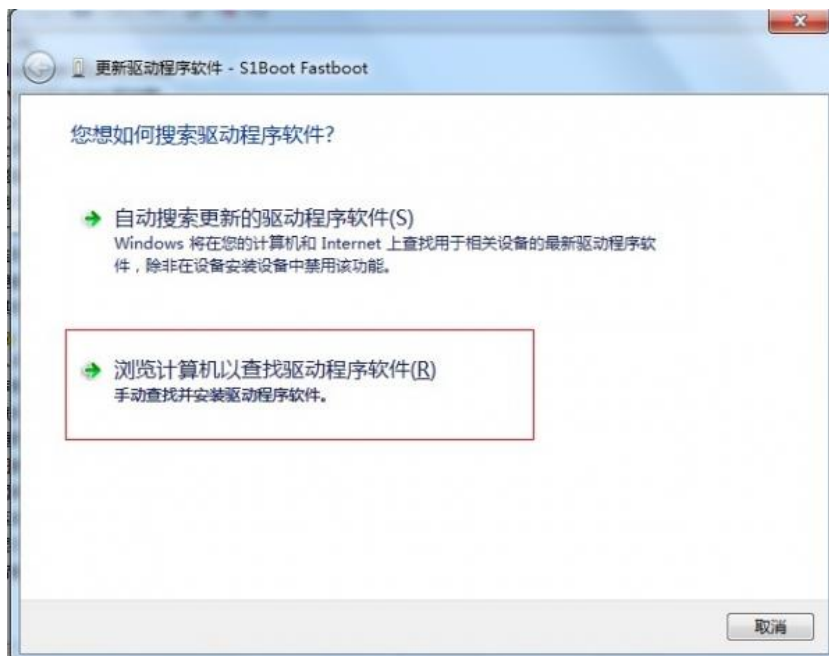


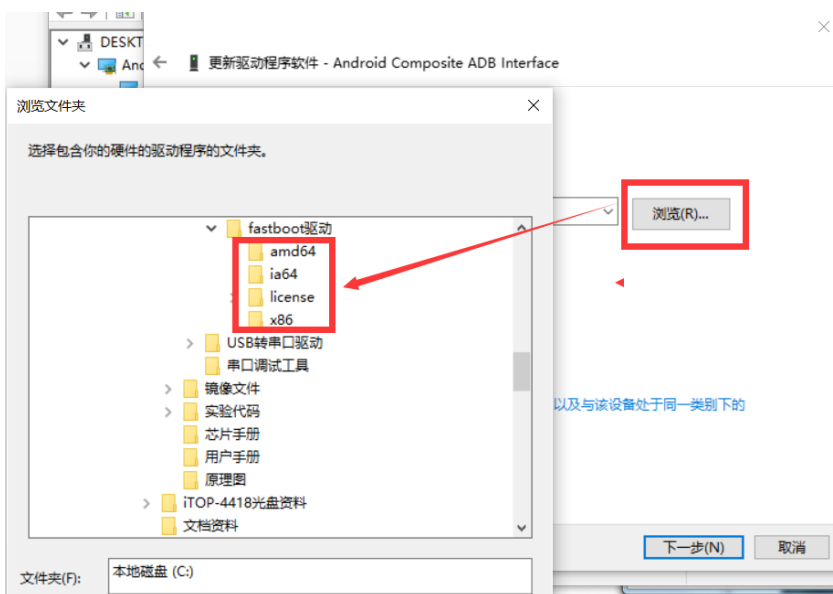
如没有安装驱动，在 windows 系统设备管理器出现如下现象时：



此时需要右键点击 Android 1.0，选择更新驱动程序软件，手动安装 Fastboot 驱动。

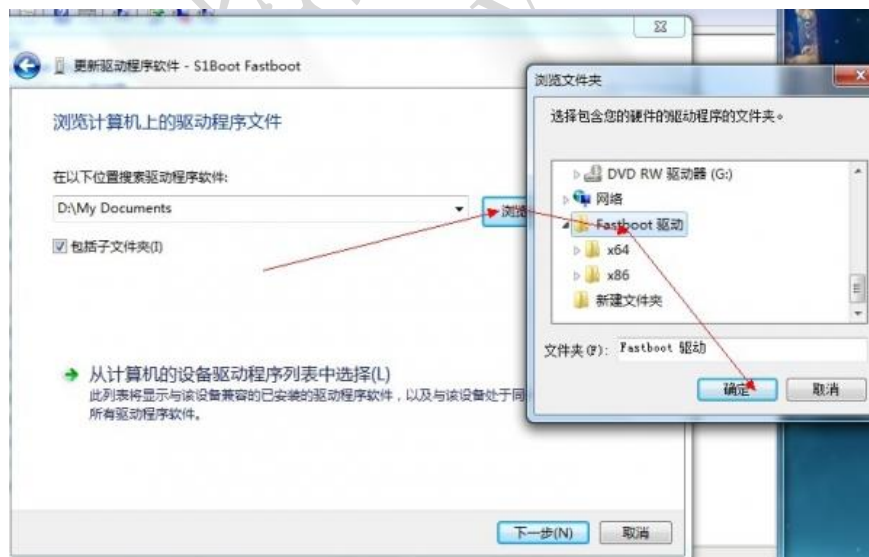
注意：由于系统差异有些系统无法成功安装驱动，可安装 qq 手机助手或 360 手机助手由手机助手自动安装。







注：XP 下也是如此操作，只不过某些地方名称不一样！第五步如果确定按钮是灰色的话，选择到 X86 里面的 usbwin 就行了，如果是 64 位 XP 就选择 X64 的。WIIN8 下要关闭强制驱动签名才能安装上驱动。





注：XP 下也是如此操作，只不过某些地方名称不一样！第五步如果确定按钮是灰色的话，选择到 X86 里面的 usbwin 就行了，如果是 64 位 XP 就选择 X64 的。

3.5.3 安装 ADB 驱动

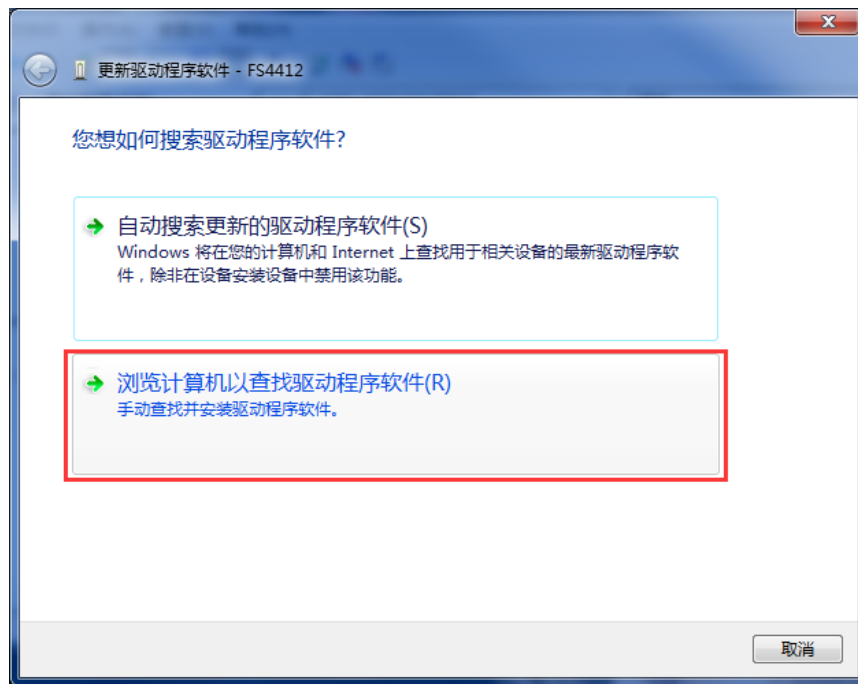
ADB 中文解释就是调试桥的作用。既然是调试作用，需要开机并连接电脑才可以使用该工具。而 fastboot 相关工作需进入 fastboot 模式完成，这两个是不同的命令，需要安装不同的驱动，完成刷机工作不需要任何 adb 命令即可完成，只需要 fastboot 即可，后期调试工作中会用到 ADB 相关工具。

第一次使用 ADB 需要安装驱动，驱动位置在【华清远见-CORTEXA9 资料\工具软件\Windows\Android ADB Driver】下，连接开发板串口和 USB 口并启动安卓系统。

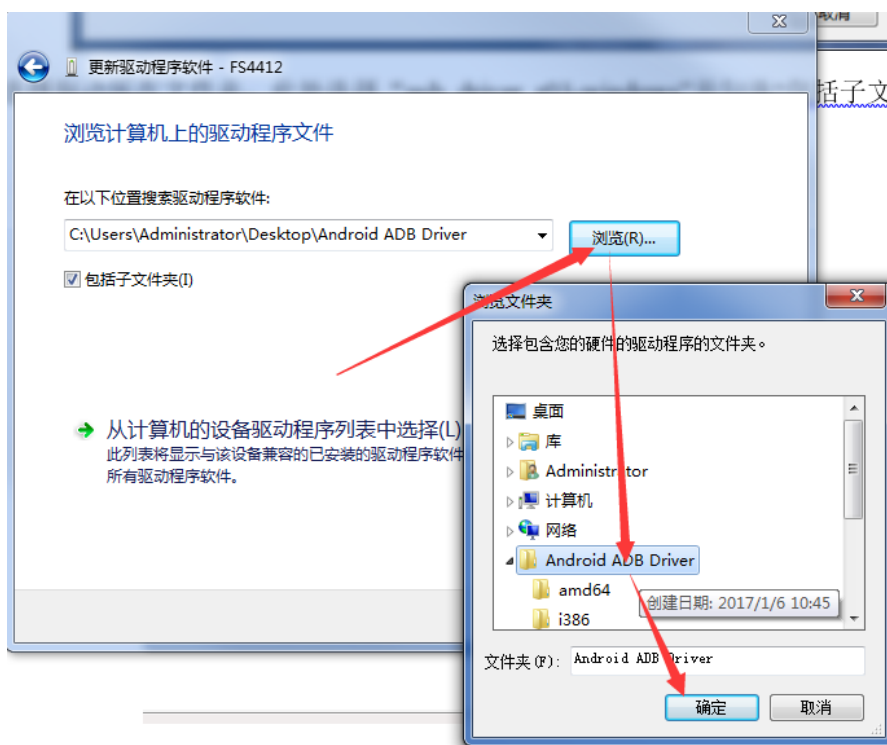
此时，如果系统安装了驱动，在设备管理器中应该如下显示：



如没有安装驱动，需要手动安装，通过 USB 线连接开发板和电脑，开发板进入安卓系统，在设备管理器右击未识别的设备【FS4412】选择“更新驱动程序软件”弹出如下窗口，点击“浏览计算机以查找驱动程序软件”：



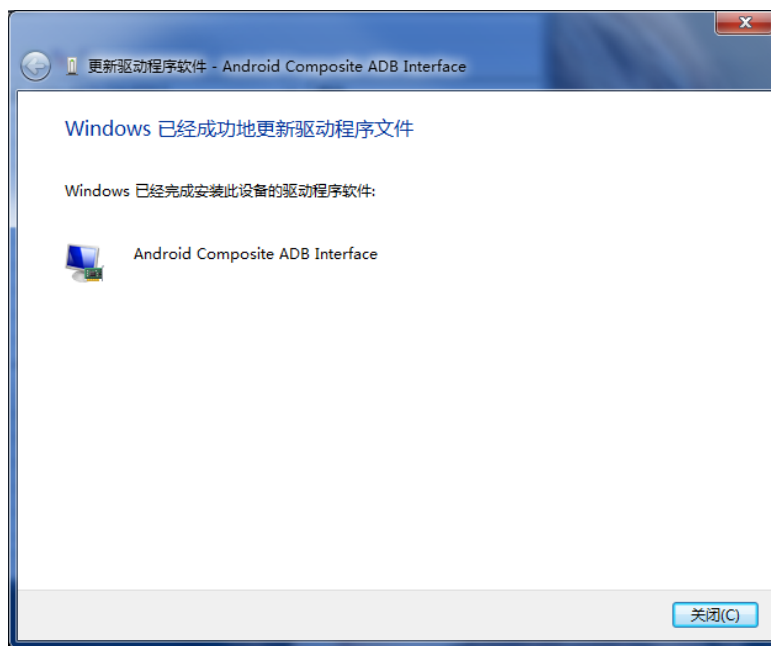
点击浏览选择驱动所在文件夹，此处选择“Android ADB Driver”并勾选“包括子文件夹”即可。



选择“始终安装此驱动程序软件”



安装成功后，选择关闭



注：WIN8 下要关闭强制驱动签名才能安装上驱动。

3.5.4 安装常见驱动提示错误

错误：文件的哈希值不在指定的文件中！

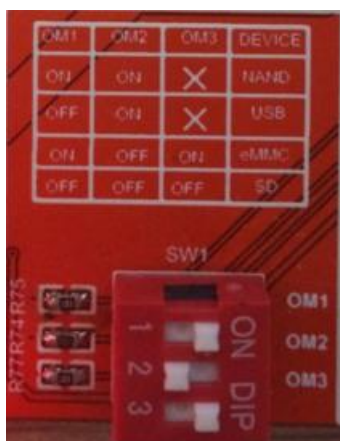
在 win8 及以上的系统，安装部分驱动可能会提示如上的错误，这是因为需要强制签名的原因。

Win8 以上系统关闭驱动强制签名的方式在【系统设置/更新和安全/恢复/高级启动/立即重启】，点击立即重启后，在后面的界面中选择【疑难解答/高级选项/启动设置/重启】，在重启过程中，按 7 或者 F7 选择关闭驱动强制签名。

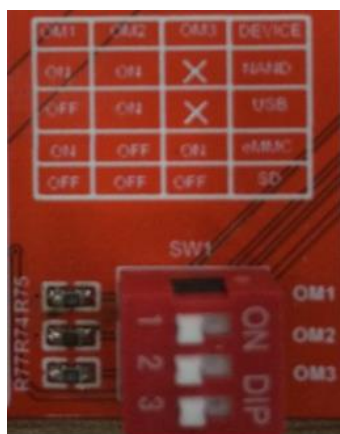
3.6 烧写系统

如果用 SD 卡上的 uboot 启动则将拨码拨至【000】，如果上电后串口终端无输出则说明 SD 卡启动盘没有制作成功，重新制作即可。

如果是用 eMMC 中 uboot 启动则将拨码拨至【101】，如果上电后串口终端无输出则说明 eMMC 中无 uboot 需制作 SD 启动盘通过 SD 卡启动。



eMMC 启动



SD 卡启动

打开开发板，在 uboot 倒计时期，按任意键结束然后输入“fastboot”，如下图所示。

fastboot

```
COM12 - PuTTY
** Partition 1 not valid on device 0 **
** Partition 1 not valid on device 0 **
Hit any key to stop autoboot: 0
FS6818# fastboot

Fastboot Partitions:
mmc.2: ubootpak, img : 0x200, 0x78000
mmc.2: 2ndboot, img : 0x200, 0x4000
mmc.2: bootloader, img : 0x8000, 0x70000
mmc.2: boot, fs : 0x100000, 0x4000000
mmc.2: system, fs : 0x4100000, 0x2f200000
mmc.2: cache, fs : 0x33300000, 0x1ac00000
mmc.2: misc, fs : 0x4e000000, 0x800000
mmc.2: recovery, fs : 0x4e900000, 0x1600000
mmc.2: userdata, fs : 0x50000000, 0x0
Support fstype : 2nd boot factory raw fat ext4 emmc nand ubi ubifs
Reserved part : partmap mem env cmd
DONE: Logo bmp 1024 by 600 (3bpp), len=1843256
DRAW: 0x47000000 -> 0x46000000
Load USB Driver: android
Core usb device tie configuration done
OTG cable Connected!
-----
```

安卓镜像在【华清远见-嵌入式 ARM 实验箱资料- I -FS6818\烧写镜像\Android 镜像\】目录下对应平台下的所有文件，用户需根据自己的设备平台进行选择，将镜像拷贝到任意路径下，打开 CMD 终端（同时按下 win 键+r 键后输入 cmd）进入安卓镜像拷贝的路径，执行下列操作：

```
fastboot flash ubootpak ubootpak.bin (指令一)
fastboot flash boot boot.img (指令二)
fastboot flash system system.img (指令三)
fastboot flash userdata userdata.img (指令四)
fastboot flash cache cache.img (指令五)
```

注意：烧写过程可能出现错误，错误原因：

1、 fastboot 驱动没有正确安装，正确安装即可



2、系统中一些手机助手软件占用了设备，退出或卸载这些软件即可

ubootpak.img 烧写：

cmd 终端显示如下：

```
E:\FS6818B光盘资料\镜像文件\Android镜像>fastboot flash ubootpak ubootpak.bin
target reported max download size of 38000000 bytes
sending 'ubootpak' (296 KB)...
OKAY [ 0.329s]
writing 'ubootpak'...
OKAY [ 0.155s]
finished. total time: 0.488s
```

串口终端显示如下：

```
Ready : ubootpak
Starting download of 303380 bytes
downloading 301056 -- 99% complete.
downloading of 303380 bytes to 0x48000000 (0x38000000) finished
Flash : ubootpak
head boot dev = 2
update mmc.2 type 2ndboot = 0x200(0x1) ~ 0x4a114(0x251): Done
Flash : ubootpak - DONE
```

boot.img 烧写：

cmd 终端显示如下：

```
E:\FS6818B光盘资料\镜像文件\Android镜像>fastboot flash boot boot.img
target reported max download size of 38000000 bytes
sending 'boot' (19544 KB)...
OKAY [ 3.067s]
writing 'boot'...
OKAY [ 3.912s]
finished. total time: 6.990s
```

串口终端显示如下：

```
Ready : boot
Starting download of 20013288 bytes
downloading 19813888 -- 99% complete.
downloading of 20013288 bytes to 0x48000000 (0x38000000) finished
Flash : boot
Flash : boot - DONE
```

system.img 烧写，system 镜像比较大所以烧写时分为多次烧写完成：

cmd 终端显示如下：

```
writing 'system'...
OKAY [ 3.026s]
sending sparse 'system' (32257 KB)...
OKAY [ 5.051s]
writing 'system'...
OKAY [ 3.575s]
finished. total time: 111.336s
```

串口终端显示如下：

```
downloading of 37999376 bytes to 0x48000000 (0x38000000) finished
Flash : system
Flash : system - DONE
Starting download of 33031468 bytes
downloading 32701952 -- 99% complete.
downloading of 33031468 bytes to 0x48000000 (0x38000000) finished
Flash : system
Flash : system - DONE
```



userdata.img 烧写:

cmd 终端显示如下:

```
E:\FS6818B光盘资料\镜像文件\Android镜像>fastboot flash userdata userdata.img
target reported max download size of 38000000 bytes
sending sparse 'userdata' (37108 KB)...
OKAY [ 5.779s]
writing 'userdata'...
OKAY [ 2.508s]
sending sparse 'userdata' (37108 KB)...
OKAY [ 5.785s]
writing 'userdata'...
OKAY [ 2.069s]
sending sparse 'userdata' (21413 KB)...
OKAY [ 3.385s]
writing 'userdata'...
OKAY [ 2.222s]
finished. total time: 21.769s
```

串口终端显示如下:

```
Ready : userdata
Starting download of 37998704 bytes
downloading 37619712 -- 99% complete.
downloading of 37998704 bytes to 0x48000000 (0x38000000) finished
Flash : userdata
Flash : userdata - DONE
Starting download of 37998656 bytes
downloading 37619200 -- 99% complete.
downloading of 37998656 bytes to 0x48000000 (0x38000000) finished
Flash : userdata
Flash : userdata - DONE
Starting download of 21927332 bytes
downloading 21708800 -- 99% complete.
downloading of 21927332 bytes to 0x48000000 (0x38000000) finished
Flash : userdata
Flash : userdata - DONE
```

cache.img 烧写:

cmd 终端显示如下:

```
E:\FS6818B光盘资料\镜像文件\Android镜像>fastboot flash cache cache.img
target reported max download size of 38000000 bytes
sending 'cache' (8752 KB)...
OKAY [ 1.437s]
writing 'cache'...
OKAY [ 1.517s]
finished. total time: 2.961s
```

串口终端显示如下:

```
Ready : cache
Starting download of 8962352 bytes
downloading 8873472 -- 99% complete.
downloading of 8962352 bytes to 0x48000000 (0x38000000) finished
Flash : cache
Flash : cache - DONE
```

安卓镜像烧写完成后我们需要重启开发板并查看当前环境变量是否正确:

重启开发板在串口查看开机倒计时结束前按任意键进入 bootloadrt 交互界面。使用 print 查看当前的



环境变量，如果用户在使用过程中修改了其中的参数，可以按照如下的值来设定：(使用 `setenv` 修改环境变量的值，修改完成之后使用 `saveenv` 保存环境变量)

使用 `print` 查看当前的环境变量，下图是使用 **eMMC 启动 Android 系统**时的默认参数。

```
FS6818# print
androidcrc=-1889320695
baudrate=115200
bootargs=lcd=vs070cxn tp=gs1x680
bootcmd=ext4load mmc 2:1 0x48000000 uImage;ext4load mmc 2:1 0x49000000 root.img.
gz;bootm 0x48000000
bootdelay=3
bootfile=uImage
ethaddr=00:e2:1c:ba:e8:60
fastboot=flash=mmc,2:ubootpak:2nd:0x200,0x78000;flash=mmc,2:2ndboot:2nd:0x200,0x
4000;flash=mmc,2:bootloader:boot:0x8000,0x70000;flash=mmc,2:boot:ext4:0x00100000
,0x04000000;flash=mmc,2:system:ext4:0x04100000,0x2F200000;flash=mmc,2:cache:ext4
:0x33300000,0x1AC00000;flash=mmc,2:misc:emmc:0x4E000000,0x00800000;flash=mmc,2:r
ecover:emmc:0x4E900000,0x01600000;flash=mmc,2:userdata:ext4:0x50000000,0x0;
filesize=1c2038
gatewayip=192.168.1.254
ipaddr=192.168.1.165
netmask=255.255.255.0
serverip=192.168.1.164
stderr=serial
stdin=serial
stdout=serial
```

如果用户在使用过程中修改了其中的参数，可以按照如下的值来设定：(使用 `setenv` 修改环境变量的值，修改完成之后使用 `saveenv` 保存环境变量)

```
FS6818 # setenv bootcmd ext4load mmc 2:1 0x48000000 uImage\;ext4load mmc 2:1 0x49000000
root.img.gz\;bootm 0x48000000

FS6818 # saveenv

FS6818 # setenv bootargs lcd=vs070cxn

FS6818 # saveenv
```

上边的命令一定需要保证不能够出现错误，否则会出现系统引导失败!!!!!!!!!!!!

修改完成后重启开发板即可。



第 4 章 Android 系统与嵌入式开发

嵌入式开发是当前最热门的领域之一，它无处不在，受到了社会各方面的广泛关注，越来越多的人开始学习嵌入式系统开发，Android 作为当下最热门的手机操作系统，与传统的嵌入式开发既有联系也有区别，本章将向读者介绍嵌入式开发的基本知识，以及它和 Android 系统的联系和区别，主要内容如下：

- (1) Android 与传统嵌入式 Linux 系统。
- (2) 嵌入式系统的概述。
- (3) 嵌入式系统的组成。
- (4) 嵌入式系统开发概述。
- (5) Android 系统的概述。
- (6) Android 系统架构。

4.1 Android 与传统嵌入式 Linux 系统

众所周知，Android 操作系统是基于 Linux 内核的，Google 使用 Linux 内核构建了一个可移植的和健壮的手机平台，而并没有使用传统 Linux 的任何其他东西，基于这个特点，现在从事 Android 开发的人员一共分两大类，一类是从事 Android 应用开发人员，他们使用的开发语言主要是 Java；另一类是 Android 底层开发人员，他们使用的语言主要是 C/C++ 语言。

Android 系统不是完全依赖于 Linux 内核的，这和传统的 Linux 系统还是不一样的，但是从系统移植和驱动开发者的角度来讲，Android 底层的开发移植和传统嵌入式 Linux 系统开发关系非常密切，Android 系统的驱动与 Linux 的驱动在开发上几乎保持了完全一致，另外，Android 底层开发和移植的环境也与嵌入式 Linux 的环境保持了基本一致。

4.2 嵌入式系统概述

4.2.1 嵌入式系统简介

在我们这个时代，嵌入式系统早就广泛应用在科学研究、工程设计、军事技术、各类产业、商业文化艺术、娱乐业，以及人们的日常生活等方方面面，随着数字信息技术和网络技术的飞速发展，计算机、通信、消费电子的一体化趋势日益明显，这必将培育出一个庞大的嵌入式应用市场。嵌入式系统技术也成了当前关注、学习研究的热点。大家可能会问，究竟什么是嵌入式系统呢？嵌入式系统本身是一个相对模糊的定义，不同的组织对其定义也略有不同，但大意是相同的，我们来看一下嵌入式系统的相关定义。按照 IEEE 的定义，嵌入式系统是用来控制、监控或辅助操作机器、装置、工厂等大规模系统的设备（devices used to control, monitor, or assist the operation of equipment, machinery or plants）。这主要是从嵌入式系统的用途方面来进行定义的。

更具一般性，也是在多数书籍资料中使用的关于嵌入式系统的定义：嵌入式系统是指以应用为中心，以计算机技术为基础，软件硬件可裁剪，适应应用系统对功能、可靠性、成本、体积、功耗严格要求的专用计算机系统。

根据以上嵌入式系统的定义，我们可以看出，嵌入式系统是由硬件和软件相结合组成的具有特定功



能、用于特定场合的独立系统。其硬件主要由嵌入式微处理器、外围硬件设备组成；其软件主要由底层系统软件 and 用户应用软件组成。

4.2.2 嵌入式系统的特点

从嵌入式系统的定义可以看出，嵌入式系统是面向应用的，与通用系统最大的区别在于嵌入式系统功能专一。根据这个特性，嵌入式系统的软、硬件可以根据需要进行精心设计、量体裁衣、去除冗余，以实现低成本、高性能。也正因如此，嵌入式系统采用的微处理器和外围设备种类繁多，系统不具有通用性。

嵌入式系统大多用在特定场合，要么是环境条件恶劣，要么要求其长时间连续运转，因此嵌入式系统应具有高可靠性、高稳定性、低功耗等特点。

由于成本和应用场合的特殊性，通常嵌入式系统的硬件资源（如内存等）都比较少，因此对嵌入式系统设计也提出了较高的要求。嵌入式系统的软件设计要求高质量，要在有限资源上实现高可靠性、高性能的系统。虽然随着硬件技术的发展和成本的降低，在高端嵌入式产品上也开始采用嵌入式操作系统，但其和 PC 资源比起来还是少得可怜，所以，嵌入式系统的软件代码依然要在保证性能的情况下，占用尽量少的资源，保证产品的高性价比，使其具有更强的竞争力。

为了提高执行速度和系统可靠性，嵌入式系统中的软件一般都固化在存储器芯片或单片机本身中，而不是存储于磁盘中。

很多采用嵌入式系统的应用具有实时性要求，所以大多数嵌入式系统采用实时性系统。但需要注意的是，嵌入式系统不等于实时系统。

嵌入式系统不仅功能强大，而且要求使用灵活、方便，一般不需要键盘、鼠标等。人机交互以简单方便为主。

嵌入式软件开发有别于桌面软件系统开发的一个显著特点是，它一般需要一个交叉编译和调试环境，即编辑和编译软件在主机上进行（如在 PC 的 Windows 操作系统下），编译好的软件需要下载到目标机上运行（如在一个 PC 目标机上的 VxWorks 操作系统下），主机和目标机建立起通信连接，并传输调试命令和数据。由于主机和目标机往往运行着不同的操作系统，而且处理器的体系结构也彼此不同，这就提高了嵌入式开发的复杂性。

嵌入式系统是将先进的计算机技术、半导体技术和电子技术与各个行业的具体应用相结合后的产物。这一点就决定了它必然是一个技术密集、资金密集、高度分散、不断创新的知识集成系统，从事嵌入式系统开发的人才也必须是复合型人才。

4.2.3 嵌入式系统的发展

第一阶段是以单芯片为核心的可编程控制器形式的系统。这类系统大部分应用于一些专业性强的工业控制系统中，一般没有操作系统的支持，软件通过汇编语言编写。这一阶段系统的主要特点是：系统结构和功能相对单一，处理效率较低，存储容量较小，几乎没有用户接口。由于这种嵌入式系统使用简单、价格低，因此当时在国内工业领域应用较为普遍，但是现在已经远不能适应高效的、需要大容量存储的现代工业控制和新兴信息家电等领域的需求。



第二阶段是以嵌入式 CPU 为基础、以简单操作系统为核心的嵌入式系统。其主要特点是：CPU 种类繁多，通用性比较弱；系统开销小，效率高；操作系统达到一定的兼容性和扩展性；应用软件较专业化，用户界面不够友好。

第三阶段是以嵌入式操作系统为标志的嵌入式系统。其主要特点是：嵌入式操作系统能运行于各种不同类型的微处理器上，兼容性好；操作系统内核小、效率高，并且具有高度的模块化和扩展性；具备文件和目录管理、支持多任务、支持网络应用、具备图形窗口和用户界面；具有大量的应用程序接口（API），开发应用程序较简单；嵌入式应用软件丰富。

第四阶段是以 Internet 为标志的嵌入式系统。这是一个正在迅速发展的阶段，目前大多数嵌入式系统还孤立于 Internet 之外，但随着 Internet 的发展及 Internet 技术与信息家电、工业控制技术结合日益密切，嵌入式设备与 Internet 的结合将代表嵌入式系统的未来。

随着技术水平的提高和人们生活的需要，嵌入式设备（尤其是消费类产品）正朝着小型化便携式和智能化的方向发展。如果携带笔记本电脑外出办事，肯定希望它轻薄小巧，甚至希望有一种更便携的设备来替代它，目前的上网本、MID（移动互联网设备）、便携投影仪等都是应类似的需求而出现的。对嵌入式而言，可以说是已经进入了嵌入式互联网时代（有线网、无线网、广域网和局域网的组合），嵌入式设备和互联网的紧密结合，更为我们的日常生活带来了极大的方便和无限的想象空间。嵌入式设备功能越来越强大，未来我们的冰箱、洗衣机等家用电器都将实现网上控制；异地通信、协同工作、无人操控场所、安全监控场所等的可视化已经成为了现实，随着网络运载能力的提升，可视化将得到进一步完善。人工智能、模式识别技术也将在嵌入式系统中得到应用，使得嵌入式系统更具人性化、智能化。

人们需要处理的信息越来越多，这就要求嵌入式设备运算能力更强，因此需要设计出更强大的嵌入式处理器，多核技术处理器在嵌入式中的应用将更为普遍。

在嵌入式系统的硬件和软件设计中都在追求更低的功耗，以求嵌入式系统能获得更长的可靠工作时间。如手机的通话和待机时间、MP3 听音乐的时间，等等。同时，绿色环保型嵌入式产品将更受人们青睐，在嵌入式系统设计中也会更多的考虑，如辐射和静电等问题。

简单来讲，云计算是将计算分布在大量的分布式计算机上，这样只需要一个终端，就可以通过网络服务来实现需要的计算任务，甚至是超级计算任务。云计算（Cloud Computing）是分布式处理（Distributed Computing）、并行处理（Parallel Computing）和网格计算（Grid Computing）的发展，或者说是这些计算机科学概念的商业实现。在未来几年内，云计算将得到进一步发展与应用。

可重构性是指在一个系统中，其硬件模块或（和）软件模块均能根据变化的数据流或控制流对系统结构和算法进行重新配置（或重新设置）。可重构系统最突出的优点就是能够根据不同的应用需求，改变自身的体系结构，以便与具体的应用需求相匹配。

虚拟化是指计算机软件在一个虚拟的平台上而不是真实的硬件上运行。虚拟化技术可以简化软件的重新配置过程，易于实现软件标准化。其中，CPU 的虚拟化可以单 CPU 模拟多 CPU 并行运行，允许一个平台同时运行多个操作系统，并且都可以在相互独立的空间内运行而互不影响，从而提高工作效率和安全性，虚拟化技术是降低多内核处理器系统开发成本的关键。虚拟化技术是未来几年最值得期待和关注的关键技术之一。



随着各种技术的成熟与在嵌入式系统中的应用，将不断为嵌入式系统增添新的魅力和发展空间。

嵌入式操作系统将进一步走向开放、开源、标准化、组件化。嵌入式软件开发平台化也将是今后的一个趋势，越来越多的嵌入式软、硬件行业标准将出现，最终的目标是使嵌入式软件开发简单化，这也是一个必然规律。同时随着系统复杂度的提高，系统可升级和代码复用技术在嵌入式系统中得到更多的应用。另外，因为嵌入式系统采用的微处理器种类多，不够标准，所以在嵌入式软件开发中将更多地使用跨平台的软件开发语言与工具，目前，Java 语言正在被越来越多地使用到嵌入式软件开发中。

需求和网络技术的发展是嵌入式系统发展的一个源动力，随着移动互联网的发展，将进一步促进嵌入式系统软件 PC 化。如前所述，结合跨平台开发语言的广泛应用，那么未来嵌入式软件开发的概念将被逐渐淡化，也就是嵌入式软件开发和非嵌入式软件开发的区别将逐渐减小。

嵌入式系统软硬件融合、产品功能融合、嵌入式设备和互联网的融合趋势加剧。嵌入式系统设计中软硬件结合将更加紧密，软件将是其核心。消费类产品将在运算能力和便携方面进一步融合。传感器网络迅速发展，将极大地促进嵌入式技术和互联网技术的融合。

随着嵌入式技术和互联网技术的结合发展，嵌入式系统的信息安全问题日益凸显，保证信息安全也成为了嵌入式系统开发的重点和难点。

4.3 嵌入式系统的组成

由前面的介绍我们可以知道，嵌入式系统总体上是由硬件和软件组成的，硬件是基础，软件是核心与灵魂。它们之间的关系如图 1.1 所示。

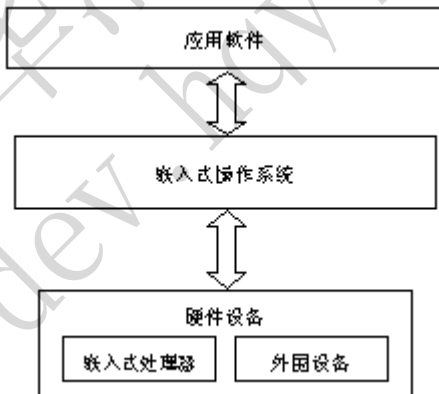


图 1.1

4.3.1 嵌入式系统的硬件组成

嵌入式系统硬件设备包括嵌入式处理器和外围设备。其中，嵌入式处理器（CPU）是嵌入式系统的核心部分，它与通用处理器最大的区别在于，嵌入式处理器大多工作在为特定用户群所专门设计的系统中，它将通用处理器中许多由板卡完成的任务集成到芯片内部，从而有利于嵌入式系统在设计时趋于小型化，同时还具有很高的效率和可靠性。如今，全世界嵌入式处理器已经超过 1000 多种，流行的体系结构有 30 多个系列，其中以 ARM、PowerPC、MC 68000、MIPS 等使用得最为广泛。

外围设备是嵌入式系统中用于完成存储、通信、调试、显示等辅助功能的其他部件。目前常用的嵌入式外围设备按功能可以分为存储设备（如 RAM、SRAM、Flash 等）、通信设备（如 RS-232 接口、SPI



接口、以太网接口等）和显示设备（如显示屏等）3 类。

常见存储器概念辨析：RAM、SRAM、SDRAM、ROM、EPROM、EEPROM、Flash。

存储器可以分为很多种类，其中根据掉电后数据是否丢失可以分为 RAM（随机存取存储器）和 ROM（只读存储器），其中 RAM 的访问速度比较快，但掉电后数据会丢失，而 ROM 掉电后数据不会丢失。人们通常所说的内存即指系统中的 RAM。

RAM 又可分为 SRAM（静态存储器）和 DRAM（动态存储器）。SRAM 是利用双稳态触发器来保存信息的，只要不掉电，信息是不会丢失的。DRAM 是利用 MOS（金属氧化物半导体）电容存储电荷来存储信息的，因此必须通过不停地给电容充电来维持信息，所以 DRAM 的成本、集成度、功耗等明显优于 SRAM。

而通常人们所说的 SDRAM 是 DRAM 的一种，它是同步动态存储器，利用一个单一的系统时钟同步所有的地址数据和控制信号。使用 SDRAM 不但能提高系统表现，还能简化设计、提供高速的数据传输。在嵌入式系统中经常使用。

EPROM、E2PROM 都是 ROM 的一种，分别为可擦除可编程 ROM 和电可擦除 ROM，但使用不是很方便。

Flash 也是一种非易失性存储器（掉电不会丢失），它擦写方便，访问速度快，已大大取代了传统的 EPROM 的地位。由于它具有和 ROM 一样掉电不会丢失的特性，因此很多人称其为 Flash ROM。

4.3.2 嵌入式系统的软件组成

在嵌入式系统不同的应用领域和不同的发展阶段，嵌入式系统软件组成也不完全相同，大致如图 1.2 所示。

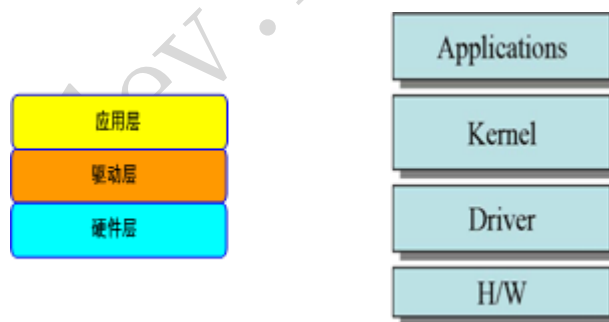


图 1.2

如图 2.2（a）所示，在某些特殊领域，嵌入式系统软件没有使用如通用计算机系统的。嵌入式操作系统从嵌入式发展的第三阶段起开始引入。嵌入式操作系统不仅具有通用操作系统的一般功能，如向上提供对用户的接口（如图形界面、库函数 API 等），向下提供与硬件设备交互的接口（硬件驱动程序等），管理复杂的系统资源，同时，它还在系统实时性、硬件依赖性、软件固化性及应用专用性等方面，具有更加鲜明的特点。

应用软件是针对特定应用领域，基于某一固定的硬件平台，用来达到用户预期目标的计算机软件。由于嵌入式系统自身的特点，决定了嵌入式应用软件不仅要求做到准确性、安全性和稳定性等方面的需



要，而且还要尽可能地进行代码优化，以减少对系统资源的消耗，降低硬件成本。

4.4 嵌入式系统开发概述

由嵌入式系统本身的特性所影响，嵌入式系统开发与通用系统的开发有很大的区别。嵌入式系统的开发主要分为系统总体开发、嵌入式硬件开发和嵌入式软件开发三大部分，其总体流程图如图 1.3 所示。

在系统总体开发中，由于嵌入式系统与硬件依赖程序非常紧密，往往某些需求只能通过特定的硬件才能实现，因此需要进行处理器选型，以更好地满足产品的需求。另外，对于有些硬件和软件都可以实现的功能，就需要在成本和性能上做出选择。往往通过硬件实现会增加产品的成品，但能大大提高产品的性能和可靠性。

再次，开发环境的选择对于嵌入式系统的开发也有很大的影响。这里的开发环境包括嵌入式操作系统的选择及开发工具的选择等。1.2 节对各种不同的嵌入式操作系统进行了比较，读者可以以此为依据进行相关的选择。例如，对开发成本和进度限制较大的产品可以选择嵌入式 Linux，对实时性要求非常高的产品可以选择 VxWorks 等。

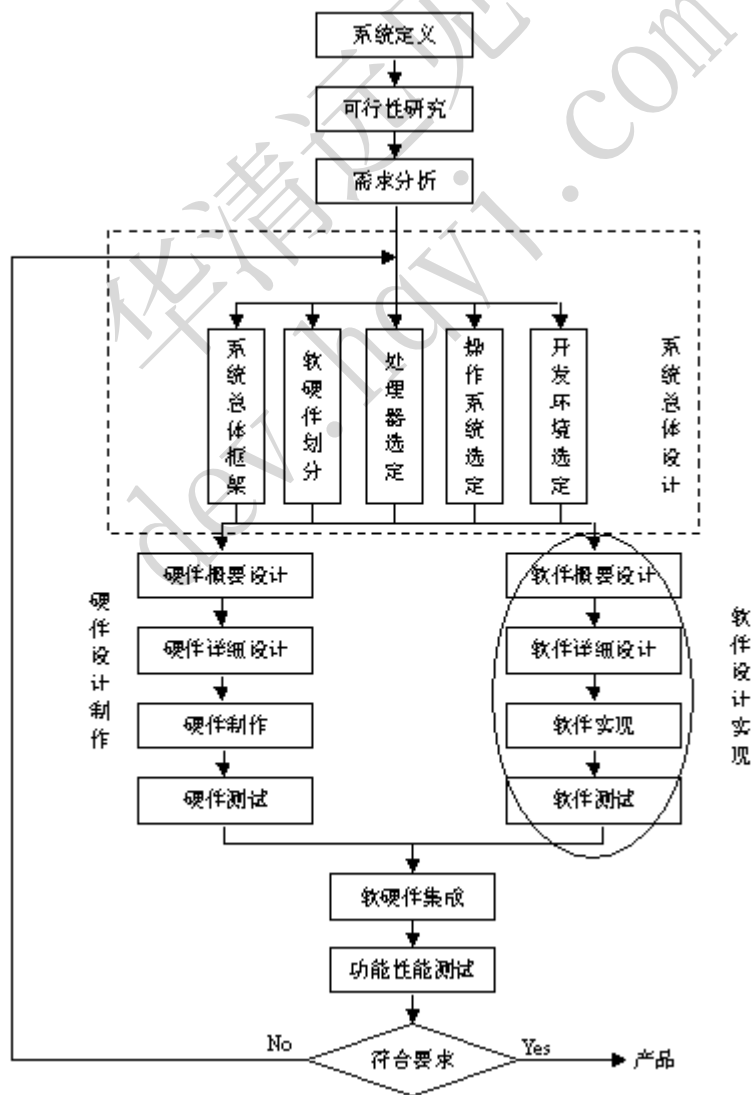


图 1.3



嵌入式软件开发总体流程为图 1.3 中“软件设计实现”部分所示，它同通用计算机软件开发一样，分为需求分析、软件概要设计、软件详细设计、软件实现和软件测试。其中嵌入式软件需求分析与硬件的需求分析合二为一，故没有分开画出。

由于嵌入式软件开发的工具非常多，为了更好地帮助读者选择开发工具，下面首先对嵌入式软件开发过程中所使用的工具进行简单归纳。

嵌入式软件的开发工具根据不同的开发过程划分，例如，在需求分析阶段，可以选择 IBM 的 Rational Rose 等软件；而在程序开发阶段可以采用 CodeWarrior（下面要介绍的 ADS 的一个工具）等；在调试阶段所用的 Multi-ICE 等。同时，不同的嵌入式操作系统往往会有配套的开发工具，如 VxWorks 的集成开发环境 Tornado、WinCE 的集成开发环境 WinCE Platform 等。此外，不同的处理器可能还有针对的开发工具，如 ARM 的常用集成开发工具 ADS 等。在这里，大多数软件都有比较高的使用费用，但也可以大大加快产品的开发进度，用户可以根据需求自行选择。如图 1.4 所示为嵌入式开发的不同阶段的常用软件。

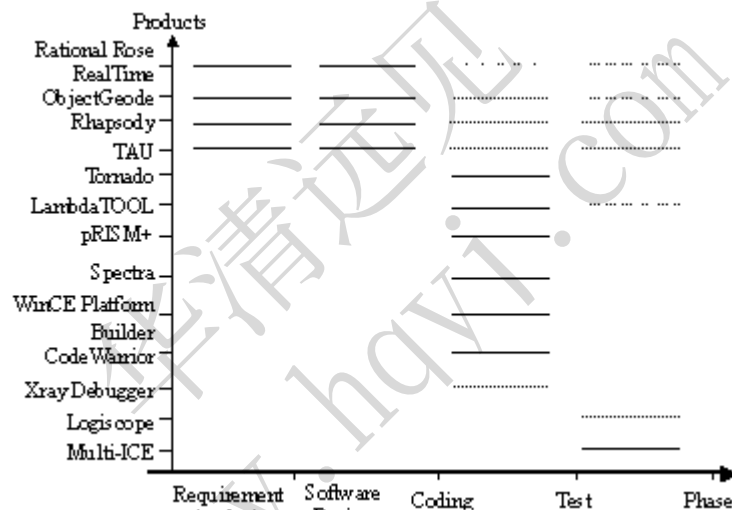


图 1.4

嵌入式系统的软件开发与通常软件开发的区别主要在于软件实现部分，其中又可以分为编译和调试两部分，下面分别对这两部分进行讲解。

4.4.1 交叉编译

嵌入式软件开发所采用的编译为交叉编译。所谓交叉编译就是在一个平台上生成可以在另一个平台上执行的代码。因此，不同的 CPU 需要有相应的编译器，而交叉编译就如同翻译一样，把相同的程序代码翻译成不同的 CPU 对应语言。需要注意的是，编译器本身也是程序，也要在与之对应的某一个 CPU 平台上运行。

一般把进行交叉编译的主机称为宿主机，也就是普通的通用计算机，而把程序实际的运行环境称为目标机，也就是嵌入式系统环境。由于一般通用计算机拥有非常丰富的系统资源、使用方便的集成开发环境和调试工具等，而嵌入式系统的系统资源非常紧缺，没有相关的编译工具，因此，嵌入式系统的开发需要借助宿主机（通用计算机）来编译出目标机的可执行代码。



由于编译的过程包括编译、链接等几个阶段，因此，嵌入式的交叉编译也包括交叉编译、交叉链接等过程，通常 ARM 的交叉编译器为 `arm.elf.gcc`，交叉链接器为 `arm.elf.ld`，交叉编译过程如图 1.5 所示。

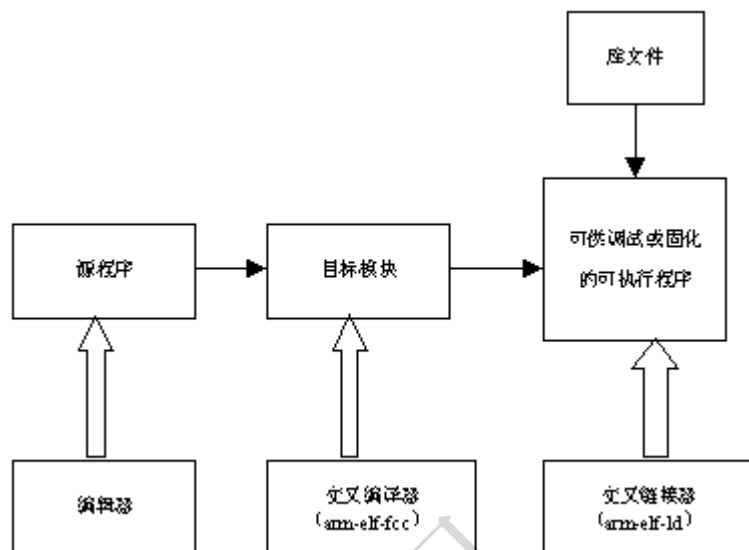


图 1.5

4.4.2 交叉调试

嵌入式软件经过编译和链接后即进入调试阶段，调试是软件开发过程中必不可少的一个环节，嵌入式软件开发过程中的交叉调试与通用软件开发过程中的调试方式有很大的差别。在常见软件开发中，调试器与被调试的程序往往运行在同一台计算机上，调试器是一个单独运行着的进程，它通过操作系统提供的调试接口来控制被调试的进程。而在嵌入式软件开发中，调试时采用的是在宿主机和目标机之间进行的交叉调试，调试器仍然运行在宿主机的通用操作系统之上，但被调试的进程却运行在基于特定硬件平台的嵌入式操作系统中，调试器和被调试进程通过串口或者网络进行通信，调试器可以控制、访问被调试进程，读取被调试进程的当前状态，并能够改变被调试进程的运行状态。

- (1) 嵌入式系统的交叉调试有多种方法，主要可分为软件方式和硬件方式两种。它们一般都具有如下一些典型特点。
- (2) 调试器和被调试进程运行在不同的机器上，调试器运行在 PC 或者工作站上（宿主机），而被调试的进程则运行在各种专业调试板上（目标机）。
- (3) 调试器通过某种通信方式（串口、并口、网络、JTAG 等）控制被调试进程。
- (4) 在目标机上一般会具备某种形式的调试代理，它负责与调试器共同配合完成对目标机上运行着的进程的调试。这种调试代理可能是某些支持调试功能的硬件设备，也可能是某些专门的调试软件（如 GDBServer）。
- (5) 目标机可能是某种形式的系统仿真器，通过在宿主机上运行目标机的仿真软件，整个调试过程可以在一台计算机上运行。此时物理上虽然只有一台计算机，但逻辑上仍然存在着宿主机和目标机的区别。

下面分别就软件调试桩方式和硬件片上调试两种方式进行详细介绍。



1 软件调试

软件调试方式主要是通过插入调试桩的方式来进行的。调试桩方式进行调试是通过目标操作系统和调试器内分别加入某些功能模块，二者互通信息来进行调试。该方式的典型调试器有 GDB 调试器。

GDB 的交叉调试器分为 GDBServer 和 GDBClient，其中 GDBServer 作为调试桩安装在目标板上，GDBClient 即驻于本地的 GDB 调试器。它们的调试原理图如图 1.6 所示。

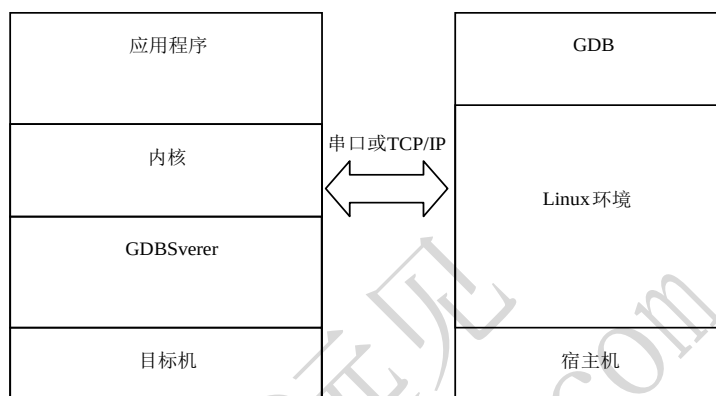


图 1.6

GDB 调试桩的工作流程如下：

- (6) 建立调试器（本地 GDB）与目标操作系统的通信连接，可通过串口、网卡、并口等多种方式。
- (7) 在目标机上开启 GDBServer 进程，并监听对应端口。
- (8) 在宿主机上运行调试器 GDB，这时，GDB 就会自动寻找远端的通信进程，也就是 GDBServer 所在的进程。
- (9) 在宿主机上的 GDB 通过 GDBServer 请求对目标机上的程序发出控制命令。这时，GDBServer 将请求转化为程序的地址空间或目标平台的某些寄存器的访问，这对于没有虚拟存储器的简单嵌入式操作系统而言，是十分容易的。
- (10) GDBServer 把目标操作系统的所有异常处理转向通信模块，并告知宿主机上 GDB 当前异常。

宿主机上的 GDB 向用户显示被调试程序产生了哪类异常。

这样就完成了调试的整个过程。这个方案的实质是用软件接管目标机的全部异常处理及部分中断处理，并在其中插入调试端口通信模块，与主机的调试器进行交互。但是它只能在目标机系统初始化完毕、调试通信端口初始化完成后才能起作用，因此，一般只能用于调试运行于目标操作系统之上的应用程序，而不宜用来调试目标操作系统的内核代码及启动代码。而且，它必须改变目标操作系统，因此，也就多了一个不用于正式发布的调试版。



2 硬件调试

相对于软件调试而言，使用硬件调试器可以获得更强大的调试功能和更优秀的调试性能。硬件调试器的基本原理是通过仿真硬件的执行过程，让开发者在调试时可以随时了解到系统的当前执行情况。目前，嵌入式系统开发中最常用到的硬件调试器是 ROMMonitor、ROMEmulator、In.Circuit Emulator 和 In.Circuit Debugger。

采用 ROMMonitor 方式进行交叉调试需要在宿主机上运行调试器，在目标机上运行 ROM 监视器（ROMMonitor）和被调试程序，宿主机通过调试器与目标机上的 ROM 监视器遵循远程调试协议建立通信连接。ROM 监视器可以是一段运行在目标机 ROM 上的可执行程序，也可以是一个专门的硬件调试设备，它负责监控目标机上被调试程序的运行情况，能够与宿主机端的调试器一同完成对应用程序的调试。

4.5 Android 系统概述

Android 一词英文本义是指“机器人”，它是由 Google 公司于 2007 年 11 月宣布的基于 Linux 平台的开源手机操作系统，依靠 Google 的强大开发和媒体资源，Android 成为众多手机厂商竞相追逐的对象，逐渐发展成为目前最流行的手机开发平台。

Android 是一个包括操作系统、中间件、用户界面和关键应用软件的移动设备软件堆。换句话说，就是 Android 是基于 Java 并运行在 Linux 内核上的轻量级操作系统，功能全面，包括一系列 Google 公司在其上内置的应用软件，如电话、短信等基本应用功能。

4.6 Android 系统架构

Android 系统从下至上分为 4 层：Linux 内核、Android 核心库及 Android 运行时环境（Android Runtime）、应用程序框架、应用程序及小部件，如下图所示。





4.6.1 Linux 内核 (Linux Kernel)

Android 的核心系统服务依赖于 Linux，例如，安全、内存管理、进程管理、网络堆栈、驱动模型。Linux Kernel 也作为硬件和软件之间的抽象层，它隐藏具体硬件细节而为上层提供统一的服务。

除了标准的 Linux 内核以外，Android 系统还增加了内核的驱动程序，例如，显示驱动、蓝牙驱动、相机驱动、闪存卡驱动、Binder IPC 驱动、输入设备驱动、USB 驱动、WiFi 驱动、音频系统驱动、电源管理等，为 Android 系统的运行提供基础性支持。

这样分层的好处就是使用下层提供的服务而为上层提供统一的服务，屏蔽本层及以下层的差异，当本层及以下层发生了变化时不会影响到上层。也就是说各层各尽其职，各层提供固定的 SAP (Service Access Point)，即高内聚、低耦合。

4.6.2 Android 核心库 (Libraries)

Android 包含一个 C/C++ 库的集合，以供 Android 系统的各个组件使用。这些功能通过 Android 的应用程序框架 (Application Framework) 暴露给开发者。下面列出一些核心库。

- (1) 系统 C 库：由 BSD 继承衍生的标准 C 系统函数库 (libc)，调整为基于嵌入式 Linux 设备的库。
- (2) 媒体库：基于 PacketVideo 的 OpenCore。这些库支持播放和录制多种流行的音频和视频格式，以及多种媒体格式的编码/解码格式，包括 MPEG4、H.264、MP3、AAC、AMR、JPG、PNG。
- (3) 界面管理：显示子系统的管理器，管理访问显示子系统和无缝组合多个应用程序的二维和三维图形层。
- (4) LibWebCore：新式的 Web 浏览器引擎，驱动 Android 浏览器和可嵌入的 Web 视图。
- (5) SGL：Skia 图形库，基本的 2D 图形引擎。
- (6) 3D libraries：基于 OpenGL ES APIs 的实现。该库使用硬件 3D 加速或使用高度优化的 3D 软加速。
- (7) FreeType：位图 (Bitmap) 和矢量 (Vector) 字体渲染。
- (8) SQLite：所有应用程序都可以使用的强大而轻量级的关系数据库引擎。

4.6.3 Android 运行时环境 (Android Runtime)

在 Linux 内核层上还有一个 Android 运行时层，该层包括 Dalvik 虚拟机及 Java 核心库，提供了 Java 编程语言核心库的大多数功能。

Dalvik 虚拟机是 Android 使用的 Java 虚拟机。每一个 Android 应用程序是 Dalvik 虚拟机中的实例，运行在它们自己的进程中。Dalvik 虚拟机设计成在一个设备中可以高效地运行多个虚拟机。Dalvik 虚拟机可执行文件格式是 .dex，dex 格式是专为 Dalvik 设计的一种压缩格式，适合内存和处理器速度有限的系统。

大多数虚拟机包括 JVM 都是基于栈的，而 Dalvik 虚拟机则是基于寄存器的。两种架构各有优劣，一般而言，基于栈的机器需要更多指令，而基于寄存器的机器指令更大。dx 是一套工具，可以将 Java .class 转换成 dex 格式。一个 .dex 文件通常会有多个 .class。由于 .dex 有时必须进行最佳化，会使文件大小增加 1.4 倍，以 ODEX 结尾。

Dalvik 虚拟机依赖于 Linux 内核提供基本功能，如线程和底层内存管理。



4.6.4 Android 应用程序框架（Application Framework）

位于 Android 程序库和运行时上面的是应用程序框架层。通过提供开放的开发平台，Android 使开发者能够访问核心应用程序所使用的 API 框架，这样使得组件的重用得以简化，任何应用程序都能发布它的功能且任何其他应用程序可以使用这些功能（需要服从框架执行的安全限制）。从而使得开发者可以编制极其丰富和新颖的应用程序，自由地利用设备硬件优势、访问位置信息、运行后台服务、设置闹钟、向状态栏添加通知等。

每个应用程序其实是一组服务和系统，包括如下几项。

- (1) 视图（View）：丰富的、可扩展的视图集合，用来构建应用程序。包括列表（ListView）、网格（Grid）、文本框（EditText/TextView）、按钮（Button）等，甚至是可嵌入的网页浏览器（WebView）。
- (2) 内容提供者（Content Providers）：使应用程序可以访问其他应用程序（如通讯录）的数据，或共享自己的数据。
- (3) 资源管理器（Resource Manager）：提供对于非代码资源的访问，如本地化字符串、图形和布局文件。
- (4) 通知管理器（Notification Manager）：使得应用程序能够在状态栏显示自定义的提示信息。
- (5) 活动管理器（Activity Manager）：管理应用程序生命周期，并提供常用的导航回退功能。

4.6.5 Android 应用程序和小部件

Android 装配一个核心应用程序集合，连同系统一起发布，这些应用程序包括电子邮件客户端、SMS 程序、日历、地图、浏览器、联系人和其他设置等。所有应用程序都是用 Java 语言编写的，由用户开发的 Android 应用程序和 Android 核心应用程序是同一层次的。

4.7 小结

本章主要介绍了传统嵌入式开发和 Android 系统之间的关系，如果以前是一个嵌入式软件工程师，那么阅读了本章后就会有一种恍然大悟的感觉，从而更快了解 Android 系统。另外，本章在前面的基础上分析了 Android 系统的层次结构，说明了 Android 系统和嵌入式 Linux 系统之间的关系。



第 5 章 Android JNI 层解析

5.1 JNI 介绍

Android 的应用层由 Java 语言编写，Framework 框架层则是由 Java 代码与 C/C++ 语言实现，之所以由两种不同的语言组合开发框架层，是因为 Java 代码是与硬件环境彻底“隔离”的跨平台语言，Java 代码无法直接操作硬件。比如：Android 系统支持大量传感器，Java 运行在虚拟机中，无法直接得到传感器数据，而 Android 系统基于 Linux 操作系统，在 Linux 操作系统中 C/C++ 通过 Linux 提供的系统调用接口可以直接访问传感器硬件驱动，Java 代码可以将自己的请求，交给底层的本地 C/C++ 代码实现间接的对传感器的访问。另外，Java 代码的执行效率要比 C/C++ 执行效率要低，在一些对性能要求比较高的场合，也要使用 C/C++ 来实现程序逻辑。

既然 Java 代码要请求本地 C/C++ 代码，那么二者必须要通过一种媒介或桥梁联系起来，这种媒介就是 Java 本地接口（Java Native Interface），它是 Java 语言支持的一种本地语言访问方式，JNI 提供了一系列接口，它允许 Java 与 C/C++ 语言之间通过特定的方式相互调用、参数传递等交互操作。

5.2 JNI 原理

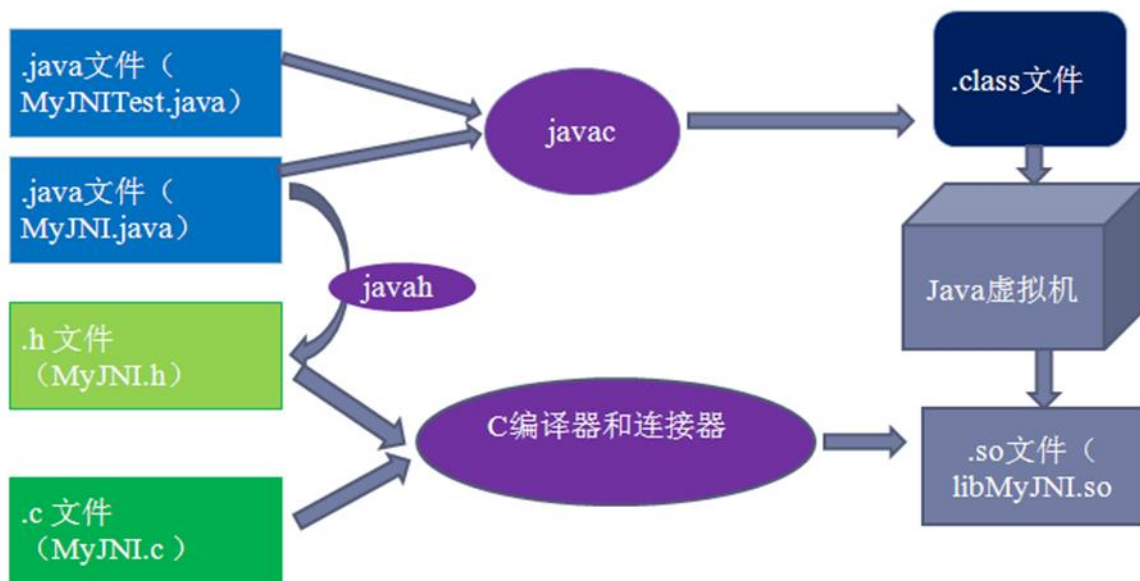


图 5.1

JNI 的调用结构如图 5.1 所示，Java 类中需要实现的是声明本地方法和加载共享库，然后就可以像平时一样使用对象了。

Java 程序与 JNI 编译成的共享库之间联系如下：

Java 类加载共享库用 `loadlibrary` 方法，名称与 JNI 共享库名称对应。

Java 调用 JNI 中的类需要声明本地方法，其中本地方法的类型需要与 JNI 中 `JNINativeMethod` 中提供的方法声明一致。

JNI 类中需要在 `jni helper` 函数声明调用 Java 类，这样才能实现 Java 虚拟机与本地方法的联系。



Java 服务类的全称需与 JNI 的文件名有一定的对应关系，如 Java 服务类名为 com.farsight.Service.LedService，对应 JNI 的 service 应用名为 com_farsight_service_LedService.cpp。

5.3 JNI 使用方法

首先，我们需要编写一个需要调用本地方法的 java 程序,在 java 中声明一个本地方法。

```
public class A
{
    int size = 5;
    public static void main(String args[])
    {
        A a = new A();
        System.out.println(a.size);
        a.addSize();
        System.out.println("after addsize size = "+a.size);
    }

    public native void addSize();    //声明本地方法

    static{
        System.loadLibrary("A");    //启动时加载对应的库文件
    }
}
```

此时，上面的程序只能编译，不能运行。运行将产生一个异常对象 java.lang.UnsatisfiedLinkError。需要实现对应的 C 代码。运行 javah 获得头文件，包括这个本地方法的声明格式。

```
/* DO NOT EDIT THIS FILE - it is machine generated */
#include <jni.h>
/* Header for class A */

#ifndef _Included_A
#define _Included_A
#ifdef __cplusplus
extern "C" {
#endif
/*
 * Class:      A
 * Method:     addSize
 */
}
```



```
* Signature: ()V
*/
JNIEXPORT void JNICALL Java_A_addSize (JNIEnv *, jobject);    //我们需要的函数原型

#ifdef __cplusplus
}
#endif
#endif
```

使用 c 语言来实现本地方法。

```
#include <stdio.h>
#include "A.h"    //需要包含生成的头文件
JNIEXPORT void JNICALL Java_A_addSize(JNIEnv *env, jobject oj)
{
    //get the class
    jclass class_A = (*env)->GetObjectClass(env,oj);
    //get the field ID
    jfieldID id_size = (*env)->GetFieldID(env,class_A,"size","I");
    //get value
    jint size = (*env)->GetIntField(env,oj,id_size);
    size++;    //实现变量的自加
    //set the field value
    (*env)->SetIntField(env,oj,id_size,size);
}
```

通过 `gcc -share` 命令来生成.so 文件，将其放入到共享类库中并把当前目录添加到库路径当中。

之后再通过 `java` 命令来编译.java 文件即可生成可以执行的.class 文件了。

5.4 小结

本章主要介绍了 JNI 的相关理论和原理，并通过一个实例来说明 Java 中如何使用 JNI 来调用本地 C 语言的方法。读者可以通过这一章来了解整个程序实现的框架，结合下一章 HAL 层解析可以更好地理解 Android 底层实现。

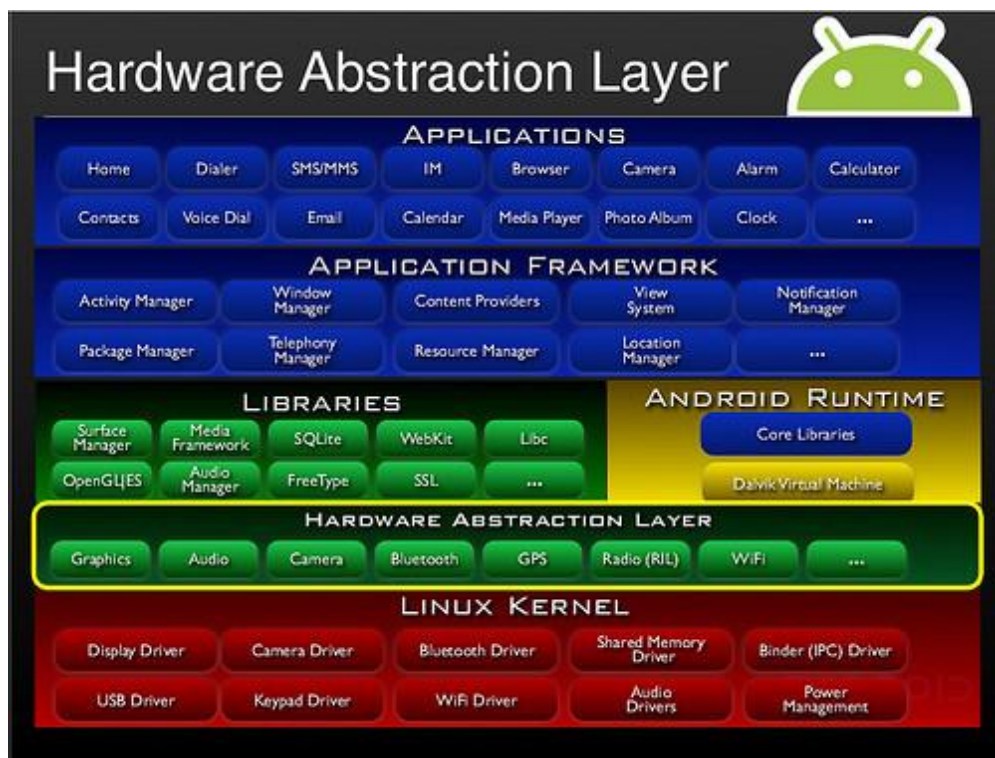


第 6 章 Android HAL 层解析

Android 的 HAL 层是为了一些硬件提供商提出的保护专利的驱动程序而产生的，简而言之，就是为了避开 Linux Kernel 的 GPL license 的束缚。Android 把控制硬件的动作都放到了用户空间中，而在 Kernel Driver 中只有最简单的读/写寄存器的操作，而完全去掉了各种功能性的操作（如控制逻辑等），这些能够体现硬件特性的操作都放到了 Android 的 HAL 层，而 Android 是基于 Apache 的 license，因此硬件厂商可以只提供二进制代码，所以说 Android 只是一个开放的平台，并不是一个开源的平台。

6.1 HAL 介绍

现有 HAL 架构是由 Patrick Brady (Google) 在 2008 年 Google I/O 演讲中提出的，如下图所示：



Android 的 HAL 是为了保护一些硬件提供商的知识产权而提出的，是为了避开 Linux 的 GPL 束缚。思路是把控制硬件的动作都放到了 Android HAL 中，而 Linux driver 仅完成一些简单的数据交互功能，甚至把硬件寄存器空间直接映射到用户空间。而 Android 是基于 Apache 的 license，因此硬件厂商可以只提供二进制代码，所以说 Android 只是一个开放的平台，并不是一个开源的平台。也许也正是因为 Android 不遵从 GPL，所以 Greg Kroah-Hartman 才在 2.6.33 内核中将 Android 驱动从 Linux 中删除。GPL 和硬件厂商目前还是有着无法弥合的裂痕。Android 想要把这个问题处理好也是不容易的。

总结下来，Android HAL 存在的原因主要有：

并不是所有的硬件设备都有标准的 Linux Kernel 接口。

Kernel Driver 涉及 GPL 的版权。某些设备制造商并不愿意公开硬件驱动，所以才去用 HAL 方式绕过



GPL。

针对某些硬件，Android 有一些特殊的需求。

6.2 HAL 内容

6.2.1 HAL 主要的存储目录

HAL 主要的存储目录有如下几个。

libhardware_legacy/：旧的架构、采取链接库模块的观念进行。

libhardware/：新架构、调整为 HAL stub 的观念。

ril/：Radio Interface Layer。

msm7k：QUAL 平台相关。

主要包含以下一些模块：Gps、Vibrator、Wifi、Copybit、Audio、Camera、Lights、Ril、Overlay 等。

6.2.2 两种 HAL 架构比较

目前存在两种 HAL 架构，分别是位于 libhardware_legacy 目录下的“旧 HAL 架构”和位于 libhardware 目录下的“新 HAL 架构”

两种框架如图 3.1 和图 3.2 所示。

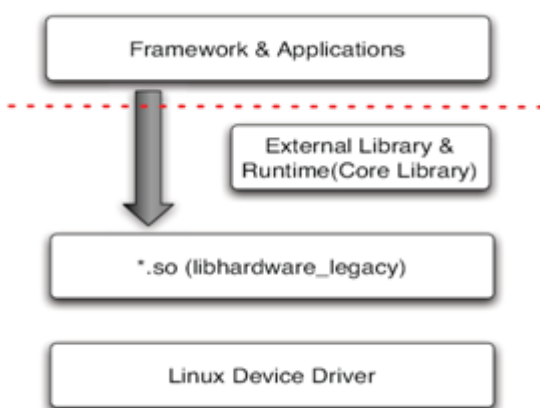


图 3.1

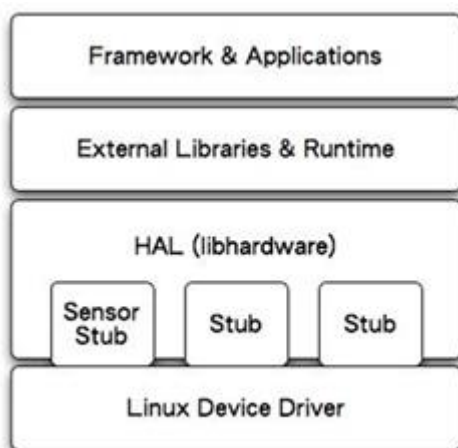


图 3.2

旧的 libhardware_legacy 架构是将 *.so 文件当做 shared library 来使用的，通过直接函数调用的方式，来操作驱动程序。当然，应用程序也可以不需要通过 JNI 的方式进行，直接加载 *.so（dlopen）的做法调用 *.so 中的符号（symbol）也是一种方式。

现在的 libhardware 架构，就有 stub 的味道了。HAL stub 是一种代理人（proxy）的概念，stub 虽然仍是以 *.so 的形式存在，但 HAL 已经将 *.so 隐藏起来了。Stub 向 HAL 提供操作函数（operations），而 runtime 则向 HAL 取得特定模块（stub）的 operations，再回调这些操作函数。这种以 indirect function call 的架构，让 HAL stub 变成是一种包含关系，即 HAL 中包含了许多 stub（代理人）。Runtime 只需要说明类型，即 module ID，就可以取得操作函数。对于目前的 HAL，可以认为 Android 定义了 HAL 层结构框架，通过几个接口访问硬件从而统一了调用方式。

6.3 mokoid 工程代码下载与结构分析

6.3.1 mokid 项目概述

modkoid 工程提供了一个 LedTest 示例程序，是中国台湾的 Jollen 用于培训的。对于理解 Android 层次结构、Hal 编程方法都非常有意义。

6.3.2 结构分析

结构分析如下：

```

|-- Android.mk
|-- apps          //两种应用测试方法
|   |-- Android.mk
|   |-- LedClient  //直接调用 service 来调用 jni
|   |   |-- AndroidManifest.xml
|   |   |-- Android.mk
|   |   `-- src

```



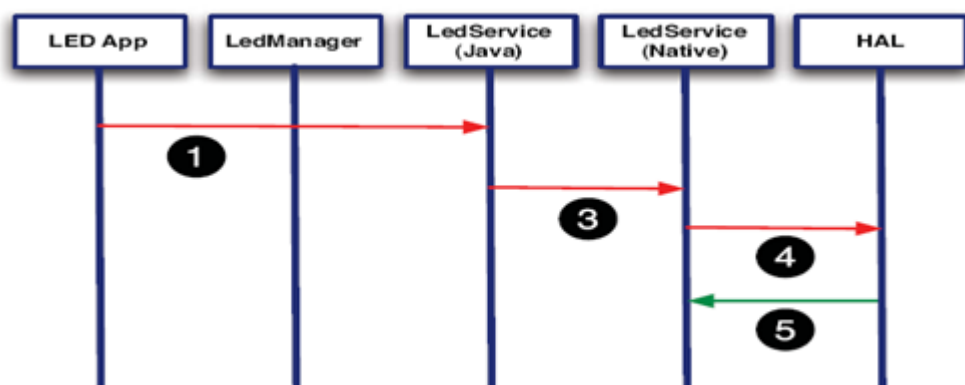
```
| |      `-- com
| |          `-- mokoid
| |              `-- LedClient
| |                  `-- LedClient.java           //第 1 种方式应用程序实现代码
| `-- LedTest           //通过 manager 来调用 jni
|     |-- AndroidManifest.xml
|     |-- Android.mk
|     `-- src
|         `-- com
|             `-- mokoid
|                 `-- LedTest
|                     |-- LedSystemService.java     //开启了一个后台 service，下文会有解释
|                     `-- LedTest.java             //第 2 种方式应用程序实现代码
|-- dma6410xp    //这个目录可以不要
|   |-- AndroidBoard.mk
|   |-- AndroidProducts.mk
|   |-- BoardConfig.mk
|   |-- dma6410xp.mk
|   |-- init.dma6410xp.rc
|   |-- init.goldfish.sh
|   `-- init.rc
|-- frameworks    //框架代码
|   |-- Android.mk
|   `-- base
|       |-- Android.mk
|       |-- core
|       |   `-- java
|       |       `-- mokoid
|       |           `-- hardware
|       |               |-- ILedService.aidl
|       |               `-- LedManager.java        //实现了 Manager，给第 2 种方法用
|       `-- service
|           |-- Android.mk
|           |-- com.mokoid.server.xml
|           `-- java
```



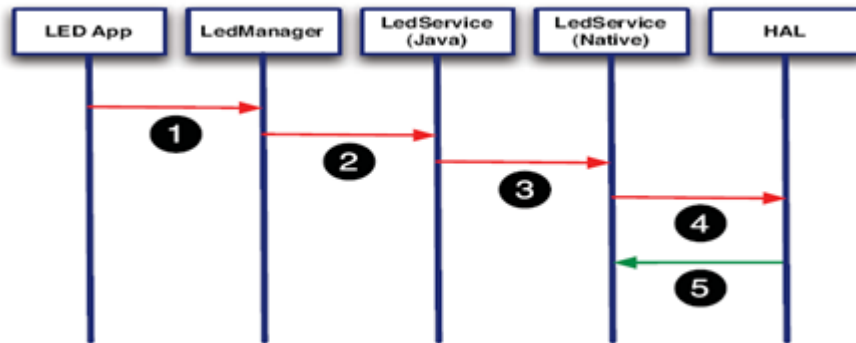
```
|
|   |-- com
|
|       |-- mokoid
|
|           |-- server
|
|               |-- LedService.java    //Framework service 代码
|
|   |-- jni
|
|       |-- Android.mk
|
|           |-- com_mokoid_server_LedService.cpp  //jni 代码
|-- hardware
|
|   |-- Android.mk
|
|   |-- libled
|
|       |-- Android.mk
|
|       |-- libled.c
|
|   |-- modules
|
|       |-- Android.mk
|
|       |-- include
|
|           |-- mokoid
|
|           |-- led.h
|
|       |-- led
|
|           |-- Android.mk
|
|               |-- led.c    //led stub 硬件控制代码
|-- README.txt
```

Android 的 HAL 实现需要通过 JNI (Java Native Interface)，JNI 是 Java 程序调用 C/C++写的动态链接库，这样，HAL 就可以使用 C/C++语言编写，效率更高。在 Android 下访问 HAL 大致有以下两种方式：

(1) Android 的 app 可以直接通过 Service 调用.so 格式的 JNI，如下图所示。



(2) 通过 Manager 调用 Service，如下图所示。



上述两种方法各有优缺点，第一种方法简单高效，但不正规。第二种方法实现起来比较复杂，但更符合目前的 Android 框架。第二种方法中，LedManager 和 LedService (java) 在两个进程中，需要通过进程通信的方式来通信。

mokoid 工程中实现了上述两种方法——下面将详细介绍这两种方法的实现原理。

6.3.3 第一种方法 直接调用 service 方法的实现过程

下面分析第一种方法中，各层的关键代码。

1. HAL 层

一般来说，HAL module 需要涉及的是如下 3 个关键结构体：

```

struct hw_module_t;
struct hw_module_methods_t;
struct hw_device_t;

```

下面结合代码说明这 3 个结构体的用法。部分代码经过修改，后面的章节会给出修改的原因。

文件：mokoid-read-only/hardware/modules/include/mokoid/led.h。

```

/*****
struct led_module_t {
    struct hw_module_t common;
};
//HAL 规定不能直接使用 hw_module_t 结构，因此需要做这么一个继承
struct led_control_device_t {
//自定义的一个针对 Led 控制的结构，包含 hw_device_t 和支持的 API 操作
    struct hw_device_t common;
    /* attributes */
    int fd; //可用于具体的设备描述符
    /* supporting control APIs go here */

```



```
int (*set_on)(struct led_control_device_t *dev, int32_t led);
int (*set_off)(struct led_control_device_t *dev, int32_t led);
};
#define LED_HARDWARE_MODULE_ID "led"
//定义一个 MODULE_ID，HAL 层可以根据这个 ID 找到 HAL stub
```

文件：mokoid-read-only/hardware/modules/led/led.c。

```
#define LOG_TAG "MokoidLedStub"
#include <hardware/hardware.h>
#include <fcntl.h>
#include <errno.h>
#include <cutils/log.h>
#include <cutils/atomic.h>
//#include <mokoid/led.h>
#include "../include/mokoid/led.h"

/*****

int fd;                //硬件 LED 的设备描述符，也可以用 led_control_device_t 结构中定义的 fd
#define GPG3DAT2_ON 0x4800                //ioctl 控制命令
#define GPG3DAT2_OFF 0x4801
int led_device_close(struct hw_device_t* device)
{
    struct led_control_device_t* ctx = (struct led_control_device_t*)device;
    if (ctx) {
        free(ctx);
    }
    close(fd);
    return 0;
}

int led_on(struct led_control_device_t *dev, int32_t led)
{
    LOGI("LED Stub: set %d on.", led);
    ioctl(fd,GPG3DAT2_ON,NULL);            //控制 LED 亮灭，和硬件相关
    return 0;
}

int led_off(struct led_control_device_t *dev, int32_t led)
{

```



```

        LOGI("LED Stub: set %d off.", led);
        return 0;
    }

static int led_device_open(const struct hw_module_t* module, const char* name,
                           struct hw_device_t** device)
{
    struct led_control_device_t *dev;
    dev = (struct led_control_device_t *)malloc(sizeof(*dev));
    memset(dev, 0, sizeof(*dev));
    dev->common.tag =    HARDWARE_DEVICE_TAG;
    dev->common.version = 0;
    dev->common.module = module;
    dev->common.close = led_device_close;
    dev->set_on = led_on;        //实例化支持的操作
    dev->set_off = led_off;
    *device = &dev->common;    //将实例化后的 led_control_device_t 地址返回给 JNI 层
                                //这样 JNI 层就可以直接调用 led_on、led_off、led_device_close 方法了
    if((fd=open("/dev/led",O_RDWR))!=-1)    //打开硬件设备
    {
        LOGE("LED open error");
    }
    else
        LOGI("open ok");
success:
    return 0;
}

static struct hw_module_methods_t led_module_methods = {
    open: led_device_open
};

const struct led_module_t HAL_MODULE_INFO_SYM = {
    //定义这个对象等于向系统注册了一个 ID 为 LED_HARDWARE_MODULE_ID 的 stub。注意，这里
    //HAL_MODULE_INFO_SYM 的名称不能改
    common: {
        tag: HARDWARE_MODULE_TAG,
        version_major: 1,
    }
}
    
```




```
version_minor: 0,
id: LED_HARDWARE_MODULE_ID,
name: "Sample LED Stub",
author: "The Mokoid Open Source Project",
methods: &led_module_methods, //实现了一个 open 的方法供 jni 层调用,
                                   //从而实例化 led_control_device_t
}
/* supporting APIs go here */
};
```

2. JNI 层

文件：mokoid-read-only/frameworks/base/service/jni/com_mokoid_server_LedService.cpp。

```
struct led_control_device_t *sLedDevice = NULL;

static jboolean mokoid_setOn(JNIEnv* env, jobject thiz, jint led)
{
    LOGI("LedService JNI: mokoid_setOn() is invoked.");

    if (sLedDevice == NULL) {
        LOGI("LedService JNI: sLedDevice was not fetched correctly.");
        return -1;
    } else {
        return sLedDevice->set_on(sLedDevice, led); //调用 HAL 层的注册的方法
    }
}

static jboolean mokoid_setOff(JNIEnv* env, jobject thiz, jint led)
{
    LOGI("LedService JNI: mokoid_setOff() is invoked.");

    if (sLedDevice == NULL) {
        LOGI("LedService JNI: sLedDevice was not fetched correctly.");
        return -1;
    }
}
```



```

    } else {
        return sLedDevice->set_off(sLedDevice, led); //调用 HAL 层的注册的方法
    }
}

/** helper APIs */
static inline int led_control_open(const struct hw_module_t* module,
    struct led_control_device_t** device) {
    return module->methods->open(module,
        LED_HARDWARE_MODULE_ID, (struct hw_device_t**)device);
//这个过程非常重要, JNI 通过 LED_HARDWARE_MODULE_ID 找到对应的 stub
}

static jboolean mokoid_init(JNIEnv *env, jclass clazz)
{
    led_module_t* module;
    LOGI("jni init-----.");
    if (hw_get_module(LED_HARDWARE_MODULE_ID, (const hw_module_t**)&module) == 0) {
//根据 LED_HARDWARE_MODULE_ID 找到 hw_module_t, 参考 HAL 层的实现
        LOGI("LedService JNI: LED Stub found.");
        if (led_control_open(&module->common, &sLedDevice) == 0) {
//通过 hw_module_t 找到 led_control_device_t
            LOGI("LedService JNI: Got Stub operations.");
            return 0;
        }
    }

    LOGE("LedService JNI: Get Stub operations failed.");
    return -1;
}

/*
 * Array of methods.
 * Each entry has three fields: the name of the method, the method
 * signature, and a pointer to the native implementation.

```



```

*/

static const JNINativeMethod gMethods[] = {
    { "_init",      "()Z",   (void *)mokoid_init },//Framework 层调用_init 时触发
    { "_set_on",    "(I)Z",  (void *)mokoid_setOn },
    { "_set_off",   "(I)Z",  (void *)mokoid_setOff },
};

/*
*JNINativeMethod 是 jni 层注册的方法，Framework 层可以使用这些方法
*_init 、 _set_on、 _set_off 是在 Framework 中调用的方法名称，函数的类型及返回值如下：
* ( ) Z   无参数    返回值为 bool 型
* (I)Z   整型参数  返回值为 bool 型
*/

static int registerMethods(JNIEnv* env) {
    static const char* const kClassName =
        "com/mokoid/server/LedService";//注意：必须和 Framework 层的 service 类名相同
    jclass clazz;
    /* look up the class */
    clazz = env->FindClass(kClassName);
    if (clazz == NULL) {
        LOGE("Can't find class %s\n", kClassName);
        return -1;
    }
    /* register all the methods */
    if (env->RegisterNatives(clazz, gMethods,
        sizeof(gMethods) / sizeof(gMethods[0])) != JNI_OK)
    {
        LOGE("Failed registering methods for %s\n", kClassName);
        return -1;
    }
    /* fill out the rest of the ID cache */
    return 0;
}

jint JNI_OnLoad(JavaVM* vm, void* reserved) { //Framework 层加载 jni 库时调用
    JNIEnv* env = NULL;
    jint result = -1;

```



```
LOGI("JNI_OnLoad LED");
    if (vm->GetEnv((void**) &env, JNI_VERSION_1_4) != JNI_OK) {
        LOGE("ERROR: GetEnv failed\n");
        goto fail;
    }

    assert(env != NULL);

    if (registerMethods(env) != 0) { //注册 JNINativeMethod
        LOGE("ERROR: PlatformLibrary native registration failed\n");
        goto fail;
    }

    /* success -- return valid version number */
    result = JNI_VERSION_1_4;

fail:
    return result;
}
```

3. service（属于 Framework 层）

文件：frameworks/base/service/java/com/mokoid/server/LedService.java。

```
package com.mokoid.server;
import android.util.Config;
import android.util.Log;
import android.content.Context;
import android.os.Binder;
import android.os.Bundle;
import android.os.RemoteException;
import android.os.IBinder;
import mokoid.hardware.ILedService;
public final class LedService extends ILedService.Stub {
```

//对于这种直接模式不需要进程通信，所以可以不加 extends ILedService.Stub，此处加上主要是为了后面的第二

//种模式

```
    static {
        System.load("/system/lib/libmokoid_runtime.so");//加载 JNI 的动态库
    }
```



```
public LedService() {
    Log.i("LedService", "Go to get LED Stub...");
    _init();
}
/*
 * Mokoid LED native methods.
 */
public boolean setOn(int led) {
    Log.i("MokoidPlatform", "LED On");
    return _set_on(led);
}
public boolean setOff(int led) {
    Log.i("MokoidPlatform", "LED Off");
    return _set_off(led);
}
private static native boolean _init();           //声明 JNI 库可以提供的方法
private static native boolean _set_on(int led);
private static native boolean _set_off(int led);
}
```

4. APP 测试程序 （属于 APP 层）

文件：apps/LedClient/src/com/mokoid/LedClient/LedClient.java。

```
package com.mokoid.LedClient;
import com.mokoid.server.LedService; // 导入 Framework 层的 LedService
import android.app.Activity;
import android.os.Bundle;
import android.widget.TextView;

public class LedClient extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        // Call an API on the library.
        LedService ls = new LedService(); //实例化 LedService
    }
}
```



```
ls.setOn(1); //通过 LedService 提供的方法，控制底层硬件
ls.setOff(2);

    TextView tv = new TextView(this);
    tv.setText("LED 1 is on. LED 2 is off.");
    setContentView(tv);
}
}
```

6.3.4 第二种方法——经过 Manager 调用 service

HAL、JNI 两层和第一种方法一样，所以后面只分析其他的层次。

1. Manager （属于 Framework 层）

APP 通过这个 Manager 和 service 通信。

文件：mokoid-read-only/frameworks/base/core/java/mokoid/hardware/LedManager.java。

```
package mokoid.hardware;
import android.content.Context;
import android.os.Binder;
import android.os.Bundle;
import android.os.Parcelable;
import android.os.ParcelFileDescriptor;
import android.os.Process;
import android.os.RemoteException;
import android.os.Handler;
import android.os.Message;
import android.os.ServiceManager;
import android.util.Log;
import mokoid.hardware.ILedService;

/*
 * Class that lets you access the Mokoid LedService.
 */
public class LedManager
{
```



```
private static final String TAG = "LedManager";  
private ILedService mLedService;  
public LedManager() {  
    mLedService = ILedService.Stub.asInterface(ServiceManager.getService("led"));
```

```
/*
```

*这一步是关键，利用 ServiceManager 获取 LedService，从而调用它提供的方法。这要求 LedService 必

*须已经添加到了 ServiceManager 中，这个过程将在 App 中的一个 service 进程中完成

```
*/
```

```
    if (mLedService != null) {  
        Log.i(TAG, "The LedManager object is ready.");  
    }  
}  
  
public boolean LedOn(int n) {  
    boolean result = false;  
    try {  
        result = mLedService.setOn(n);  
    } catch (RemoteException e) {  
        Log.e(TAG, "RemoteException in LedManager.LedOn:", e);  
    }  
    return result;  
}  
  
public boolean LedOff(int n) {  
    boolean result = false;  
    try {  
        result = mLedService.setOff(n);  
    } catch (RemoteException e) {  
        Log.e(TAG, "RemoteException in LedManager.LedOff:", e);  
    }  
    return result;  
}  
}
```

因为 LedService 和 LedManager 在不同的进程中，所以要考虑到进程通信的问题。Manager 通过增加一个 aidl 文件来描述通信接口

文件：mokoid-read-only/frameworks/base/core/java/mokoid/hardware/ILedService.aidl



```
package mokoid.hardware;

interface ILedService
{
    boolean setOn(int led);
    boolean setOff(int led);
}
```

2. SystemServer（属于 APP 层）

文件：mokoid-read-only/apps/LedTest/src/com/mokoid/LedTest/LedSystemServer.java。

```
package com.mokoid.LedTest;
import com.mokoid.server.LedService;
import android.os.IBinder;
import android.os.ServiceManager;
import android.util.Log;
import android.app.Service;
import android.content.Context;
import android.content.Intent;

public class LedSystemServer extends Service {
    //这里的 Service 是 APP 中的概念，代表一个后台进程。注意，区别和 Framework 中的 service 的概念
    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
    public void onStart(Intent intent, int startId) {
        Log.i("LedSystemServer", "Start LedService...");

        /* Please also see SystemServer.java for your interests. */
        LedService ls = new LedService();
        try {
            ServiceManager.addService("led", ls); //将 LedService 添加到 ServiceManager 中
        } catch (RuntimeException e) {
            Log.e("LedSystemServer", "Start LedService failed.");
        }
    }
}
```




}

3. APP 测试程序（属于 APP 层）

文件：mokoid-read-only/apps/LedTest/src/com/mokoid/LedTest/LedTest.java。

```
package com.mokoid.LedTest;

import mokoid.hardware.LedManager;
import com.mokoid.server.LedService;
import android.app.Activity;
import android.os.Bundle;
import android.util.Log;
import android.widget.TextView;
import android.widget.Button;
import android.content.Intent;
import android.view.View;

public class LedTest extends Activity implements View.OnClickListener {
    private LedManager mLedManager = null;

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        // Start LedService in a seperated process.
        startService(new Intent("com.mokoid.systemserver")); //开启后台进程
        Button btn = new Button(this);
        btn.setText("Click to turn LED 1 On");
        btn.setOnClickListener(this);
        setContentView(btn);
    }

    public void onClick(View v) {
        // Get LedManager.
        if (mLedManager == null) {
            Log.i("LedTest", "Creat a new LedManager object.");
            mLedManager = new LedManager(); //实例化 Framework 层中的 Manager
        }
        if (mLedManager != null) {
```



```
        Log.i("LedTest", "Got LedManager object.");
    }

    /** Call methods in LedService via proxy object
     * which is provided by LedManager.
     */

    mLedManager.LedOn(1);
    TextView tv = new TextView(this);
    tv.setText("LED 1 is On.");
    setContentView(tv);
}
}
```

6.4 小结

本章从概述 HAL 开始，介绍了 HAL 的内容，分析了 HAL 主要的存储目录，比较了 HAL 的两种架构；解析了 Android HAL 实例：mokid 工程，介绍了工程代码的下载方法和结构分析。引出了在 Android 下访问 HAL 的两种方式。



第 7 章 Android 驱动介绍

7.1 Android 移植和驱动的关系

假设我们出产基于 Android 的代号为“大米”的手机的摄像头是 800 万像素，伴随着硬件的发展，我们打算升级这款手机为“老米”手机，它的摄像头升级为 1000 万像素，那么这时原先“大米”手机的 Android 镜像中的摄像头驱动就要进行修改，然后重新进行编译和烧写，所以在进行 Android 移植中的难点在于驱动程序的编写和修改。

7.2 设备驱动程序

7.2.1 设备驱动概念

任何一个计算机系统的运行都是系统中软硬件协作的结果，没有硬件的软件是空中楼阁，而没有软件的硬件则只是一堆废铁。硬件是底层基础，是所有软件得以运行的平台，代码最终会落实为硬件上的组合逻辑与时序逻辑；软件则实现了具体应用，它按照各种不同的业务需求而设计，满足了用户的需求。硬件较固定，软件则很灵活，可以适应各种复杂多变的应用。可以说，计算机系统的软硬件互相成就了对方。但是，软硬件之间同样存在着悖论，那就是软件和硬件不应该互相渗透到对方的领地。为了尽可能快速地完成设计，应用软件工程师不想也不必关心硬件，而硬件工程师也难有足够的闲暇和能力来顾及软件。

例如，应用软件工程师在调用套接字发送和接收数据包时，不必关心网卡上的中断、寄存器、存储空间、I/O 端口、片选，以及其他任何硬件词汇；在使用 `printf()` 函数输出信息时，不用知道底层究竟是怎样把相应的信息输出到屏幕或串口的。也就是说，应用软件工程师需要看到一个没有硬件的纯粹的软件世界，硬件必须被透明地呈现给他们。谁来实现硬件对应用软件工程师的隐形？这个艰巨的任务就落在了驱动工程师的身上。对设备驱动最通俗的解释就是“驱使硬件设备行动”。设备驱动与底层硬件直接打交道，按照硬件设备的具体工作方式读/写设备寄存器，完成设备的轮询、中断处理、DMA 通信，进行物理内存向虚拟内存的映射，最终使通信设备能够收发数据，使显示设备能够显示文字和画面，使存储设备能够记录文件和数据。由此可见，设备驱动充当了硬件和应用软件之间的纽带，它使得应用软件只需要调用系统软件的应用编程接口（API）就可让硬件去完成要求的工作。在系统中没有操作系统的情况下，工程师可以根据硬件设备的特点自行定义接口，如对串口定义 `SerialSend()`、`SerialRecv()`；对 LED 定义 `LightOn()`、`LightOff()`；以及对 Flash 定义 `FlashWrite()`、`FlashRead()` 等。而在有操作系统的情况下，设备驱动的架构则由相应的操作系统定义，驱动工程师必须按照相应的架构设计设备驱动，这样，设备驱动才能良好地整合到操作系统的内核中。驱动程序沟通着硬件和应用软件，而驱动工程师则沟通着硬件工程师和应用软件工程师。

随着通信、电子行业的迅速发展，全世界每天都会有大量的新芯片被生产，大量的新电路板被设计，因此，也会有大量设备驱动需要开发。这些设备驱动，或运行在简单的单任务环境中，或运行在 VxWorks、Linux、Windows 等多任务操作系统环境中，发挥着不可替代的作用。



7.2.2 内核驱动程序

内核是操作系统最基本的部分。它是为众多应用程序提供对计算机硬件的安全访问的一部分软件，这种访问是有限的，并且内核决定一个程序在什么时候对某部分硬件操作多长时间。直接对硬件操作是非常复杂的，所以内核通常提供一种硬件抽象的方法来完成这些操作。硬件抽象隐藏了复杂性，为应用软件和硬件提供了一套简洁、统一的接口，使程序设计更为简单。这就是内核空间实现的驱动程序，它使用内核资源、内核栈，包括可加载的内核驱动模块。

7.2.3 用户空间驱动程序

在用户空间实现驱动程序是把驱动程序的一部分放在用户空间，但不能将驱动程序的任意部分放在用户空间内。而且并非所有设备驱动都在用户空间内实现，因为对于大部分设备驱动程序是在内核空间实现的。只有设备种类众多、驱动程序较复杂、能够分层时，为了使用方便，其中上层或增加的驱动程序部分可在用户空间实现。

7.2.4 Linux 设备驱动概述

1. Linux 设备驱动

操作系统是通过各种驱动程序来驾驭硬件设备的，它为用户屏蔽了各种各样的设备，驱动硬件是操作系统最基本的功能，并且提供统一的操作方式。设备驱动程序是操作系统最基本的组成部分之一，在 Linux 内核源程序中也占有 60% 以上。因此，熟悉驱动的编写是很重要的。

Linux 的一个重要特点就是将所有的设备都当做文件进行处理，这类特殊文件就是设备文件（通常在 /dev 目录下），这样，在应用程序看来，硬件设备只是一个设备文件，应用程序可以像操作普通文件一样对硬件设备进行操作，从而大大方便了对设备的处理。

Linux 系统的设备分为 3 类：字符设备、块设备和网络设备。

字符设备通常指像普通文件或字节流一样，以字节为单位顺序读/写的设备，如串口设备、虚拟控制台等。字符设备可以通过设备文件节点访问，它与普通文件之间的区别在于，普通文件可以被随机访问（可以前后移动访问指针），而大多数字符设备只能提供顺序访问，因为对它们的访问不会被系统所缓存。但也有例外，例如，帧缓存（framebuffer）是一个可以被随机访问的字符设备。

块设备通常指一些需要以块为单位随机读/写的设备，如 IDE 硬盘、SCSI 硬盘、光驱等。它不仅可以提供随机访问，而且可以容纳文件系统（如硬盘、闪存等）。Linux 可以使用户态程序像访问字符设备一样每次进行任意字节的操作，只是在内核态内部中的管理方式和内核提供的驱动接口上不同。

通过文件属性可以查看它们是哪种设备文件（字符设备文件或块设备文件）。

```
$ ls -l /dev
```

```
crw-rw---- 1 root uucp 4, 64 08-30 22:58 ttyS0 /*串口设备， c 表示字符设备*/
```

```
brw-r----- 1 root floppy 2, 0 08-30 22:58 fd0 /*软盘设备， b 表示块设备*/
```

网络设备通常是指通过网络能够与其他主机进行数据通信的设备，如网卡等。



内核和网络设备驱动程序之间的通信调用一套数据包处理函数，它们完全不同于内核和字符，以及块设备驱动程序之间的通信（`read()`、`write()`等函数）。Linux 网络设备不是面向流的设备，因此不会将网络设备的名称（如 `eth0`）映射到文件系统中去。

2. Linux 设备驱动程序的特点

Linux 中的设备驱动程序有如下几个特点。

- (1) 内核代码：设备驱动程序是内核的一部分，如果驱动程序出错，则可能导致系统崩溃。
- (2) 内核接口：设备驱动程序必须为内核或者其子系统提供一个标准接口。比如，一个终端驱动程序必须为内核提供一个文件 I/O 接口；一个 SCSI 设备驱动程序应该为 SCSI 子系统提供一个 SCSI 设备接口，同时 SCSI 子系统也必须为内核提供文件的 I/O 接口及缓冲区。
- (3) 内核机制和服务：设备驱动程序使用一些标准的内核服务，如内存分配等。
- (4) 可装载：大多数的 Linux 操作系统设备驱动程序都可以在需要时装载进内核，在不需要时从内核中卸载。
- (5) 可设置：Linux 操作系统设备驱动程序可以集成为内核的一部分，并可以根据需要把其中的某一部分集成到内核中，这只需要在系统编译时进行相应的设置。

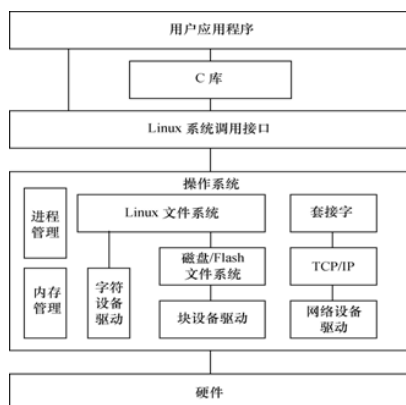
动态性：在系统启动且各个设备驱动程序初始化后，驱动程序将维护其控制的设备。如果该设备驱动程序控制的设备不存在也不影响系统的运行，那么此时的设备驱动程序只是多占用了一点系统内存。

3. Linux 设备驱动程序与整个软硬件系统的关系

除网络设备外，字符设备与块设备都被映射到 Linux 文件系统的文件和目录，通过文件系统的系统调用接口 `open()`、`write()`、`read()`、`close()` 等函数即可访问字符设备和块设备。所有的字符设备和块设备都被统一地呈现给用户。块设备比字符设备复杂，在它上面会首先建立一个磁盘/Flash 文件系统，如 FAT、Ext3、YAFFS、JFFS2 等。它们规范了文件和目录在存储介质上的组织。

应用程序可以使用 Linux 系统调用接口编程，也可以使用 C 库函数，出于代码可移植性的考虑，后者更值得推荐。C 库函数本身也通过系统调用接口而实现的，如 C 库函数中的 `fopen()`、`fwrite()`、`fread()`、`fclose()` 分别会调用操作系统 API 的 `open()`、`write()`、`read()`、`close()` 函数。

Linux 设备驱动程序与整个软硬件系统的关系如下图所示：



7.3 Linux 核心与 Android 驱动

Android 中内核的结构和标准的 Linux 2.6 内核是基本相同的，Android 在其基础上增加了私有内容。Android 在 Linux 内核中增加的主要是一些驱动程序，这些驱动程序主要分为两种：Android 专用驱动和 Android 使用的设备驱动。

7.3.1 Android 专用驱动

- (1) Android 专用驱动程序不是 Linux 的标准驱动，它们的作用是辅助系统运行，一般不操作实际硬件。
- (2) Ashmem：匿名共享内存驱动。
- (3) Logger：轻量级的 log 驱动。
- (4) Binder 驱动（Binder Driver）：基于 OpenBinder 驱动，为 Android 平台提供 IPC 的支持。
- (5) 能源管理（Android Power Management）：轻量级的能源管理，基于 Linux 的能源管理，为嵌入式系统做了优化。
- (6) Android Power Management：定时器驱动，用于唤醒设备。
- (7) Low Memory Killer：在缺少内存的情况下，杀死进程。
- (8) Android PMEM：物理内存驱动。

下面重点介绍 Android 专用驱动中最重要的 3 个驱动。

1. Ashmem 匿名共享内存驱动

Ashmem（Anonymous Shared Memory），即匿名共享内存，通过内核的机制，为用户空间程序提供分配内存的机制。

Ashmem 设备节点名称为/dev/ashmem，主设备号为 10（Misc Driver），次设备号动态生成。

Ashmem 的代码路径如下：

```
kernel/include/linux/ashmem.h
kernel/mm/ashmem.c
```



2. Logger：轻量级的 log 驱动

Android 的 Logger 驱动程序为用户层程序提供 log 的支持，这个驱动作为一个工具来使用。

Logger 有 3 个设备节点，分别为/dev/log/main、/dev/log/event、/dev/log/radio。

主设备号为 10（Misc Driver），次设备号动态生成。

Logger 驱动的代码路径如下：

```
kernel/include/linux/logger.h
kernel/drivers/misc/logger.c
```

在用户空间 logcat 程序调用 Logger 驱动：

```
system/core/logcat/可执行程序。
```

3. Binder 驱动（Binder Driver）：

Android 的 Binder 驱动程序为用户层程序提供 IPC（进程间通信）支持，Android 整个系统的运行依赖 Binder 驱动。

Binder 设备节点名称为/dev/binder，主设备号为 10（Misc Driver），次设备号动态生成。

Binder 的代码路径如下：

```
kernel/include/linux/binder.h
kernel/drivers/misc/binder.c
```

Android 系统上层用户空间的 libutil 工具库和 ServiceManager 守护进程都是调用 Binder 接口驱动提供对整个系统进程间通信功能的支持，它们的代码路径如下：

```
frameworks/base/cmds/servicemanager/
frameworks/base/include/utils/
frameworks/base/libs/utils/
```

Binder 是 Android 中主要使用的 IPC 方式，通常需要按照模板定义相关的类，不需要直接调用 Binder 驱动程序的设备节点。

7.3.2 Android 使用的设备驱动

Android 中常使用的设备主要有 Framebuffer 驱动、输入设备驱动、v4l2 摄像头—视频驱动、OSS 音频驱动、ALSA 音频驱动、MTD 驱动、蓝牙驱动、Wlan 驱动。

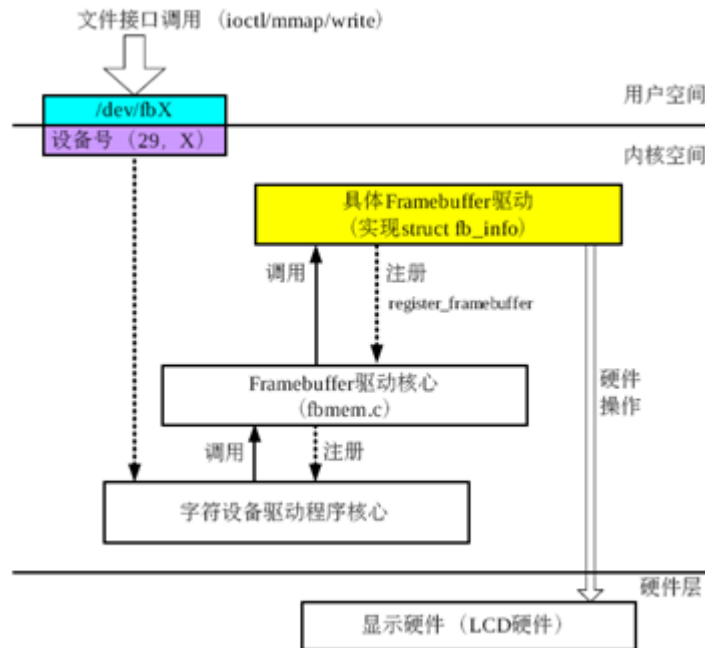
1. Framebuffer 显示驱动

framebuffer 驱动的设备节点：/dev/fb0，/dev/graphics/fb0，主设备号为 29，次设备号递增生成。

代码路径：include/linux/fb.h，drivers/video/fbmem.c。



Framebuffer 显示驱动工作原理如下图所示。



2. 输入设备驱动

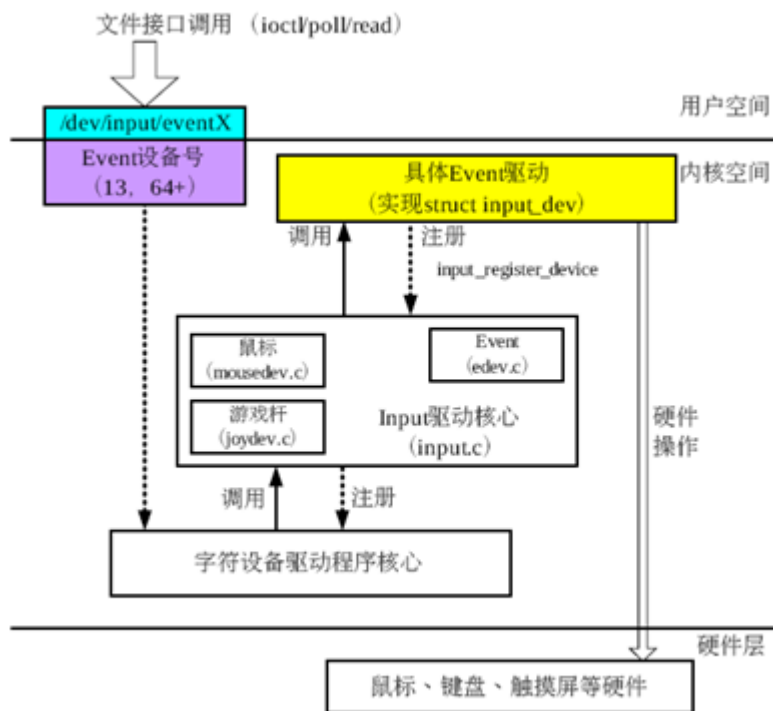
Android 中的输入设备驱动主要包括：游戏杆 (Joystick)、鼠标 (Mouse) 和事件设备 (Event)。

输入设备驱动同样也是字符设备，主设备号是 13，次设备号是 64~95 之间自动生成的，这个输入设备驱动程序那是相当相当的复杂。在 Android 内核中主要需要关注以下几个文件：

- `include/linux/input.h` (驱动头文件)。
- `driver/input/input.c` (驱动核心实现，包含大量的操作接口)。
- `driver/input/event.c` (event 机制)。
- `driver/input/joydev.c` (joystick 驱动)。
- `driver/input/mousedev.c` (鼠标驱动)。

其实上面这些东西都不需我们去实现，内核已经帮我们实现好了，不过在写硬件驱动时需要和 Inputcore 交互，所以需要用到上面这些函数中的接口，也就是说上面这些函数是透明的。

Event 输入设备驱动工作原理如下图所示。



3. v4l2 摄像头—视频驱动

摄像头 (Camera) —视频驱动通常使用 Video For Linux。

v4l2 驱动的设备节点: `/dev/video/videoX`, 主设备号为 81, 次设备号 0-63。

v4l2 驱动主要头文件路径:

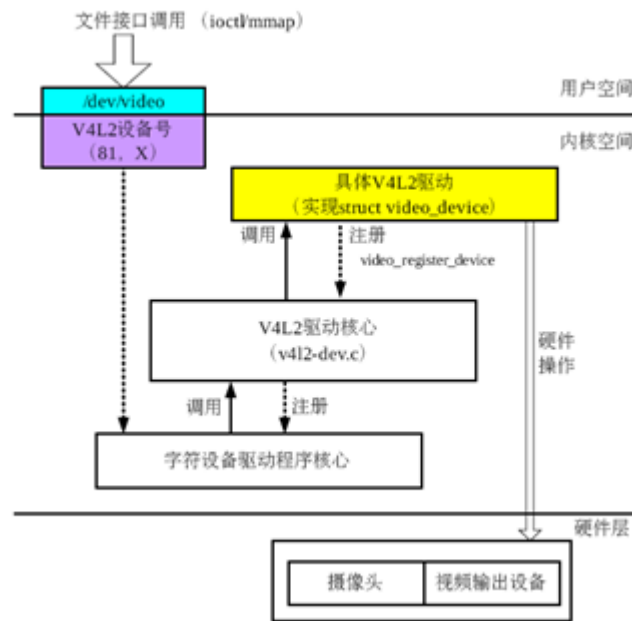
`include/linux/videodev.h`: v4l 第一版的头文件。

`include/linux/videodev2.h`: 定义主要的数据接口和常量。

`include/media/v4l2-dev.h`: 设备头文件, 具体设备使用其中的接口注册。

v4l2 驱动核心实现路径: `driver/media/video/v4l2-dev.c`。

v4l2 摄像头-视频驱动工作原理如下图所示。



4. OSS 音频驱动

OSS (Open Sound System) 开放声音系统。

OSS 驱动的设备节点: `/dev/mixer`、`/dev/sndstat`、`/dev/dsp`, OSS 主设备号为 14, 次设备号为各个设备。

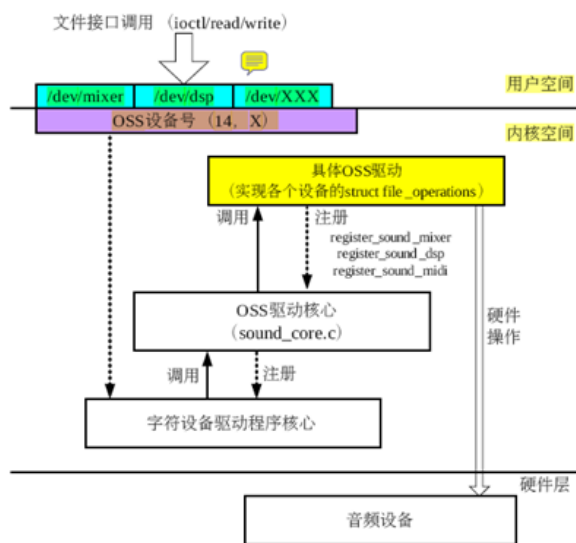
OSS 驱动程序的主要头文件:

`include/linux/soundcard.h`: OSS 驱动的主要头文件。

`include/linux/sound.h`: 定义 OSS 驱动的次设备号和注册函数。

OSS 驱动程序的核心: `sound/sound_core.c`。

OSS 音频驱动工作原理如下图所示。



5. ALSA 音频驱动

ALSA (Advanced Linux Sound Architecture) 即高级 Linux 声音体系。

ALSA 驱动的设备节点: `/dev/snd/controlCX`、`/dev/snd/pcmXXXc`、`/dev/snd/pcmXXXp`、`/dev/snd/seq`、`/dev/snd/timer`，主设备号为 116，次设备号为各个设备。

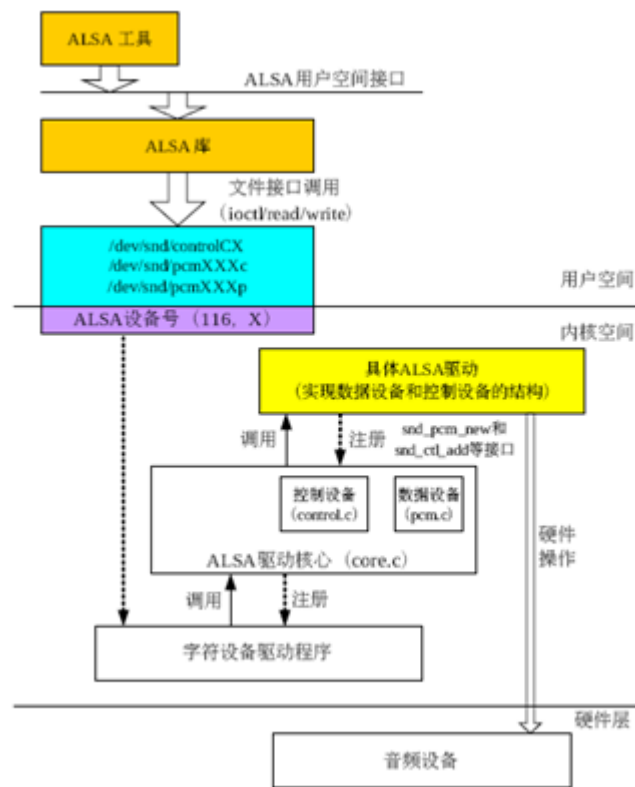
ALSA 驱动程序的头文件:

`include/sound/asound.h`: ALSA 驱动的主要头文件。

`include/sound/core.h`: ALSA 驱动核心数据结构和具体驱动的注册函数。

ALSA 驱动程序的核心实现: `sound/core/sound.c`。

ALSA 音频驱动工作原理如下图所示。



6. MTD 驱动

Flash 驱动通常使用 MTD (Memory Technology Device, 内存技术设备)。

MTD 的字符设备: `/dev/mtdX`, 主设备号为 90。

MTD 的块设备: `/dev/block/mtdblockX`, 主设备号为 13。

MTD 驱动程序头文件路径: `include/linux/mtd/mtd.h`。

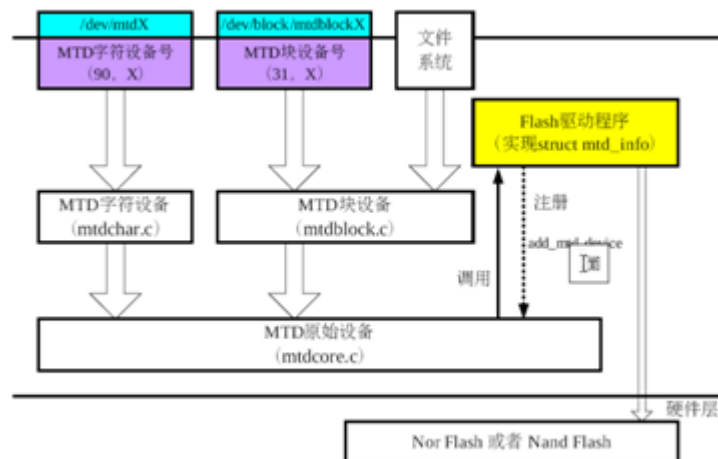
MTD 源代码路径:

`drivers/mtd/mtdcore.c`: MTD 核心, 定义 MTD 原始设备。

`drivers/mtd/mtdchar.c`: MTD 字符设备。

`drivers/mtd/mtdblock.c`: MTD 块设备。

MTD 驱动工作原理如下图所示。



7. 蓝牙驱动

在 Linux 中，蓝牙设备驱动是网络设备，使用网络接口。

蓝牙设备的网络协议：协议族 AF_BLUETOOTH (31)。

蓝牙协议部分头文件：

include/net/bluetooth/hci_core.h。

include/net/bluetooth/bluetooth.h。

蓝牙协议源代码文件：net/bluetooth/*。

蓝牙驱动程序部分的文件：drivers/bluetooth/*。

8. WLAN 驱动

在 Linux 中，WLAN 设备驱动是网络设备，使用网络接口。WLAN 在用户空间使用标准的 Socket 接口进行控制。

WiFi 协议部分头文件：include/net/wireless.h。

WiFi 协议部分源文件：net/wireless/*。

WiFi 驱动程序部分：drivers/net/wireless/*。

7.4 小结

本章讲述了 Android 驱动开发的基本概念、设备驱动的概念、内核驱动程序和用户空间驱动个程序。介绍了什么是嵌入式系统，概述了 Android 平台的概念和特点，以及嵌入式 Android 的系统架构。重点介绍 Linux 核心与 Android 驱动的知识。通过本章的学习，可初步了解驱动程序的概念和 Android 驱动的开发原理。另外，还讲述了 Android 设备驱动的开发环境的搭建，并创建简单的开发示例。



第 8 章 Android LED 点灯实验

8.1 实验目的

通过实验熟悉 android app 层、jni 层、hal 层之间的调用关系。

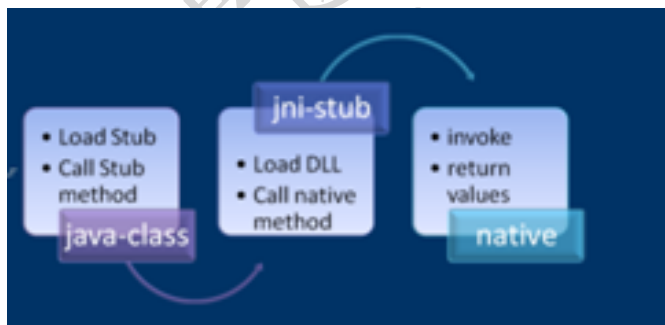
8.2 实验平台

FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

8.3 实验原理

android 实现了很好的分层机制，从而使得开发者开发更加专注，但是 android 的应用层和框架层使用的开发语言为 java，Java 是一种平台无关性的语言，平台对于上层的 java 代码来说是透明的，而 android 是基于 linux 的一个操作系统，linux 只提供了 c/c++或者是汇编的接口给开发，因此，为了使得 android 系统运行正常，必须想办法把 java 语言与 c/c++连接起来，这个连接螺丝就是 jni 技术，Android HAL（硬件抽象层）是一个 Linux 底层驱动与 Java ApplicationFramework（应用层框架）的中间层，它可以将硬件的操作逻辑封装起来，实现 Java 类的本地接口，交由 Android 应用框架统一管理：

JNI 的调用结构如图：



下为 JNI 的使用步骤：



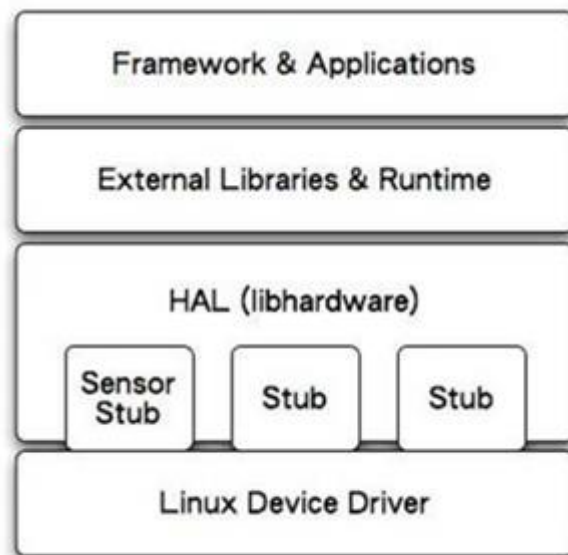
由上述介绍可知，Java 程序与 JNI 编译成的共享库之间联系如下：

- (1) Java 类加载共享库用 loadlibrary 方法，名称与 JNI 共享库名称对应；
- (2) Java 调用 JNI 中的类需要声明本地方法，其中本地方法的类型需要与 JNI 中 JNINativeMethod 中提供的方法声明、signature 一致。
- (3) JNI 类中需要在 jni helper 函数声明调用 Java 类，这样才能实现 Java 虚拟机与本地方法的联系。
- (4) Java 服务类的全称需与 JNI 的文件名有一定的对应关系，如 Java 服务类名为 com.farsight.Service.LedService，对应 JNI 的 service 应用名为 com_farsight_service_LedService.cpp。

HAL Stub 是一种代理人（Proxy）的概念，Stub 虽然仍是以 *.so 的形式存在的，但 HAL 已经将*.so



隐藏起来了.Stub 作为直接调用 Linux 驱动的上层,实现了简单的 I/O 操作,Stub 向 HAL 提供操作函数 (Operations),而 Runtime 则向 HAL 取得特定模块 (Stub) 的 Operations,再回调这些操作函数。这种以 indirect function call 的架构,让 HAL Stub 变成一种包含关系,即 HAL 中包含了许多 Stub (代理人)。Runtime 只要说明类型,即 module ID,就可以取得操作函数。对于目前的 HAL,可以认为 Android 定义了 HAL 层结构框架,通过几个接口访问硬件从而统一了调用方式。



8.4 实验步骤

本例实验代码在:【华清远见-嵌入式 ARM 实验箱资料-I-FS6818: 程序源码\Android 底层实验源码\Android LED 点灯实验\实验源码】目录下

led应用	2017/8/18 10:22	文件夹	
Farsight_Test.apk	2015/7/21 16:57	APK 文件	52 KB
led.slsiap.so	2017/8/25 18:30	SO 文件	6 KB
led.tar.xz	2016/11/16 12:52	WinRAR 压缩文件	3 KB
libled_runtime.so	2017/8/25 18:34	SO 文件	6 KB

led 应用: led 应用程序源码

Farsight_Test.apk: led 灯测试程序

led.tar.xz: android 底层代码

两个 so 文件分别为 hel 层和 jni 层编译生成的共享库

8.4.1 底层代码编译

打开虚拟机 ubuntu 系统,进入终端 android 源码目录:

执行:

```
$ cd /home/linux/workdir/Fs6818
```



因 Android 源码中 hardware/farsight/led 目录下在出厂时已有这部分代码，就不再进行拷贝工作，该源码在编译安卓源码时已经编译，所以此时直接编译安卓源码即可，或者用户在安卓源码编译完成后单独编译该部分源码（由于安卓源码编译时间长，且随机出厂已经编译该库，用户可直接烧写安卓镜像即可）。

执行命令：

```
$ ./mk -a
```

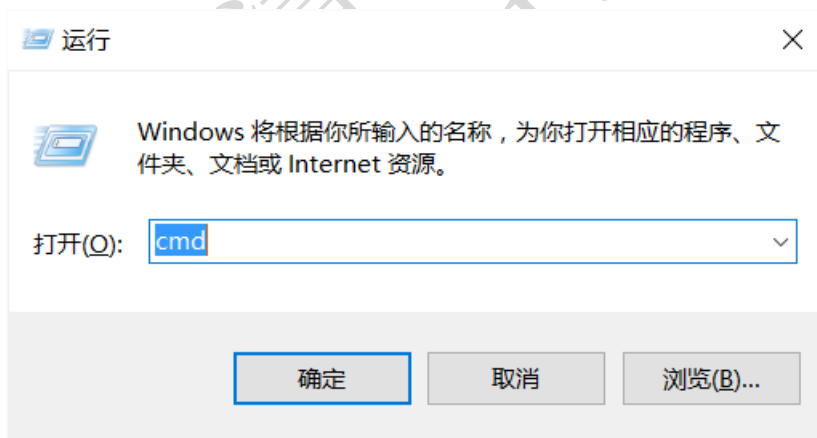
编译过程中有如下信息表示编译成功，其中 led.slsiap.so 是 HAL，libled_runtime.so 是 JNI。因编译过程中打印信息较多，可能无法找到如下信息，为了确保库被编译了，可分别进入 out/target/product/fs6818/system/lib/hw 和 out/target/product/fs6818/system/lib/目录下查找是否有这两个文件

```
Install: out/target/product/fs6818/system/lib/hw/led.slsiap.so
target_strip: ip-down (out/target/product/fs6818/obj/EXECUTABLES/ip-down_intermediate
Copy: out/target/product/fs6818/system/etc/ppp/ip-up
Copy: out/target/product/fs6818/system/bin/chat
Install: out/target/product/fs6818/system/lib/libled_runtime.so
```

编译完成后重新烧写 system.img 即可。

8.4.2 应用程序安装

在键盘上按下 Win+r 打开运行界面，输入 cmd 点击确定。



确定后进入 cmd 界面，输入命令进入 Farsight_test.apk 所在目录后执行：

```
adb install Farsight_Test.apk
```

```
部分\Android LED 点灯实验\实验源码>adb install Farsight_Test.apk
3277 KB/s (52444 bytes in 0.015s)
pkg: /data/local/tmp/Farsight_Test.apk
Success
```

8.5 实验现象

在试验前为使得实验效果明显，首先输入如下命令关闭所有 led 灯：

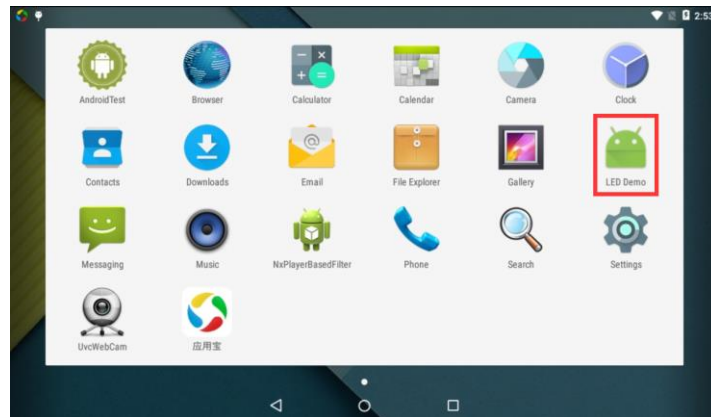
```
echo 1 > /sys/class/leds/led1/brightness
```

```
echo 1 > /sys/class/leds/led2/brightness
```



```
echo 1 > /sys/class/leds/led3/brightness
```

打开 FS6818 上的 android 应用程序 LED Demo:



应用程序打开后显示如下，默认是开灯状态：



点击图标，会看到 FS6818 开发板上的 LED 灯关掉，再次点击会把灯打开：





8.1 实验代码

8.1.1 led 灯实验应用层代码

Mainmenu.java

```
1 package com.farsight.Resource;
2 import cn.com.farsight.led.R;
3
4 import com.farsight.service.LedService;
5
6 import android.content.Context;
7 import android.content.DialogInterface;
8 import android.app.Activity;
9 import android.app.AlertDialog;
10 import android.app.AlertDialog.Builder;
11 import android.os.Bundle;
12 import android.os.Handler;
13 import android.os.Message;
14 import android.util.Log;
15 import android.widget.EditText;
16 import android.widget.ImageButton;
17 import android.hardware.SensorManager;
18 import android.hardware.SensorListener;
19 import android.widget.Button;
20 import android.view.KeyEvent;
21 import android.view.View;
22 import android.view.View.OnClickListener;
23 public class Mainmenu extends Activity
24 {
25
26     final String tag = "farsight.res";
27
28     ImageButton btn0;
29
30     static LedService led_srv;
31
32     static boolean iflag = true;
33
34
35     protected void onCreate(Bundle savedInstanceState)
36     {
37         super.onCreate(savedInstanceState);
38         setContentView(R.layout.main);
39
```



```
40     led_srv = new LedService();
41
42
43     btn0 = (ImageButton)this.findViewById(R.id.imageButton1);
44
45
46     this.btn0.setOnClickListener(new OnClickListener()
47     {
48         public void onClick(View v)
49         {
50             String title = new String();
51             if (iflag)
52             {
53                 title = led_srv.set_on(1);
54                 setTitle(title);
55                 btn0.setImageResource(R.drawable.bulb_on);
56                 iflag = false;
57             }
58             else
59             {
60                 title = led_srv.set_off(1);
61                 setTitle(title);
62                 btn0.setImageResource(R.drawable.bulb_off);
63                 iflag = true;
64             }
65         }
66     });
67 }
68 protected void onResume()
69 {
70     super.onResume();
71 }
72 protected void onStop()
73 {
74     super.onStop();
75 }
76 }
```

LedService.java

```
1 package com.farsight.service;
2 import android.util.Log;
3 public class LedService
4 {
5
6     static
```



```
7 {
8     Log.i ( "Java Service" , "*****before load led so" );
9     System.loadLibrary ( "led_runtime" );
10    Log.i ( "Java Service" , "*****after load led so" );
11 }
12
13 public LedService()
14 {
15     String icount;
16     Log.i ( "Java Service" , "do init Native Call" );
17     _init ();
18     // icount = String.valueOf(_get_count ());
19     // Log.d ( "Java Service Init OK ",icount );
20 }
21
22
23 public String set_on(int arg)
24 {
25     Log.i ( "LedService,LED On", String.valueOf(arg) );
26     _set_on (arg);
27     Log.i ( "Java Service" , "*****led on" );
28     return "led " + String.valueOf(arg) + "on";
29 }
30 public String set_off(int arg)
31 {
32     Log.i ( "LedService,LED Off", String.valueOf(arg) );
33     _set_off (arg);
34     Log.i ( "Java Service" , "*****led off" );
35     return "led " + String.valueOf(arg) + "off" ;
36 }
37 private static native boolean _init();
38 private static native int _set_on(int arg);
39 private static native int _set_off(int arg);
40 private static native int _get_count();
41 }
```

8.1.2 LED 灯实验 jni 层代码

com_farsight_service_LedService.cpp

```
1#define LOG_TAG "LedService"
2#include "utils/Log.h"
3#include <stdlib.h>
4#include <string.h>
```



```
5#include <unistd.h>
6#include <assert.h>
7#include <jni.h>
8#include "../led_stub/include/led.h"
9
10static led_control_device_t *sLedDevice = 0;
11static led_module_t *sLedModule = 0;
12
13static jint get_count(void)
14{
15    if(sLedDevice)
16        sLedDevice->getcount_led(sLedDevice);
17    else
18        ALOGI("sLedDevice is null");
19    return 0;
20}
21
22static jint led_setOn(JNIEnv *env, jobject thiz, jint arg)
23{
24    if (sLedDevice)
25    {
26        ALOGI("led_set_on");
27        sLedDevice->set_on(sLedDevice, (int)arg);
28    }
29    else
30    {
31        ALOGI("sLedDevice is null");
32    }
33    return 0;
34}
35
36static jint led_setOff(JNIEnv *env, jobject thiz, jint arg)
37{
38    if (sLedDevice)
39    {
40        ALOGI("led_set_off");
41        sLedDevice->set_off(sLedDevice, (int)arg);
42    }
43    else
44    {
45        ALOGI("sLedDevice is null");
46    }
47
48    return 0;
```



```

49}
50
51/** helper APIs */
52static inline int led_control_open(const struct hw_module_t *module,
53                                   struct led_control_device_t **device)
54{
55    ALOGI("led_control_open");
56    return module->methods->open(module,
57                                  LED_HARDWARE_MODULE_ID, (struct hw_device_t **)device);
58}
59
60static jint led_init(JNIEnv *env, jclass clazz)
61{
62    led_module_t const *module;
63    ALOGI("led_init");
64
65    if (hw_get_module(LED_HARDWARE_MODULE_ID, (const hw_module_t **)&module) == 0)
66    {
67        ALOGI("get Module OK");
68        sLedModule = (led_module_t *) module;
69        if (led_control_open(&module->common, &sLedDevice) != 0)
70        {
71            ALOGI("led_init error");
72            return -1;
73        }
74    }
75
76    ALOGI("led_init success");
77    return 0;
78}
79
80/*
81 *
82 ** Array of methods.
83 ** Each entry has three fields: the name of the method, the method
84 ** signature, and a pointer to the native implementation.
85 */
86static const JNINativeMethod gMethods[] =
87{
88    {
89        "_init",      "()Z",
90        (void *)led_init
91    },
92    {

```



```

93     "_set_on",          "(I)I",
94     (void *)led_setOn
95 },
96 {
97     "_set_off",         "(I)I",
98     (void *)led_setOff
99 },
100 {
101     "_get_count",       "()I",
102     (void *)get_count
103 },
104 };
105
106 static int registerMethods(JNIEnv *env)
107 {
108     static const char *const kClassName =
109         "com/farsight/service/LedService";
110     jclass clazz;
111     /* look up the class */
112     clazz = env->FindClass(kClassName);
113     if (clazz == NULL)
114     {
115         ALOGE("Can't find class %s\n", kClassName);
116         return -1;
117     }
118
119     /* register all the methods */
120     if (env->RegisterNatives(clazz, gMethods,
121                             sizeof(gMethods) / sizeof(gMethods[0])) != JNI_OK)
122     {
123         ALOGE("Failed registering methods for %s\n", kClassName);
124         return -1;
125     }
126     /* fill out the rest of the ID cache */
127     return 0;
128 }
129
130 /*
131  *
132  * * This is called by the VM when the shared library is first loaded.
133  */
134 jint JNI_OnLoad(JavaVM *vm, void *reserved)
135 {
136     JNIEnv *env = NULL;

```



```
137     jint result = -1;
138     ALOGI("JNI_OnLoad");
139
140     if (vm->GetEnv((void **) &env, JNI_VERSION_1_4) != JNI_OK)
141     {
142         ALOGE("ERROR: GetEnv failed\n");
143         goto fail;
144     }
145
146     assert(env != NULL);
147     if (registerMethods(env) != 0)
148     {
149         ALOGE("ERROR: PlatformLibrary native registration failed\n");
150         goto fail;
151     }
152     /* success -- return valid version number */
153     result = JNI_VERSION_1_4;
154
155 fail:
156     return result;
157 }
```

8.1.3 LED 灯实验 hal 层代码

led.c

```
1#define LOG_TAG "LedStub"
2#include <hardware/hardware.h>
3#include <fcntl.h>
4#include <errno.h>
5#include <cutils/log.h>
6#include <cutils/atomic.h>
7#include <sys/ioctl.h>
8#include "../include/led.h"
9#define LED_ON 1
10#define LED_OFF 0
11int fd;
12
13static int led_device_close(struct hw_device_t *device)
14{
15     struct led_control_context_t *ctx = (struct led_control_context_t *)device;
16     if (ctx)
17     {
18         free(ctx);
```




```
19     }
20     close(fd);
21     return 0;
22 }
23
24 static int led_getcount(struct led_control_device_t *dev)
25 {
26     ALOGI("led_getcount");
27     return 4;
28 }
29
30
31 static int led_set_on(struct led_control_device_t *dev, int arg)
32 {
33     //FIXME: do system call to control gpio led
34     ALOGI("led_set_on");
35     write(fd, "0", 1);
36     return 0;
37 }
38
39 static int led_set_off(struct led_control_device_t *dev, int arg)
40 {
41     //FIXME: do system call to control gpio led
42     ALOGI("led_set_off");
43     write(fd, "1", 1);
44     return 0;
45 }
46
47 static int led_device_open(const struct hw_module_t *module, const char *name,
48                             struct hw_device_t **device)
49 {
50     struct led_control_context_t *context;
51     ALOGD("led_device_open");
52     context = (struct led_control_context_t *)malloc(sizeof(*context));
53     memset(context, 0, sizeof(*context));
54
55     //HAL must init property
56     context->device.common.tag = HARDWARE_DEVICE_TAG;
57     context->device.common.version = 0;
58     context->device.common.module = module;
59     context->device.common.close = led_device_close;
60
61     // 初始化控制 API
62     context->device.set_on = led_set_on;
```



```
63 context->device.set_off = led_set_off;
64 context->device.getcount_led = led_getcount;
65 *device = (struct hw_device_t *) & (context->device);
66
67 if((fd = open("/sys/class/leds/led1/brightness", O_RDWR)) == -1)
68 {
69     ALOGI("open error");
70     // exit(1);
71 }
72 else
73     ALOGI("open ok\n");
74
75 return 0;
76}
77
78
79static struct hw_module_methods_t led_module_methods =
80{
81open:
82    led_device_open
83};
84
85struct hw_module_t HAL_MODULE_INFO_SYM =
86{
87    .tag = HARDWARE_MODULE_TAG,
88    .version_major = 1,
89    .version_minor = 0,
90    .id = LED_HARDWARE_MODULE_ID,
91    .name = "led HAL module",
92    .author = "farsight",
93    .methods = &led_module_methods,
94    /* supporting APIs go here */
95};
96
97
98
99
100
101
102
```



第 9 章 Android UART 通讯实验

Android UART 通讯实验对于理解 android app 层、jni 层、hal 层的调用关系有重要意义。

9.1 实验目的

通过实验熟悉 android app 层、jni 层、hal 层之间的调用关系。

9.2 实验原理

同“第 8 章 Android LED 点灯实验原理”

9.3 实验步骤

本例实验代码在：【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Android 底层实验源码\Android UART 通讯实验\实验源码\串口实验】目录下

Uart	2017/8/18 10:22	文件夹	
libuart_runtime.so	2017/8/25 18:34	SO 文件	6 KB
Uart.apk	2013/3/27 22:45	APK 文件	15 KB
uart.slsiap.so	2017/8/25 18:30	SO 文件	6 KB
uart.tar.xz	2016/11/16 12:59	WinRAR 压缩文件	5 KB

Uart.tar.xz : JNI 和 HAL 源码

UART 应用：应用程序源码

Uart.apk: 应用程序

两个 so 文件分别为 hel 层和 jni 层编译生成的共享库

9.3.1 底层代码编译

打开虚拟机 ubuntu 系统，进入终端 android 源码目录：

执行：

```
$ cd /home/linux/workdir/Fs6818
```

因 Android 源码中 hardware/farsight/uart 目录下在出厂时已有这部分代码，就不再进行拷贝工作，该源码在编译安卓源码时已经编译，所以此时直接编译安卓源码即可，或者用户在安卓源码编译完成后单独编译该部分源码（由于安卓源码编译时间长，且随机出厂已经编译该库，用户可直接烧写安卓镜像即可）。

执行命令：

```
$ ./mk -a
```

编译过程中有如下信息表示编译成功，其中 uart.slsiap.so 是 HAL，libuart_runtime.so 是 JNI。因编译过程中打印信息较多，可能无法找到如下信息，为了确保库被编译了，可分别进 out/target/product/fs6818/system/lib/hw 和 out/target/product/fs6818/system/lib/目录下查找是否有这两个文件

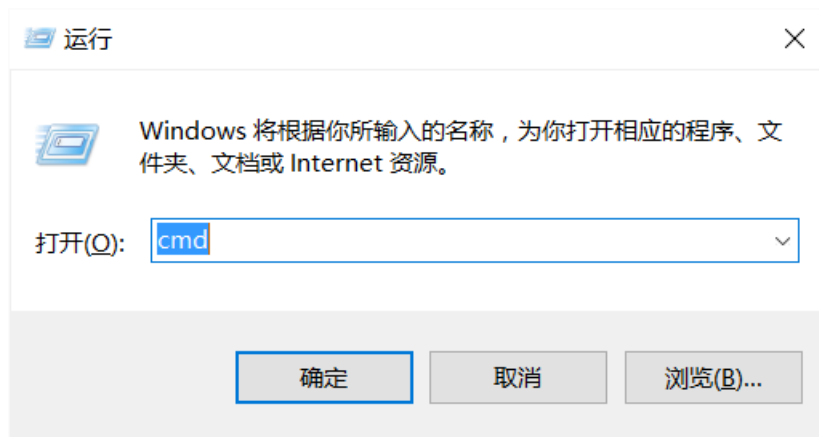
```
Install: out/target/product/fs6818/system/lib/hw/uart.slsiap.so
Install: out/target/product/fs6818/system/lib/libuart_runtime.so
```



编译完成后重新烧写 system.img 即可。

9.3.2 应用程序安装

在键盘上按下 Win+r 打开运行界面，输入 cmd 点击确定



确定后进入 cmd 界面,输入命令进入 Uart.apk 所在目录后执行:

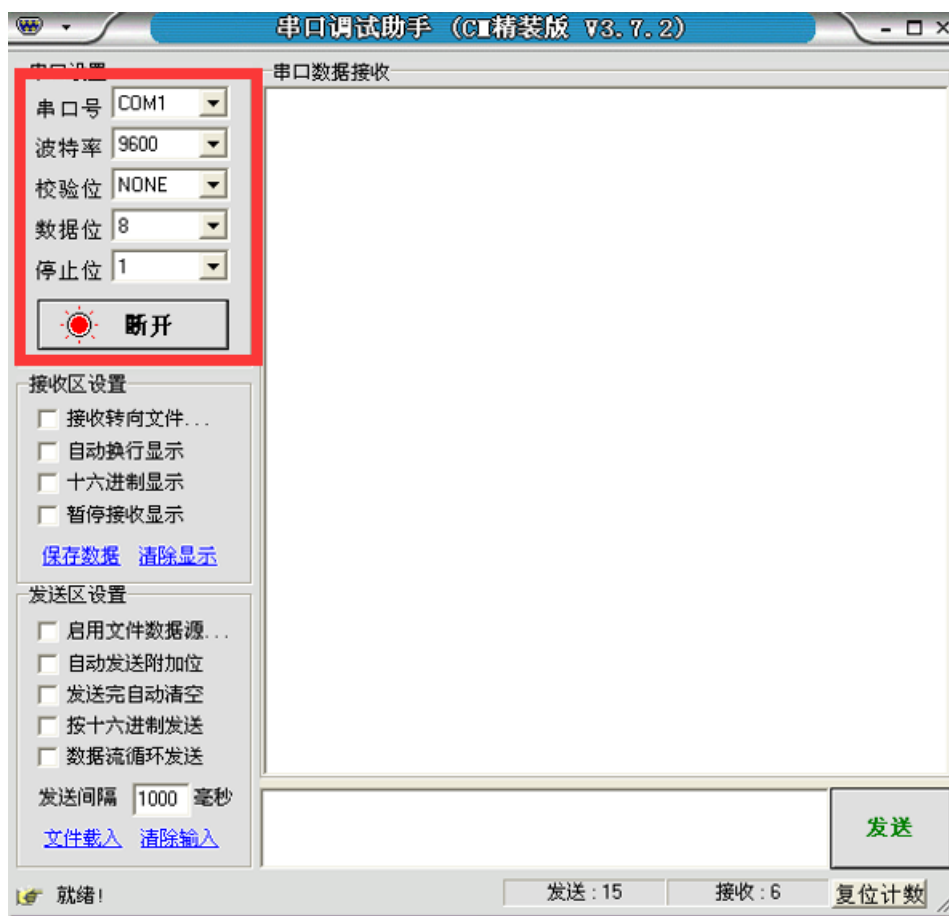
```
adb install Uart.apk
```

```
部分\Android UART 通讯实验\实验源码>adb install Uart.apk
14 KB/s (14842 bytes in 1.000s)
  pkg: /data/local/tmp/Uart.apk
Success
```

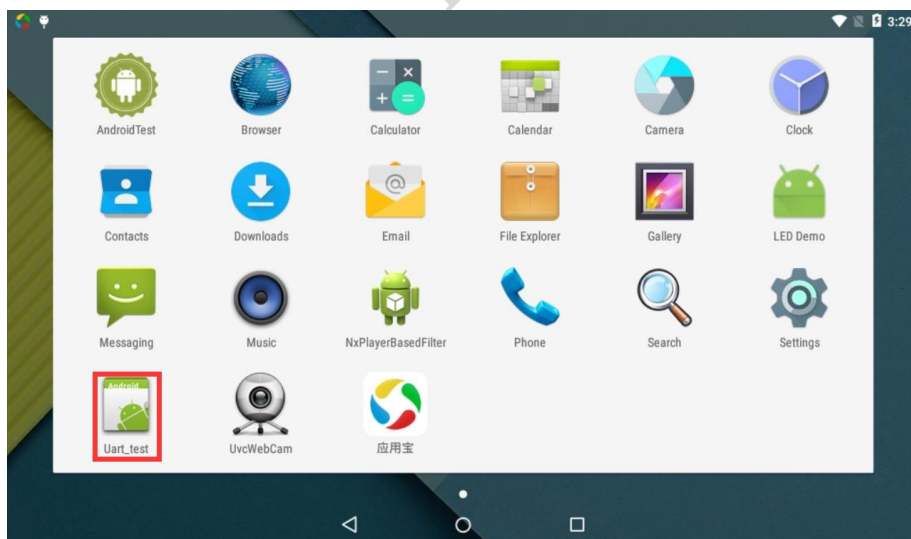
9.4 实验现象

通过 usb 转串口线将 FS6818 的 UART1 与 PC 连接。在 PC 设备管理器中查看 usb 转串口的设备编号。如笔者 USB 转串口识别为 COM1。

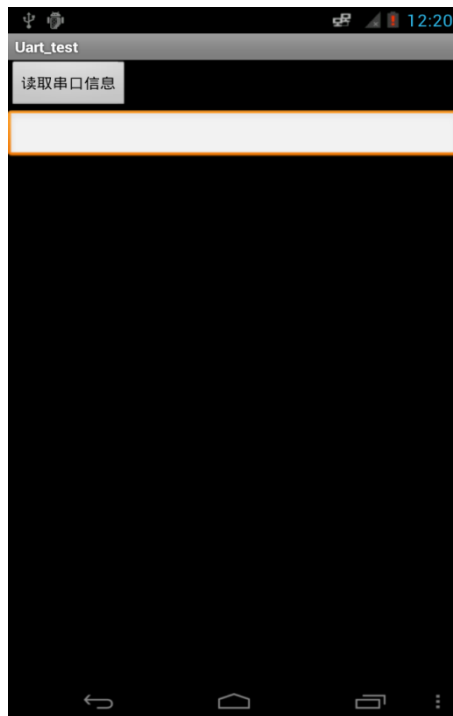
打开【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Android 底层实验源码\Android UART 通讯实验\工具相关】路径下的 UartAssist.exe，配置串口，如图所示：



打开 FS6818 上的 android 应用程序 Uart_test:



应用程序打开后:



pc 机端输入 “abc”，然后点击“发送”：



点击“读取串口信息”按钮，此时 android 应用程序会逐个解析字符并显示：



Android 应用程序读取到串口的字符后，再向 pc 机串口端发送所收到的字符，如图所示：





9.5 实验代码

9.5.1 串口通讯实验应用层代码

myuart.java

```
1 package cn.com.muart.display;
2
3
4 import cn.com.mgps.display.R;
5 import cn.com.muart.uart.Uart;
6 import android.app.Activity;
7 import android.os.Bundle;
8 import android.util.Log;
9 import android.view.View;
10 import android.view.View.OnClickListener;
11 import android.widget.Button;
12 import android.widget.EditText;
13
14 public class MGpsActivity extends Activity
15 {
16     /** Called when the activity is first created. */
17     EditText et;
18     static Uart uartuart = new Uart();
19     char s;
20     //String s;
21     String s1, s2;
22
23     @Override
24     public void onCreate(Bundle savedInstanceState)
25     {
26         super.onCreate(savedInstanceState);
27         setContentView(R.layout.main);    //布局视图
28
29         et = (EditText)this.findViewById(R.id.et);
30         Button button = (Button)this.findViewById(R.id.button1);
31
32         button.setOnClickListener(new OnClickListener()//开启串口通信
33         {
34
35             public void onClick(View v)
36             {
37                 s = uartuart.GpsDataUpdate();
38                 updateView(s);
39                 uartuart.GpsDataUpdate_write(String.valueOf(s));
```




```
40         };
41     });
42 }
43
44
45
46 public void updateView(char s)
47 {
48     //if(s !=null){    //判断是否为空
49     et.setText(s + "");
50     // }
51     //else{
52     //如果传入的 Location 对象为空则清空 EditText
53     //et.setEditableText().clear();//清空 EditText 对象
54     // et.setText("loading...");
55     //}
56 }
57}
58
59package cn.com.muart.uart;
60import java.io.BufferedReader;
61import java.io.IOException;
62import java.io.InputStream;
63import java.io.InputStreamReader;
64
65import android.util.Log;
66
67public class Uart
68{
69     static
70     {
71         System.loadLibrary("uart0_runtime");
72     }
73     public Uart()
74     {
75         _init();
76     }
77
78     public char GpsDataUpdate()
79     {
80         char s = _uart_data_update();
81         return s;
82     }
83     public int GpsDataUpdate_write(String s )
```



```
84     {
85         _uart_data_update_write(s);
86         return 0 ;
87     }
88 }
89
90 private static native boolean _init();
91 private static native char _uart_data_update();
92 private static native int _uart_data_update_write(String s);
93
94 }
```

9.5.2 串口通讯实验 jni 层代码

cn_com_uart_MyUartService.cpp

```
#define LOG_TAG "MyUartService"
#include "utils/Log.h"
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <assert.h>
#include <jni.h>
#include "../uart_stub/include/uart.h"

static uart_control_device_t *sUartDevice = 0;
static uart_module_t *sUartModule = 0;

static jchar uart_read(JNIEnv *env, jobject thiz)
{
    char s_buf[100] = {0};
    //char* s = s_buf;

    if (sUartDevice)
    {
        sUartDevice->uart_read_hal(sUartDevice, s_buf, 1);
    }
    else
    {
        ALOGI("sUartDevice is null");
    }

    if (s_buf == 0)
    {
        return 0;
    }
}
```



```

        ALOGI("JNI: Lost info");
    else
    {
        ALOGI("*****JNI:read info is : %s", s_buf);
        ALOGI("*****JNI:info length is : %d", strlen(s_buf));
    }

    return s_buf[0]; //将 Cstring 转换为 Jstring
}
static int uart_write(JNIEnv *env, jobject thiz, jstring s )
{
    char *rtn = NULL;
    jclass clsstring = env->FindClass("java/lang/String");
    jstring strencode = env->NewStringUTF("GB2312");
    jmethodID mid = env->GetMethodID(clsstring, "getBytes", "(Ljava/lang/String;)[B");
    jbyteArray barr = (jbyteArray)env->CallObjectMethod(s, mid, strencode);
    jsize alen = env->GetArrayLength(barr);
    jbyte *ba = env->GetByteArrayElements(barr, JNI_FALSE);
    if(alen > 0)
    {
        rtn = (char *)malloc(alen + 1); //new char[alen+1];
        memcpy(rtn, ba, alen);
        rtn[alen] = 0;
    }
    env->ReleaseByteArrayElements(barr, ba, 0);
    if (sUartDevice)
    {
        ALOGI("*****JNI:my uart to write");
        sUartDevice->uart_write_hal(sUartDevice, rtn, strlen(rtn));
    }
    else
    {
        ALOGI("sUartDevice is null");
    }

    if (rtn == 0)
        ALOGI("JNI: Lost info");
    else
    {
        ALOGI("*****JNI:write info is : %s", rtn);
        ALOGI("*****JNI:write info length is : %d", strlen(rtn));
    }

    //return env->NewStringUTF(s); //将 Cstring 转换为 Jstring
}

```



```

    return 0 ;
}

/** helper APIs */

static inline int uart_control_open(const struct hw_module_t *module,
                                    struct uart_control_device_t **device)
{
    ALOGI("*****JNI:uart_control_open");
    return module->methods->open(module,
                                  UART_HARDWARE_MODULE_FS_ID, (struct hw_device_t **)device);
    //通过标准的 Open 接口, 调用 stub 中的 open 函数
}

static jint uart_init(JNIEnv *env, jclass clazz)
{
    uart_module_t const *module;
    ALOGI("*****JNI_1.2:uart_init");

    //调用 Android HAL 标准函数 hw_get_module, 通过 LED_HARDWARE_MODULE_ID 获取 LED Stub 的句柄,
    //句柄保存在 module 变量中
    if (hw_get_module(UART_HARDWARE_MODULE_FS_ID, (const hw_module_t **)&module) == 0)
    {
        ALOGI("*****get Module OK");
        sUartModule = (uart_module_t *) module;
        if (uart_control_open(&module->common, &sUartDevice) != 0)
        {
            ALOGI("*****uart_init error");
            return -1;
        }
    }
    ALOGI("*****uart_init success");
    return 0;
}

/*
JNINativeMethod 是 jni 层注册的方法, Framework 层可以使用这些方法
*_init 、uart_read、uart_write 是在 Framework 中调用的方法名称, 函数的类型及返回值如下:
*( )Z 无参数 返回值为 bool 型
*( )Ljava/lang/String; 无参数 返回值为 String 型
*( II)I 参数为 2 个 int 型 返回值为 int 型
*/
static const JNINativeMethod gMethods[] =

```



```
{
    {
        "_init",      "()Z",
        (void *)uart_init
    },

    {
        "_uart_data_update",
        //"(Ljava/lang/String;",
        "()C",
        (void *)uart_read
    },
    {
        "_uart_data_update_write",      "(Ljava/lang/String;)I", //Ljava/lang/String;
        (void *)uart_write
    },
};

static int registerMethods(JNIEnv *env)
{
    static const char *const kClassName = "cn/com/muart/uart/Uart";
    // "cn/com/farsight/refrigerator/energy/view/Uart";

    // "cn/com/muart/uart/Uart"; //注意：必须和 Framework 层的 service 类名相
    jclass clazz;
    /* look up the class */
    clazz = env->FindClass(kClassName); //查找被注册的类
    if (clazz == NULL)
    {
        ALOGE("Can't find class %s\n", kClassName);
        return -1;
    }

    /* register all the methods */
    if (env->RegisterNatives(clazz, gMethods,
                            sizeof(gMethods) / sizeof(gMethods[0])) != JNI_OK)
    {
        ALOGE("Failed registering methods for %s\n", kClassName);
        return -1;
    } //向类中注册本 SO 中 Native 的接口，接口定义在 gMethods 数组中
    /* fill out the rest of the ID cache */
    return 0;
}
```



```
}

/*
 *
 * * This is called by the VM when the shared library is first loaded.
 */
/* 当第一次加载此库时，调用此函数注册本地方法 */
jint JNI_OnLoad(JavaVM *vm, void *reserved)
{
    JNIEnv *env = NULL;
    jint result = -1;
    ALOGI("zxs_JNI_OnLoad");

    if (vm->GetEnv((void **) &env, JNI_VERSION_1_4) != JNI_OK)
    {
        LOGE("ERROR: GetEnv failed\n");
        goto fail;
    } //获取当前的VM的环境，保存在env变量中

    assert(env != NULL);
    if (registerMethods(env) != 0)
    {
        LOGE("ERROR: PlatformLibrary native registration failed\n");
        goto fail;
    } //自己写的函数，向当前JAVA环境中注册接口
    /* success -- return valid version number */
    result = JNI_VERSION_1_4;

fail:
    return result;
}
```

9.5.3 串口通讯实验 hal 层代码

uart.c

```
1#define LOG_TAG "UartStub"
2#include <hardware/hardware.h>
3#include <fcntl.h>
4#include <termios.h>
5#include <errno.h>
6#include <cutils/log.h>
7#include <cutils/atomic.h>
8#include <sys/ioctl.h>
```



```

9#include <errno.h>
10#include <string.h>
11#include "../include/uart.h"
12
13int fd;
14
15static int uart_device_close(struct hw_device_t *device)
16{
17    struct uart_control_device_t *ctx = (struct uart_control_device_t *)device;
18    if (ctx)
19    {
20        free(ctx);
21    }
22    close(fd);
23    return 0;
24}
25
26static int uart_read_drv(struct uart_control_device_t *dev, char *buf, int count)
27{
28    int count1 = 0;
29    count1 = read(fd, buf, count);
30    if(count1 < 0)
31    {
32        ALOGI("*****hal1.2 read buf count1: %d\n", count1);
33        perror("read");
34    }
35
36    LOGI("*****hal1.2 read buf : %s\n", buf);
37
38    return 0;
39}
40
41/*****/
42static int uart_write_drv(struct uart_control_device_t *dev, char *buf, int count)
43{
44    count = write(fd, buf, strlen(buf));
45    LOGI("*****hal1.2 write buf count : %d\n", count);
46    if(count < 0)
47    {
48        perror("write");
49    }
50
51    ALOGI("*****hal1.1 write buf : %s\n", buf);
52    ALOGI("*****hal1.1 write buf strlen(buf) : %d\n", strlen(buf));

```



```
53
54     LOGI("*****moni write buf");
55     // write(fd, "moni", 5);
56     return 0;
57 }
58 /*****
59
60
61 static int uart_device_open(const struct hw_module_t *module, const char *name,
62                             struct hw_device_t **device)
63 {
64     struct uart_control_device_t *dev;
65     struct termios opt;
66
67     ALOGI("*****HAL_1.1:uart_device_open");
68     dev = (struct uart_control_device_t *)malloc(sizeof(*dev));
69     memset(dev, 0, sizeof(*dev));
70
71     //HAL must init property
72     dev->common.tag = HARDWARE_DEVICE_TAG; //必须写这个
73     dev->common.version = 0;
74     dev->common.module = module;
75
76     dev->uart_read_hal = uart_read_drv;
77
78     dev->uart_write_hal = uart_write_drv;
79     *device = &dev->common;
80
81     if((fd = open("/dev/ttySAC1", O_RDWR | O_NOCTTY | O_NDELAY)) < 0)
82     {
83         ALOGE("open error");
84         return -1;;
85     }
86     else
87     {
88         ALOGI("*****serial 1 : open ok\n");
89     }
90
91     //初始化串口
92     tcgetattr(fd, &opt);
93     //tcflush(fd, TCIOFLUSH);
94     cfsetispeed(&opt, B9600);
95     cfsetospeed(&opt, B9600);
96
```




```

97     //tcflush(fd, TCIOFLUSH);
98
99     opt.c_cflag |= (CLOCAL | CREAD);
100
101     opt.c_cflag &= ~CSIZE;
102     opt.c_cflag &= ~CRTSCTS;
103     opt.c_cflag |= CS8;
104
105     /*
106     opt.c_cflag |= PARENB; // enable; 允许输入奇偶校验
107     opt.c_cflag |= PARODD; // J check    对输入使用奇校验
108     opt.c_iflag |= (INPCK | ISTRIP); //
109     */
110     opt.c_iflag |= IGNPAR;
111
112     opt.c_cflag &= ~CSTOPB;
113
114     opt.c_oflag = 0;
115     opt.c_lflag = 0;
116
117
118     //opt.c_cc[VTIME] = 0;
119     //opt.c_cc[VMIN] = 0;
120
121     tcsetattr(fd, TCSANOW, &opt);
122
123     LOGI("*****serial 1 : init ok\n");
124
125     return 0;
126 }
127
128 //定一个 hw_module_methods_t 结构体, 关联入口函数
129 static struct hw_module_methods_t uart_module_methods =
130 {
131     open:
132         uart_device_open
133 };
134
135 //定义 Stub 入口
136 //注意必须使用:
137 //1. hw_module_t 继承类
138 //2. 必须使用 HAL_MODULE_INFO_SYM 这个名字
139
140 struct hw_module_t HAL_MODULE_INFO_SYM =
    
```



```
141{
142    .tag = HARDWARE_MODULE_TAG,
143    .version_major = 1,
144    .version_minor = 0,
145    .id = UART_HARDWARE_MODULE_FS_ID,
146    //模块 ID, 上层的 Service 通过这个 ID 应用当前 Stub
147    .name = "uart HAL module",
148    .author = "farsight",
149    .methods = &uart_module_methods, //入口函数管理结构体
150    /* supporting APIs go here */
151};
152
153
154
155
156
157
158
```

华清远见
dev.havi.com



第 10 章 Android 与传感器设计与实现

传感器是一种物理装置或生物器官，能够探测、感受外界的信号、物理条件（如光、热、湿度）或化学组成（如烟雾），并将探知的信息传递给其他装置或器官。在 Android 设备中，有很多常见的传感器，如光感应器、重力感应器等，本章主要介绍在 S5PV210 平台上 Android 传感器的原理和示例分析。

10.1 Android Sensor 实现原理概述

在 Android 系统中实现了 HAL 的 stub，并向 JNI 层注册相应服务，包括 LED 服务、BEEP 服务、KEY 服务，以及由 Android 系统实现的 Sensor 服务。

Linux 驱动与普通的 Linux 驱动完全一致，而在 HAL 中实现的 stub 需要实现 hw_module_t 结构体，包括填充 version、HAL 的 ID，以及上层调用时所需要的方法，下面针对 Android sensor 的 stub 实现进行具体分析。

10.2 JNI 原理

Android HAL（硬件抽象层）是一个 Linux 底层驱动与 Java ApplicationFramework（应用层框架）的中间层，它可以将硬件的操作逻辑封装起来，实现 Java 类的本地接口，交由 Android 应用框架统一管理。HAL 下与驱动层打交道，上与 Android runtime（Java 虚拟机）联系，同时 Android 的很多应用库也依赖于 HAL 提供的接口。HAL 层由 C/C++ 实现，具体采用哪种语言则依据于实际情况。一个通用的原则是，操纵硬件的代码用 C 语言实现，为上层提供接口的函数用 C++ 更为合适，尤其是与 Java 类进行交互时，用 C++ 实现将容易很多。那么 C/C++ 是如何与 Java 实时进行交互的呢？这就引入了 JNI。

JNI 的调用结构如图 5.1 所示，Java 类中需要实现的是声明本地方法和加载共享库，然后就可以像平时一样使用对象了。

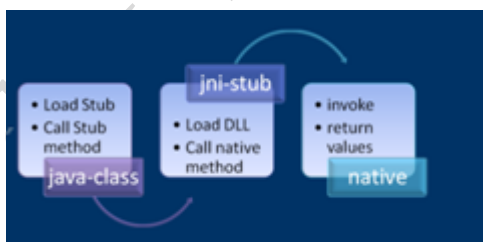


图 5.1

如图 5.2 所示为 JNI 的使用步骤。



图 5.2

由上述介绍可知，Java 程序与 JNI 编译成的共享库之间联系如下：

Java 类加载共享库用 loadlibrary 方法，名称与 JNI 共享库名称对应。

Java 调用 JNI 中的类需要声明本地方法，其中本地方法的类型需要与 JNI 中 JNINativeMethod 中提供的方法声明、signature 一致。



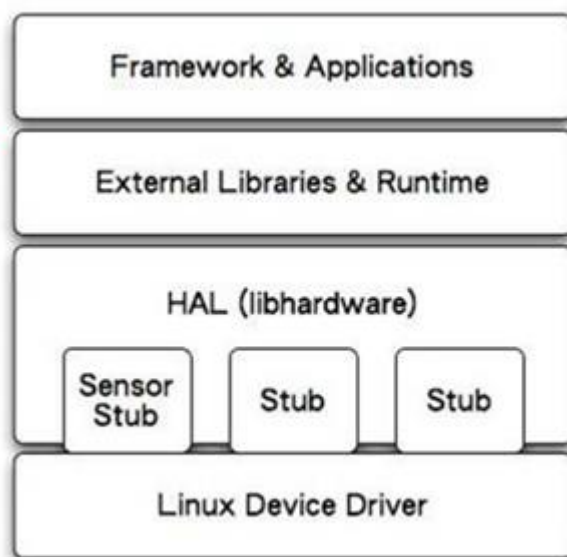
JNI 类中需要在 jni helper 函数声明调用 Java 类，这样才能实现 Java 虚拟机与本地方法的联系。

Java 服务类的全称需与 JNI 的文件名有一定的对应关系，如 Java 服务类名为 com.farsight.Service.LedService，对应 JNI 的 service 应用名为 com_farsight_service_LedService.cpp。

10.3 Android HAL

HAL Stub 是一种代理人（Proxy）的概念，Stub 虽然仍是以 *.so 的形式存在的，但 HAL 已经将*.so 隐藏起来了。Stub 作为直接调用 Linux 驱动的上层，实现了简单的 I/O 操作，Stub 向 HAL 提供操作函数（Operations），而 Runtime 则向 HAL 取得特定模块（Stub）的 Operations，再回调这些操作函数。这种以 indirect function call 的架构，让 HAL Stub 变成一种包含关系，即 HAL 中包含了许多 Stub（代理人）。Runtime 只要说明类型，即 module ID，就可以取得操作函数。对于目前的 HAL，可以认为 Android 定义了 HAL 层结构框架，通过几个接口访问硬件从而统一了调用方式。

如图 5.3 所示是在以前的 HAL 层与驱动层之间又加了一层，那么它最大的好处是什么呢？一定是在对设备的管理上。以前一个设备实现一个 HAL 层的 Service，而加入 Stub 则可以将多个设备统一到一个 Service 之下。一个典型的例子就是本文所分析的 Sensor Stub。我们知道传感器是一大类的设备，它们功能多样，形制也大相迥异，却有着很多通用的思想，如都是自主采样，交由上层去处理，采样得到的数据结构有通用的数据格式，同类型传感器很多接口规范高度一致，易于扩展，等等。传统的 HAL 模式对这种具有高度规范性的设备显然通用性不足，于是 Sensor 统一架构应运而生。



10.4 Sensor 传感器系统

Android 传感器源码结构如下图所示。



用户空间	Java应用程序	
	Java Framework框架层	SensorManager.java SensorService.java
	JNI	android.hardware.SensorManager.cpp com.android.server.SensorService.cpp
	硬件抽象层	sensors.cpp
内核空间	G-sensor设备驱动程序	bma220_driver.c
硬件	G-sensor重力感应器	

Android 中涉及传感器的几个文件位于 Android 源码的如下几个位置：

frameworks/base/core/java/Android/hardware/SensorManager.java。

frameworks/base/core/jni/Android_hardware_SensorManager.cpp。

frameworks/base/services/java/com/Android/server/SensorService.java。

frameworks/base/services/jni/com_Android_server_SensorService.cpp。

再加上需要由我们实现的 sensorstub，以及 Linux 内核驱动（如 ADC 驱动、三轴传感器驱动、加速度计驱动等），构成了整个 Sensor 框架，用户应用程序只需调用 sensormanager 类，即可操作所有的传感器。

10.4.1 Sensor Stub 分析

Sensor Stub 作为直接与驱动程序打交道的接口，必须实现所有传感器的基本操作程序接口，同时还要兼顾传感器系统的核心特点，将所有传感器纳入到统一框架之下，实现多个传感器的访问控制。我们在 Linux 应用学习中已经了解了并发中的 I/O 多路复用的概念，在驱动学习中知道了设备通过设置 POLL 标志位，可以将可读/写状态交由内核处理，实现 Select 和 Poll 方法。那么在 Sensor 系统中多个传感器也可以引入多路复用的概念。当传感器有数据更新时，我们再对其进行操作，从而提高效率，减小功耗，因此 Sensor Stub 的核心即是实现 Poll 方法。

下面通过一个 Sensor Stub 的实例源码分析其中的要点。

Sensor Stub 的一个核心理念就是控制与分离数据，这样做的好处就是可以更灵活地控制所有传感器的状态，包括睡眠、唤醒、读数、关闭等，从而最精细化电源管理，毕竟 Android 多用在手持设备中，能耗控制可以说是核心功能。

于是，我们有了 structensors_control_context_t 与 structensors_data_context_t。这两个结构体一层套一层，下面来分析一下。

structensors_control_context_t 结构体如下图所示，其中 sensor_fd 是传感器的文件描述符，active_sensors 是当前激活的 sensor 个数，而 structensors_control_device_t 结构体如下图所示，则包含了



一个 struct hw_device_t 及一系列设备的操作方法。

```
struct sensors_control_context_t {
    struct sensors_control_device_t device;
    int sensor_fd;
    uint32_t active_sensors;
};
```

```
struct sensors_control_device_t {
    struct hw_device_t common;
    native_handle_t* (*open_data_source)(struct sensors_control_device_t *dev);
    int (*close_data_source)(struct sensors_control_device_t *dev);
    int (*activate)(struct sensors_control_device_t *dev,
        int handle, int enabled);
    int (*set_delay)(struct sensors_control_device_t *dev, int32_t ms);
    int (*wake)(struct sensors_control_device_t *dev);
};
```

如下图所示，可以看到这个 sensors_control_device_t 结构体包含了标签、版本信息，以及一个关闭设备的函数方法，最关键的则是 hw_module_t，指向这个设备属于的模块，那么这个模块又是什么呢？

```
typedef struct hw_device_t {
    /** tag must be initialized to HARDWARE_DEVICE_TAG */
    uint32_t tag;

    /** version number for hw_device_t */
    uint32_t version;

    /** reference to the module this device belongs to */
    struct hw_module_t* module;

    /** padding reserved for future use */
    uint32_t reserved[12];

    /** Close this device */
    int (*close)(struct hw_device_t* device);
} hw_device_t;
```

如下图所示，hw_module_t 结构体是 HAL 模块的核心结构体，所有硬件模块都必须将这个结构体作为第一个成员，表示继承了这个结构体。这个结构体中有表示模块的 ID，就是打开 HAL 层的钥匙。



```
typedef struct hw_module_t {
    /** tag must be initialized to HARDWARE_MODULE_TAG */
    uint32_t tag;

    /** major version number for the module */
    uint16_t version_major;

    /** minor version number of the module */
    uint16_t version_minor;

    /** Identifier of module */
    const char *id;

    /** Name of this module */
    const char *name;

    /** Author/owner/implementor of the module */
    const char *author;

    /** Modules methods */
    struct hw_module_methods_t* methods;

    /** module's dso */
    void* dso;

    /** padding to 128 bytes, reserved for future use */
    uint32_t reserved[32-7];
} hw_module_t;
```

另外，如下图所示的结构体中还有一个方法结构体 `hw_module_methods_t`，其中只有一个 `open` 方法，这个方法称为 helper API，用来打开这个模块，即通过 `id` 使 `hw_module_t` 得到 `hw_device_t`。

```
typedef struct hw_module_methods_t {
    /** Open a specific device */
    int (*open)(const struct hw_module_t* module, const char* id,
                struct hw_device_t** device);
} hw_module_methods_t;
```

至此，`sensors_control_context_t` 结构体中的数据结构就介绍完了，硬件的具体调用流程总结如下：
通过 `id` 得到硬件模块。

从硬件模块中得到 `hw_module_methods_t`，打开得到 `hw_device_t`。

调用 `hw_device_t` 中的各个方法。

调用 `hw_device_t` 中的 `close` 关闭设备。

其中第（2）步打开是通过 `hw_get_module` 函数实现的，其原型是：

```
int hw_get_module(const char *id, const struct hw_module_t **module);
```

位于 `hardware/libhardware/include/hardware/hardware.h`。

在调用过程中通过 `hw_get_module` 根据模块 `id` 寻找硬件模块动态链接库地址，然后调用 `load` 函数打开动态链接库并从动态链接库中获取硬件模块结构体地址。硬件模块路径格式如下：



HAL_LIBRARY_PATH/id.prop.so。

HAL_LIBRARY_PATH 定义为/system/lib/hw。

id 是 hw_get_module 函数的第一个参数所传入的，prop 部分首先按照 variant_keys 数组中的名称逐一调用 property_get 获取对应的系统属性，然后访问 HAL_LIBRARY_PATH/id.prop.so，如果能找到，访问就结束，否则就访问 HAL_LIBRARY_PATH/id.default.so。

所以开发硬件模块时 Makefile 文件（Android.mk）中模块的命名 LOCAL_MODULE 要参考上面的内容，否则就无法访问。

回到 Sensor 设备，在 structsensors_control_device_t 中，open_data_source 是一个核心的方法，它连接了 sensors_control_device_t 与 sensors_data_device_t，简而言之，就是将 control 中打开的设备号 device_fd 转换为 I/O 多路复用中的设备号 event_fd，这是通过一个 native_handle_t 结构体实现的。

如下图所示，这个结构体保存了当前打开的文件描述符和注册的中断的总数，以及它们的具体内容。这样一个 handle 传给了数据层，自然是加入了 POLL 池里用做统一管理。

```
typedef struct
{
    int version;           /* sizeof(native_handle_t) */
    int numFds;            /* number of file-descriptors at &data[0] */
    int numInts;           /* number of ints at &data[numFds] */
    int data[0];           /* numFds + numInts ints */
} native_handle_t;
```

如下图所示是 structsensors_data_context_t 结构体，这个结构体与之前分析的 structsensors_control_context_t 结构相似，event_fd 是 I/O 多路复用中的设备号，而 sensors_data_t 则是传感器数据的统一结构。

```
struct sensors_data_context_t {
    struct sensors_data_device_t device;
    int event_fd;
    sensors_data_t sensors[MAX_NUM_SENSORS];
};
```

如下图所示，sensors_data_t 结构体的结构一目了然，sensor 是传感器的识别符，而联合体则将多种类型的传感器数据统一封装起来，从而可以将任何传感器的数据进行统一处理。



```
typedef struct {
    /* sensor identifier */
    int sensor;

    union {
        /* x,y,z values of the given sensor */
        sensors_vec_t vector;

        /* orientation values are in degrees */
        sensors_vec_t orientation;

        /* acceleration values are in meter per second per second (m/s^2) */
        sensors_vec_t acceleration;

        /* magnetic vector values are in micro-Tesla (uT) */
        sensors_vec_t magnetic;

        /* temperature is in degrees centigrade (Celsius) */
        float temperature;

        /* distance in centimeters */
        float distance;

        /* light in SI lux units */
        float light;
    };

    /* time is in nanosecond */
    int64_t time;

    uint32_t reserved;
} sensors_data_t;
```

如下图所示，这个结构体中的方法是必须实现的核心方法，其中的 data_open、data_close 提供了一个更细粒度的控制策略，因为多个程序可以进行传感器数据的开启、关闭，而不会影响硬件层的操作，这也是一种安全上的隔离。

```
struct sensors_data_device_t {
    struct hw_device_t common;
    int (*data_open)(struct sensors_data_device_t *dev, native_handle_t* nh);
    int (*data_close)(struct sensors_data_device_t *dev);
    int (*poll)(struct sensors_data_device_t *dev,
                sensors_data_t* data);
};
```

接下来就是核心的 poll 方法。在这里可以实现网络编程中的 Select 模型，实现 I/O 多路复用。当然并不是一定要实现 I/O 端口复用，在传感器较少时，采用轮询加睡眠的策略也能够满足需求。

当然，从 sensor_stub 的功能来说，第一步要做的是注册可用传感器列表，并提交给 sensor_manager。

除了这些之外，sensor_stub 一般还需提供之前提到过但没有实现的 wake、active、delay 等方法，这些都可以根据具体情况进行设计，以达到精细控制的目的。

至此，sensor_stub 的分析基本结束了，事实上我们只是分析了整个框架的最下层，可以大概浏览一下 sensor_service 与 sensor_manager 的 JNI 部分，会发现它们主要是将 stub 中的 open、close 等方法封装提交给 android application 框架。

```
static jint
android_init(JNIEnv *env, jclass clazz)
```



在 init 中会调用 open 方法初始化:

```
static jobject
android_open(JNIEnv *env, jclass clazz)
```

在 open 中会打开所有设备文件与中断, 当然这已经在 stub 中进行过隔离了。

其中值得注意的是, JNI 的数据类型与 Java 数据类型的相互转化及函数注册, 这是通过 JNINative_Method 结构体实现的, 代码如下:

```
typedef struct {
    const char* name; // java方法名称
    const char* signature; // java方法签名
    void*      fnPtr; // c/c++的函数指针
} JNINativeMethod;
```

其中的 signature 规则如下图所示。

字符 Java类型 C类型

V	void	void
Z	jboolean	boolean
I	jint	int
J	jlong	long
D	jdouble	double
F	jfloat	float
B	jbyte	byte
C	jchar	char
S	jshort	short

数组则以“[]”开始, 用两个字符表示

[I	jintArray	int[]
[F	jfloatArray	float[]
[B	jbyteArray	byte[]
[C	jcharArray	char[]
[S	jshortArray	short[]
[D	jdoubleArray	double[]
[J	jlongArray	long[]
[Z	jbooleanArray	boolean[]

如上图所示都是基本类型, 如果参数是 Java 类, 则以“L”开头, 以“;”结尾, 中间用“/”隔开包及类名, 而对应的 C 函数的参数则为 jobject, 一个例外是 String 类, 它对应 C 类型 jstring, 例如, Ljava/lang/String; 、 Ljava/net/Socket; 等, 如果 Java 函数位于一个嵌入类 (又称内部类), 则用“\$”作为类名间的分隔符, 例如, "LAndroid/os/FileUtils\$FileStatus;"。

使用 registerNativeMethods 方法不仅是为了改变方法名, 最重要的是可以提高效率, 因为当 Java 类通过 VM 呼叫到本地函数时, 通常是依靠 VM 去动态寻找。so 中的本地函数 (因此它们才需要特定规则的命名格式), 如果某方法需要连续呼叫很多次, 则每次都要寻找一遍, 所以使用 RegisterNatives 将本



地函数向 VM 进行登记，可以让其更有效率的找到函数。

接下来看一下 sensor_service 的 Java 部分。

在 SensorService 中必须实现的核心方法就是 enablesensors 方法，在 sensor_manager 的 Java 部分中声明了本地方法，如下图所示。

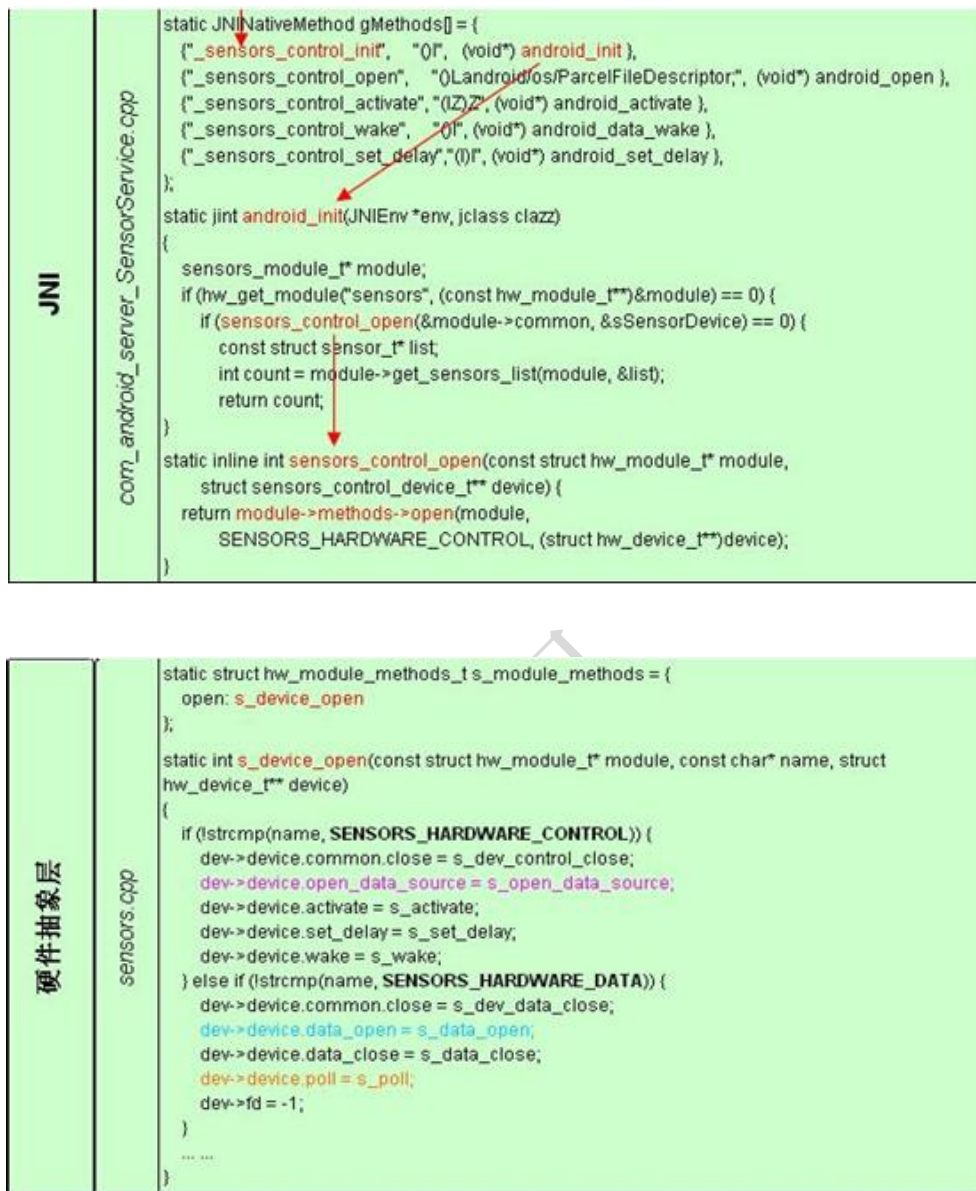
```
private static native void nativeClassInit();

private static native int sensors_module_init();
private static native int sensors_module_get_next_sensor(Sensor sensor, int next);

// Used within this module from outside SensorManager, don't make private
static native int sensors_data_init();
static native int sensors_data_uninit();
static native int sensors_data_open(FileDescriptor[] fds, int[] ints);
static native int sensors_data_close();
static native int sensors_data_poll(float[] values, int[] status, long[] timestamp);
```

同时也实现了各类型传感器数据的处理，提供了 listener 接口，是用户程序操作的核心类。
init 方法和 open 方法之间的调用关系如下图所示。

Framework	SensorService.java <pre>class SensorService extends ISensorService.Stub { ... @SuppressWarnings("unused") public SensorService(Context context) { _sensors_control_init(); } public ParcelFileDescriptor getDataChanel() throws RemoteException { return _sensors_control_open(); } ... _sensors_control_activate(sensor, true); _sensors_control_set_delay(minDelay); _sensors_control_wake(); }</pre>
-----------	--



综上，可以看到，虽然在 Android 系统中，层层包装给代码的阅读与理解带来了一定的麻烦，但从实际角度出发，这样的架构给未来的扩展与更精细的控制带来的好处远大于这些麻烦。

10.5 Logcat 调试

在项目实践中，我们会遇到程序调试的很多问题，特别是在 JNI 层与 HAL Stub 的编写调试中过程，printf 函数族不能输出到终端，使用 logcat 成为了唯一的选择。下面介绍 logcat 的一些典型用法。

V — Verbose (lowest priority)。

D — Debug。

I — Info。

W — Warning。

E — Error。



F — Fatal。

S — Silent (highest priority, on which nothing is ever printed)。

上述是 logcat 的日志等级，优先级从低到高，可以指定 logcat 输出等级以过滤不相干日志。如 logcat *:W 可以指定输出 Warning 以上等级的日志。

在程序中可以设置 TAG，使用 logcat -b TAG 指定输出 TAG，可以观察特定 TAG 代码的执行日志。

可以在环境变量 ANDROID_LOG_TAGS 中添加一个参数来设置默认的过滤地址输入一个参数来设置默认的过滤地址，例如：

```
export ANDROID_LOG_TAGS="ActivityManager:IMyApp:D *:S"
```

日志系统有循环缓冲区，并不是所有的日志系统都有默认循环缓冲区。为了得到日志信息，需要通过 -b 选项来启动 Logcat 。如果要使用循环缓冲区，需要查看剩余的循环缓冲期，命令如下：

radio — 查看缓冲区的相关信息。

events — 查看和事件相关的的缓冲区。

main — 查看主要的日志缓冲区。

下面的例子表示怎么查看日志缓冲区包含 radio 和 telephony 信息：

```
adb logcat -b radio
```

10.6 小结

本章介绍了传感器的原理，以及传感器系统在 Android 下的实现原理和细节，通过具体的 Android 传感器的代码分析，我们会更进一步了解 Android 整体的体系结构及实现细节。



应用部分

华清远见
dev.hqvi.com



第 11 章 Android 应用基本概念

本章介绍 Android 应用基本概念方面的内容，包括 Android 平台特性、Android 系统架构、Android 开发框架以及 OMS 介绍。

11.1 Android 简介

Android 一词的本义指“机器人”，它是由 Google 公司于 2007 年 11 月推出的基于 Linux 平台的开源手机操作系统，依靠 Google 的强大开发和媒体资源，Android 成为众多手机厂商竞相追逐的对象，逐渐发展成为目前最流行的手机开发平台。图 1-1 所示为 Android 系统的 Logo。



图 1-1 Android 系统的 Logo

Android 是一个包括操作系统、中间件、用户界面和关键应用程序的移动设备软件堆。换句话说，Android 是基于 Java 并运行在 Linux 内核上的轻量级操作系统，功能全面，包括一系列 Google 公司在其内置的应用软件，如电话、短信等基本应用功能。图 1-2 所示为 Android 的模拟器，从中可以大概了解 Android 的运行界面。



图 1-2 Android 模拟器



11.2 Android 平台特性

随着科技的发展，移动电话（Mobile Phone）正向着智能化的方向迈步，并逐步成为多种工具的功能载体，而 Android 就是这样一个智能手机的平台，一个多种工具的功能载体。

1. 通信工具

移动电话的最基本功能即为通信，因此，使用运营商提供的通信网络进行语音通话也是 Android 平台的最基本功能。除了传统的语音通话功能外，Android 平台还具有短消息功能，以及通常移动电话都具有的个人信息系统管理方面的功能（如电话本等）。

2. 网络工具

随着数字业务使用的普遍化，移动电话通常作为网络工具使用，这样移动电话可以完成电脑的部分功能。由此，Android 平台在网络方面的功能主要包括浏览器、IM（即时信息）、邮件等，基本包含了网络方面的大部分功能。

3. 媒体播放器

随着多媒体技术的应用，在移动电话上进行音频和视频播放已经成为经常使用的功能。由此，Android 平台具有支持更多的音频/视频格式，支持更高分辨率的视频、更流畅地播放，以及和网络结合的流媒体方面等功能。

4. 媒体获取设备

随着移动电话与媒体获取设备的集成日益增强，Android 平台提供了照相机、录音机、摄像机等功能。

5. 多类型的连接设备

Android 平台提供了多种连接方式，如 USB、GPS、红外、蓝牙、无线局域网等。

6. 具有友好和绚丽的用户界面

Android 平台具有友好的用户界面，使用户容易学习和操作，同时具有绚丽的用户界面，具有良好的视觉效果。



7. 可以个性化定制的平台

Android 平台对于用户的个性化需求，提供了全面自定义手机的功能。

除了以上介绍 Android 平台的功能以外，其在技术上还具有以下几个方面的特性。

全开放智能移动电话平台。

支持多硬件平台。

使用众多的标准化技术。

核心技术完整、统一。

完善的 SDK 和文档。

完善的辅助开发工具。

11.3 Android 开发框架

Android 系统作为一个开放的系统，它体积庞大，对于不同的开发者来说，在开发过程中并不需要掌握整个 Android 系统，只需要进行其中某一部分的开发。由此，从功能上将 Android 开发分为移植开发移动电话系统、Android 应用程序开发，以及 Android 系统开发三种。

从商业模式的角度来讲，移植开发移动电话系统和 Android 应用程序开发是 Android 开发的主流。移植开发移动电话系统主要是由移动电话的制造者来进行开发，其产品主要是 Android 手机；而公司、个人和团体一般进行 Android 应用程序的开发，产生各种各样的 Android 应用程序。

对于 Android 移植开发，其主要工作集中于 Linux 内核中的相关设备驱动程序及 Android 本地框架中的硬件抽象层接口的开发；对于 Android 应用程序开发，其开发的应用程序与 Android 系统的第四个层次的应用程序是一个层次的内容；对于 Android 系统的开发，涉及 Android 系统的各个层次，一般情况下是从底层到上层的整体开发。

Android 开发框架包括基本的应用功能开发、数据存储、网络访问三大块。

11.3.1 应用方面

一般而言，一个标准的 Android 程序包括 Activity、Broadcast Intent Receiver、Service、Content Provider 四部分。

1. Activity

Activity 是在 Android 应用开发中最频繁、最基本的模块。在 Android 中，Activity 类主要与界面资源文件相关联（res/layout 目录下的 xml 资源，也可以不含任何界面资源），包含控件的显示设计、界面交互设计、事件的响应设计以及数据处理设计、导航设计等 Application 设计的方方面面。

因此，对于一个 Activity 来说，它就是手机上的一个界面，相当于一个网页，所不同的是，每个 Activity 运行结束时都返回一个返回值，类似一个函数。Android 系统会自动记录从首页到其他页面的所



有跳转记录并且自动将以前的 Activity 压入系统堆栈，用户可以通过编程的方式删除历史堆栈中的 Activity Instance。

2. Broadcast Intent Receiver

Intent 为不同的 Activity 进行跳转提供了机制，譬如从 A Activity 跳转到 B Activity，使用 Intent 来实现，语句如下。

```
Intent in = new Intent(A.this, B.class);  
startActivity(in);
```

Broadcast Intent Receiver 为各种不同的 Android 应用程序间进行进程间的通信提供了可能。如当电话呼叫来临时，可以通过 Broadcast Intent Receiver 发布广播消息。对用户而言，用户是无法看到 Broadcast Intent Receiver 事件的，它对用户是不透明的，Broadcast Intent Receiver 通过 Notification Manager 来通知用户这些事件发生了，它既可以在资源 AndroidManifest.xml 中注册，也可以在代码中通过 Context.registerReceiver() 进行注册在 AndroidManifest.xml 中注册以后，当事件来临时，即使程序没有启动，系统也会自动启动此应用程序。另外，各应用程序可以很方便地通过 Context.sendBroadcast() 将自己的事件广播给其他应用程序。

3. Service

Android 中的 Service 和 Windows 中的 Service 完全是一个概念，用户可以通过 StartService(Intent service) 启动一个 Service，也可通过 Context.bindService 来绑定一个 Service。

4. Content Provider

Content Provider 提供了应用程序之间数据交换的机制，一个应用程序通过实现一个 Content Provider 的抽象接口将自己的数据暴露出去，并且隐蔽了具体的数据存储实现，这样实现了 Android 应用程序内部数据的保密性。标准的 Content Provider 提供了基本的 CRUD (Create、Read、Update、Delete) 接口，并且实现了权限机制，保护了数据交互的安全性。

一个标准的 Android 应用程序的工程文件包含以下几大部分。

Java 源代码部分（包含 Activity）——放置在 src 目录中。

R.java 文件——由 Eclipse 自动生成与维护，开发者不需要修改，提供了对 Android 资源的全局索引。

Android Library——应用程序运行的 Android 库。

assets 目录——主要用于放置多媒体等文件。

res 目录——放置的是资源文件：drawable 包含的是图片文件，layout 里面包含的是布局文件，



values 里面主要包含的是字符串（strings.xml）、颜色（colors.xml）以及数组（arrays.xml）资源。

AndroidManifest.xml——应用的配置文件，在这个文件中，需要声明所有用到的 Activity、Service、Receiver 等。

11.3.2 数据存储

Android 中提供的存储方式包括：SharedPreferences、文件存储、SQLite 数据库存储方式、内容提供者方式 Content Provider 以及网络方式 5 种，具体介绍如下。

1. SharedPreferences

作为 Android 提供的一种配置文件读/写方式，默认保存在应用的目录 data/<package name>/shared_prefs 下，通过方法 getSharedPreferences(xx, 0);来获取 SharedPreferences 对象进行读/写操作。

2. 文件存储

Android 系统提供了进行数据读/写访问的 API，例如，openFileInput、openFileOutput 等，需要特别注意的是，Android 中应用程序的数据是私有的，也就是说，其他应用程序无法访问当前应用程序产生的文件。

3. SQLite 数据库存储方式

通过继承 SQLiteOpenHelper 类提供的 CRUD 接口来进行数据库操作，方便了应用程序的数据存储操作。

4. 内容提供者方式（Content Provider）

通过调用其他应用程序的数据接口来实现数据的读/写访问。

5. 网络方式

主要通过下面要提到的网络访问该网络提供的网络服务接口，实现数据的读/写服务（如 Webservice 数据访问接口）。

11.3.3 网络访问方面

Android 主要通过 java.net.*及 Android.net.*来进行 HTTP 访问技术的封装；利用其提供的 HttpPost、

11.3.4 开发流程

软件开发生命周期

```

graph TD
    A((需求)) --> B((设计))
    B --> C((实现))
    C --> D((测试))
    D --> E((运维))
    E --> A
  
```

- 142 -



测试经理及测试人员：测试经理主要负责计划和组织测试人员对目标产品进行测试，发现 bug、跟踪 bug，直到解决 bug；测试人员则根据测试经理的计划和测试总体方案对目标产品进行测试，编写测试用例和测试代码，发现和跟踪 bug；编写用户手册；进行用户培训和教育。

开发人员：根据设计师的设计成果进行具体编码工作，对自己的代码进行基本的单元测试。

项目实施人员：负责软件系统安装配置、系统割接、运行期间的维护工作。

11.4 本章小结

本章从介绍 Android 基本概念开始，让读者了解了 Android 平台特性、Android 系统架构、Android 开发框架，以及 OMS 等知识，使读者可以对 Android 的应用前景有一个很好的认识。

华清远见
dev.havi.com



第 12 章 Android 应用开发环境搭建

12.1 Android 开发环境的安装与配置

Android 应用软件开发需要的开发环境在光盘路径【华清远见开发环境：Android 应用开发环境】下：

JDK：JDK\JDK8\jdk-8u5-windows-i586.exe（32bit）或者 jdk-8u5-windows-x64.exe（64bit）

（从 JDK 8.0 开始不支持 Windows XP 操作系统，使用 Windows XP 的用户可以使用 JDK7 目录下的内容）

ADT：adt-bundle-windows-x86.7z（32bit）或者 adt-bundle-windows-x86_64.7z（64bit）

以下主要介绍在 Windows 环境下搭建 Android 开发环境的步骤和注意事项。

12.1.1 安装 JDK 和配置 Java 开发环境

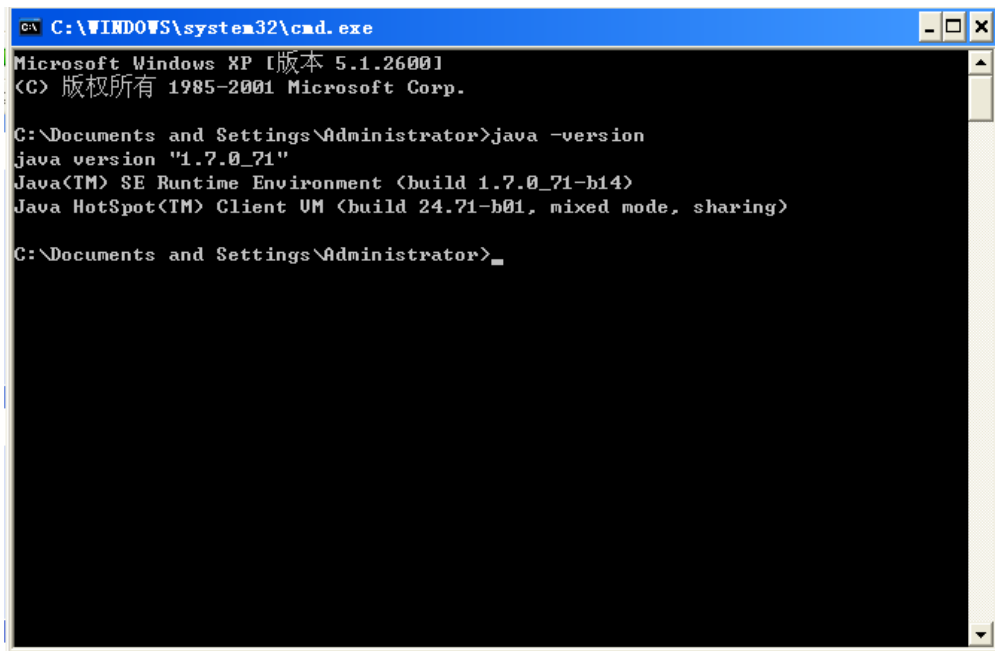
双击 JDK\JDK8\jdk-8u5-windows-i586.exe（32bit 操作系统）或者 jdk-8u5-windows-x64.exe（64bit 操作系统）进行安装（从 JDK 8.0 开始不支持 Windows XP 操作系统，使用 Windows XP 的用户可以使用 JDK7 目录下的内容选择代替 JDK8 目录下的内容）。接受许可证，选择需要安装的组件和安装路径后，单击“下一步”按钮，完成安装过程。



安装完成后，利用以下步骤检查安装是否成功：打开 Windows CMD 窗口，在 CMD 窗口中输入 java -version 命令，如果屏幕出现下图所示的代码信息，说明 JDK 安装成功。



XP 下安装 JDK7 如下：

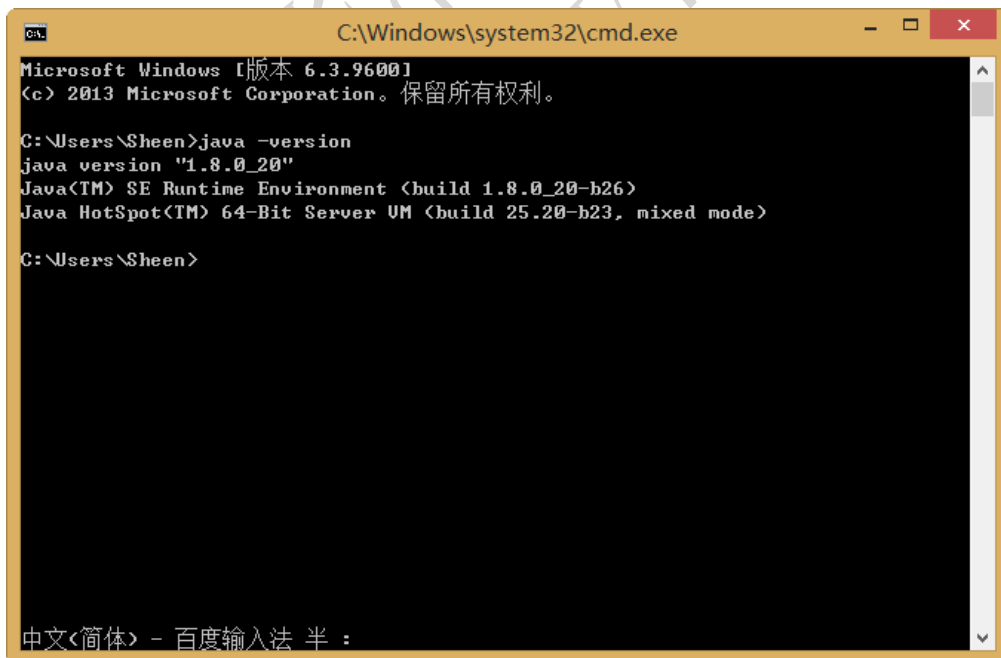


```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows XP [版本 5.1.2600]
(C) 版权所有 1985-2001 Microsoft Corp.

C:\Documents and Settings\Administrator>java -version
java version "1.7.0_71"
Java(TM) SE Runtime Environment (build 1.7.0_71-b14)
Java HotSpot(TM) Client VM (build 24.71-b01, mixed mode, sharing)

C:\Documents and Settings\Administrator>
```

非 XP 下安装 JDK8 如下：



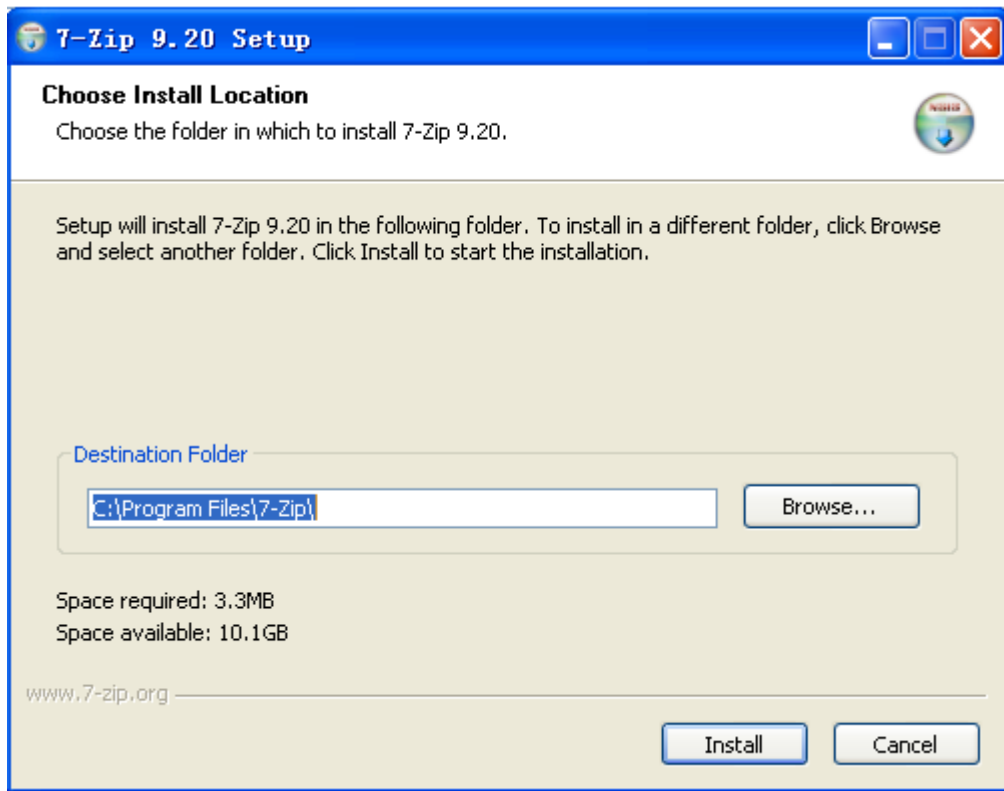
```
C:\Windows\system32\cmd.exe
Microsoft Windows [版本 6.3.9600]
(c) 2013 Microsoft Corporation。保留所有权利。

C:\Users\Sheen>java -version
java version "1.8.0_20"
Java(TM) SE Runtime Environment (build 1.8.0_20-b26)
Java HotSpot(TM) 64-Bit Server VM (build 25.20-b23, mixed mode)

C:\Users\Sheen>
```

12.1.2 安装 7Zip 压缩软件

双击光盘\7zip\7z920-x86.exe（32bit 操作系统）或者 7z920-x64.msi（64bit 操作系统）进行安装。

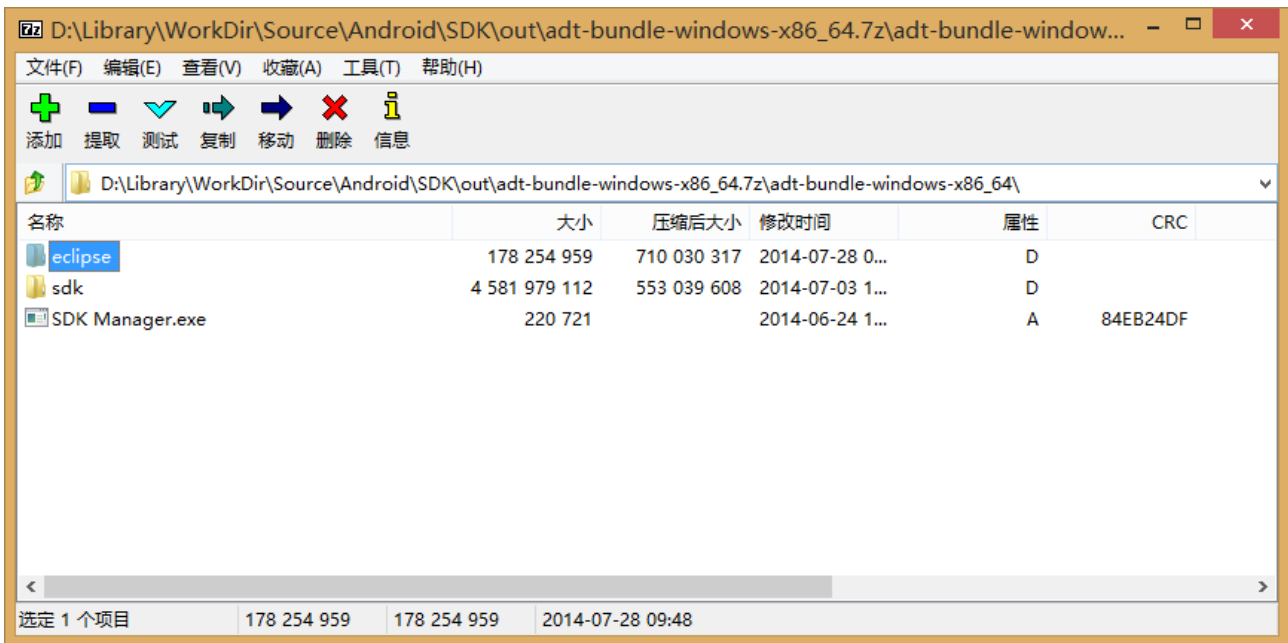


12.1.3 解压 **adt-bundle-windows**

JDK 安装成功后，使用 7Zip 软件解压 ADT 目录下的 **adt-bundle-windows-x86.7z**（32bit）或者 **adt-bundle-windows-x86_64.7z**（64bit）。

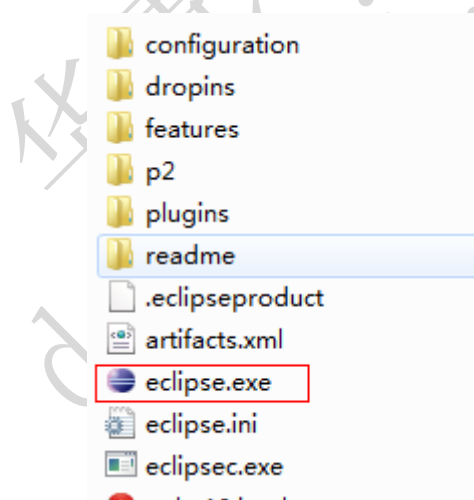
注意 1：必须使用光盘下的 7Zip 软件解压本镜像，否则解压可能会出错；

注意 2：解压路径不包含中文；



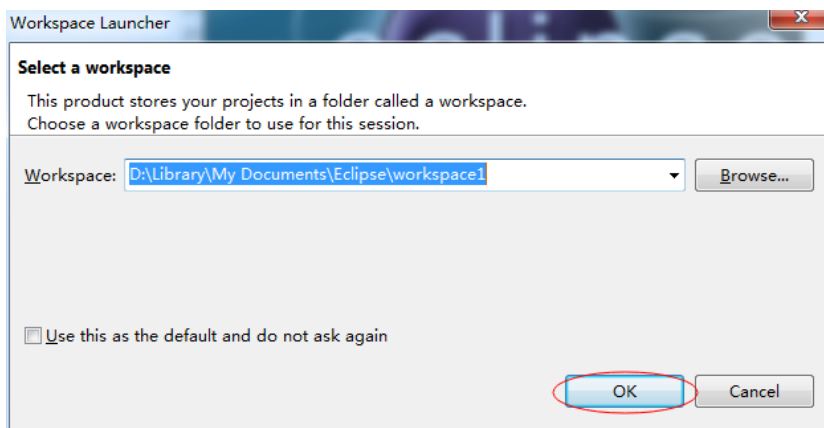
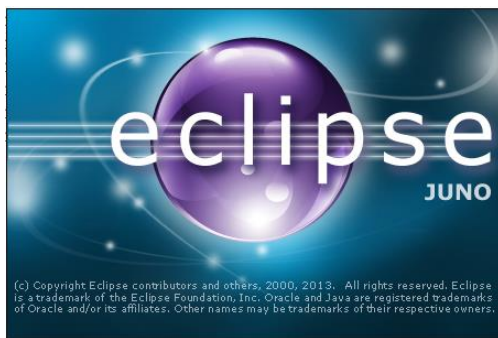
12.1.4 运行 Eclipse

解压完毕后，直接执行其中的 eclipse\eclipse.exe 文件，Eclipse 可以自动找到用户前期安装的 JDK 路径。

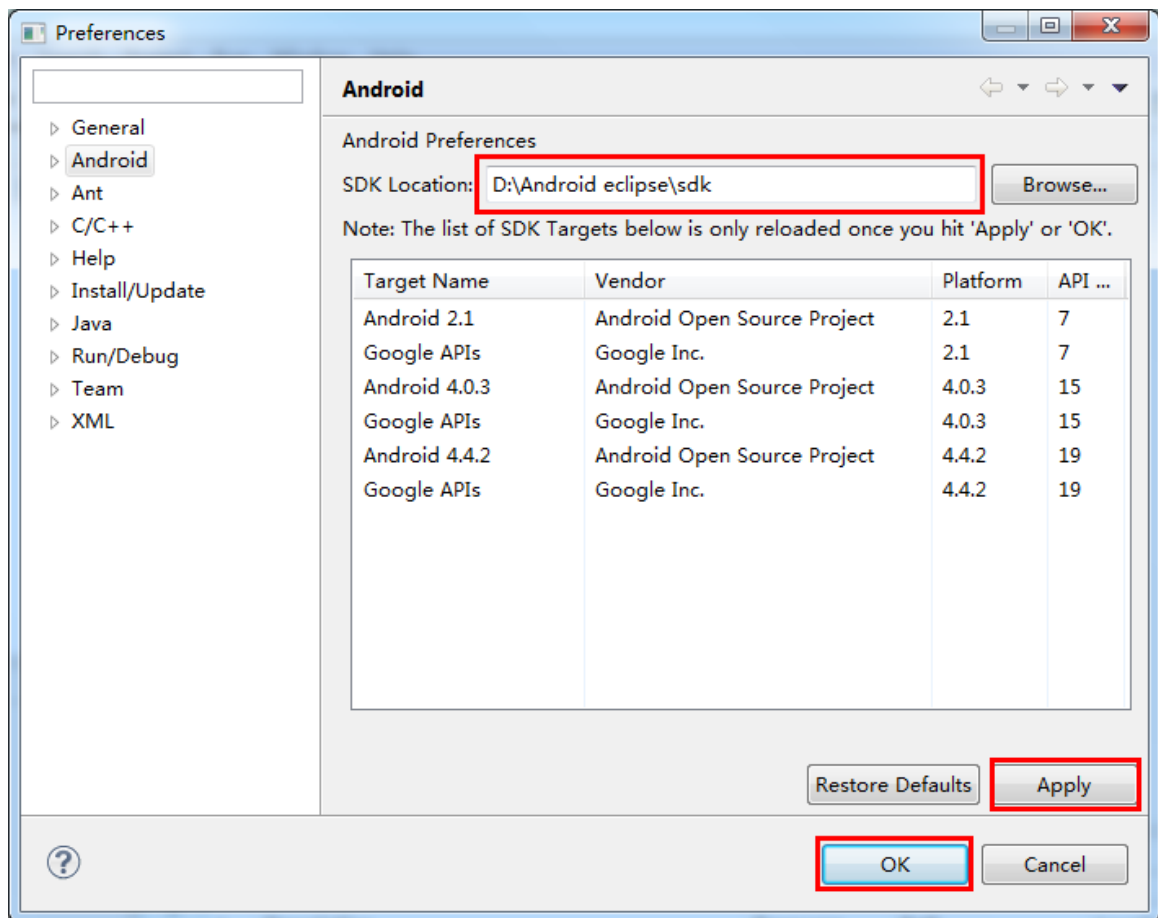


12.1.5 配置 SDK

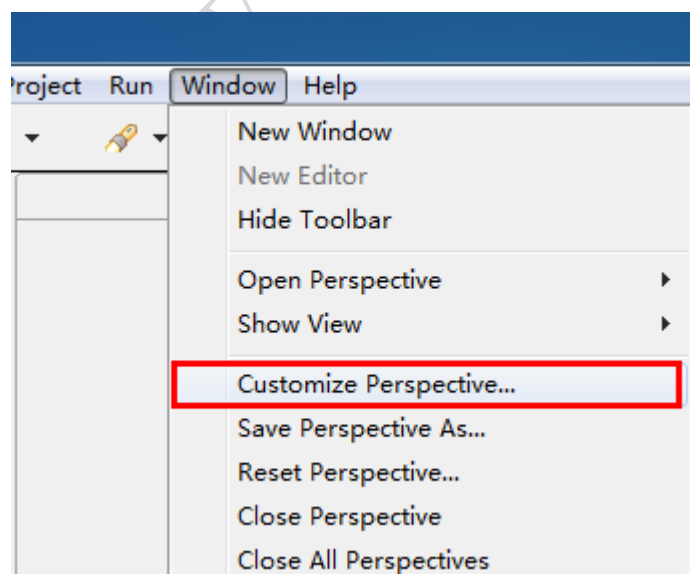
运行解压目录下的 eclipse\eclipse.exe，为自己选择一个工作目录 Workspace，不要有中文路径，不选择默认也可以。



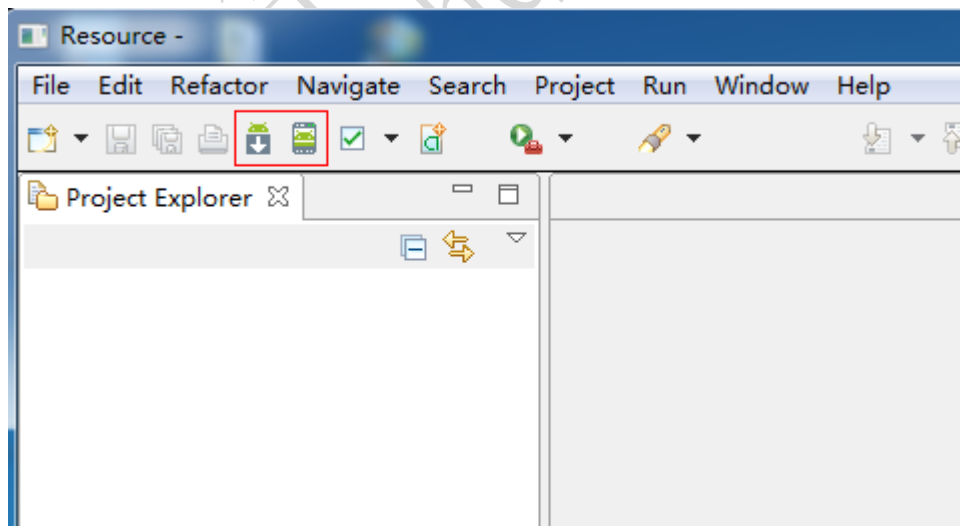
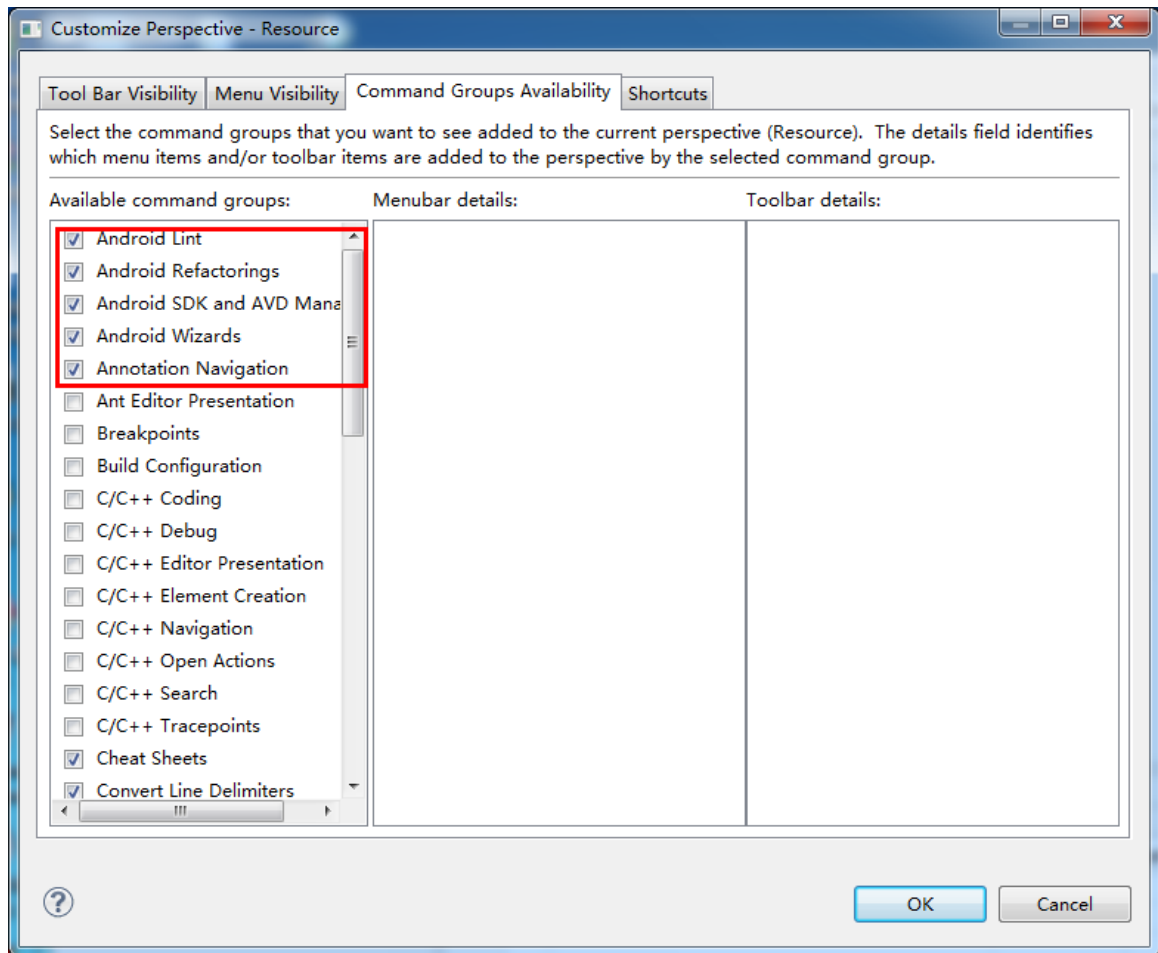
需要为 Eclipse 关联 SDK 的安装路径，即解压路径下的 sdk 目录。在 Eclipse 中，点击 Window->Preferences，会看到其中添加了 Android 的配置，按图所示的操作，然后点击 Apply，最后点击 OK 即可。



完成以上步骤后，设置 Eclipse 环境



勾选 Android 相关的工具，点击 OK（如果已经勾选，则不理睬）。

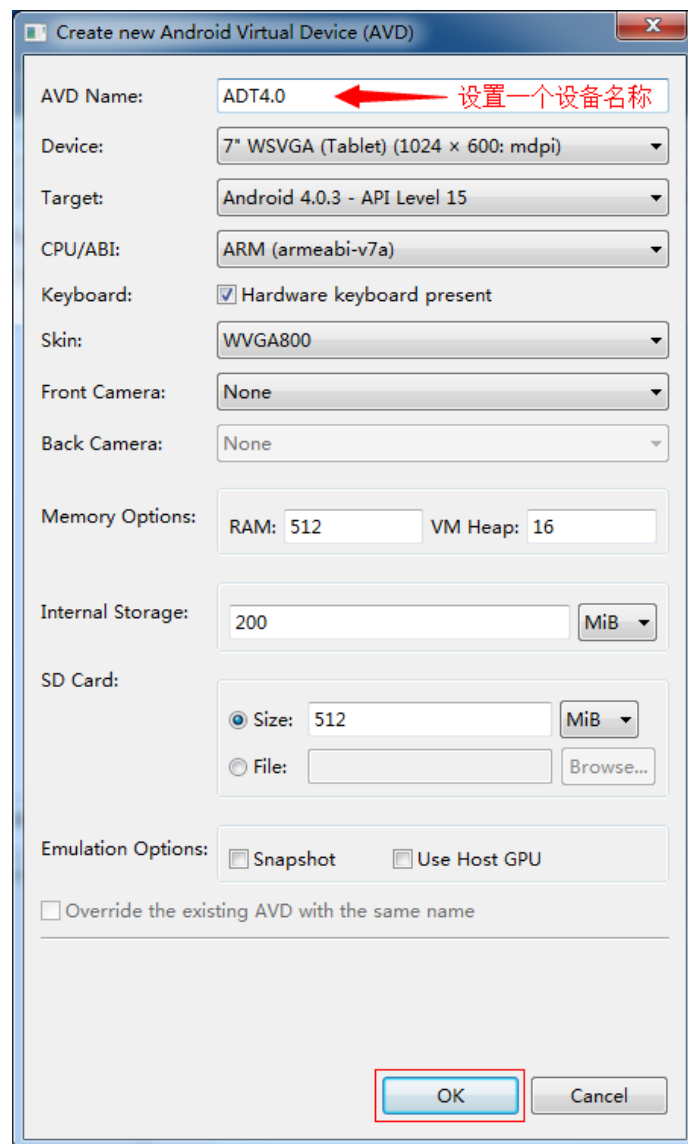
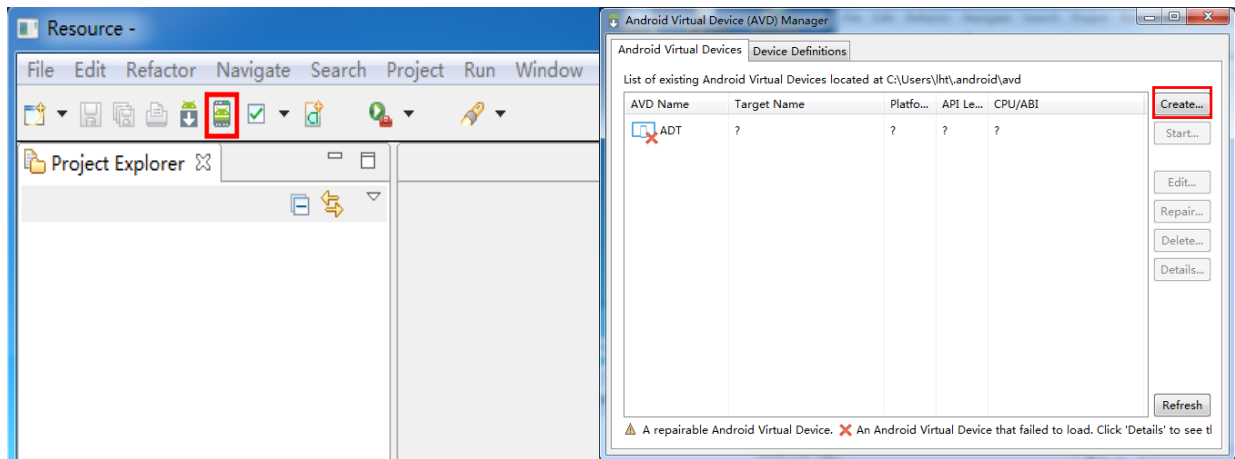


12.1.6 配置 ADT

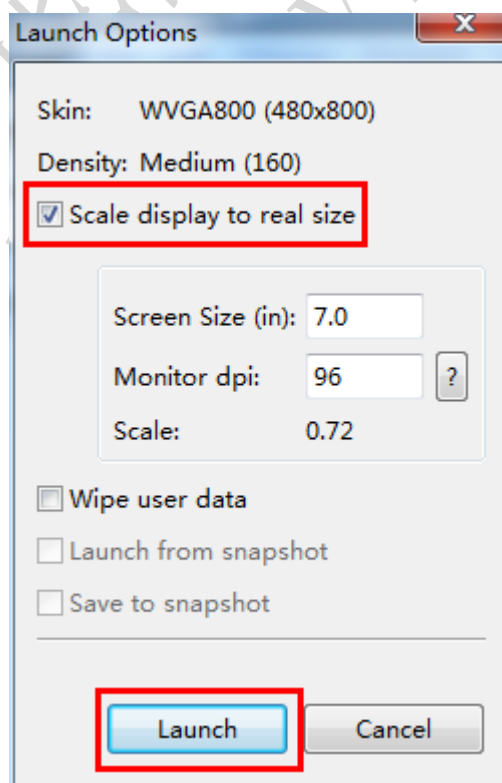
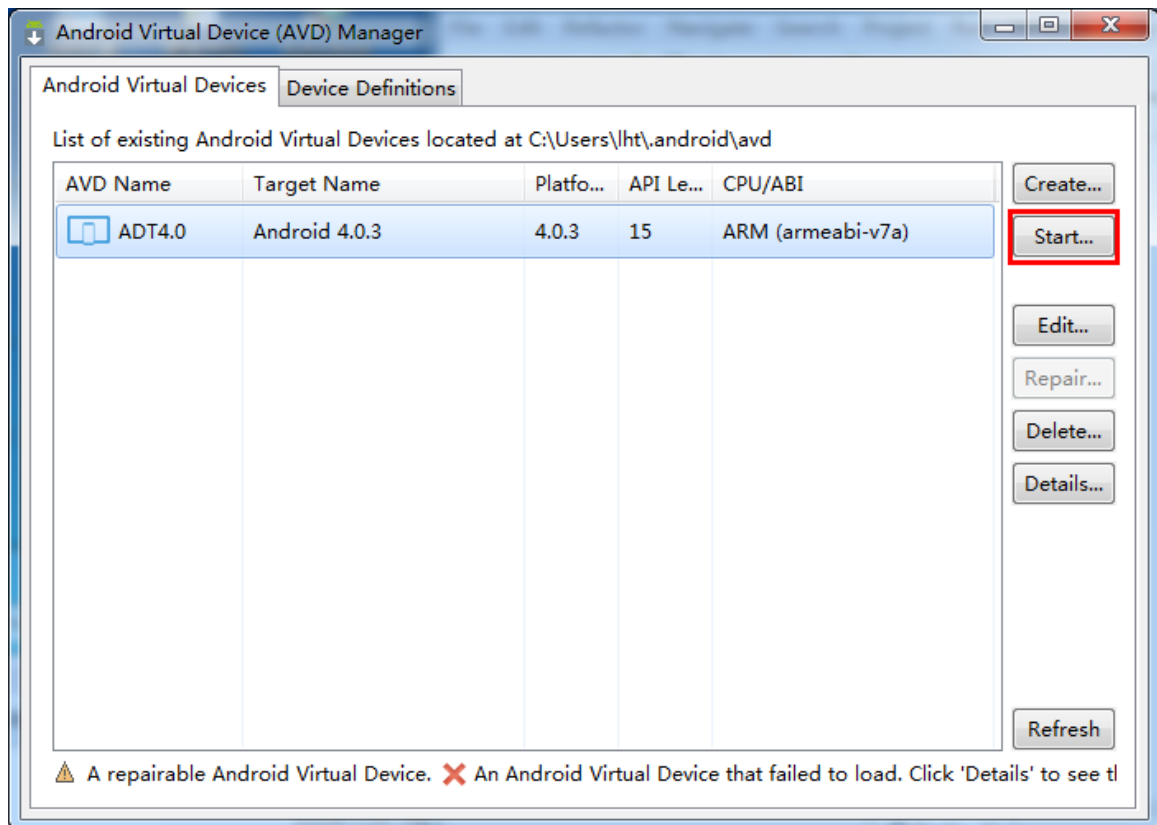
ADT (Android Development Tools) 是 Android 为 Eclipse 定制的一个插件，为用户提供了一个强大的用户开发 Android 应用程序的综合环境。ADT 扩展了 Eclipse 的功能，可以让用户快速地建立 Android 项目，创建应用程序界面，在基于 Android 框架 API 的基础上添加组件，以及用 SDK 工具集调试应用程序，甚至导出签名（或未签名）的 APKs 以便发行应用程序。



在 Eclipse 中安装 ADT，首先在启动 Eclipse，点击 “Android Virtual Device (AVD) Manager” 图标。

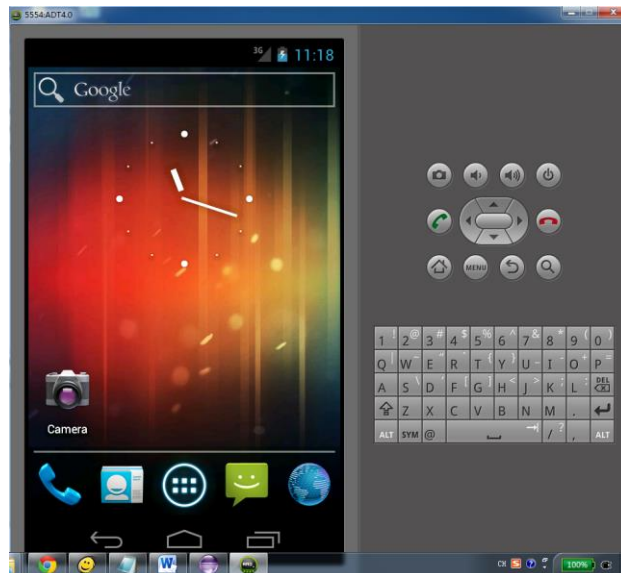


测试运行 AVD。





虚拟设备启动完成后，如下图：



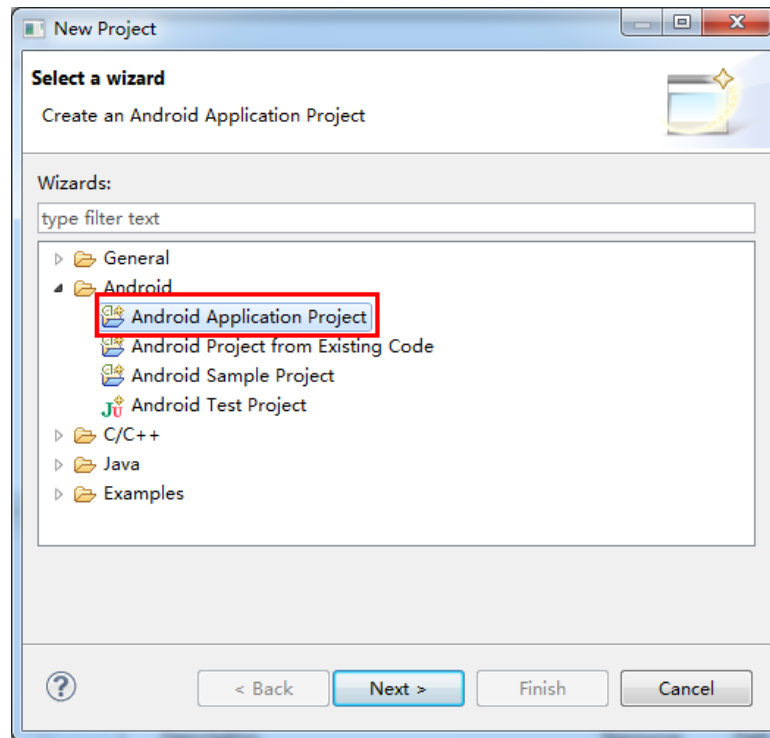
注意：虚拟机启动速度较慢，如果用虚拟机调试程序，可以保持虚拟机打开状态

12.2 创建第一个 Android 应用

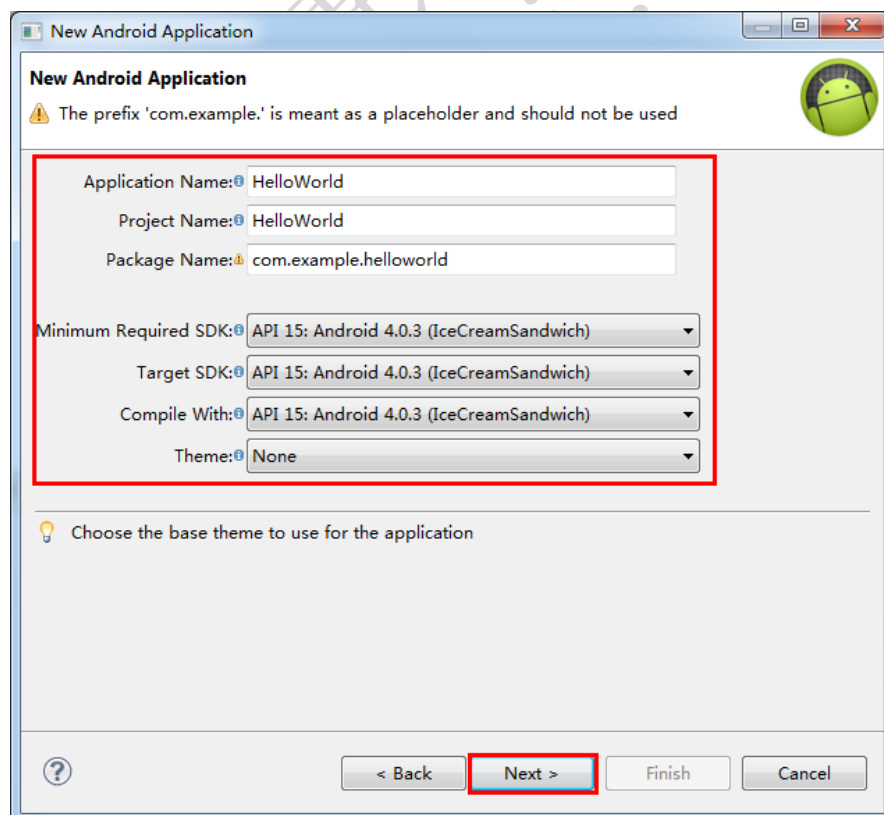
Android 的 SDK 环境安装完成后，就可以在 SDK 中建立工程并进行调试了。

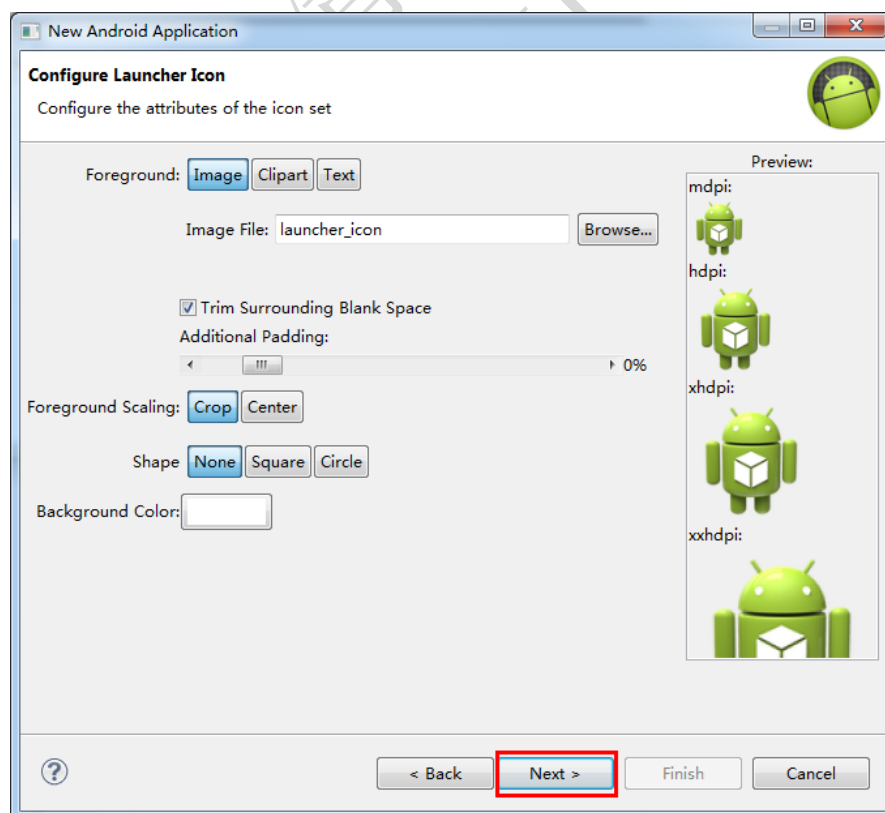
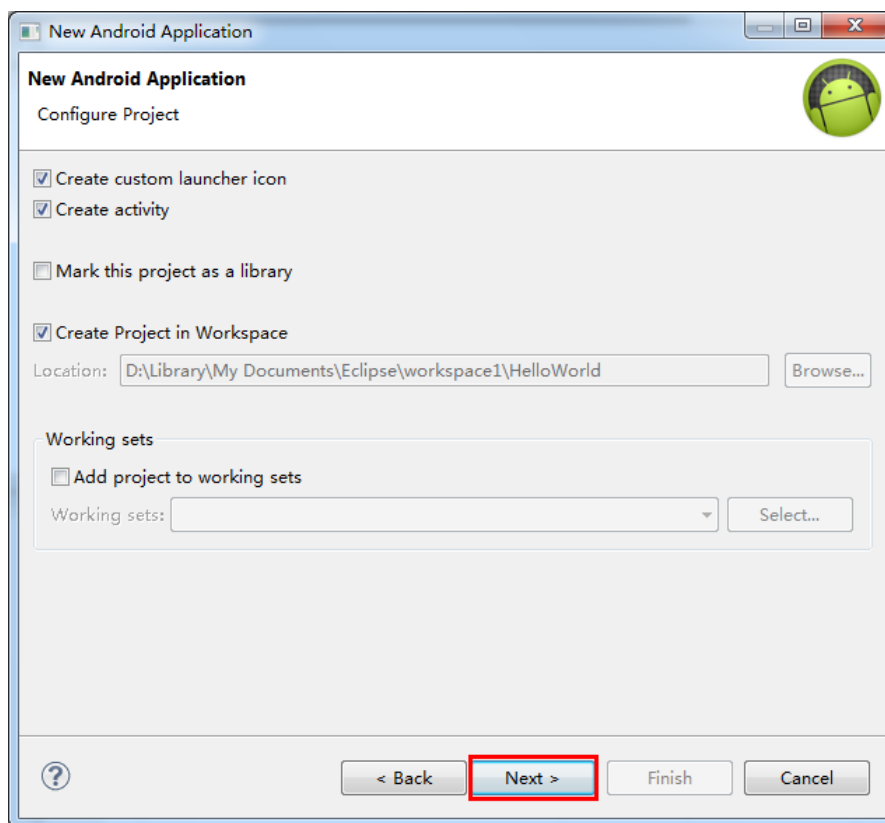
建立 Android 工程的步骤如下：

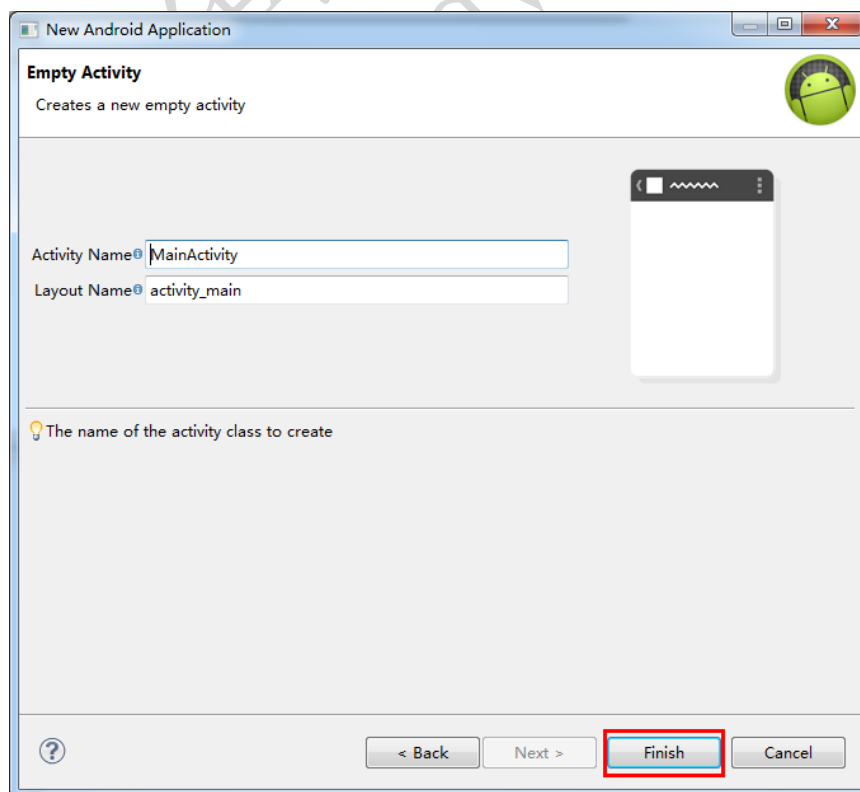
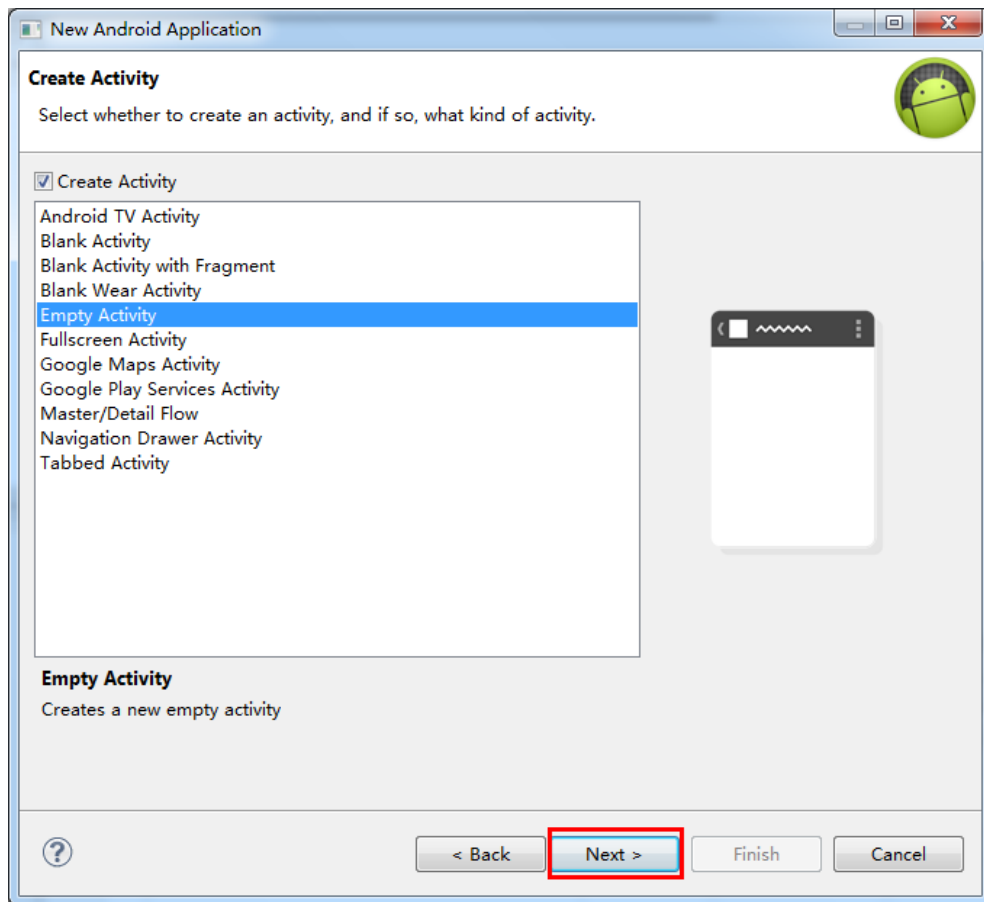
- (1) 选择 “File” → “New” → “Project” 命令。
- (2) 选择 “Android” → “Android Application Project” 命令，单击 “Next” 按钮，如图 2-6 所示。



(3) 在“New Android Application”对话框中，输入项目名称等信息，最后单击“Next”按钮。



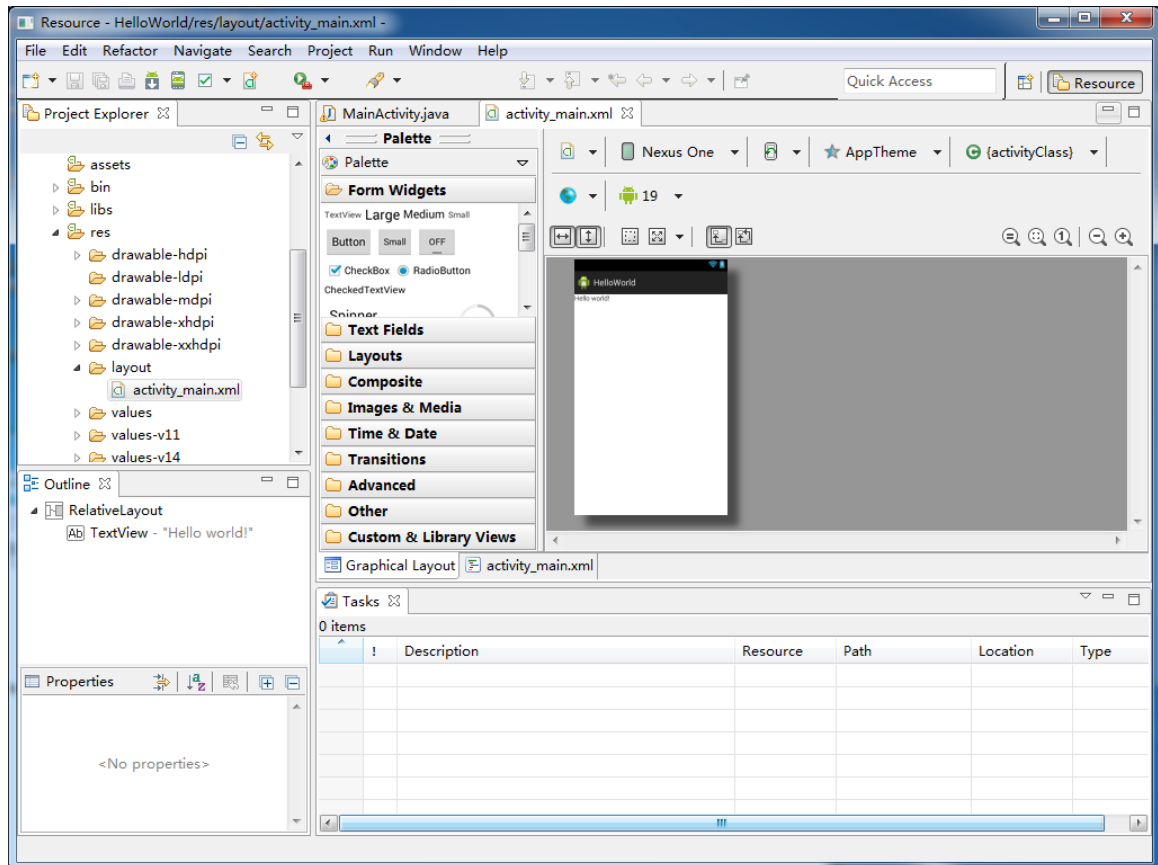




(4) 工程建立后，可以通过 Eclipse 环境查看 Android 应用程序中各个文件，例如

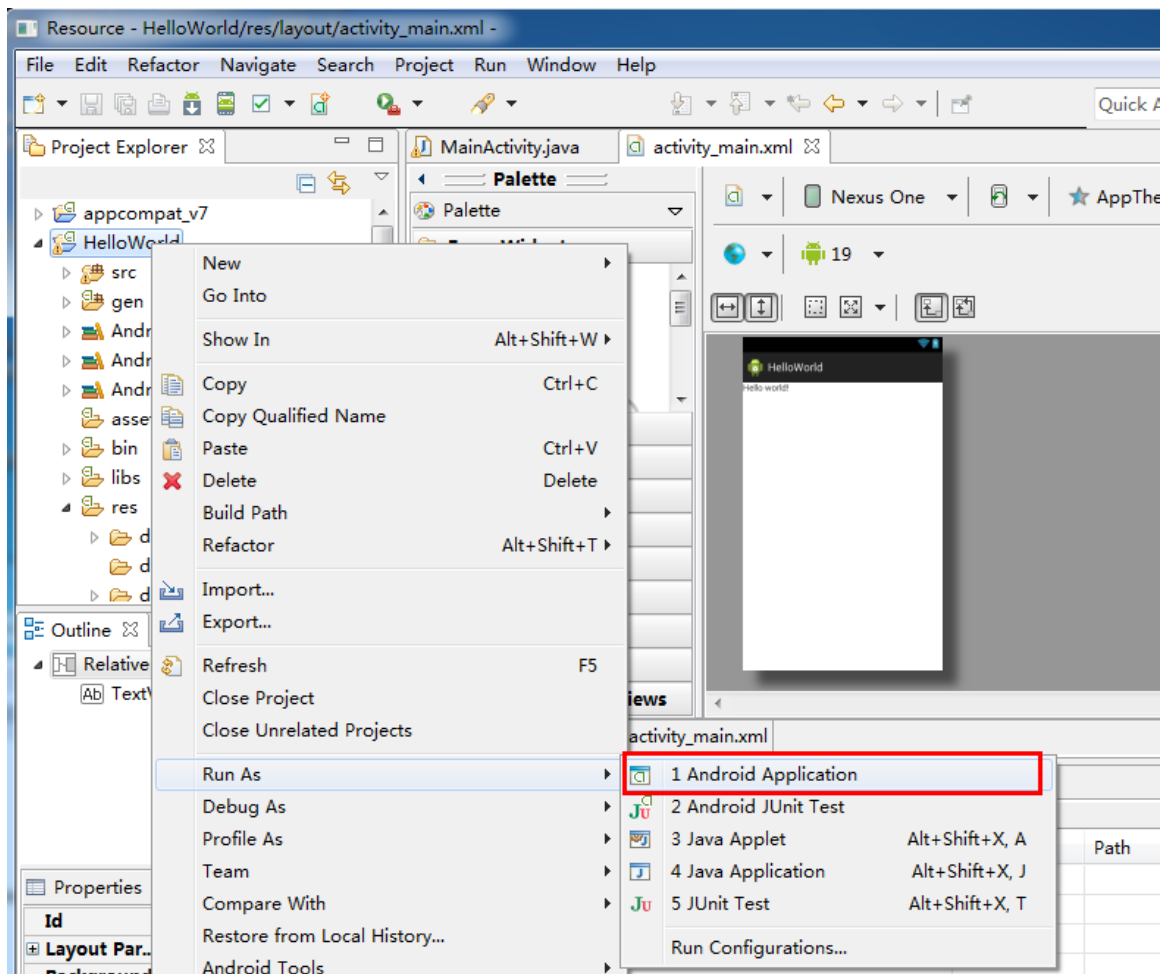


AndroidManifest.xml 文件、布局文件、代码等。如图 2-8 所示为布局文件的编辑界面，可以直观地查看程序的 UI 布局。

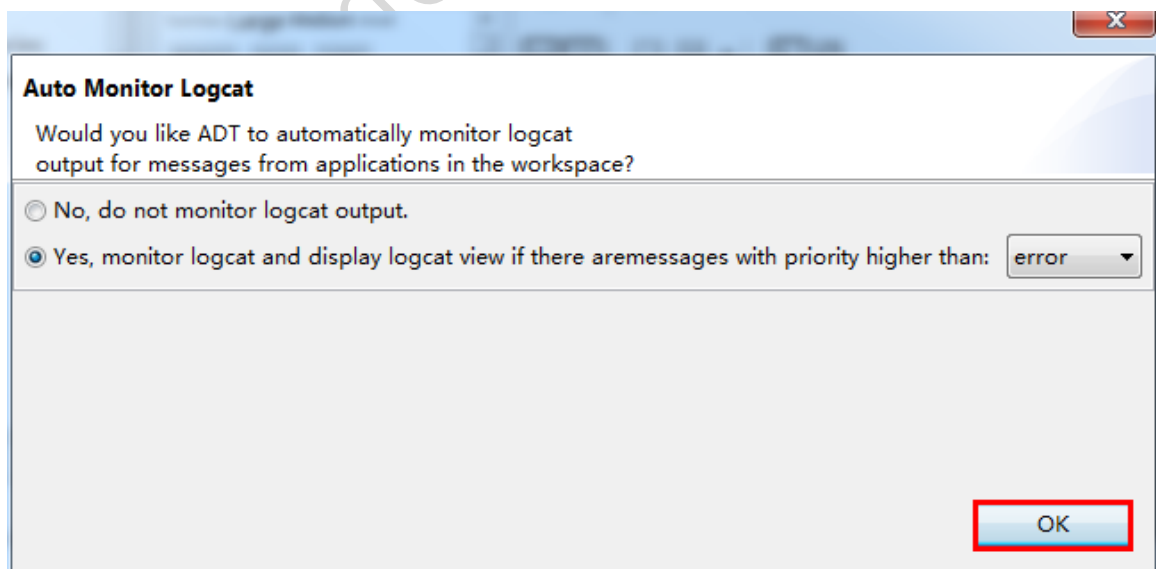


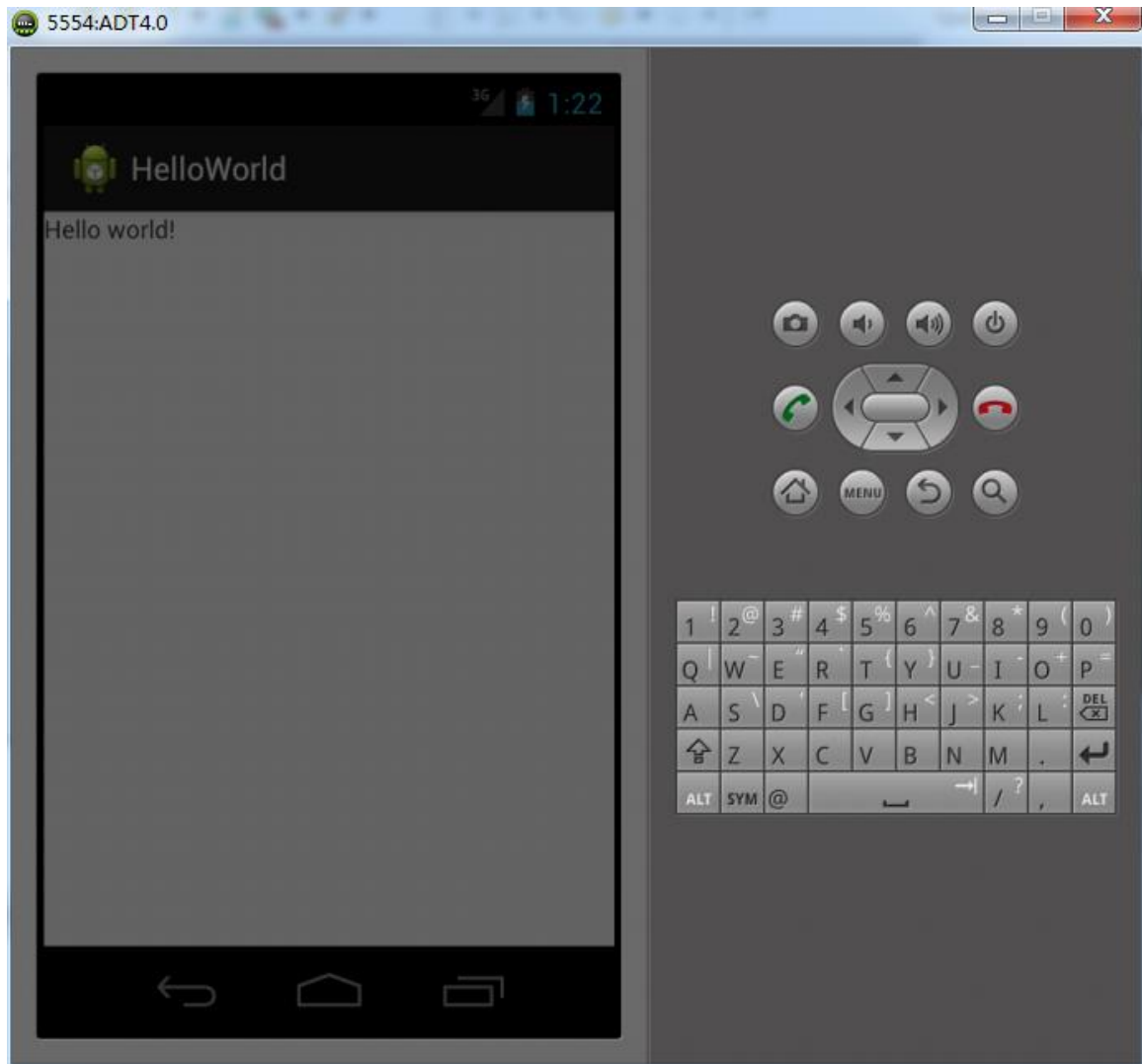
12.3 在模拟器上运行程序

右键选择 HelloWorld 工程，然后选择“Run As”，最后选择“Android Application”。



Eclipse 将打开刚才建立的默认的 Android 模拟器，运行画面如同真的手机开机一般，开机后，随即打开运行的程序，运行画面如下图所示。





12.4 在目标开发平台上运行程序

注意：如果在调试开发板的时候，出现 ADB 连接不上的问题（已知华清远见 FSPAD723 开源平板），可以试着替换 Android SDK 的 ADB 工具（把光盘\Android 应用开发环境\ADB\ADB1.0.26\下的 4 个文件拷贝到用户 ADT 解压目录下的 sdk\platform-tools 中）

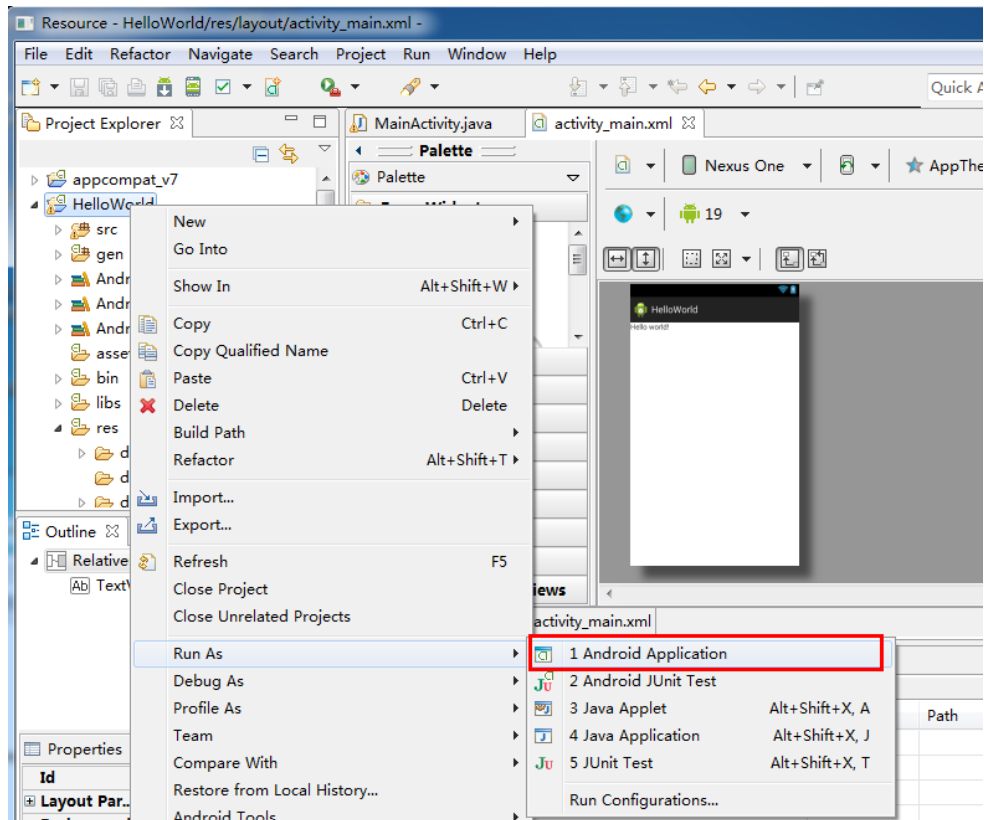
组织	新建	打开	选择	
WorkDir > Source > Android > SDK > adt-bundle-windows-x86_64 > sdk > platform-tools				
名称	修改日期	类型	大小	
api	2014/6/21 周六 ...	文件夹		
systrace	2014/6/21 周六 ...	文件夹		
adb.exe	2014/6/21 周六 ...	应用程序	888 KB	
AdbWinApi.dll	2014/6/21 周六 ...	应用程序扩展	94 KB	
AdbWinUsbApi.dll	2014/6/21 周六 ...	应用程序扩展	60 KB	
dmtracedump.exe	2014/6/21 周六 ...	应用程序	62 KB	
etc1tool.exe	2014/6/21 周六 ...	应用程序	291 KB	
fastboot.exe	2014/6/21 周六 ...	应用程序	165 KB	
hprof-conv.exe	2014/6/21 周六 ...	应用程序	29 KB	
NOTICE.txt	2014/6/21 周六 ...	文本文档	704 KB	
source.properties	2014/6/21 周六 ...	PROPERTIES 文件	1 KB	

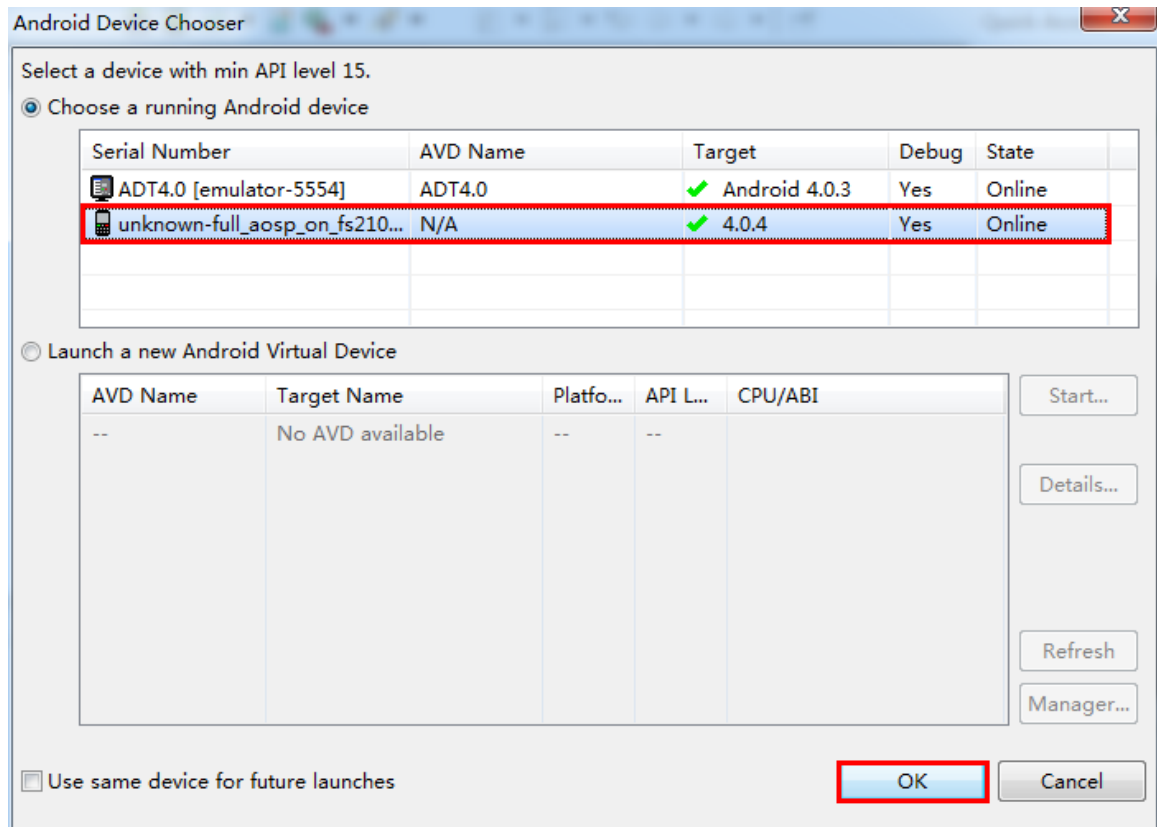
开发期间，在实际的设备上运行 Android 程序与在模拟器上运行该程序的效果几乎相同，需要做的就



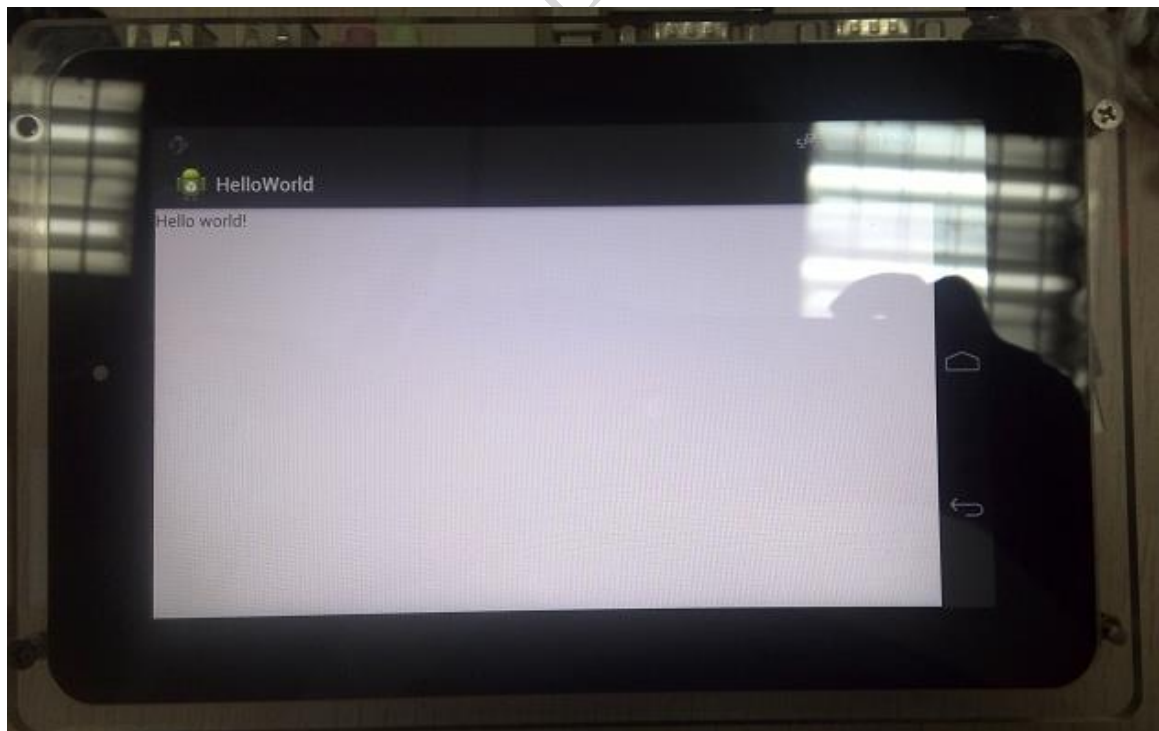
是用 USB 电缆连接手机与计算机，并安装一个对应的设备驱动程序。如果模拟器窗口已打开，请将其关闭。只要将开发平台（正常启动 Android 系统状态，如果平台不能正常启动 Android，请参考镜像烧写章节）通过 USB 下载线与计算机相连，应用程序就会在开发平台上加载并运行。本例使用华清远见 FS6818 开发板作为目标设备，用户可以根据自己的设备参考选择。

在 Eclipse 中选择“Run” → “Run”（或 Debug）命令，这时会弹出一个窗口（见图 2-13），让你选择用模拟器还是手机来显示，如果选择手机，即可在手机上运行该程序。





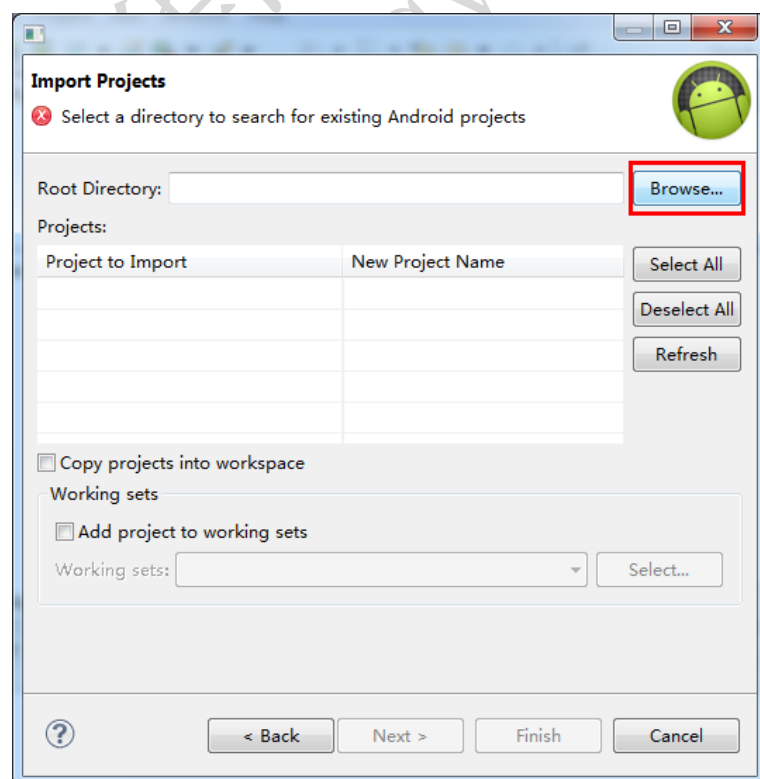
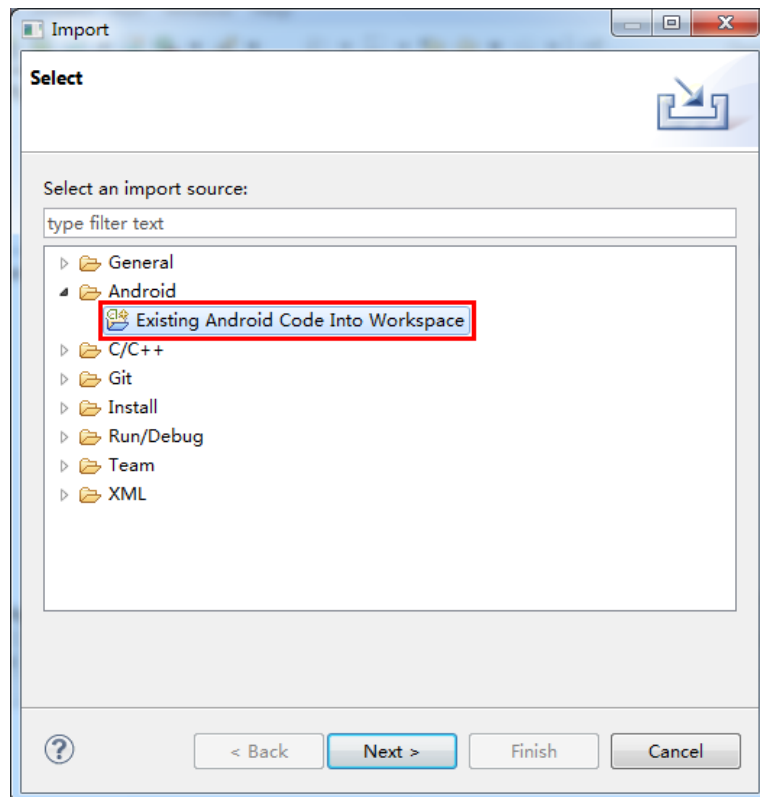
选择 FS6818 设备，点击“OK”，程序运行在开发平台显示屏上。（偶尔遇到错误时，可以试着关闭 Eclipse，再重新打开测试。）





12.5 导入一个已有工程

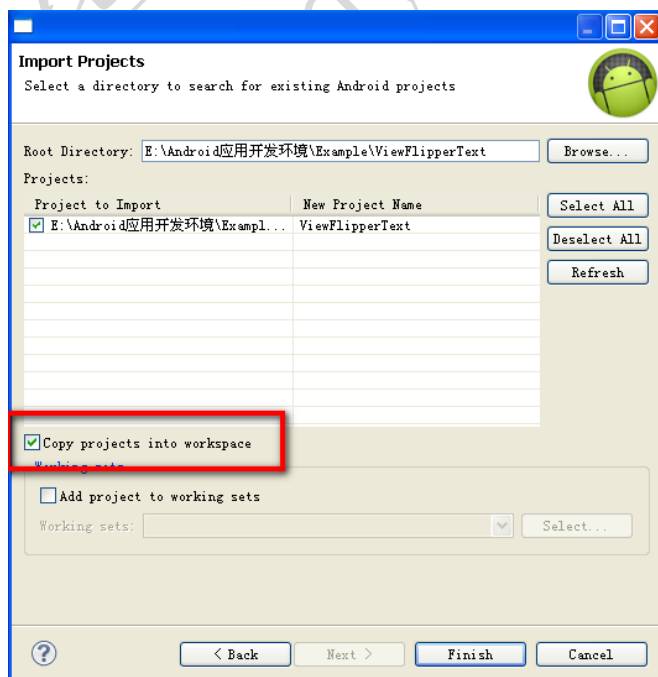
打开 Eclipse 环境，选择 File->Import。



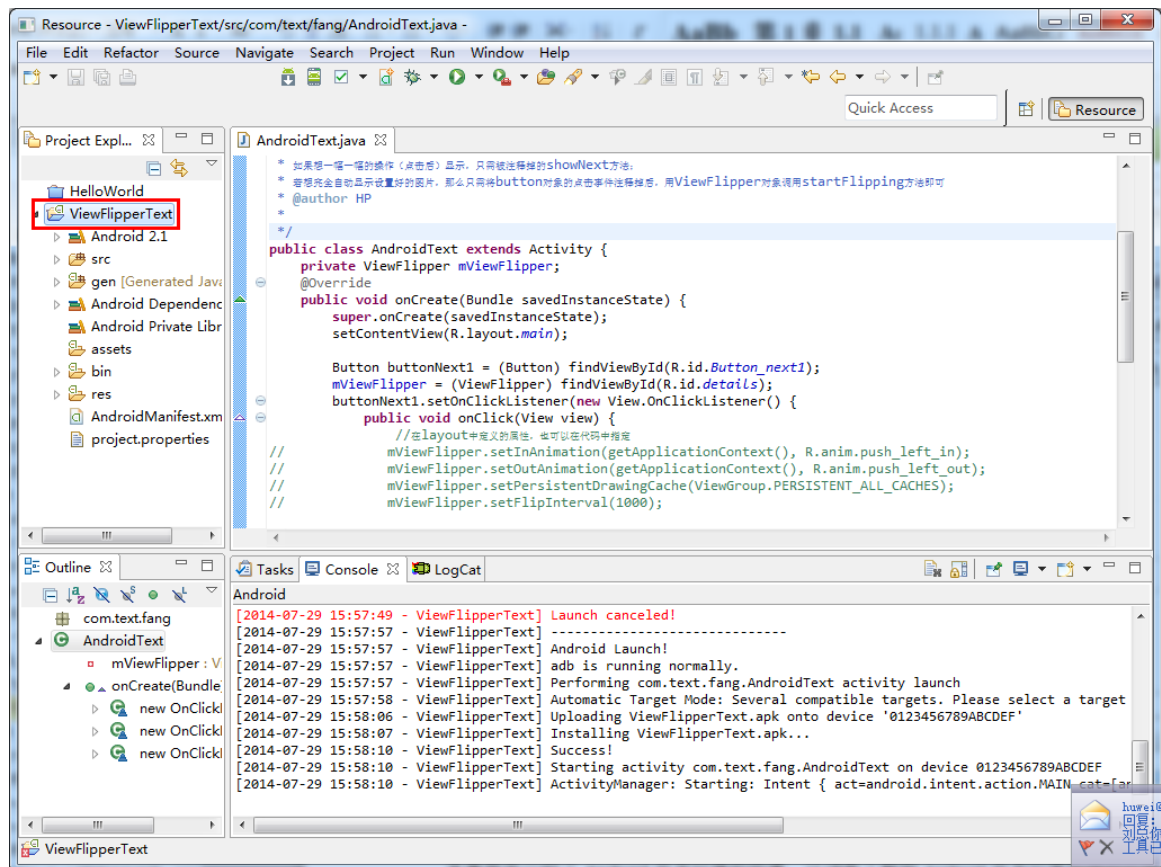
例如：导入光盘【华清远见开发环境：Android 应用开发环境\Example\ViewFlipperText】。



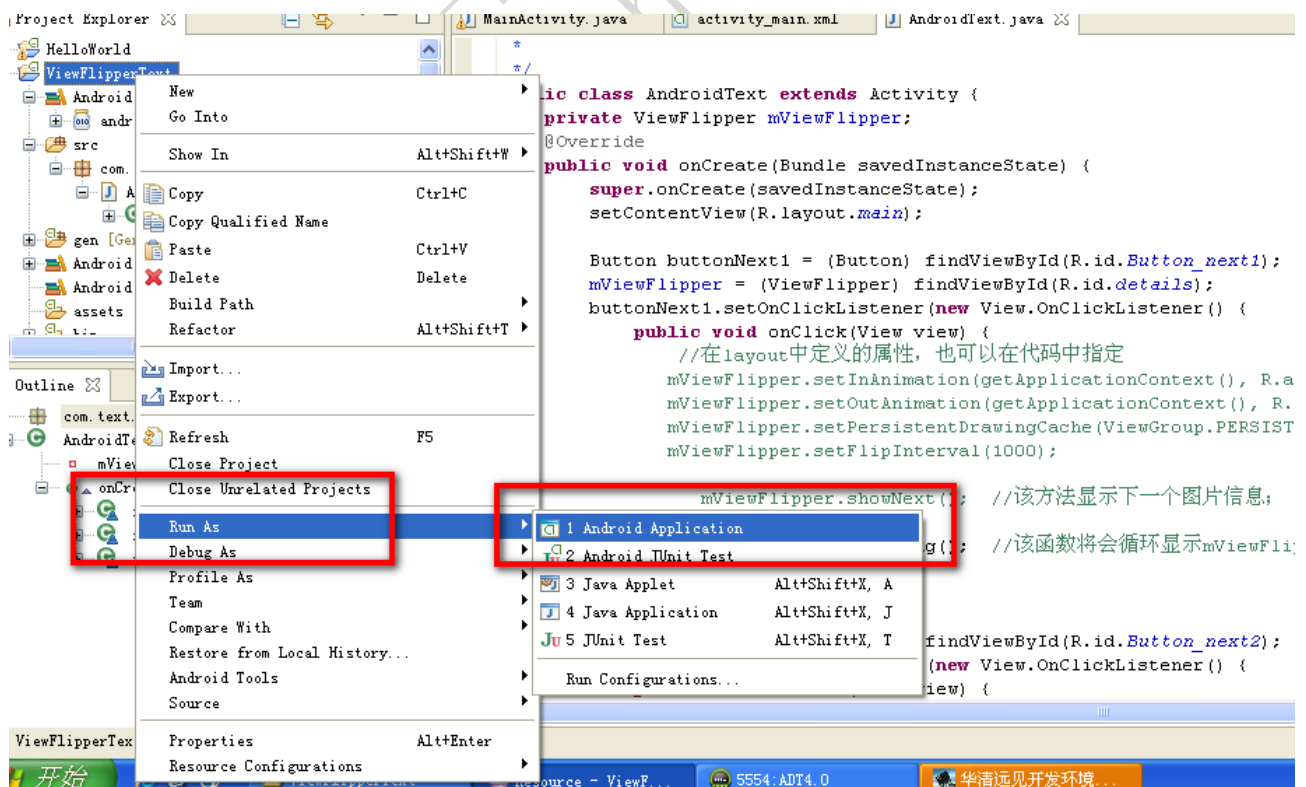
建议把下图红色框内勾选上。

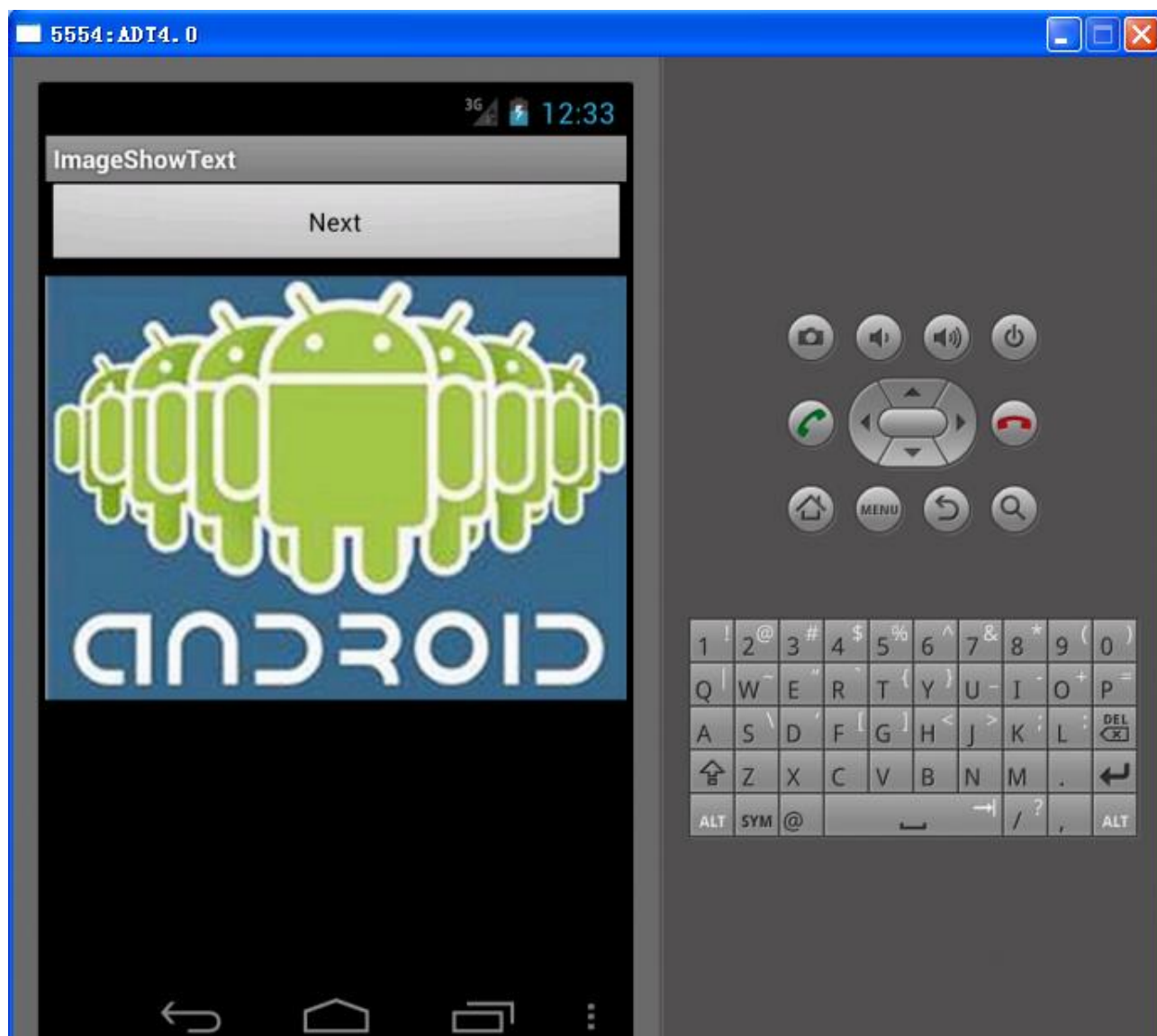


点击 Finish 完成。



尝试运行一下测试程序。

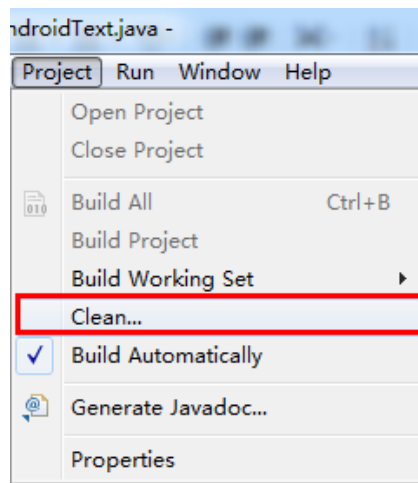




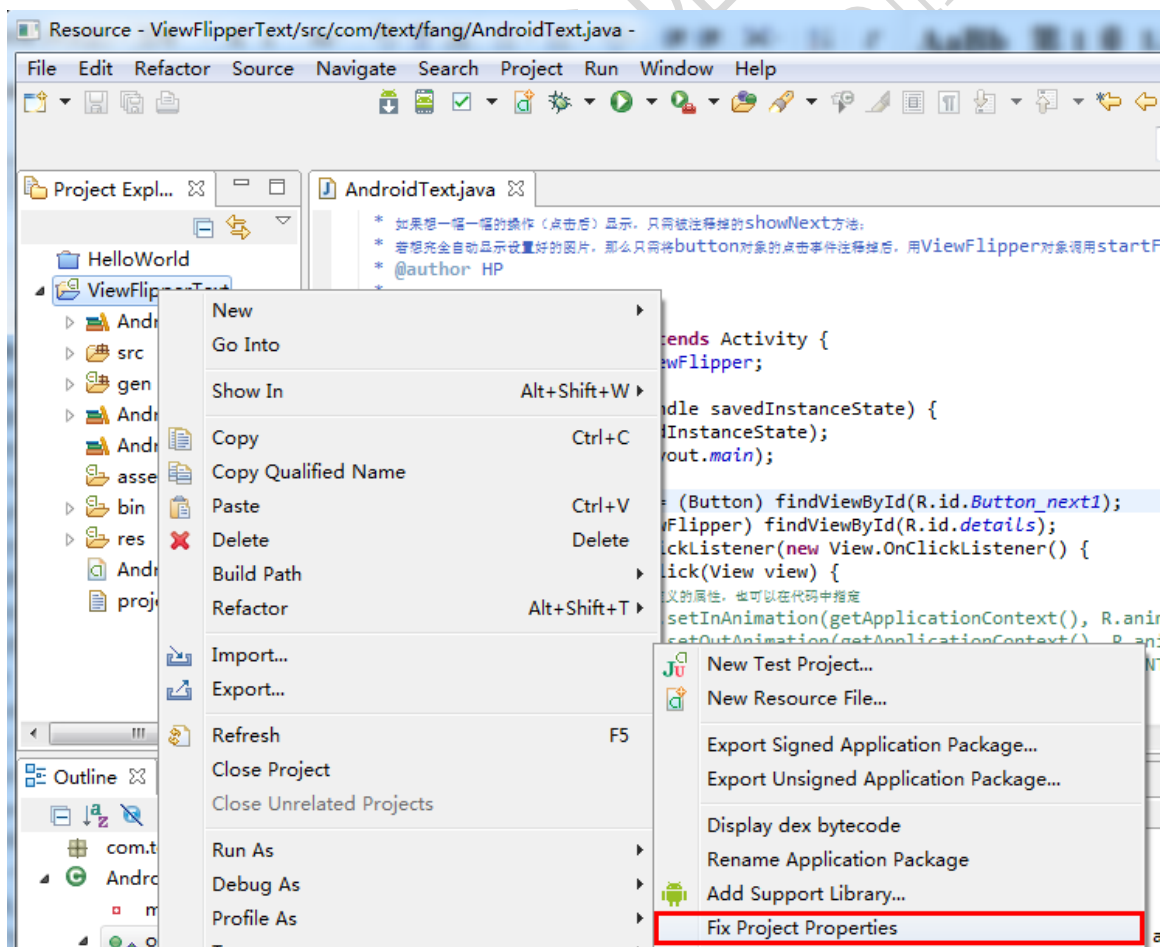


12.6 可能遇到的错误及解决方法

- 1、如果错误提示是 R 文件有问题，可以 clean 一次工程。



- 2、如果有其它文件错误，可以 Fix 一下错误



- 3、如果还是解决不了，可以重新启动 Eclipse 软件。



12.7 本章小结

本章首先介绍了 Android 开发环境的搭建，让读者了解到 Android 开发环境搭建的步骤及注意事项，同时介绍了 Android 应用程序开发的过程和步骤，使读者可以对 Android 的应用程序开发有一个很好的认识。

华清远见
dev.havi.com



第 13 章 程序设计基础

在上一章的学习中，主要了解了 Eclipse+ADT 的开发流程，对其有了初步的认识和了解。对初学者来说，这一章的内容比较烦琐，但是又必须掌握，这也是进行 Android 开发必须经过的第一步，有了这个基础，我们将进行 Android 应用程序设计。

13.1 Android 程序框架

上一章我们建立了 AndroidTest 项目，在项目中，所有的代码是由 ADT 插件自动生成的，我们并没有对其进行编码，所以没有对其框架进行分析。其实每一个平台都有自己的结构框架，比如，在最初学习 Java 或者 C/C++ 时，第一个程序总是 main 方法，以及文件类型和存储方式等。本节将对 Android 平台的目录结构、文件类型及其负责的功能和 Android 平台的 main 方法进行剖析。

13.1.1 Android 项目目录结构

有了前两章的基础，再来打开上一章建立的 AndroidTest 项目，分析其项目目录结构，对 Android 项目进一步深入了解。首先启动 Eclipse，展开“Package Explorer”导航器中的“AndroidTest”项目，如图 3-1 所示。

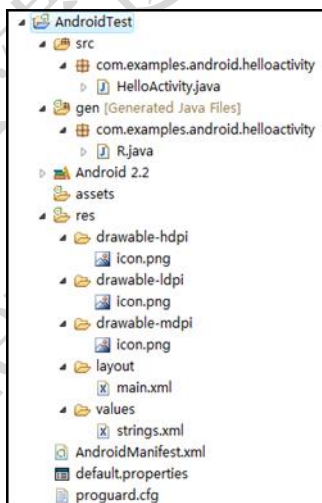


图 3-1 JDK 安装检查 Android 项目结构

与一般的 Java 项目一样，src 文件夹是项目的所有包及源文件（.java），res 文件夹中则包含了项目中的所有资源，比如，程序图标（drawable）、布局文件（layout）、常量（values）等。下面来介绍其他 Java 项目中没有的 gen 文件夹中的 R.java 文件和每个 Android 项目都必须有的 AndroidManifest.xml 文件。

R.java 是在建立项目时自动生成的，这个文件是只读模式，不能更改，R.java 文件是定义该项目所有资源的索引文件。先来看看 AndroidTest 项目的 R.java 文件，如代码清单 3-1 所示。

代码清单 3-1 R.java

```
package com.examples.android.helloactivity;
```



```
public final class R{
    public static final class attr{
    }
    public static final class drawable{
        public static final int icon=0x7f020000;
    }
    public static final class layout{
        public static final int main=0x7f030000;
    }
    public static final class string{
        public static final int app_name=0x7f040001;
        public static final int hello=0x7f040000;
    }
}
```

可以看到这里定义了很多常量，仔细一看就发现这些常量的名字都与 `res` 文件夹中的文件名相同，再次证明 `R.java` 文件中所存储的是该项目所有资源的索引。有了这个文件，在程序中使用资源将变得更加方便，可以很快地找到要使用的资源，由于这个文件不能被手动编辑，所以当在项目中加入了新的资源时，只需要刷新一下该项目，`R.java` 文件便自动生成了所有资源的索引。

`AndroidManifest.xml` 文件则包含了该项目中所使用的 `Activity`、`Service`、`Receiver`，先来打开 `AndroidTest` 项目中的 `AndroidManifest.xml` 文件，如代码清单 3-2 所示。

代码清单 3-2 `AndroidManifest.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    package=" com.examples.android.helloactivity"
    android:versionCode="1"
    android:versionName="1.0" >

    <application android:icon="@drawable/icon"
        android:label="@string/app_name">
        <activity android:name=".HelloActivity"
            android:label="@string/app_name">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category
                    android:name="android.intent.category.LAUNCHER" />
```



```
</intent-filter>

</activity>

</application>

</manifest>
```

代码清单 3-2 中，intent-filters 描述了 Activity 启动的位置和时间。每当一个 Activity（或操作系统）要执行一个操作时，它将创建出一个 Intent 的对象，这个 Intent 对象能承载的信息可描述你想做什么，你想处理什么数据，数据的类型，以及一些其他信息。而 Android 则会和每个 Application 所暴露的 intent-filter 的数据进行比较，找到最合适 Activity 来处理调用者所指定的数据和操作。下面我们来仔细分析 AndroidManifest.xml 文件，如表 3-1 所示。

表 3-1 AndroidManifest.xml 分析

manifest	根节点，描述了 package 中所有的内容
xmlns:android	包含命名空间的声明。 xmlns:android=http://schemas.android.com/apk/res/android，使得 Android 中各种标准属性能在文件中使用，提供了大部分元素中的数据
package	声明应用程序包
application	包含 Package 中 Application 级别组件声明的根节点。此元素也可包含 Application 的一些全局和默认的属性，如标签、icon、主题、必要的权限，等等。一个 manifest 能包含零个或一个此元素（不能大于一个）
android:icon	应用程序图标
android:label	应用程序名字
activity	用来与用户交互的主要工具。Activity 是用户打开一个应用程序的初始页面，大部分被使用到的其他页面也由不同的 Activity 所实现，并声明在另外的 Activity 标记中。注意，每一个 Activity 必须有一个<Activity>标记对应，无论它给外部使用或是只用于自己的 Package 中。如果一个 Activity 没有对应的标记，你将不能运行它。另外，为了支持运行时查找 Activity，可包含一个或多个<intent-filter>元素来描述 Activity 所支持的操作
android:name	应用程序默认启动的 Activity
intent-filter	声明了指定的一组组件支持的 Intent 值，从而形成了 IntentFilter。除了能在此元素下指定不同类型的值，属性也能放在这里来描述一个操作所需的唯一的标签、icon 和其他信息
action	组件支持的 Intent action
category	组件支持的 Intent Category。这里指定了应用程序默认启动的



	Activity
uses-sdk	该应用程序所使用的 SDK 版本相关

下面我们看看资源文件中一些常量的定义，如 String.xml，如代码清单 3-3 所示。

代码清单 3-3 String.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="hello">Hello World, HelloActivity!</string>
    <string name="app_name">HelloWorld</string>
</resources>
```

这个文件很简单，就定义了两个字符串资源，与 R.java 中对应的索引如代码清单 3-4 所示。

代码清单 3-4 R.java 中的 String 类

```
public static final class string{
    public static final int app_name=0x7f040001;
    public static final int hello=0x7f040000;
}
```

在程序中装载并使用这个字符串资源如代码清单 3-5 所示。

代码清单 3-5 String 资源的使用

```
Resources r = this.getContext().getResources();
String appname = ((String) r.getString(R.string.appname));
String hello = ((String) r.getString(R.string.hello));
```

基本上可以定义出项目中所有使用的常量，例如颜色。所以，可根据需要对资源常量进行定义。下面是定义了颜色的常量 colors.xml，如代码清单 3-6 所示。

代码清单 3-6 colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="status_idle">#cccccc</color>
    <color name="status_done">#637a47</color>
    <color name="status_sync">#cc9900</color>
    <color name="status_error">#ac4444</color>
</resources>
```

现在我们来分析 AndroidTest 项目的布局文件（layout），首先打开 res→layout→main.xml 文件，如代码清单 3-7 所示。

代码清单 3-7 main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
```



```
        android:orientation="vertical"
        android:layout_width="fill_parent"
        android:layout_height="fill_parent"
    >
    <TextView
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="@string/hello"
    />
</LinearLayout>
```

< LinearLayout>: 线性版面配置, 在这个标签中, 所有元件都是由上到下排成的。

android:orientation: 表示这个介质的版面配置方式是从上到下垂直地排列其内部的视图。这里“vertical”表示是水平排列。

android:layout_width: 定义当前视图在屏幕上所占的宽度, fill_parent/ match_parent 即填充整个屏幕。

android:layout_height: 定义当前视图在屏幕上所占的高度, fill_parent/ match_parent 即填充整个屏幕。

wrap_content: 随着文字大小的不同而改变这个视图的宽度或高度。

layout_weight : 用于给一个线性布局中的多个视图的重要度赋值。所有视图都有 layout_weight 值, 默认为零, 即需要显示多大的视图就占据多大的屏幕空间。如果值大于零, 则将父视图中的可用空间分割, 分割大小具体取决于每一个视图的 layout_weight 值和该值在当前屏幕布局的整体 layout_weight 值, 以及其他视图屏幕布局的 layout_weight 值中所占的比例。

在这里, 布局中设置了一个 TextView, 用来配置文本标签 Widget, 其中设置的属性 android:layout_width 为整个屏幕的宽度, android:layout_height 可以根据文字来改变高度, 而 android:text 则设置了这个 TextView 要显示的文字内容, 这里引用了 @string 中的 hello 字符串, 即 String.xml 文件中的 hello 所代表的字符串资源。hello 字符串的内容 "Hello World, HelloActivity!" 这就是在 AndroidTest 项目运行时看到的字符串。

最后, 分析 AndroidTest 项目的主程序文件 HelloActivity.java, 如代码清单 3-8 所示。

代码清单 3-8 HelloActivity.java

```
package com.examples.android.helloactivity;

import android.app.Activity;
import android.os.Bundle;

public class HelloActivity extends Activity
```



```
{  
  
    public void onCreate(Bundle savedInstanceState)  
    {  
  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.main);  
  
    }  
  
}
```

主程序 `HelloActivity` 类继承自 `Activity` 类，重写了 `void onCreate (Bundle savedInstanceState)` 方法。在 `onCreate` 方法中通过 `setContentView (R.layout.main)` 设置了 `Activity` 要显示的布局文件 (`/layout/main.xml`)。

到这里，是不是明白了为什么在创建项目时没有进行编码就可以直接运行程序呢？当然，这也是 `Android` 开发的特点，这样可以很轻松地将代码和 `UI` 分开，在国际化 and 程序维护方面有着巨大的作用。如果 `Android` 程序需要自适应国际化，比如说多国语言等问题，那么就可以定义不同语言的 `UI` 布局，在程序装载时调用不同的布局。而且，如果需要修改 `UI` 的一些问题，就不必查看代码了，直接更改这些布局文件即可，是不是很方便？当然，这需要开发者在开发时使用这种 `MVC` 框架，尽量减少使用“硬编码”。

13.1.2 Android 应用解析

上面了解了 `Android` 应用程序的目录结构和其中每个文件的功能，要对其进行应用开发，还需要对 `Android` 应用构造进行深入的分析。下面介绍一下 `Android` 应用开发中所应用到的重要模块。

`Activity`

`Intent`

`Content Provider`

`Service`

当然，并非所有的 `Android` 应用程序都必须由这 4 部分组成，它可以根据开发者需求来进行组合，比如，上面建立的 `HelloActivity` 项目就只使用了 `Activity` 这一个模块。但是，对于任何一个应用程序来说，都必须在 `AndroidManifest.xml` 文件中声明使用到的模块。

1. Activity

`Android` 中最基本的模块就是 `Activity`，在之前的 `HelloActivity` 项目中已经使用过。我们称之为“活动”，在应用程序中，一个活动 (`Activity`) 通常就是一个单独的屏幕。每一个活动都被实现为一个独立的类，并且继承于从活动基类，活动类将会显示由视图控件组成的用户接口，并对事件作出响应。例如，`HelloActivity` 项目中的 `HelloActivity.java` 即继承了活动 (`Activity`) 类。大多数的应用都是由多个 `Activity` 显示组成，例如，对一个文本信息应用而言，第一个屏幕用来显示发送消息的联系人列表，第二个屏幕用来写文本消息和选择收件人，第三个屏幕查看消息历史或者消息设置操作等。



这里的每一个屏幕就是一个活动，很容易实现从一个屏幕到一个新的屏幕，并且完成新的活动。当一个新的屏幕打开后，前一个屏幕将会暂停，并保存在历史栈中。用户可以返回到历史栈中的前一个屏幕，当屏幕不再使用时，还可以从历史栈中删除。

简单理解，Activity 就代表着用户所能看到的屏幕，它主要处理了应用程序的整体性工作，例如，为用户显示指定的 View，启动其他 Activity，监听系统事件（按键事件、触摸屏事件等）等。

所有应用的 Activity 都继承于 Android 提供的基类 android.app.Activity 类，其他的 Activity 继承该父类后，通过父类的方法来实现各种功能，这种设计在其他领域也较为常见。

2. Intent

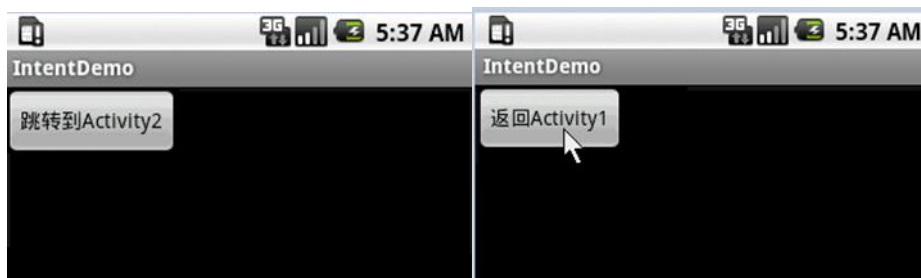
在 Android 中，利用 Intent 类实现了在 Activity1 与 Activity2 之间的切换（以及启动 Service 等）。Intent 类主要用于描述应用的功能。在 Intent 的描述结构中，有动作（Action）和动作对应的数据（data）两个最重要的部分。其中，典型的动作类型有：MAIN、VIEW、PICK、EDIT 等，而动作对应的数据以 URI 的形式表示。例如，如果要查看一个人的联系方式，需要先创建一个动作类型为 VIEW 的 Intent，以及一个表示这个动作对应的数据（这里就是这个人）的 URI。

从一个屏幕导航到另一个屏幕，我们需要解析各种 Intent。为了实现向前导航，首先，我们调用 Activity 的 startActivity(Intent myIntent) 方法。此时，系统为找到最匹配 myIntent 的 Intent 对应的 Activity，就将在所有已安装的应用程序中定义的 IntentFilter 中查找。而匹配的对应的新的 Activity 在接收到 myIntent 的通知后，开始运行。当 startActivity 方法被调用时，将触发解析 myIntent 的动作，该机制提供了两个关键好处。

（1）Activities 能够重复利用从其他组件中以 Intent 的形式产生的请求。

（2）Activities 可以在任何时候被具有相同 Action 的新的 Activity 取代。

下面举例说明两个 Activity 之间的切换。运行效果：当应用程序启动时，显示布局 main.xml，如图 3-2 所示，当我们单击“切换”按钮时，屏幕显示布局 main2.xml，如图 3-3 所示，再单击“切换”按钮，又回到如图 3-2 所示的状态。就这样通过 Intent 完成了两个 Activity 之间的切换。



以下是两个 Activity 的代码。

代码清单 3-9 Activity1.java

```
package com.example.android.Examples_03_01;
```



```
import android.app.Activity;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;

/**
 * 在 Examples_02_01 项目中一共使用了两个 Activity,
 * 每使用一个 Activity, 都必须在 AndroidManifest.xml 中
 * 进行声明
 */
public class Activity1 extends Activity
{
    public void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        /* 设置显示 main.xml 布局 */
        setContentView(R.layout.main);
        /* findViewById(R.id.button1)取得布局 main.xml 中的 button1 */
        Button button = (Button) findViewById(R.id.button1);
        /* 监听 button 的事件信息 */
        button.setOnClickListener(new Button.OnClickListener() {
            public void onClick(View v)
            {
                /* 新建一个 Intent 对象 */
                Intent intent = new Intent();
                /* 指定 intent 要启动的类 */
                intent.setClass(Activity1.this, Activity2.class);
                /* 启动一个新的 Activity */
                startActivity(intent);
                /* 关闭当前的 Activity */
                Activity1.this.finish();
            }
        });
    }
}
```



```
}
```

代码清单 3-10 Activity02.java

```
package com.example.android.Examples_03_01;

import android.app.Activity;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;

public class Activity2 extends Activity
{
    public void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        /* 设置显示 main2.xml 布局 */
        setContentView(R.layout.main2);
        /* findViewById(R.id.button2)取得布局 main.xml 中的 button2 */
        Button button = (Button) findViewById(R.id.button2);
        /* 监听 button 的事件信息 */
        button.setOnClickListener(new Button.OnClickListener() {
            public void onClick(View v)
            {
                /* 新建一个 Intent 对象 */
                Intent intent = new Intent();
                /* 指定 intent 要启动的类 */
                intent.setClass(Activity2.this, Activity1.class);
                /* 启动一个新的 Activity */
                startActivity(intent);
                /* 关闭当前的 Activity */
                Activity2.this.finish();
            }
        });
    }
}
```



代码清单 3-11 main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    >

    <Button
        android:id="@+id/button1"
        android:layout_width="100px"
        android:layout_height="wrap_content"
        android:layout_x="100px"
        android:layout_y="80px"
        android:text="跳转到 Activity2"
        >

    </Button>
</LinearLayout>
```

代码清单 3-12 Main2.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    >

    </TextView>

    <Button
        android:id="@+id/button2"
        android:layout_width="100px"
        android:layout_height="wrap_content"
        android:layout_x="100px"
        android:layout_y="80px"
        android:text="返回 Activity1"
        >
```



```
>  
  
</Button>  
  
</LinearLayout>
```

如代码清单 3-9 所示，需要在 AndroidManifest.xml 中声明使用的 Activity2，如代码清单 3-13 所示。

代码清单 3-13 AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>  
<manifest xmlns:android="http://schemas.android.com/apk/res/android"  
    package="com.example.android.Examples_03_01"  
    android:versionCode="1"  
    android:versionName="1.0">  
    <application android:icon="@drawable/icon" android:label="@string/app_name">  
        <activity android:name=".Activity1"  
            android:label="@string/app_name">  
            <intent-filter>  
                <action android:name="android.intent.action.MAIN" />  
                <category android:name="android.intent.category.LAUNCHER" />  
            </intent-filter>  
        </activity>  
        <activity android:name="Activity2"></activity>  
    </application>  
    <uses-sdk android:minSdkVersion="8" />  
</manifest>
```

3. Content Provider

Android 应用能够将它们的数据保存到文件和 SQLite 数据库中，甚至是任何有效的设备中。利用 Content Provider 实现应用数据与其他的应用共享。因为 Content Provider 类实现了一组标准的方法，能够让其他的应用保存或读取此内容提供者处理的各种数据类型。

Content Provider 可以理解为在不同的应用包之间共享数据的工具。在 Android 中，默认的系统数据库为 SQLite。但是在 Android 中，使用方法略有区别。在 Android 中，每一个应用都单独运行在各自的进程中，当一个应用需要访问其他应用的数据时，这里就需要在不同的虚拟机之间传递数据，这样的情况操作起来可能有些困难（正常情况下，你不能读取其他应用的 db 文件）。Content Provider 就解决了上述困难。

在 Android 中，Content Provider 是一个特殊的存储数据的类型，它提供了一套标准的接口用来获取和



操作数据。并且，Android 自身也提供了现成的 Content Provider: Contacts、Browser、CallLog、Settings、MediaStore。当通过 Content Resolver 提供的方法来使用 Content Provider 时，应用可以通过唯一的 Content Resolver interface 来使用具体的某个 Content Provider。其中，Content Resolver 提供的方法包括 query()、insert()、update()等。要使用这些方法，还会涉及 URI。可以将它理解成 string 形式的 Content Provider 的完全路径。

4. Service

Service 即“服务”的意思，既然是服务，那么 Service 将是一个生命周期长而且没有用户界面的程序。比如，一个正在从播放列表中播放歌曲的媒体播放器，在这个媒体播放器应用中，应该会有多个 Activity，让使用者可以选择歌曲并播放歌曲。然而，音乐重放这个功能并没有对应的 Activity，因为使用者会认为在导航到其他屏幕时音乐应该还在播放。在这个例子中，媒体播放器这个 Activity 会使用 Context.Startservice()来启动一个 Service，从而可以在后台保持音乐的播放。同时，系统也将保持这个 Service 一直执行，直到这个 Service 运行结束。另外，还可以通过使用 Context.Bindservice()方法连接到一个 Service 上（如果这个 Service 当前还没有处于启动状态，则将启动它）。当连接到一个 Service 之后，还可用 Service 提供的接口与它进行通信。以媒体播放器为例，还可以执行暂停、重播等操作。

13.2 Android 程序 UI 设计

在前面章节的例子中，我们已经接触了 TextView、Button 等 UI 控件，其实这里所说的 UI 就是布局文件（layout），UI 是一个应用程序展现给用户的界面，一个应用程序要想受用户喜爱，那么 UI 可不能差。自从 Android SDK 1.0_r2 版本开始，ADT 提供了 UI 预览的功能。现在只需要打开一个 Android 项目的“/res/layout/main.xml”并右键单击鼠标右键，在弹出的快捷菜单中选择“Open With”→“Android Layout Editor”命令，或者直接双击 main.xml 文件，即可以切换到 UI 设计界面，如图 3-4 所示。

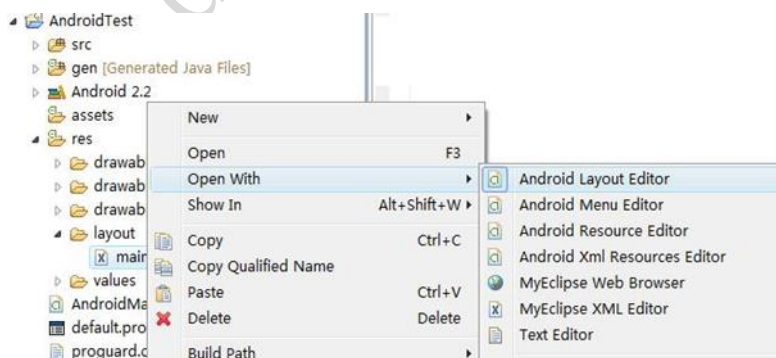


图 3-4 Android Layout Editor 命令

左边的 layouts 标签的内容则是一些线性布局，可以使用它轻松地完成对布局的排版，比如，横向或者纵向布局。Views 标签则是一些 UI 控件，可以将这些控件直接拖动到右边的窗口进行编辑，这些 UI 控件的类型如图 3-5 所示。

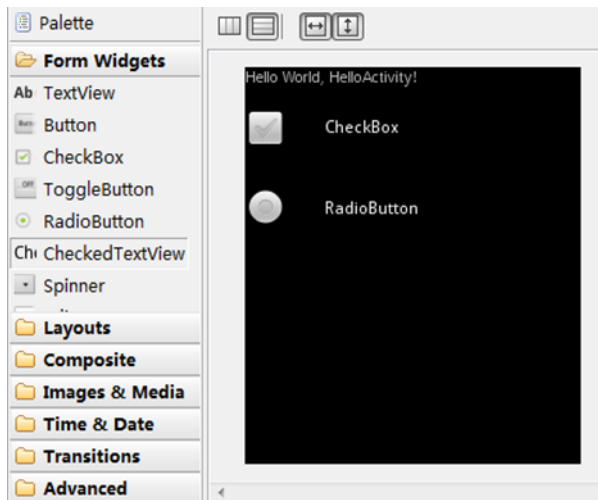


图 3-5 Android layout Editor

当然，还可以单击右下角的 main.xml 标签来切换到 XML 编辑器，对代码进行编排，如图 3-6 所示。将这些功能配合起来使用，基本可以满足开发者需求。

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    >
    <TextView
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="@string/hello"
    />
    <CheckBox
        android:text="CheckBox"
        android:id="@+id/checkBox1"
    />
</LinearLayout>
```

图 3-6 XML 编辑器

13.3 Java 语言在 Android 程序中的使用

Android 应用程序采用 Java 语言编写，Java 语法和 C/C++ 有很大的相似性，但也有一些特别之处。

13.3.1 Interface 的使用

从名字上看，Interface 即为接口的意思，多用于实现回调（Call Back）方法。

在 Interface 的定义中，一般的代码架构如代码清单 3-14 所示。

代码清单 3-14 InterfaceServer.java

```
public class InterfaceServer {
    public interface OnClickListener{
        public void onClick();
    }
}
```



```

    }

    private OnClickListener mOnClickListener=null;

    public void onClick(){
        if(mOnClickListener!=null)
            mOnClickListener.onClick();
    }

    public void setOnClickListener(OnClickListener l){
        mOnClickListener = l;
    }

```

对于 Interface 内部的方法而言，只需要声明，而不需要具体实现。从编译器的角度来看，Interface 会被认为是一个指向方法的指针。

使用 InterfaceServer 的代码一般如代码清单 3-15 所示。

代码清单 3-15 使用 InterfaceServer

```

public void addToButton {
    Button b = (Button)findViewById(R.id.button);
    OnClickListener l = new OnClickListener(){
        public void onClick(View v){
            TextView tv1 = (TextView) findViewById(R.id.tv1);
            tv1.setText("The Button has been clicked");
        }
    };
    b.setOnClickListener(l);
}

```

13.3.2 abstract class 的使用

abstract 是一个修饰符，其类似于 Static 这样的关键字。Android 程序中常用 abstract 修饰一个类，如 abstract class，当然它也可以修饰一些变量或是方法。

抽象类所包含的方法可以只是定义，也可以是已实现的。对于没有实现的方法，基于该方法的子类必须实现；而对于已经实现的方法，子类可以重写该方法，若没有重写，则使用父类的方法。

在一定程度上，abstract class 可以代替 Interface，例如，3.3.1 节中 Interface 的例子做如下的 abstract class 替换，其效果是等价的。

代码清单 3-16 InterfaceServer.java

```

public class InterfaceServer {
    abstract class OnClickListener2{
        public void onClick2();
    }
}

```



```

    }

    private OnClickListener2 mOnClickListener2=null;

    public void onClick2(){
        if(mOnClickListener2!=null)
            mOnClickListener2.onClick2();
    }

    public void setOnClickListener2(OnClickListener2 l){
        mOnClickListener2 = l;
    }
}

```

13.3.3 Interface 与 abstract class 的区别

从语法角度讲，接口和抽象类有以下区别。

Java 语法规则，一个子类只能有一个父类，但可以实现多个接口。

abstract class 可以代替 Interface。

定义 Interface 时，只需要列出所包含方法的定义而不必实现。而定义 Abstract 类时，方法必须有实现部分，这就是所谓的默认实现，除非该方法也是 Abstract 类型。

接口的子类必须实现接口所定义的全部方法，而抽象类的子类不必实现抽象类所定义的任何方法，除非该方法是 Abstract 或者子类想重写某个方法。

接口中的成员变量必须是 Static Final 类型（实际应用中则很少包含变量，因为接口多用于引用），而 abstract class 内部可以包含任意变量。

从应用的角度来讲，Interface 和 abstract class 的区别在于：Interface 提供了一个方法集合的接口，该接口用于客户端和服务端的方法调用，如图 3-7 所示。

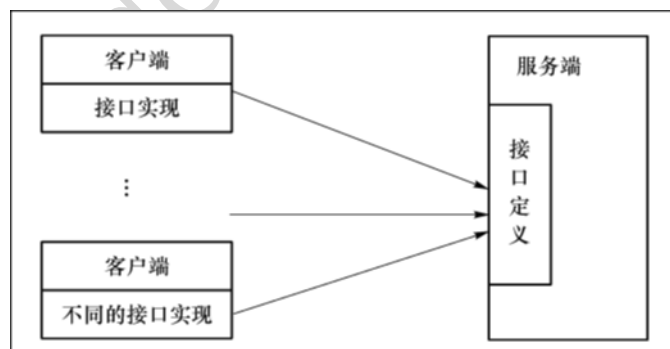


图 3-7 Interface 的使用机制

接口一般是由服务端定义，比如操作系统，客户端根据自己的需求对接口做不同的实现；而 abstract class 则仅提供了一个基类，该基类没有任何服务端或者客户端的概念，它的作用就是为了继承并重写，如图 3-8 所示。

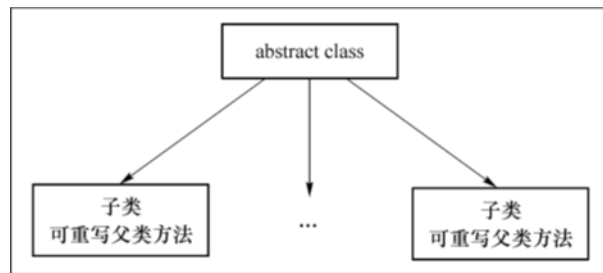


图 3-8 abstract class 的使用机制

13.3.4 for 循环的使用

除了传统的 for 循环（语法是 `for(int i=0;i<N;i++){}`；）以外还有 foreach 循环。举例如代码清单 3-17 所示。

代码清单 3-17 For 循环语法

```

public void doSomething(){
    int[] ages = new int[20];
    for(int age:ages){
        //to do,add something to process age.
        Log.i("Haiii" . "The age is" +age);
        ...
    }
}

```

该 for 语法是对某个集合进行循环，第 1 个参数是循环过程集合元素值的引用，第 2 个参数是集合对象。第 1 个参数的类型必须和集合元素的类型相同。

以下代码示例了传统的 for 语法和新 for 语法的等效使用，但由于编译器对新 for 语法的优化，其执行效率将更高。

代码清单 3-18 传统 for 语法和新 for 语法比较

```

public void doSomething(){
    int[] counts =new int[20];
    int total1=0,total2=0;
    for(int i=0;i<20;i++)
        total1 += counts[i];
    for(int count:counts)
        total2 += count;
}

```

13.3.5 Map 类的使用

在 Android 系统中，有着多种存储数据的方式，例如，文件、数据库及程序内参数式存储、网络存储



等。对于参数式存储时，使用的就是 Map 类。Map 本身是 Interface，Java 基于该接口实现三个具体的 Map 类，分别是 HashMap、TreeMap，以及 EnumMap，常用的为 HashMap，本节也主要介绍 HashMap。

Map 定义了访问特定集合的标准方法，这种集合用来存储 key-value 类型的键值对，比如，对于 name:Haiai 和 age:22 这两组数据来讲，其中 name、age 称为键（key），与此对应的是键值（value）。在一个 Map 集合类中，每对键或值其类型都可以是任意的，比如 int、String 等都是可以的。

Map 类又是一个类模板，一个 Map 类对象在初始化时必须指定键的类型，可以是任何 Object 类，比如，Map<String,Object> mMap= new HashMap<String,Object>()。

<>里面的数据类型用于指定 Map 集合中“键值对”的类型。

给 Map 集合添加和删除键值对的方法如表 3-2 所示。

表 3-2 Map 集合添加和删除键值对的方法

方法	描述
clear()	删除该 Map 集合中的全部元素
remove(Objectkey)	删除键名为 key 所对应的键值对
put(Objectkey,Objectvalue)	添加一个新的键值对
putAll(Mapmap)	将该 Map 集合的元素全部复制到新的 Map 中

Map 类没有提供直接遍历键值对的方法，要遍历所有键值对需要一个中间过程。Map 提供了 3 个方法用于间接遍历键值对，如下：

entrySet() 返回所有键值对类型为 Set 对象。

keySet() 返回所有键值对类型为 Set 对象。

valueSet() 返回所有键值对类型为 Collection 对象。

要得到具体的键值对，需要再解析 Set 和 Collection 对象，但仅有这两个对象还不能获得键值对，还需要借助于 Iterator 类。到这里，可能觉得有些复杂，别着急，结果马上就要出来了。

Set、Collection、Iterator 实际上是 Map 内部进行操作的 3 个辅助类，要得到具体 Map 键值对，如代码清单 3-19 所示。

代码清单 3-19 得到具体的 Map 键值对

```
Map<String,Object> mMap= new HashMap<String,Object>();
```

```
Iterator kv = mMap.entrySet().iterator();
```

```
Iterator k = mMap.keySet().iterator();
```

```
Iterator v = mMap.values().iterator();
```

```
Int size = mMap.size();
```

```
for(int i = 0;i<size;i++)
```

```
{
```

```
    Map.Entryentry = (Map.Entry)kv.next();
```



```
Object key = entry.getKey();  
Object value = entry.getValue();  
}
```

用以上代码读取键值对时，Object 可以强制转换为 int 类型。

13.3.6 Integer 与 String 之间的转换

在实际程序设计中，经常需要把 Integer 类型转换为 String 类型，或者相反，Java 类库中提供了这样的方法。

整形转换为字符串：String.valueOf(Stringstr)。

字符串转换为整形：Integer.parseInt(int)、Integer.parseInt(long)等。

面向对象编程中，一般不能直接调用类中的方法，而是需要先定义类的一个对象，然后才能使用其包含的方法。而以上两个方法直接调用 String 和 Integer 的方法，这就是 static 关键字的作用。

在定义一个类时,如果其中的方法声明为 static，那么外部程序就可以直接调用该方法，该方法所引用的一些变量也必须为 static 类型的变量。

Java 内部有一种安全机制：对于一个普通的类，必须声明该类的对象才能访问类中的方法或者变量，实现这种安全机制的是 Java 编译器。编译器隐藏了所有类的地址，因此不能通过类名称定位到类的地址；而如果使用 static 修饰符，无论是变量还是方法,编译器都会把该变量或者方法的名称导出，以便程序能够根据类名定位到类所在的地址，从而能够调用相应的方法或者变量。

13.3.7 synchronized 同步关键字

synchronized 关键字属于操作系统的范畴，与同步对应的是异步。在程序设计的概念中,有同步调用或者异步调用，同步是指该段代码（方法）从调用开始,直到内部执行完毕后才能返回；异步调用是指调用该段代码（方法）后立即返回，无论该段代码内部所执行的物理操作是否执行完毕，异步代码一般存在于多线程程序设计中，单线程程序内部不存在异步调用。

synchronized 关键字的作用就是告诉操作系统，在执行该关键字所限定的代码片段内，不允许被其他线程打断。在一般的操作系统设计中，会提供一个类似于 synchronized 的 API 方法，而 Java 则是给了这样一个关键字，相当于说，Java 编译器为操作系统分担了一部分工作。

那么，什么情况下需要使用 synchronized 关键字呢？凡是需要某段代码在执行时不被其他线程打断时，都可以加上 synchronized 关键字，举例如代码清单 3-20 所示。

代码清单 3-20 synchronized 关键字

```
public class MyMusicWidgetProvider{  
    private static MyMusicWidgetProvider sInstance;  
    static synchronized MyMusicWidgetProvider getInstance(){  
        if(sInstance==null)  
            sInstance=new MyMusicWidgetProvider();  
        return sInstance;  
    }  
}
```



```
}  
  
}
```

以上代码定义了一个类，并希望该类在运行时仅有一个实例，每次调用该类所包含的方法时，先得到该类的实例，然后再通过实例调用其他方法。该类有一个 `static` 的 `getInstance()` 方法，该方法的作用就是检查是否存在一个实例，如果没有就创建一个，否则返回存在的实例。

`getInstance()` 方法前面加了 `synchronized` 关键字，为的是该方法在执行时不能被其他线程打断，那么，为什么有这样的要求呢？试想一下，假设没有使用该关键字，第 1 个线程 A 在执行 `getInstance()` 方法时，方法体中的 `new` 运算符刚刚创建了一个 `MyMusicWidgetProvider` 实例，但还没来得及把实例赋值给 `sInstance`，而此时 B 线程又来调用 `getInstance()` 方法，出现这种情况的原因是：`sInstance = new MyMusicWidgetProvider()` 这句代码会被 Java 编译器编译成多条机器指令，其中给 `sInstance` 赋值的机器指令和创建一个 `MyMusicWidgetProvider` 对象的机器指令是分开的。此时对于 B 线程来讲，它会重新检查 `sInstance` 是否为空，由于 A 线程还没有来得及给 `sInstance` 赋值，因此，B 线程就会再次创建一个新的 `MyMusicWidgetProvider` 实例，当 B 线程暂停并返回 A 线程时，A 线程会把第 1 次创建的实例重新赋值给 `sInstance`，这就会导致 B 线程将来对 `MyMusicWidgetProvider` 实例的错误引用，因为 B 线程所创建实例的地址被 A 线程修改了。

因此，这就要求 `getInstance()` 一旦开始执行，必须执行完毕后才能返回，中间不能被其他线程打断。

`synchronized` 除了约束整个方法外，也可以约束一小段代码，这样做的好处有两个：一个是可以避免定义新方法；另一个是能够避免一些线程同步问题，如代码清单 3-21 所示。

代码清单 3-21 `synchronized` 约束小段代码

```
Object obj;  
  
int c;  
  
void addMethod(){  
    synchronized(obj){  
        c++;  
        //其他代码  
    }  
}
```

`obj` 可以是任意类型的一个对象，该对象的唯一作用就是给 `synchronized` 提供一个“锁”。同步设计的原则之一是尽量减少同步中包含的代码大小，从而在线程间能够更平衡地运行。因此，同步一小段代码是一个不错的做法。同时，假设 `addMethod()` 内部还调用了其他同步方法，那么，从这个同步跳到另外的同步会增加线程间死锁的风险，因此不同步整个 `addMethod()` 而仅同步局部代码，这就是第 2 个好处。

13.3.8 new 的使用

Java 语言中，`new` 的作用是为一个对象（Object）分配内存，代码清单 3-22 说明了为各种 Object 分配内存的方法。



代码清单 3-22 为各种 Object 分配内存的方法

```
int a = 20;
int A[] = new int[100];
float A2[] = new float[100];
int A3[] = {10, 20, 30};
String str = new String();
String str1 = ;
String str2 = null;
String[] Str = new String[100];
MyMusicWidgetProvider myProvider = new MyMusicWidgetProvider();
str1 += "Android is from... ";
MyMusicWidgetProvider commonProvider = myProvider.getInstance()
```

一般情况下，没有用 `new` 修饰符定义的数据都是在栈（Stack）中分配内存，但有一个例外，对于 `String` 定义的变量，总是从系统内存堆（Heap）中分配内存，栈中仅有对该 `String` 的引用。另外，从系统堆中分配的实际内存大小并不是按指定的大小分配的，比如，`int A[] = new int[100]` 所分配的内存大小并不是 100B，而是 128B，内存分配机制为了提高分配效率以及分配算法的可实现性，实际上的内存颗粒大小会按照 2 的幂次方进行划分，实际分配的内存大小是最接近指定大小的一个值。另外，最小的内存颗粒大小会根据不同的内存分配算法有所不同，一般会取 512B 或者 1KB。

13.4 本章小结

本章主要介绍了 Android 应用程序框架、UI 设计，以及 Java 在 Android 程序中的使用。首先彻底地分析了上一章的 `AndroidTest` 项目，从其项目目录结构、文件功能等方面分析了 Android 应用程序的基本框架，其次逐一分析了 Android 应用程序的构成，并分别通过示例程序演示了其功能的运用。其次介绍了有关 UI 设计的工具，使得程序界面更加漂亮。最后介绍了在 Android 程序中 Java 语法的使用。



第 14 章 Android 生命周期

经过上一章的学习，主要了解了 Android 应用程序设计的基础知识，对 Android 程序的开发有了一定的了解。在此基础上，本章将对 Android 系统的进程优先级的变化方式、Android 系统的 4 大基本组件、Activity 的生命周期中各个状态的变化关系、Android 应用程序的调试方法和工具进行学习。

14.1 程序生命周期

所谓的应用程序生命周期就是应用程序进程从创建到消亡的整个过程。在 Android 中，多数情况下每个程序都是在各自独立的 Linux 进程中运行的。当一个程序或其某些部分被请求时，它的进程就“出生”了；当这个程序没有必要再运行下去且系统需要回收这个进程的内存用于其他程序时，这个进程就“死亡”了。可以看出，Android 程序的生命周期是由系统控制而非程序自身直接控制。这和编写桌面应用程序时的思维有一些不同，一个桌面应用程序的进程也是在其他进程或用户请求时被创建，但是往往是在程序自身收到关闭请求后执行一个特定的动作（如从 main 方法中 return）而导致进程结束的。

简而言之，程序的生命周期是在 Android 系统中进程从启动到终止的所有阶段，也就是 Android 程序启动到停止的全过程，程序的生命周期是由 Android 系统进行调度和控制的。

但是，一个不容忽视的问题就是，手机的内存是有限的，随着打开的应用程序数量的增多，随之而来的可能会是应用程序响应时间过长或者系统假死的糟糕情况。所以，若将 Android 应用程序生命周期交由系统处理的话，那么在系统内存不足的情况下，便由 Android 系统舍车保帅，选择性地来终止一些重要性较次的应用程序，以便回收内存供更重要的应用程序使用。

那么，系统是根据一个什么样的重要性标准来终止 Android 应用程序的呢？

Android 根据应用程序的组件及组件当前运行状态将所有的进程按重要性程度从高到低划分了五个优先级：前台进程、可见进程、服务进程、后台进程、空进程。

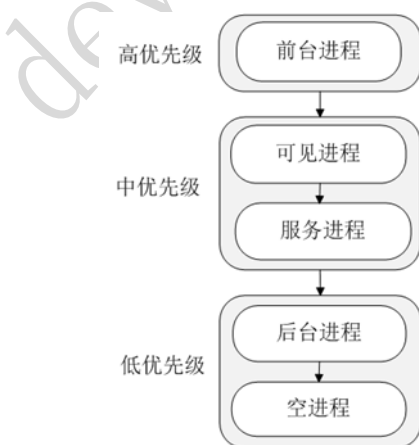


图 4-1 Android 系统进程优先级

以下就按优先级由高到低的顺序介绍 Android 系统中的进程。

1. 前台进程

前台进程是显示在屏幕最前端并与用户正在交互的进程，是 Android 系统中最重要



4 种情况。

进程中的 Activity 正在与用户进行交互。

进程服务被 Activity 调用，而且这个 Activity 正在与用户进行交互。

进程服务正在执行声明周期中的回调方法，如 onCreate()、onStart()或 onDestroy()。

进程的 BroadcastReceiver 正在执行 onReceive()方法。

Android 系统在多个前台进程同时运行时，可能会出现资源不足的情况，此时会清除部分前台进程，保证主要的用户界面能够及时响应。

2. 可见进程

可见进程指部分程序界面能够被用户看见，却不在前台与用户交互，不响应界面事件（其 onPause()方法已被调用）的进程。如果一个进程包含服务，且这个服务正在被用户可见的 Activity 调用，此进程同样被视为可见进程。

Android 系统一般存在少量的可见进程，只有在特殊的情况下，Android 系统才会为保证前台进程的资源而清除可见进程。

3. 服务进程

服务进程是指包含由 startService()方法启动服务的进程。它有以下特性：没有用户界面；在后台长期运行。例如，后台 MP3 播放器或后台上传下载数据的网络服务。

Android 系统除非不能保证前台进程或可见进程所必要的资源，否则不强行清除服务进程。

4. 后台进程

后台进程是指不包含任何已经启动的服务，而且没有任何用户可见的 Activity 的进程。这些进程不直接影响用户的体验。

Android 系统中一般存在数量较多的后台进程，因此这些进程会被保存在一个列表中，以保证在系统资源紧张时，系统将优先清除用户较长时间没有见到的后台进程。

5. 空进程

空进程是不包含任何活跃组件的进程。一般保留这些进程，是为了将其作为一个缓存，在它所属的应用组件下一次需要时，缩短启动的时间。

空进程在系统资源紧张时会被首先清除，但为了提高 Android 系统应用程序的启动速度，Android 系统会将空进程保存在系统内存中，在用户重新启动该程序时，空进程会被重新使用。

14.2 Android 组件

组件是可以调用的基本功能模块。Android 应用程序就是由组件组成的，Android 系统有 4 个重要的组件，分别是 Activity、Service、BroadcastReceiver 和 ContentProvider。

Activity 是 Android 程序的呈现层，显示可视化的用户界面，并接收与用户交互所产生的界面事件。在界面上的呈现形式就是全屏窗体、非全屏悬浮窗体的对话框，与在桌面系统上的独立事业，如办公应用等类似。Activities 是可执行的代码块，由用户或者操作系统来进行初始实例化，并在他们被需求时致以运行。Activities 可以与用户、请求数据或者其他 Activity、Service 的服务通过 query 或 Intent 进行交互。大部分为 Android 编写的可执行代码将以 Activity 的形式执行。对于一个 Android 应用程序来说，可



以包含一个或多个 Activity，一般在程序启动后会呈现一个 Activity，用于提示用户程序已经正常启动。当它不积极运行时，Activity 可以被操作系统终止以节省内存。

Service 用于没有用户界面，但需要长时间在后台运行的应用。它类似于桌面应用或者服务器操作系统上的服务或守护进程。Service 是在后台运行的可执行的代码块，从它被初始化一直运行到该程序关闭。一个 Service 的典型例子是一个 MP3 播放器，尽管用户已经使用其他应用程序，但仍然需要持续播放文件。你的应用程序可能需要在没有用户界面的情况下一直执行 Service 来实现后台任务。

Broadcast 和 Intent Receivers 对从其他的应用程序的服务请求做出一个全系统广播的响应，这些广播响应可能来自于 Android 系统本身或者是任何在其系统上运行的程序。BroadcastReceiver 是用来接受并响应广播消息的组件。它不包含任何用户界面，但可以通过启动 Activity 或者 Notification 通知用户接收到重要信息。

Activity 或 Service 通过执行一个 IntentReceiver 为其他应用程序提供了访问其功能的功能。Intent Receiver 是一段可执行代码块，对其他 Activity 的数据或服务请求做出响应。请求的 Activity（客户端）生成一个 Intent，将其添加至 Android Framework 中，来指出哪些应用程序（目标程序）接收并对其做出响应。Intent 是 Android 的主要构成元素之一，它从现有的应用程序中创造新的应用程序。Intent 实现了应用程序和其他的应用程序和服务交换所需信息的功能。

ContentProvider 是 Android 系统提供了一种标准的共享数据的机制，应用程序可以通过 ContentProvider 访问其他应用程序的私有数据（私有数据可以是存储在文件系统中的文件，也可以是 SQLite 中的数据库）。Android 系统内部也提供一些内置的 ContentProvider，能够为应用程序提供重要的数据信息。

所有 Android 组件都具有自己的生命周期，是从组件建立到组件销毁的整个过程。在生命周期中，组件会在可见、不可见、活动、非活动等状态中不断变化。

14.3 Activity 生命周期

Activity 生命周期指 Activity 从启动到销毁的过程。Activity 表现为 4 种状态，分别是活动状态、暂停状态、停止状态和非活动状态。

活动状态，Activity 在用户界面中处于最上层，完全能被用户看到，能够与用户进行交互。

暂停状态，Activity 在界面上被部分遮挡，该 Activity 不再处于用户界面的最上层，且不能够与用户进行交互；或者屏幕被锁定。

停止状态，Activity 在界面上完全不能被用户看到，也就是说这个 Activity 被其他 Activity 全部遮挡。

非活动状态，不在以上 3 种状态中的 Activity 则处于非活动状态。

这四种状态是可以相互转换的，转换关系图如图 4-2 所示。

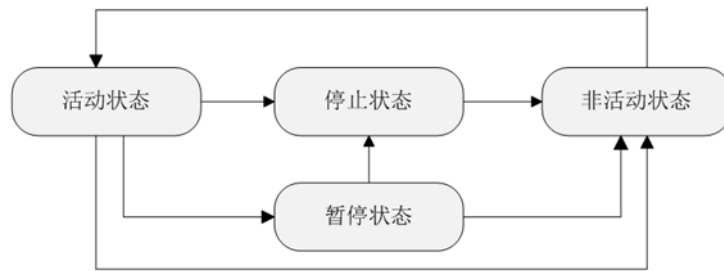


图 4-2 Activity 的 4 种状态的转换关系图

图 4-3 解释了状态之间转化的可能路径。其中着色的椭圆表示活动的主要状态，矩形表示当活动在状态之间转换时会被调用的回调方法。

Android 调用以下的事件回调方法通知 Activity 从某一状态转变到另一状态。

代码清单 4-1 事件的回调方法

```

public class MyActivity extends Activity {
    protected void onCreate(Bundle savedInstanceState);
    protected void onStart();
    protected void onRestart();
    protected void onResume();
    protected void onPause();
    protected void onStop();
    protected void onDestroy();
}
  
```

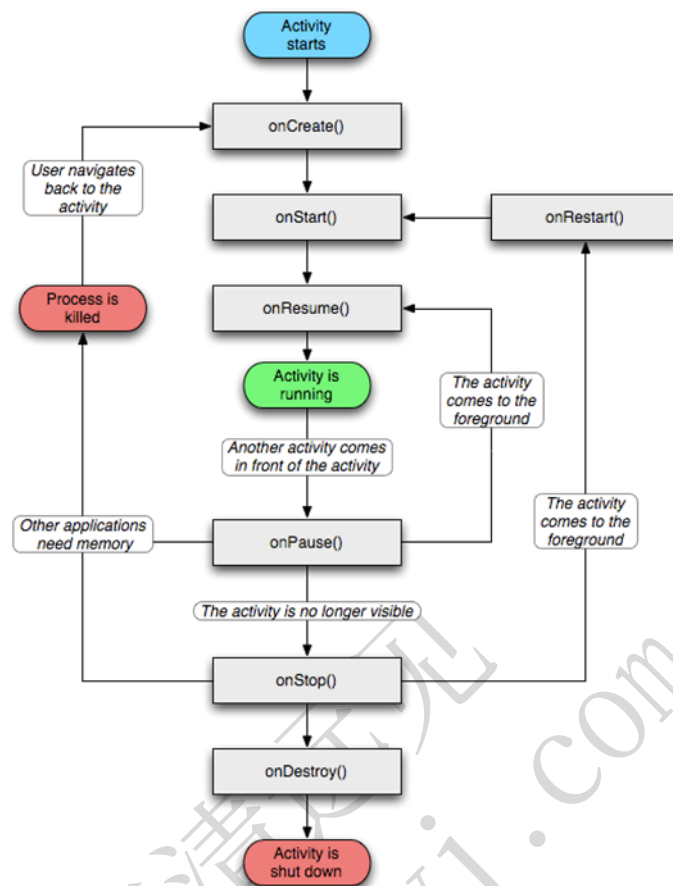


表 4-1 对各个事件回调方法做出说明。

表 4-1 Activity 生命周期的事件回调方法

方法	是否可终止	说明
onCreate()	否	Activity 启动后第一个被调用的方法，常用来进行 Activity 的初始化，例如创建 View、绑定数据或恢复信息等
onStart()	否	当 Activity 显示在屏幕上时，该方法被调用
onRestart()	否	当 Activity 从停止状态进入活动状态前，调用该方法
onResume()	否	当 Activity 能够与用户交互，接收用户输入时，该方法被调用。此时的 Activity 位于 Activity 栈的栈顶



方法	是否可终止	说明
onPause()	是	当 Activity 进入暂停状态时，该方法被调用。一般用来保存持久的数据或释放占用的资源
onStop()	是	当 Activity 进入停止状态时，该方法被调用
onDestroy()	是	在 Activity 被终止前，即进入非活动状态前，该方法被调用
onSaveInstanceState()	否	Android 系统因资源不足终止 Activity 前调用该方法，用以保存 Activity 的状态信息，供 onRestoreInstanceState()或 onCreate()恢复之用
onRestoreInstanceState()	否	恢复 onSaveInstanceState()保存的 Activity 状态信息，在 onStart()和 onResume ()之间被调用

Activity 事件回调方法的调用顺序，如图 4-4 所示。

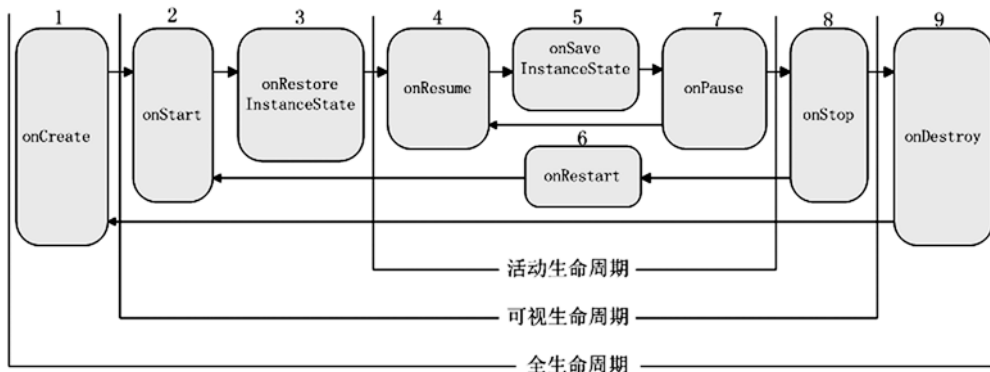


图 4-4 Activity 事件回调方法的调用顺序

Activity 的生命周期可分为全生命周期、可视生命周期和活动生命周期。每个生命周期中包含不同的事件回调方法。

14.3.1 全生命周期

全生命周期是从 Activity 建立到销毁的全部过程，始于 onCreate()，结束于 onDestroy()。

使用者通常在 onCreate()中初始化用户界面，分配引用类变量，绑定数据控件，并创建服务和线程等 Activity 所能使用的全局资源和状态，并在 onDestroy()中释放这些资源，并确保所有外部连接被关闭，例如，网络或数据库的联系等；在一些极端的情况下，Android 系统会不调用 onDestroy()方法，而直接终止进程。



为了避免创造短期对象和增加垃圾收集的时间，以致对用户体验产生直接影响。如果你的 Activity 需要创建一些对象的话，最好在 onCreate 方法中创建，因为它在一个 Activity 的完整生命周期中仅调用一次。

14.3.2 可视生命周期

可视生命周期是 Activity 在界面上从可见到不可见的过程，开始于 onStart()，结束于 onStop()。

onStart()一般用来初始化或启动与更新界面相关的资源。

onStop()一般用来暂停或停止一切与更新用户界面相关的线程、计时器和服务。

onRestart()方法在 onStart()前被调用，用来在 Activity 从不可见变为可见的过程中，进行一些特定的处理过程。

onStart()和 onStop()会被多次调用。

onStart()和 onStop()也经常被用来注册和注销 BroadcastReceiver 或者传感器。

在 onStart()和 onStop()这两个方法中间，Activity 对用户将会是可见的，尽管它可能部分被遮挡着。在一个 Activity 完整的生命周期中可能会经过几个 Activity 可见的生命周期，因为 Activity 可能会经常在前台和后台之间切换。在极端情况下，系统将销毁掉一个 Activity 即使它在可见状态并且不调用 onStop 方法。

14.3.3 活动生命周期

活动生命周期是 Activity 在屏幕的最上层，并能够与用户交互的阶段，开始于 onResume()，结束于 onPause()。在 Activity 的状态变换过程中 onResume()和 onPause()经常被调用，因此这两个方法中应使用更为简单、高效的代码。

onPause()是第一个被标识为“可终止”的方法。

在 onPause()返回后，onStop()和 onDestroy()随时能被 Android 系统调用。

onPause()常用来保存持久数据，如界面上用户的输入信息等。

当系统而不是用户关闭一个活动来节省内存时，用户可能希望返回到活动且是它之前的状态。为了获得活动被关闭之前的状态，可以执行活动的 onSaveInstanceState()方法。Android 在活动容易被销毁前调用这个方法，也就是调用 onPause()之前。该方法的参数是一个 Bundle 对象，这个对象可以名值对记录活动的动态状态。当活动再次启动时，Bundle 同时被传递到 onCreate()和调用 onCreate()之后的方法 onRestoreInstanceState()。

因为 onSaveInstanceState()方法不总被调用，你应该仅使用 onSaveInstanceState()来记录活动的临时状态，而不是持久的数据。应该使用 onPause()来存储持久数据。

当一个活动启动另一个活动时，这两个活动都经历生命周期转换。一个暂停或是停止，然而被启动的活动则启动。有时，这些活动可能需要协调。当这两个活动在同一个进程中，生命周期的回调顺序是明确界定的：调用当前活动的 onPause()方法；然后，按序调用启动活动的 onCreate()、onStart()、onResume()方法；之后，如果该活动不需再在屏幕上可见，则调用它的 onStop()方法。下面我们就来详细学习一下关于 Android 如何管理多个 Activity。



(1) Android 用 Activity Stack 来管理多个 Activity，因此，同一时刻只会有最顶上的 Activity 是处于 active 或者 running 状态。其他的 Activity 都被压在下面。

(2) 如果非活动的 Activity 仍是可见的（如果上面压着的是一个非全屏的 Activity 或透明的 Activity），它是处于 paused 状态的。在系统内存不足的情况下，paused 状态的 Activity 是有可能被系统销毁掉的。

14.4 Activity 启动模式

Activity 作为 Android 中重要一环，它有 4 种不同的启动模式，类似于 C 语言中的局部变量、全局变量及静态变量等。这 4 种启动模式如下。

standard: 标准模式，调用 startActivity() 方法就会产生一个新的实例。

singleTop: 检查是否已经存在了一个实例位于 Activity Stack 的顶部，如果存在就不产生新的实例，反之则调用 Activity 的 newInstance() 方法产生一个新实例。

singleTask: 在一个新的 Task 中产生这个实例，以后每次调用都会使用此实例，而避免产生新的实例。

singleInstance: 这个基本上跟 singleTask 一样，只是有一点不同，那就是在这个模式下的 Activity 实例所处的 Task 中，只能有这一个 Activity 实例，而不能有其他的实例。

这些启动模式在 Android 清单文件 AndroidManifest.xml 中，通过<activity>中的 launchMode 属性进行设置，如代码清单 4-2 所示。

```
<activity android:name=".Activity2"
    android:launchMode="singleTask"></activity>
```

也可以在 Eclipse ADT 图形界面中编辑，如图 4-5 所示。

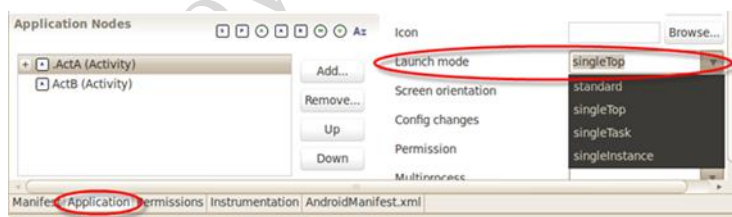


图 4-5 设置 Activity 启动模式

下面通过一个简单的例子——LaunchMode_Test 来对四种启动模式进行简要分析。在该例中涉及 Fx_Main、Activity2 及 Activity3 三个 Activity。

下面介绍一下例子中涉及的两个 Activity 及其界面。

首先是 Fx_Main，其界面如图 4-6 所示。



图 4-6 Fx_Main 的界面

在图 4-6 所示的界面中，单击“跳转到 AC2”按钮之后，跳转至 Activity2，具体代码如代码清单 4-3 所示。

代码清单 4-3 Fx_Main.Activity

```
import android.app.Activity;
import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.TextView;

public class Fx_Main extends Activity {
    /** Called when the activity is first created. */
    private Button b;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        // TODO Auto-generated method stub
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
        TextView tv=(TextView)findViewById(R.id.TextView01);
        tv.setText("Main---->" + getTaskId());
        Log.i("System.out", "Main---->" + this.toString() + "Task ID---->" + getTaskId());
        b=(Button)findViewById(R.id.Button01);
        b.setOnClickListener(new OnClickListener(){

            @Override
```



```
public void onClick(View v) {  
    // TODO Auto-generated method stub  
    Intent i=new Intent(Fx_Main.this,Activity2.class);  
    startActivity(i);  
    });  
}  
  
@Override  
protected void onDestroy() {  
    // TODO Auto-generated method stub  
    super.onDestroy();  
    Log.i("System.out", "Fx_Main--->Destory");  
}  
}
```

其次是 Activity2，其界面如图 4-7 所示。



图 4-7 Activity2 的界面

在该界面中，单击“跳回到 Main”按钮，则跳转至 Fx_Main，而单击“跳到本页面”则仍显示 Activity2 的界面，单击“跳到 AC3”则跳转到 Activity3 的界面，具体代码如代码清单 4-4 所示。

代码清单 4-4 Activity2.Activity

```
import android.app.Activity;  
import android.content.Intent;  
import android.os.Bundle;  
import android.util.Log;  
import android.view.View;
```



```
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.TextView;

public class Activity2 extends Activity {
    private Button b;
    private Button b2;
    private Button b3;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        // TODO Auto-generated method stub
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity2);
        b=(Button)findViewById(R.id.Button02);
        b2=(Button)findViewById(R.id.Button03);
        b3=(Button)findViewById(R.id.Button04);
        TextView tv=(TextView)findViewById(R.id.TextView02);
        tv.setText("Ac2---->"+getTaskId());
        Log.i("System.out", "Ac2---->"+this.toString()+"Task ID---->"+getTaskId());
        b.setOnClickListener(new OnClickListener(){

            @Override
            public void onClick(View v) {
                // TODO Auto-generated method stub
                Intent i=new Intent(Activity2.this,Fx_Main.class);
                startActivity(i);
            }
        });
        b2.setOnClickListener(new OnClickListener(){

            @Override
            public void onClick(View v) {
                // TODO Auto-generated method stub
                Intent i=new Intent(Activity2.this,Activity2.class);
                startActivity(i);
            }
        });
    }
}
```

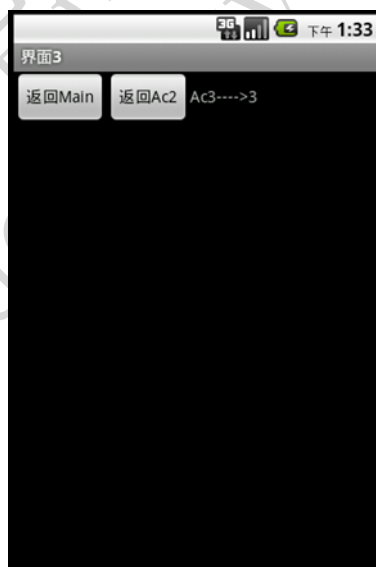


```
b3.setOnClickListener(new OnClickListener(){

    @Override
    public void onClick(View v) {
        // TODO Auto-generated method stub
        Intent i=new Intent(Activity2.this,Activity3.class);
        startActivity(i);
    });
}

@Override
protected void onDestroy() {
    // TODO Auto-generated method stub
    super.onDestroy();
    Log.i("System.out", "Ac2--->destory");
}
}
```

最后是 Activity3，其界面如图 4-8 所示。



如图 4-8 所示，单击“返回 Main”则跳转至 Fx_Main，单击“返回 AC2”，则跳转到 Activity2。具体代码如代码清单 4-5 所示。

代码清单 4-5 Activity3.Activity

```
import android.app.Activity;
import android.content.Intent;
```



```
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.TextView;

public class Activity3 extends Activity {
    private Button b;
    private Button b2;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        // TODO Auto-generated method stub
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity3);
        b=(Button)findViewById(R.id.Button03);
        b2=(Button)findViewById(R.id.Button04);
        TextView tv=(TextView)findViewById(R.id.TextView03);
        tv.setText("Ac3---->"+getTaskId());
        Log.i("System.out", "Ac3---->"+this.toString()+"Task ID---->"+getTaskId());
        b.setOnClickListener(new OnClickListener(){

            @Override
            public void onClick(View v) {
                // TODO Auto-generated method stub
                Intent i=new Intent(Activity3.this,Fx_Main.class);
                startActivity(i);
            }
        });
        b2.setOnClickListener(new OnClickListener(){

            @Override
            public void onClick(View v) {
                // TODO Auto-generated method stub
                Intent i=new Intent(Activity3.this,Activity2.class);
                startActivity(i);
            }
        });
    }
}
```



```

    });
}

@Override
protected void onDestroy() {
    // TODO Auto-generated method stub
    super.onDestroy();
    Log.i("System.out", "Ac3--->Destory");
}
}

```

14.4.1 standard 标准模式

在 standard 模式也就是默认模式下，不需要配置 launchMode。此时的 AndroidManifest.xml 如代码清单 4-6 所示。

代码清单 4-6 AndroidManifest.xml

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="feixun.com.jiang"
    android:versionCode="1"
    android:versionName="1.0">
    <application android:icon="@drawable/icon" android:label="@string/app_name">
        <activity android:name=".Fx_Main"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity android:name=".Activity2" android:label="@string/Ac2"/>
        <activity android:name=".Activity3" android:label="@string/Ac3"/>

    </application>
    <uses-sdk android:minSdkVersion="4" />

</manifest>

```

运行例子，从 Fx_Main 开始，一直点回到 Activity2 按钮时，Log 信息如图 4-9 所示。



Time	pid	tag	Message
05-02 ...	I 310	Syste...	Main-----> Fx_Main@43e09d38Task ID----->4
05-02 ...	I 310	Syste...	Ac2-----> Activity2@43e16dd0Task ID----->4
05-02 ...	I 310	Syste...	Ac2-----> Activity2@43e1f318Task ID----->4
05-02 ...	I 310	Syste...	Ac2-----> Activity2@43e26788Task ID----->4
05-02 ...	I 310	Syste...	Ac2-----> Activity2@43e2dc10Task ID----->4

图 4-9 Standard 启动模式下 Log 信息

发现每次都创建了 Activity2 的新实例。standard 的加载模式就是这样的，Intent 将发送给它新的 Activity 实例。

现在点击 Android 设备的回退键，可以看到 Log 信息按照刚才创建 Activity 实例的倒序依次出现，类似退栈的操作，而刚才操作跳转按钮的过程是压栈的操作。

14.4.2 singleTop

singleTop 和 standard 模式，都会将 Intent 发送到新的实例（如果已经有了，singleTask 模式和 singleInstance 模式不发送到新的实例）。不过，singleTop 要求如果创建 intent 时栈项已经有要创建 Activity 的实例，则将 Intent 发送给该实例，而不发送给新的实例。

还是用刚才的示例，只需将 Activity2 的 launchMode 改为 singleTop，就能看到区别。修改后 AndroidManifest.xml 中代码如代码清单 4-7 所示。

代码清单 4-7 AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="feixun.com.jiang"
    android:versionCode="1"
    android:versionName="1.0">
    <application android:icon="@drawable/icon" android:label="@string/app_name">
        <activity android:name=".Fx_Main"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity android:name=".Activity2" android:label="@string/Ac2"
            android:launchMode="singleTop" />
        <activity android:name=".Activity3" android:label="@string/Ac3"/>
    </application>
</uses-sdk android:minSdkVersion="4" />
```




</manifest>

运行 Fx_Main, 跳转到 Activity2---->Activity2 时会发现, 单击多少遍按钮, 都是相同的 Activity2 实例, 因为该实例在栈顶, 所以不会创建新的实例。如果回退, 回到 Fx_Main, 将退出应用, 如图 4-10 所示。

```
05-02 ... I 339 Syste... Main----> Fx_Main@43e09d38Task ID---->5
05-02 ... I 339 Syste... Ac2----> Activity2@43e16db8Task ID---->5
```

图 4-10 singleTop 模式下“跳转到 AC2”的 Log 信息

singleTop 模式, 可用来解决栈顶多个重复相同的 Activity 的问题。

如果是 Fx_Main 跳转到 Activity2, 再跳转到 Fx_Main, 行为就和 standard 一样了, 会在 Activity2 跳转到 Fx_Main 时创建 Fx_Main 的新实例, 因为当时的栈顶不是 Activity2 实例, 如图 4-11 所示。

```
05-02 ... I 339 Syste... Main----> Fx_Main@43e27960Task ID---->6
05-02 ... I 339 Syste... Ac2----> Activity2@43e2d590Task ID---->6
05-02 ... I 339 Syste... Main----> Fx_Main@43e34960Task ID---->6
```

图 4-11 singleTop 模式下“跳转到 AC2”后“跳回到 Main”的 Log 信息

14.4.3 singleTask

singleTask 模式和后面的 singleInstance 模式都是只创建一个实例的。

当 Intent 到来, 需要创建 singleTask 模式 Activity 时, 系统会检查栈里面是否已经有该 Activity 的实例。如果有直接将 Intent 发送给它 (注意此时原在此 Activity 栈中上面的 Activity 将会被关闭)。

把 Activity2 的启动模式改成 singleTask, 修改后 AndroidManifest.xml 中代码如代码清单 4-8 所示。

代码清单 4-8 AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="feixun.com.jiang"
    android:versionCode="1"
    android:versionName="1.0">
    <application android:icon="@drawable/icon" android:label="@string/app_name">
        <activity android:name=".Fx_Main"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity android:name=".Activity2" android:label="@string/Ac2"
```



```

        android:launchMode="singleTask"/>

        <activity android:name=".Activity3" android:label="@string/Ac3">

</application>

<uses-sdk android:minSdkVersion="4" />

</manifest>

```

启动 Fx_Main, 跳转到 Activity2---->Activity3---->Activity2, 此时看 Log 信息, 如图 4-12 所示。

```

05-02 ... I 398 Syste... Main----> Fx_Main@43e09d38Task ID---->8
05-02 ... I 398 Syste... Ac2----> Activity2@43e16dd0Task ID---->8
05-02 ... I 398 Syste... Ac3----> Activity3@43e1f550Task ID---->8
05-02 ... I 398 Syste... Activity3---->onDestroy

```

图 4-12 singleTask 启动模式下 Log 信息

可见从 AC3 再跳转到 AC2 时, 因为 AC2 之前在栈中是存在的所以不生成新的 AC2 实例, 而是在栈中找到此 AC2, 并将在 AC2 上面的 AC3 关闭, 所以此时栈中只有 Fx_Main 和 AC2, 在 AC2 点返回会直接退到 Fx_Main 然后退出。

14.4.4 singleInstance

在 singleInstance 模式下, 加载该 Activity 时如果没有实例化, 它会在创建新的 Task 后, 实例化入栈, 如果已经存在, 则直接调用 onNewIntent, 该 Activity 的 Task 中不允许启动其他的 Activity, 任何从该 Activity 启动的其他 Activity 都将被放到其他 Task 中, 先检查是否有在应用的 Task, 没有的话就创建。

在这里介绍一下 Task (任务) 的概念。按照字面意思, 任务就是自己要实现的一个目的, 而在 Android 中的 Task 的定义是一系列 Activity 的集合, 即要达到自己最终要到的 Activity, 之前所有经历过的 Activity 的集合。它可以是同一个应用内部的, 也可以是两个不同应用的。Task 可以认为是一个栈, 可放入多个 Activity。比如, 启动一个应用, 那么 Android 就创建了一个 Task, 然后启动这个应用的入口 Activity, 就是 intent-filter 中配置为 main 和 launch 的那个。这个 Activity 是根 (Root) Activity, 可能会在它的界面调用其他 Activity, 这些 Activity 如果按照上面那 3 个模式, 也会在这个栈 (Task) 中, 只是实例化的策略不同而已。

把 Activity2 的启动模式改成 singleInstance, 修改后 AndroidManifest.xml 中代码如代码清单 4-9 所示。

代码清单 4-9 AndroidManifest.xml

```

<?xml version="1.0" encoding="utf-8"?>

<manifest xmlns:android="http://schemas.android.com/apk/res/android"

    package="feixun.com.jiang"

    android:versionCode="1"

```



```

        android:versionName="1.0">
        <application android:icon="@drawable/icon" android:label="@string/app_name">
            <activity android:name=".Fx_Main"
                android:label="@string/app_name" >
                <intent-filter>
                    <action android:name="android.intent.action.MAIN" />
                    <category android:name="android.intent.category.LAUNCHER" />
                </intent-filter>
            </activity>
            <activity android:name=".Activity2" android:label="@string/Ac2"
                android:launchMode="singleInstance"/ >
            <activity android:name=".Activity3" android:label="@string/Ac3/>

        </application>
        <uses-sdk android:minSdkVersion="4" />

    </manifest>

```

然后进行测试，启动 Fx_Main---->Activity2---->Activity3 然后看一下 Log 信息，如图 4-13 所示。

图 4-13 singleInstance 启动模式下 Log 信息

可以看到 Fx_Main 以及 Activity3 的 Task ID 为 9，而 Activity2 的 Task ID 为 10，此时在 Activity3 单击“返回”按钮会发现先退到 Fx_Main，继续返回会回到 Activity2 最后退出。从该过程可以看出：如果从其他应用程序调用 singleInstance 模式的 Activity (Fx_Main)，从该 Activity 开启其他 Activity (Activity2) 时，会创建一个新的 Task (Task ID 为 10 的那个)，实际上，如果包含该 Activity (Activity2) 的 Task 已经运行的话，他会在该运行的 Task 中重新创建。

经过上述的介绍，用下面的表格来进行一个简单的总结，如表 4-2 所示。

表 4-2 Activity4 种启动模式对比

区别	是否允许多个实例	如何决定所属 Task	是否每次都生成新实例	是否允许其他 Activity 存在于本 Task 内
standard	可被多次实例化，同一个 Task	存放于 Start Activity()的	是	允许



	的不同的实例可位于不同的 Task 中，每个 Task 也可包含多个实例	Task。除非设置 FLAG_ACTIVITY_NEW_TASK 标记		
singleTop	同 standard	同 standard	如果寄存 Activity 的栈顶为该 Activity，则直接用该 Activity 处理；否则，创建新实例	允许
singleTask	不能有多个实例。由于该模式下 Activity 总是位于栈顶，所以 Activity 在同一个设备里最多只有一个实例	放入新的 Task 内，并且位于该 Task 的根	只有在第一次才创建新的实例，其他情况复用该 Activity	允许。如果存在 singleTask 的栈寄存 Task 内，响应一个 Intent 时，如果 singleTask 位于栈顶，则处理 Intent，否则会丢失 Intent，但该 Task 会处于前台
singleInstance	同 singleTask	同 singleTask	同 singleTask	不允许

14.5 程序调试

Android 系统提供了两种调试工具 LogCat 和 DevTools，用于定位、分析及修复程序中出现的错误。

14.5.1 LogCat 命令行工具

LogCat 是可以显示在 Eclipse 集成开发环境中的用来获取系统日志信息的工具。它的主要功能就是能够捕获包括 Dalvik 虚拟机产生的信息、进程信息、ActivityManager 信息、PackageManager 信息、Homeloader 信息、WindowsManager 信息、Android 运行时信息和应用程序信息等可被捕获的信息。



1. LogCat 的使用方法

打开方式：选择“Window”→“Show View”→“Other”命令，打开 Show View 的选择菜单，然后在 Andoird → LogCat 中选择 LogCat。打开 LogCat 后，它便显示在 Eclipse 的下方区域，其界面如图 4-14 所示。

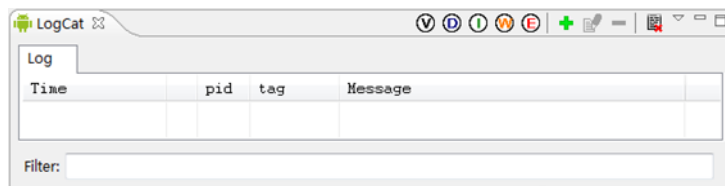


图 4-14 LogCat 界面

从图中我们可以看到 LogCat 的右上方有 5 个不同的字母，这 5 个字母分别表示 5 种不同类型的日志信息，它们的级别依次增高，表示含义如下。

V：详细（Verbose）信息。

D：调试（Debug）信息。

I：通告（Info）信息。

W：警告（Warn）信息。

E：错误（Error）信息。

在 LogCat 中，用户可以通过 5 个字母图标选择显示的信息类型，级别高于所选类型的信息也会在 LogCat 中显示，但级别低于所选类型的信息则不会被显示。

同时，LogCat 提供了“过滤”功能，在右上角的“+”号和“-”号，分别是添加和删除过滤器。用户可以根据日志信息的标签（Tag）、产生日志的进程编号（Pid）或信息等级（Level），对显示的日志内容进行过滤。

2. 程序调试原理

引入 android.util.Log 包。

使用 Log.v()、Log.d()、Log.i()、Log.w() 和 Log.e() 5 个方法在程序中设置“日志点”。

Log.v()用来记录详细信息。

Log.d()用来记录调试信息。

Log.i()用来记录通告信息。

Log.w()用来记录警告信息。

Log.e()用来记录错误信息。

当程序运行到“日志点”时，应用程序的日志信息便被发送到 LogCat 中。



判断“日志点”信息与预期的内容是否一致。

进而判断程序是否存在错误。

下面的例子演示了 Log 类的具体使用方法。

```
package com.example.LogCat;

import android.app.Activity;
import android.os.Bundle;
import android.util.Log;

public class LogCat extends Activity {
    final static String TAG = "LOGCAT";

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);

        Log.v(TAG, "Verbose");
        Log.d(TAG, "Debug");
        Log.i(TAG, "Info");
        Log.w(TAG, "Warn");
        Log.e(TAG, "Error");
    }
}
```

在本段代码中，程序第 5 行“import android.util.Log;”引入 android.util.Log 包；第 8 行定义标签，标签帮助用户在 LogCat 中找到目标程序生成的日志信息，同时也能够利用标签对日志进行过滤；第 14 行记录一个详细信息，Log.v()方法的第一个参数是日志的标签，第二个参数是实际的信息内容；第 15~18 行分别产生了调试信息、通告信息、警告信息和错误信息。

最终运行结果如图 4-15 所示，从图中还可以看出 LogCat 对不同类型的信息使用了不同的颜色加以区别。

Time	pid	tag	Message
06-07 00:33:43.700	D 656	dalvikvm	GC freed 2807 objects / 172736 bytes in 297ms
06-07 00:33:48.653	D 612	dalvikvm	GC freed 3111 objects / 189488 bytes in 228ms
06-07 00:34:01.919	I 570	ActivityManager	Starting activity: Intent { action=android.inte
06-07 00:34:02.000	V 714	LOGCAT	Verbose
06-07 00:34:02.000	D 714	LOGCAT	Debug
06-07 00:34:02.000	I 714	LOGCAT	Info
06-07 00:34:02.000	W 714	LOGCAT	Warn
06-07 00:34:02.000	E 714	LOGCAT	Error
06-07 00:34:02.216	W 612	IInputConnectionW...	showStatusIcon on inactive InputConnection
06-07 00:34:02.251	I 570	ActivityManager	Displayed activity edu.hrbeu.LogCat/.LogCat: 32
06-07 00:34:07.482	D 612	dalvikvm	GC freed 1842 objects / 98152 bytes in 208ms



图 4-15 LogCat 工程的运行结果

3. 添加过滤器

上文中提到 LogCat 提供了“过滤”功能，下面就来介绍一下 LogCat 是如何添加过滤器的。

首先，单击右上角的“+”，在弹出的对话框中填入过滤器的名称：LogcatFilter，设置过滤条件为“标签=LOGCAT”即可，操作方法如图 4-16 所示。

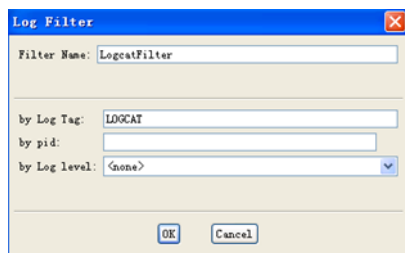


图 4-16 添加过滤器

经过上述过滤器过滤后，无论什么类型的日志信息，属于哪一个进程，只要标签为 LogCat，都将显示在 LogcatFilter 区域内。LogCat 过滤后的输入结果如图 4-17 所示。

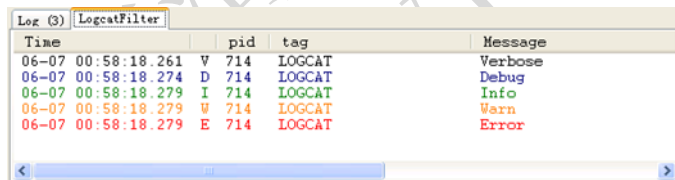


图 4-17 LogCat 过滤后的输入结果

14.5.2 DevTools 开发调试工具

DevTools 是用于调试和测试的工具，它包括了如下所示一系列各种用途的用户小工具：Development Settings、Exception Browser、Google Login Service、Instrumentation、Media Scanner、Package Browser、Pointer Location、Raw Image Viewer、Running processes 和 Terminal Emulator。

如图 4-18 所示，为 DevTools 使用时的界面。由使用时的界面也可以看出其中的各个小工具。

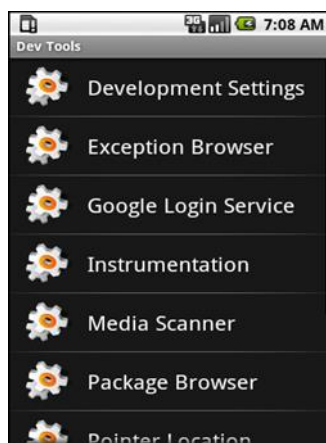


图 4-18 DevTools 的使用界面

以下着重讲解 Dev Tools 的一些小工具。

Development Settings

Development Settings 中包含了程序调试的相关选项，单击功能前面的选择框，出现绿色的“对号”表示功能启用，模拟器会自动保存设置。

图 4-19 显示了 Development Settings 的运行界面。

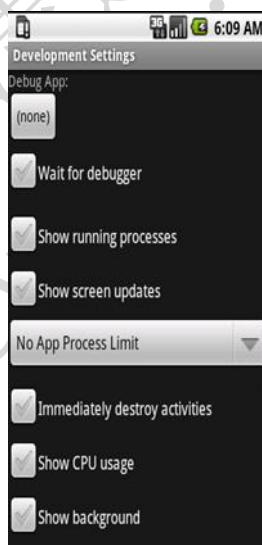


图 4-19 Development Settings 运行界面

下面就详细介绍 Development Settings 中各个选项的含义，如表 4-3 所示。

表 4-3 Development Settings 中各选项的含义

选项	说明
Debug App	为 Wait for debugger 选项指定应用程序，如果不指定（选择 none），Wait for debugger 选项将适用于所有应用程序。Debug App 可以有效地防止 Android 程序长时间停留在断点而产生异常



Wait for debugger	阻塞加载应用程序，直到关联到调试器（Debugger）。 用于在 Activity 的 onCreate()方法的进行断点调试
Show running processes	在屏幕右上角显示运行中的进程
Show screen updates	选中该选项时，界面上任何被重绘的矩形区域会闪现粉红色，有利于发现界面中不必要的重绘区域
No App Process limit	允许同时运行进程的数量上限
Immediately destroy activities	Activity 进入停止状态后立即销毁，用于测试在方法 onSaveInstanceState()、onRestoreInstanceState()和 onCreate()中的代码
Show CPU usage	在屏幕顶端显示 CPU 使用率，上层红线显示总的 CPU 使用率，下层绿线显示当前进程的 CPU 使用率
Show background	应用程序没有 Activity 显示时，直接显示背景面板，一般这种情况仅在调试时出现
Show Sleep state on LED	在休眠状态下开启 LED
Windows Animation Scale	窗口动画规模
Transition Animation	转换动画
Light Hinting	轻显示
Show GTalk service connection status	显示 GTalk 服务连接状态

Pointer Location

Pointer Location 是屏幕点位置查看工具，能够显示触摸点的 X 轴坐标和 Y 轴坐标，如图 4-20 所示。



图 4-20 Pointer Location 的使用画面

Running processes

Running processes 能够查看在 Android 系统中正在运行的进程，并能查看进程的详细信息，包括进程

名称和进程所调用的程序包。

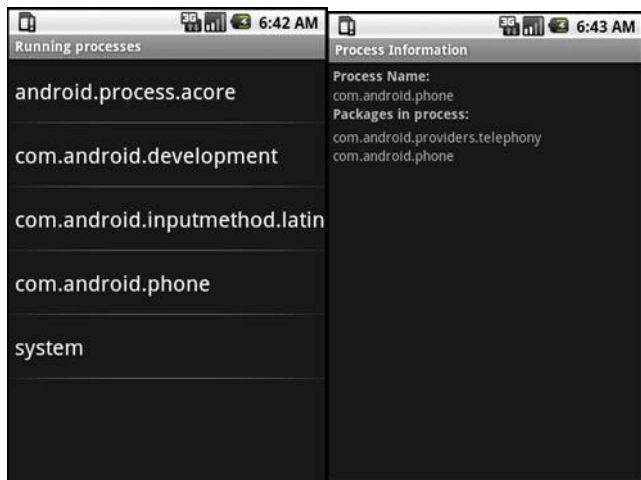


图 4-21 Andoird 模拟器默认情况下运行的进程和 com.android.phone 进程的详细信息

Terminal Emulator

Terminal Emulator 可以打开一个连接底层 Linux 系统的虚拟终端，但具有的权限较低，且不支持提升权限的 su 命令。如果需要使用 root 权限的命令，可以使用 ADB 工具。

图 4-22 是 Terminal Emulator 运行时的画面，输入 ls 命令，显示出根目录下的所有文件夹。

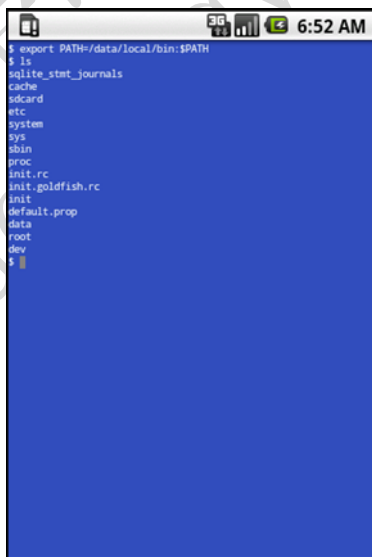


图 4-22 Terminal Emulator 运行时的画面

14.6 本章小结

本章主要介绍了 Android 系统的进程优先级排序、不同优先级进程之间的变化方式，Android 系统的 4 大基本组件及其用途，Activity 的生命周期中各个状态及状态间的变化关系、Android 应用程序的调试方法和工具。



第 15 章 用户界面开发

在上一章的学习中，主要了解了 Android 系统的进程优先级排序、不同优先级进程之间的变化方式，Android 系统的 4 大基本组件及其用途，Activity 的生命周期中各个状态及状态间的变化关系、Android 应用程序的调试方法和工具。在此基础上，本章将对 Android 程序界面开发的学习，包括用户界面基础、用户界面的控件的使用、界面布局的特点及使用方法、菜单的使用方法、界面事件的处理方法等。

15.1 用户界面基础

用户界面（User Interface，UI）是系统和用户之间进行信息交换的媒介，实现信息的内部形式与用户可以接收形式之间的转换。

在 Android 系统中，Android 自带有许多要求，预示着其用户界面的复杂性：它是一个支持多个并发应用程序的多处理系统，接受多种形式的输入，有着高交互性，必须具有足够的灵活性，以支持现在和未来广泛的设备。令人印象深刻的是丰富的用户界面及其易用性，实现了所有给定的功能。但为了使用应用程序在不同的设备上正常的显示以及运行，避免对系统性能造成过大的负担，应该明白其工作原理。

Android 使用 XML 文件描述用户界面；资源文件独立保存在资源文件夹中；对用户界面描述非常灵活，允许不明确定义界面元素的位置和尺寸，仅声明界面元素的相对位置和粗略尺寸。以下就来介绍一下 Android 的用户界面框架。

Android 是在 Java 环境中增加了一个图形用户界面（GUI）工具包，联合了 AWT，Swing，SWT，和 J2ME（撇开 Web UI 的工具包）。Android 框架和他们一样，它是单线程的，事件驱动的，并建立一个嵌套的组件库。

Android 用户界面框架（Android UI Framework），像其他的 UI 框架一样，采用了 MVC（Model-View-Controller）模型，提供了处理用户输入的控制器（Controller），显示用户界面和图像的视图（View），以及保存数据和代码的模型（Model）。

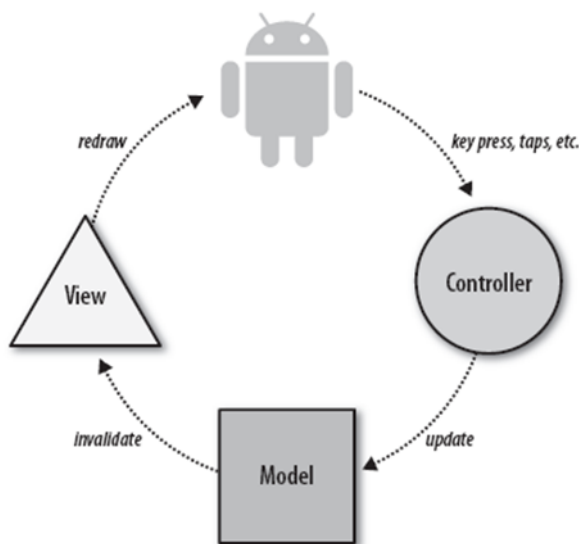




图 5-1 Android 用户界面框架 MVC 模型

其中 Model 是应用程序的核心。虽然特定应用程序的视图（View）和控制器（Controller）必然反映他们操纵的 Model，但一个 Model 可能是由几个不同的应用使用。想想看，例如，一个 MP3 播放器的应用程序以及一个将 MP3 文件转换成 WAV MP3 文件的程序，对于这两个应用程序，Model 包括它的 MP3 文件格式和编解码器。然而，前者的应用程序，有熟悉的停止，启动和暂停控制等操作。后者可能不会产生任何声音；相反，它会设置比特率的控制等。此时，他们的 Model 都是对所有的文件数据。

其中的控制器（Controller）能够接收并响应程序的外部动作，如按键动作或触摸屏动作等。控制器使用队列处理外部动作，每个外部动作作为一个对应的事件被加入队列中，然后 Android 用户界面框架按照“先进先出”的规则从队列中获取事件，并将这个事件分配给所对应的事件处理方法。例如，当用户按下他的手机上的键，Android 系统生成的 KeyEvent，并将其添加到事件队列中。最后，在之前已排队的事件被处理后，KeyEvent 是从队列中删除的，并作为当前选择 View 的 dispatchKeyEvent 方法的调用参数传递。一旦事件被分派到的焦点组件，该组件可能会采取适当的行动来改变程序的内部状态。例如，在 MP3 播放器应用程序中，当用户点击屏幕上的播放/暂停按钮时，触发该按钮的事件，处理方法可能更新 Model，恢复播放一些先前所选乐曲。

视图（View）是应用程序给用户的反馈。它负责应用程序的部分渲染显示，发送音频扬声器，产生触觉反馈等。视图部分应用视图树（View Tree）模型。视图树是由 Android 用户界面框架中的界面元素以一种树形结构组织在一起的，Android 系统会依据视图树的结构从上至下绘制每一个界面元素。每个元素负责对自身的绘制，如果元素包含子元素，该元素会通知其下所有子元素进行绘制。

下面就来详细介绍一下视图树。Android 当中的可视化界面单元，可分为“容器”与“非容器”两类，容器类继承 ViewGroup，非容器类则从 View 衍生出来，如图 5-2 所示。

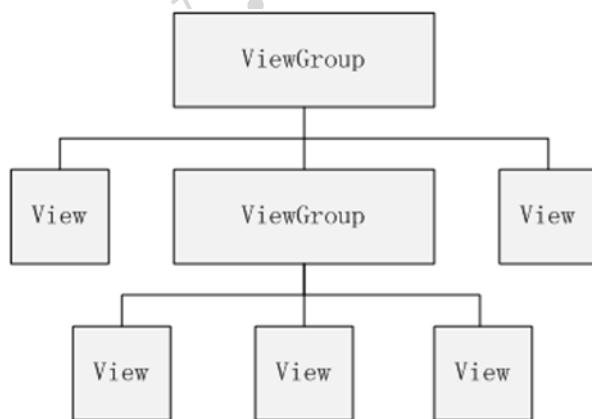


图 5-2 Android 视图树（View Tree）

视图树由 View 和 ViewGroup 构成。其中，View 是界面的最基本的可视单元，存储了屏幕上特定矩形区域内所显示内容的数据结构，并能够实现所占据区域的界面绘制、焦点变化、用户输入和界面事件处理等功能。同时 View 也是一个重要的基类，所有在界面上的可见元素都是 View 的子类。ViewGroup 是一种能够承载含多个 View 的显示单元，它承载了界面布局，同时还承载了具有原子特性的重构模块。

如图 5-3 所示，这些 Layout 可以套叠式地组成一棵视图树。其中，父节点的 Layout 与子节点的



LayoutParams 之间有控制关系，例如，若父节点是 RelativeLayout，则子节点的单元中可以指定 RelativeLayout.LayoutParams 中的属性，以控制子节点在父节点中的排列状况。

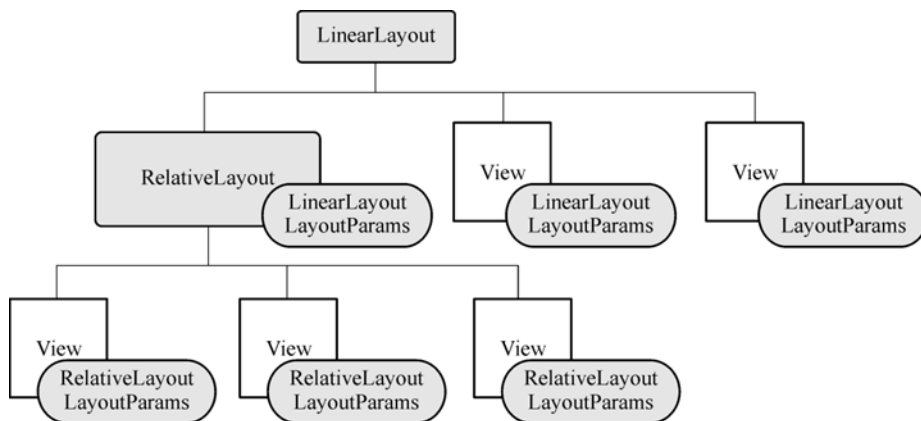


图 5-3 ViewGroup 树形层次结构

在单线程用户界面中，控制器从队列中获取事件和视图在屏幕上绘制用户界面，使用的都是同一个线程。这样的单线程用户界面使得处理方法具有顺序性，能够降低应用程序的复杂程度，同时也能降低开发的难度。

15.2 界面布局

界面布局（Layout）是用户界面结构的描述，定义了界面中所有的元素、结构和相互关系。

界面布局（Layout）是为了适应多种 Android 设备上的屏幕而设计的解决方案：它们可以有不同的像素密度、尺寸和不同的纵横比。典型的 Android 设备，如 HTC G1 手机，甚至允许应用程序运行时改变屏幕的方向（纵向或横向），因此布局的基础设施需要能够应对这种情况。布局的目的是为开发人员提供一种方式来表示 View 之间的物理关系，因为它们是在屏幕上绘制。作为 Android 的界面布局，它使用开发需求来满足与开发要求最接近的屏幕布局。

Android 开发者使用术语“布局”，指的是两种含意中的一种。布局的两种定义如下。

一种资源，它定义了要在屏幕上画什么。布局资源存储在应用程序的/res/layout 资源目录下的 XML 文件中。布局资源简单地说就是一个用于用户界面屏幕，或屏幕的一部分，以及内容的模板。

一种视图类，它主要是组织其他控件。这些布局类（LinearLayout，RelativeLayout，TableLayout 等）用于在屏幕上显示子控件，如文本控件、按钮或图片。

Eclipse 的 Android 开发插件包含了一个很方便的用于设计和预览布局资源的布局资源设计器。这个工具包括两个标签视图：布局视图允许你预览在不同的屏幕下及对于每一个方向控件会如何展现；XML 视图告诉你资源的 XML 定义。

这里有一些关于在 Eclipse 中使用布局资源编辑器的技巧。

使用概要（Outline）窗格在你的布局资源中添加和删除控件。

选择特定的控件（在预览或概要窗口）并使用属性窗格来调整特定控件的属性。

使用 XML 标签来直接编辑 XML 定义。



很重要的是要记住一点，Eclipse 布局资源编辑器不能完全精确的模拟出布局在最终用户设备上的呈现形式。对此，必须在适当配置的模拟器中测试，更重要的是在目标设备上测试。而且一些“复杂”控件，包括标签或视频查看器，也不能在 Eclipse 中预览。

声明 Android 程序的界面布局有两种方法。

使用 XML 文件描述界面布局。

在程序运行时动态添加或修改界面布局。

用户既可以独立使用任何一种声明界面布局的方式，也可以同时使用两种方式。

使用 XML 文件声明界面布局有以下 3 个特点：将程序的表现层和控制层分离；在后期修改用户界面时，无须更改程序的源代码；用户还能够通过可视化工具直接看到所设计的用户界面，有利于加快界面设计的过程，并且为界面设计与开发带来极大的便利性。

设计程序用户界面最方便且可维护的方式是创建 XML 布局资源。这个方法极大地简化了 UI 设计过程，将许多用户界面控件的布局，以及控件属性定义移到 XML 中，代替了写代码。它适应了 UI 设计师（更关心布局）和开发者（了解 Java 和实现应用程序功能）潜在的区别。开发者依然可以在必要时动态地改变屏幕内容。复杂控件，像 ListView 或 GridView，通常用程序动态地处理数据。

XML 布局资源必须存放在项目目录的/res/layout 下。对于每一屏（与某个活动紧密关联）都创建一个 XML 布局资源是一个通用的做法，但这并不是必需的。理论上来说，可以创建一个 XML 布局资源并在不同的活动中使用它，为屏幕提供不同的数据。如果需要的话，也可以分散布局资源并用另外一个文件包含它们。

现在把注意力转向对组织其他控件很有用的布局控件。Android 中 Layout 的列表，如表 5-1 所示。

表 5-1 Layout 分类表

Layout 类别	说明
线性布局 LinearLayout	线性（水平或垂直）排版的容器
框架布局 FrameLayout	单一界面的容器
表格布局 TableLayout	以表格方式排版的容器
相对布局 RelativeLayout	以相对坐标排版的容器
绝对布局 AbsoluteLayout	以绝对坐标排版的容器，不推荐使用

15.2.1 线性布局（LinearLayout）

线性布局是最简单的布局之一，它提供了控件水平或者垂直排列的模型。如图 5-4 所示，线性布局中，所有的子元素如果垂直排列，则每行仅包含一个界面元素；如果水平排列，则每列仅包含一个界面元素。



图 5-4 线性布局（LinearLayout）效果图

同时，使用此布局时可以通过设置控件的 **Weight** 参数控制各个控件在容器中的相对大小。

LinearLayout 布局的属性既可以在布局文件（XML）中设置，也可以通过成员方法进行设置。表 5-2 给出了 **LinearLayout** 常用的属性及这些属性的对应设置方法。

表 5-2 **LinearLayout** 常用属性及对应方法

属性名称	对应方法	描述
android:orientation	setOrientation(int)	设置线性布局的朝向，可取 horizontal
android:gravity	setGravity(int)	设置线性布局的内部元素的布局方式

在线性布局中可使用 **gravity** 属性来设置控件的对齐方式，**gravity** 可取的值及说明如表 5-3 所示。

提示：当需要为 **gravity** 设置多个值时，用“|”分隔即可。

表 5-3 **gravity** 可取的属性及说明

属性	说明
top	不改变控件大小，对齐到容器顶部
bottom	不改变控件大小，对齐到容器底部
left	不改变控件大小，对齐到容器左侧
right	不改变控件大小，对齐到容器右侧
center_vertical	不改变控件大小，对齐到容器纵向中央位置
center-horizontal	不改变控件大小，对齐到容器横向中央位置
center	不改变控件大小，对齐到容器中央位置
fill_vertical	若有可能，纵向拉伸以填满容器
fill_horizontal	若有可能，横向拉伸以填满容器
fill	若有可能，纵向横向同时拉伸以填满容器

以下用一个线性布局的例子来加深对线性布局的理解。

1. 创建一个名为 **LinearLayout** 的 Android 工程

包名称是 **edu.hrbeu.LinearLayout**，Activity 名称为 **LinearLayout**。为了能够完整体验创建线性布局的



过程，我们需要删除 Eclipse 自动建立的/res/layout/main.xml 文件，之后我们将手动创建一个 XML 布局文件。

2. 建立 XML 线性布局文件

首先，删除 Eclipse 自动建立的/res/layout/main.xml 文件；其次，建立用于显示垂直排列线性布局的 XML 文件：右击/res/layout 文件夹，选择“New”→“File”命令打开新文件建立向导，文件名为 main_vertical.xml，保存位置为 LinearLayout/res/layout，如图 5-5 所示。

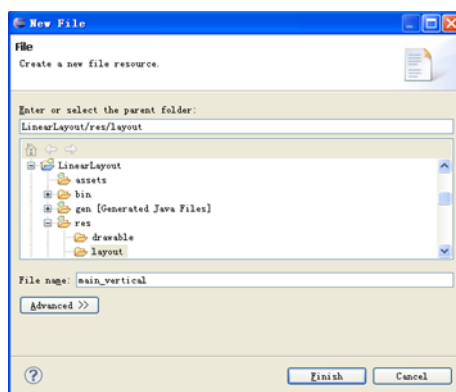


图 5-5 新建线性布局 XML 文件

3. 编辑 XML 线性布局文件

打开 XML 文件编辑器，对 main_vertical.xml 文件的代码做如代码清单 5-1 所示的修改。

代码清单 5-1 main_vertical.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">
</LinearLayout>
```

第 2 行代码是声明 XML 文件的根元素为线性布局；第 4、5、6 行代码是在属性编辑器中修改过的宽度、高度和排列方式的属性。同样地，用户可以在可视化编辑器和属性编辑器中对页面布局进行修改，这些修改会同步地反映在 XML 文件中。



4.添加控件

将四个界面控件 TextView、EditText、Button、Button 先后拖曳到可视化编辑器中，所有控件都自动获取控件名称，并把该名称显示在控件上，如 TextView01、EditText01、Button01 和 Button02。

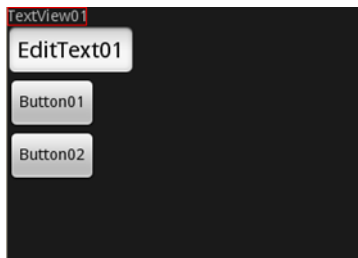


图 5-6 线性布局添加控件

修改界面控件的属性如表 5-4 所示。

表 5-4 线性布局控件属性

编号	类 型	属性	值
1	TextView	id	@+id/label
		text	用户名:
2	EditText	id	@+id/entry
		Layout_width	match_parent
3	Button	text	[null]
		id	@+id/ok
4	Button	text	确认
		id	@+id/cancel

打开 XML 文件编辑器查看 main_vertical.xml 文件代码，发现在属性编辑器内填入的文字已经正常写入 XML 文件中，如代码清单 5-2 中第 11、20、25 行代码。

代码清单 5-2 main_vertical.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <TextView android:id="@+id/label"
```



```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="用户名: " >
    </TextView>
    <EditText android:id="@+id/entry"
        android:layout_height="wrap_content"
        android:layout_width="match_parent">
    </EditText>
    <Button android:id="@+id/ok"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="确认">
    </Button>
    <Button android:id="@+id/cancel"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="取消" >
    </Button>
</LinearLayout>
```

5. 修改 LinearLayout.java 文件

将 LinearLayout.java 文件中的 setContentView(R.layout.main)，更改为 setContentView(R.layout.main_vertical)。

同理，按照以上步骤，可以得到横向线性布局。

建立 main_horizontal.xml 文件。

线性布局的 Orientation 属性的值设置为 horizontal。

将 EditText 的 Layout width 属性的值设置为 wrap_content。

将 LinearLayout.java 文件中的 setContentView(R.layout.main_vertical)修改为 setContentView(R.layout.main_horizontal)。

15.2.2 框架布局（FrameLayout）

框架布局（FrameLayout）是最简单的界面布局，它在屏幕上开辟出了一块区域，在这块区域中可以添加多个子控件，但是所有的子控件都被对齐到屏幕的左上角。框架布局的大小由子控件中尺寸最大的那个子控件来决定。如果子控件一样大，同一时刻只能看到最上面的子控件。

FrameLayout 继承自 ViewGroup，除了继承自父类的属性和方法，FrameLayout 类中包含了自己特有



的属性和方法，如表 5-5 所示。

表 5-5 FrameLayout 常用属性及对应方法

属性名称	对应方法	描述
android:foreground	setForeground(Drawable)	设置绘制在所有子控件之上的内容
android:foregroundGravity	setForegroundGravity(int)	设置绘制在所有子控件之上内容的 gravity

以下用一个 FrameLayout 的例子来加深对 FrameLayout 的理解。

在 Eclipse 中新建一个项目 FrameLayout。打开其 res/values 目录下的 strings.xml，在其中输入如代码清单 5-3 所示代码。在该段代码中声明了应用程序总会用到的字符串资源。

代码清单 5-3 strings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<string name="app_name">FrameExample</string>
<string name="big">大的</string>
<string name="middle">中的</string>
<string name="small">小的</string>
</resources>
```

在项目 rers/values 目录下新建一个 colors.xml，在其中输入如代码清单 5-4 所示代码。该段代码声明了应用程序中将会用到的颜色资源。这样将所有颜色资源统一管理有助于提高程序的可读性及可维护性。

代码清单 5-4 colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<color name="red">#FF0000</color>
<color name="green">#00FF00</color>
<color name="blue">#0000FF</color>
<color name="white">#FFFFFF</color>
</resources>
```

打开项目 res/layout 目录下的 main.xml 文件，将其中已有的代码替换为如代码清单 5-5 所示代码。

代码清单 5-5 main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout
android:id="@+id/FrameLayout01"
android:layout_width="match_parent"
```



```
android:layout_height="match_parent"
android:background="@color/white"
xmlns:android="http://schemas.android.com/apk/res/android"><
    <TextView
        android:text="@string/big"
        android:id="@+id/TextView01"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="60px"
        android:textColor="@color/green"
    > <!-- 声明一个 TextView 控件 -->
    </TextView>
    <TextView
        android:text="@string/middle"
        android:id="@+id/TextView02"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="40px"
        android:textColor="@color/red"
    > <!-- 声明一个 TextView 控件 -->
    </TextView>
    <TextView
        android:text="@string/small"
        android:id="@+id/TextView03"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="20px"
        android:textColor="@color/blue"
    > <!-- 声明一个 TextView 控件 -->
    </TextView>
</FrameLayout>
```

代码第 2~7 行声明了一个框架布局，并设置其在父控件中的显示方式及自身的背景颜色；代码第 8~16 行声明了一个 TextView 控件，该控件 ID 为 TextView01，第 13 行定义了其显示内容的字号为 60px，第 14 行定义了所显示内容的字体颜色为绿色；代码第 17~25 行声明了一个 TextView 控件，该控件 ID 为 TextView02，第 22 行定义了其显示内容的字号为 40px，第 23 行定义了所显示内容的字体颜色



为红色；代码第 26~34 行声明了一个 `TextView` 控件，该控件 id 为 `TextView03`，第 22 行定义了其显示内容的字号为 20px，第 23 行定义了所显示内容的字体颜色为蓝色。

运行程序，在图 5-7 所示的运行效果图中可以看到，程序运行时所有的子控件都自动地对齐到容器的左上角，由于子控件的 `TextView` 是按照字号从大到小排列的，所以字号小的在最上层。



图 5-7 框架布局运行效果图

15.2.3 表格布局（`TableLayout`）

`TableLayout` 类以行和列的形式管理控件，每行为一个 `TableRow` 对象，也可以为一个 `View` 对象，当为 `View` 对象时，该 `View` 对象将跨越该行的所有列。在 `TableRow` 中可以添加子控件，每添加一个子控件为一系列。

`TableLayout` 布局中并不会为每一行、每一列或每个单元格绘制边框，每一行可以有 0 或多个单元格，每个单元格为一个 `View` 对象。`TableLayout` 中可以有空的单元格，单元格也可以像 HTML 中那样跨越多个列。

图 5-8 是表格布局的示意图。

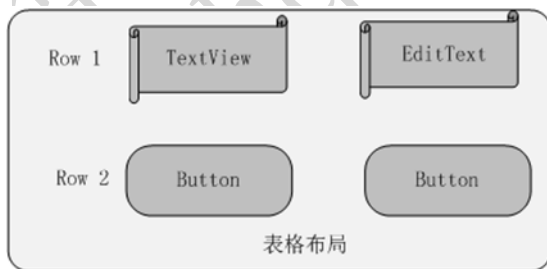


图 5-8 表格布局示意图

在表格布局中，一个列的宽度由该列中最宽的那个单元格指定，而表格的宽度是由父容器指定的。在 `TableLayout` 中，可以为列设置 3 种属性。

Shrinkable，如果一个列被标识为 `shrinkable`，则该列的宽度可以进行收缩，以使表格能够适应其父容器的大小。

Stretchable，如果一个列被标识为 `stretchable`，则该列的宽度可以进行拉伸，以填满表格中空闲的空间。

Collapsed，如果一个列被标识为 `collapsed`，则该列将会被隐藏。

`TableLayout` 继承自 `LinearLayout` 类，除了继承来自父类的属性和方法，`TableLayout` 类中还包含表格布局所特有的属性和方法。这些属性和方法说明如表 5-6 所示。



表 5-6 TableLayout 类常用属性及对应方法说明

属性名称	对应方法	描述
android:collapseColumns	setColumnCollapsed(int,boolean)	设置指定列号的列为 Collapsed，列号从 0
android:shrinkColumns	setShrinkAllColumns(boolean)	设置指定列号的列为 Shrinkable，列号从 0
android:stretchColumns	setStretchAllColumns(boolean)	设置指定列号的列为 Stretchable，列号从 0

以下我们用一个表格布局的例子来加深对表格布局的理解。

首先，建立表格布局要注意以下几点。

向界面中添加一个表格布局，无须修改布局的属性值。其中，ID 属性为 TableLayout01，Layout width 和 Layout height 属性都为 wrap_content。

向 TableLayout01 中添加两个 TableRow。TableRow 代表一个单独的行，每行被划分为几个小的单元，单元中可以添加一个界面控件。其中，ID 属性分别为 TableRow01 和 TableRow02，Layout width 和 Layout height 属性都为 wrap_content。

通过 Outline，向 TableRow01 中添加 TextView 和 EditText；向 TableRow02 中添加两个 Button。

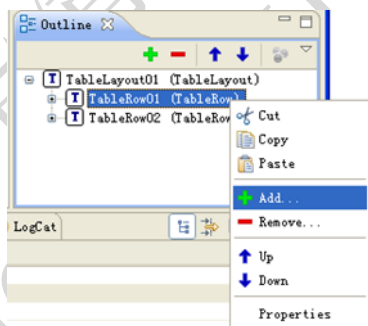


图 5-9 向 TableRow01 中添加 TextView 和 EditText

参考表 5-7 设置 TableRow 中 4 个界面控件的属性值。

表 5-7 表格布局控件属性

编号	类型	属性	值
1	TextView	id	@+id/label
		text	用户名:
		gravity	right
		padding	3dip
		Layout_width	160dip
2	EditText	id	@+id/entry
		text	[null]



		padding	3dip
		Layout_width	160dip
3	Button	id	@+id/ok
		text	确认
		padding	3dip
4	Button	id	@+id/cancel
		text	取消

代码清单 5-6 main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<TableLayout android:id="@+id/TableLayout01"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    xmlns:android="http://schemas.android.com/apk/res/android">
    <TableRow android:id="@+id/TableRow01"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content">
        <TextView android:id="@+id/label"
            android:layout_height="wrap_content"
            android:layout_width="160dip"
            android:gravity="right"
            android:text="用户名: "
            android:padding="3dip" >
        </TextView>
        <EditText android:id="@+id/entry"
            android:layout_height="wrap_content"
            android:layout_width="160dip"
            android:padding="3dip" >
        </EditText>
    </TableRow>
    <TableRow android:id="@+id/TableRow02"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content">
        <Button android:id="@+id/ok"
            android:layout_height="wrap_content"
```



```

        android:padding="3dip"
        android:text="确认">
    </Button>
    <Button android:id="@+id/Button02"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:padding="3dip"
        android:text="取消">
    </Button>
</TableRow>
</TableLayout>

```

代码中，第 3 行代码使用了<TableLayout>标签声明表格布局；第 7 行和第 23 行代码声明了两个 TableRow 元素；第 12 行设定宽度属性 android:layout_width: 160dip；第 13 行设定属性 android:gravity，指定文字为右对齐；第 15 行使用属性 android:padding，声明 TextView 元素与其他元素的间隔距离为 3dip。

表格布局运行效果如图 5-10 所示。



图 5-10 表格布局运行图

15.2.4 相对布局（RelativeLayout）

相对布局（RelativeLayout）是一种非常灵活的布局方式，能够通过指定界面元素与其他元素的相对位置关系，确定界面中所有元素的布局位置，能够最大限度保证在各种屏幕类型的手机上正确显示界面布局。

在相对布局中，子控件的位置是相对兄弟控件或父容器而决定的。出于性能考虑，在设计相对布局时要按照控件之间的依赖关系排列，如 View A 的位置相对于 View B 来决定，则需要保证在布局文件中 View B 在 View A 的前面。

在进行相对布局时用到的属性很多，首先来看属性值只为 true 或 false 的属性，如表 5-8 所示。



表 5-8 相对布局中只取 true 或 false 的属性及说明

属性名称	属性说明
android:layout_centerHorizontal	当前控件位于父控件的横向中间位置
android:layout_centerVertical	当前控件位于父控件的纵向中间位置
android:layout_centerInParent	当前控件位于父控件的中央位置
android:layout_alignParentBottom	当前控件底端与父控件底端对齐

接下来看属性值为其他控件 ID 的属性，如表 5-9 所示。

表 5-9 相对布局中取值为其他控件 ID 的属性及说明

属性名称	属性说明
android:layout_toRightOf	使当前控件位于给出 ID
android:layout_toLeftOf	使当前控件位于给出 ID
android:layout_above	使当前控件位于给出 ID
android:layout_below	使当前控件位于给出 ID
android:layout_alignTop	使当前控件的上边界位于给出 ID
android:layout_alignBottom	使当前控件的下边界位于给出 ID
android:layout_alignLeft	使当前控件的左边界位于给出 ID
android:layout_alignRight	使当前控件的右边界位于给出 ID

最后要介绍的是属性值以像素为单位的属性及说明，如表 5-10 所示。

表 5-10 相对布局中取值为像素的属性及说明

属性名称	属性说明
android:layout_marginLeft	当前控件左侧的留白
android:layout_marginRight	当前控件右侧的留白
android:layout_marginTop	当前控件上方的留白
android:layout_marginBottom	当前控件下方的留白

需要注意的是在进行相对布局时要避免出现循环依赖，例如，设置相对布局在父容器中的排列方式为 WRAP_CONTENT，就不能再将相对布局的子控件设置为 ALIGN_PARENT_BOTTOM。因为这样会造成子控件和父控件相互依赖和参照的错误。

以下用一个相对布局的例子来加深对线性布局的理解。首先来看一下相对布局的效果图，如图 5-11 所示。

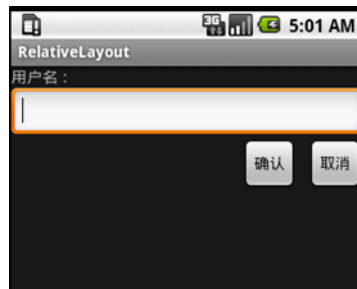


图 5-11 相对布局效果图

为达到以上效果，按以下步骤进行操作。

添加 TextView 控件（用户名），相对布局会将 TextView 控件放置在屏幕的最上方。

添加 EditText 控件（输入框），并声明该控件的位置在 TextView 控件的下方，相对布局会根据 TextView 的位置确定 EditText 控件的位置。

添加第一个 Button 控件（“取消”按钮），声明在 EditText 控件的下方，且在父控件的最右边。

添加第二个 Button 控件（“确认”按钮），声明该控件在第一个 Button 控件的左方，且与第一个 Button 控件处于相同的水平位置。

相对布局在 main.xml 文件的完整代码如代码清单 5-7 所示。

代码清单 5-7 main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<RelativeLayout android:id="@+id/RelativeLayout01"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    xmlns:android="http://schemas.android.com/apk/res/android">
    <TextView android:id="@+id/label"
        android:layout_height="wrap_content"
        android:layout_width="match_parent"
        android:text="用户名: ">
    </TextView>
    <EditText android:id="@+id/entry"
        android:layout_height="wrap_content"
        android:layout_width="match_parent"
        android:layout_below="@id/label">
    </EditText>
    <Button android:id="@+id/cancel"
        android:layout_height="wrap_content"
        android:layout_width="wrap_content"
```



```

        android:layout_alignParentRight="true"
        android:layout_marginLeft="10dip"
        android:layout_below="@id/entry"
        android:text="取消" >
    </Button>

    <Button android:id="@+id/ok"
        android:layout_height="wrap_content"
        android:layout_width="wrap_content"
        android:layout_toLeftOf="@id/cancel"
        android:layout_alignTop="@id/cancel"
        android:text="确认">
    </Button>
</RelativeLayout>

```

在代码中，第 3 行使用了<RelativeLayout>标签声明一个相对布局；第 15 行使用位置属性 android:layout_below，确定 EditText 控件在 ID 为 label 的元素下方；第 20 行使用属性 android:layout_alignParentRight，声明该元素在其父元素的右边边界对齐；第 21 行设定属性 android:layout_marginLeft，左移 10dip；第 22 行声明该元素在 ID 为 entry 的元素下方；第 28 行声明使用属性 android:layout_toLeftOf，声明该元素在 ID 为 cancel 元素的左边；第 29 行使用属性 android:layout_alignTop，声明该元素与 ID 为 cancel 的元素在相同的水平位置。

15.2.5 绝对布局（AbsoluteLayout）

绝对布局（AbsoluteLayout）能通过指定界面元素的坐标位置，来确定用户界面的整体布局。所谓绝对布局，是指屏幕中所有控件的摆放由开发人员通过设置控件的坐标来指定，控件容器不再负责管理其子控件的位置。由于子控件的位置和布局都通过坐标来指定，因此 AbsoluteLayout 类中并没有开发特有的属性和方法。

绝对布局是一种不推荐使用的界面布局，因为通过 X 轴和 Y 轴确定界面元素位置后，Android 系统不能够根据不同屏幕对界面元素的位置进行调整，降低了界面布局对不同类型和尺寸屏幕的适应能力。每一个界面控件都必须指定坐标（X，Y），例如图 5-12 中，“确认”按钮的坐标是（40，120），“取消”按钮的坐标是（120，120）。坐标原点（0，0）在屏幕的左上角。



图 5-12 绝对布局效果图

绝对布局示例在 main.xml 文件的完整代码如代码清单 5-8 所示。

代码清单 5-8 main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<AbsoluteLayout android:id="@+id/AbsoluteLayout01"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    xmlns:android="http://schemas.android.com/apk/res/android">
    <TextView android:id="@+id/label"
        android:layout_x="40dip"
        android:layout_y="40dip"
        android:layout_height="wrap_content"
        android:layout_width="wrap_content"
        android:text="用户名: ">
    </TextView>
    <EditText android:id="@+id/entry"
        android:layout_x="40dip"
        android:layout_y="60dip"
        android:layout_height="wrap_content"
        android:layout_width="150dip">
    </EditText>
    <Button android:id="@+id/ok"
        android:layout_width="70dip"
        android:layout_height="wrap_content"
        android:layout_x="40dip"
        android:layout_y="120dip"
```



```
        android:text="确认">
    </Button>
    <Button android:id="@+id/cancel"
        android:layout_width="70dip"
        android:layout_height="wrap_content"
        android:layout_x="120dip"
        android:layout_y="120dip"
        android:text="取消">
    </Button>
</AbsoluteLayout>
```

上述涉及的界面布局（`LinearLayout`，`TableLayout`，`RelativeLayout` 等）像其他控件一样也是一个控件。这意味着布局控件可以被嵌套。比如，为了组织屏幕上的控件你可以在一个 `LinearLayout` 中使用一个 `RelativeLayout`，反过来也行。但是需注意在界面设计过程中，尽量保证屏幕相对简单，复杂布局加载很慢并且可能引起性能问题。

同时，在设计程序布局资源时需要考虑设备的差异性。通常情况下是可能设计出在各种不同设备上看着都不错的灵活布局的，不管是竖屏还是横屏模式。必要时可以引入可选布局资源来处理特殊情况。例如，可以根据设备的方向或设备是不是有超大屏幕（如网络平板）来提供不同的布局供加载。

Android SDK 提供了几个可以帮助我们设计、调试和优化布局资源的工具。除了 Eclipse 的 Android 插件中内置的布局资源设计器，还可以使用 Android SDK 提供的 `Hierarchy Viewer`（层次结构查看器）和 `layoutopt`。这些工具在 Android SDK 的 `/tools` 目录下可以找到。可以使用 `Hierarchy Viewer` 来查看布局运行时的详细情况；可以使用 `layoutopt`（布局优化）命令行工具来优化你的布局文件。优化布局非常重要，因为复杂的布局文件加载很慢。`layoutopt` 工具简单地扫描 XML 布局文件并找出不必要的控件。在 Android 开发者网站的 `layoutopt` 部分查看更多信息。

15.3 界面控件

Android 系统的界面控件分为定制控件和系统控件。

定制控件是用户独立开发的控件，或通过继承并修改系统控件后所产生的新控件。能够为用户提供特殊的功能或与不同的显示需求方式；系统控件是 Android 系统提供给用户已经封装的界面控件，它提供应用程序开发过程中常见功能控件。同时，系统控件更有利于帮助用户进行快速开发，能够使 Android 系统中应用程序的界面保持一致性。

这里着重讲解一下系统控件的使用。

常见的系统控件包括 `TextView`、`EditText`、`Button`、`ImageButton`、`Checkbox`、`RadioButton`、`Spinner`、`ListView` 和 `TabHost`。

15.3.1 `TextView` 和 `EditText`

`TextView` 是一种用于显示字符串的控件；`EditText` 则是用来输入和编辑字符串的控件,它是一个具有



编辑功能的 TextView。

每个 TextView 期望的这样一个组件的属性：可以改变它的高度、宽度、字体、文字颜色、背景颜色等。TextView 也有一些有用的独特属性，如表 5-11 所示。

表 5-11 TextView 也有一些有用的独特属性

属性名称	属性说明
autoLink	如果设置 (TRUE)，发现文本中所显示的 URL，并自动将它们转换为可点击链接
autoText	如果设置 (TRUE)，发现并纠正在文本简单的拼写错误
editable	如果设置 (TRUE)，表示程序已定义的输入方法来接收输入文字（对 TextView 来说默认是 false，对 EditText 来说默认是 true）
inputMethod	标识的输入法（EditText 上定义一个通用文本）

下面就通过一个例子来加深对这两个控件的理解。

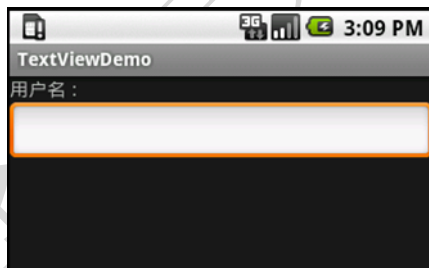


图 5-13 TextView 与 EditText 效果图

首先，建立一个“TextViewDemo”的程序，包含 TextView 和 EditText 两个控件，如图 5-13 所示。上方“用户名”部分使用的是 TextView，下方的文字输入框使用的是 EditText。

TextViewDemo 在 XML 文件中的代码如代码清单 5-9 所示。

代码清单 5-9 main.xml

```
<TextView android:id="@+id/TextView01"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="TextView01" >
</TextView>
<EditText android:id="@+id/EditText01"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="EditText01" >
</EditText>
```



在上述代码中，第 1 行 `android:id` 属性声明了 `TextView` 的 ID，这个 ID 主要用于在代码中引用这个 `TextView` 对象；“`@+id/TextView01`”表示所设置的 ID 值；`@`表示后面的字符串是 ID 资源；加号（+）表示需要建立新资源名称，并添加到 `R.java` 文件中；斜杠后面的字符串（`TextView01`）表示新资源的名称；如果资源不是新添加的，或属于 Android 框架的 ID 资源，则不需要使用加号（+），对于 Android 框架中的 ID 资源，还必须添加 Android 包的命名空间，如 `android:id="@android:id/empty"`。

第 2 行的 `android:layout_width` 属性用来设置 `TextView` 的宽度，`wrap_content` 表示 `TextView` 的宽度只要能够包含所显示的字符串即可。

第 3 行的 `android:layout_height` 属性用来设置 `TextView` 的高度。

第 4 行表示 `TextView` 所显示的字符串，在后面将通过代码更改 `TextView` 的显示内容。

第 7 行中“`fill_content`”表示 `EditText` 的宽度将等于父控件的宽度。

在上述步骤之后，修改 `TextViewDemo.java` 文件中代码为代码清单 5-10 所示的代码：

代码清单 5-10 `TextViewDemo.java`

```
TextView textView = (TextView)findViewById(R.id.TextView01);
EditText editText = (EditText)findViewById(R.id.EditText01);
textView.setText("用户名：");
editText.setText("");
```

第 1 行代码的 `findViewById()` 方法能够通过 ID 引用界面上的任何控件，只要该控件在 XML 文件中定义过 ID 即可。

第 3 行代码的 `setText()` 方法用来设置 `TextView` 所显示的内容。

15.3.2 Button 和 ImageButton

`Button` 是一种按钮控件，用户能够在该控件上点击，并后引发相应的事件处理方法；`ImageButton` 用以实现能够显示图像功能的控件按钮。

下面通过一个例子来加深对这两个控件的理解。

建立一个“`ButtonDemo`”的程序

程序包含 `Button` 和 `ImageButton` 两个按钮，上方是“`Button` 按钮”，下方是一个 `ImageButton` 控件，如图 5-14 所示。



图 5-14 `Button` 与 `ImageButton` 效果图



ButtonDemo 在 XML 文件中的代码如代码清单 5-11 所示。

代码清单 5-11 main.xml

```
<Button android:id="@+id/Button01"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Button01" >
</Button>
<ImageButton android:id="@+id/ImageButton01"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content">
</ImageButton>
```

在上述代码中，定义 Button 控件的高度、宽度和内容及 ImageButton 控件的高度和宽度，但是没定义显示的图像，在后面的代码中进行定义。

引入资源

将 download.png 文件复制到/res/drawable 文件夹下，在/res 目录上选择 Refresh，就可以看到新添加的文件显示在/res/drawable 文件夹下，同时 R.java 文件内容也得到了更新，否则提示无法找到资源的错误。

更改 Button 和 ImageButton 内容

在 ButtonDemo.java 中引入 android.widget.Button 和 android.widget.ImageButton，并修改其代码如代码清单 5-12 所示。

代码清单 5-12 ButtonDemo.java

```
Button button = (Button)findViewById(R.id.Button01);
ImageButton imageButton = (ImageButton)findViewById(R.id.ImageButton01);
button.setText("Button 按钮");
imageButton.setImageResource(R.drawable.download);
```

上述代码中，第 1 行代码用于引用在 XML 文件中定义的 Button 控件。

第 2 行代码用于引用在 XML 文件中定义的 ImageButton 控件。

第 3 行代码将 Button 的显示内容更改为“Button 按钮”。

第 4 行代码利用 setImageResource() 方法，将新加入的 png 文件 R.drawable.download 传递给 ImageButton。

按钮响应点击事件：添加点击事件的监听器

在 ButtonDemo.java 中添加代码清单 5-13 所示的代码。

代码清单 5-13 ButtonDemo.java

```
final TextView textView = (TextView)findViewById(R.id.TextView01);
button.setOnClickListener(new View.OnClickListener() {
```




```
public void onClick(View view) {  
    textView.setText("Button 按钮");  
}  
});  
imageButton.setOnClickListener(new View.OnClickListener() {  
    public void onClick(View view) {  
        textView.setText("ImageButton 按钮");  
    }  
});
```

在本段代码中，第 2 行代码中 button 对象通过调用 `setOnClickListener()` 方法，注册一个点击（Click）事件的监听器 `View.OnClickListener()`。

第 3 行代码是点击事件的回调方法。

第 4 行代码将 `TextView` 的显示内容更改为“Button 按钮”。

这里我们来了解一下 `View.OnClickListener()`。

`View.OnClickListener()` 是 `View` 定义的点击事件的监听器接口，并在接口中仅定义了 `onClick()` 方法。当 `Button` 从 `Android` 界面框架中接收到事件后，首先检查这个事件是否是点击事件，如果是点击事件，同时 `Button` 又注册了监听器，则会调用该监听器中的 `onClick()` 方法。每个 `View` 仅可以注册一个点击事件的监听器，如果使用 `setOnClickListener()` 方法注册第二个点击事件的监听器，之前注册的监听器将被自动注销。

多个按钮注册到同一个点击事件的监听器上，代码如代码清单 5-14 所示。

代码清单 5-14 多个按钮注册到一个点击事件的监听器上

```
Button.OnClickListener buttonListener = new Button.OnClickListener(){  
    @Override  
    public void onClick(View v) {  
        switch(v.getId()){  
            case R.id.Button01:  
                textView.setText("Button 按钮");  
                return;  
            case R.id.ImageButton01:  
                textView.setText("ImageButton 按钮");  
                return;  
        }  
    }  
});  
Button.setOnClickListener(buttonListener);  
ImageButton.setOnClickListener(buttonListener);
```



该段代码中，第 1 行至第 12 行代码定义了一个名为 `buttonListener` 的点击事件监听器；第 13 行代码将该监听器注册到 `Button` 上；第 14 行代码将该监听器注册到 `ImageButton` 上。

15.3.3 CheckBox 和 RadioButton

`CheckBox` 是一个同时可以选择多个选项的控件；而 `RadioButton` 则是仅可以选择一个选项的控件；`RadioGroup` 是 `RadioButton` 的承载体，程序运行时不可见，应用程序中可能包含一个或多个 `RadioGroup`，一个 `RadioGroup` 包含多个 `RadioButton`，在每个 `RadioGroup` 中，用户仅能够选择其中一个 `RadioButton`。

下面就通过一个例子来加深对这两个控件的理解，其效果如图 5-15 所示。

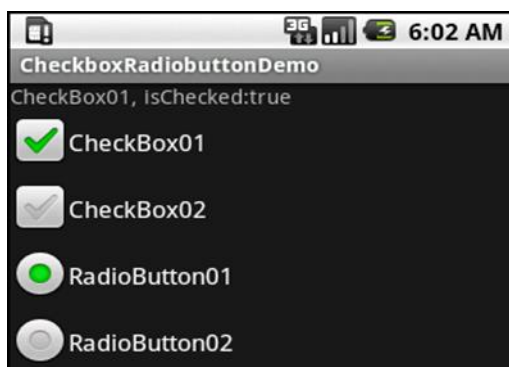


图 5-15 CheckBox 与 RadioButton 效果图

建立一个“`CheckboxRadiobuttonDemo`”程序

程序包含 5 个控件，从上至下分别是 `TextView01`、`CheckBox01`、`CheckBox02`、`RadioButton01`、`RadioButton02`，当选择 `RadioButton01` 时，`RadioButton02` 则无法选择。

`CheckboxRadiobuttonDemo` 在 XML 文件中的代码如代码清单 5-15 所示。

代码清单 5-15 main.xml

```
<TextView android:id="@+id/TextView01"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/hello"/>
<CheckBox android:id="@+id/CheckBox01"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="CheckBox01" >
</CheckBox>
<CheckBox android:id="@+id/CheckBox02"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
```



```
        android:text="CheckBox02" >
    </CheckBox>
    <RadioGroup android:id="@+id/RadioGroup01"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content">
        <RadioButton android:id="@+id/RadioButton01"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="RadioButton01" >
        </RadioButton>
        <RadioButton android:id="@+id/RadioButton02"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="RadioButton02" >
        </RadioButton>
    </RadioGroup>
```

上述代码中，第 15 行<RadioGroup>标签声明了一个 RadioGroup；在第 18 行和第 23 行分别声明了两个 RadioButton，这两个 RadioButton 是 RadioGroup 的子元素。

引用 CheckBox 和 RadioButton

引用 CheckBox 和 RadioButton 的方法参考代码清单 5-16 所示的代码。

代码清单 5-16 引用 CheckBox 和 RadioButton

```
CheckBox checkBox1= (CheckBox)findViewById(R.id.CheckBox01);
RadioButton radioButton1 =(RadioButton)findViewById(R.id.RadioButton01);
```

响应点击事件：添加点击事件的监听器

CheckBox 设置点击事件监听器的方法与 Button 设置点击事件监听器中介绍的方法相似，唯一不同在于将 Button.OnClickListener 换成了 CheckBox.OnClickListener。

代码清单 5-17 设置 CheckBox 点击事件监听器

```
CheckBox.OnClickListener checkboxListener = new CheckBox.OnClickListener(){
    @Override
    public void onClick(View v) {
        //过程代码
    };
    checkBox1.setOnClickListener(checkboxListener);
    checkBox2.setOnClickListener(checkboxListener);
```

RadioButton 设置点击事件监听器的方法如代码清单 5-18 所示。

代码清单 5-18 设置 RadioButton 点击事件监听器

```
RadioButton.OnClickListener radioButtonListener = new RadioButton.OnClickListener(){
    @Override
    public void onClick(View v) {
        //过程代码
    };
    radioButton1.setOnClickListener(radioButtonListener);
    radioButton2.setOnClickListener(radioButtonListener);
```

通过上述的讲解，可以得出这样的结论：CheckBox 是可以选择多个选项的复选框控件，当其中选项被选中时，显示相应的 checkmark。这时，需要创建一个“OnClickListener”捕获点击事件，并可以添加所需的功能代码。

RadioGroup 是一个包含一些 RadioButton 的 ViewGroup。用户可选择的一个按钮，通过对每一个 RadioButton 设置监听 OnClickListener 来获取其选择。这里需注意，点击 RadioButton 并不触发 RadioGroup 的 Click 事件。

15.3.4 Spinner

Spinner 是一种能够从多个选项中选择选项的控件，类似于桌面程序的组合框（ComboBox），但没有组合框的下拉菜单，而是使用浮动菜单为用户提供选择，如图 5-16 所示。

下面就通过一个例子来加深对 Spinner 的理解。

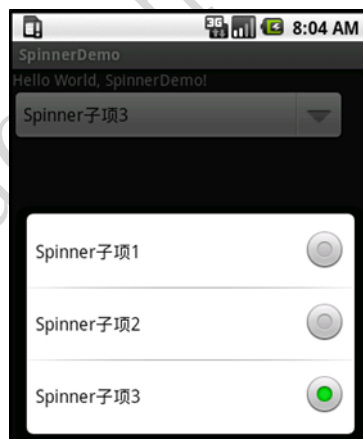


图 5-16 Spinner 效果图

建立一个程序“SpinnerDemo”

程序包含 3 个子项，Spinner 控件在 XML 文件中的代码如代码清单 5-19 所示。

代码清单 5-19 main.xml

```
<TextView android:id="@+id/TextView01"
    android:layout_width="match_parent"
```



```
        android:layout_height="wrap_content"
        android:text="@string/hello"/>
<Spinner android:id="@+id/Spinner01"
        android:layout_width="300dip"
        android:layout_height="wrap_content">
</Spinner>
```

在上述代码中，第 5 行使用<Spinner>标签声明了一个 Spinner 控件；第 6 行代码中指定了该控件的宽度为“300dip”。

修改 SpinnerDemo.java 文件

在 SpinnerDemo.java 文件中，定义一个 ArrayAdapter 适配器，在 ArrayAdapter 中添加 Spinner 的内容，需要在代码中引入 android.widget.ArrayAdapter 和 android.widget.Spinner。

代码清单 5-20 SpinnerDemo.java

```
Spinner spinner = (Spinner) findViewById(R.id.Spinner01);
List<String> list = new ArrayList<String>();
list.add("Spinner 子项 1");
list.add("Spinner 子项 2");
list.add("Spinner 子项 3");
ArrayAdapter<String> adapter = new ArrayAdapter<String>(this,
        android.R.layout.simple_spinner_item, list );
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
spinner.setAdapter(adapter);
```

本段代码中，第 2 行代码建立了一个字符串数组列表（ArrayList），这种数组列表可以根据需要进行增减，<String>表示数组列表中保存的是字符串类型的数据。

第 6 行代码建立了一个 ArrayAdapter 的数组适配器，数组适配器能够将界面控件和底层数据绑定在一起。

第 7 行代码设定了 Spinner 的浮动菜单的显示方式，其中，android.R.layout.simple_spinner_dropdown_item 是 Android 系统内置的一种浮动菜单。

第 8 行代码实现绑定过程，所有 ArrayList 中的数据，将显示在 Spinner 的浮动菜单中。

利用该段代码，适配器绑定界面控件和底层数据，如果底层数据更改了，用户界面也相应修改显示内容，因此不需要应用程序再监视，从而极大地简化了代码的复杂性。

由上述例子可以得出结论：与上一小节中的 CheckBox 和 RadioButton 相比，Spinner 需要的工作量最大，但可以为用户提供相对来说较好的屏幕显示。如上所示，Spinner 显示当前选中的选项，当单击右侧的下拉列表时，弹出一个可供选择的选项列表。为了实现该功能需满足以下条件。

创建一个可供选择的选项列表（该列表可以是动态创建并被应用程序修改）。

为 Spinner 的列表创建一个 ArrayAdapter 以实现其下拉列表的显示。这里需注意 ArrayAdapter 的格式



（simple_spinner_item 和 simple_spinner_dropdown_item）是由 Android 系统定义的，它们不会出现在资源 XML 文件中。

创建 onItemSelectedListener 来捕捉 Spinner 的选择事件。监听 onItemSelected Listener 包含 onItemSelected()方法和 onNothingSelected()方法。

15.3.5 ListView

ListView 是一种用于垂直显示的列表控件，如果显示内容过多，则会出现垂直滚动条。

ListView 能够通过适配器将数据和自身绑定，在有限的屏幕上提供大量内容供用户选择，所以是经常使用的用户界面控件。同时，ListView 支持点击事件处理，用户可以用少量的代码实现复杂的选择功能。例如，调用 setAdapter()提供的数据和 View 子项，并通过 setOnItemSelectedListener()方法监听 ListView 上子项选择事件。

若 Activity 由一个单一的列表控制，则 Activity 需继承 ListActivity 类而不是之前介绍的常规的 Activity 类。如果主视图仅仅只是列表，甚至不需要建立一个 layout，ListActivity 会为用户构建一个全屏的列表。如果想自定义布局，则需要确定 ListView 的 id 为 @android:id/list，以便 ListActivity 知道其 Activity 的主要清单。

下面就通过一个例子来加深对 ListView 的理解，如图 5-17 所示。

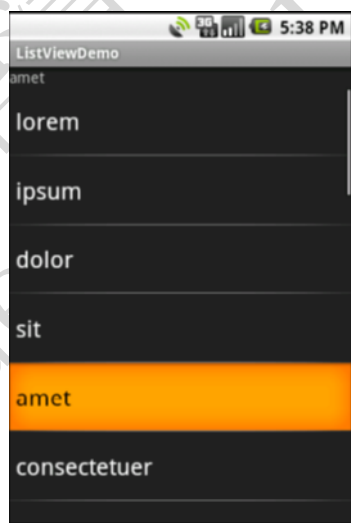


图 5-17 ListView 效果图

建立一个“ListViewDemo”程序

XML 文件中的代码如代码清单 5-21 所示。

代码清单 5-21 main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:orientation="vertical"
```



```
android:layout_width="match_parent"
android:layout_height="match_parent" >
<TextView
android:id="@+id/selection"
android:layout_width="match_parent"
android:layout_height="wrap_content"/>
<ListView
android:id="@android:id/list"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:drawSelectorOnTop="false"
/>
</LinearLayout>
```

修改 ListViewDemo.java 文件

在 ListViewDemo.java 文件中，首先需要为 ListView 创建适配器，配置和连接列表，添加 ListView 中所显示的内容。

代码清单 5-22 ListViewDemo.java

```
public class ListViewDemo extends ListActivity {
    TextView selection;
    String[] items={"lorem", "ipsum", "dolor", "sit", "amet",
        "consectetuer", "adipiscing", "elit", "morbi", "vel",
        "ligula", "vitae", "arcu", "aliquet", "mollis",
        "etiam", "vel", "erat", "placerat", "ante",
        "porttitor", "sodales", "pellentesque", "augue", "purus"};

    @Override
    public void onCreate(Bundle icle) {
        super.onCreate(icle);
        setContentView(R.layout.main);
        setListAdapter(new ArrayAdapter<String>(this,
            android.R.layout.simple_list_item_1,items));
        selection=(TextView)findViewById(R.id.selection);
    }
    public void onItemClick(ListView parent, View v, int position,long id) {
        selection.setText(items[position]);
    }
}
```



```
}
```

继承 `ListActivity` 后，可以通过 `setListAdapter()` 方法设置列表。这种情况下，提供了一个 `ArrayAdapter` 包装的字符串数组。其中 `ArrayAdapter` 的第二个参数 `android.R.layout.simple_list_item_1` 控制了 `ListView` 中行的显示，上例中 `android.R.layout.simple_list_item_1` 该值提供了标准的 Android 清单行：大字体、很多的填充、文本和白色。重写 `onListItemClick` 方法以在列表上子项的选择发生变化时及时更新其文本。

在默认情况下，`ListView` 只对列表子项的点击事件进行监听。但 `ListView` 也跟踪用户的选择，或多个可能的选择列表，但它需要一些变化。

在 Java 代码中调用 `ListView` 的 `setChoiceMode()` 方法来设置选择模式，可供选择的模式有：`CHOICE_MODE_SINGLE` 和 `CHOICE_MODE_MULTIPLE` 两种。可以通过 `getListView()` 方法在 `ListActivity` 中获取 `ListView`。

在构造 `ArrayAdapter` 时，第二个参数选择使用以下两种参数可以使列表上子项单选或是复选：`android.R.layout.simple_list_item_single_choice` 和 `android.R.layout.simple_list_item_multiple_choice`，如图 5-18 所示。

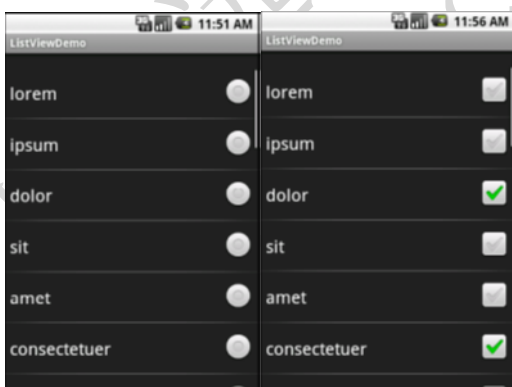


图 5-18 单选、复选模式

通过调用 `getCheckedItemPositions()` 方法来判断用户选择的子项。

15.3.6 TabHost

Tab 标签页是界面设计时经常使用的界面控件，可以实现多个分页之间的快速切换，每个分页可以显示不同的内容。

对 Tab 标签页的使用，首先要设计所有的分页的界面布局，在分页设计完成后，使用代码建立 Tab 标签页，并给每个分页添加标识和标题，最后确定每个分页所显示的界面布局。其中，每个分页建立一个 XML 文件，用以编辑和保存分页的界面布局，使用的方法与设计普通用户界面一样。

下面就通过一个例子来加深对 Tab 标签页的理解，如图 5-19 所示的效果图。



图 5-19 Tab 标签页效果图

建立一个“TabDemo”程序

程序包含两个 XML 文件，分别为 tab1.xml 和 tab2.xml，这两个文件分别使用线性布局、相对布局和绝对布局示例中的 main.xml 的代码，并将布局的 ID 分别定义为 layout01 和 layout02。

其中，tab1.xml 文件代码如代码清单 5-23 所示。

代码清单 5-23 tab1.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout android:id="@+id/layout01"
    ...
</LinearLayout>
```

tab2.xml 文件代码如代码清单 5-24 所示。

代码清单 5-24 tab2.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout android:id="@+id/layout03"
    ...
</RelativeLayout>
```

修改 TabDemo.java 文件

在 TabDemo.java 文件中输入代码清单 5-25 所示的代码，创建 Tab 标签页，并建立子页与界面布局直接的关联关系。

代码清单 5-25 TabDemo.java

```
package com.example.TabDemo;

import android.app.TabActivity;
import android.os.Bundle;
import android.widget.TabHost;
import android.view.LayoutInflater;
```



```
public class TabDemo extends TabActivity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        TabHost tabHost = getTabHost();  
        LayoutInflater.from(this).inflate(R.layout.tab1,  
            tabHost.getTabContentView(),true);  
        LayoutInflater.from(this).inflate(R.layout.tab2,  
            tabHost.getTabContentView(),true);  
        tabHost.addTab(tabHost.newTabSpec("TAB1")  
            .setIndicator("线性布局").setContent(R.id.layout01));  
        tabHost.addTab(tabHost.newTabSpec("TAB2")  
            .setIndicator("相对布局").setContent(R.id.layout02));  
    }  
}
```

该段代码中，第 8 行代码“public class TabDemo extends TabActivity”的声明 TabDemo 类继承于 TabActivity，与以往继承 Activity 不同，TabActivity 支持内嵌多个 Activity 或 View。

第 12 行代码“TabHost tabHost = getTabHost();”通过 getTabHost()方法获得 Tab 标签页的容器，用以承载可以点击的 Tab 标签和分页的界面布局。

第 13 行代码“LayoutInflater.from(this).inflate(R.layout.tab1, tabHost.getTabContentView(),true);”通过 LayoutInflater 将 tab1.xml 文件中的布局转换为 Tab 标签页可以使用的 View 对象。

第 14 行代码“tabHost.addTab(tabHost.newTabSpec("TAB1").setIndicator("线性布局").setContent(R.id.layout01));”使用 addTab()方法添加了第 1 个分页，tabHost.newTabSpec("TAB1")表明在第 12 行代码中建立的 tabHost 上，添加一个标识为 TAB1 的 Tab 分页，同时使用 setIndicator()方法设定分页显示的标题，使用 setContent()方法设定分页所关联的界面布局。

15.4 菜单

菜单是应用程序中非常重要的组成部分，能够在不占用界面空间的前提下，为应用程序提供统一的功能和设置界面，并为程序开发人员提供了易于使用的编程接口。Android 系统支持 3 种菜单：选项菜单（Option Menu）、子菜单（Submenu）、快捷菜单（Context Menu）。

15.4.1 选项菜单

选项菜单是一种经常被使用的 Android 系统菜单，可以分为图标菜单（Icon Menu）和扩展菜单（Expanded Menu）两类，可通过“菜单键”（Menu key）打开。

图标菜单能够同时显示文字和图标，最多支持 6 个子项，但图标菜单不支持单选框和复选框。

扩展菜单在图标菜单子项多余 6 个时才出现，通过点击图标菜单最后的子项“More”才能打开。扩展菜单是垂直的列表型菜单，不能够显示图标，但支持单选框和复选框。



图 5-20 图标菜单



图 5-21 扩展菜单

重写 onCreateOptionsMenu()方法

在 Android 应用程序中使用选项菜单，需重载 Activity 的 onCreateOptionsMenu()方法。初次使用选项菜单时，会调用 onCreateOptionsMenu()方法，用来初始化菜单子项的相关内容，因此这里需要设置菜单子项自身的子项 ID 和组 ID、菜单子项显示的文字和图片等。代码如代码清单 5-26 所示。

代码清单 5-26 重载 onCreateOptionsMenu()方法

```
final static int MENU_DOWNLOAD = Menu.FIRST;
final static int MENU_UPLOAD = Menu.FIRST+1;
@Override
public boolean onCreateOptionsMenu(Menu menu){
    menu.add(0,MENU_DOWNLOAD,0,"下载设置");
    menu.add(0,MENU_UPLOAD,1,"上传设置");
    return true;
}
```

第 1 行和第 2 行代码将菜单子项 ID 定义成静态常量，并使用静态常量 Menu.FIRST（整数类型，值为 1）定义第一个菜单子项，以后的菜单子项仅需在 Menu.FIRST 增加相应的数值即可。

第 4 行代码 Menu 对象作为一个参数被传递到方法内部，因此在 onCreateOptionsMenu()方法中，用户可以使用 Menu 对象的 add()方法添加菜单子项。其中 add()方法的语法如下。

```
MenuItem android.view.Menu.add(int groupId, int itemId, int order, CharSequence title)
```

第 1 个参数 groupId 是组 ID，用以批量的对菜单子项进行处理和排序；第 2 个参数 itemId 是子项 ID，是每一个菜单子项的唯一标识，通过子项 ID 使应用程序能够定位到用户所选择的菜单子项；第 3 个参数 order 是定义菜单子项在选项菜单中的排列顺序；第 4 个参数 title 是菜单子项所显示的标题。

第 7 行代码是 onCreateOptionsMenu()方法返回值，方法的返回值类型为布尔型：返回 true 将显示方法中设置的菜单，否则不能够显示菜单。

做完以上步骤后，使用 setIcon()方法和 setShortcut()方法，添加菜单子项的图标和快捷键，如代码清单 5-27 所示。

代码清单 5-27 添加菜单子项的图标和快捷键

```
menu.add(0,MENU_DOWNLOAD,0,"下载设置")
.setIcon(R.drawable.download);
```



```
.setShortcut(' ', 'd');
```

代码中，利用 MENU_DOWNLOAD 菜单设置图标和快捷键的代码；第 2 行代码中使用了新的图像资源，用户将需要使用的图像文件复制到/res/drawable 目录下；setShortcut()方法第一个参数是为数字键盘设定的快捷键，第二个参数是为全键盘设定的快捷键，且不区分字母的大小写。

重写 onPrepareOptionsMenu()方法

重载 onPrepareOptionsMenu()方法，能够动态地添加、删除菜单子项，或修改菜单的标题、图标和可见性等内容。onPrepareOptionsMenu()方法的返回值的含义与 onCreateOptionsMenu()方法相同：返回 true 则显示菜单，返回 false 则不显示菜单。

代码清单 5-28 所示的代码是在用户每次打开选项菜单时，在菜单子项中显示用户打开该子项的次数。

代码清单 5-28 菜单子项中显示用户打开该子项的次数

```
static int MenuUploadCounter = 0;

@Override
public boolean onPrepareOptionsMenu(Menu menu){
    MenuItem uploadItem = menu.findItem(MENU_UPLOAD);
    uploadItem.setTitle("上传设置:" +String.valueOf(MenuUploadCounter));
    return true;
}
```

第 1 行代码设置一个菜单子项的计数器，用来统计用户打开“上传设置”子项的次数；第 4 行代码是通过将菜单子项的 ID 传递给 menu.findItem()方法，获取到菜单子项的对象；第 5 行代码是通过 MenuItem 的 setTitle()方法修改菜单标题。

onOptionsItemSelected()方法

onOptionsItemSelected()方法能够处理菜单选择事件，且该方法在每次单击菜单子项时都会被调用。下面的代码说明了如何通过菜单子项的 ID 执行不同的操作。

代码清单 5-29 onOptionsItemSelected()

```
@Override
public boolean onOptionsItemSelected(MenuItem item){
    switch(item.getItemId()){
        case MENU_DOWNLOAD:
            MenuDownlaodCounter++;
            return true;
        case MENU_UPLOAD:
            MenuUploadCounter++;
            return true;
    }
}
```



```
return false;

}
```

onOptionsItemSelected() 的返回值表示是否对菜单的选择事件进行处理，如果已经处理过则返回 true，否则返回 false；第 3 行的 MenuItem.getItemId() 方法可以获取到被选择菜单子项的 ID。

程序运行后，通过单击“菜单键”可以调出程序设计的两个菜单子项，如图 5-22 所示。

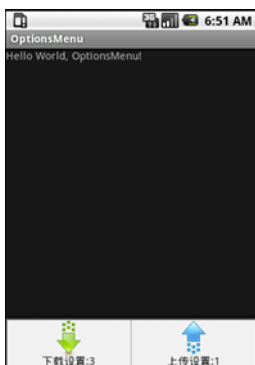


图 5-22 运行效果图

15.4.2 子菜单

子菜单是能够显示更加详细信息的菜单子项，如图 5-23 所示。其中，菜单子项使用了浮动窗体的显示形式，能够更好地适应小屏幕的显示方式。



图 5-23 菜单子项

Android 系统的子菜单使用非常灵活，可以在选项菜单或快捷菜单中使用子菜单，有利于将相同或相似的菜单子项组织在一起，便于显示和分类。但是，子菜单不支持嵌套。

子菜单的添加使用 addSubMenu() 方法实现，代码如代码清单 5-30 所示。

代码清单 5-30 onOptionsItemSelected()

```
SubMenu uploadMenu = (SubMenu) menu.addSubMenu
    (0,MENU_UPLOAD,1,"上传设置").setIcon(R.drawable.upload);
uploadMenu.setHeaderIcon(R.drawable.upload);
```



```
uploadMenu.setHeaderTitle("上传参数设置");
uploadMenu.add(0,SUB_MENU_UPLOAD_A,0,"上传参数 A");
uploadMenu.add(0,SUB_MENU_UPLOAD_B,0,"上传参数 B");
```

第 1 行代码在 onCreateOptionsMenu() 方法传递的 menu 对象上调用 addSubMenu() 方法，在选项菜单中添加一个菜单子项，用户单击后可以打开子菜单；addSubMenu() 方法与选项菜单中使用过的 add() 方法支持相同的参数，同样可以指定菜单子项的 ID、组 ID 和标题等参数，并且能够通过 setIcon() 方法显示菜单的图标。

第 2 行代码使用 setHeaderIcon () 方法，定义子菜单的图标。

第 3 行定义子菜单的标题，若不规范子菜单的标题，子菜单将显示父菜单子项标题，即第 1 行代码中“上传设置”。

第 4 行和第 5 行在子菜单中添加了两个菜单子项，菜单子项的更新方法和选择事件处理方法，仍然使用 onPrepareOptionsMenu() 方法和 onOptionsItemSelected () 方法。

以上一小节的代码为基础，将“上传设置”改为子菜单，并在子菜单中添加“上传参数 A”和“上传参数 B”两个菜单子项。运行结果如图 5-24 所示。



图 5-24 运行效果图

15.4.3 上下文菜单（Context Menu）

快捷菜单同样采用了浮动窗体的显示方式，与子菜单的实现方式相同，但两种菜单的启动方式却截然不同。

启动方式：快捷菜单类似于普通桌面程序中的“右键菜单”，当用户点击界面元素超过 2 秒后，将启动注册到该界面元素的快捷菜单。

使用方法：与使用选项菜单的方法非常相似，需要重载 onCreateContextMenu() 方法和 onContextItemSelected() 方法。

onCreateContextMenu() 方法

onCreateContextMenu() 方法主要用来添加快捷菜单所显示的标题、图标和菜单子项等内容，选项菜单中的 onCreateOptionsMenu() 方法仅在选项菜单第一次启动时被调用一次，而快捷菜单的 onCreateContextMenu() 方法每次启动时都会被调用一次。



代码清单 5-31 onCreateContextMenu ()

```
final static int CONTEXT_MENU_1 = Menu.FIRST;
final static int CONTEXT_MENU_2 = Menu.FIRST+1;
final static int CONTEXT_MENU_3 = Menu.FIRST+2;

@Override
public void onCreateContextMenu(ContextMenu menu, View v,
    ContextMenuInfo menuInfo){
    menu.setHeaderTitle("快捷菜单标题");
    menu.add(0, CONTEXT_MENU_1, 0,"菜单子项 1");
    menu.add(0, CONTEXT_MENU_2, 1,"菜单子项 2");
    menu.add(0, CONTEXT_MENU_3, 2,"菜单子项 3");
}
```

ContextMenu 类支持 add()方法（代码第 7 行）和 addSubMenu()方法，可以在快捷菜单中添加菜单子项和子菜单。

第 5 行代码的 onCreateContextMenu()方法中的参数：第 1 个参数 menu 是需要显示的快捷菜单；第 2 个参数 v 是用户选择的界面元素；第 3 个参数 menuInfo 是所选择界面元素的额外信息。

onContextItemSelected ()方法

菜单选择事件的处理需要重载 onContextItemSelected()方法，该方法在用户选择快捷菜单中的菜单子项后被调用，与 onOptionsItemSelected ()方法的使用方法基本相同。

代码清单 5-32 onContextItemSelected()

```
@Override
public boolean onContextItemSelected(MenuItem item){
    switch(item.getItemId()){
        case CONTEXT_MENU_1:
            LabelView.setText("菜单子项 1");
            return true;
        case CONTEXT_MENU_2:
            LabelView.setText("菜单子项 2");
            return true;
        case CONTEXT_MENU_3:
            LabelView.setText("菜单子项 3");
            return true;
    }
    return false;
}
```



registerForContextMenu()方法

使用 registerForContextMenu()方法，将快捷菜单注册到界面控件上（代码清单 5-33 中第 7 行）。这样，用户在长时间点击该界面控件时，便会启动快捷菜单。

为了能够在界面上直接显示用户所选择快捷菜单的菜单子项，在代码中引用了界面元素 TextView（代码清单 5-33 中第 6 行），通过更改 TextView 的显示内容，显示用户所选择的菜单子项。

代码清单 5-33 registerForContextMenu ()

```
TextView LabelView = null;
@Override
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main);
    LabelView = (TextView)findViewById(R.id.label);
    registerForContextMenu(LabelView);
}
```

main.xml

代码清单 5-34 main.xml

```
<TextView android:id="@+id/label"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:text="@string/hello"
/>
```

上述代码为/src/layout/main.xml 文件的部分内容，第 1 行声明了 TextView 的 ID 为 label，在代码清单 5-33 的第 6 行中，通过 R.id.label 将 ID 传递给 findViewById()方法，这样用户便能够引用该界面元素，并能够修改该界面元素的显示内容。

需要注意的一点，代码清单 5-34 的第 2 行，将 android:layout_width 设置为 match_parent，这样 TextView 将填满父节点的所有剩余屏幕空间，用户点击屏幕 TextView 下方任何位置都可以启动快捷菜单。如果将 android:layout_width 设置为 wrap_content，则用户必须准确单击 TextView 才能启动快捷菜单。

图 5-25 为快捷菜单的运行效果图。



图 5-25 运行效果图

下面将快捷菜单的示例程序 MyContextMenu 改用 XML 实现，新程序的工程名称为 MyXMLMContextMenu。

首先需要创建保存菜单内容的 XML 文件：在/src 目录下建立子目录 menu，并在 menu 下建立 context_menu.xml 文件，代码如代码清单 5-35 所示。

代码清单 5-35 context_menu.xml

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/contextMenu1"
        android:title="菜单子项 1"/>
    <item android:id="@+id/contextMenu2"
        android:title="菜单子项 2"/>
    <item android:id="@+id/contextMenu3"
        android:title="菜单子项 3"/>
</menu>
```

在描述菜单的 XML 文件中，必须以<menu>标签（代码第 1 行）作为根节点，<item>标签（代码第 2 行）用来描述菜单中的子项，<item>标签可以通过嵌套实现子菜单的功能。

XML 菜单的显示结果如图 5-26 所示。



图 5-26 XML 菜单的显示结果

在 XML 文件中定义菜单后，在 onCreateContextMenu()方法中调用 inflater.inflate()方法，将 XML 资源文件传递给菜单对象，代码如代码清单 5-36 所示。



代码清单 5-36 onCreateContextMenu()

```
@Override
public void onCreateContextMenu(ContextMenu menu,
    View v, ContextMenuInfo menuInfo){
    MenuInflater inflater = getMenuInflater();
    inflater.inflate(R.menu.context_menu, menu);
}
```

第 4 行代码中的 `getMenuInflater()` 为当前的 Activity 返回 `MenuInflater`；第 5 行代码将 XML 资源文件 `R.menu.context_menu`，传递给 `menu` 这个快捷菜单对象。

15.5 界面事件

在 Android 系统中，存在多种界面事件，如点击事件、触摸事件、焦点事件和菜单事件等，在这些界面事件发生时，Android 界面框架调用界面控件的事件处理方法对事件进行处理。

Android 系统界面事件的传递和处理遵循以下规则。

如果界面控件设置了事件监听器，则事件将先传递给事件监听器。

如果界面控件没有设置事件监听器，界面事件则会直接传递给界面控件的其他事件处理方法。

即使界面控件设置了事件监听器，界面事件也可以再次传递给其他事件处理方法。

是否继续传递事件给其他处理方法是由事件监听器处理方法的返回值决定的。

如果监听器处理方法的返回值为 `true`，表示该事件已经完成处理过程，不需要其他处理方法参与处理过程，这样事件就不会再继续进行传递。

如果监听器处理方法的返回值为 `false`，则表示该事件没有完成处理过程，或需要其他处理方法捕获到该事件，事件会被传递给其他的事件处理方法。

在 MVC 模型中，控制器根据界面事件（UI Event）类型不同，将事件传递给界面控件不同的事件处理方法。

按键事件（KeyEvent）将传递给 `onKey()` 方法进行处理。

触摸事件（TouchEvent）将传递给 `onTouch()` 方法进行处理。

15.5.1 按键事件

下面以 `EditText` 控件中的按键事件为例，说明 Android 系统界面事件传递和处理过程。

假设 `EditText` 控件已经设置了按键事件监听器，当用户按下键盘上的某个按键时，控制器将产生 `KeyEvent` 按键事件。Android 系统会首先判断 `EditText` 控件是否设置了按键事件监听器，因为 `EditText` 控件已经设置按键事件监听器 `OnKeyListener`，所以按键事件先传递到监听器的事件处理方法 `onKey()` 中，事件能够继续传递给 `EditText` 控件的其他事件处理方法，完全根据 `onKey()` 方法的返回值来确定：如果 `onKey()` 方法返回 `false`，事件将继续传递，这样 `EditText` 控件就可以捕获到该事件，将按键的内容显示在 `EditText` 控件中；如果 `onKey()` 方法返回 `true`，将阻止按键事件的继续传递，这样 `EditText` 控件就不能够捕获到按键事件，也就不能够将按键内容显示在 `EditText` 控件中。



Android 界面框架支持对按键事件的监听，并能够将按键事件的详细信息传递给处理方法。为了处理控件的按键事件，先需要设置按键事件的监听器，并重载 `onKey()` 方法，示例代码如代码清单 5-37 所示。

代码清单 5-37 设置按键事件的监听器，并重载 `onKey()` 方法

```
entryText.setOnKeyListener(new OnKeyListener(){
    @Override
    public boolean onKey(View view, int keyCode, KeyEvent keyEvent) {
        //过程代码.....
        return true/false;
    }
}
```

第 1 行代码是设置控件的按键事件监听器。

第 3 行代码的 `onKey()` 方法中的参数：第 1 个参数 `View` 表示产生按键事件的界面控件；第 2 个参数 `keyCode` 表示按键代码；第 3 个参数 `KeyEvent` 则包含了事件的详细信息，如按键的重复次数、硬件编码和按键标志等。

第 5 行代码是 `onKey()` 方法的返回值：返回 `true`，阻止事件传递；返回 `false`，允许继续传递按键事件。

`KeyEventDemo` 是一个说明如何处理按键事件的示例。

`KeyEventDemo` 用户界面如图 5-27 所示。

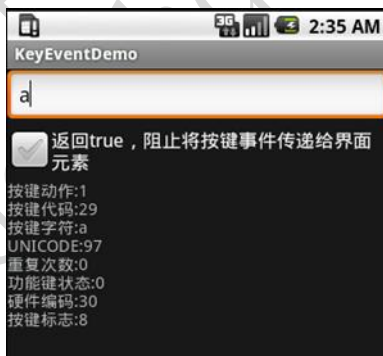


图 5-27 `KeyEventDemo` 用户界面

从图 5-27 中可以看出，最上方的 `EditText` 控件是输入字符的区域，中间的 `CheckBox` 控件用来控制 `onKey()` 方法的返回值，最下方的 `TextView` 控件用来显示按键事件的详细信息，包括按键动作、按键代码、按键字符、UNICODE 编码、重复次数、功能键状态、硬件编码和按键标志。

界面的 XML 文件的代码如代码清单 5-38 所示

代码清单 5-38 界面 XML 文件

```
<EditText android:id="@+id/entry"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
```



```
</EditText>
<CheckBox android:id="@+id/block"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="返回 true，阻止将按键事件传递给界面元素">
</CheckBox>
<TextView android:id="@+id/label"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="按键事件信息">
</TextView>
```

在 EditText 中，当任何一个键按下或抬起时，都会引发按键事件。为了能够使 EditText 处理按键事件，需要使用 `setOnKeyListener()` 方法在代码中设置按键事件监听器，并在 `onKey()` 方法中添加按键事件的处理过程，代码如代码清单 5-39 所示。

代码清单 5-39 `setOnKeyListener()`

```
entryText.setOnKeyListener(new OnKeyListener(){
    @Override
    public boolean onKey(View view, int keyCode, KeyEvent keyEvent) {
        int metaState = keyEvent.getMetaState();
        int unicodeChar = keyEvent.getUnicodeChar();
        String msg = "";
        msg += "按键动作:" + String.valueOf(keyEvent.getAction())+"\n";
        msg += "按键代码:" + String.valueOf(keyCode)+"\n";
        msg += "按键字符:" + (char)unicodeChar+"\n";
        msg += "UNICODE:" + String.valueOf(unicodeChar)+"\n";
        msg += "重复次数:" + String.valueOf(keyEvent.getRepeatCount())+"\n";
        msg += "功能键状态:" + String.valueOf(metaState)+"\n";
        msg += "硬件编码:" + String.valueOf(keyEvent.getScanCode())+"\n";
        msg += "按键标志:" + String.valueOf(keyEvent.getFlags())+"\n";
        labelView.setText(msg);
        if (checkBox.isChecked())
            return true;
        else
            return false;
    }
});
```



在上述代码中，第 4 行代码用来获取功能键状态。功能键包括左 Alt 键、右 Alt 键和 Shift 键，当这 3 个功能键被按下时，功能键代码 metaState 值分别为 18、34 和 65；但没有功能键被按下时，功能键代码 metaState 值分别为 0。

第 5 行代码获取了按键的 Unicode 值，而在第 9 行中，将 Unicode 转换为了字符，显示在 TextView 中。

第 7 行代码获取了按键动作，0 表示按下按键，1 表示抬起按键。第 7 行代码获取按键的重复次数，但当按键被长时间按下时，则会产生这个属性值。

第 13 行代码获取了按键的硬件编码，各硬件设备的按键硬件编码都不相同，因此该值一般用于调试。

第 14 行获取了按键事件的标志符。

15.5.2 触摸事件

Android 界面框架支持对触摸事件的监听，并能够将触摸事件的详细信息传递给处理方法，不过需要设置触摸事件的监听器，并重载 onTouch() 方法。

设置触摸事件的监听器，并重载 onTouch() 方法的代码如代码清单 5-40 所示。

代码清单 5-40 设置触摸事件的监听器，并重载 onTouch() 方法

```
touchView.setOnTouchListener(new View.OnTouchListener(){
    @Override
    public boolean onTouch(View v, MotionEvent event) {
        //过程代码...
        return true/false;
    }
});
```

在上述代码中，第 1 行代码是设置控件的触摸事件监听器；在第 3 行的 onTouch() 方法中，第 1 个参数 View 表示产生触摸事件的界面控件；第 2 个参数 MontionEvent 表示触摸事件的详细信息，如产生时间、坐标和触点压力等；第 5 行是 onTouch() 方法的返回值。

TouchEventDemo 是一个说明如何处理触摸事件的示例。

TouchEventDemo 用户界面如图 5-28 所示。



图 5-28 TouchEventDemo 用户界面

由图 5-28 可以看出，上半部分的浅蓝色区域是可以接受触摸事件的区域，用户可以在 Android 模拟器中使用鼠标点击屏幕用以模拟触摸手机屏幕；下方黑色区域是显示区域，用来显示触摸事件类型、相对坐标、绝对坐标、触点压力、触点尺寸和历史数据量等信息。

在用户界面中使用了线性布局，并加入了 3 个 TextView 控件：第 1 个 TextView（ID 为 touch_area）用来标识触摸事件的测试区域；第 2 个 TextView（ID 为 history_label）用来显示触摸事件的历史数据量；第 3 个 TextView（ID 为 event_label）用来显示触摸事件的详细信息，包括类型、相对坐标、绝对坐标、触点压力和触点尺寸。

XML 文件的代码如代码清单 5-41 所示。

代码清单 5-41 XML 文件

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView android:id="@+id/touch_area"
        android:layout_width="match_parent"
        android:layout_height="300dip"
        android:background="#80A0FF"
        android:textColor="#FFFFFF"
        android:text="触摸事件测试区域">
    </TextView>
    <TextView android:id="@+id/history_label"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content">
```



```

        android:text="历史数据量： " >
    </TextView>
    <TextView android:id="@+id/event_label"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="触摸事件： " >
    </TextView>
</LinearLayout>

```

在上述代码中，第 9 行代码定义了 TextView 的背景颜色，#80A0FF 是颜色代码；第 10 行代码定义了 TextView 的字体颜色。

在代码中为了能够引用 XML 文件中声明的界面元素，使用了代码清单 5-42 所示的代码。

代码清单 5-42 在代码中引用 XML 文件中声明的界面元素

```

TextView labelView = null;
labelView = (TextView)findViewById(R.id.event_label);
TextView touchView = (TextView)findViewById(R.id.touch_area);
final TextView historyView = (TextView)findViewById(R.id.history_label);

```

当手指接触到触摸屏、在触摸屏上移动或离开触摸屏时，分别会引发 ACTION_DOWN、ACTION_UP 和 ACTION_MOVE 触摸事件，而无论是哪种触摸事件，都会调用 onTouch() 方法进行处理。事件类型包含在 onTouch() 方法的 MotionEvent 参数中，可以通过 getAction() 方法获取到触摸事件的类型，然后根据触摸事件的不同类型进行不同的处理。为了能够使屏幕最上方的 TextView 处理触摸事件，需要使用 setOnTouchListener() 方法在代码中设置触摸事件监听器，并在 onTouch() 方法添加触摸事件的处理过程。代码如代码清单 5-43 所示。

代码清单 5-43 onTouch()

```

touchView.setOnTouchListener(new View.OnTouchListener(){
    @Override
    public boolean onTouch(View v, MotionEvent event) {
        int action = event.getAction();
        switch (action) {
            case (MotionEvent.ACTION_DOWN):
                Display("ACTION_DOWN",event);
                break;
            case (MotionEvent.ACTION_UP):
                int historySize = ProcessHistory(event);
                historyView.setText("历史数据量： "+historySize);
                Display("ACTION_UP",event);

```



```

        break;

        case (MotionEvent.ACTION_MOVE):
            Display("ACTION_MOVE",event);
            break;
    }

    return true;
}

});

```

第 7 行代码的 Display() 是一个自定义方法，主要用来显示触摸事件的详细信息，方法的代码和含义将在后面进行介绍；第 10 行代码的 ProcessHistory() 也是一个自定义方法，用来处理触摸事件的历史数据；第 11 行代码是使用 TextView 显示历史数据的数量。

MotionEvent 参数中不仅有触摸事件的类型信息，还有触点的坐标信息，获取方式是使用 getX() 和 getY() 方法，这两个方法获取到的是触点相对于父界面元素的坐标信息。如果需要获取绝对坐标信息，则可使用 getRawX() 和 getRawY() 方法。

触点压力是一个介于 0 和 1 之间的浮点数，用来表示用户对触摸屏施加压力的大小，接近 0 表示压力较小，接近 1 表示压力较大，获取触摸事件触点压力的方式是调用 getPressure() 方法。

触点尺寸指用户接触触摸屏的接触点大小，也是一个介于 0 和 1 之间的浮点数，接近 0 表示尺寸较小，接近 1 表示尺寸较大，可以使用 getSize() 方法获取。

Display() 将 MotionEvent 参数中的事件信息提取出来，并显示在用户界面上。代码如代码清单 5-44 所示。

代码清单 5-44 Display()

```

private void Display(String eventType, MotionEvent event){
    int x = (int)event.getX();
    int y = (int)event.getY();
    float pressure = event.getPressure();
    float size = event.getSize();
    int RawX = (int)event.getRawX();
    int RawY = (int)event.getRawY();

    String msg = "";
    msg += "事件类型: " + eventType + "\n";
    msg += "相对坐标: " + String.valueOf(x) + "," + String.valueOf(y) + "\n";
    msg += "绝对坐标: " + String.valueOf(RawX) + "," + String.valueOf(RawY) + "\n";
    msg += "触点压力: " + String.valueOf(pressure) + ",    ";
    msg += "触点尺寸: " + String.valueOf(size) + "\n";
}

```




```
labelView.setText(msg);  
}
```

一般情况下，如果用户将手指放在触摸屏上，但不移动，然后抬起手指，应先后产生 ACTION_DOWN 和 ACTION_UP 两个触摸事件。但如果用户在屏幕上移动手指，然后再抬起手指，则会产生这样的事件序列：ACTION_DOWN → ACTION_MOVE → ACTION_MOVE → ACTION_MOVE → ... → ACTION_UP。

在手机上运行的应用程序，效率是非常重要的。如果 Android 界面框架不能产生足够多的触摸事件，则应用程序就不能够很精确地描绘触摸屏上的触摸轨迹。如果 Android 界面框架产生了过多的触摸事件，虽然能够满足精度的要求，但也降低了应用程序效率。

针对以上问题 Android 界面框架使用了“打包”的解决方法。在触点移动速度较快时会产生大量的数据，每经过一定的时间间隔便会产生一个 ACTION_MOVE 事件，在这个事件中，除了有当前触点的相关信息外，还包含这段时间间隔内触点轨迹的历史数据信息，这样既能够保持精度，又不至于产生过多的触摸事件。

通常情况下，在 ACTION_MOVE 的事件处理方法中，都先处理历史数据，然后再处理当前数据，代码如代码清单 5-45 所示。

代码清单 5-45 ProcessHistory(MotionEvent event)

```
private int ProcessHistory(MotionEvent event)  
{  
    int historySize = event.getHistorySize();  
    for (int i = 0; i < historySize; i++) {  
        long time = event.getHistoricalEventTime(i);  
        float pressure = event.getHistoricalPressure(i);  
        float x = event.getHistoricalX(i);  
        float y = event.getHistoricalY(i);  
        float size = event.getHistoricalSize(i);  
  
        // 处理过程...  
    }  
    return historySize;  
}
```

在上述代码中，第 3 行代码获取了历史数据的数量；然后在第 4 行至 12 行中循环处理这些历史数据；第 5 行代码获取了历史事件的发生时间；第 6 行代码获取历史事件的触点压力；第 7 行和第 8 行代码获取历史事件的相对坐标；第 9 行获取历史事件的触点尺寸；在第 14 行返回历史数据的数量，主要是用于界面显示。



15.6 自定义样式和主题

Android 也可以像 HTML/CSS 中的 style 一样，使用自定义的 style 样式。Android 一般通过 value 文件夹下面新建一个 styles.xml 文件来设置自定义样式。这里开发者可以设置高度、填充字体颜色、字体大小、背景颜色等描述一个 View 或者一个窗口的显示属性。这就像 Web 开发中的 CSS 样式表，使我们的样式独立于内容进行设计开发。

主题和样式都是通过在 xml 文件中预定义一系列属性值，通过这些属性值来形成统一的显示风格。不同的是，样式只能应用于某种类型的 View；而主题刚好相反，不能应用于特定的 View，而只能作用于一个或多个 Activity，或是整个应用。

下面通过代码学习一下如何自定义样式与主题，并在程序中应用。

首先是自定义样式和主题。在项目的 res/values/目录下添加 styles.xml。如代码清单 5-46 所示。

代码清单 5-46 styles.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<!-- 定义 my_style_1，没有指定 parent，用系统默认的 -->
<style name="my_style_1">
    <!-- 定义与指定 View 相关的若干属性 -->
    <item name="android:hint">load from style 1</item>
</style>

<!-- 定义 my_style_2，用自定义的 my_style_1 作为 parent -->
<style name="my_style_2" parent="@style/my_style_1">
    <!-- 定义与指定 View 相关的若干属性 -->
    <item name="android:textSize">30sp</item>
    <item name="android:textColor">#FFFF0000</item>
    <item name="android:hint">load from style 2</item>
</style>

<!-- 定义 my_style_3，用 android 的 EditText 作为 parent -->
<style name="my_style_3" parent="@android:style/Widget.EditText">
    <!-- 定义与指定 View 相关的若干属性 -->
    <item name="android:hint">"load from style 3"</item>
    <item name="android:textStyle">bold|italic</item>
    <item name="android:typeface">monospace</item>
    <item name="android:background">@drawable/mybackground</item>
</style>
```



```
<!-- 定义 MyTheme，用 android 的 Theme 作为 parent -->
<style name="MyTheme" parent="@android:style/Theme">
    <item name="android:textSize">20sp</item>
    <item name="android:textColor">#FF0000FF</item>
    <item name="android:hint">"load from style 3"</item>
    <item name="android:textStyle">bold|italic</item>
    <item name="android:typeface">monospace</item>
    <item name="android:background">@drawable/gallery_selected_pressed</item>
    <item name="myStyle">@style/my_style_3</item>
</style>
</resources>
```

由上述代码可以看出，主题和样式都可以通过在<style>下添加多个<item>来完成其定义。下面来介绍一下<style>的属性，如表 5-12 所示。

表 5-12 <style>的属性

属性名称	描述
name	style 的名字，引用该 style 时使用此名字
parent	一些在自定义的 style 中没有指定的属性会继承 parent style 中的值。parent 可以是 android 预定义的 resource，也可以是自己定义的 style。为可选项

表 5-12 <style>的属性

属性名称	描述
name	style 的名字，引用该 style 时使用此名字
parent	一些在自定义的 style 中没有指定的属性会继承 parent style 中的值。parent 可以是 android 预定义的 resource，也可以是自己定义的 style。为可选项

在<item>中定义需要改变的属性值，例如，可以定义颜色、高度等。Android 中能使用的属性可以在<sdk>/docs/reference/android/R.styleable.html 中查到；也可以用自己定义的属性值。

下面讲解如何在程序中使用样式：一方面可以在 layout 的布局文件中指定自定义的样式；另一方面可以在 java 代码中指定。具体代码如代码清单 5-47、5-48 所示。

代码清单 5-47 在布局文件中指定自定义样式

```
<EditText android:id="@+id/EditText03"
    style="@style/my_style_3"
    android:layout_width="fill_parent"
    android:layout_height="wrap_content">
```



</EditText>

代码清单 5-48 在 java 代码中指定自定义样式

```
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main);
    LinearLayout ll = (LinearLayout)findViewById(R.id.main);

    EditText et = new EditText(this, null, R.attr.myStyle);//指定样式
    ll.addView(et);
}
```

如何在程序中使用自定义主题呢？这与使用自定义样式相似，都可以通过两个方法指定，不同的是主题除了可以在 java 代码中指定以外，还可以在 AndroidManifest 中被指定。具体代码如代码清单 5-49、5-50、5-51 所示。

代码清单 5-49 在 AndroidManifest.xml 中指定自定义主题——应用于整个程序

```
<application android:theme="@style/MyTheme">
```

代码清单 5-50 在 AndroidManifest.xml 中指定自定义主题——应用于 Activity

```
<activity android:theme="@style/MyTheme">
```

代码清单 5-51 在 java 代码中指定自定义主题

```
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setTheme(R.style.MyTheme);//设置主题
    setContentView(R.layout.main);
}
```

15.7 9Patch

9Patch 是一个对 png 图片做处理的工具。经 9Patch 处理过的图片以 *.9.png 结尾，和普通图片相比，四周多了一个边框，如图 5-29 所示。



图 5-29 9Patch 处理过的图片 *.9.png

“*.9.png”是一种被 Android os 所支持的特殊图片格式，是经过 9Patch 特殊处理过的图片。用它可以实现部分拉伸而不出现拉伸以后图片失真等不良现象。为将普通的 PNG 图片编辑转化为 9Patch 图片，可以使用 Android SDK/tools 目录下提供的 draw9patch.bat 编辑器。draw9patch.bat 编辑器界面如图 5-30 所示。

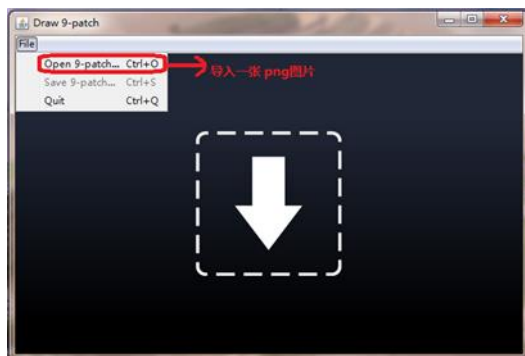


图 5-30 draw9Patch.bat 编辑器界面

在 draw9patch.bat 编辑器中，可以对导入的 png 图片进行以下操作来达到想要的图片目标。

Zoom: 缩放左边编辑区域的大小。

Patch scale: 缩放右边预览区域的大小。

Show lock: 当鼠标在图片区域时，显示不可编辑区域。

Show patches: 在编辑区域显示图片拉伸的区域。

Show content: 在预览区域显示图片的内容区域。

Show bad patches: 在拉伸区域周围用显示可能会对拉伸后的图片产生变形的区域；根据图片的颜色值来区分是否为 bad patch。

15.8 本章小结

本章主要对 Android 程序界面开发的学习，包括用户界面基础，界面布局特点及使用方法，用户界面控件的使用，选项菜单、子菜单和快捷菜单的使用方法，界面事件的处理方法等。



第 16 章 组件间通信

在第 5 章的学习中，主要了解了 Android 程序界面的开发，包括用户界面基础、用户界面控件的使用、界面布局的特点及使用方法、菜单的使用方法、界面事件的处理方法等。在此基础上，本章将对 Android 组件间的通信进行学习，包括 Intent 进行组件通信的原理、Intent 启动 Activity 的方法、获取 Activity 返回值的方法、Intent 过滤器的原理及其匹配机制、发送和接收广播消息的方法等。

16.1 Intent 对象及其属性

Intent 是一个动作的完整描述，包含了动作的产生组件、接收组件和传递的数据信息。Android 则根据 Intent 的描述，在不同组件间传递消息，负责找到对应的组件，将 Intent 传递给调用的组件，组件接收到传递的消息，执行相关动作，完成组件的调用。

Intent 不仅可用于应用程序之间，也可用于应用程序内部的 Activity/Service 之间的交互。Intent 为 Activity、Service 和 BroadcastReceiver 等组件提供交互能力，还可以启动 Activity 和 Service，在 Android 系统上发布广播消息。这里的广播消息是指可以接收到的特定数据或消息，也可以是手机的信号变化或电池的电量过低等信息。

因此，Intent 在这里起着一个媒体中介的作用，专门提供组件互相调用的相关信息，实现调用者与被调用者之间的解耦。在 SDK 中给出了 Intent 作用的表现形式。

通过 Context.startActivity() or Activity.startActivityForResult() 启动一个 Activity。

通过 Context.startService() 启动一个服务，或者通过 Context.bindService() 和后台服务交互。

通过广播方法（比如 Context.sendBroadcast(), Context.sendOrderedBroadcast(), Context.sendStickyBroadcast()）发给 broadcast receivers。

一般情况下，Intent 对某操作的抽象描述包含下面几个部分。

对执行动作的描述：操作（action）。

对这次动作相关联的数据进行描述：数据（data）。

对数据类型的描述：数据类型（type）。

对执行动作的附加信息进行描述：类别（category）。

其他一切附加信息的描述：附件信息（extras）。

对目标组件的描述：目标组件（component）。

16.1.1 Intent 的 action 属性

action 是要执行的动作，也可以是在广播 Intent 中已发生且正被报告的动作。action 部分是一个字符串对象。它描述了 Intent 会触发的动作。Android 系统中已经预定义了一些 action 常量，可以参看 SDK 帮助文档，下表给出了一些标准的 action 常量，如表 6-1 所示。

表 6-1 SDK 中定义的标准动作

常量	目标组件	描述
----	------	----



ACTION_CALL	activity	初始化一个电话呼叫
ACTION_EDIT	activity	显示可供用户编辑的数据
ACTION_MAIN	activity	将该 Activity 作为 task 的第一个 Activity，没有数据输入，也没有数据返回
ACTION_SYNC	activity	使服务器上的数据与移动设备上数据同步
ACTION_BATTERY_LOW	broadcast receiver	提示电池电量低
ACTION_HEADSET_PLUG	broadcast receiver	提示耳机塞入或拔出
ACTION_SCREEN_ON	broadcast receiver	屏幕已点亮
ACTION_TIMEZONE_CHANGED	broadcast receiver	时区设置改变

除上表介绍的 action 常量外，开发者也可以定义自己的 action 描述。一般来讲，定义自己的 action 字符串应该以应用程序的包名为前缀（防止重复定义）。由于 action 部分很大程度上决定了一个 Intent 的内容，特别是数据（data）和附加（extras）字段，就像一个方法名决定了参数和返回值。正是这个原因，应该尽可能明确指定动作，并紧密关联到其他 Intent 字段。即应该定义组件能够处理的 Intent 对象的整个协议，而不仅仅是单独地定义一个动作。一个 Intent 对象的动作通过 setAction()方法设置，通过 getAction()方法读取。

16.1.2 Intent 的 data 属性

data，即执行动作要操作的数据。

data 描述了 Intent 的动作所能操作数据的 MIME 类型和 URL，不同的 Action 用不同的操作数据。例如，如果 Activity 字段是 ACTION_EDIT，data 字段将显示包含用于编辑的文档的 URI；如果 Activity 是 ACTION_CALL，data 字段是一个 tel://URI 和将拨打的号码；如果 Activity 是 ACTION_VIEW，data 字段是一个 http://URI，接收活动将被调用去下载和显示 URI 指向的数据。在许多情况下，数据类型能够从 URI 中推测出来，特别是 content://URIs，它表示位于设备上的数据且被内容提供者（Content Provider）控制。但是类型也能够显示设置，setData()方法指定数据的 URI，setType()指定 MIME 类型，setDataAndType()指定数据的 URI 和 MIME 类型。通过 getData()读取 URI，getType()读取类型。

匹配一个 Intent 到一个能够处理 data 的组件，知道 data 的类型（它的 MIME 类型）和它的 URI 很重要。例如，一个组件能够显示图像数据就不应该被调用去播放音频文件。

16.1.3 Intent 的 type 属性

数据类型（type），显式指定 Intent 的数据类型（MIME）。一般 Intent 的数据类型能够根据数据本身



进行判定，但是通过设置这个属性，可以强制采用显式指定的类型而不再进行推导。

16.1.4 Intent 的 category 属性

category（类别），被执行动作的附加信息。例如，LAUNCHER_CATEGORY 表示 Intent 的接受者应该在 Launcher 中作为顶级应用出现；而 ALTERNATIVE_CATEGORY 表示当前的 Intent 是一系列的可选动作中的一个，这些动作可以在同一块数据上执行。其他的如表 6-2 所示。

表 6-2 SDK 中定义的标准动作

常量	描述
CATEGORY_BROWSABLE	目标 Activity 可通过浏览器安全启动以显示一个链接相关的数据，如图片或邮件信息
CATEGORY_GADGET	Activity 可被嵌入另外一个拥有 gadget 的 Activity 中
CATEGORY_HOME	Activity 显示主页，即设备打开时用户看到的第一个界面或是用户按 Home 键时的界面
CATEGORY_LAUNCHER	Activity 是一个 task 的初始 Activity，是程序启动的高优先级 Activity
CATEGORY_PREFERENCE	目标 Activity 为 preference panel。

通过 addCategory()方法添加一个种类到 Intent 对象中；通过 removeCategory()方法删除一个之前添加的种类；通过 getCategories()方法获取 Intent 对象中的所有种类。

16.1.5 Intent 的 extras 属性

extras（附加信息）是一组键值对，包含了需要传递给目标组件并有其处理的一些附加信息。

就像动作关联的特定种类的数据 URIs，也关联到某些特定的附加信息。例如，一个 ACTION_TIMEZONE_CHANGE intent 有一个“time-zone”的附加信息，标识新的时区，ACTION_HEADSET_PLUG 有一个“state”附加信息，标识头部现在是否塞满或未塞满；有一个“name”附加信息，标识头部的类型。如果你自定义了一个 SHOW_COLOR 动作，颜色值将可以设置在附加的键值对中。例如，如果要执行“发送电子邮件”这个动作，可以将电子邮件的标题、正文等保存在 extras 里，传给电子邮件发送组件。

Intent 有一系列 putXXX()方法用于插入各种附加数据，有一系列 getXXX()方法可以取出一系列数据。

使用 Extras 可以为组件提供扩展信息。

16.1.6 Intent 的 ComponentName 属性

ComponentName（组件），指定 Intent 的目标组件的类名称。ComponentName 包含两个 String 成员，分别代表组件的全称类名和包名，包名必须和 AndroidManifest.xml 文件标记中的对应信息一致。

ComponentName 通过 setComponent()、setClass()或 setClassName()设置，通过 getComponent()读取。

通常 Android 会根据 Intent 中包含的其他属性的信息（如 action、data/type、category）进行查找，



最终找到一个与之匹配的目标组件。但是，如果 `ComponentName` 这个属性有指定，将直接使用指定的组件，而不再执行上述查找过程。指定了这个属性以后，`Intent` 的其他所有属性都是可选的。

对于 `Intent`，组件名并不是必需的。如果一个 `Intent` 对象添加了组件名，则称该 `Intent` 为“显式 `Intent`”，这样的 `Intent` 在传递时会直接根据组件名去寻找目标组件。如果没有添加组件名，则称为“隐式 `Intent`”，`Android` 会根据 `Intent` 中的其他信息来确定响应该 `Intent` 的组件。

总之，`action`、`data/type`、`category` 和 `extras` 一起使系统能够理解诸如“查看某联系人的详细信息”或“给某人打电话”之类的短语。随着应用不断地加入系统中，`Android` 系统可以添加新的 `action`、`data/type`、`category` 来扩展功能。当然，最受益的还是应用本身，可以利用这套语言机制来处理不同的动作和数据。

16.2 系统标准 `ActivityAction` 应用

16.2.1 启动 `Activity`

在 `Android` 系统中，应用程序一般都有多个 `Activity`，`Intent` 可以实现不同 `Activity` 之间的切换和数据传递。

启动 `Activity` 方式有以下两种方式。

显式启动，必须在 `Intent` 中指明启动的 `Activity` 所在的类。

隐式启动，`Android` 系统根据 `Intent` 的动作和数据来决定启动哪一个 `Activity`，即在隐式启动时，`Intent` 中只包含需要执行的动作和所包含的数据，而无须指明具体启动哪一个 `Activity`，选择权由 `Android` 系统和最终用户来决定。

显式启动

使用 `Intent` 显式启动 `Activity`，首先需要创建一个 `Intent`，指定当前的应用程序上下文及要启动的 `Activity`，并把创建好的 `Intent` 作为参数传递给 `startActivity()` 方法。代码如代码清单 6-1 所示。

代码清单 6-1 显式启动

```
Intent intent = new Intent(IntentDemo.this, Activity2.class);
startActivity(intent);
```

以下将通过一个 `IntentDemo` 示例来详细讲解如何使用 `Intent` 显式启动新的 `Activity`。

`IntentDemo` 示例中包含 `IntentDemo` 和 `Activity2` 这两个 `Activity` 类，程序启动是默认启动 `IntentDemo` 这个 `Activity`，启动画面如图 6-1 所示。

在图 6-1 的界面中，单击“跳转到 `Activity2`”按钮后，程序启动 `Activity2` 这个 `Activity`，界面如图 6-2 所示。

为使程序达到上述效果，首先，在 `AndroidManifest.xml` 文件中注册上面两个 `Activity`，应使用 `<activity>` 标签，嵌套在 `<application>` 标签内部。代码如代码清单 6-2 所示。

代码清单 6-2 `AndroidManifest.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
```



```
package="com.example.IntentDemo"
android:versionCode="1"
android:versionName="1.0">
<application android:icon="@drawable/icon" android:label="@string/app_name">
    <activity android:name=".IntentDemo"
        android:label="@string/app_name">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
    <activity android:name=".Activity2"
        android:label="@string/app_name">
    </activity>
</application>
<uses-sdk android:minSdkVersion="3" />
</manifest>
```

在 Android 应用程序中，用户使用的每个组件都必须在 `AndroidManifest.xml` 文件中的 `<application>` 节点内定义，所以 `<application>` 节点下共有两个 `<activity>` 节点，分别代表应用程序中所使用的两个 Activity：IntentDemo 和 Activity2。

在 IntentDemo.java 文件中，包含了显示使用 Intent 启动 Activity2 的核心代码，如代码清单 6-3 所示。

代码清单 6-3 IntentDemo.java

```
Button button = (Button)findViewById(R.id.btn);
button.setOnClickListener(new OnClickListener(){
    public void onClick(View view){
        Intent intent = new Intent(IntentDemo.this, Activity2.class);
        startActivity(intent);
    }
});
```

在单击事件的处理函数中，Intent 构造函数的第 1 个参数是应用程序上下文，应用程序上下文是 IntentDemo；第 2 个参数是接收 Intent 的目标组件，使用的是显式启动方式，直接指明了需要启动的 Activity。

同理，在 Activity2.java 文件中，包含了显示使用 Intent 启动 IntentDemo 的核心代码，如代码清单 6-4 所示。



代码清单 6-4 Activity2.java

```
Button button = (Button)findViewById(R.id.btn);
button2.setOnClickListener(new OnClickListener(){
    public void onClick(View view){
        Intent intent = new Intent(Activity2.this, IntentDemo.class);
        startActivity(intent);
    }
});
```

2. 隐式启动

隐式启动 Activity 时，Android 系统在应用程序运行时解析 Intent，并根据一定的规则对 Intent 和 Activity 进行匹配，使 Intent 上的动作、数据与 Activity 完全吻合。而匹配的 Activity 可以是应用程序本身的，也可以是 Android 系统内置的，还可以是第三方应用程序提供的。因此，这种方式更强调了 Android 应用程序中组件的可复用性。

由此可以看出，隐式启动不需要指明需要启动哪一个 Activity，而由 Android 系统来决定，有利于使用第三方组件。

在默认情况下，Android 系统会调用内置的 Web 浏览器，代码如代码清单 6-5 所示。

代码清单 6-5 隐式启动默认情况调用内置 Web 浏览器

```
Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse("http://www.google.com"));
startActivity(intent);
```

在上述代码中，Intent 的动作是 Intent.ACTION_VIEW，是根据 Uri 的数据类型来匹配动作；数据部分的 Uri 是 Web 地址，使用 Uri.parse(urlString)方法，可以简单地把一个字符串解释成 Uri 对象。

Intent 的语法如代码清单 6-6 所示。

代码清单 6-6 Intent 语法

```
Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse(urlString));
```

Intent 构造函数的第 1 个参数是 Intent 需要执行的动作；第 2 个参数是 Uri，表示需要传递的数据。

Android 系统支持的常见动作字符串常量表，如表 6-3 所示。

表 6-3 Android 系统支持的常见动作字符串常量表

动 作	说 明
ACTION_ANSWER	打开接听电话的 Activity，默认为 Android 内置的拨号盘界面
ACTION_CALL	打开拨号盘界面并拨打电话，使用 Uri 中的数字部分作为电话号码
ACTION_DELETE	打开一个 Activity，对所提供的数据进行删除操作
ACTION_DIAL	打开内置拨号盘界面，显示 Uri 中提供的电话号码
ACTION_EDIT	打开一个 Activity，对所提供的数据进行编辑操作



ACTION_INSERT	打开一个 Activity，在提供数据的当前位置插入新项
ACTION_PICK	启动一个子 Activity，从提供的数据列表中选择一项
ACTION_SEARCH	启动一个 Activity，执行搜索动作
ACTION_SENDTO	启动一个 Activity，向数据提供的联系人发送信息
ACTION_SEND	启动一个可以发送数据的 Activity
ACTION_VIEW	最常用的动作，对以 Uri 方式传送的数据，根据 Uri 协议部分以最佳方式启动相应的 Activity 进行处理。对于 http:address 将打开浏览器查看；对于 tel:address 将打开拨号呼叫指定的电话号码
ACTION_WEB_SEARCH	打开一个 Activity，对提供的数据进行 Web 搜索

以下将通过一个 WebViewIntentDemo 示例来了解如何隐式启动 Activity。

如图 6-3 所示，当用户在文本框中输入要访问的网址后，通过单击“Go”按钮，程序根据用户输入的网址生成一个 Intent，并以隐式启动的方式调用 Android 内置的 Web 浏览器，并打开指定的 Web 页面。本例输入的网址是 google 主页的主站地址，地址是：<http://www.google.com>。

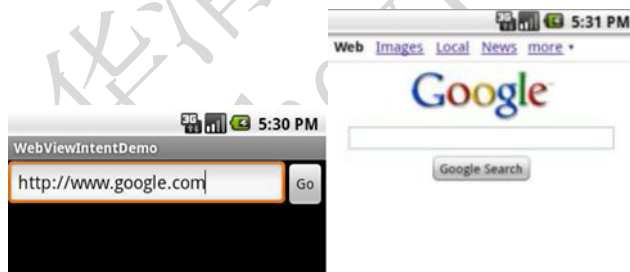


图 6-3 隐式启动 Activity

其中，main.xml 代码如代码清单 6-7 所示。

代码清单 6-7 main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:orientation="horizontal"
android:layout_width="fill_parent"
android:layout_height="fill_parent" >
<EditText
    android:id="@+id/url_field"
    android:layout_width="wrap_content"
```



```
        android:layout_height="wrap_content"
        android:layout_weight="1.0"
        android:lines="1"
        android:inputType="textUri"
        android:imeOptions="actionGo" />
<Button
    android:id="@+id/go_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/go_button" />
</LinearLayout>
```

其中，strings.xml 代码如代码清单 6-8 所示。

代码清单 6-8 strings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name"> WebViewIntentDemo </string>
    <string name="go_button">Go</string>
</resources>
```

WebViewIntentDemo.java 代码如代码清单 6-9 所示。

代码清单 6-9 WebViewIntentDemo.java

```
import android.app.Activity;
import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.view.KeyEvent;
import android.view.View;
import android.view.View.OnClickListener;
import android.view.View.OnKeyListener;
import android.widget.Button;
import android.widget.EditText;

public class WebViewIntentDemo extends Activity {
    private EditText urlText;
    private Button goButton;
```



```
@Override
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main);

    // Get a handle to all user interface elements
    urlText = (EditText) findViewById(R.id.url_field);
    goButton = (Button) findViewById(R.id.go_button);

    // Setup event handlers
    goButton.setOnClickListener(new OnClickListener() {
        public void onClick(View view) {
            openBrowser();
        }
    });

    urlText.setOnKeyListener(new OnKeyListener() {
        public boolean onKey(View view, int keyCode, KeyEvent event) {
            if (keyCode == KeyEvent.KEYCODE_ENTER) {
                openBrowser();
                return true;
            }
            return false;
        }
    });
}
```

在上述代码中第 26 行对按钮“Go”添加监听，当触摸单击或通过手机按键单击该按钮时，触发 openBrowser() 方法。其中，openBrowser() 方法代码如代码清单 6-10 所示。

代码清单 6-10 openBrowser() 方法

```
/** Open a browser on the URL specified in the text box */
private void openBrowser() {
    Uri uri = Uri.parse(urlText.getText().toString());
    Intent intent = new Intent(Intent.ACTION_VIEW, uri);
    startActivity(intent);
}
```



```
}
```

16.2.2 获取 Activity 返回值

在 6.2.1 节的 IntentDemo 示例中，通过使用 startActivity(Intent) 方法启动 Activity 后，两个 Activity 之间相互独立，没有任何关联。然而，在很多情况下，后启动的 Activity 是为了让用户对特定信息进行选择，在关闭这个 Activity 后，用户的选择信息需要返回给未关闭的那个 Activity。由此，按照 Activity 启动的先后顺序，先启动的称为父 Activity，后启动的称为子 Activity，如果需要将子 Activity 的部分信息返回给父 Activity，则可以使用 Sub-Activity 的方式去启动子 Activity。

获取子 Activity 的返回值，一般分以下 3 个步骤。

以 Sub-Activity 的方式启动子 Activity

首先，调用 startActivityForResult(Intent, requestCode) 函数，其中，参数 Intent 用于决定启动哪个 Activity，参数 requestCode 是唯一的标识子 Activity 的请求码。

显式启动子 Activity 的代码如代码清单 6-11 所示。

代码清单 6-11 显式启动子 Activity

```
int SUBACTIVITY1 = 1;
Intent intent = new Intent(this, SubActivity1.class);
startActivityForResult(intent, SUBACTIVITY1);
```

隐式启动子 Activity 的代码如代码清单 6-12 所示。

代码清单 6-12 隐式启动子 Activity

```
int SUBACTIVITY2 = 2;
Uri uri = Uri.parse("content://contacts/people");
Intent intent = new Intent(Intent.ACTION_PICK, uri);
startActivityForResult(intent, SUBACTIVITY2);
```

设置子 Activity 的返回值

在子 Activity 调用 finish() 函数关闭前，调用 setResult() 函数将所需的数据返回给父 Activity。其中，setResult() 函数有两个参数：结果码和返回值。结果码表明了子 Activity 的返回状态，通常为 Activity.RESULT_OK 或 Activity.RESULT_CANCELED，或自定义的结果码，结果码均为整数类型；返回值封装在 Intent 中，子 Activity 通过 Intent 将需要返回的数据传递给父 Activity。数据主要是 Uri 形式，可以附加一些额外信息，这些额外信息用 Extra 的集合表示。

代码清单 6-13 所示代码说明了如何在子 Activity 中设置返回值。

代码清单 6-13 在子 Activity 中设置返回值

```
Uri data = Uri.parse("tel:" + tel_number);
Intent result = new Intent(null, data);
result.putExtra("address", "(");
setResult(RESULT_OK, result);
finish();
```



在父 Activity 中获取返回值

当子 Activity 关闭时，启动它的父 Activity 的 `onActivityResult()` 函数将被调用；如果需要在父 Activity 中处理子 Activity 的返回值，则覆盖此函数即可。此函数的语法如代码清单 6-14 所示。

代码清单 6-14 `onActivityResult()` 语法

```
public void onActivityResult(int requestCode, int resultCode, Intent data);
```

在上述代码中，第 1 个参数 `requestCode`，用来表示是哪一个子 Activity 的返回值；第 2 个参数 `resultCode` 用于表示子 Activity 的返回状态；第 3 个参数 `data` 是子 Activity 的返回数据，返回数据类型是 `Intent`。返回数据的用途不同，`Uri` 数据的协议也就不同。也可以使用 `Extra` 方法返回一些原始类型的数据。

代码清单 6-15 所示代码说明如何在父 Activity 中处理子 Activity 的返回值。

代码清单 6-15 在父 Activity 中处理子 Activity 的返回值

```
private static final int SUBACTIVITY1 = 1;
private static final int SUBACTIVITY2 = 2;

@Override
public void onActivityResult(int requestCode, int resultCode, Intent data){
    Super.onActivityResult(requestCode, resultCode, data);
    switch(requestCode){
        case SUBACTIVITY1:
            if (resultCode == Activity.RESULT_OK){
                Uri uriData = data.getData();
            }else if (resultCode == Activity.RESULT_CANCELED){
            }
            break;
        case SUBACTIVITY2:
            if (resultCode == Activity.RESULT_OK){
                Uri uriData = data.getData();
            }
            break;
    }
}
```

在上述代码中，第 1 行代码和第 2 行代码是两个子 Activity 的请求码；第 7 行代码对请求码进行匹配；第 9 行和第 11 行代码对结果码进行判断，如果返回的结果码是 `Activity.RESULT_OK`，则在代码的第 10 行使用 `getData()` 函数获取 `Intent` 中的 `Uri` 数据；反之，若返回的结果码是 `Activity.RESULT_CANCELED`，则不进行任何操作。



ActivityCommunication 示例说明了如何以 Sub-Activity 方式启动子 Activity，以及使用 Intent 进行组件间的通信。



图 6-4 ActivityCommunication 主界面

在如图 6-4 所示的主界面中，当用户单击“启动 Activity1”和“启动 Activity2”按钮时，程序将分别启动两个不同的子 Activity：子 SubActivity1 和 SubActivity2，它们的界面分别如图 6-5 所示。



图 6-5 SubActivity1 和 SubActivity2 界面

SubActivity1 提供了一个输入框，以及“接受”和“撤销”两个按钮，如果在输入框中输入信息后单击“接受”按钮，程序会把输入框中的信息传递给父 Activity，并在父 Activity 界面上显示；如果用户单击“撤销”按钮，则程序不会向父 Activity 传递任何信息。

SubActivity2 主要为了说明如何在父 Activity 中处理多个子 Activity，因此仅提供了用于关闭 SubActivity2 的“关闭”按钮。

下面介绍 ActivityCommunication 的核心代码。

首先是 ActivityCommunication.java 文件，如代码清单 6-16 所示。

代码清单 6-16 ActivityCommunication.java

```
public class ActivityCommunication extends Activity {
    private static final int SUBACTIVITY1 = 1;
    private static final int SUBACTIVITY2 = 2;
    TextView textView;
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
    }
}
```



```
textView = (TextView)findViewById(R.id.textShow);
final Button btn1 = (Button)findViewById(R.id.btn1);
final Button btn2 = (Button)findViewById(R.id.btn2);

btn1.setOnClickListener(new OnClickListener(){
    public void onClick(View view){
        Intent intent = new Intent(ActivityCommunication.this, SubActivity1.class);
        startActivityForResult(intent, SUBACTIVITY1);
    }
});

btn2.setOnClickListener(new OnClickListener(){
    public void onClick(View view){
        Intent intent = new Intent(ActivityCommunication.this, SubActivity2.class);
        startActivityForResult(intent, SUBACTIVITY2);
    }
});
}

@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);

    switch(requestCode){
        case SUBACTIVITY1:
            if (resultCode == RESULT_OK){
                Uri uriData = data.getData();
                textView.setText(uriData.toString());
            }
            break;
        case SUBACTIVITY2:
            break;
    }
}
}
```



在上述代码中，第 2 行和第 3 行分别定义了两个子 Activity 的请求码，而在第 16 行和第 23 行以 Sub-Activity 的方式分别启动两个子 Activity。

第 29 行代码是子 Activity 关闭后的返回值处理函数，其中 requestCode 是子 Activity 返回的请求码，应与第 2 行和第 3 行定义的两个请求码相匹配；resultCode 是结果码，在第 32 行代码对结果码进行判断，如果等于 RESULT_OK，则在第 35 行代码获取子 Activity 的返回值中的数据，其中，data 是返回值，子 Activity 需要返回的数据就保存在 data 中。

SubActivity1.java 的核心代码如代码清单 6-17 所示。

代码清单 6-17 SubActivity1.java

```
public class SubActivity1 extends Activity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.subactivity1);  
        final EditText editText = (EditText)findViewById(R.id.edit);  
        Button btnOK = (Button)findViewById(R.id.btn_ok);  
        Button btnCancel = (Button)findViewById(R.id.btn_cancel);  
  
        btnOK.setOnClickListener(new OnClickListener(){  
            public void onClick(View view){  
                String uriString = editText.getText().toString();  
                Uri data = Uri.parse(uriString);  
                Intent result = new Intent(null, data);  
                setResult(RESULT_OK, result);  
                finish();  
            }  
        });  
  
        btnCancel.setOnClickListener(new OnClickListener(){  
            public void onClick(View view){  
                setResult(RESULT_CANCELED, null);  
                finish();  
            }  
        });  
    }  
}
```



上述 SubActivity1.java 的核心代码中，第 13 行代码将 EditText 控件的内容作为数据保存在 Uri 中；第 14 行代码中使用这个 Uri 构造 Intent；第 15 行代码中，将 Intent 作为返回值，RESULT_OK 作为结果码，通过调用 setResult() 函数，将返回值和结果码传递给父 Activity；第 16 行代码调用 finish() 函数关闭当前的子 Activity。

SubActivity2.java 的核心代码如代码清单 6-18 所示。

代码清单 6-18 SubActivity2.java

```
public class SubActivity2 extends Activity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.subactivity2);  
  
        Button btnReturn = (Button)findViewById(R.id.btn_return);  
        btnReturn.setOnClickListener(new OnClickListener(){  
            public void onClick(View view){  
                setResult(RESULT_CANCELED, null);  
                finish();  
            }  
        });  
    }  
}
```

上述 SubActivity2.java 的核心代码中，第 10 行的 setResult() 函数仅设置了结果码，第 2 个参数为 null，表示数据需要传递给父 Activity。

16.3 Intent 过滤器

Intent 过滤器是一种根据 Intent 中的动作 (Action)、类别 (Category) 和数据 (Data) 等内容，对适合接收该 Intent 的组件进行匹配和筛选的机制。它可以匹配数据类型、路径和协议，还可以用来确定多个匹配项顺序的优先级 (Priority)。

应用程序的 Activity 组件、Service 组件和 BroadcastReceiver 都可以注册 Intent 过滤器，这些组件在特定的数据格式上就可以产生相应的动作。

16.3.1 注册 Intent 过滤器

注册 Intent 过滤器的方法如下：

在 AndroidManifest.xml 文件的各个组件的节点下定义 <intent-filter> 节点，然后在 <intent-filter> 节点中声明该组件所支持的动作、执行的环境和数据格式等信息。

<intent-filter> 节点支持 <action> 标签、<category> 标签和 <data> 标签，其中：<action> 标签定义 Intent



过滤器的“类别”；<category>标签定义 Intent 过滤器的“动作”；<data>标签定义 Intent 过滤器的“数据”。

<intent-filter>节点支持的标签和属性如表 6-4 所示。

表 6-4 <intent-filter>节点支持的标签和属性

标 签	属 性	说 明
<action>	Android:name	指定组件所能响应的动作，用字符串表示，通常使用 Java 类名和包的完全限定名构成
<category>	Android:category	指定以何种方式去服务 Intent 请求的动作
<data>	Android:host	指定一个有效的主机名
	Android:mimetype	指定组件能处理的数据类型
	Android:path	有效的 Uri 路径名
	Android:port	主机的有效端口号
	Android:scheme	所需要的特定的协议

<category>标签用来指定 Intent 过滤器的服务方式，每个 Intent 过滤器可以定义多个<category>标签，程序开发人员可使用自定义的类别，或使用 Android 系统提供的类别。其中，Android 系统提供的类别如表 6-5 所示。

表 6-5 Android 系统提供的类别

值	说 明
ALTERNATIVE	Intent 数据默认动作的一个可替换的执行方法
SELECTED_ALTERNATIVE	和 ALTERNATIVE 类似，但替换的执行方法不是指定的，而是被解析出来的
BROWSABLE	声明 Activity 可以由浏览器启动

续表

值	说 明
DEFAULT	为 Intent 过滤器中定义的数据提供默认动作
HOME	设备启动后显示的第一个 Activity
LAUNCHER	在应用程序启动时首先被显示

AndroidManifest.xml 文件中的每个组件的<intent-filter>都被解析成一个 Intent 过滤器对象。当应用程序安装到 Android 系统时，所有的组件和 Intent 过滤器都会注册到 Android 系统中。这样，Android 系统便知道了如何将任意一个 Intent 请求通过 Intent 过滤器映射到相应的组件上。



16.3.2 Intent 解析

Intent 到 Intent 过滤器的映射过程称为“Intent 解析”。Intent 解析可以在所有组件中找到一个可与请求的 Intent 达成最佳匹配的 Intent 过滤器。

Intent 解析的匹配规则。

Android 系统把所有应用程序包中的 Intent 过滤器集合在一起，形成一个完整的 Intent 过滤器列表。

在 Intent 与 Intent 过滤器进行匹配时，Android 系统会将列表中所有 Intent 过滤器的“动作”和“类别”与 Intent 进行匹配，任何不匹配的 Intent 过滤器都将被过滤掉。没有指定“动作”的 Intent 过滤器可以匹配任何的 Intent，但没有指定“类别”的 Intent 过滤器只能匹配没有“类别”的 Intent。

把 Intent 数据 Uri 的每个子部与 Intent 过滤器的<data>标签中的属性进行匹配，如果<data>标签指定了协议、主机名、路径名或 MIME 类型，那么这些属性都要与 Intent 的 Uri 数据部分进行匹配，任何不匹配的 Intent 过滤器均被过滤掉。

如果 Intent 过滤器的匹配结果多于一个，则可以根据在<intent-filter>标签中定义的优先级标签来对 Intent 过滤器进行排序，优先级最高的 Intent 过滤器将被选择。

在此以 6.2.1 节中隐式启动 Activity 的例子 WebViewIntentDemo 为基础，在 AndroidManifest.xml 文件中注册 Intent 过滤器，以及设置<intent-filter>节点属性来捕获指定的 Intent。

在 AndroidManifest.xml 中添加如代码清单 6-19 所示。

代码清单 6-19 AndroidManifest.xml 中添加代码

```
<intent-filter>
<action android:name="android.intent.action.VIEW" />
<data android:schema="http" />
</intent-filter>
```

利用 <intent-filter> 可以把应用程序的操作注册到系统中，当用户调用 Intent 时，可以根据输入的 ACTION 和 Uri 参数来找到这个应用程序。例如，在上述代码中，以 http 协议为例，打开 Google Map 可以用 URI: geo:38.899533,-77.036476。

此外，“协议”在 Android 里都可以随便定义。例如，写一个打开文件的关联 Intent，如 <file:///sdcard/abc.txt>。也可以用 type 进行关联，其代码如代码清单 6-20 所示。

代码清单 6-20 用 type 进行关联

```
<intent-filter>
<action android:name="android.intent.action.VIEW" />
<type android:value="test.item"/>
</intent-filter>
```

然后，通过代码清单 6-21 所示代码就能定位到应用。

代码清单 6-21 用 type 定位到应用代码

```
Intent it = new Intent(Intent.ACTION_VIEW);
it.setType("test.item");
```



startActivity(it)

16.4 广播消息

应用程序和 Android 系统都可以使用 Intent 发送广播消息。其中，广播消息的内容可以是与应用程序密切相关的数据信息，也可以是 Android 的系统信息，例如，网络连接变化、电池电量变化、接收短信和系统设置变化等。如果应用程序注册了 **BroadcastReceiver**，则可以接收到指定的广播消息。

下面将介绍广播信息的使用方法。

首先，创建一个 Intent。调用 **sendBroadcast()** 函数，就可把 Intent 携带的消息广播出去，如果要在 Intent 传递额外数据，可以用 Intent 的 **putExtra()** 方法。

利用 Intent 发送广播消息，并添加了额外的数据，然后调用 **sendBroadcast()** 发送广播消息的代码如代码清单 6-22 所示。

代码清单 6-22 利用 Intent 发送广播消息

```
String UNIQUE_STRING = "com.example.BroadcastReceiverDemo";
Intent intent = new Intent(UNIQUE_STRING);
intent.putExtra("key1", "value1");
intent.putExtra("key2", "value2");
sendBroadcast(intent);
```

广播消息发送后，利用 **BroadcastReceiver** 监听广播消息。具体方法如下：在 **AndroidManifest.xml** 文件或在代码中注册一个 **BroadcastReceiver**，并在其中使用 Intent 过滤器指定要处理的广播消息。在 **BroadcastReceiver** 接收到与之匹配的广播消息后，**onReceive()** 方法会被调用；**onReceive()** 方法必须要在 5 秒内执行完毕，否则 Android 系统会认为该组件失去响应，并提示用户强行关闭该组件。

创建 **BroadcastReceiver** 需继承 **BroadcastReceiver** 类，并重载 **onReceive()** 方法。代码如代码清单 6-23 所示。

代码清单 6-23 利用 **BroadcastReceiver** 监听广播消息

```
public class MyBroadcastReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        //TODO: React to the Intent received.
    }
}
```

BroadcastReceiver 的应用程序不需要一直运行，当 Android 系统接收到与之匹配的广播消息时，会自动启动此 **BroadcastReceiver**。基于以上特征，**BroadcastReceiver** 适合做一些资源管理的工作。

如图 6-6 所示，**BroadcastReceiverDemo** 示例说明了如何在应用程序中注册 **BroadcastReceiver**，并接收指定类型的广播消息。



图 6-6 BroadcastReceiverDemo 示例

如图 6-6 所示，在单击“发送广播消息”按钮后，EditText 控件中内容将以广播消息的形式发送出去，示例内部的 BroadcastReceiver 将接收这个广播消息，并显示在用户界面的下方。

BroadcastReceiverDemo.java 文件中包含发送广播消息的代码，其关键代码如代码清单 6-24 所示。

代码清单 6-24 BroadcastReceiverDemo.java

```
button.setOnClickListener(new OnClickListener(){
    public void onClick(View view){
        Intent intent = new Intent("com.example.BroadcastReceiverDemo");
        intent.putExtra("message", entryText.getText().toString());
        sendBroadcast(intent);
    }
});
```

在上述代码中，第 3 行代码创建 Intent，将 com.example.BroadcastReceiverDemo 作为识别广播消息的字符串标识；第 4 行代码添加了额外信息；第 5 行代码调用 sendBroadcast() 函数发送广播消息。

为了能够使应用程序中的 BroadcastReceiver 接收指定的广播消息，首先要在 AndroidManifest.xml 文件中添加 Intent 过滤器，声明 BroadcastReceiver 可以接收的广播消息。其中，AndroidManifest.xml 文件的完整代码如代码清单 6-25 所示。

代码清单 6-25 AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.BroadcastReceiverDemo"
    android:versionCode="1"
    android:versionName="1.0">
    <application android:icon="@drawable/icon" android:label="@string/app_name">
        <activity android:name=".BroadcastReceiverDemo"
            android:label="@string/app_name">
```




```
<intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
</activity>
<receiver android:name=".MyBroadcastReceiver">
    <intent-filter>
        <action android:name="com.example.BroadcastReceiverDemo" />
    </intent-filter>
</receiver>
</application>
<uses-sdk android:minSdkVersion="3" />
</manifest>
```

在上述代码中，第 14 行代码中创建了一个<receiver>节点；在第 15 行中声明了 Intent 过滤器的动作为“com.example.BroadcastReceiverDemo”，这与 BroadcastReceiverDemo.java 文件中 Intent 的动作相一致，表明这个 BroadcastReceiver 可以接收动作为“com.example.BroadcastReceiverDemo”的广播消息。

MyBroadcastReceiver.java 文件创建了一个自定义的 BroadcastReceiver，其核心代码如代码清单 6-26 所示。

代码清单 6-26 MyBroadcastReceiver.java

```
public class MyBroadcastReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        String msg = intent.getStringExtra("message");
        Toast.makeText(context, msg, Toast.LENGTH_SHORT).show();
    }
}
```

在上述代码中，第 1 行代码首先继承了 BroadcastReceiver 类；第 3 行代码重载了 onReceive() 函数，当接收到 AndroidManifest.xml 文件定义的广播消息后，程序将自动调用 onReceive() 函数。

16.5 本章小结

本章主要对 Android 组件间的通信进行学习，包括 Intent 进行组件通信的原理、Intent 启动 Activity 的方法、获取 Activity 返回值的方法、Intent 过滤器的原理及其匹配机制、发送和接收广播消息的方法。



第 17 章 数据存储与访问

在上一章的学习中，主要了解了 Android 组件间的通信，包括 Intent 进行组件通信的原理、Intent 启动 Activity 的方法、获取 Activity 返回值的方法、Intent 过滤器的原理及其匹配机制、发送和接收广播消息的方法等。本章主要学习简单存储的使用方法、文件存储的类别以及区别和适用情况，数据库存储的特点与操作，以及数据共享的原理与用途等。

17.1 简单存储

17.1.1 SharedPreferences

SharedPreferences 是 Android 平台上的一种轻量级的数据保存方式，主要用来存储一些简单的配置信息，例如一些默认欢迎语、登录的用户名和密码等。通过 SharedPreferences 可以将 NVP（Name-Value Pair，名称-值对）保存在 Android 的文件系统中，而且 SharedPreferences 对文件系统的操作过程完全屏蔽，这使得开发人员仅需通过调用 SharedPreferences 对 NVP 很方便地进行保存和读取。SharedPreferences 不仅能够保存数据，还能够实现不同应用程序间的数据共享。

SharedPreferences 支持 3 种访问模式。

私有（MODE_PRIVATE）：只有创建程序有读取或写入 SharedPreferences 的权限。

全局读（MODE_WORLD_READABLE）：不仅创建程序有读取或写入 SharedPreferences 的权限，其他应用程序也有读取操作的权限，但没有写入操作的权限。

全局写（MODE_WORLD_WRITEABLE）：创建程序和其他程序都可以对其进行写入操作，但没有读取的权限。

在使用 SharedPreferences 之前，需先定义 SharedPreferences 的访问模式。代码清单 7-1 所示的代码将访问模式定义为私有模式。

代码清单 7-1 定义 SharedPreferences 私有访问模式

```
public static int MODE = MODE_PRIVATE;
```

有的时候需要将 SharedPreferences 的访问模式设定为全局读/写模式，这时就需要其既可以全局读 MODE_WORLD_READABLE，又可以全局写 MODE_WORLD_WRITEABLE，这样就需要写成代码清单 7-2 所示的代码方式：

代码清单 7-2 定义 SharedPreferences 全局读/写模式

```
public static int MODE = Context.MODE_WORLD_READABLE +  
Context.MODE_WORLD_WRITEABLE;
```

在定义完 SharedPreferences 的访问模式之后，需定义 SharedPreferences 的名称（如代码清单 7-3 所示），这个名称与在 Android 文件系统中保存的文件同名。由此可知，若 NVP 内容具有相同的 SharedPreferences 名称，它们那么都会保存在同一个文件中。

代码清单 7-3 定义 SharedPreferences 的名称

```
public static final String PREFERENCE_NAME = "SETTING_Infos";
```

为了可以使用 SharedPreferences，需要将访问模式和 SharedPreferences 名称作为参数，传递到



getSharedPreferences()方法中，方法返回一个 SharedPreferences 对象，代码如代码清单 7-4 所示。

代码清单 7-4 getSharedPreferences()方法，获取 SharedPreferences 对象

```
SharedPreferences sharedPreferences = getSharedPreferences
(PREFERENCE_NAME, MODE);
```

在获取到 SharedPreferences 对象后，则可以通过 SharedPreferences.Editor 类对 SharedPreferences 进行修改（如代码清单 7-5 所示），最后调用 commit()方法保存修改内容。

其中，SharedPreferences 广泛支持包括整型、布尔型、浮点型和长型等在内的各种基本数据类型。

代码清单 7-5 SharedPreferences.Editor 对 SharedPreferences 修改

```
SharedPreferences.Editor editor = sharedPreferences.edit();
editor.putString("NAME", "Tom");
editor.putFloat("HEIGHT", 1.81f);
editor.commit();
```

以上讲述了如何修改 SharedPreferences 的数据，那么如何读取其中的数据呢？对于一个已经保存了的 SharedPreferences，若是从其中读取数据，则首先调用 getSharedPreferences()方法，并在方法的第 1 个参数中指明需要访问的 SharedPreferences 名称，获取到 SharedPreferences 对象之后，先通过 get<Type>()方法获取保存在 SharedPreferences 中的 NVP，get<Type>()方法的第 1 个参数是 NVP 的名称，第 2 个参数是在无法获取到数值的时候使用的默认值。代码如代码清单 7-6 所示。

代码清单 7-6 SharedPreferences.Editor 对 SharedPreferences 修改

```
SharedPreferences sharedPreferences = getSharedPreferences
(PREFERENCE_NAME, MODE);

String name = sharedPreferences.getString("NAME", "");
String height = sharedPreferences.getString("HEIGHT", "");
```

17.1.2 示例

下面通过 SimplePreferenceDemo 示例介绍具体说明 SharedPreferences 的文件保存位置和保存格式及其读取。

图 7-1 是 SimplePreferenceDemo 示例的用户界面。

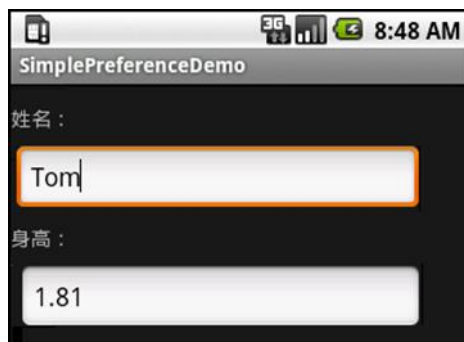




图 7-1 SimplePreferenceDemo 示例的用户界面

在图 7-1 所示的界面中，用户在界面上输入信息，然后退出这个应用，即关闭 Activity 时将 SharedPreferences 进行保存。当在应用程序列表中找到这个应用重新开启时，保存在 SharedPreferences 的信息（刚刚输入的信息）将被读取出来，并重新呈现在用户界面上。

界面代码如代码清单 7-7 所示。

代码清单 7-7 main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    >

    <TextView
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="姓名： "
        >
    </TextView>

    <EditText
        android:id="@+id/name"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text=""
        >
    </EditText>

    <TextView
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="身高： "
        >
    </TextView>
```



```
<EditText
    android:id="@+id/height"
    android:layout_width="fill_parent"
    android:layout_height="wrap_content"
    android:text=""
>

</EditText>
```

```
</LinearLayout>
```

主要实现代码如代码清单 7-8 所示。

代码清单 7-8 SimplePreferenceDemo.java

```
package cn.com.farsight. SimplePreferenceDemo;
```

```
import android.app.Activity;
```

```
import android.content.SharedPreferences;
```

```
import android.os.Bundle;
```

```
import android.widget.EditText;
```

```
public class SharedPreferencesDemo extends Activity {
```

```
    public static final String SETTING_INFOS = "SETTING_Infos";
```

```
    public static final String NAME = "NAME";
```

```
    public static final String HEIGHT = "HEIGHT ";
```

```
    private EditText field_name; //接收用户名的组件
```

```
    private EditText filed_height; //接收身高的组件
```

```
@Override
```

```
public void onCreate(Bundle savedInstanceState) {
```

```
    super.onCreate(savedInstanceState);
```

```
    setContentView(R.layout.main);
```

```
//Find View
```

```
field_name = (EditText) findViewById(R.id.name); //首先获取用来输入用户名的组件
```

```
filed_height = (EditText) findViewById(R.id.height); //同时也需要获取输入身高的组件
```

```
// Restore preferences
```



```
//获取一个 SharedPreferences 对象
SharedPreferences settings =
    getSharedPreferences(SETTING_INFOS, 0);

String name = settings.getString(NAME, "");           //取出保存的 NAME
String height = settings.getFloat(HEIGHT, "").toString(); //取出保存的 HEIGHT

//Set value
field_name.setText(name); //将取出来的用户名赋予 field_name
filed_height.setText(height); //将取出来的身高赋予 filed_height
}

@Override
protected void onStop(){
    super.onStop();
    //首先获取一个 SharedPreferences 对象
    SharedPreferences settings =
        getSharedPreferences(SETTING_INFOS, 0);
    settings.edit()
        .putString(NAME, field_name.getText().toString())
        .putFloat(HEIGHT,
            Float.parseFloat(filed_height.getText().toString())
        ).commit();
    } //将用户名和身高保存进去
}
```

通过上述代码可以看出，在 onCreate 中使用 findViewById 得到两个 EditText 后，使用 getSharedPreferences 取得 SharedPreferences 对象 settings，然后使用 getString 取得其中保存的值，最后使用 setText 将其值设置为两个 EditText 的值。

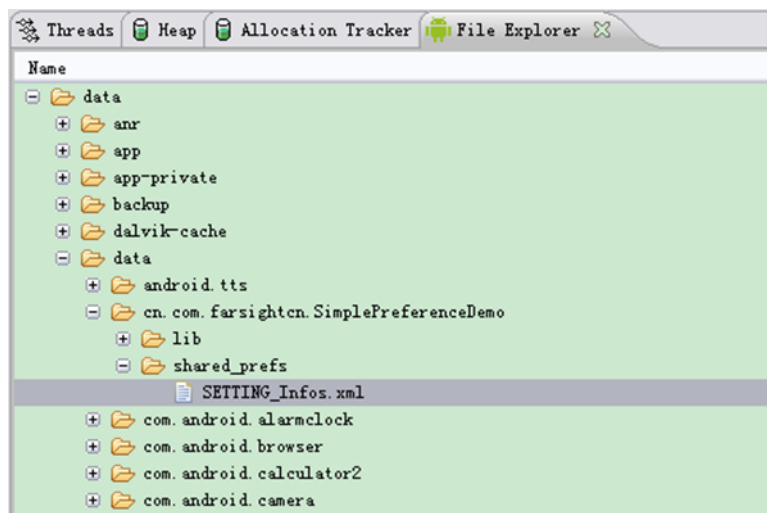
而在程序运行 onStop 过程，也就是在程序退出时，首先使用 getSharedPreferences 得到 settings；其次调用 edit()方法使其处于可以编辑状态，并使用 putString 将两个 EditText 中的值保存起来；最后使用 commit()方法提交即可保存。

SharedPreferences 保存到哪里去了？

SharedPreferences 是以 XML 的格式以文件的方式自动保存的，在 SimplePreferenceDemo 示例运行后，DDMS 中的 File Explorer 展开到/data/data，可以看到 Android 为每个应用程序建立了与包同名的目录，用来保存应用程序产生的数据，这些数据包括文件、SharedPreferences 文件和数据库等；将目录展开



到/data/data/<package name>/shared_prefs 下，以上面这个为例，可以看到一个叫做 SETTING_Infos.xml 的文件，如图 7-2 所示。



将 SETTING_Infos.xml 导出至设备，其代码如代码清单 7-9 所示。

代码清单 7-9 SaveSetting.xml

```
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <float name="Height" value="1.81" />
  <string name="Name">Tom</string>
</map>
```

17.2 文件存储

由于 Android 使用的文件系统是基于 Linux 的文件系统，这就允许程序开发人员建立和访问程序自身的私有文件，也可以访问保存在资源目录中的原始文件和 XML 文件，还可以保存文件在 SD 卡等外部存储设备中。

17.2.1 内部存储

Android 系统允许应用程序创建仅能够自身访问的私有文件，文件保存在设备的内部存储器上的 /data/data/<package name>/files 目录中。和传统的 Java 中实现 I/O 的程序类似，在 Android 中，其提供了 openFileInput 和 openFileOutput 方法读取设备上的文件。

openFileOutput()

openFileOutput()方法打开的应用程序私有文件，为写入数据做准备。在默认情况下，写入的时候会覆盖原文件内容，如果想把新写入的内容附加到原文件内容后，则可以指定其 mode 为



Context.MODE_APPEND；如果指定的打开文件不存在时，则创建一个新的文件。默认情况下，使用 openFileOutput 方法创建的文件只能被其调用的应用使用，其他应用无法读取这个文件。

openFileOutput()方法的语法格式如下：

代码清单 7-10 openFileOutput()

```
public FileOutputStream openFileOutput(String name, int mode)
```

其中，第 1 个参数是文件名称，这个参数不可以包含描述路径的斜杠，也就是说，它并不允许我们去修改保存文件的路径；第 2 个参数是操作模式；方法的返回值是 FileOutputStream 类型。

关于第 2 个参数——操作模式来讲，Android 系统支持 4 种文件操作模式，具体如表 7-1 所示。

表 7-1 Android 系统支持 4 种文件操作模式

模式	说明
MODE_PRIVATE	私有模式，缺陷模式，文件仅能够被文件创建程序访问，或具有相同 UID 的程序访问
MODE_APPEND	追加模式，如果文件已经存在，则在文件的结尾处添加新数据
MODE_WORLD_READABLE	全局读模式，允许任何程序读取私有文件
MODE_WORLD_WRITEABLE	全局写模式，允许任何程序写入私有文件

使用 openFileOutput()方法建立新文件的示例代码如代码清单 7-11 所示。

代码清单 7-11 openFileOutput()建立新文件

```
//建立文件的名称 fileDemo.txt
String FILE_NAME = "fileDemo.txt";
//使用 openFileOutput()方法以私有模式建立文件
FileOutputStream fos = openFileOutput
                        (FILE_NAME,Context.MODE_PRIVATE);

String text = "Some data";
//将数据写入文件
fos.write(text.getBytes());
//将所有剩余的数据写入文件
fos.flush();
//关闭 FileOutputStream
fos.close();
```

openFileInput()

openFileInput()方法打开应用程序私有文件为从其中读取数据而做准备。openFileInput()方法的语法格式如代码清单 7-12 所示。

代码清单 7-12 openFileInput()

```
public FileInputStream openFileInput (String name)
```




第 1 个参数是文件名称，同样不允许包含描述路径的斜杠。

使用 `openFileInput()` 方法打开已有文件的示例代码如代码清单 7-13 所示。

代码清单 7-13 `openFileInput()` 打开已有文件

```
String FILE_NAME = "fileDemo.txt";
FileInputStream fis = openFileInput(FILE_NAME);
```

```
byte[] readBytes = new byte[fis.available()];
while(fis.read(readBytes) != -1){
}
```

上面的两部分代码在实际使用过程中会遇到错误提示，在实际使用过程中，应该使用 `try/catch` 捕获可能产生的异常来避免因为文件操作遇到的各种问题而导致的操作失败。

`InternalFileDemo` 示例用来演示在内部存储器上进行文件写入和读取，示例的用户界面如图 7-3 所示



图 7-3 `InternalFileDemo` 示例用户界面图

`InternalFileDemo` 示例的核心代码如代码清单 7-14 所示。

代码清单 7-14 `InternalFileDemo` 示例的核心代码

```
OnClickListener writeButtonListener = new OnClickListener() {
    @Override
    public void onClick(View v) {
        FileOutputStream fos = null;
        try {
            if (appendBox.isChecked()){
                fos = openFileOutput(FILE_NAME, Context.MODE_APPEND);
            } else {
                fos = openFileOutput(FILE_NAME, Context.MODE_PRIVATE);
            }
            String text = entryText.getText().toString();
            fos.write(text.getBytes());
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
};
```



```
        labelView.setText("文件写入成功，写入长度：" + text.length());
        entryText.setText("");
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    }
    catch (IOException e) {
        e.printStackTrace();
    }
    finally{
        if (fos != null){
            try {
                fos.flush();
                fos.close();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
};

OnClickListener readButtonListener = new OnClickListener() {
    @Override
    public void onClick(View v) {
        displayView.setText("");
        FileInputStream fis = null;
        try {
            fis = openFileInput(FILE_NAME);
            if (fis.available() == 0){
                return;
            }
            byte[] readBytes = new byte[fis.available()];
            while(fis.read(readBytes) != -1){
            }
            String text = new String(readBytes);
            displayView.setText(text);
        }
```



```
        labelView.setText("文件读取成功，文件长度: "+text.length());
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    }
    catch (IOException e) {
        e.printStackTrace();
    }
}
};
```

程序运行后，可以在/data/data/cn.com.farsight.InternalFileDemo/files/目录下，找到新建立的fileDemo.txt 文件: fileDemo.txt 从文件权限上进行分析，“-rw-rw---”表明文件仅允许文件创建者和同组用户读/写，其他用户无权使用；文件内保存的数据为“Some data”，其大小为 9 个字节。

17.2.2 外部存储

在上一小节中，使用 Activity 的 openFileOutput()方法来保存文件，其保存的文件是存放在手机空间上，而一般手机的存储空间不是很大，存放些小文件还行，如果要存放像视频这样的大文件就存在困难。对于像视频这样的大文件，可以把它存放在 SDCard。

SD 卡（Secure Digital Memory Card）是 Android 的外部存储设备，是一种广泛使用于数码设备上的记忆卡。然而，SD 卡中的文件为全局可访问的，而且可以被用户操作，因此，SD 卡适用于保存大尺寸的文件或者是一些无须设置访问权限的文件，可以保存录制的大容量的视频文件和音频文件等。

Android 系统提供了对 SD 卡便捷地访问方法。Android 模拟器支持 SD 卡，可以使用以下两种方式来创建 SD 卡。

使用<Android SDK>/tools 目录下的 mkSDcard 工具创建 SD 卡映像文件，命令如代码清单 7-15 所示。

代码清单 7-15 创建 SD 卡映像文件

```
mkSDcard -l SDCARD 256M E:\android\sdcards_file
```

该命令中，第 1 个参数-l 表示后面的字符串是 SD 卡的标签，这个新建立的 SD 卡的标签是 SDCARD；第 2 个参数 256M 表示 SD 卡的容量是 256 兆；最后一个参数表示 SD 卡映像文件的保存位置，上面的命令将映像保存在 E:\android 目录下 sdcards_file 文件中。在 CMD 中执行该命令后，则可在所指定的目录中找到生产的 SD 卡映像文件。

如果希望 Android 模拟器启动时能够自动加载指定的 SD 卡，还需要在模拟器的“运行设置”（Run Configurations）中添加 SD 卡加载命令，而 SD 卡加载命令中只要指明映像文件位置即可。SD 卡加载命令如图 7-4 所示。

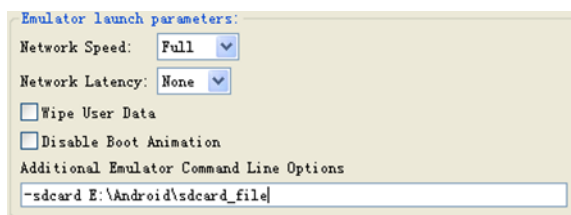


图 7-4 SD 卡加载命令

那么如何测试 SD 卡映像是否正确加载呢？在模拟器启动后，使用 FileExplorer 向 SD 卡中随意上传一个文件，如果文件上传成功，则表明 SD 卡映像已经成功加载。

在创建模拟器 Android Virtual Device (AVD) 时，自定义 SD 卡的大小，由系统来完成模拟器虚拟 SD 卡的创建，如图 7-5 所示。



图 7-5 创建模拟器时选择 SD 卡大小

创建完成后，启动该模拟器，打开 Eclipse 的 DDMS→File→Explorer，可以看到目录结构。

在加载 SD 卡后，需要通过编程来访问 SD 卡：首先需要使用外部储存之前必须调用 `getExternalStorageState()` 方法来检查存储媒介是否可用来检测系统的 `/sdcard` 目录是否可用，如果不可用，则说明设备中的 SD 卡已经被移除或者被挂载到一台电脑上、遗失、只读或处于其他状态；在 Android 模拟器则表明 SD 卡映像没有被正确加载等；如果可用，则直接通过使用标准的 `Java.io.File` 类进行访问，将数据保存在 SD 卡。

检测 SD 卡是否可用的代码如代码清单 7-16 所示。

代码清单 7-16 检验 SD 卡是否可用

```
boolean mExternalStorageAvailable = false;
boolean mExternalStorageWriteable = false;
String state = Environment.getExternalStorageState();
if (Environment.MEDIA_MOUNTED.equals(state)) {
    // 对 SD 卡上的存储可以进行读/写操作
    mExternalStorageAvailable = mExternalStorageWriteable = true;
} else
if (Environment.MEDIA_MOUNTED_READ_ONLY.equals(state))
{
```



```
//对 SD 卡上的存储可以进行读操作
mExternalStorageAvailable = true;
mExternalStorageWriteable = false;
} else {
    //对 SD 卡上的存储不可用
    mExternalStorageAvailable = mExternalStorageWriteable = false;
}
```

下面通过 SDcardFileDemo 示例说明如何将数据保存在 SD 卡。图 7-6 所示 DcardFileDemo 示例的用户界面。



图 7-6 SDcardFileDemo 示例的用户界面

在图 7-6 所示界面中，通过“生产随机数列”按钮生产 10 个随机小数；通过“写入 SD 卡”按钮将生产的数据保存在 SD 卡的目录下。SDcardFileDemo 示例运行后，在每次单击“写入 SD 卡”按钮后，都会在 SD 卡中生产一个新文件，文件名各不相同。

SDcardFileDemo 示例的核心代码如代码清单 7-17 所示。

代码清单 7-17 SDcardFileDemo 示例的核心代码

```
private static String randomNumbersString = "";
OnClickListener writeButtonListener = new OnClickListener() {
    @Override
    public void onClick(View v) {
        String fileName = "SdcardFile-"+System.currentTimeMillis()+".txt";
        File dir = new File("/sdcard/");
        if (dir.exists() && dir.canWrite()) {
            File newFile = new File(dir.getAbsolutePath() + "/" + fileName);
```



```

        FileOutputStream fos = null;
        try {
            newFile.createNewFile();
            if (newFile.exists() && newFile.canWrite()) {
                fos = new FileOutputStream(newFile);
                fos.write(randomNumbersString.getBytes());
                TextView labelView = (TextView)findViewById(R.id.label);
                labelView.setText(fileName + "文件写入 SD 卡");
            }
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            if (fos != null) {
                try{
                    fos.flush();
                    fos.close();
                }
                catch (IOException e) { }
            }
        }
    }
};

```

由上述代码可以看出，SDcardFileDemo 示例与 InternalFileDemo 示例的核心代码比较相似，但 SDcardFileDemo 示例与 InternalFileDemo 示例也有不同之处：第 7 行代码中添加了/sdcard 目录存在性检查；第 8 行代码使用“绝对目录+文件名”的形式表示新建立的文件；第 12 行代码写入文件前对文件存在性和可写入性进行检查；第 5 行代码为了保证在 SD 卡中多次写入时文件名不会重复，在文件名中使用了唯一且不重复的标识，这个标识通过调用 System.currentTimeMillis()方法获得，表示从 1970 年 00:00:00 到当前所经过的毫秒数。

17.2.3 资源文件

资源文件是在代码中使用到的并且在编译时被打包到应用程序的附加文件。Android 支持多种不同的文件，包括 XML、PNG 和 JPEG 文件。而 Android 平台中会存在很多不同的资源和资源定义文件。每一种资源定义文件的语法和格式及保存的位置取决于其依赖的对象。通常，创建资源可以通过以下 3 种文件：XML 文件（除位图和原始格式文件外），位图文件（作为图片）和原始格式文件（所有其他的类型，比如声音文件）。



XML (eXtensible Markup Language)，主要用来存储数据，使得其易于在任何应用程序中读写数据，虽然不同的应用软件也可以支持它的数据交换格式。事实上，在 Android 程序中有两种不同类型的 XML 文件：一种是作为资源被编译进应用程序；另一种是作为资源的描述，被应用程序使用。表 7-2 详细说明了这些资源文件的类型和结构。

表 7-2 资源文件

目录	资源类型
res/anim	XML 文件编译为帧序列动画或者自动动画对象
res/drawable	使用 Resources.getDrawable(id)可以获得资源类型。png、9.png、.jpg 文件被编译为 Drawable 资源子类型： * 位图文件。 * 9-patches (可变位图文件)
res/layout	资源编译为屏幕布局器
res/values	XML 文件可以被编译为多种资源 注意：不像其他 res 下的目录，这个目录可以包含多个资源描述文件。XML 文件元素类型控制着这些资源被 R 类放置在何处。 这些文件可以自定义名称。这里有一些约定俗成的文件： * arrays.xml 定义数组。 * colors.xml 定义可绘制对象的颜色和字符串的颜色。使用 Resources.getDrawable() 和 Resources.getColor() 都可以获得这些资源。 * dimens.xml 定义尺度。使用 Resources.getDimension() 可以获得这些资源。 * strings.xml 定义字符串 (使用 Resources.getString() 或者更适合的 Resources.getText() 方法获得这些资源。Resources.getText() 方法将保留所有用于描述用户界面样式的描述符，保持复杂文本的原貌。 * styles.xml 定义样式对象
res/xml	自定义的 XML 文件。这些文件将在运行时编译进应用程序，并且使用 Resources.getXML() 方法可以在运行时获取
res/raw	自定义的原始格式资源，将被直接复制给设备。这些文件将不被压缩至应用程序。使用带有 ID 参数的 Resources.getRawResource() 方法可以获得这些资源，比如 R.raw.somefilename

由表 7-2 可看出，有多种不同的 XML 文件，XML 文件描述的内容不同则决定了其不同的格式。这些文件将描述文件支持的类型、语法或格式等。

下面我们就通过 ResourceFileDemo 示例来了解如何在程序运行时访问资源文件。ResourceFileDemo 示例用户界面如图 7-7 所示。



图 7-7 ResourceFileDemo 示例用户界面图

在图 7-7 所示界面中，当用户单击“读取原始文件”按钮时，程序将读取/res/raw/raw_file.txt 文件，并将内容显示在界面上。当用户单击“读取 XML 文件”按钮时，程序将读取/res/xml/people.xml 文件，并将内容显示在界面上。

读取原始格式文件，首先需要调用 `getResource()` 方法获得资源对象，其次通过调用资源对象的 `openRawResource()` 方法，以二进制流的形式打开指定的原始格式文件。在读取文件结束后，调用 `close()` 方法关闭文件流。

ResourceFileDemo 示例中关于读取原始格式文件的核心代码如代码清单 7-18 所示。

代码清单 7-18 ResourceFileDemo 示例读取原始格式文件核心代码

```
Resources resources = this.getResources();
InputStream inputStream = null;
try {
    inputStream = resources.openRawResource(R.raw.raw_file);
    byte[] reader = new byte[inputStream.available()];
    while (inputStream.read(reader) != -1) {
    }
    //以 UTF-8 的编码方式，从字节数组中实例化一个字符串
    displayView.setText(new String(reader,"utf-8"));
} catch (IOException e) {
    Log.e("ResourceFileDemo", e.getMessage(), e);
} finally {
    if (inputStream != null) {
        try {
            inputStream.close();
        }
        catch (IOException e) { }
    }
}
```




下面介绍一下如何在程序运行时读取/res/xml 目录下的 XML 文件。

首先，在/res/xml 目录下创建一个名为 people.xml 的文件，XML 文件定义了多个<person>元素，每个<person>元素都包含三个属性 name、age 和 height，分别表示姓名、年龄和身高。/res/xml/people.xml 文件代码如代码清单 7-19 所示。

代码清单 7-19 people.xml

```
<people>
    <person name="李某某" age="21" height="1.81" />
    <person name="王某某" age="25" height="1.76" />
    <person name="张某某" age="20" height="1.69" />
</people>
```

然后，读取 XML 格式文件。通过调用资源对象的 getXml() 方法，获取到 XML 解析器 XmlPullParser，XmlPullParser 是 Android 平台标准的 XML 解析器，这项技术来自一个开源的 XML 解析 API 项目 XMLPULL。ResourceFileDemo 示例中关于读取 XML 文件的核心代码如代码清单 7-20 所示。

代码清单 7-20 ResourceFileDemo 示例读取 XML 文件的核心代码

//通过资源对象的 getXml() 方法获取到 XML 解析器

```
XmlPullParser parser = resources.getXml(R.xml.people);
String msg = "";
try {
    //parser.next()方法可以获取到高等级的解析事件，并通过对比确定事件类型
    while (parser.next() != XmlPullParser.END_DOCUMENT) {
        //使用 getName()方法获得元素的名称
        String people = parser.getName();
        String name = null;
        String age = null;
        String height = null;
        if ((people != null) && people.equals("person")) {
            //使用 getAttributeCount()方法获取元素的属性数量
            int count = parser.getAttributeCount();
            for (int i = 0; i < count; i++) {
                //通过 getAttributeName()方法得到属性名称
                String attrName = parser.getAttributeName(i);
                String attrValue = parser.getAttributeValue(i);
                //分析属性名获取到正确的属性值
                if ((attrName != null) && attrName.equals("name")) {
                    name = attrValue;
```



```

        } else if ((attrName != null) && attrName.equals("age")) {
            age = attrValue;
        } else if ((attrName != null) && attrName.equals("height")) {
            height = attrValue;
        }
    }
    //将属性值整理成需要显示的信息
    if ((name != null) && (age != null) && (height != null)) {
        msg+="姓名: "+name+", 年龄: "+age+", 身高: "+height+"\n";
    }
}
} catch (Exception e) {
    Log.e("ResourceFileDemo", e.getMessage(), e);
}
displayView.setText(msg);

```

这里介绍一下 XmlPullParser 的 XML 事件类型，如表 7-3 所示。

表 7-3 XmlPullParser 的 XML 事件类型

事件类型	说明
START_TAG	读取到标签开始标志
TEXT	读取文本内容
END_TAG	读取到标签结束标志
END_DOCUMENT	文档末尾

17.3 数据库存储

在 Android 平台上，集成了一个嵌入式关系型数据库—SQLite。SQLite3 支持 NULL、INTEGER、REAL（浮点数字）、TEXT（字符串文本）和 BLOB（二进制对象）数据类型，虽然它支持的类型只有 5 种，但实际上它也接受 varchar(n)、char(n)、decimal(p,s) 等数据类型，只不过在运算或保存时会转换成对应的 5 种数据类型。

17.3.1 SQLite 简介

SQLite 是 Android 所带的一个标准的数据库，它是在 2000 年由 D.Richard Hipp 发布的支持 SQL 语句的轻量级的嵌入式数据库。SQLite 数据库特点：更加适用于嵌入式系统，嵌入使用它的应用程序中；占用内存非常少，运行高效可靠，可移植性好；提供了零配置（zero-configuration）运行模式。SQLite 数据库不仅提高了运行效率，而且屏蔽了数据库使用和管理的复杂性，程序仅需要进行最基本的数据操作，



其他操作可以交给程序内部的数据库引擎完成。

SQLite 数据库具有很强的移植性，可以运行在 Windows、Linux、BSD、Mac OS X 和一些商用 UNIX 系统，比如，Oracle 的 Solaris、IBM 的 AIX 等；SQLite 数据库也可以工作在许多嵌入式操作系统下，例如，QNX、VxWorks、Android、Symbian 和 Windows Phone 等。

SQLite 的核心大约有 3 万行标准 C 语言代码，并且其源代码开放，代码中 95% 有较好的注释，加上其模块化的设计使这些代码更加易于理解。

SQLite 数据库的模块化设计，由 8 个独立的模块构成，这些独立模块又构成了三个主要的子系统：编译器、核心模块及后端，模块将复杂的查询过程分解为细小的工作进行处理。模块结构图如图 7-8 所示。

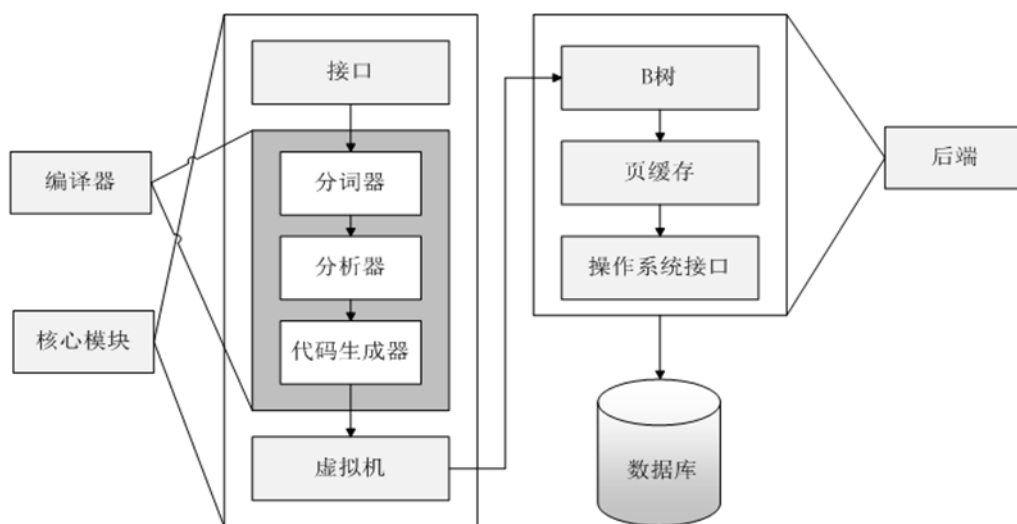


图 7-8 SQLite 数据库模块化设计

接口由 SQLite C API 组成，该 API 简单易用，无论是应用程序、脚本，还是库文件，最终都是通过接口与 SQLite 交互。

由图可以看出，编译器由分词器、分析器和代码生成器组成。分词器和分析器对 SQL 语句进行语法检查分析后，将 SQL 语句转化为对于底层来说能够更方便处理的分层的数据结构，这种分层的数据结构称为“语法树”，生成的语法树传递给代码生成器，由代码生成器进行处理，生成一种针对 SQLite 的汇编代码，该部分代码最后交与虚拟机执行。

SQLite 数据库体系结构中最核心的部分便是虚拟机，也称为虚拟数据库引擎（Virtual Database Engine, VDBE）。与 Java 虚拟机相似，虚拟机用来解释执行字节代码。虚拟机的字节代码由 128 个操作码构成，这些操作码主要用于操作数据库，每一条指令都可以完成特定的数据库操作，或以特定的方式处理栈的内容。

后端由 B-树、页缓存和操作系统接口构成。B-树的主要功能就是索引，它维护着各个页面之间的复杂关系，便于快速找到所需数据；页缓存主要就是通过操作系统接口在 B-树和磁盘之间传递页面；B-树和页缓存共同对数据进行管理。



17.3.2 手动建立数据库

手动建立数据库就是利用 `sqlite3` 工具，通过手工输入 SQL 命令来完成建立数据库的过程。这里来介绍一下 `sqlite3`。`sqlite3` 是 SQLite 数据库自带的一个 SQL 命令执行工具，它基于命令行，并可以显示命令执行结果。`sqlite3` 工具是被集成在 Android 系统中的，用户在 Linux 的命令行界面中输入 `sqlite3` 即可启动 `sqlite3` 工具，并得到工具的版本信息，如代码清单 7-21 所示。

代码清单 7-21 启动 `sqlite3` 工具

```
# sqlite3
SQLite version
Enter ".help " for instructions
sqlite>
```

由上段代码可以看出，在启动 `sqlite3` 工具后，提示符从“#”变为“`sqlite>`”，表示命令行界面已经进入与 SQLite 数据库的交互模式，此时可以输入命令对数据库的内容进行建立、删除或修改。而正确退出 `sqlite3` 工具的方法是使用 `.exit` 命令。

原则上，每个应用程序的数据库都保存在各自的 `/data/data/<package name>/databases` 目录下，但如果使用手工方式建立数据库，则必须手工建立数据库目录，目前版本无须修改数据库目录的权限。

在 SQLite 数据库中，每个数据库保存在一个独立的文件中，使用 `sqlite3` 工具后加文件名的方式打开数据库文件，如果指定文件不存在，`sqlite3` 工具则自动创建新文件。

代码清单 7-23 所示的代码将创建名为 `people` 的数据库，在文件系统中将产生一个名为 `people.db` 的数据库文件。

代码清单 7-23 创建名为 `people` 的数据库

```
# sqlite3 people.db
SQLite version
Enter ".help " for instructions
sqlite>
```

代码清单 7-24 所示的代码在新创建的数据库中，构造了一个名为 `peopleinfo` 的表，使用 `create table` 命令，关系模式为 `peopleinfo (_id, name, age, height)`。表包含 4 个属性，`_id` 是整型的主键；`name` 表示姓名，数据类型为字符型，`not null` 表示这个属性一定要填写，不可以为空值；`age` 表示年龄，数据类型为整数型；`height` 表示身高，数据类型为浮点型。

代码清单 7-24 创建名为 `peopleinfo` 的表

```
sqlite> create table peopleinfo
...> (_id integer primary key autoincrement,
...> name text not null,
...> age integer,
...> height float);
sqlite>
```



为了确认数据表是否创建成功，可以使用.tables 命令，显示当前数据库中的所有表。从代码清单 7-25 所示的代码中可以观察到，当前数据库仅有一个名为 peopleinfo 的表。

代码清单 7-25 查看数据库中表情况

```
sqlite> .tables
peopleinfo
sqlite>
```

当然，也可以使用.schema 命令查看建立表时使用的 SQL 命令。如果当前数据库中包含多个表，则可以使用[.schema 表名]的形式，显示指定表的建立命令，如代码清单 7-26 所示。

代码清单 7-26 查看数据库中指定表的建立命令

```
sqlite>.schema
CREATE TABLE peopleinfo (_id integer primary key autoincrement,
name text not null, age integer, height float);
sqlite>
```

向 peopleinfo 表中添加数据，使用 insert into ... values 命令，如代码清单 7-27 所示。

代码清单 7-27 向 peopleinfo 表中添加数据

```
sqlite> insert into peopleinfo values(null,'Tom',21,1.81);
sqlite> insert into peopleinfo values(null,'Jim',22,1.78);
sqlite> insert into peopleinfo values(null,'Lily',19,1.68);
sqlite> insert into peopleinfo values(null,'Lucy',21,1.68);
sqlite> insert into peopleinfo values(null,'John',21,1.86);
```

代码运行成功后，数据库的 peopleinfo 表将有 3 条数据。因为_id 是自增加的主键，因此在输入 null 后，SQLite 数据库会自动填写该项的内容。最终得到的表数据如表 7-4 所示。

表 7-4 peopleinfo 表数据

_id	name	age	height
1	Tom	21	1.81
2	Jim	22	1.78
3	Lily	19	1.68
4	Lucy	21	1.68
5	John	21	1.86

在数据添加完毕后，使用 select 命令，显示指定数据表中的所有数据信息，命令格式为[select 属性 from 表名]。代码清单 7-28 所示的代码用来显示 peopleinfo 表的所有数据。

代码清单 7-28 显示 peopleinfo 表的所有数据

```
select * from peopleinfo;
```



```
1|Tom|21|1.81
2|Jim|22|1.78
3|Lily|19|1.68
4|Lucy|21|1.68
5|John|21|1.86
sqlite>
```

上面的查询结果看起来不是非常直观，可以使用 `mode` 命令将结果输出格式更改为“表格”方式。

代码清单 7-29 以表格格式显示 `peopleinfo` 表的所有数据

```
sqlite> .mode column
sqlite> select * from peopleinfo;
1          Tom          21          1.81
2          Jim          22          1.78
3          Lily          19          1.68
4          Lucy          21          1.68
5          John          21          1.86
sqlite>
```

更新数据可以使用 `update` 命令，命令格式为[`update 表名 set 属性=“新值” where 条件`]。更新数据后，同样使用 `select` 命令显示数据，则可以确定数据是否正确更新。

代码清单 7-30 所示的代码将姓名为 Lily 数据中的高度值更新为 1.88。

代码清单 7-30 以表格格式显示 `peopleinfo` 表更新后的数据

```
sqlite> update peopleinfo set height=1.88 where name='Lily';
sqlite> select * from peopleinfo;
select * from peopleinfo;
1          Tom          21          1.81
2          Jim          22          1.78
3          Lily          19          1.88
4          Lucy          21          1.68
5          John          21          1.86
sqlite>
```

删除数据可以使用 `delete` 命令，命令格式为[`delete from 表名 where 条件`]。代码清单 7-31 所示的代码将 `_id` 为 3 的数据从表 `peopleinfo` 中删除。

代码清单 7-31 删除数据

```
sqlite> delete from peopleinfo where _id=3;
sqlite> select * from peopleinfo;
select * from peopleinfo;
```



1	Tom	21	1.81
2	Jim	22	1.78
4	Lucy	21	1.68
5	John	21	1.86

sqlite>

sqlite3 除了支持上述提到的增删改查的数据库操作外，对于复杂的查询，例如，order by、group by，sum、count 类的聚合函数等也支持。如代码清单 7-32、7-33 所示。

代码清单 7-32 order by

```
sqlite> select * from peopleinfo order by height;
```

```
select * from peopleinfo order by height;
```

4	Lucy	21	1.68
2	Jim	22	1.78
1	Tom	21	1.81
5	John	21	1.86

sqlite>

代码清单 7-33 group by、count

```
sqlite> select age,count(_id) as count from peopleinfo group by age having count(_id)>1;
```

```
select * from peopleinfo order by height;
```

21	3
----	---

sqlite>

sqlite3 工具还支持大量的命令，可以使用.help 命令查询 sqlite3 的命令列表。以下简单介绍几种命令，如表 7-5 所示。

表 7-5 Sqlite3 支持命令



编 号	命令	说明
1	.bail ON OFF	遇到错误时停止，默认为 OFF
2	.databases	显示数据库名称和文件位置
3	.dump ?TABLE? ...	将数据库以 SQL 文本形式导出
4	.echo ON OFF	开启和关闭回显
5	.exit	退出
6	.explain ON OFF	开启或关闭适当输出模式，如果开启模式更改为 column，并自动设置宽度
7	.header(s) ON OFF	开启或关闭标题显示
8	.help	显示帮助信息
9	.import FILE TABLE	将数据从文件导入表中
10	.indices TABLE	显示表中所有列名
11	.load FILE ?ENTRY?	导入扩展库
12	.mode MODE ?TABLE?	设置输入格式
13	.nullvalue STRING	打印时使用 STRING 代替 NULL
14	.output FILENAME	将输入保存到文件
15	.output stdout	将输入显示在屏幕上
16	.prompt MAIN CONTINUE	替换标准提示符
17	.quit	退出
18	.read FILENAME	在文件中执行 SQL 语句
19	.schema ?TABLE?	显示表的创建语句
20	.separator STRING	更改输入和导入的分隔符
21	.show	显示当前设置变量值

续表



编号	命令	说明
22	.tables ?PATTERN?	显示符合匹配模式的表名
23	.timeout MS	尝试打开被锁定的表 ms（毫秒）
24	.timer ON OFF	开启或关闭 CPU 计时器
25	.width NUM NUM ...	设置 column 模式的宽度

17.3.3 代码建库（SQLiteOpenHelper）

最常用的建立数据库的方法，即在代码中进行对数据库的建立。在程序运行过程中，当程序需要进行数据库操作时，应用程序会首先尝试打开数据库，此时如果数据库并不存在，程序则会自动建立数据库，然后再打开数据库。在编程实现时，一般将所有对数据库的操作都封装在类中，因此只要调用这个类，就可以完成对数据库的添加、更新、删除和查询等操作。下面我们就来介绍一些常用的接口和类。

本节中主要介绍一下如何通过 SQLiteOpenHelper 类来实现建立数据库。

SQLiteOpenHelper 是一个抽象的辅助类（Helper），用来打开（若数据库已存在）或创建数据库。当在程序当中调用 SQLiteOpenHelper 的 getWritableDatabase() 方法，或者 getReadableDatabase() 方法的时候，这两个方法根据数据库是否存在、版本号和是否可写等情况，决定在返回数据库对象前是否需要建立数据库，如果当时没有数据，那么 Android 系统就会自动生成一个数据库。关于抽象类我们都知道，如果要使用它，一定是继承它。所以使用它都是通过自己定义一个类继承于它并实现其抽象方法：

onCreate(SQLiteDatabase)，onUpgrade(SQLiteDatabase, int, int)，onOpen(SQLiteDatabase) 3 个方法，如表 7-6 所示。

表 7-6 SQLiteOpenHelper 类中重要方法

方法	描述
onCreate(SQLiteDatabase)	在数据库第一次生成的时候会调用这个方法，一般在这个方法里边生成数据库表
onUpgrade(SQLiteDatabase, int, int)	当数据库需要升级的时候，Android 系统会主动地调用这个方法。一般在这个方法里边删除数据表，并建立新的数据表，当然是否还需要做其他的操作，完全取决于应用的需求
onOpen(SQLiteDatabase)	这是当打开数据库时的回调函数，一般也不会用到

下面通过对 DBAdapter 封装类的代码来学习如何通过继承 SQLiteOpenHelper 类来实现数据库的建立、打开和关闭等操作。

代码清单 7-34 DBAdapter



```
public class DBAdapter {  
    //声明了数据库的基本信息  
    //数据库文件的名称  
    private static final String DB_NAME = "people.db";  
    //数据库表格名称和数据库版本  
    private static final String DB_TABLE = "peopleinfo";  
    private static final int DB_VERSION = 1;  
  
    //数据库表中的属性名称  
    public static final String KEY_ID = "_id";  
    public static final String KEY_NAME = "name";  
    public static final String KEY_AGE = "age";  
    public static final String KEY_HEIGHT = "height";  
  
    //声明 SQLiteDatabase 对象 db  
    private SQLiteDatabase db;  
    private final Context context;  
    //DBOpenHelper 继承了 SQLiteOpenHelper, 声明 DBOpenHelper 类对象,  
    //辅助建立、更新和打开数据库  
    private DBOpenHelper dbOpenHelper;  
  
    private static class DBOpenHelper extends SQLiteOpenHelper {}  
  
    public DBAdapter(Context _context) {  
        context = _context;  
    }  
  
    //调用了 SQLiteOpenHelper 类的 getWritableDatabase()方法和 getReadableDatabase()方法, 打开数  
    据库  
    public void open() throws SQLiteException {  
        dbOpenHelper = new DBOpenHelper(context, DB_NAME, null, DB_VERSION);  
        try {  
            //负责得到一个可写的 SQLite 数据库, 如果这个数据库还没有  
            //建立, 那么 mOpenHelper 辅助类负责建立这个数据库。如果数  
            //据库已经建立, 那么直接返回一个可写的数据库  
        }  
    }  
}
```



```

        db = dbOpenHelper.getWritableDatabase();
    } catch (SQLiteException ex) {
        db = dbOpenHelper.getReadableDatabase();
    }
}

public void close() {
    if (db != null){
        //关闭数据库
        db.close();
        db = null;
    }
}
}

```

如代码清单 7-34 所示，我们注意到，在声明了数据库基本信息后，紧接着声明了 SQLiteDatabase 对象 db。这里的 SQLiteDatabase 类是由 Android 提供的封装了一些数据库操作的 API 的类，使用该类可以完成对数据进行添加（Create）、查询（Retrieve）、更新（Update）和删除（Delete）操作（这些操作简称为 CRUD），执行 SQL 命令，对数据进行管理等工作。在下一节的数据操作中，将对使用该类实现数据操作进行具体讲解。

之前有讲解到，继承 SQLiteOpenHelper 类覆盖其抽象方法，这里来覆盖 onCreate(SQLiteDatabase db) 和 onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion)。为了实现对数据库进行管理，重载 onCreate() 方法和 onUpgrade() 方法的代码如代码清单 7-35 所示。

代码清单 7-35 DBOpenHelper

```

private static class DBOpenHelper extends SQLiteOpenHelper {
    public DBOpenHelper(Context context, String name,
                        CursorFactory factory, int version){
        super(context, name, factory, version);
    }

    private static final String DB_CREATE = "create table " + DB_TABLE +
        "(" + KEY_ID + " integer primary key autoincrement, " +
        KEY_NAME + " text not null, " + KEY_AGE + " integer, " +
        KEY_HEIGHT + " float)";

    @Override
    public void onCreate(SQLiteDatabase _db) {
        db.execSQL(DB_CREATE);
    }
}

```



```

}

@Override

public void onUpgrade(SQLiteDatabase _db, int _oldVersion, int _newVersion) {

    _db.execSQL("DROP TABLE IF EXISTS " + DB_TABLE);

    onCreate(_db);

}

}
    
```

在上述代码中，在声明了创建表的 SQL 命令语 DB_CREATE 之后，分别重载了 onCreate()方法和 onUpgrade()方法，这是继承 SQLiteOpenHelper 类必须重载的两个方法。

onCreate()方法在数据库第一次建立时被调用，一般用来创建数据库中的表，并做适当的初始化工作，在 onCreate()方法中，通过调用 SQLiteDatabase 对象的 execSQL()方法，执行创建表的 SQL 命令。

onUpgrade()方法在数据库需要升级时被调用，一般用来删除旧的数据库表，并将数据转移到新版本的数据库表中。为了简单起见，在本段代码的 onUpgrade()方法中，并没有做任何的数据转移，而仅仅删除原有的表后建立新的数据库表。

程序开发人员不应直接调用 onCreate()和 onUpgrade()方法，而应该由 SQLiteOpenHelper 类来决定何时调用这两个方法。SQLiteOpenHelper 类的 getWritableDatabase()方法和 getReadableDatabase()方法是可以直接调用的方法。getWritableDatabase()方法用来建立或打开可读/写的数据库对象，一旦方法调用成功，数据库对象将被缓存，何时需要使用数据库对象，都可以调用这个方法获取到数据库对象，但一定要在不使用时调用 close()方法关闭数据库。如果保存数据库文件的磁盘空间已满，调用 getWritableDatabase()方法则无法获得可读/写的数据库对象，这时可以调用 getReadableDatabase()方法，获得一个只读的数据库对象。

如果程序开发人员不希望使用 SQLiteOpenHelper 类，同样可以通过 SQLiteDatabase 对象直接创建数据库。通过 SQLiteDatabase (android.database.sqlite.SQLiteDatabase) 这个类，除了可以实现数据库的创建或打开以外，还可以进行创建表、插入数据、删除数据、查询数据、修改数据等操作，如表 7-7 所示。

表 7-7 SQLiteDatabase 类中重要方法

方法	描述
public static SQLiteDatabase openOrCreateDatabase (String path, SQLiteDatabase.CursorFactory factory)	用于打开或创建一个数据库（对于 sqlite 来说，就是打开或产生一个数据库文件），参数中的 File 类就是 java 中表示系统文件路径的 File 类，而 SQLiteDatabase.CursorFactory 则是一个产生 Cursor 对象的工厂类
public long insert (String table, String nullColumnHack, ContentValues values)	用于在数据库中加入数据。ContentValue 类似于 java 中 HashMap 类，用于以键值对的方式保存数据
public int delete (String table, String	删除表中的数据



whereClause, String[] whereArgs)	
public Cursor query (boolean distinct, String table, String[] columns, String selection, String[] selectionArgs, String groupBy, String having, String orderBy, String limit)	查询数据库中的数据
public int update (String table, ContentValues values, String whereClause, String[] whereArgs)	用于修改数据
public void execSQL (String sql)	执行 String 表示的非查询的 SQL 语句, 例如 Create 等
public void close ()	用来关闭数据库并释放数据库占用的相关资源

通过 SQLiteDatabase 对象直接创建数据库方法如下：首先调用 openOrCreateDatabases() 方法创建数据库对象，然后执行 SQL 命令建立数据库中的表和直接的关系。示例代码如代码清单 7-36 所示。

代码清单 7-36 openOrCreateDatabases() 方法创建数据库对象

```
private static final String DB_CREATE = "create table " +
    DB_TABLE + " (" + KEY_ID + " integer primary key autoincrement, " +
    KEY_NAME + " text not null, " + KEY_AGE + " integer, " + KEY_HEIGHT + " float); ";
public void create() {
    db.openOrCreateDatabases(DB_NAME, context.MODE_PRIVATE, null)
    db.execSQL(DB_CREATE);
}
```

17.3.4 数据操作（SQLiteDatabase）

数据操作是指对数据的添加、删除、查找和更新（CRUD）的操作。通过执行 SQL 命名完成数据操作，但推荐使用 Android 提供的专用类和方法，这些类和方法更加简洁、易用。

在上述实现了创建数据库的 DBAdapter 类的基础上，为了使 DBAdapter 类支持对数据的添加、删除、更新和查找等功能，在 DBAdapter 类中增加下面的这些方法：

insert(People people)用来添加一条数据。

queryAllData()用来获取全部数据。

queryOneData(long id)根据 id 获取一条数据。

deleteAllData()用来删除全部数据。

deleteOneData(long id)根据 id 删除一条数据。

updateOneData(long id, People people)根据 id 更新一条数据。



代码清单 7-37 DBAdapter

```
public class DBAdapter {  
    public long insert(People people) {}  
    public long deleteAllData() { }  
    public long deleteOneData(long id) { }  
    public People[] queryAllData() {}  
    public People[] queryOneData(long id) { }  
    public long updateOneData(long id , People people){ }  
  
    private People[] ConvertToPeople(Cursor cursor){}  
}
```

其中，ConvertToPeople（Cursor cursor）是私有方法，作用是将查询结果转换为用来存储数据的自定义 People 类对象。

首先就来介绍一下自定义的 People 类。People 类包含 4 个公共属性，分别为 ID、Name、Age 和 Height，对应数据库中的 4 个属性值。覆盖 Object 中的 toString()方法，主要是为了便于界面显示的需要。People 类的代码如代码清单 7-38 所示。

代码清单 7-38 People 类

```
public class People {  
    public int ID = -1;  
    public String Name;  
    public int Age;  
    public float Height;  
  
    @Override  
    public String toString(){  
        String result = "";  
        result += "ID: " + this.ID + ", ";  
        result += "姓名: " + this.Name + ", ";  
        result += "年龄: " + this.Age + ", ";  
        result += "身高: " + this.Height + ", ";  
        return result;  
    }  
}
```

下面分别介绍如何使用 SQLiteDatabase 类的公共方法，完成数据的添加、删除、更新和查询等数据操作。



SQLiteDatabase 类的公共方法 insert()、delete()、update()和 query()，封装了执行的添加、删除、更新和查询功能的 SQL 命令。

添加功能

首先构造一个 ContentValues 对象；其次调用 ContentValues 对象的 put()方法，将每个属性的值写入 ContentValues 对象中；最后使用 SQLiteDatabase 对象的 insert()方法，将 ContentValues 对象中的数据写入指定的数据库表中。

insert()方法的返回值是新数据插入的位置，即 ID 值。ContentValues 类是一个数据承载容器，主要用来向数据库表中添加一条数据。

代码清单 7-39 insert

```
public long insert(People people) {  
    ContentValues newValues = new ContentValues();  
  
    newValues.put(KEY_NAME, people.Name);  
    newValues.put(KEY_AGE, people.Age);  
    newValues.put(KEY_HEIGHT, people.Height);  
  
    return db.insert(DB_TABLE, null, newValues);  
}
```

在上述代码中，第 4 行代码向 ContentValues 对象 newValues 中添加一个名称/值对，put()方法的第 1 个参数是名称，第 2 个参数是值；在第 8 行代码的 insert()方法中，第 1 个参数是数据表的名称，第 2 个参数是在 NULL 时的替换数据，第 3 个参数是需要向数据库表中添加的数据。

删除功能

删除数据比较简单，只需要调用当前数据库对象的 delete()方法，并指明表名称和删除条件即可。

代码清单 7-40 删除

```
public long deleteAllData() {  
    return db.delete(DB_TABLE, null, null);  
}  
  
public long deleteOneData(long id) {  
    return db.delete(DB_TABLE, KEY_ID + "=" + id, null);  
}
```

其中，delete()方法的第 1 个参数是数据库的表名称，第 2 个参数是删除条件。在第 2 行代码中，删除条件为 null，表示删除表中的所有数据；第 6 行代码指明了需要删除数据的 id 值，因此 deleteOneData()方法仅删除一条数据，此时 delete()方法的返回值表示被删除的数据的数量。

更新功能

更新数据同样要使用 ContentValues 对象，首先构造 ContentValues 对象，其次调用 put()方法将属性的



值写入 ContentValues 对象中，最后使用 SQLiteDatabase 对象的 update() 方法，并指定数据的更新条件。

代码清单 7-41 更新

```
public long updateOneData(long id , People people){
    ContentValues updateValues = new ContentValues();
    updateValues.put(KEY_NAME, people.Name);
    updateValues.put(KEY_AGE, people.Age);
    updateValues.put(KEY_HEIGHT, people.Height);

    return db.update(DB_TABLE, updateValues, KEY_ID + "=" + id, null);
}
```

在代码的第 7 行中，update() 方法的第 1 个参数表示数据表的名称，第 2 个参数是更新条件，update() 方法的返回值表示数据库表中被更新的数据数量。

查询功能

首先介绍 Cursor 类。在 Android 系统中，数据库查询结果的返回值并不是数据集合的完整复制，而是返回数据集的指针，这个指针就是 Cursor 类。Cursor 类支持在查询的数据集合中多种移动方式，并能够获取数据集的属性名称和序号。

Cursor 类的方法和说明如表 7-8 所示。

表 7-8 Cursor 类的方法和说明

方法	说明
moveToFirst	将指针移动到第一条数据上，返回 boolean 类型值，为 true 表示指针移动成功
moveToNext	将指针移动到下一条数据上，返回 boolean 类型值，为 true 表示指针移动成功
moveToPrevious	将指针移动到上一条数据上，返回 boolean 类型值，为 true 表示指针移动成功
getCount	获取集合的数据数量
getColumnIndexOrThrow	返回指定属性名称的序号，如果属性不存在则产生异常
getColumnName	返回指定序号的属性名称
getColumnNames	返回属性名称的字符串数组
getColumnIndex	根据属性名称返回序号
moveToPosition	将指针移动到指定的数据上，返回 boolean 类型值，为 true 表示指针移动成功
getPosition	返回当前指针的位置

从 Cursor 中提取数据可以参考 ConvertToPeople() 方法的实现方法，具体代码如代码清单 7-42 所示。



代码清单 7-42 从 Cursor 中提取数据

```
private People[] ConvertToPeople(Cursor cursor){
    int resultCounts = cursor.getCount();
    if (resultCounts == 0 || !cursor.moveToFirst()){
        return null;
    }
    People[] peoples = new People[resultCounts];
    for (int i = 0 ; i<resultCounts; i++){
        peoples[i] = new People();
        peoples[i].ID = cursor.getInt(0);
        peoples[i].Name=cursor.getString(cursor.getColumnIndex(KEY_NAME));
        peoples[i].Age = cursor.getInt(cursor.getColumnIndex(KEY_AGE));
        peoples[i].Height=cursor.getFloat(cursor.getColumnIndex(KEY_HEIGHT));
        cursor.moveToNext();
    }
    return peoples;
}
```

在提取 Cursor 数据的数据前，推荐测试 Cursor 中的数据数量，避免在数据获取中产生异常，例如，代码的第 3 行到第 5 行。从 Cursor 中提取数据使用类型安全的 get<Type>()方法，方法的输入值为属性的序号，为了获取属性的序号，可以使用 getColumnIndex()方法获取指定属性的序号，如代码的第 10 行到第 12 行。

要进行数据查询就需要调用 SQLiteDatabase 类的 query()方法，query()方法的语法如代码清单 7-43 所示。

```
Cursor android.database.sqlite.SQLiteDatabase.query(String table, String[] columns, String selection,
String[] selectionArgs, String groupBy, String having, String orderBy)
```

query()方法的参数说明如表 7-9 所示。

表 7-9 query()方法的参数

位置	类型+名称	说明
1	String table	表名称
2	String[] columns	返回的属性列名称
3	String selection	查询条件
4	String[] selectionArgs	如果在查询条件中使用的问号，则需要定义替换符的具体内容
5	String groupBy	分组方式
6	String having	定义组的过滤器



7	String orderBy	排序方式
---	----------------	------

根据 id 查询数据的代码如代码清单 7-44 所示。

代码清单 7-44 根据 id 查询数据的代码

```
public People[] getOneData(long id) {
    Cursor results = db.query(DB_TABLE, new String[] { KEY_ID, KEY_NAME, KEY_AGE,
KEY_HEIGHT}, KEY_ID + "=" + id, null, null, null, null);
    return ConvertToPeople(results);
}
```

查询全部数据的代码如代码清单 7-45 所示。

代码清单 7-45 查询全部数据的代码

```
public People[] getAllData() {
    Cursor results = db.query(DB_TABLE, new String[] { KEY_ID, KEY_NAME, KEY_AGE,
KEY_HEIGHT}, null, null, null, null, null);
    return ConvertToPeople(results);
}
```

17.4 数据共享(ContentProviders)

ContentProvider 是在应用程序间共享数据的一种接口机制，它使其他的应用程序保存或读取此 ContentProvider 的各种数据类型。

ContentProvider 提供了更为高级的数据共享方法，应用程序可以指定需要共享的数据，而其他应用程序则可以在不知数据来源、路径的情况下，提供了查询、添加、删除和更新共享数据等操作的接口。许多 Android 系统的内置数据也通过 ContentProvider 提供给用户使用，如通讯录、音视频文件和图像文件等。

ContentProvider 的数据模式类似于数据库的数据表，每行是一条记录，每列具有相同的数据类型。每条记录都包含一个 long 型的字段 _ID，用来唯一标识每条记录。ContentProvider 可以提供多个数据集，调用者使用 URI 对不同的数据集的数据进行操作。

ContentProvider 数据模型如表 7-10 所示。

表 7-10 ContentProvider 数据模型

_ID	NAME	AGE	HEIGHT
1	Tom	21	1.81
2	Jim	22	1.78

ContentProvider 完全屏蔽了数据提供组件的数据存储方法。在使用者看来，数据提供者通过 ContentProvider 提供了一组标准的数据操作接口，却无法得知数据提供者的数据存储方法。数据提供者可以使用 SQLite 数据库存储数据，也可以通过文件系统或 SharedPreferences 存储数据，甚至是使用网络存



储的方法，这些内容对数据使用者都是不可见的。同时也正是因为屏蔽数据的存储方法，很大程度上简化了 ContentProvider 的使用难度，使用者只要调用 ContentProvider 提供的接口方法，就可完成所有的数据操作。

图 7-9 显示了 ContentProvider 的调用关系。

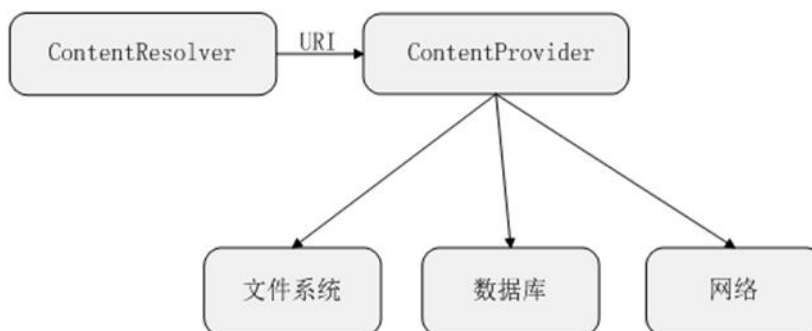


图 7-9 ContentProvider 的调用关系图

首先，在创建 ContentProvider 时，需要先实现底层的存储功能，可以是数据库、文件系统或是网络存储等，然后在继承 ContentProvider 的类中实现基本数据操作的接口方法，包括添加、删除、查找和更新等功能。接口主要有以下几个。

query(Uri uri, String[] projection, String selection, String[] selectionArgs, String sortOrder): 通过 Uri 进行查询，返回一个 Cursor。

insert(Uri url, ContentValues values): 将一组数据插入 Uri 指定的地方。

update(Uri uri, ContentValues values, String where, String[] selectionArgs): 更新 Uri 指定位置的数据。

delete(Uri url, String where, String[] selectionArgs): 删除指定 Uri 并且符合一定条件的数据。

其次，我们看出调用者不能够直接调用 ContentProvider 的接口方法，而需要使用 ContentResolver 对象，通过 Uri 间接调用 ContentProvider。那么什么是 ContentResolver 对象呢？

ContentResolver 接口是外界的程序访问 ContentProvider 提供的桥梁，ContentResolver 提供的接口和 ContentProvider 中需要实现的接口对应，主要有以下几个。

query(Uri uri, String[] projection, String selection, String[] selectionArgs, String sortOrder): 通过 Uri 进行查询，返回一个 Cursor。

insert(Uri url, ContentValues values): 将一组数据插入 Uri 指定的地方。

update(Uri uri, ContentValues values, String where, String[] selectionArgs): 更新 Uri 指定位置的数据。

delete(Uri url, String where, String[] selectionArgs): 删除指定 Uri 并且符合一定条件的数据。

程序开发人员在 Activity 当中通过 `getContentResolver()` 可以得到当前应用的 ContentResolver 对象，然后使用该 ContentResolver 对象与 ContentProvider 进行交互，而 ContentResolver 则通过 Uri 确定需要访问的 ContentProvider 的数据集。在发起一个请求的过程中，Android 首先根据 Uri 确定处理这个查询的 ContentResolver，然后初始化 ContentResolver 所有需要的资源，这个初始化的工作无须程序开发人员参与，是 Android 系统完成的。一般情况下只有一个 ContentProvider 对象，但却可以同时与多个



ContentResolver 在不同的应用程序和不同的进程中进行交互。

Uri 是通用资源标志符 (Uniform Resource Identifier)，用来定位任何远程或本地的可用资源。在 ContentProvider 和 ContentResolver 当中用到的 Uri 语法结构如代码清单 7-46 所示。

代码清单 7-46 ContentProvider 使用的 Uri 语法结构

```
content://<authority>/<data_path>/<id>
```

content://是通用前缀，表示该 Uri 用于 ContentProvider 定位资源，无须修改；<authority>是授权者名称，用来确定具体由哪一个 ContentProvider 提供资源。因此，一般<authority>都由类的包名+类名小写全称组成，以保证唯一性。

<data_path>是数据路径，用来确定请求的是哪个数据集。如果 ContentProvider 仅提供一个数据集，数据路径则是可以省略的；如果 ContentProvider 提供多个数据集，数据路径则必须指明具体是哪一个数据集。数据集的数据路径可以写成多段格式，例如/people/girl 和/people/boy。

Uri 可以指定全部数据，也可以是指定某个 ID 的数据。这里我们就用到了上述语法结构中的最后一个参数：<id>。<id>是数据编号，用来唯一确定数据集中的一条记录，用来匹配数据集中_ID字段的值，如果没有指定<id>，那么表示返回全部。请求整个 people 数据集的 Uri 应写为如代码清单 7-47 所示的代码。

代码清单 7-47 请求数据的 Uri 例子

```
//请求全部的联系人数据的 Uri
```

```
content://contacts/people/
```

```
//请求 ID 为 1 的联系人数据的 Uri
```

```
content://contacts/people/1
```

```
//请求整个 people 数据集的 Uri
```

```
content://cn.com.farsight.peopleprovider/people
```

```
//请求 people 数据集中第 3 条数据的 Uri
```

```
content://cn.com.farsight.peopleprovider/people/3
```

17.4.1 创建 ContentProvider

通过之前的讲解，对 ContentProvider 有了比较全面的理解。那么如何创建 ContentProvider 呢？一般情况下，程序开发人员在创建自己的 ContentProvider 的过程可以分为以下三步。

首先，继承 ContentProvider 类，并覆盖 ContentProvider 的抽 6 个抽象方法。新建的类继承 ContentProvider 后，需覆盖的 6 个方法如下。

public boolean onCreate()，当 ContentProvider 生成时调用此方法，主要实现初始化底层数据集和建立数据链接等工作。

public Cursor query(Uri uri, String[] projection, String selection, String[] selectionArgs, String sortOrder) ，此方法返回一个 Cursor 对象作为查询结果集。

public Uri insert(Uri uri, ContentValues initialValues)，此方法负责往数据集当中插入一行，并返回这一



列的 Uri。

`public int delete(Uri uri, String where, String[] whereArgs)`，此方法负责删除指定 Uri 的数据。

`public int update(Uri uri, ContentValues values, String where, String[] whereArgs)`，此方法负责更新指定 Uri 的数据。

`public String getType(Uri uri)`，返回所给 Uri 的 MIME 类型，如果 Uri 是单条数据，则返回的 MIME 数据类型应以 `vnd.android.cursor.item` 开头；如果 Uri 是多条数据，则返回的 MIME 数据类型应以 `vnd.android.cursor.dir` 开头。

新建的类继承 `ContentProvider` 后，Eclipse 会提示程序开发人员需要覆盖部分代码，并自动生成需要覆盖的代码框架。代码清单 7-48 所示的代码是 Eclipse 自动生成的代码框架。

代码清单 7-48 继承 `ContentProvider` 后的代码框架

```
import android.content.*;
import android.database.Cursor;
import android.net.Uri;

public class PeopleProvider extends ContentProvider{

    @Override
    public int delete(Uri uri, String selection, String[] selectionArgs) {
        // TODO Auto-generated method stub
        return 0;
    }

    @Override
    public String getType(Uri uri) {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public Uri insert(Uri uri, ContentValues values) {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
```



```

public boolean onCreate() {
    // TODO Auto-generated method stub
    return false;
}

@Override
public Cursor query(Uri uri, String[] projection, String selection,
    String[] selectionArgs, String sortOrder) {
    // TODO Auto-generated method stub
    return null;
}

@Override
public int update(Uri uri, ContentValues values, String selection,
    String[] selectionArgs) {
    // TODO Auto-generated method stub
    return 0;
}
}
    
```

声明 CONTENT_URI，实现 UriMatcher。

在上一步中，知道在返回所给 Uri 的 MIME 类型即实现 getType(Uri uri)时，需通过 Uri 是单条数据还是多条数据来确认其返回的 MIME 数据类型。在新构造的 ContentProvider 类中，通过构造一个 UriMatcher，判断 Uri 是单条数据还是多条数据。为了便于判断和使用 Uri，一般将 Uri 的授权者名称和数据路径等内容声明为静态常量，并声明 CONTENT_Uri。

声明 CONTENT_Uri 和构造 UriMatcher 的代码如代码清单 7-49 所示。

代码清单 7-49 声明 CONTENT_URI 和构造 UriMatcher

```

//声明了 Uri 的授权者名称
public static final String AUTHORITY = "cn.com.farsight.peopleprovider";
//声明了单条数据的数据路径
public static final String PATH_SINGLE = "people/#";
//声明了多条数据的数据路径
public static final String PATH_MULTIPLE = "people";
//声明了 CONTENT_URI 的字符串形式
public static final String CONTENT_URI_STRING = "content://" +
    AUTHORITY + "/" + PATH_MULTIPLE;
    
```



```
//声明了 CONTENT_URI
public static final Uri CONTENT_URI = Uri.parse
                                (CONTENT_URI_STRING);

//声明了多条数据的返回代码
private static final int MULTIPLE_PEOPLE = 1;
//声明了单条数据的返回代码
private static final int SINGLE_PEOPLE = 2;

//声明了 UriMatcher
private static final UriMatcher uriMatcher;
//静态构造方法，声明了 UriMatcher 的匹配方式和返回代码
static {
    // UriMatcher.NO_MATCH 表示 Uri 无匹配时的返回代码
    uriMatcher = new UriMatcher(UriMatcher.NO_MATCH);
    //addURI()方法用来添加新的匹配项
    uriMatcher.addURI(AUTHORITY, PATH_SINGLE, MULTIPLE_PEOPLE);
    uriMatcher.addURI(AUTHORITY, PATH_MULTIPLE, SINGLE_PEOPLE);
}
```

其中，addURI()方法用来添加新的匹配项，语法如代码清单 7-50 所示。

代码清单 7-50 addURI()方法语法

```
public void addURI (String authority, String path, int code)
```

其中，authority 表示匹配的授权者名称；path 表示数据路径，#可以代表任何数字；code 表示返回代码。

使用 UriMatcher 时，则可以直接调用 match()方法，对指定的 Uri 进行判断，示例代码如代码清单 7-51 所示。

代码清单 7-51 调用 match()方法，对指定的 URI 进行判断

```
switch(uriMatcher.match(uri)){
    case MULTIPLE_PEOPLE:
        //多条数据的处理过程
        break;
    case SINGLE_PEOPLE:
        //单条数据的处理过程
        break;
    default:
        throw new IllegalArgumentException("不支持的 URI:" + uri);
}
```



```
}
```

注册 ContentProvider。

在完成 ContentProvider 类的代码实现后，需要在 AndroidManifest.xml 文件中进行注册。注册 ContentProvider 使用<provider>标签，例如，注册了一个授权者名称为 cn.com.farsight.peopleprovider 的 ContentProvider，其实现类是 PeopleProvider 的示例代码如代码清单 7-52 所示。

代码清单 7-52 注册 ContentProvider

```
<application android:icon="@drawable/icon"
              android:label="@string/app_name">
    <provider android:name=".PeopleProvider"
              android:authorities="cn.com.farsight.peopleprovider"/>
</application>
```

17.4.2 使用数据提供者

使用 ContentProvider 是通过 Android 组件都具有的 ContentResolver 对象，通过 Uri 进行数据操作。程序开发人员只需要知道 Uri 和数据集的数据格式，则可以进行数据操作，解决不同应用程序间的数据共享问题。

每个 Android 组件都具有一个 ContentResolver 对象，获取 ContentResolver 对象的方法是调用 getContentResolver()方法。

代码清单 7-53 获取 ContentResolver 对象

```
ContentResolver resolver = getContentResolver();
```

查询操作

在获取到 ContentResolver 对象后，程序开发人员则可以使用 query()方法查询目标数据。代码清单 7-54 所示的代码是查询 ID 为 2 的数据。

代码清单 7-54 获取 ContentResolver 对象

```
String KEY_ID = "_id";
String KEY_NAME = "name";
String KEY_AGE = "age";
String KEY_HEIGHT = "height";

Uri uri = Uri.parse(CONTENT_URI_STRING + "/" + "2");
Cursor cursor = resolver.query(uri,
    new String[] {KEY_ID, KEY_NAME, KEY_AGE, KEY_HEIGHT},
    null, null, null);
```

在 Uri 中定义了需要查询数据的 ID，在 query()方法并没有额外声明查询条件。

如果需要获取数据集中的全部数据，则可直接使用 CONTENT_URI，此时 ContentProvider 在分析 Uri 时将认为需要返回全部数据。ContentResolver 的 query()方法与 SQLite 数据库的 query()方法非常相似，语



法结构如代码清单 7-55 所示。

代码清单 7-55 ContentResolver 的 query()方法语法

```
Cursor query(Uri uri, String[] projection, String selection,  
             String[] selectionArgs, String sortOrder)
```

其中, uri 定义了查询的数据集; projection 定义了从数据集返回哪些数据项; selection 定义了返回数据的查询条件; selectionArgs 是对应的实际查询条件; sortOrder 是排序方式。

2. 添加操作

向 ContentProvider 中添加数据有两种方法: 一种是使用 insert()方法, 向 ContentProvider 中添加一条数据; 另一种是使用 bulkInsert()方法, 批量地添加数据。

代码清单 7-56 所示的代码说明了如何使用 insert()方法添加单条数据。

代码清单 7-56 使用 insert()方法添加单条数据

```
ContentValues values = new ContentValues();  
values.put(KEY_NAME, "Tom");  
values.put(KEY_AGE, 21);  
values.put(KEY_HEIGHT, );  
Uri newUri = resolver.insert(CONTENT_URI, values);
```

代码清单 7-57 所示的代码说明了如何使用 bulkInsert()方法添加多条数据。

代码清单 7-57 使用 bulkInsert()方法添加多条数据

```
ContentValues[] arrayValues = new ContentValues[10];  
//实例化每一个 ContentValues  
int count = resolver.bulkInsert(CONTENT_URI, arrayValues);
```

3. 删除操作

删除操作需要使用 delete()方法。如果需要删除单条数据, 则可以在 Uri 中指定需要删除数据的 ID; 如果需要删除多条数据, 则可以在 selection 中声明删除条件。

代码清单 7-58 所示的代码说明了如何删除 ID 为 2 的数据。

代码清单 7-58 删除 ID 为 2 的数据

```
Uri uri = Uri.parse(CONTENT_URI_STRING + "/" + "2");  
int result = resolver.delete(uri, null, null);
```

也可以在 selection 中将删除条件定义为 ID 大于 4 的数据, 如代码清单 7-59 所示。

代码清单 7-59 删除条件定义为 ID 大于 4 的数据

```
String selection = KEY_ID + ">4";  
int result = resolver.delete(CONTENT_URI, selection, null);
```

更新操作

更新操作需要使用 update()方法, 参数定义与 delete()方法相同, 同样可以在 Uri 中指定需要更新数据的 ID, 也可以在 selection 中声明更新条件。



代码清单 7-60 所示的代码说明了如何更新 ID 为 7 的数据。

代码清单 7-60 更新 ID 为 7 的数据

```
ContentValues values = new ContentValues();  
values.put(KEY_NAME, "Tom");  
values.put(KEY_AGE, 21);  
values.put(KEY_HEIGHT, );  
Uri uri = Uri.parse(CONTENT_URI_STRING + "/" + "7");  
int result = resolver.update(uri, values, null, null);
```

17.5 本章小结

本章主要对数据存储及其访问进行学习，包括简单存储——主要是 `SharedPreferences` 的使用方法，文件存储的类别及区别和适用情况，数据库存储的特点、体系结构、建立与操作方法，以及数据共享的原理与用途、创建与使用方法。

华清远见
dev.havi.com



第 18 章 多媒体开发

在上一章的学习中，主要了解了 Android 中简单存储的使用方法，文件存储的类别以及区别和适用情况，数据库存储的特点、体系结构、建立与操作方法，以及数据共享的原理与用途、创建与使用方法。

Android 提供了常见媒体的编码、解码机制，因此可以非常容易地集成音频、视频和图片等多媒体文件到应用程序中。可以调用 Android 提供的现有 API，非常容易地实现相册、播放器、录音和摄像等应用程序。当然，有些需要硬件的支持。通过前面学习的 Activity 和 Intent，可以非常直接地来访问这些媒体文件。

播放音频及视频会用到 MediaPlayer 和 JetPlayer 类，录制音频及视频会用到 MediaRecorder 类。文件的来源可以是存储在 Android 应用程序中的资源文件（源文件中）、存储在本地文件系统的标准媒体文件，还可以是通过网络连接取得的文件流（URL）。

18.1 Android 系统提供内置的媒体格式

表 8-1 所示的是 Android 系统提供内置的音频、视频和图像媒体格式。

表 8-1 Android 系统的音频、视频和图像媒体格式

种 类	格 式	编 码	解 码	说 明	文 件 类 型
音 频	AAC LC/LTP		X	Mono/Stereo content in any combination of standard bit rates up to 160 kbps and sampling rates from 8 to 48kHz	3gpp(.3gp)、 MPEG-4(.mp4、 m4a)。不提供 rawAAC(.aac)
	HE- AACv1 (AAC+)		X		
	HE- AACv2 (enhanced AAC+)		X		
	AMR- NB	X	X	4.75 to 12.2 kbps sampled @ 8kHz	3gpp(.3gp)
	AMR- WB		X	9 rates from 6.60 kbit/s to 23.85 kbit/s sampled @ 16kHz	3gpp(.3gp)
	MP3		X	Mono/Stereo 8-320Kbps constant (CBR) or variable bit-rate (VBR)	MP3(.mp3)



	MIDI		X	MIDI Type 0 and 1. DLS Version 1 and 2. XMF and Mobile XMF. Support for ringtone formats RTTTL/RTX, OTA, and iMelody	Type 0 and 1 (.mid, .xmf, .mxmf). Also RTTTL/RTX (.rtttl, .rtx), OTA (.ota), and iMelody (.imy)
	Ogg		X		Ogg (.ogg)
	PCM/WAVE		X	8- and 16-bit linear PCM (rates up to limit of hardware)	WAV (.wav)
视 频	H.263	X	X		3gpp(.3gp)、 MPEG-4(.mp4)
	H.264(AVC)		X		3gpp(.3gp)、 MPEG-4(.mp4)
	MPEG-4 SP		X		3gpp(.3gp)
图 像	Jpeg	X	X		Jpeg (.jpg)
	PNG		X		PNG(.png)
	GIF		X		GIF(.gif)
	BMP		X		BMP(.bmp)

18.2 在窗体布局上——VideoView

VideoView 类可以从不同的来源（如文件系统和内容提供商的网站）下载视频文件，并显示在窗体布局上。因为它可以配合窗体布局设计上的需要来控制视频窗体的大小，提供各种选项，比如缩放和着色。

在 VideoView 类上，使用 setVideoPath()方法来设置视频文件路径，没使用 setMediaController()方法来控制媒体播放，使用 start()方法来开始播放视频。示例代码如代码清单 8-1 所示。

代码清单 8-1 使用 VideoView 类播放视频

```
public class HelloVideoView extends Activity {
    //HelloVideoView 主程序
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
        VideoView videoView =
```



```
(VideoView)findViewById(R.id.VideoView01);

//设置资源文件路径
videoView.setVideoPath("/sdcard/navy.3gp");

//实现 MediaController 对象
MediaController mc = new MediaController(this);

//设置 VideoView 的控制媒体播放机制
videoView.setMediaController(mc);

videoView.requestFocus();

//开始播放视频
videoView.start();

}

}
```

18.3 音频及视频播放——MediaPlayer

播放媒体的来源可以来自源文件、文件系统或网络的数据流（URL）。

只能在标准的输出设备上播放音频，输出设备指的是移动设备上扬声器或蓝牙耳机，不可以从电话的话筒播放音频。一般而言，移动手机会有播放音乐用的扬声器和通话用的话筒。

这里我们主要使用 MediaPlayer 来实现音频及视频的播放功能。下面来介绍一下 MediaPlayer 的常用方法（表 8-2）及监听事件（表 8-3）。

表 8-2 MediaPlayer 的常用方法



方法	描述
create(Context context, Uri uri)	静态方法，通过 Uri 创建一个多媒体播放器
create(Context context, int resid)	静态方法，通过资源 ID 创建一个多媒体播放器
create(Context context, Uri uri, SurfaceHolder holder)	静态方法，通过 Uri 和指定 SurfaceHolder 【抽象类】 创建一个多媒体播放器
getCurrentPosition()	返回 Int，得到当前播放位置
getDuration()	返回 Int，得到文件的时间
getVideoHeight()	返回 Int，得到视频的高度
getVideoWidth()	返回 Int，得到视频的宽度
isLooping()	返回 boolean，是否循环播放
isPlaying()	返回 boolean，是否正在播放
pause()	无返回值，暂停
prepare()	无返回值，准备同步
prepareAsync()	无返回值，准备异步
release()	无返回值，释放 MediaPlayer 对象
reset()	无返回值，重置 MediaPlayer 对象
seekTo(int msec)	无返回值，指定播放的位置（以毫秒为单位的时间）
setAudioStreamType(int streamtype)	无返回值，指定流媒体的类型
setDataSource(String path)	无返回值，设置多媒体数据来源【根据 路径】



setDataSource(FileDescriptor fd, long offset, long length)	无返回值，设置多媒体数据来源【根据 FileDescriptor】
setDataSource(FileDescriptor fd)	无返回值，设置多媒体数据来源【根据 FileDescriptor】
setDataSource(Context context, Uri uri)	无返回值，设置多媒体数据来源【根据 Uri】
setDisplay(SurfaceHolder sh)	无返回值，设置用 SurfaceHolder 来显示多媒体
setLooping(boolean looping)	无返回值，设置是否循环播放
setScreenOnWhilePlaying(boolean screenOn)	无返回值，设置是否使用 SurfaceHolder 显示
setVolume(float leftVolume, float rightVolume)	无返回值，设置音量
start()	• 无返回值，开始播放
stop()	无返回值，停止播放

表 8-3 MediaPlayer 的监听事件

事件	描述
setOnBufferingUpdateListener(MediaPlayer.OnBufferingUpdateListener listener)	网络流媒体的缓冲监听
setOnCompletionListener(MediaPlayer.OnCompletionListener listener)	网络流媒体播放结束监听
setOnErrorListener(MediaPlayer.OnErrorListener listener)	设置错误信息监听
setOnVideoSizeChangedListener(MediaPlayer.OnVideoSizeChangedListener listener)	视频尺寸监听



18.3.1 从源文件中播放

使用 MediaPlayer 类播放音频及视频，最简单的一种方式就是从源文件中播放。要实现该功能需要以下步骤。

将 Android 支持的媒体文件放在项目的 res/raw 文件夹下，如一个 MP3 文件，Eclipse 开发环境的内置功能会发现这个文件，并将它生成一个信息，可以自 R 类引用到这个文件。

创建一个 MediaPlayer 实例，可以使用 MediaPlayer 的静态方法 create 来完成，读出媒体资源。

调用 MediaPlayer 实例的 start() 方法开始播放，调用 stop() 方法停止播放，调用 pause() 方法暂停播放，如暂停后再播放，需再次调用 start() 方法。如果希望重复播放，在调用 start() 方法之前，必须调用 reset() 和 prepare() 方法。

播放流程如图 8-1 所示。

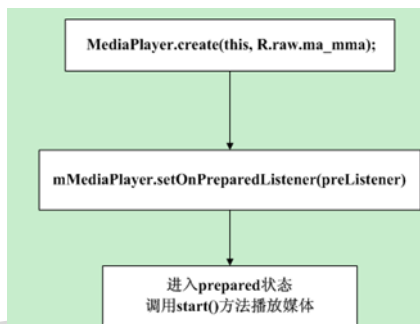


图 8-1 播放流程

程序代码如代码清单 8-2 所示。

代码清单 8-2 播放源文件中视频或音频方法

```
//实例化 MediaPlayer
MediaPlayer mMediaPlayer = MediaPlayer.create(this,R.raw.test);
//开始播放
mMediaPlayer.start();
```

由上述可知，从源文件中播放是放在“<工程>/res/raw”，然后发布的时候被打成 APK 包一起安装在手机上。很显然这种方式不适合用于播放娱乐为主的多媒体文件，由于娱乐的多媒体文件是经常更新的，而放置在这个 raw 下面的文件用户是没有权限更新的，因此这种方式一般用于应用自己的一些音频和视频播放，如按键音、开机启动音、信息提示音等应用使用的声音。

18.3.2 从文件系统中播放

从文件系统中播放也是比较常见的一种方式。例如，在 SD 卡中有一些音频及视频文件，那么就可以直接进行播放。

从文件系统中播放需要以下步骤。

实例化一个 MediaPlayer。



调用 `setDataSource()` 方法来设置想要播放文件的路径，取得播放的媒体文件。

首先调用 `prepare()`，然后调用 `start()` 来进行播放。

播放流程如图 8-2 所示。

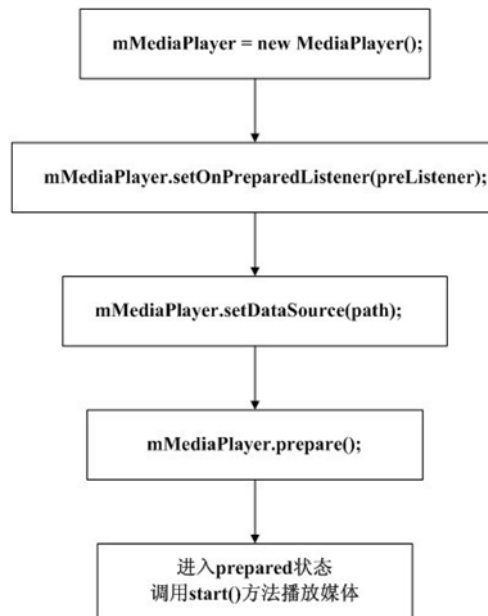


图 8-2 从文件系统中播放音频视频的播放流程

程序代码如代码清单 8-3 所示。

代码清单 8-3 从文件系统中播放音频或视频

```

private void play() {
    File sdCardDir = Environment.getExternalStorageDirectory();
    //播放路径
    String path = sdCardDir.getPath() + "/Lover's Concerto.mp3";
    try {
        if (mMediaPlayer == null || state == STOP) {
            // 创建 MediaPlayer 对象并设置 Listener
            mMediaPlayer = new MediaPlayer();
            mMediaPlayer.setOnPreparedListener(preListener);
        } else {
            // 复用 MediaPlayer 对象
            mMediaPlayer.reset();
        }
        //为 MediaPlayer 设置数据源
        mMediaPlayer.setDataSource(path);
        mMediaPlayer.prepare();
    }
}
    
```



```
mMediaPlayer.start();  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}
```

从源文件中播放音频视频和从本地文件系统中播放音频视频的差别就在于 `play()` 方法中的 `mMediaPlayer = new MediaPlayer()`，而不是 `MediaPlayer.Create()`，然后设置预处理监听器，设置 `setDataSource(path)`，其中的 `path` 是指定的音频文件，这里放置 SD 卡中的 MP3 文件，`mMediaPlayer.prepare()` 开始预处理。

18.3.3 从网络中播放

随着 3G 的不断完善和推广，自费不断降低，直接利用网络资源已经不是什么问题。移动互联网时代已经到了。这里就看一看如何通过网络来播放音视频文件。

要实现该功能有两种方法。

方法一

创建网络 URI 实例。

创建一个 `MediaPlayer` 实例，通过传递 URI 参数，使用 `MediaPlayer` 的静态方法 `create` 完成。

调用 `start()` 方法开始播放。

程序代码如代码清单 8-4 所示。

代码清单 8-4 从网络中播放音频或视频方法一

```
private void play() {  
    try {  
        //播放路径  
        String path =  
            "http://ms1.m.mop.com/2008/08/27/012/12198150127399164.mp3";  
        //将字符串 Uri 解析为 Uri 实例  
        Uri uri = Uri.parse(path);  
        //实例化 MediaPlayer  
        MediaPlayer player = MediaPlayer.create(this, uri);  
        //播放  
        player.start();  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}
```

方法二



实例化一个 MediaPlayer。

调用 setDataSource()方法来设置想要播放文件的路径，该路径是网络上的可用路径。

首先调用 prepare()，然后调用 start()来进行播放。

程序代码如代码清单 8-5 所示。

代码清单 8-5 从网络中播放音频或视频方法二

```
private void play() {  
    try {  
        //实例化 MediaPlayer  
        MediaPlayer mp = new MediaPlayer();  
        //播放路径  
        String path =  
            "http://ms1.m.mop.com/2008/08/27/012/12198150127399164.mp3";  
        try{  
            //设置数据源  
            mp.setDataSource(path);  
            //准备  
            mp.prepare();  
            //播放  
            mp.start();  
        }catch(IllegalArgumentException e){  
            e.printStackTrace();  
        }catch(IllegalStateException e){  
            e.printStackTrace();  
        }catch(IOException e){  
            e.printStackTrace();  
        }  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}
```

虽然从网络中播放音频视频不用存储比较方便，但它的弊端就是要通过网络，速度要比从源文件中
和从文件系统中播放两种方式慢，因此上述两种播放方式中，利用同步预处理方法 prepare()播放网络文
件用户体验不好。一般网络播放方式要采用异步预处理方式。还有网络流量费用也会增加用户的使用成
本。



播放流程如图 8-3 所示。

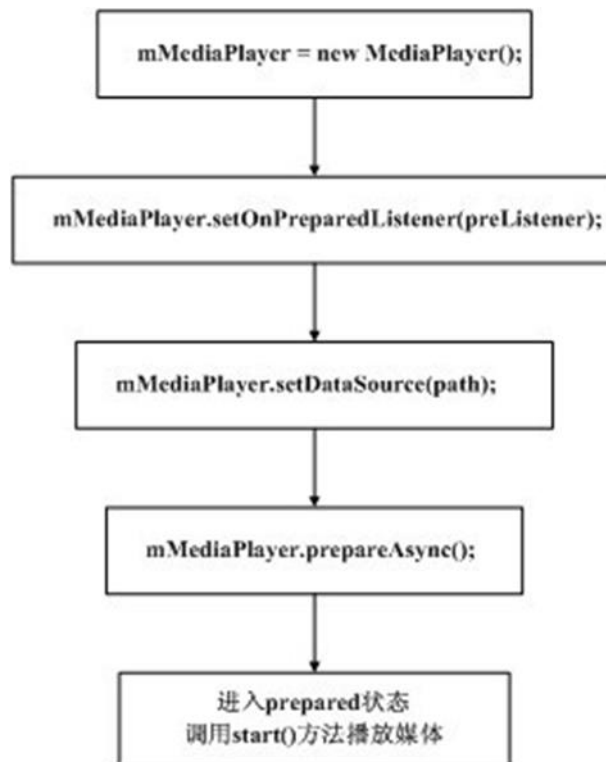


图 8-3 从网络中播放音频视频流程图

程序代码如代码清单 8-6 所示。

代码清单 8-6 从网络中播放音频或视频——异步处理方式

```
private void play() {  
    try {  
        if (mMediaPlayer == null || state == STOP) {  
            // 创建 MediaPlayer 对象并设置 Listener  
            mMediaPlayer = new MediaPlayer();  
            mMediaPlayer.setOnPreparedListener(preListener);  
        } else {  
            // 复用 MediaPlayer 对象  
            mMediaPlayer.reset();  
        }  
        // 播放网络媒体最好不要调用 prepare()方法  
        mMediaPlayer  
            .setDataSource("http://ms1.m.mop.com/2008/08/27/012/12198150127399164.mp3");  
        mMediaPlayer.prepareAsync();  
    } catch (Exception e) {
```



```
e.printStackTrace();
}
}
```

18.3.4 音频播放示例

本示例“MediaPlayerAudio.java”主要功能是播放音乐文件，因读取音乐文件的来源不同，现设置一个文本框来显示其来源，如表 8-4 所示。

表 8-4 MediaPlayerAudio.java 文本框显示

文件来源	文本框显示
从源文件中播放	LOCAL_AUDIO-播放音乐中
从文件系统中播放	RESOURCES_AUDIO-播放音乐中
从网络中播放	STREAM_AUDIO-播放音乐中

MediaPlayerAudio.java 代码如代码清单 8-7 所示。

代码清单 8-7 MediaPlayerAudio.java

```
package cn.com.farsight.media;

import android.app.Activity;
import android.media.MediaPlayer;
import android.os.Bundle;
import android.util.Log;
import android.widget.TextView;
import android.widget.Toast;

public class MediaPlayerAudio extends Activity {
    private static final String TAG = "MediaPlayerDemo";
    private MediaPlayer mMediaPlayer;
    private static final String MEDIA = "media";
    private static final int LOCAL_AUDIO = 1;
    private static final int STREAM_AUDIO = 2;
    private static final int RESOURCES_AUDIO = 3;
    private static final int LOCAL_VIDEO = 4;
    private static final int STREAM_VIDEO = 5;
    private static final int RESOURCES_VIDEO = 6;
    private String path;
    private TextView mTextView;
```



```
//MediaPlayerAudio
@Override
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.mediaplayer02);
    mTextView = (TextView) findViewById(R.id.TextView01);
    Bundle extras = getIntent().getExtras();
    playAudio(extras.getInt(MEDIA));
}
//播放音乐
private void playAudio(Integer media) {
    try {
        switch (media) {
            case LOCAL_AUDIO:
                //设定音频文件路径
                path = "/sdcard/song01.mp3";//测试用的音频文件
                //path = "";
                if (path == "") {
                    //警告 User 尚未建立音频文件
                    Toast.makeText(MediaPlayerAudio.this,"尚未建立音频文件",
Toast.LENGTH_LONG).show();
                }
                mMediaPlayer = new MediaPlayer();
                mMediaPlayer.setDataSource(path);
                mMediaPlayer.prepare();
                mMediaPlayer.start();
                mTextView.setText("LOCAL_AUDIO-播放音乐中...");
                break;
            case STREAM_AUDIO:
                //设定音频文件路径的 URL
                path = " http://116.28.63.250/7705.mp3";
                //path = "";
                if (path == "") {
                    //警告 User 尚未建立音频文件的 URL.
                    Toast.makeText(MediaPlayerAudio.this,"Please edit MediaPlayerDemo_Video
```



```
Activity and set the path variable to your media file URL.", Toast.LENGTH_LONG).show();

        }

        mMediaPlayer = new MediaPlayer();
        mMediaPlayer.setDataSource(path);
        mMediaPlayer.prepare();
        mMediaPlayer.start();
        mTextView.setText("STREAM_AUDIO-播放音乐中...");
        break;

    case RESOURCES_AUDIO:
        //本地音频文件要放在/res/raw 目录内且提供 resid 给 MediaPlayer.create()
        mMediaPlayer = MediaPlayer.create(this, R.raw.song21);
        mMediaPlayer.start();
        mTextView.setText("RESOURCES_AUDIO-播放音乐中...");
        break;

    }

} catch (Exception e) {
    Log.e(TAG, "error: " + e.getMessage(), e);
}

}

//关闭 MediaPlayerAudio 程序且释放 MediaPlayer
@Override
protected void onDestroy() {
    super.onDestroy();
    if (mMediaPlayer != null) {
        mMediaPlayer.release();
        mMediaPlayer = null;
    }
}

}
```

18.3.5 视频播放示例

本示例“MediaPlayerVideo.java”主要功能是播放视频文件，因读取视频文件的来源不同，现设置一个文本框来显示其来源，如表 8-5 所示。

表 8-5 MediaPlayerVideo.java 文本框显示



文件来源	文本框显示
从源文件中播放	LOCAL_VIDEO-播放音乐中...
从文件系统中播放	RESOURCES_VIDEO-播放音乐中...
从网络中播放	STREAM_VIDEO-播放音乐中...

播放视频文件前，要确认是否已进入 Prepared 状态和读入视频文件，取得播放视频的长度。为了确认这两件事情的同步完成时刻，首先要实现 onPrepared()和 onVideoSizeChanged()方法的监听功能，管理两个事件的同步完成时刻变量 mIsVideoReadyToBePlayed 和 mIsVideoSizeKnown，当 onPrepared()方法被执行时，会设置变量 mIsVideoReadyToBePlayed 为 true，由 Initialized 状态进入 Prepared 状态；当 onVideoSizeChanged()方法执行时，会设置变量 mIsVideoSizeKnown 为 true，取得播放视频的长度。当变量 mIsVideoReadyToBePlayed 和 mIsVideoSizeKnown 都为 true 时，调用 startVideoPlayback()方法，设置好播放视频的长度，再开始播放。

代码清单 8-8 MediaPlayerAudio.java

```
package cn.com.farsight.media;

import android.app.Activity;
import android.media.AudioManager;
import android.media.MediaPlayer;
import android.media.MediaPlayer.OnBufferingUpdateListener;
import android.media.MediaPlayer.OnCompletionListener;
import android.media.MediaPlayer.OnPreparedListener;
import android.media.MediaPlayer.OnVideoSizeChangedListener;
import android.os.Bundle;
import android.util.Log;
import android.view.SurfaceHolder;
import android.view.SurfaceView;
import android.widget.TextView;
import android.widget.Toast;

public class MediaPlayerVideo extends Activity implements
    OnBufferingUpdateListener, OnCompletionListener,
    OnPreparedListener, OnVideoSizeChangedListener, SurfaceHolder.Callback {
    private static final String TAG = "MediaPlayerDemo";
    private int mVideoWidth;
    private int mVideoHeight;
```




```
private MediaPlayer mMediaPlayer;
private SurfaceView mPreview;
private SurfaceHolder holder;
private String path;
private TextView mTextView;
private Bundle extras;
private static final String MEDIA = "media";
private static final int LOCAL_AUDIO = 1;
private static final int STREAM_AUDIO = 2;
private static final int RESOURCES_AUDIO = 3;
private static final int LOCAL_VIDEO = 4;
private static final int STREAM_VIDEO = 5;
private static final int RESOURCES_VIDEO = 6;
private boolean mIsVideoSizeKnown = false;
private boolean mIsVideoReadyToBePlayed = false;
//MediaPlayerVideo 程序
@Override
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.mediaplayer01);
    mTextView = (TextView) findViewById(R.id.TextView01);
    mPreview = (SurfaceView) findViewById(R.id.surface);
    holder = mPreview.getHolder();
    holder.addCallback(this);
    holder.setType(SurfaceHolder.SURFACE_TYPE_PUSH_BUFFERS);
    extras = getIntent().getExtras();
}
//播放视频
private void playVideo(Integer Media) {
    doCleanup();
    try {
        switch (Media) {
            case LOCAL_VIDEO:
                //设定视频文件路径
                path = "/sdcard/navy.3gp";//测试用的视频文件
```



```
//path = "";
if (path == "") {
    //警告 User 尚未建立视频文件
    Toast.makeText(MediaPlayerVideo.this, "尚未建立视频文件
",Toast.LENGTH_LONG).show();
}
mTextView.setText("LOCAL_VIDEO-播放视频中...");
break;
case STREAM_VIDEO:
    //警告 User 尚未建立视频文件的 URL.
    path = " http://www.yinyuetai.com/video/307729 ";//测试用的视频文件
    //path = "";
    if (path == "") {
        //警告 User 尚未建立视频文件的 URL.
        Toast.makeText(MediaPlayerVideo.this,"Please edit MediaPlayerDemo_
Video Activity and set the path variable to your media file URL.",Toast.LENGTH_LONG).show();
    }
    mTextView.setText("STREAM_VIDEO-播放视频中...");
    break;
case RESOURCES_VIDEO:
    //本地视频文件要放在/res/raw 目录内且提供 resid 给 MediaPlayer.create()
    //mMediaPlayer = new MediaPlayer();
    //mMediaPlayer.setDataSource("/sdcard/navy.3gp");
    mMediaPlayer = MediaPlayer.create(this, R.raw.navy);
    //执行 create()时, 会调用 prepare()
    mMediaPlayer.setDisplay(holder);
    //mMediaPlayer.prepare();
    mMediaPlayer.setOnBufferingUpdateListener(this);
    mMediaPlayer.setOnCompletionListener(this);
    mMediaPlayer.setOnPreparedListener(this);
    mMediaPlayer.setOnVideoSizeChangedListener(this);
    mMediaPlayer.setAudioStreamType(AudioManager.STREAM_MUSIC);
    mMediaPlayer.start();
    mTextView.setText("RESOURCES_VIDEO-播放视频中... ");
    break;
```



```
    }

    //实例化 MediaPlayer 且设定监听功能
    if (Media == LOCAL_VIDEO || Media == STREAM_VIDEO) {
        mMediaPlayer = new MediaPlayer();
        mMediaPlayer.setDataSource(path);
        mMediaPlayer.setDisplay(holder);
        mMediaPlayer.prepare();
        mMediaPlayer.setOnBufferingUpdateListener(this);
        mMediaPlayer.setOnCompletionListener(this);
        mMediaPlayer.setOnPreparedListener(this);
        mMediaPlayer.setOnVideoSizeChangedListener(this);
        mMediaPlayer.setAudioStreamType(AudioManager.STREAM_MUSIC);
    }
} catch (Exception e) {
    Log.e(TAG, "error: " + e.getMessage(), e);
}

}

//onBufferingUpdate
public void onBufferingUpdate(MediaPlayer arg0, int percent) {
    Log.d(TAG, "onBufferingUpdate percent: " + percent);
}

//onCompletion
public void onCompletion(MediaPlayer arg0) {
    Log.d(TAG, "onCompletion called");
}

//onVideoSizeChanged
public void onVideoSizeChanged(MediaPlayer mp, int width, int height) {
    Log.v(TAG, "onVideoSizeChanged called");
    Toast.makeText(MediaPlayerVideo.this, "onVideoSizeChanged called", Toast.LENGTH_
LONG).show();
    if (width == 0 || height == 0) {
        Log.e(TAG, "invalid video width(" + width + ") or height(" + height + ")");
        return;
    }
    mIsVideoSizeKnown = true;
}
```



```
        mVideoWidth = width;
        mVideoHeight = height;
        if (mIsVideoReadyToBePlayed && mIsVideoSizeKnown) {
            startVideoPlayback();
        }
    }

    //onPrepared
    public void onPrepared(MediaPlayer mediaplayer) {
        Log.d(TAG, "onPrepared called");
        Toast.makeText(MediaPlayerVideo.this, "onPrepared called", Toast.LENGTH_LONG).
show();

        mIsVideoReadyToBePlayed = true;
        if (mIsVideoReadyToBePlayed && mIsVideoSizeKnown) {
            startVideoPlayback();
        }
    }

    //surfaceChanged
    public void surfaceChanged(SurfaceHolder surfaceholder, int I, int j, int k) {
        Log.d(TAG, "surfaceChanged called");
    }

    //surfaceDestroyed
    public void surfaceDestroyed(SurfaceHolder surfaceholder) {
        Log.d(TAG, "surfaceDestroyed called");
    }

    //调用 playVideo(), 播放视频
    public void surfaceCreated(SurfaceHolder holder) {
        Log.d(TAG, "surfaceCreated called");
        playVideo(extras.getInt(MEDIA));
    }

    //onPause
    @Override
    protected void onPause() {
        super.onPause();
        releaseMediaPlayer();
        doCleanup();
    }
}
```



```
}  
  
//关闭 MediaPlayerVideo 程序且释放 MediaPlayer  
@Override  
protected void onDestroy() {  
    super.onDestroy();  
    releaseMediaPlayer();  
    doCleanUp();  
}  
  
//releaseMediaPlayer  
private void releaseMediaPlayer() {  
    if (mMediaPlayer != null) {  
        mMediaPlayer.release();  
        mMediaPlayer = null;  
    }  
}  
  
//doCleanUp  
private void doCleanUp() {  
    mVideoWidth = 0;  
    mVideoHeight = 0;  
    mIsVideoReadyToBePlayed = false;  
    mIsVideoSizeKnown = false;  
}  
  
//startVideoPlayback  
private void startVideoPlayback() {  
    Log.v(TAG, "startVideoPlayback");  
    Toast.makeText(MediaPlayerVideo.this, "startVideoPlayback()", Toast.LENGTH_LONG).show();  
    holder.setFixedSize(mVideoWidth, mVideoHeight);  
    mMediaPlayer.start();  
}  
}
```

18.4 音/视频的录制

鉴于现在大部分的手机都提供了音频录制的功能，Android 系统相应地提供了对音频及视频录制的支持，当然这也需要有硬件的支持。要想录制音频及视频需要通过 **MediaRecorder** 来完成。

MediaRecorder 类位于 **android.media** 包中，它包含了音频及视频的记录功能。常用的方法如表 8-6 所示。



表 8-6 MediaRecorder 的常用方法

方法	描述	方法	描述
start()	开始录制	prepare()	准备录制
pause()	暂停录制	reset()	重新开始
stop()	停止录制	release()	释放资源

MediaRecorder 类的工作流程如图 8-4 所示。

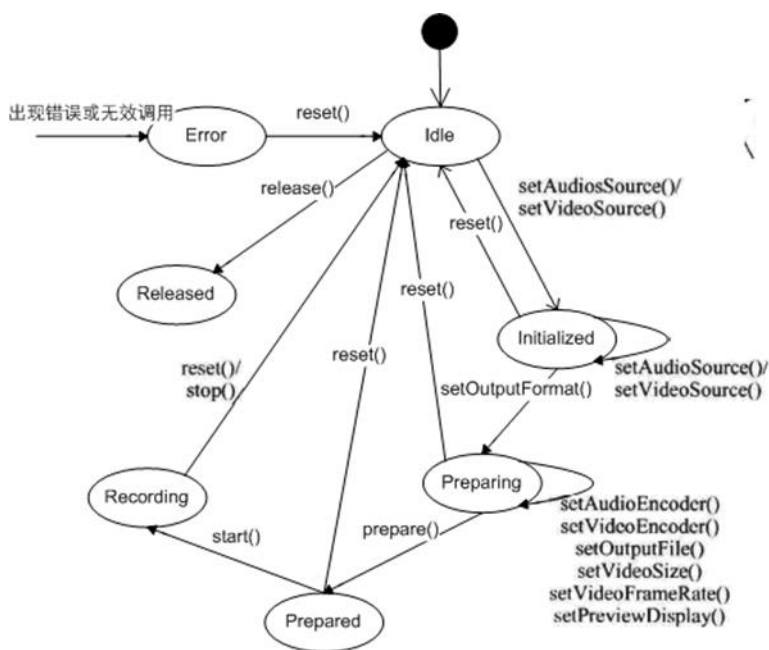


图 8-4 MediaRecorder 类的工作流程

要实现音频及视频的录制的步骤说明如下所示。

在工程的 res/drawable 目录下放置两个图片文件，用来设置控制按钮的图片。

在布局文件 main.xml 中添加 3 个 View 组件，其中 TextView 用来显示标题，其余两个按钮用来控制开始录制和停止录制。

创建 Activity 类 MainActivity。

生成两个 ImageButton 实例。

创建 loadCip()方法，实例化 MediaRecorder。

在 onCreate()方法中实例化。

创建 record()方法，调用 MediaRecorder 的 start()方法录制，同时使录制按钮失效，使停止按钮生效。

创建 stop()方法，调用 MediaRecorder 的 stop()方法停止录制。

创建 error()方法捕获异常。

创建 setProperty()方法，设置 MediaRecorder 的属性，包括用 MediaRecorder.setAudioSource()方法来设置音频源，你可以用 MediaRecorder.AudioSource.MIC 方法来设置麦克风；用 MediaRecorder.setOutputFormat()方法来设置输出文件格式；用 MediaRecorder.setAudioEncoder()方法来设



置音频编码格式。

创建 `setup()` 方法，调用 `loadClip()`，使录制按钮生效，使停止按钮失效。

为录制按钮添加单击事件，调用 `play()` 方法录制。

为停止按钮添加单击事件，调用 `stop()` 方法停止录制。

当实现的 `MediaRecorder` 类完成音频的捕捉后，调用 `release()` 方法。

布局文件 `main.xml` 的代码如代码清单 8-9 所示。

代码清单 8-9 `main.xml`

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    >

    <TextView
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="迷你录音器"
    />

    <LinearLayout
        android:orientation="horizontal"
        android:layout_width="fill_parent"
        android:layout_height="fill_parent">

        <ImageButton
            android:id="@+id/record"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:src="@drawable/record"
        ></ImageButton>

        <ImageButton
            android:id="@+id/stop"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
```



```
        android:src="@drawable/stop"  
    ></ImageButton>  
  
</LinearLayout>  
</LinearLayout>
```

Activity 代码如代码清单 8-10 所示。

代码清单 8-10 MainActivity.java

```
package cn.com.farsight.record;  
  
import java.io.File;  
import java.io.IOException;  
  
import com.examples.android.helloactivity.R;  
  
import android.app.Activity;  
import android.app.AlertDialog;  
import android.media.MediaRecorder;  
import android.os.Bundle;  
import android.view.View;  
import android.widget.ImageButton;  
  
public class MainActivity extends Activity {  
    //声明图片按钮 ImageButton  
    private ImageButton record,stop;  
    //声明 MediaRecorder  
    private MediaRecorder mr;  
    //声明路径  
    private String path;  
  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        //设置当前 Activity 界面布局  
        setContentView(R.layout.main);  
        //通过 findViewById 方法获得 ImageButton 实例  
        record = (ImageButton)findViewById(R.id.record);
```




```
stop = (ImageButton)findViewById(R.id.stop);

//为录制按钮设置单击事件监听器
record.setOnClickListener(new View.OnClickListener() {

    @Override
    public void onClick(View v) {
        // TODO Auto-generated method stub
        //录制
        record();
    }
});

//为停止按钮设置单击事件监听器
stop.setOnClickListener(new View.OnClickListener() {

    @Override
    public void onClick(View v) {
        // TODO Auto-generated method stub
        //停止录制
        stop();
    }
});

//准备
setup();
}

//设置路径
private void setPath() throws IOException{
    //路径为 sd 扩展卡
    path = "/sdcard/test.mp3";
    //获得 sd 扩展卡状态
    String state = android.os.Environment.getExternalStorageState();
    //判断是否安装扩展卡
    if(!state.equals(android.os.Environment.MEDIA_MOUNTED)){
        throw new IOException("SD 没有安装，它的状态是"+state+".");
    }
}
```



```
    }  
    //获取文件父目录  
    File directory = new File(path).getParentFile();  
    if(!directory.exists() && !directory.mkdirs()){  
        throw new IOException("文件不能被创建。");  
    }  
}  
  
@Override  
protected void onDestroy() {  
    // TODO Auto-generated method stub  
    super.onDestroy();  
    if(stop.isEnabled()){  
        //停止录制  
        stop();  
    }  
}  
  
//录制方法  
private void record(){  
    try{  
        //准备  
        mr.prepare();  
    }catch(IllegalStateException e){  
        e.printStackTrace();  
    }catch(IOException e){  
        e.printStackTrace();  
    }  
    //开始录制  
    mr.start();  
    //使录制按钮失效  
    record.setEnabled(false);  
    //使停止按钮生效  
    stop.setEnabled(true);  
}
```



```
//停止
private void stop(){
    //停止录制
    mr.stop();
    //使停止按钮失效
    stop.setEnabled(false);
    //使录制按钮生效
    record.setEnabled(true);
    //释放资源
    mr.release();
}

//设置属性
private void setProperty(){
    try{
        //实例化 MediaRecorder
        mr = new MediaRecorder();
        //设置音频源
        mr.setAudioSource(MediaRecorder.AudioSource.MIC);
        //设置输出格式
        mr.setOutputFormat(MediaRecorder.OutputFormat.THREE_GPP);
        //设置音频编码器
        mr.setAudioEncoder(MediaRecorder.AudioEncoder.AMR_NB);
        try{
            //设置路径
            setPath();
        }catch(IOException e){
            e.printStackTrace();
        }
        //设置输出文件
        mr.setOutputFile(path);
    }catch(Throwable t){
        error(t);
    }
}
```



```
}

//准备
private void setup(){
    //设置属性
    setPeoperty();
    //使录制按钮生效
    record.setEnabled(true);
    //使停止按钮生效
    stop.setEnabled(false);
}

//错误提示
private void error(Throwable t){
    //对话框
    AlertDialog.Builder builder = new AlertDialog.Builder(this);
    builder.setTitle("报错啦！").setMessage(t.toString()).setPositiveButton("确定", null).show();
}
}
```

18.5 Camera 照相

除了手机录制音/视频外，手机拍照也已经不是什么新鲜事了，Android 相应地提供了手机拍照功能 API。要使用 Android 系统进行拍照，使用到的类有很多。具体介绍如下。

SurfaceView：该类是一个视图组件，实现了照片的预览功能。

SurfaceHolder：一个抽象接口，是 SurfaceView 的支持类，一般通过 SurfaceView 的 `getHolder()` 方法获得。允许你控制界面的尺寸和格式、编辑界面的像素，以及监控界面尺寸的改变。

SurfaceHolder.Callback：SurfaceHolder 的一个内部接口，可以实现该接口获取界面改变的信息。该接口中有三个方法，`surfaceCreated()` 方法在界面创建时被调用，一般在该方法中打开照相机并设置预览。`surfaceChanged()` 方法在界面尺寸改变时被调用，一般在该方法中设置相机的参数（大小、格式等）。`surfaceDestroyed()` 方法在界面被销毁时调用，在该方法中清除相机实例，释放资源。

Camera：相机类，实现拍照功能。

Camera.PictureCallback：Camera 的一个内部接口，处理照片准备好后的回调。

通过上面类的介绍，可以实现一个拍照程序了。下面是程序的实现步骤。

创建一个 Android 工程“Camera”，在该工程的“com.example”包中，创建一个 MainActivity 类，在该类的顶部声明 SurfaceView、SurfaceHolder 和 Camera 对象，代码如代码清单 8-11 所示。



代码清单 8-11 MainActivity 类中声明对象

```
package com.example;

import android.app.Activity;
import android.hardware.Camera;
import android.os.Bundle;
import android.view.SurfaceHolder;
import android.view.SurfaceView;

public class MainActivity extends Activity {
    //声明拍照界面组件 SurfaceView
    private SurfaceView surfaceView;
    //声明界面控制组件 SurfaceHolder
    private SurfaceHolder surfaceHolder;
    //声明照相机
    private Camera camera;

    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        //设置当前 Activity 界面布局
        setContentView(R.layout.main);
    }
}
```

在该工程 res\layout\目录下的布局文件 main.xml 中添加 SurfaceView 视图组件。代码如代码清单 8-12 所示。

代码清单 8-12 main.xml 中添加 SurfaceView 视图组件

```
<?xml version="1.0" encoding="utf-8"?>
<SurfaceView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/preview"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent">
</SurfaceView>
```



在 onCreate()方法中获得 SurfaceView 和 SurfaceHolder 实例对象。代码如代码清单 8-13 所示。

代码清单 8-13 onCreate()获取 SurfaceView、SurfaceHolder 实例对象

```
//实例化拍照界面组件
surfaceView = (SurfaceView) findViewById(R.id.preview);
//从 SurfaceView 中获得 SurfaceHolder
surfaceHolder=surfaceView.getHolder();
```

实现 SurfaceHolder.Callback 回调接口，实现该接口中的 3 个方法。代码如代码清单 8-14 所示。

代码清单 8-14 实现 SurfaceHolder.Callback 回调接口

```
SurfaceHolder.Callback surfaceCallback = new SurfaceHolder.Callback() {
    @Override
    public void surfaceDestroyed(SurfaceHolder holder) {
        //停止预览
        camera.stopPreview();
        //释放相机资源
        camera.release();
        camera=null;
    }

    @Override
    public void surfaceCreated(SurfaceHolder holder) {
        //打开相机
        camera = Camera.open();
        try{
            //设置预览
            camera.setPreviewDisplay(holder);
        }catch(IOException e){
            e.printStackTrace();
        }
    }

    @Override
    public void surfaceChanged(SurfaceHolder holder, int format, int width,
                               int height) {
        //获得相机参数
        Camera.Parameters parameters = camera.getParameters();
```



```
//设置相片大小
parameters.setPictureSize(width, height);
//设置照片格式
parameters.setPictureFormat(PixelFormat.JPEG);
//开始预览
camera.startPreview();
}
};
```

为 SurfaceHolder 添加回调，并设置其类型。代码如代码清单 8-15 所示。

代码清单 8-15 为 SurfaceHolder 添加回调

```
surfaceHolder.addCallback(surfaceCallback);
surfaceHolder.setType(SurfaceHolder.SURFACE_TYPE_PUSH_BUFFERS);
```

创建一个保存图片任务类，该类的主要功能是将硬件获得的字节流保存到设备中。代码如代码清单 8-16 所示。

代码清单 8-16 创建保存图片任务类

```
public class SavePictureTask extends AsyncTask<byte[], String, String> {
    @Override
    protected String doInBackground(byte[]... params) {
        // TODO Auto-generated method stub
        //创建文件
        File picture = new File(Environment.getExternalStorageDirectory(),
                                "picture.jpg");
        //如果文件存在，删除现存文件
        if(picture.exists())
            picture.delete();
        try{
            //获取文件输出流
            FileOutputStream fos = new
                FileOutputStream(picture.getParent());

            //写到该文件
            fos.write(params[0]);
            //关闭文件流
            fos.close();
        }catch(Exception e){
```



```
e.printStackTrace();  
}  
return null;  
}  
}
```

实现 Camera.PictureCallback 回调方法，执行保存照片任务。代码如代码清单 8-17 所示。

代码清单 8-17 实现 Camera.PictureCallback 回调方法

```
//照片回调  
Camera.PictureCallback pictureCallback = new Camera.PictureCallback() {  
    @Override  
    public void onPictureTaken(byte[] data, Camera camera) {  
        new SavePictureTask().execute(data);  
        camera.startPreview();  
    }  
};
```

创建保存照片方法。代码如代码清单 8-18 所示。

代码清单 8-18 创建保存照片方法

```
//拍照方法  
private void takePic(){  
    //停止预览  
    camera.stopPreview();  
    //拍照  
    camera.takePicture(null, null, pictureCallback);  
}
```

相应按键事件，当拍照键或搜索键被按下时进行拍照。代码如代码清单 8-19 所示。

代码清单 8-19 相应按键事件

```
//相应按键事件  
@Override  
public boolean onKeyDown(int keyCode, KeyEvent event) {  
    if(keyCode==KeyEvent.KEYCODE_CAMERA||  
    keyCode==KeyEvent.KEYCODE_SEARCH){  
        takePic();  
        return true;  
    }  
    return super.onKeyDown(keyCode, event);  
}
```




}

18.6 本章小结

本章主要学习了调用 Android 提供的现有 API 来进行多媒体的开发，主要包括音频及视频的播放，音/视频的录制及照相功能的实现。

华清远见
dev.havi.com



第 19 章 定位与地图

在上一章的学习中，主要了解了如何调用 Android 提供的现有 API 来进行多媒体的开发，包括音频及视频的播放、音频视频的录制，以及照相功能的实现。本章主要学习位置服务的相关概念、地图密钥的申请、位置信息的获取、MapView 和 MapController 的使用方法、Google 地图覆盖层的使用方法等。

19.1 位置服务

Android 支持 GPS 和网络地图，通常将各种不同的定位技术称为位置服务。位置服务（Location-Based Services, LBS），又称定位服务或基于位置的服务，它通过电信运营商的无线电通信网络（如 GSM 网、CDMA 网）或外部定位方式（如 GPS），融合了 GPS 定位、移动通信、导航等多种技术，获取移动终端用户的位置信息，为移动用户提供了与空间位置相关的综合应用服务。

位置服务首先在日本得到商业化的应用。2001 年 7 月，DoCoMo 发布了第一款具有三角定位功能的手持设备；2001 年 12 月，KDDI 发布第一款具有 GPS 功能的手机；如今，基于位置的服务发展迅速，已涉及商务、医疗、工作和生活的各个方面，为用户提供定位、追踪和敏感区域警告等一系列服务。

19.1.1 Android Location API

Android 平台支持提供位置服务的 API 主要包括以下几种。

LocationManager 可以用来获取当前的位置，追踪设备的移动路线，或设定敏感区域，在进入或离开敏感区域时设备会发出特定警报。

LocationProviders 是能够提供定位功能的组件集合，集合中的每种组件以不同的技术提供设备的当前位置，区别在于定位的精度、速度和成本等方面。

LocationListener 接口用于接收从 LocationManager 的位置发生改变时的通知。如果 LocationListener 被注册添加到 LocationManager 对象，并且此 LocationManager 对象调用了 requestLocationUpdates（String, long, float, LocationListener）方法，那么接口中的相关方法将会被调用。其公共方法如表 9-1 所示。

表 9-1 LocationListener 类的方法

抽象方法	参数描述	说明
public abstract void onLocationChanged (Location location)	location: 位置发生变化后的新位置	当位置 发生改变后 被调用
public abstract void onProviderDisabled(String provider)	provider: 与之关联的 location provider 名称	在 provider 被 用户关闭后 被调用
public abstract void onPorviderEnabled (String provider)	provider: 与之关联的 location provider 名称	provider 被用户开启 后调用



<pre>public abstract void onStatusChanged (String provider, int Status, Bundle extras)</pre>	provider: 与变化相关的 location provider 名称。 status : 如果服务已停止, 并且在短时间内不会改变, 状态码为 OUT_OF_SERVICE; 如果服务暂时停止, 并且在短时间内会恢复, 状态码为 TEMPORARILY_UNAVAILABLE; 如果服务正常有效, 状态码为 AVAILABLE。 extras: 一组可选参数, 其包含 provider 的特定状态。下面列出一组共用的键值对, 其实任何键的 provider 都需要提供的值	在 Provider 的状态在可用、暂时不可用和无服务三个状态直接切换时被调用
--	---	--

Location 类, 表示某一特定时间地理位置的相关信息。其方法如表 9-2 所示。

表 9-2 LocationListener 类的方法

方法	说明
getLatitude()	取得纬度——Latitude
getLongitude()	取得经度——Longitude
getAccuracy()	取得精确度——Accuracy
getAltitude()	取得高度——Altitude
getTime()	取得时间——Time
getSpeed()	取得速度——Speed
getBearing()	取得方位——Bearing

19.1.2 GPS 定位步骤

为确定设备所在位置, 需要进行以下步骤的操作。

提供位置服务, 需要获得 LocationManager 对象。使用 LOCATION_SERVICE 参数调用

android.app.Activity.getSystemService()函数获取一个 LocationManager 实例。

android.app.Activity.getSystemService()函数代码如代码清单 9-1 所示。

```
代码清单 9-1 getSystemService()
//Context.LOCATION_SERVICE 指明获取的服务是位置服务
String serviceString = Context.LOCATION_SERVICE;

//根据服务名称获取 Android 提供的系统级服务
LocationManager locationManager =
    (LocationManager) getSystemService(serviceString);
```

这里介绍一下 Android 支持的系统级服务有哪些, 如表 9-3 所示。



表 9-3 Android 支持的系统级服务表

Context 类的静态常量	值	返回对象	说明
LOCATION_SERVICE	location	LocationManager	控制位置等设备的更新
WINDOW_SERVICE	window	WindowManager	最顶层的窗口管理器
LAYOUT_INFLATER_SERVICE	layout_inflater	LayoutInflater	将 XML 资源实例化为 View
POWER_SERVICE	power	PowerManager	电源管理
ALARM_SERVICE	alarm	AlarmManager	在指定时间接受 Intent
NOTIFICATION_SERVICE	notification	NotificationManager	后台事件通知
KEYGUARD_SERVICE	keyguard	KeyguardManager	锁定或解锁键盘
SEARCH_SERVICE	search	SearchManager	访问系统的搜索服务
VIBRATOR_SERVICE	vibrator	Vibrator	访问支持振动的硬件
CONNECTIVITY_SERVICE	connection	ConnectivityManager	网络连接管理
WIFI_SERVICE	wifi	WifiManager	WiFi 连接管理
INPUT_METHOD_SERVICE	input_method	InputMethodManager	输入法管理



选择 LocationManager 的定位方法。在获取到 LocationManager 后，还需要指定 LocationManager 的定位方法，然后才能够调用 LocationManager，LocationManager 支持的定位方法有以下两种。

GPS 定位：可以提供更加精确的位置信息，但定位速度和质量受到卫星数量和环境情况的影响。

网络定位：提供的位置信息精度差，但速度较 GPS 定位快。

LocationManager 支持的定位方法比较，如表 9-4 所示。

表 9-4 LocationManager 支持的定位方法比较

LocationManager 类的静态常量	值	说明
GPS_PROVIDER	gps	使用 GPS 定位，利用卫星提供精确的位置信息，需要 android.permissions.ACCESS_FINE_LOCATION 用户权限
NETWORK_PROVIDER	network	使用网络定位，利用基站或 WiFi 提供近似的位置信息，需要具有如下权限： android.permission.ACCESS_COARSE_LOCATION 或 android.permission.ACCESS_FINE_LOCATION.

在指定 LocationManager 的定位方法后，则可以调用 getLastKnownLocation() 方法获取当前的位置信息。

以使用 GPS 定位为例，获取位置信息的代码如代码清单 9-2 所示。

代码清单 9-2 GPS 定位获取位置信息

```
String provider = LocationManager.GPS_PROVIDER;
Location location = locationManager.getLastKnownLocation(provider);
```

在上述代码中，第 2 行返回的 Location 对象中，包含了可以确定位置的信息，如经度、纬度和速度等。然后通过调用 Location 中的 getLatitude() 和 getLongitude() 方法可以分别获取位置信息中的纬度和经度，示例代码如代码清单 9-3 所示。

代码清单 9-3 获取经纬度信息

```
double lat = location.getLatitude();
double lng = location.getLongitude();
```

实现 LocationListener 类。代码如代码清单 9-4 所示。

代码清单 9-4 LocationListener

```
LocationListener locationManager = new LocationListener(){
    //在设备的位置改变时被调用
    public void onLocationChanged(Location location) {
    }
    //在用户禁用具有定位功能的硬件时被调用
    public void onProviderDisabled(String provider) {
```



```

    }

    //在用户启用具有定位功能的硬件时被调用
    public void onProviderEnabled(String provider) {
    }

    //在提供定位功能的硬件的状态改变时被调用，如从不可获取位置信息状态到可以获取位置信息
    的状态，反之亦然
    public void onStatusChanged(String provider, int status, Bundle extras) {
    }
};

```

利用 requestLocationUpdates() 方法启动位置信息的接收。

LocationManager 提供了一种便捷、高效的位置监视方法 requestLocationUpdates()，可以根据位置的
距离变化和时间间隔设定产生位置改变事件的条件，这样可以避免因微小的距离变化而产生大量的位置
改变事件。

LocationManager 中设定监听位置变化的代码如代码清单 9-5 所示。

代码清单 9-5 监听位置变化

```
locationManager.requestLocationUpdates(provider, 2000, 10, locationListener);
```

其中，第 1 个参数是定位的方法，GPS 定位或网络定位；第 2 个参数是产生位置改变事件的时间间
隔，单位为微秒；第 3 个参数是距离条件，单位是米；第 4 个参数是回调函数，在满足条件后的位置改
变事件的处理函数。代码将产生位置改变事件的条件设定为距离改变 10 米，时间间隔为 2 秒。

为了使 GPS 定位功能生效，还需要在 AndroidManifest.xml 文件中加入用户许可。没有这些权限，应
用程序在运行时是无法获取到位置更新的。实现代码如下所示：

代码清单 9-6 AndroidManifest.xml

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
```

此处需注意，若使用 GPS_PROVIDER，则使用 ACCESS_FINE_LOCATION 权限；若使用
NETWORK_PROVIDER，则使用 ACCESS_FINE_LOCATION 和 ACCESS_COARSE_
LOCATION 权限。

位置服务一般都需要使用设备上的硬件，最理想的调试方式是将程序上传到物理设备上运行，但在
没有物理设备的情况下，也可以使用 Android 模拟器提供的虚拟方式模拟设备的位置变化，调试具有位置
服务的应用程序。

首先打开 DDMS 中的模拟器控制，在 Location Controls 中的 Longitude 和 Latitude 部分输入设备当前
的经度和纬度，然后单击“Send”按钮，就将虚拟的位置信息发送到 Android 模拟器中，如图 9-1 所示。

在程序运行过程中，可以在模拟器控制器中改变经度和纬度坐标值，程序在检测到位置的变化后，
会将最新的位置信息显示在界面上。

CurrentLocationDemo 是一个提供位置服务的基本示例，提供了显示当前位置新的功能，并能够监视
设备的位置变化。



CurrentLocationDemo 示例中 LocationBasedServiceDemo.java 文件的完整代码如代码清单 9-7 所示。

代码清单 9-7 LocationBasedServiceDemo.java

```
package cn.com.farsight.LocationBasedServiceDemo;

import android.app.Activity;
import android.content.Context;
import android.os.Bundle;
import android.widget.TextView;
import android.location.Location;
import android.location.LocationListener;
import android.location.LocationManager;

public class LocationBasedServiceDemo extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);

        String serviceString = Context.LOCATION_SERVICE;
        LocationManager locationManager =
            (LocationManager) getSystemService(serviceString);
        String provider = LocationManager.GPS_PROVIDER;
        Location location = locationManager.getLastKnownLocation(provider);
        getLocationInfo(location);
        locationManager.requestLocationUpdates(provider, 2000, 0, locationListener);
    }

    private void getLocationInfo(Location location){
        String latLongInfo;
        TextView locationText = (TextView)findViewById(R.id.txtshow);
        if (location != null){
            double lat = location.getLatitude();
            double lng = location.getLongitude();
            latLongInfo = "Lat: " + lat + "\nLong: " + lng;
        }
    }
}
```



```
        else{
            latLongInfo = "No location found";
        }
        locationText.setText("Your Current Position is:\n" + latLongInfo);
    }

    private final LocationListener locationListener = new LocationListener(){
        @Override
        public void onLocationChanged(Location location) {
            getLocationInfo(location);
        }
        @Override
        public void onProviderDisabled(String provider) {
            getLocationInfo(null);
        }
        @Override
        public void onProviderEnabled(String provider) {
            getLocationInfo(null);
        }
        @Override
        public void onStatusChanged(String provider, int status, Bundle extras) {
        }
    };
}
```




华清远见
dev.havi.com



第 20 章 Android 中的图形图像

在上一章的学习中，主要了解了 Android 平台下定位与位置方面的知识，包括位置服务的相关概念，本章来了解一下 Android 系统的图形处理能力。

Android 系统拥有强大的图形处理能力，它并没有使用 Java 的图形处理类来进行对 2D 图形的处理，而是自定义了一系列 2D 图形处理类，这些类分别位于 `android.graphics`、`android.graphics.drawable.shapes` 和 `android.view.animation` 包中。对于 3D 图形的处理，在 Android 系统中，Android 提供了高效的 3D 图形处理技术，这些类分别位于 `javax.microedition.khronos.openhles` 和 `android.opengl` 包中。

Android 系统的图形处理基本可以分为两类。一类是静态图形处理，也就是处理一些不经常变化的图片，例如，一些图标、Logo、动画等，针对这些图形的处理，一般是将图形作为资源文件添加到工程（`res/drawable/`、`res/anim`）当中，然后通过各种 `Drawable` 类来处理使用。另一类是动态图形处理，也就是处理一些经常变化的图片（这些变化包括大小、位置、形状等的变化），需要不断地重复绘制，例如，游戏中的各种场景、任务和道具等。

本章主要学习在 Android 中访问图片、动画的实现、动态图形绘制的基本思路、图形特效的实现。

20.1 在 Android 中访问图片

图片在 Android 系统中随处可见。例如，列表图标、快捷方式图标、图片按钮等。在 Android 中我们通过使用 `Drawable` 类来完成对图片的操作。`Drawable` 类有很多个子类，如用来操作位图的 `BitmapDrawable` 类、用来操作颜色的 `ColorDrawable` 类，以及用来操作各种形状的 `ShapeDrawable` 类。

可以通过以下 3 种方法来实例化 `Drawable` 对象：一是使用保存在工程中的一个图片文件；二是使用 XML 文件定义 `Drawable` 属性；三是构造方法实例化，这种方法在实际开发中一般用不到。

20.1.1 使用图片文件创建 `Drawable` 对象

最简单的一种方式是在工程的资源文件下面保存图片，该图片会被 Eclipse 自动在 `R` 类中创建引用，然后可以通过 `R.drawable.my_image` 使用该图片对象。下面看一个实例，实例步骤说明如下。

将一个 `test.jpg` 图片文件放置在工程的资源文件夹下。

创建布局文件 `main.xml` 并在其中添加一个 `ImageView` 组件。

创建 `Activity`，并实例化 `ImageView` 组件对象。

调用 `ImageView` 的 `setImageResource()` 方法，引用资源 `id`。

`Activity` 代码如代码清单 10-1 所示。

代码清单 10-1 MainActivity.java

```
package cn.com.farsight.drawable;
```

```
import android.app.Activity;
```

```
import android.os.Bundle;
```

```
import android.widget.ImageView;
```



```
public class MainActivity extends Activity {  
    //声明图片视图 ImageView  
    private ImageView myImageView;  
  
    /** Called when the activity is first created. */  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.main);  
        //通过 findViewById 方法获得 ImageView  
        myImageView = (ImageView) findViewById(R.id.ImageView01);  
        //为 ImageView 设置图片资源  
        myImageView.setImageResource(R.drawable.test);  
    }  
}
```

布局文件 main.xml 代码如代码清单 10-2 所示。

代码清单 10-2 main.xml

```
<?xml version="1.0" encoding="utf-8"?>  
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"  
    android:orientation="vertical"  
    android:layout_width="fill_parent"  
    android:layout_height="fill_parent"  
    >  
    <TextView  
        android:layout_width="fill_parent"  
        android:layout_height="wrap_content"  
        android:text="Drawable Test"/>  
    <ImageView  
        android:id="@+id/ImageView01"  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
    ></ImageView>  
</LinearLayout>
```

程序运行结果如图 10-1 所示。

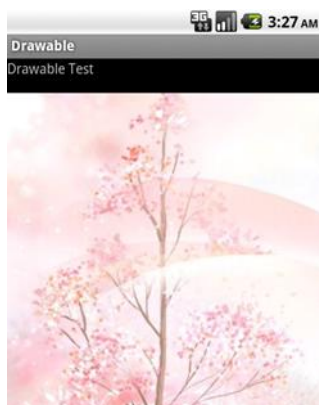


图 10-1 从资源文件加载图片

20.1.2 使用 XML 文件定义 Drawable 属性

在布局文件中定义 Drawable 属性。例如，可以在布局文件中定义图片按钮的图片及应用程序的图标等。

代码清单 10-3 所示的代码演示了如何在 AndroidManifest.xml 配置文件中引用资源图标。

代码清单 10-3 AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="cn.com.farsight.drawable"
    android:versionCode="1"
    android:versionName="1.0">

    <application android:icon="@drawable/icon"
        android:label="@string/app_name">
        <activity android:name=".MainActivity"
            android:label="@string/app_name">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

    <uses-sdk android:minSdkVersion="3"/>
</manifest>
```

在上一个例子中也可以通过这种方式来配置 ImageView 的图片资源。代码如代码清单 10-4 所示。



代码清单 10-4 main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    >

    <TextView
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="Drawable Test"/>

    <ImageView
        android:id="@+id/ImageView01"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:src="@drawable/test"
    ></ImageView>

</LinearLayout>
```

20.1.3 Bitmap 和 BitmapFactory

除了以上提到的两种图片保存以外，还可以将图片文件保存在 SDCard 中，那么此时如何访问保存在 SDCard 中图片文件呢？在这种情况下，可以通过 Bitmap 和 BitmapFactory 两个类来读取文件。下面的代码演示了如何从 SDCard 中读取图片文件并将其设置为壁纸。

程序步骤说明如下：

在 SDCard 中添加一个名称为 wallpaper.jpg 的图片文件。

创建 Activity。

在 Activity 的 onCreate() 方法中通过 BitmapFactory 的 decodeFile() 方法传递文件路径，获取 Bitmap 对象。

调用 setWallpaper() 方法设置桌面。

程序代码如代码清单 10-5 所示。

代码清单 10-5 MainActivity.java

```
package cn.com.farsight.drawable;

import java.io.IOException;
import android.app.Activity;
```



```
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.os.Bundle;

public class MainActivity extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
        //图片路径
        String path="/sdcard/wallpaper.jpg";
        //通过 BitmapFactory 获取 Bitmap 实例
        Bitmap bm = BitmapFactory.decodeFile(path);
        try{
            //设置桌面
            setWallpaper(bm);
        }catch(IOException e){
            e.printStackTrace();
        }
    }
}
```

程序运行后桌面被改变，如图 10-2 所示。



图 10-2 Bitmap 和 BitmapFactory 的使用

20.2 Android 中的动画

在 Android 系统中提供了两种动画实现方式：一种是 Tween 动画，通过视图组件移动、放大、缩小，以及产生透明度的变化等来实现动画；另一种是 Frame 动画，这是一种传统的通过顺序播放排列好的图片来实现的动画方法，类似电影。



20.2.1 Tween 动画

Tween 动画能完成一系列诸如位置、尺寸、透明度和旋转等简单的变化。例如，对程序中的 ImageView 组件，可以通过 Tween 动画使该视图实现放大、缩小、旋转、渐变等变化。下面就对 Tween 动画类进行简单介绍。

Tween 动画类位于 `android.view.animation` 包中，该包中包含了一些常用的动画实现类。

Animation: 动画抽象类，其他几个实现类继承该类。

ScaleAnimation: 控制尺寸变化的动画类。

AlphaAnimation: 控制透明度变化的动画类。

TranslateAnimation: 控制位置变化的动画类。

AnimationSet: 定义动画属性集合类。

AnimationUtils: 动画工具类。

总体来讲，在 Android 系统中，Tween 动画有 4 种表现方式：渐变、缩放、平移，以及旋转。对这 4 种方式的实现方法详细说明如表 10-1 所示。

表 10-1 Tween 动画实现表

名称	实现类	常用构造方法	参数说明
Alpha (渐变动画)	AlphaAnimation	AlphaAnimation(float fromAlpha, float toAlpha)	fromAlpha: 动画开始透明度 toAlpha: 动画结束透明度 (取值范围 0.0~1.0)
Scale (尺寸变化动画)	ScaleAnimation	public ScaleAnimation (float fromX, float toX, float fromY, float toY, int pivotXType, float pivotXValue, int pivotYType, float pivotYValue)	fromX: 起始 X 坐标上的伸缩尺寸 toX: 结束 X 坐标上的伸缩尺寸 fromY: 起始 Y 坐标上的伸缩尺寸 toY: 结束 Y 坐标上的伸缩尺寸 pivotXType: X 坐标伸缩模式 (取值有 Animation.ABSOLUTE、Animation.RELATIVE_TO_SELF、Animation.RELATIVE_TO_PARENT) pivotXValue: 相对于 X 坐标伸缩值 pivotYType: Y 坐标伸缩模式 (取值有 Animation.ABSOLUTE、Animation.RELATIVE_TO_SELF、Animation.RELATIVE_TO_PARENT)



			) pivotYValue: 相对于 Y 坐标伸缩值
Translate (位置变化动画)	TranslateAnimation	public TranslateAnimation (float fromXDelta, float toXDelta, float fromYDelta, float toYDelta)	fromXDelta: 起始 X 坐标 toXDelta: 结束 X 坐标 fromYDelta: 起始 Y 坐标 toYDelta: 结束 Y 坐标
Rotate (旋转动画)	RotateAnimation	public RotateAnimation (float fromDegrees, float toDegrees, int pivotXType, float pivotXValue, int pivotYType, float pivotYValue)	fromDegrees: 旋转开始角度 toDegrees: 旋转结束角度 pivotXType: X 坐标伸缩模式 (取值有 Animation.ABSOLUTE、 Animation.RELATIVE_TO_SELF、 Animation.RELATIVE_TO_PARENT) ● pivotXValue: 相对于 X 坐标伸缩值 pivotYType: Y 坐标伸缩模式 (取值有 Animation.ABSOLUTE、 Animation.RELATIVE_TO_SELF、 Animation.RELATIVE_TO_PARENT.) pivotYValue: 相对于 Y 坐标伸缩值

Tween 动画可以通过两种编码方式来实现：一种是直接通过硬编码的方式在程序代码中实现；另一种是在配置文件中定义。对于 Android 系统来讲，建议使用配置文件的方法，这种方式可扩展性较高，便于维护。

Tween 动画可以通过两种编码方式来实现：一种是直接通过硬编码的方式在程序代码中实现；另一种是在配置文件中定义。对于 Android 系统来讲，建议使用配置文件的方法，这种方式可扩展性较高，便于维护。

创建一个 Android 工程 “GraphicAnimation”。

在工程的 res//drawable// 目录中添加一个 girl.png 图片资源。

在该工程的 res//layout// 目录中创建一个 main.xml 布局文件，在该布局文件中添加 4 个 Button 组件和一个 ImageView 组件，并为 ImageView 组件设置资源引用 android:src 属性。代码如代码清单 10-6 所示。



代码清单 10-6 main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    >
    <ImageView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/ImageView01"
        android:src="@drawable/girl"
    ></ImageView>

    <Button
        android:layout_height="wrap_content"
        android:id="@+id/Button01"
        android:text="Test Scale..."
        android:layout_width="wrap_content"></Button>

    <Button
        android:layout_height="wrap_content"
        android:id="@+id/Button02"
        android:text="Test Alpha..."
        android:layout_width="wrap_content"></Button>

    <Button
        android:layout_height="wrap_content"
        android:id="@+id/Button03"
        android:text="Test Translate..."
        android:layout_width="wrap_content"></Button>

    <Button
        android:layout_height="wrap_content"
        android:id="@+id/Button04"
        android:text="Test Ratate..."
        android:layout_width="wrap_content"></Button>
</LinearLayout>
```



在 MainActivity 类的顶部声明使用到的 Button 和 ImageView 组件，在 onCreate()方法中实例化。代码如代码清单 10-7 所示。

代码清单 10-7 MainActivity.java

```
package cn.com.farsight.graphicanimation;

import android.app.Activity;
import android.os.Bundle;
import android.widget.Button;
import android.widget.ImageView;

public class MainActivity extends Activity {
    //声明使用到的 Button 视图组件
    private Button b1,b2,b3,b4;
    //声明使用到的 ImageView 组件
    private ImageView girlImage;

    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
        //实例化视图组件
        girlImage = (ImageView) findViewById(R.id.ImageView01);
        b1 = (Button) findViewById(R.id.Button01);
        b2 = (Button) findViewById(R.id.Button02);
        b3 = (Button) findViewById(R.id.Button03);
        b4 = (Button) findViewById(R.id.Button04);
    }
}
```

分别为 4 个按钮添加单击事件监听器，在监听器的方法中通过构造方法定义不同的动画，调用动画的 setDuration()方法设置动画持续事件，调用 ImageView 的 startAnimation()方法开始动画。代码如代码清单 10-8 所示。

代码清单 10-8 添加事件监听

```
//为按钮添加监听事件
b1.setOnClickListener(new OnClickListener){
```



```
@Override
public void onClick(View v) {
    // TODO Auto-generated method stub
    //创建 Scale（尺寸）变化动画
    Animation scaleAnimation = new ScaleAnimation(0f, 1f, 0f, 1f,
        Animation.RELATIVE_TO_SELF, 0.5f,
        Animation.RELATIVE_TO_SELF, 0.5f);

    //设置动画持续时间
    scaleAnimation.setDuration(3000);
    //开始动画
    girlImage.startAnimation(scaleAnimation);
}

});
b2.setOnClickListener(new OnClickListener(){
    @Override
    public void onClick(View v) {
        // TODO Auto-generated method stub
        //创建 Alpha（渐变）动画
        Animation alphaAnimation = new AlphaAnimation(0.1f,0.1f);
        //设置动画持续时间
        alphaAnimation.setDuration(3000);
        //开始动画
        girlImage.startAnimation(alphaAnimation);
    }

});
b3.setOnClickListener(new OnClickListener(){
    @Override
    public void onClick(View v) {
        // TODO Auto-generated method stub
        //创建 Translate（位置变化）动画
        Animation translateAnimation = new TranslateAnimation(10,100,10,100);
        //设置动画持续时间
        translateAnimation.setDuration(3000);
        //开始动画
        girlImage.startAnimation(translateAnimation);
    }
});
```



```

    }
});
b4.setOnClickListener(new OnClickListener(){
    @Override
    public void onClick(View v) {
        // TODO Auto-generated method stub
        //创建 Rotate（旋转）动画
        Animation rotateAnimation = new RotateAnimation(0f,+360f,
            Animation.RELATIVE_TO_SELF,0.5f,
            Animation.RELATIVE_TO_SELF,0.5f);

        //设置动画持续时间
        rotateAnimation.setDuration(3000);
        //开始动画
        girlImage.startAnimation(rotateAnimation);
    }
});

```

以上讲述的是通过硬编码的方式来实现的 4 种不同的动画效果，其实 Android 系统更推荐我们使用 XML 配置文件的方式来实现，该 XML 文件位于工程的 `res/anim/` 目录中。XML 文件的格式如代码清单 10-9 所示。

代码清单 10-9 XML 配置文件格式

```

<set android:shareInterpolator=boolean>
    <alpha android:fromAlpha=float
        android:toAlpha=float>
    <scale android:fromXScale=float
        android:toXScale=float
        android:fromYScale=float
        android:toYScale=float
        android:pivotX=string
        android:pivotY=string>
    <translate android:fromX=string
        android:toX=string
        android:fromY=string
        android:toY=string>
    <rotate android:fromDegrees=float
        android:toDegrees=float

```



```
        android:pivotX=string
        android:pivotY=string>
    <interpolator tag>
    <set>
</set>
```

如上所示，必须要有一个 set 根元素，根元素里边定义不同的动画，如渐变（Alpha）、尺寸（Scale）、位置变化（Translate）和旋转（Rotate）等。

下面通过实例来演示如何使用 XML 配置文件实现各种动画效果。基本思路是在工程的 res/anim/目录下定义不同的动画配置文件。通过调用 AnimationUtils.loadAnimation()方法获得动画实例，调用视图组件的 startAnimation()方法开始动画。实例具体步骤说明如下。

创建一个 Android 工程 “Graphic_Animation”。

在工程的 res/drawable/目录中添加一个 girl.png 图片资源。

在该工程的 res/layout/目录中创建一个 main.xml 布局文件，在该布局文件中添加 4 个 Button 组件和一个 ImageView 组件，并为 ImageView 组件设置资源引用 android:src 属性。代码如代码清单 10-10 所示。

代码清单 10-10 main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    >
    <ImageView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/ImageView01"
        android:src="@drawable/girl"
    ></ImageView>

    <Button
        android:layout_height="wrap_content"
        android:id="@+id/Button01"
        android:text="Test Scale..."
        android:layout_width="wrap_content"></Button>

    <Button
```



```
        android:layout_height="wrap_content"
        android:id="@+id/Button02"
        android:text="Test Alpha..."
        android:layout_width="wrap_content"></Button>
<Button
    android:layout_height="wrap_content"
    android:id="@+id/Button03"
    android:text="Test Translate..."
    android:layout_width="wrap_content"></Button>
<Button
    android:layout_height="wrap_content"
    android:id="@+id/Button04"
    android:text="Test Rotate..."
    android:layout_width="wrap_content"></Button>
</LinearLayout>
```

在 MainActivity 类的顶部声明使用到的 Button 和 ImageView 组件，在 onCreate() 方法中将其实例化。

代码清单 10-11 MainActivity.java

```
package cn.com.farsight.graphicanimation;

import android.app.Activity;
import android.os.Bundle;
import android.view.View;
import android.view.View.OnClickListener;
import android.view.animation.AlphaAnimation;
import android.view.animation.Animation;
import android.view.animation.RotateAnimation;
import android.view.animation.ScaleAnimation;
import android.view.animation.TranslateAnimation;
import android.widget.Button;
import android.widget.ImageView;

public class MainActivity extends Activity {
    //声明使用到的 Button 视图组件
    private Button b1,b2,b3,b4;
    //声明使用到的 ImageView 组件
```



```
private ImageView girlImage;

/** Called when the activity is first created. */
@Override
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main);
    //实例化视图组件
    girlImage = (ImageView) findViewById(R.id.ImageView01);
    b1 = (Button) findViewById(R.id.Button01);
    b2 = (Button) findViewById(R.id.Button02);
    b3 = (Button) findViewById(R.id.Button03);
    b4 = (Button) findViewById(R.id.Button04);
}
}
```

在工程的 res/anim/目录下创建各种动画的 XML 配置文件，代码如代码清单 10-12~10-15 所示。

Alpha:

代码清单 10-12 Alpha

```
<set xmlns:android="http://schemas.android.com/apk/res/android">
    <alpha android:fromAlpha="0.1"
        android:toAlpha="1.0"
        android:duration="5000"
    />
</set>
```

Translate:

代码清单 10-13 Translate

```
<set xmlns:android="http://schemas.android.com/apk/res/android">
    <translate
        android:fromXDelta="10"
        android:toXDelta="100"
        android:fromYDelta="10"
        android:toYDelta="100">
    </translate>
</set>
```

Scale:



代码清单 10-14 Scale

```
<set xmlns:android="http://schemas.android.com/apk/res/android">
    <Scale
        android:fromXScale="0.0"
        android:toXScale="1.0"
        android:fromYScale="0.0"
        android:toYScale="1.0"
        android:pivotX="50%"
        android:pivotY="50%"
        android:duration="5000">
    </scale>
</set>
```

Rotate:

代码清单 10-15 Rotate

```
<set xmlns:android="http://schemas.android.com/apk/res/android">
    <rotate
        android:fromDegrees="0"
        android:toDegrees="-180"
        android:pivotX="50%"
        android:pivotY="50%"
        android:duration="5000">
    </rotate>
</set>
```

分别为 4 个按钮添加单击事件监听器，在监听器的方法中通过 AnimationUtils.loadAnimation()方法定义不同的动画，调用 ImageView 的 startAnimation()方法开始动画。

代码清单 10-16 添加事件监听

```
//为按钮添加监听事件
b1.setOnClickListener(new OnClickListener(){
    @Override
    public void onClick(View v) {
        //创建 Scale（尺寸）变化动画
        Animation scaleAnimation = AnimationUtils.loadAnimation
            (MainActivity.this,R.anim.my_scale);

        //开始动画
        girlImage.startAnimation(scaleAnimation);
    }
});
```




```
    }  
});  
b2.setOnClickListener(new OnClickListener(){  
    @Override  
    public void onClick(View v) {  
        //创建 Alpha（渐变）动画  
        Animation alphaAnimation = AnimationUtils.loadAnimation  
            (MainActivity.this,R.anim.my_alpha);  
  
        //开始动画  
        girlImage.startAnimation(alphaAnimation);  
    }  
});  
b3.setOnClickListener(new OnClickListener(){  
    @Override  
    public void onClick(View v) {  
        //创建 Translate（位置变化）动画  
        Animation translateAnimation = AnimationUtils.loadAnimation  
            (MainActivity.this,R.anim.my_translate);  
  
        //开始动画  
        girlImage.startAnimation(translateAnimation);  
    }  
});  
b4.setOnClickListener(new OnClickListener(){  
    @Override  
    public void onClick(View v) {  
        //创建 Rotate（旋转）动画  
        Animation rotateAnimation = AnimationUtils.loadAnimation  
            (MainActivity.this,R.anim.my_rotate);  
  
        //开始动画  
        girlImage.startAnimation(rotateAnimation);  
    }  
});
```

20.2.2 Frame 动画

Frame 动画是顺序播放图片来产生动画效果的，类似电影。例如，要实现一个人物走动的动画，可以通过三张图片来实现，第一张两脚都着地，第二张左脚着地，第三张右脚着地。然后，按顺序播放就实



现任务行走的动画效果了。

Frame 动画是通过 `AnimationDrawable` 类实现的，该类中的两个重要方法是 `start()` 和 `stop()`，分别用来开始和停止动画。动画一般通过 XML 配置文件来进行配置，在工程的 `res/anim/` 目录下创建一个 XML 配置文件，该配置文件中有一个 `<animation-list>` 根元素和若干个 `<item>` 子元素。

下面通过一个实例来演示 Frame 动画的实现。该实例通过顺序播放 6 张图片来实现一个人跳起来的动画效果。6 张静态图片如图 10-7 所示。



图 10-7 Frame 动画静态图片

实例步骤说明如下：

创建一个工程 “Graphic_Animation2”。

在该工程的 `res/drawable/` 目录下添加上述 6 张资源文件。

在该工程的 `res/anim/` 目录下创建一个 XML 动画文件，代码如代码清单 10-17 所示。

代码清单 10-17 jump.xml

```
<animation-list xmlns:android="http://schemas.android.com/apk/res/android"
    android:oneshot="true">
    <item android:drawable="@drawable/p01" android:duration="500"/>
    <item android:drawable="@drawable/p02" android:duration="500"/>
    <item android:drawable="@drawable/p03" android:duration="500"/>
    <item android:drawable="@drawable/p04" android:duration="500"/>
    <item android:drawable="@drawable/p05" android:duration="500"/>
    <item android:drawable="@drawable/p06" android:duration="500"/>
</animation-list>
```

在工程的 `res/layout/` 目录下创建一个 `main.xml` 布局文件，在该布局文件中添加两个 `Button`：一个用来开始动画，另一个用来停止动画。添加一个 `ImageView` 显示动画。设置 `ImageView` 的背景色 `android:background="@anim/dance"`。代码如代码清单 10-18 所示。

代码清单 10-18 jump.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent">
```



```
>
<ImageView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:background="@anim/jump"
    android:id="@+id/ImageView01"></ImageView>

<Button
    android:layout_height="wrap_content"
    android:text="Start..."
    android:layout_width="wrap_content"
    android:id="@+id/Button01"></Button>

<Button
    android:layout_height="wrap_content"
    android:text="Stop..."
    android:layout_width="wrap_content"
    android:id="@+id/Button02"></Button>

</LinearLayout>
```

在 MainActivity 类中声明 Button、ImageView 和 AnimationDrawable 实例，在 onCreate() 方法中将其实例化。获得 ImageView 视图的背景色，并转化为 AnimationDrawable 对象。为按钮添加单击事件监听器，在事件方法中分别开始动画和结束动画。

代码清单 10-19 MainActivity.java

```
package cn.com.farsight.graphic_Animation;

import android.app.Activity;
import android.graphics.drawable.AnimationDrawable;
import android.os.Bundle;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.ImageView;

public class MainActivity extends Activity {
    //声明使用到的 Button 视图组件
    private Button b1,b2;
```



```
//声明使用到的 ImageView 组件
private ImageView myImage;

//声明 AnimationDrawable
private AnimationDrawable jumpAnimation;

/** Called when the activity is first created. */
@Override
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main);
    //实例化视图组件
    myImage=(ImageView) findViewById(R.id.ImageView01);
    b1 = (Button) findViewById(R.id.Button01);
    b2 = (Button) findViewById(R.id.Button02);

    //获得背景色，并转换为 AnimationDrawable 对象
    jumpAnimation = (AnimationDrawable) myImage.getBackground();

    //为按钮添加监听事件
    b1.setOnClickListener(new OnClickListener(){
        @Override
        public void onClick(View v) {
            // TODO Auto-generated method stub
            //开始动画
            jumpAnimation.start();
        }
    });
    b2.setOnClickListener(new OnClickListener(){
        @Override
        public void onClick(View v) {
            // TODO Auto-generated method stub
            //停止动画
            jumpAnimation.stop();
        }
    });
};
```



```
}
```

```
}
```

20.3 动态图形绘制

20.3.1 动态图形绘制类简介

如何动态绘制图形呢？首先来看在 Android 中如何绘制图形。其实，在 Android 中涉及的这些工具类都很形象。想一下真正地画一张画需要哪些东西呢？首先需要一张画布，这里就是 Android 中的 Canvas；其次还需要画笔，这里就是 Android 中的 Paint；再次需要不同的颜色，这里就是 Android 中的 Color。接下来如果要画线还需要连接路径，这里就是 Android 中的 Path。还可以借助工具直接画出各种图形如圆、椭圆、矩形等，这里就是 Android 中的 ShapeDrawable 类，当然它还有很多子类，例如，OvalShape（椭圆）、RectShape（矩形）等。

Canvas

Canvas 就是我们所说的画布，位于 android.graphics 包中，提供了一些画各种图形的方法，例如，矩形、圆、椭圆等。该类的详细方法如表 10-2 所示。

表 10-2 Canvas 常用方法

方法名称	方法描述
drawText(String text,float x,float y,Paint paint)	以(x,y)为起始坐标，使用 paint 绘制文本
drawPoint(float x,float y,Paint paint)	在坐标(x,y)上使用 paint 画点
drawLine(float startX,float startY,float stopX,float stopY,Paint paint)	以(startX, startY)为起始坐标点，(stopX, stopY)为终止坐标点，使用 paint 画线
drawCircle(float cx,float cy,float radius,Paint paint)	以(cx, cy)为原点，radius 为半径，使用 paint 画圆
drawOval(RectF oval,Paint paint)	使用 paint 画矩形 oval 的内切椭圆
DrawRect(RectF rect,Paint paint)	使用 paint 画矩形 rect
drawRoundRect(RectF rect,float rx,float ry,Paint paint)	画圆角矩形
clipRect(float left,float top,float right,float bottom)	剪辑矩形
clipRegion(Region region)	剪辑区域

Paint

Paint 就是涂料的意思，用来描述图形的颜色和风格，如线宽、颜色、字体等信息。Paint 位于 android.graphics 包中，该类的详细方法如表 10-3 所示。

表 10-3 Paint 常用方法

方法名称	方法描述
------	------



Paint()	构造方法，使用默认设置
setColor(int color)	设置颜色

续表

方法名称	方法描述
setStrokeWidth(float width)	设置线宽
setTextAlign(Paint.Align align)	设置文字对齐
setTextSize(float textSize)	设置文字尺寸
setShader(Shader shader)	设置渐变
setAlpha(int a)	设置 Alpha 值
reset()	复位 Paint 默认设置

Color

Color 类定义了一些颜色变量和一些创建颜色的方法。颜色的定义一般使用 RGB 三原色定义。Color 位于 android.graphics 包中，其常用属性和方法如表 10-4 所示。

表 10-4 Color 常用属性和方法

属性名称	方法描述
BALCK	黑色
BLUE	蓝色
CYAN	青色
DKGRAY	深灰色
GRAY	灰色
GREEN	绿色
LTGRAY	浅灰色
MAGENTA	紫色
RED	红色
TRANSPARENT	透明
WHITE	白色
YELLOW	黄色

Path

当我们想要画一个圆的时候，我们只需要制定圆心（点）和半径就可以了。那么，如果要画一个梯形呢？这里我们需要有点和连线。Path 一般用来从一点移动到另一个点连线。Path 位于 android.graphics 包中。详细方法如表 10-5 所示。

表 10-5 Path 常用方法

方法名称	方法描述
lineTo(float x,float y)	从最后点到指定点划线



moveTo(float x,float y)	移动到指定点
reset()	复位

20.3.2 动态图形绘制的基本思路

动态图形绘制的基本思路是，创建一个类继承 View 类（或者继承 SurfaceView 类）。覆盖 onCreate() 方法，使用 Canvas 对象在界面上绘制不同的图形，使用 invalidate() 方法刷新界面。下面通过一个弹球实例来讲述动态图形绘制的基本思路。该实例是在界面上动态绘制一个小球，当小球触顶或者触底时自动改变方向继续运行。实例步骤说明如下：

创建一个 Android 工程，入口 Activity 的名称为 MainActivity。MainActivity 代码如代码清单 10-20 所示。

代码清单 10-20 MainActivity.java

```
package cn.com.farsight.draw;

import android.app.Activity;
import android.content.Context;
import android.graphics.Canvas;
import android.graphics.Color;
import android.graphics.Paint;
import android.os.Bundle;
import android.os.Handler;
import android.os.Message;
import android.util.AttributeSet;
import android.view.View;

public class MainActivity extends Activity {
    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
    }
}
```

在 MainActivity 类中创建一个 MyView 内部类，该类实现 Runnable 接口支持多线程

在 onDraw() 方法中，定义 Paint 画笔并设置画笔颜色，使用 Canvas 的 drawCircle() 方法画圆。

定义一个 update() 方法，用于更新 Y 坐标。

定义一个消息处理器类 RefreshHandler，它继承 Handler 并覆盖 handleMessage() 方法，在该方法中处理消息。



在线程的 run()方法中设置并发送消息。

在构造方法中启动线程。代码如代码清单 10-21 所示。

代码清单 10-21 内部类 MyView

```
class MyView extends View implements Runnable{
    //图形当前坐标
    private int x=20,y=20;
    //构造方法
    public MyView(Context context, AttributeSet attrs) {
        super(context, attrs);
        // TODO Auto-generated constructor stub
        //获得焦点
        setFocusable(true);
        //启动线程
        new Thread(this).start();
    }

    @Override
    public void run() {
        // TODO Auto-generated method stub
        RefreshHandler mRedrawHandler = new RefreshHandler();
        while(!Thread.currentThread().isInterrupted()){
            //通过发送消息更新界面
            Message m = new Message();
            m.what = 0x101;
            mRedrawHandler.sendMessage(m);
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        }
    }

    @Override
```




```
protected void onDraw(Canvas canvas) {  
    // TODO Auto-generated method stub  
    super.onDraw(canvas);  
    //实例化画笔  
    Paint p = new Paint();  
    //设置画笔颜色  
    p.setColor(Color.GREEN);  
    //画图  
    canvas.drawCircle(x, y, 10, p);  
}  
  
//更新界面处理器  
class RefreshHandler extends Handler{  
    @Override  
    public void handleMessage(Message msg) {  
        // TODO Auto-generated method stub  
        if(msg.what==0x101){  
            MyView.this.update();  
            MyView.this.invalidate();  
        }  
        super.handleMessage(msg);  
    }  
}  
  
//更新坐标  
private void update(){  
    int h=getHeight();  
    y+=5;  
    if(y>=h){  
        y=20;  
    }  
}  
}
```

(3) 在 MainActivity 类的 onCreate()方法中实例化 MyView 类，并将其设置为 Activity 的内容视图。

代码清单 10-22 onCreate()

```
public void onCreate(Bundle savedInstanceState) {
```



```
super.onCreate(savedInstanceState);  
MyView v = new MyView(this,null);  
setContentView(v);  
}
```

20.3.3 绘制几何图形

通过上述 4 个类的学习，我们可以使用它们来画出一些简单的几何图形了。下面的实例演示了如何使用这些类来画出一些常见的几何图形。程序步骤说明如下：

创建一个 Activity；

创建 Activity 的内部类 MyView 继承 View；

覆盖 View 的 onDraw()方法；

在 onDraw()方法中创建 Paint 对象、Path 对象，设置 Canvas 属性画出各种图形；

在 Activity 的 onCreate()方法中实例化 MyView，调用 Activity 的 setContentView()方法，将其设置为当前视图。

程序代码如代码清单 10-23 所示。

代码清单 10-23 MainActivity.java

```
package cn.com.farsight.drawDemo;  
  
import android.app.Activity;  
import android.content.Context;  
import android.graphics.Canvas;  
import android.graphics.Color;  
import android.graphics.LinearGradient;  
import android.graphics.Paint;  
import android.graphics.Path;  
import android.graphics.RectF;  
import android.graphics.Shader;  
import android.os.Bundle;  
import android.view.View;  
  
public class MainActivity extends Activity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(new MyView(this));  
    }  
}
```



```
//自定义视图类
private class MyView extends View{
    public MyView(Context context){
        super(context);
    }

    @Override
    protected void onDraw(Canvas canvas) {
        super.onDraw(canvas);
        //设置 Canvas 颜色
        canvas.drawColor(Color.WHITE);

        //实例化 Paint
        Paint paint = new Paint();
        paint.setAntiAlias(true);
        //设置画笔颜色
        paint.setColor(Color.RED);
        //设置画笔样式
        paint.setStyle(Paint.Style.STROKE);
        //设置画笔粗细
        paint.setStrokeWidth(3);
        //画圆
        canvas.drawCircle(40, 40, 30, paint);
        //画矩形（正方形）
        canvas.drawRect(10, 90, 70,150, paint);
        //画矩形
        canvas.drawRect(10, 170, 70,200, paint);
        //声明矩形
        RectF re = new RectF(10,220,70,250);
        //画椭圆
        canvas.drawOval(re, paint);

        //画三角形
        //实例化路径
```



```
Path path = new Path();
//移动到指定点
path.moveTo(10, 330);
//画线
path.lineTo(70,330);
path.lineTo(40, 270);
//关闭路径
path.close();
//画路径
canvas.drawPath(path, paint);

//画梯形
//实例化路径
Path path1 = new Path();
//移动到指定点
path1.moveTo(10,410);
//画线
path1.lineTo(70, 410);
path1.lineTo(55, 350);
path1.lineTo(25, 350);
//关闭路径
path1.close();
//画路径
canvas.drawPath(path1, paint);

//设置样式
paint.setStyle(Paint.Style.FILL);
//设置颜色
paint.setColor(Color.BLUE);
//画圆
canvas.drawCircle(120, 40, 30, paint);
//画矩形（正方形）
canvas.drawRect(90, 90, 150, 150, paint);
//画矩形
```



```
canvas.drawRect(90, 170, 150, 200, paint);
```

```
//实例化矩形
```

```
RectF re2 = new RectF(90,220,150,250);
```

```
//画椭圆
```

```
canvas.drawOval(re2, paint);
```

```
//画三角形
```

```
//实例化路径
```

```
Path path2 = new Path();
```

```
//移动到指定点
```

```
path2.moveTo(90, 330);
```

```
//画线
```

```
path2.lineTo(150, 330);
```

```
path2.lineTo(120, 270);
```

```
//关闭路径
```

```
path2.close();
```

```
//画路径
```

```
canvas.drawPath(path2, paint);
```

```
//画梯形
```

```
//实例化路径
```

```
Path path3 = new Path();
```

```
//移动到指定点
```

```
path3.moveTo(90, 410);
```

```
//画线
```

```
path3.lineTo(150, 410);
```

```
path3.lineTo(135, 350);
```

```
path3.lineTo(105, 350);
```

```
//关闭路径
```

```
path3.close();
```

```
//画路径
```

```
canvas.drawPath(path3, paint);
```

```
//线性渲染
```



```
Shader mShader = new LinearGradient(0,0,100,100,new int[]
    {Color.RED,Color.GREEN,Color.BLUE,Color.YELLOW},
    null,Shader.TileMode.REPEAT);
//为 paint 设置线性渲染
paint.setShader(mShader);
//画圆
canvas.drawCircle(200, 40, 30, paint);
//画矩形（正方形）
canvas.drawRect(170, 90, 230, 150, paint);
//画矩形
canvas.drawRect(170, 170, 230, 200, paint);
//矩形
RectF re3 = new RectF(170,220,230,250);
//画椭圆
canvas.drawOval(re3, paint);

//画三角形
//实例化路径
Path path4 = new Path();
//移动到指定点
path4.moveTo(170, 330);
//画线
path4.lineTo(230, 330);
path4.lineTo(200, 270);
//关闭路径
path4.close();
//画路径
canvas.drawPath(path4, paint);

//画梯形
//实例化路径
Path path5 = new Path();
//移动到指定点
path5.moveTo(170, 410);
//画线
```



```

        path5.lineTo(230, 410);
        path5.lineTo(215, 350);
        path5.lineTo(185, 350);
        //关闭路径
        path5.close();
        //画路径
        canvas.drawPath(path5, paint);

        //设置文本大小
        paint.setTextSize(24);
        //写文本
        canvas.drawText(getResources().getString(R.string.str_text1), 240, 50, paint);
        canvas.drawText(getResources().getString(R.string.str_text2), 240, 120, paint);
        canvas.drawText(getResources().getString(R.string.str_text3), 240, 190, paint);
        canvas.drawText(getResources().getString(R.string.str_text4), 240, 250, paint);
        canvas.drawText(getResources().getString(R.string.str_text5), 240, 320, paint);
        canvas.drawText(getResources().getString(R.string.str_text6), 240, 390, paint);
    }

}
}

```

20.4 图形特效

20.4.1 使用 **Matrix** 实现旋转、缩放和平移

在 Android 图形 API 中提供了一个 **Matrix** 矩形类，该类具有一个 3×3 的矩阵坐标。通过该类可以实现图形的旋转、平移和缩放。该类的详细方法如表 10-6 所示。

表 10-6 **Matrix** 常用方法

方法名称	方法描述
<code>void reset()</code>	重置一个 matrix 对象
<code>void set(Matrix src)</code>	复制一个源矩阵，与构造方法 <code>Matrix(Matrix src)</code> 一样
<code>boolean isIdentity()</code>	返回这个矩阵是否定义
<code>void setRotate(float degrees)</code>	指定一个角度以 (0, 0) 为坐标进行旋转



void setRotate(float degrees, float px, float py)	指定一个角度以 (px,py) 为坐标进行旋转
void serScale(float sx, float sy)	缩放处理
void serScale(float sx, float sy, float px, float py)	以坐标 (px,py) 进行缩放
void setTranslate(float dx, float dy)	平移
void setSkew(float kx, float ky)	倾斜处理
void setSkew(float kx, float ky, float px, float py)	以坐标(px,py)进行倾斜

下面通过一个实例来演示 Matrix 的具体应用，在本实例中我们自定义一个 View 类，在该类中拥有一个 Bitmap 和 Matrix 实例，Bitmap 实例从系统资源加载一张图片，覆盖 View 类的 onDraw() 方法，在该方法中通过 reset() 方法初始化 Matrix，并设置其旋转或缩放属性，使用 Canvas 的 drawBitmap() 方法将 Bitmap 重新绘制在视图中。通过键盘事件 onKeyDown() 实现旋转属性和缩放属性的改变，调用 postInvalidate() 方法重新绘制 Bitmap。实例步骤说明如下：

创建一个 Android 工程 “GraphicMatrix”，入口 Activity 的名称为 MainActivity。

代码清单 10-24 MainActivity.java

```
package cn.com.farsight.graphicMatrix;

import android.app.Activity;
import android.content.Context;
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.graphics.Canvas;
import android.graphics.Matrix;
import android.os.Bundle;
import android.view.KeyEvent;
import android.view.View;

public class MainActivity extends Activity {
    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
    }
}
```




在该工程的 res/drawable/目录下添加一张图片资源 girl.jpg。

在 MainActivity 中定义一个内部类 MyView 继承 View。在该类的顶部声明使用到的变量，在构造方法中初始化变量，覆盖 onDraw()方法和 onKeyDown()方法。

代码清单 10-25 MyView

//自定义视图类

```
class MyView extends View{
```

```
    //位图实例
```

```
    private Bitmap bm;
```

```
    //Matrix 实例
```

```
    private Matrix matrix = new Matrix();
```

```
    //旋转角度
```

```
    private float angle = 0.0f;
```

```
    //位图的宽和高
```

```
    private int w,h;
```

```
    //缩放比例
```

```
    private float scale = 1.0f;
```

```
    //判断缩放还是旋转
```

```
    private boolean isScale = false;
```

```
    //构造方法
```

```
    public MyView(Context context) {
```

```
        super(context);
```

```
        // TODO Auto-generated constructor stub
```

```
        //获得位图
```

```
        bm = BitmapFactory.decodeResource(this.getResources(),  
                                           R.drawable.girl);
```

```
        //获得位图宽
```

```
        w = bm.getWidth();
```

```
        //获得位图高
```

```
        h = bm.getHeight();
```

```
        //使当前视图获得焦点
```

```
        this.setFocusable(true);
```

```
    }
```

```
@Override
```



```
protected void onDraw(Canvas canvas) {
    // TODO Auto-generated method stub
    super.onDraw(canvas);
    //重置 Matrix
    matrix.reset();
    if(!isScale){
        //旋转 Matrix
        matrix.setRotate(angle);
    }else{
        //缩放 Matrix
        matrix.setScale(scale, scale);
    }
    //根据原始位图和 Matrix 创建新视图
    Bitmap bm2 = Bitmap.createBitmap(bm, 0, 0, w, h,matrix, true);
    //绘制新视图
    canvas.drawBitmap(bm2, matrix, null);
}

@Override
public boolean onKeyDown(int keyCode, KeyEvent event) {
    // TODO Auto-generated method stub
    //向左旋转
    if(keyCode == KeyEvent.KEYCODE_DPAD_LEFT){
        isScale = false;
        angle++;
        postInvalidate();
    }
    //向右旋转
    if(keyCode == KeyEvent.KEYCODE_DPAD_RIGHT){
        isScale = false;
        angle--;
        postInvalidate();
    }
    //放大
    if(keyCode == KeyEvent.KEYCODE_DPAD_UP){
```



```

        isScale =true;
        if(scale < 2.0)
            scale += 0.1;
        postInvalidate();
    }
    //缩小
    if(keyCode == KeyEvent.KEYCODE_DPAD_DOWN){
        isScale = true;
        if(scale > 0.5)
            scale -= 0.1;
        postInvalidate();
    }
    return super.onKeyDown(keyCode, event);
}
}

```

在 MainActivity 的 onCreate()方法中，实例化 MyView 并将其设置为当前 Activity 的视图内容。

代码清单 10-26 onCreate()

```

public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    MyView myView = new MyView(MainActivity.this);
    //设置当前视图布局
    setContentView(myView);
}

```

20.4.2 使用 Shader 类渲染图形

经常会看到一些屏保或者音乐播放器的界面中有一些非常绚丽的图形，这些其实是在一些原有图形的基础上进行渲染的结果。Android 系统中提供了 Shader 渲染类来实现这一功能。Shader 是一个抽象父类，其子类有很多个，如 BitmapShader（位图渲染）、ComposeShader（混合渲染）、LinearGradient（线性渲染）、RadialGradient（光束渲染）和 SweepGradient（梯度渲染）等。通过 Paint 对象的 paint.setShader（shader）方法来使用 Shader。

下面通过一个实例来演示不同 Shader 的渲染效果。该实例自定义一个 MyView 继承 View 类。在该类中构建不同的 Shader 对象，在 onDraw()方法中绘制各种渲染后的图形。实例步骤说明如下：

创建一个 Android 工程“GraphicShader”，入口 Activity 类的名称为 MainActivity。

代码清单 10-27 MainActivity.java

```

package cn.com.farsight.graphicShader;

```



```
import android.app.Activity;
import android.content.Context;
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.graphics.BitmapShader;
import android.graphics.Canvas;
import android.graphics.Color;
import android.graphics.ComposeShader;
import android.graphics.LinearGradient;
import android.graphics.Paint;
import android.graphics.PorterDuff;
import android.graphics.RadialGradient;
import android.graphics.Shader;
import android.graphics.SweepGradient;
import android.graphics.PorterDuff.Mode;
import android.graphics.Shader.TileMode;
import android.os.Bundle;
import android.view.KeyEvent;
import android.view.View;

public class MainActivity extends Activity {
    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
    }
}
```

在工程 res/drawable/目录下添加一个 girl.jpg 资源文件。

在 MainActivity 中自定义一个 MyView 类，该类继承 View 类并覆盖其 onDraw() 方法。

在该类的顶部声明各种渲染对象及 Paint 对象。

在构造方法中进行实例化。

在 onDraw 方法中根据当前的渲染绘图。

在 onKeyDown() 方法中通过键盘按键改变渲染对象。

代码清单 10-28 MyView

//自定义视图类



```
class MyView extends View{
//声明 Bitmap 对象
private Bitmap bm;
//声明位图渲染对象
private Shader bitmapShader;
//声明线性渲染对象
private Shader linearGradient;
//声明光束渲染对象
private Shader radialGradient;
//声明梯度渲染对象
private Shader sweepGradient;
//声明混合渲染对象
private Shader composeShader;
//声明画笔
private Paint paint;
//声明颜色数组
private int[] colors;
private boolean isFirst = true;

public MyView(Context context) {
    super(context);
    // TODO Auto-generated constructor stub
    //获取 Bitmap 实例
    bm = BitmapFactory.decodeResource(getResources(), R.drawable.girl);
    //实例化画笔
    paint = new Paint();
    colors = new int[]{Color.RED,Color.GREEN,Color.BLUE};
    //实例化位图渲染对象，x 坐标方向重复图形，y 坐标方向镜像图形
    bitmapShader = new BitmapShader(bm,
                                    TileMode.REPEAT,TileMode.MIRROR);
    //实例化线性渲染
    linearGradient = new LinearGradient(0,0,100,100,
                                       colors,null,TileMode.REPEAT);
    //实例化光束渲染
    radialGradient = new RadialGradient(100, 100, 80,
```



```
        colors, null, TileMode.REPEAT);

//实例化梯度渲染
sweepGradient = new SweepGradient(200,200,colors,null);

//实例化混合渲染
composeShader = new ComposeShader(linearGradient, radialGradient,
                                   PorterDuff.Mode.DARKEN);

//使其获得焦点
setFocusable(true);
}

@Override
protected void onDraw(Canvas canvas) {
    // TODO Auto-generated method stub
    super.onDraw(canvas);
    if(isFirst){
        //写字，用来提示用户操作
        String content ="按上/下/左/右/中间键测试";
        paint.setColor(Color.BLUE);
        canvas.drawText(content, 0, content.length()-1, 20, 20, paint);
    }else{
        //全屏画矩形
        canvas.drawRect(0, 0, getWidth(), getHeight(), paint);
    }
}

@Override
public boolean onKeyDown(int keyCode, KeyEvent event) {
    // TODO Auto-generated method stub
    isFirst = false;
    if(keyCode == KeyEvent.KEYCODE_DPAD_UP){
        //将画笔渲染设置为位图渲染
        paint.setShader(bitmapShader);
    }
    if(keyCode == KeyEvent.KEYCODE_DPAD_DOWN){
        //将画笔渲染设置为线性渲染
    }
}
```



```
        paint.setShader(linearGradient);
    }
    if(keyCode == KeyEvent.KEYCODE_DPAD_LEFT){
        //将画笔渲染设置为光束渲染
        paint.setShader(radialGradient);
    }
    if(keyCode == KeyEvent.KEYCODE_DPAD_RIGHT){
        //将画笔渲染设置为梯度渲染
        paint.setShader(sweepGradient);
    }
    if(keyCode == KeyEvent.KEYCODE_DPAD_CENTER){
        //将画笔渲染设置为混合渲染
        paint.setShader(composeShader);
    }
    //重绘界面
    postInvalidate();
    return super.onKeyDown(keyCode, event);
}
}
```

在 MainActivity 类的 onCreate()方法中，实例化 MyView 并将其设置为 Activity 的内容视图。

代码清单 10-29 MainActivity.java

```
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    MyView v = new MyView(this);
    setContentView(v);
}
```

20.5 本章小结

本章主要对 Android 平台中的图形图像进行了学习，内容包括：在 Android 中访问图片、Tween 动画和 Frame 动画的实现、动态图片绘制的基本思路、动态图形类的学习、绘制集合图形，以及使用 Matrix 和 Shader 实现图形特效。



华清远见
dev.havi.com



第 21 章 Android NDK 开发

在本章节的学习中，将了解 Android NDK 的用途、其编译环境的安装配置方法及其开发方法。

21.1 Android NDK 简介

Android NDK (Android Native Development Kit) 是一系列的开发工具，是允许程序开发人员在 Android 应用程序中嵌入 C/C++ 语言编写的非托管代码。

Android NDK 的版本是 r7，集成了交叉编译器，支持 ARMv5TE 处理器指令集、JNI 接口和一些稳定的库文件。由于 Android NDK 仅支持 Android SDK 1.5 以上版本，因此 1.0 和 1.1 版本的应用程序不能够使用 Android NDK。Android NDK 提供一系列的说明文档、示例代码和开发工具，指导程序开发人员使用 C/C++ 语言进行库文件开发，并提供便捷工具将库文件打包到 apk 文件中。

Android 虚拟机允许应用程序源代码通过 JNI 调用在本地实现的源代码。

应用程序将声明一个或多个用 “native” 关键字的方法，用来指明它们是通过本地代码实现的，如代码清单 11-1 所示。

代码清单 11-1 native 用法

```
native byte[] loadFile(String filePath)
```

必须提供包含实现这些方法的共享库（就是 .so），将共享库打包到你的应用程序包 apk 中，这些库文件必须根据标准的 Unix 约定来命名为 lib<something>.so，并且是需要包含一个标准的 JNI 的接口，例如，libFileLoader.so。

应用程序必须明确地装载这些库文件（.so 文件），比如，在程序的开始装载它，只需要简单地添加几句源代码（如代码清单 11-2 所示）。

代码清单 11-2 装载库文件

```
static {  
    System.loadLibrary("FileLoader");  
}
```

Android NDK 对于 Android SDK 只是个组件，它可以使生成 JNI 兼容的共享库可以在大于 Android 1.5 平台的 ARM CPU 上运行，将生成的共享库复制到合适的程序工程路径的位置上，以保证它们自动地添加到 apk 包中（并且签名的）。

而且，Android NDK 还提供了：

一组交叉编译链（编译器、链接器等）来生成可以在 Linux，OS X 和 Windows（用 Cygwin）运行的二进制文件。

一组与由 Android 平台提供的稳定的本地 API 列表的头文件，它们在 docs/STABLE-APIS.html 中有说明。

一个编译系统（build system）需要开发者写一个非常短的编译文件（build files）去描述哪个源代码需要编译，并且怎样编译。编译系统可以解决所有的 toolchain/platform/CPU/ABI 细节的问题。并且，较新的 NDK 版本中还添加了更多的可以不用改变开发者的编译文件的情况下的 toolchains、platforms、系统



接口。

但同时 Android NDK 也存在一些不足。

NDK 并不是一个可以编写通用的源代码并且可以在 Android 设备上运行的方法，应用程序还是需要使用 Java 程序，适当地处理系统事件来避免“应用程序没有反应”的对话框或者处理 Android 应用程序的生命周期。注意：可以适当地在源代码中写一个复杂的应用程序，用于启动/停止一个小型的“应用程序包”。

NDK 在 Android 平台仅仅提供了有限的本地 API 和库文件的支持的系统头文件，然而一个标准的 Android 系统镜像包括许多本地共享库，这些都应该被考虑在更新和发行版本中可以彻底改变的实现细节。

21.2 NDK 编译环境

Android NDK 编译环境支持 Windows XP、Linux 和 MacOS，本章节主要介绍 Windows 系统的编译环境配置方法。

首先，下载 Android NDK 的安装包。

在 Google 的官方网站下载 Android NDK 的安装包，下载地址是 <http://developer.android.com/sdk/ndk/index.html>，打开下载页面后，我们看到最新的 NDK 版本为 r7，选择的下载文件为 [android-ndk-r7-windows.zip](#)，下载至本地计算机，如图 11-1 所示。

Download the Android NDK

The Android NDK is a companion tool to the Android SDK that lets you build performance-critical portions of your apps in native code, handle user input, use hardware sensors, access application resources, and more, when programming in C or C++ and they still run inside of a virtual machine on the device. The fundamental Android application model does not

Using native code does not result in an automatic performance increase, but always increases application complexity. If you are using the Android APIs, you probably do not need the NDK. Read [What is the NDK?](#) for more information about what the NDK offers and when to use it.

The NDK is designed for use only in conjunction with the Android SDK. If you have not already installed and setup the Android SDK, see [Installing the Android SDK](#).

Platform	Package	Size	MD5 Checksum
Windows	android-ndk-r7-windows.zip	81270552 bytes	55483482cf2b75e8dd1a5d9a7caeb6e5
Mac OS X (intel)	android-ndk-r7-darwin-x86.tar.bz2	71262092 bytes	817ca5675a1dd44078098e43070f19b6
Linux 32/64-bit (x86)	android-ndk-r7-linux-x86.tar.bz2	64884365 bytes	bf15e6b47bf50824c4b96849bf003ca3

图 11-1 android-ndk-r7-windows.zip

将下载的 ZIP 文件解压到用户的 Android 开发目录中，例如，将 Android NDK 解压到 E:\Android 目录中，ZIP 文件中包含一层目录，因此 Android NDK 的最终路径为 E:\Android\android-ndk。

在 Windows 环境下，NDK 的编译有两种方式：一种是通过脚本直接编译，这种情况下，我们只需切换到目标目录下，运行 ndk-build 脚本即可完成编译；另一种即是通过 Eclipse 来完成编译，由于使用 Eclipse 进行开发，如果可以直接通过 Eclipse 对 NDK 自动进行编译，对开发来说是十分方便的。那么如何在 Eclipse 中自动编译 NDK 呢？只需为工程添加一个 Builders 即可。



其次，我们为工程添加 Builders。

对于 NDK 为 r7 之前的版本，Eclipse 需要配置的 builder，其实是执行 Cygwin，然后传递 ndk-build 作为参数，因此，必须要安装 Cygwin 才能使用 NDK。而对于在 NDKr7 开始，Google 的 Windows 版的 NDK 提供了一个 ndk-build.cmd 的脚本，这样，就可以直接利用这个脚本编译，而不需要使用 Cygwin 了。

这里先来讲解一下使用 NDKr7 在 Windows 下配置自动编译的 Builders 的过程，对于 Linux，只需要修改 ndk-build.cmd 为 ndk-build 即可。

测试准备：新建一个 Eclipse 工程，在工程目录下新建 jni 文件夹，其中写好必要的 NDK 编译的代码等。

选择 Project→Properties→Builders→New 命令，新建一个 Builders。

在新建 Builder 中进行配置，如图 11-2 所示。

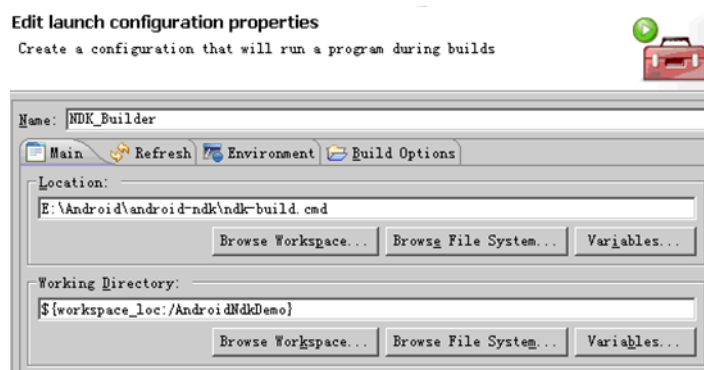


图 11-2 Builder 配置

如图 11-2 所示，其中 Location 为 ndk-build.cmd 的路径，即上一步中解压后的 Android NDK 的最终路径，例如，E:\Android\android-ndk\ndk-build.cmd；Working Directory 为当前的工程。这里的\${}的表达式是使用了 Eclipse 的环境变量，可以使用 Variables...按钮找到一些 Eclipse 的环境变量。

对于 Linux 上的配置，和这里的配置一样，只是把 Location 的地址改为 ndk-build 即可。

切换到 Refresh 选项卡进行配置。

在 Refresh 选项卡中，选择 Refresh resources upon completion 复选框，如图 11-3 所示。

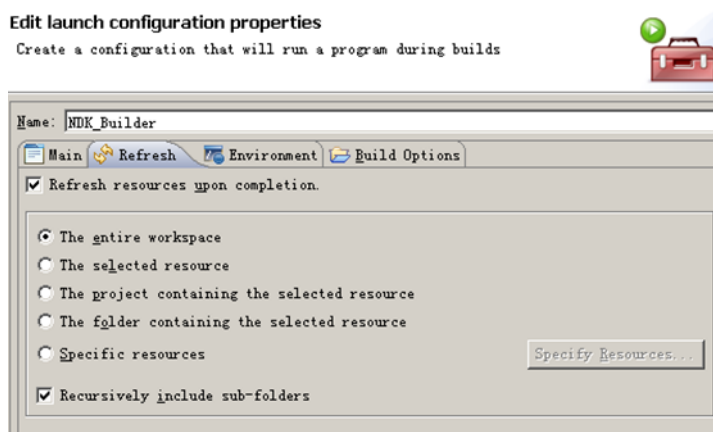


图 11-3 Builder 配置 Refresh 选项卡

切换到 Build Options 选项卡进行配置。

在 Build Options 选项卡中，配置如图 11-4 所示。

在配置 Build Options 的时候，单击“Specify Resources”按钮，选择资源目录，勾选你的项目目录下的 jni 文件夹，到此为止，Eclipse 就能自动调用 NDK 脚本编译 jni 下的 C/C++代码了。

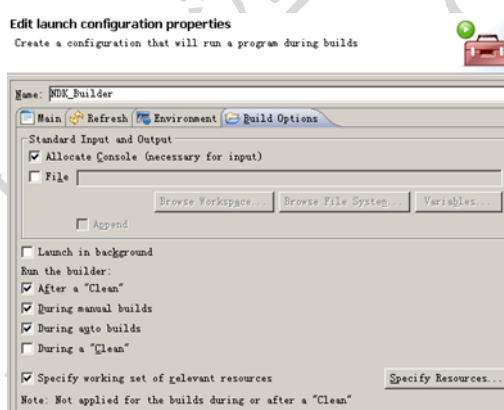


图 11-4 Builder 配置 Build Options 选项卡

至此，Android NDK 的编译环境已经安装配置完毕。下面我们来了解一下 Android NDK 的目录结构。

android-ndk 目录如图 11-5 所示。

build	文件夹
docs	文件夹
platforms	文件夹
prebuilt	文件夹
samples	文件夹
sources	文件夹
tests	文件夹
toolchains	文件夹
documentation.html	1 KB HTML Document
GNUmakefile	2 KB 文件
ndk-build	3 KB 文件
ndk-build.cmd	1 KB Windows NT 命令
ndk-gdb	21 KB 文件
ndk-stack.exe	184 KB 应用程序
README.TXT	2 KB 文本文档
RELEASE.TXT	1 KB 文本文档

图 11-5 android-ndk 目录

其中，samples 目录是 NDK 自带的示例目录；build 目录保存了交叉编译工具、编译脚本和配置文件；docs 目录是帮助文档的保存目录。

docs 目录中的帮助文件说明如表 11-1 所示。

表 11-1 docs 目录中的帮助文件说明表

文件名	说明
INSTALL.TXT	NDK 的安装与配置说明文档
OVERVIEW.TXT	NDK 的用途和使用范围的说明文档
ANDROID-MK.TXT	Android.mk 文件的说明文档，Android.mk 文件定义了需要编译的 C/C++源代码
APPLICATION-MK.TXT	Application.mk 文件的说明文档，Application.mk 文件定义了 Android 工程需要调用的 C/C++源代码
HOWTO.TXT	关于 NDK 开发的一般信息
SYSTEM-ISSUES.TXT	使用 NDK 开发时所需要注意的问题
STABLE-APIS.TXT	NDK 头文件所支持的稳定的 API 类表

platforms 对应于 Android 的各个不同版本。

sources 目录是 C/C++源代码文件的保存目录，其下的 hello-jni 和 tow-lib 子目录，分别保存了 NDK 自带示例所需要的 C/C++源代码文件。

GNUmakefile 文件、README.TXT 文件和 RELEASE.TXT 分别是 make 工具的配置文件、NDK 的说明文件以及 NDK 的版本说明。

ndk-build.cmd/ ndk-build 编译。

Android NDK 自带两个示例 hello-jni 和 tow-lib。Hello-jni 是一个非常简单的例子，非托管代码实现了一个可以返回字符串的共享库，Android 工程调用这个共享库获取字符串，然后显示在用户界面上；tow-lib 是稍微复杂一些的例子，使用非托管代码实现了一个数学运算的共享库，Android 工程动态加载这个共享库，并调用其中的函数，函数功能是通过使用静态库实现的。



21.3 NDK 开发示例

下面通过 AndroidNdkDemo 示例说明如何进行 Android NDK 开发。

AndroidNdkDemo 是一个进行加法运算的示例，程序会随机产生两个整数，然后调用 C 语言开发的共享库对这两个整数进行加法运算，最后将运算结果显示在用户界面上。AndroidNdkDemo 示例的界面如图 11-6 所示。

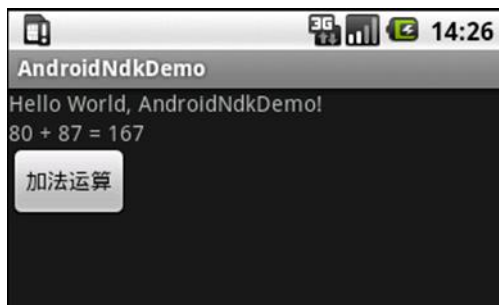


图 11-6 AndroidNdkDemo 示例的界面

进行 Android NDK 开发一般要经过以下的步骤。

建立 Android 工程

在 project 目录中建立 Android 工程 AndroidNdkDemo，如图 11-7 所示。建立完成后，在工程目录下新建 jni 文件夹。

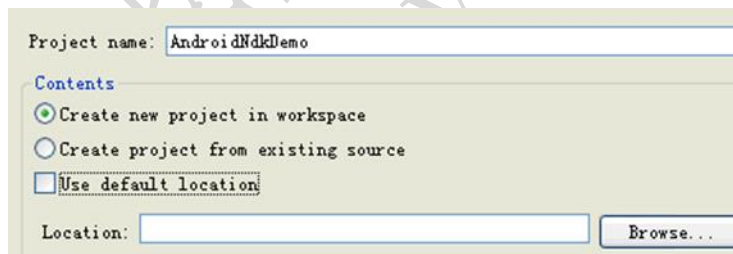


图 11-7 建立 Android 工程

在建立 AndroidNdkDemo 工程后，修改 main.xml 文件，添加一个 id 为 display 的 TextView 和一个 id 为 add_btn 的 Button 按钮。

程序中的生产随机数和调用的代码在 AndroidNdkDemo.java 文件中，代码清单 11-3 所示。是 AndroidNdkDemo.java 文件的核心代码。

代码清单 11-3 AndroidNdkDemo.java 核心代码

```
public class AndroidNdkDemo extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
    }
}
```



```
final TextView displayLable = (TextView)findViewById(R.id.display);
Button btn = (Button)findViewById(R.id.add_btn);
btn.setOnClickListener(new View.OnClickListener(){
    @Override
    public void onClick(View v) {
        double randomDouble = Math.random();
        long x = Math.round(randomDouble*100);
        randomDouble = Math.random();
        long y = Math.round(randomDouble*100);

        //System.loadLibrary("AndroidNdkDemo");//16
        long z = add(x, y); //17
        String msg = x+" + "+y+" = "+z;
        displayLable.setText(msg);
    }
});
}

//public native long add(long x, long y);//23

public long add(long x, long y){//25
    return x+y;
}
}
```

上面的代码有一个 NDK 开发的小技巧，在开发 C/C++ 的共享库前，可以使用具有相同和相近功能的 Java 函数进行替代。

在代码第 17 行本应该调用共享库的 add() 函数，但为了便于开发和调试，在代码第 25 行到第 27 行，使用 Java 代码开发了一个功能相同的 add() 函数，这样即使在没有完成 C/C++ 的共享库开发前，也可以对这个 Android 工程进行界面部分的调试。

第 16 行和第 23 行注释掉的代码，就是在 C/C++ 的共享库开发完毕后需要使用的代码，其中第 16 行是动态加载共享库的代码，加载的共享库名称为 AndroidNdkDemo；第 23 行用来声明共享库的函数，使用 C/C++ 开发的共享库必须有同名的函数。在共享库开发完毕后，取消第 16 行和第 23 行代码的注释，并注释掉第 25 行到第 27 行代码，这样程序就可以正常调用共享库内的函数进行加法运算。

建立 C 源代码文件

在 jni 目录下建立 C 源代码文件 AndroidNdkDemo.c。AndroidNdkDemo.c 文件的作用是实现两个整数加法运算功能，全部代码如代码清单 11-4 所示。



代码清单 11-4 AndroidNdkDemo.c

```
#include <jni.h>

jlong Java_cn_com_farsight_AndroidNdkDemo_AndroidNdkDemo_add(
    JNIEnv* env, jobject this, jlong x, jlong y )
{
    return x+y;
}
```

第 1 行代码引入的是 JNI（Java Native Interface）的头文件；第 3 行代码是函数名称，jlong 表示 Java 长型整数，Java_cn_com_farsight_AndroidNdkDemo_AndroidNdkDemo_add 的构成为 Java_<包名称>_<类>_<函数>，其中<函数>的名称和参数要与 AndroidNdkDemo.

java 文件定义的函数一致，AndroidNdkDemo.java 文件定义的函数为 public native long add(long x, long y)；第 5 行代码用来返回加法运算结果。

建立 Android.mk 文件

在 AndroidNdkDemo 示例的 jni 目录下建立 Android.mk。Android.mk 是为 NDK 编译系统准备的脚本文件，用来描述模块需要编译 C/C++ 文件的信息。

下面来分析 AndroidNdkDemo 示例的 jni 目录下的 Android.mk 文件，Android.mk 文件的代码如代码清单 11-5 所示。

代码清单 11-5 Android.mk

```
LOCAL_PATH := $(call my-dir) //1

include $(CLEAR_VARS) //3

LOCAL_MODULE := AndroidNdkDemo //5
LOCAL_SRC_FILES := AndroidNdkDemo.c //6

include $(BUILD_SHARED_LIBRARY) //8
```

每个 Android.mk 文件都必须以第 1 行代码开始，变量 LOCAL_PATH 用来定义需要编译的 C/C++ 源代码的位置，my-dir 由 NDK 编译系统提供，表示当前目录的位置。代码第 3 行的 include \$(CLEAR_VARS) 表示清空所有以 LOCAL_ 开始的变量，例如 LOCAL_MODULE、LOCAL_SRC_FILES、LOCAL_STATIC_LIBRARIES 等，但第 1 行定义的 LOCAL_PATH 不在清空的范围内。因为所有的脚本都将粘贴到同一个 GNU Make 的执行上下文中，而且所有变量都是全局变量，因此必须在每次使用前清空所有以前用过的变量。

第 5 行代码变量 LOCAL_MODULE 用来声明模块名称，模块名称必须唯一，而且中间不能够存在空



格。NKD 编译系统将会在模块名称前自动添加 lib 前缀，然后生产 so 文件。这里的模块名称为 AndroidNdkDemo，生产的共享库文件名为 libaAndroidNdkDemo.so。但需要注意的是，如果程序开发人员使用具有 lib 前缀的模块名称，NKD 编译系统将不再添加前缀，例如，模块名称为 libsub，生产的共享库文件名为 libsub.so。

第 6 行代码中的变量 LOCAL_SRC_FILES 表示编译模块所需要使用的 C/C++ 文件列表，但不需要给出头文件的列表，因为 NKD 编译系统会自动计算依赖关系。AndroidNdkDemo 模块仅需要一个 C 文件，文件名为 AndroidNdkDemo.c。默认情况下，结尾名为.c 的文件是 C 语言源文件，结尾名为.cpp 的文件是 C++ 语言源文件。

第 8 行代码 include \$(BUILD_SHARED_LIBRARY) 表示 NKD 编译系统构建共享库，如果变量 BUILD_SHARED_LIBRARY 更改为 BUILD_STATIC_LIBRARY，则表示需要 NKD 编译系统构建静态库。

编译共享库模块

按照 11.2 节中介绍的方法添加 Builders，并进行配置。

运行 Android 程序

在运行 AndroidNdkDemo 示例程序前，务必将 AndroidNdkDemo.java 文件中第 16 行和第 23 行的注释取消，并注释掉第 25 行到第 27 行代码。

21.4 本章小结

本章主要学习了 Android NDK 开发方面的相关知识，了解了 Android NDK 的用途、其编译环境的安装配置方法及开发方法。



第 22 章 Android 网络编程

手机的确给我们带来了不少方便，1995 年问世的第一代（1G）模拟制式手机还只能进行语音通话；1996~1997 年出现的第二代（2G）GSM、CDMA 等数字制式手机便增加了接收数据（如接收电子邮件或网页）等功能；而 2009 年诞生的 3G，其功能更是让人眼花缭乱，比如高速的无线宽带上网、视频通话、无线搜索、手机音乐、手机网游等。无线网络发展的速度也非常迅猛。有了无线网络的支持，我们就不必受时间和空间的限制，可以随时随地进行数据交换，浏览 Internet，第一时间获知新闻。随着人们知识水平的提高，生活圈也越来越广，人们更需要网络的帮助来处理一些事务，比如手机炒股、手机证券、手机银行、手机地图等，而在 Android 中，掌握了网络通信便可以开发出这些优秀的网络应用。

22.1 Android 网络通信基础

在前面的学习中，已经了解到 Android 出色的界面设计及强大的数据管理功能。除了这些强大的功能外，Android 在网络通信方面也非常优秀。例如，我们可以很轻松地使用 Android 自带的浏览器来访问网页，如图 13-1 所示。



图 13-1 Android 自带浏览器访问网页

Android 基于 Linux 内核，它包含一组优秀的联网功能，这些只是 Android 自带的一些功能，它们是开源软件，大家可以拿来学习研究。目前，Android 平台有 3 种网络接口可以使用，它们分别是：`java.net.*`（标准 Java 接口）、`org.apache`（Apache 接口）和 `android.net.*`（Android 网络接口）。下面分别来简单介绍一下这些接口的功能及其作用。

22.1.1 标准 Java 接口

`java.net.*`（标准 Java 接口）提供与联网有关的类，包括流和数据包套接字、Internet 协议、常见 HTTP 处理。比如创建 URL 及 `URLConnection/HttpURLConnection` 对象、设置连接参数、连接到服务器、向服务器写数据、从服务器读取数据等通信。以下是常见的使用 `java.net` 包的 HTTP 例子，如代码清单 13-1 所示。



代码清单 13-1 使用 java.net.*包连接网络

```
//...
try
{
    //定义地址
    URL url = new URL("http://www.google.com");
    //打开连接
    HttpURLConnection http =(HttpURLConnection)url.openConnection();
    //得到连接状态
    int nRC = http.getResponseCode();
    if(nRC == HttpURLConnection.HTTP_OK)
    {
        //取得数据
        InputStream is =http.getInputStream();
        //处理数据...
    }
}
catch (Exception e)
{
}
//...
```

22.1.2 Apache 接口

HTTP 协议可能是现在 Internet 上使用得最多、最重要的通信协议了。越来越多的 Java 应用程序需要通过 HTTP 协议来访问网络资源。虽然在 JDK 的 java.net 包中已经提供了访问 HTTP 协议的基本功能，但是对于大部分应用程序来说，JDK 库本身提供的功能远远不够。这时就需要 Android 提供的 Apache HttpClient 了。它是一个开源项目，功能更加完善，为客户端的 HTTP 编程提供高效、最新、功能丰富的工具包支持。Android 平台引入了 Apache HttpClient 的同时还提供了对它的一些封装和扩展，例如设置默认的 HTTP 超时和缓存大小等。Android 使用的是目前最新的 HttpClient 4.0(org.apache. http.*), 可以将 Apache 视为目前流行的开源 Web 服务器，主要包括创建 HttpClient、Get/Post、HttpRequest 等对象，设置连接参数，执行 HTTP 操作，处理服务器返回结果等功能。以下是一个使用 android. net. http. *包的例子，如代码清单 13-2 所示。

代码清单 13-2 使用 android. Net. http. *连接网络

```
//...
try
{
```



```
//创建 HttpClient
//这里使用 DefaultHttpClient 表示默认属性
HttpClient hc = new DefaultHttpClient();

//HttpGet 实例
HttpGet get = new HttpGet("http://www.google.com");

//连接
HttpResponse rp =hc.execute(get);

If(rp.getStatusLine().getStatusCode() == HttpStatus.SC_OK)
{
    //取得数据
    InputStream is = rp.getEntity().getContent();
    //处理数据...
}
}
catch (IOException e)
{
}
```

22.1.3 Android 网络接口

android.net.*包实际上是通过在 Apache 中 HttpClient 的封装来实现的一个 HTTP 编程接口，同时还提供了 HTTP 请求队列管理及 HTTP 连接池管理，以提高并发请求情况下（如转载网页时）的处理效率，除此之外还有网络状态监视等接口、网络访问的 Socket、常用的 Uri 类以及有关 WiFi 相关的类等。下面我们看最简单的 Socket 连接，如代码清单 13-3 所示。

代码清单 13-3 Android 中的 Socket 连接

```
//...
try
{
    //IP 地址
    InetAddress inetAddress = InetAddress.getByName ("127.0.0.1" );
    //端口
    Socket client = new Socket(inetAddress, 61203, true);
    //取得数据
    InputStream in = client.getInputStream();
```



```
OutputStream out = client.getOutputStream();
//处理数据...
out.close();
in.close();
client.close();
}
catch (UnknownHostException e)
{
}
catch (IOException e)
{
}
```

22.2 HTTP 通信

HTTP (Hyper Text Transfer Protocol, 超文本传输协议) 用于传送 WWW 方式的数据。HTTP 协议采用了请求/响应模型。客户端向服务器发送一个请求, 请求头包含了请求的方法、URI、协议版本, 以及包含请求修饰符、客户信息和内容类似于 MIME 的消息结构。服务器以一个状态行作为响应, 响应的内容包括消息协议的版本、成功或者错误编码, 还包含服务器信息、实体元信息以及可能的实体内容。它是一个属于应用层面向对象的协议, 由于其简洁、快速, 它适用于分布式超媒体信息系统。

许多 HTTP 通信是由一个用户代理初始化的, 并且包括一个申请在源服务器上资源的请求, 最简单的情况可能是在用户代理和服务器之间通过一个单独的连接来完成。在 Internet 上, HTTP 通信通常发生在 TCP/IP 连接之上, 默认端口是 TCP 80, 但其他的端口也是可用的。这并不预示着 HTTP 协议在 Internet 或其他网络的其他协议之上才能完成, HTTP 只预示着一个可靠的传输。Android 提供了 HttpURLConnection 和 HttpClient 接口来开发 HTTP 程序, 本节我们将分别介绍这两种方式。

22.2.1 HttpURLConnection 接口

HTTP 通信中使用最多的就是 Get 和 Post, Get 请求可以获取静态页面, 也可以把参数放在 URL 字符串后面, 传递给服务器。Post 与 Get 的不同之处在于 Post 的参数不是放在 URL 字符串里面, 而是放在 HTTP 请求数据中。HttpURLConnection 是 Java 的标准类, 继承自 URLConnection 类, URLConnection 与 HttpURLConnection 都是抽象类, 无法直接实例化对象。其对象主要通过 URL 的 openConnection 方法获得, 创建一个 HttpURLConnection 连接的代码如代码清单 13-4 所示。

代码清单 13-4 创建 HttpURLConnection 连接

```
URL url = new URL("http://www.google.com");
HttpURLConnection urlConn = (HttpURLConnection) url.openConnection();
```

openConnection 方法只创建 URLConnection 或 HttpURLConnection 实例, 但是并不进行真正的连接操作, 并且, 每次 openConnection 都将创建一个新的实例。因此, 在连接之前可以对其一些属性进行设



置，比如超时时间等。代码清单 13-5 所示的代码是对 HttpURLConnection 实例的属性设置。

代码清单 13-5 设置 HttpURLConnection 实例的属性

```
//设置输入/输出流
urlConn.setDoOutput(true);
urlConn.setDoInput(true);
// 设置方式为 POST
urlConn.setRequestMethod("POST");
// Post 请求不能使用缓存
urlConn.setUseCaches(false);
```

在连接完成后关闭连接代码如代码清单 13-6 所示。

代码清单 13-6 关闭连接

```
//关闭 http 连接
```

```
urlConn.disconnect();
```

这里来分别讲解一下 Get/Post 两种方式的编码情况。首先建立一个 Get 和 Post 来传递参数的网页 httpget.jsp，代码如代码清单 13-7 所示。

代码清单 13-7 httpget.jsp

```
<%@ page language="java" import="java.util.*" pageEncoding="gb2312"%>
<HTML>
<HEAD>
<TITLE>Http Test</TITLE>
</HEAD>
<BODY>
<%
String type = request.getParameter ("par");
String result = new String(type.getBytes("iso-8859-1"), "gb2312");
out.println (" <hl>parameters : "+result+"</hl>" );
%>
</BODY>
</HTML>
```

httpget.jsp 中要求传递一个 par 参数，然后在网页中显示 “parameters: par”，在浏览器中我们输入 “httpget.jsp?par=abcdefg” 来访问，则网页中显示为：“parameters: abcdefg”。

下面来看一下 Get 方式是如何传递参数的（如代码清单 13-8 所示），注意该代码中 url 地址上的传递参数。

代码清单 13-8 Get 方式传递参数

```
//http 地址"?par=abcdefg"是我们上传的参数
```



```
String httpUrl = "http://127.0.0.1:8080/httpget.jsp?par=abcdefg";
//获得的数据
String resultData = "";
URL url = null;
try
{
    //构造一个 URL 对象
    url = new URL(httpUrl);
}
catch (MalformedURLException e)
{
    Log.e(DEBUG_TAG, "MalformedURLException");
}
if (url != null){
    try
    {
        // 使用 HttpURLConnection 打开连接
        HttpURLConnection urlConn = (HttpURLConnection)
            url.openConnection();

        //得到读取的内容(流)
        InputStreamReader in = new
            InputStreamReader(urlConn.getInputStream());

        // 为输出创建 BufferedReader
        BufferedReader buffer = new BufferedReader(in);
        String inputLine = null;
        //使用循环来读取获得的数据
        while (((inputLine = buffer.readLine()) != null))
        {
            //我们在每一行后面加上一个"\n"来换行
            resultData += inputLine + "\n";
        }
        //关闭 InputStreamReader
        in.close();
        //关闭 http 连接
        urlConn.disconnect();
    }
}
```



```
//设置显示取得的内容
if ( resultData != null )
{
    mTextView.setText(resultData);
}
else
{
    mTextView.setText("读取的内容为 NULL");
}
}
catch (IOException e)
{
    Log.e(DEBUG_TAG, "IOException");
}
}
else
{
    Log.e(DEBUG_TAG, "Url NULL");
}
```

由于 HttpURLConnection 默认使用 Get 方式，所以如果要使用 Post 方式，则需要 setRequestMethod 设置，具体实现如代码清单 13-9 所示。

代码清单 13-9 Post 方式

```
String httpUrl = "http://127.0.0.1:8080/httpget.jsp";
//获得的数据
String resultData = "";
URL url = null;
try{
    //构造一个 URL 对象
    url = new URL(httpUrl);
}
catch (MalformedURLException e)
{
    Log.e(DEBUG_TAG, "MalformedURLException");
}
if (url != null) {
```




```
try
{
    // 使用 HttpURLConnection 打开连接
    HttpURLConnection urlConn = (HttpURLConnection)
                                url.openConnection();

    //因为这个是 post 请求，设立需要设置为 true
    urlConn.setDoOutput(true);
    urlConn.setDoInput(true);
    // 设置以 POST 方式
    urlConn.setRequestMethod("POST");
    // Post 请求不能使用缓存
    urlConn.setUseCaches(false);
    urlConn.setInstanceFollowRedirects(true);
    // 配置本次连接的 Content-type，
    //配置为 application/x-www-form-urlencoded 的
    urlConn.setRequestProperty("Content-Type",
                                "application/x-www-form-urlencoded");
    // 连接，postUrl.openConnection()至此的配置必须在 connect 前完成
    // 要注意的是 connection.getOutputStream 会隐含地进行 connect。
    urlConn.connect();
    //DataOutputStream 流
    DataOutputStream out = new
                                DataOutputStream(urlConn.getOutputStream());
    //要上传的参数
    String content = "par=" +
                                URLEncoder.encode("ABCDEFGF", "gb2312");
    //将要上传的内容写入流中
    out.writeBytes(content);
    //刷新、关闭
    out.flush();
    out.close();
    //获取数据
    BufferedReader reader = new BufferedReader
                                (new InputStreamReader(urlConn.getInputStream()));
    String inputLine = null;
```



```
//使用循环来读取获得的数据
while (((inputLine = reader.readLine()) != null))
{
    //我们在每一行后面加上一个"\n"来换行
    resultData += inputLine + "\n";
}
reader.close();
//关闭 http 连接
urlConn.disconnect();
//设置显示取得的内容
if ( resultData != null )
{
    mTextView.setText(resultData);
}
else
{
    mTextView.setText("读取的内容为 NULL");
}
}
catch (IOException e)
{
    Log.e(DEBUG_TAG, "IOException");
}
}
else
{
    Log.e(DEBUG_TAG, "Url NULL");
}
```

22.2.2 HttpClient 接口

上一节中，学习了通过标准 Java 接口来实现 Android 应用的联网操作。上面都只是简单地进行了网络的访问，但是在实际开发中，可能会运用到更复杂的联网操作。Apache 提供了 HttpClient，它对 java.net 中的类做了封装和抽象，更适合我们在 Android 上开发联网应用。

我们先来了解一下关于使用 HttpClient 所用到的一些类。

ClientConnectionManager 接口

ClientConnectionManager 是客户端连接管理器接口，它提供以下几个抽象方法，如表 13-1 所示。

表 13-1 ClientConnectionManager 的抽象方法



方法	说明
CloseConnectionManager	关闭所有无效、超时的连接
closeIdleConnections	关闭空闲的连接
releaseConnection	释放一个连接
requestConnection	请求一个新的连接
shutdown	关闭管理器并释放连接

DefaultHttpClient

DefaultHttpClient 是默认的一个 HTTP 客户端，可以使用它创建一个 HTTP 连接，代码如下所示：

代码清单 13-10 DefaultHttpClient 创建 HTTP 连接

```
HttpClient httpclient=new DefaultHttpClient();
```

HttpResponse

HttpResponse 是一个 HTTP 连接响应，当执行一个 HTTP 连接后，就会返回一个 HttpResponse，可以通过 HttpResponse 获得一些响应的信息。代码清单 13-11 所示是请求一个 HTTP 连接并获得该请求是否成功的代码。

代码清单 13-11 DefaultHttpClient 创建 HTTP 连接

```
HttpResponse httpResponse = httpclient.execute(httpRequest);
if (httpResponse.getStatusLine().getStatusCode() == HttpStatus.SC_OK)
{
    //连接成功
}
```

通过上面几个类的了解，下面将分别使用 Get 和 Post 方式请求一个网页。

先来看看 HttpClient 中如何使用 Get 方式获取数据，这里需要使用 HttpGet 来构建一个 Get 方式的 HTTP 请求，然后通过 HttpClient 来执行这个请求，HttpResponse 在接收这个请求后给出响应，最后通过“httpResponse.getStatusLine().getStatusCode()”来判断请求是否成功，并处理。代码如代码清单 13-12 所示。

代码清单 13-12 HttpClient 中如何使用 Get 方式获取数据

```
// http 地址
String httpUrl =
    "http://127.0.0.1:8080/httpget.jsp?par=HttpClient_android_Get";
//HttpGet 连接对象
HttpGet httpRequest = new HttpGet(httpUrl);
try
{
    //取得 HttpClient 对象
    HttpClient httpclient = new DefaultHttpClient();
```



```
//请求 HttpClient，取得 HttpResponse
HttpResponse httpResponse = httpClient.execute(httpRequest);
//请求成功
if (httpResponse.getStatusLine().getStatusCode() == HttpStatus.SC_OK)
{
    //取得返回的字符串
    String strResult = EntityUtils.toString(httpResponse.getEntity());
    mTextView.setText(strResult);
}
else
{
    mTextView.setText("请求错误!");
}
}
catch (ClientProtocolException e)
{
    mTextView.setText(e.getMessage().toString());
}
catch (IOException e)
{
    mTextView.setText(e.getMessage().toString());
}
catch (Exception e)
{
    mTextView.setText(e.getMessage().toString());
}
```

Post 方式则比 Get 方式稍微复杂一点。首先使用 `HttpPost` 来构建一个 Post 方式的请求。代码如代码清单 13-13 所示。

代码清单 13-13 构建 Post 方式的请求

```
String httpUrl = "http://127.0.0.1:8080/httpget.jsp";
//HttpPost 连接对象
```

```
HttpPost httpRequest = new HttpPost(httpUrl);
```

需要使用 `Name/ValuePair` 来保存要传递的参数，这里可以使用 `BasicNameValuePair` 来构造一个要被传递的参数，然后通过 `add` 方法添加这个参数到 `Name/ValuePair` 中，代码如代码清单 13-14 所示。



代码清单 13-14 使用 Name/ValuePair 来保存要传递的参数

```
//使用 NameValuePair 来保存要传递的 Post 参数
```

```
List<NameValuePair> params = new ArrayList<NameValuePair>();
```

```
//添加要传递的参数
```

```
params.add(new BasicNameValuePair("par", "HttpClient_android_Post"));
```

Post 方式需要设置所使用的字符集。最后就和 Get 方式一样通过 HttpClient 来请求这个连接，返回响应并处理。代码如代码清单 13-15 所示。

代码清单 13-15 Post 方式设置字符集并请求获取响应

```
//设置字符集
```

```
HttpEntity httpentity = new UrlEncodedFormEntity(params, "gb2312");
```

```
//请求 httpRequest
```

```
httpRequest.setEntity(httpentity);
```

```
//取得默认的 HttpClient
```

```
HttpClient httpclient = new DefaultHttpClient();
```

```
//取得 HttpResponse
```

```
HttpResponse httpResponse = httpclient.execute(httpRequest);
```

```
//HttpStatus.SC_OK 表示连接成功
```

```
if (httpResponse.getStatusLine().getStatusCode() == HttpStatus.SC_OK)
```

```
{
```

```
//取得返回的字符串
```

```
String strResult = EntityUtils.toString(httpResponse.getEntity());
```

```
mTextView.setText(strResult);
```

```
}
```

```
else
```

```
{
```

```
    mTextView.setText("请求错误!");
```

```
}
```

22.3 Socket 通信

Socket 通常也称做“套接字”，用于描述 IP 地址和端口，是一个通信链的句柄。应用程序通常通过“套

接字”向网络发出请求或者回复请求。它是通信的基石，是支持 TCP/IP 协议的网络通信的基本操作单元。它是网络通信过程中端点的抽象表示，包含进行网络通信必需的 5 种信息：连接使用的协议、本地主机的 IP 地址、本地进程的协议端口、远程主机的 IP 地址和远程进程的协议端口。



22.3.1 Socket 传输模式

Socket 有两种主要的操作方式：面向连接的和无连接的。面向连接的 Socket 操作就像一部电话，必须建立一个连接和一个呼叫。所有的事情到达时的顺序与它们出发时的顺序是一样的。无连接的 Socket 操作就像是一个邮件投递，没有什么保证，多个邮件到达时的顺序可能与出发时的顺序不一样。根据应用程序的需要决定 Socket 传输模式的选择。如果可靠性更高的话，用面向连接的操作会好一些。比如文件服务器需要数据的正确性和有序性，如果一些数据丢失了其时效性导致系统的有效性降低。但同时确保数据的有序性和正确性需要额外的操作，这会带来内存消耗，额外的费用将会降低系统的回应速率。

无连接的操作使用数据报协议。一个数据报是一个独立的单元。它包含了这次投递的所有信息。它包含了目的地址和要发送的内容，这个模式下的 Socket 不需要连接一个目的 Socket，它只是简单地投出数据报。无连接的操作是快速的和高效的，但是数据安全性不佳。

面向连接的操作使用 TCP 协议。该模式下的 Socket 必须在发送数据之前与目的地的 Socket 取得连接。一旦连接建立了，Socket 就可以使用一个流接口进行打开、读、写、关闭操作。所有发送的信息都会在另一端以同样的顺序被接收。面向连接的操作比无连接操作的效率要低，但是数据的安全性要高。

22.3.2 Socket 编程原理

Socket 构造

Java 在包 java.net 中提供了两个类 Socket 和 ServerSocket，分别用来表示双向连接的客户端和服务端。其构造方法如代码清单 13-16 所示。

代码清单 13-16 Socket 构造

```
Socket(InetAddress address, int port);
Socket(InetAddress address, int port, boolean stream);
Socket(String host, int port);
Socket(String host, int port, boolean stream);
Socket(SocketImpl impl);
Socket(String host, int port, InetAddress address, int localPort);
Socket(InetAddress address, int port, InetAddress localAddr, int localPort);
ServerSocket(int port);
ServerSocket(int port, int backlog);
ServerSocket(int port, int backlog, InetAddress bindAddr);
```

其中，address、host 和 port 分别是双向连接中另一方的 IP 地址、主机名和端口号，stream 指明 Socket 是流 Socket 还是数据报 Socket，localPort 表示本地主机的端口号，localAddr 和 bindAddr 是本地机器的地址（ServerSocket 的主机地址），impl 是 Socket 的父类，既可以用来创建 ServerSocket，又可以用来创建 Socket。

客户端 Socket

要想使用 Socket 来与一个服务器通信，就必须先在客户端创建一个 Socket，并指出需要连接的服务



器的 IP 地址和端口，这也是使用 Socket 通信的第一步，代码如代码清单 13-17 所示。

代码清单 13-17 创建客户端 Socket

```
try
{
//创建 Socket, 127.0.0.1 是 IP 地址, 54321 为端口
socket = new Socket("127.0.0.1",54321);
}
catch (Exception e) {}
```

ServerSocket

代码清单 13-18 创建服务器端 ServerSocket

```
try
{
//创建 ServerSocket
ServerSocket serverSocket = new ServerSocket(54321);
//接受客户端请求
Socket client = serverSocket.accept();
}catch (IOException e) {}
```

通过以上程序我们创建一个 ServerSocket 在端口 54321 监听客户请求，它是 Server 的典型工作模式，在这里 Server 只能接收一个请求，接收后 Server 就退出了。实际的应用中总是让它不停地循环接收，一旦有客户请求，Server 总是会创建一个服务线程来服务新来的客户，而自己继续监听。程序中 accept0 是一个阻塞方法，表示该方法在被调用后等待客户的请求，直到有一个客户启动并请求连接到相同的端口，然后 accept() 返回一个对应于客户的 Socket。这时，客户方和服务方都建立了用于通信的 Socket，接下来就是由各个 Socket 分别打开各自的输入、输出流。

输入（出）流

Socket 提供了方法 getInputStream() 和 getOutputStream() 来得到对应的输入（出）流以进行读（写）操作，这两个方法分别返回 InputStream 和 OutputStream 类对象。为了便于读（写）数据，我们可以在返回的输入/输出流对象上建立过滤流，如 DataInputStream、DataOutputStream 或 PrintStream 类对象，对于文本方式流对象，可以采用 InputStreamReader 和 OutputStreamWriter、PrintWriter 等处理。代码如代码清单 13-19 所示。

代码清单 13-19 Socket 输入/输出流

```
PrintStream os = new PrintStream(
new BufferedOutputStream(socket.getOutputStream()));

DataInputStream is = new DataInputStream(socket.getInputStream());

PrintWriter out = new PrintWriter(socket.getOutputStream(),true);

BufferedReader in = new BufferedReader(
```



```
new InputStreamReader(socket.getInputStream());
```

关闭 Socket

每一个 Socket 存在时都将占用一定的资源，在 Socket 对象使用完毕时，要使其关闭，关闭 Socket 可以调用 Socket 的 close() 方法。在关闭 Socket 之前，应与 Socket 相关的所有输入（出）流全部关闭，以释放所有的资源。而且要注意关闭的顺序，与 Socket 相关的所有的输入/输出应先关闭，然后再关闭 Socket。尽管 Java 有自动回收机制，网络资源最终会被释放，但是为了有效利用资源，建议读者按照合理的顺序主动释放资源。

代码清单 13-20 Socket 输入/输出流

```
os.close();  
is.close();  
socket.close();
```

22.3.3 Socket 编程实例

下面我们利用之前学到的 Java 标准 API 来开发网络应用，实现一个简单的服务器和客户端通信，客户端发送数据且接收服务器发回的数据并显示。

首先是服务器端的建立

创建服务器的步骤如下：

- 指定端口实例化一个 ServerSocket。
- 调用 ServerSocket 的 accept() 以在等待连接期间造成阻塞。
- 获取位于该底层 Socket 的流以进行读/写操作。
- 将数据封装成流。
- 对 Socket 进行读/写。
- 关闭打开的流。

服务器端实现代码如代码清单 13-21 所示。

代码清单 13-21 服务器端代码

```
import java.io.BufferedReader;  
import java.io.BufferedWriter;  
import java.io.InputStreamReader;  
import java.io.OutputStreamWriter;  
import java.io.PrintWriter;  
import java.net.ServerSocket;  
import java.net.Socket;  
  
public class Server implements Runnable  
{  
  
    public void run()
```




```
{
    try
    {
        //创建 ServerSocket
        ServerSocket serverSocket = new ServerSocket(54321);
        while (true)
        {
            //接受客户端请求
            Socket client = serverSocket.accept();
            System.out.println("accept");
            try
            {
                //接收客户端消息
                BufferedReader in = new BufferedReader(new
InputStreamReader(client.getInputStream()));

                String str = in.readLine();
                System.out.println("read:" + str);
                //向服务器发送消息
                PrintWriter out = new PrintWriter( new BufferedWriter( new
OutputStreamWriter(client.getOutputStream()),true);

                out.println("server message");
                //关闭流
                out.close();
                in.close();
            }
            catch (Exception e)
            {
                System.out.println(e.getMessage());
                e.printStackTrace();
            }
            finally
            {
                //关闭
                client.close();
                System.out.println("close");
            }
        }
    }
}
```



```
        }  
    }  
}  
catch (Exception e)  
{  
    System.out.println(e.getMessage());  
}  
}  
//main 函数，开启服务器  
public static void main(String a[])  
{  
    Thread desktopServerThread = new Thread(new Server());  
    desktopServerThread.start();  
}  
}
```

上述代码中，使用了 `java.net` 和 `java.io`。`java.net` 包提供了需要的 `Socket` 工具，`java.io` 包提供对流进行读/写的工具。设置了服务器的端口为 54321，开启一个线程，通过 `accept` 方法使服务器开始监听客户端的连接，并取得客户端的 `Socket` 对象 `client`，通过 `BufferedReader` 对象来接收客户端的输入流，如果要向客户端发送数据，可以使用 `PrintWriter` 来实现，但是需要通过 `Socket` 对象来取得其输出流，最后，关闭流和 `Socket`。`main` 函数用来开启服务器，这里为求简单，并没有对服务器端进行窗口化，而是处于命令行模式。

客户端的实现

实现客户端的步骤如下：

通过 IP 地址和端口实例化 `Socket`，请求连接服务器。

获取 `Socket` 上的流以进行读/写。

把流包装进 `BufferedReader/PrintWriter` 的实例。

对 `Socket` 进行读/写。

关闭打开的流。

实现客户端的代码如代码清单 13-22 所示。

代码清单 13-22 客户端实现

```
import java.io.BufferedReader;  
import java.io.BufferedWriter;  
import java.io.InputStreamReader;  
import java.io.OutputStreamWriter;  
import java.io.PrintWriter;
```



```
import java.net.Socket;

import android.app.Activity;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;

public class Activity01 extends Activity
{
    private final String    DEBUG_TAG= "Activity01";

    private TextView    mTextView=null;
    private EditText    mEditText=null;
    private Button      mButton=null;
    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);

        mButton = (Button)findViewById(R.id.Button01);
        mTextView=(TextView)findViewById(R.id.TextView01);
        mEditText=(EditText)findViewById(R.id.EditText01);

        //登录
        mButton.setOnClickListener(new OnClickListener()
        {
            public void onClick(View v)
            {
                Socket socket = null;
```



```
String message = mEditText.getText().toString() + "\r\n";
try
{
    //创建 Socket
    socket = new Socket("127.0.0.1",54321);
    //向服务器发送消息
    PrintWriter out = new PrintWriter( new BufferedWriter( new
OutputStreamWriter(socket.getOutputStream()),true);
    out.println(message);

    //接收来自服务器的消息
    BufferedReader br = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
    String msg = br.readLine();

    if ( msg != null )
    {
        mTextView.setText(msg);
    }
    else
    {
        mTextView.setText("数据错误!");
    }
    //关闭流
    out.close();
    br.close();
    //关闭 Socket
    socket.close();
}
catch (Exception e)
{
    // TODO: handle exception
    Log.e(DEBUG_TAG, e.toString());
}
}
```



```
});  
  
}  
  
}
```

上述实现客户端的代码中，监听了一个按钮事件，在按钮事件中通过 “socket = new Socket("127.0.0.1",54321);” 来请求连接服务器。和服务端一样，通过 `PrintWriter` 和 `BufferedReader` 来接收和发送消息。在接收到消息后，更新显示到 `TextView` 中。

22.3.4 Socket 与 HTTP 通信的区别

Android 中的 HTTP 通信中，客户端发送的每次请求不需要服务器回送响应，在请求结束后，会主动释放连接。若双方建立的是 HTTP 连接，则服务器需要等到客户端发送一次请求后才能将数据传回给客户端。

而利用 `Socket` 则可以使得客户端定时向服务器发送连接请求，服务器在收到该请求后对客户端进行回复，表明知道客户端“在线”。若服务器长时间无法收到客户端的请求，则认为客户端“下线”；若客户端长时间无法收到服务器的回复，则认为网络已经断开。很多情况下，服务器会主动向客户端发送数据，保持客户端与服务器数据的实时与同步。这样使得不仅可以保持客户端程序的在线状态，同时也是在“询问”服务器是否有新的数据，如果有就将数据传给客户端。

22.3.5 本章小结

本章主要学习了 Android 平台上网络与通信的开发，主要包括网络通信中常见的 HTTP 和 `Socket` 通信，通信过程中的中文乱码处理方案。在最常用的 HTTP 通信中分别讲述了 `HttpURLConnection` 和 `HttpClient` 接口的使用；学习 `Socket` 通信时我们通过一个简单的示例程序学习了对 `Socket` 的综合运用。



华清远见
dev.havi.com



22.4 框架布局实验

22.4.1 实验目的

通过实验学习 android 应用开发中的框架布局方法。

22.4.2 实验平台

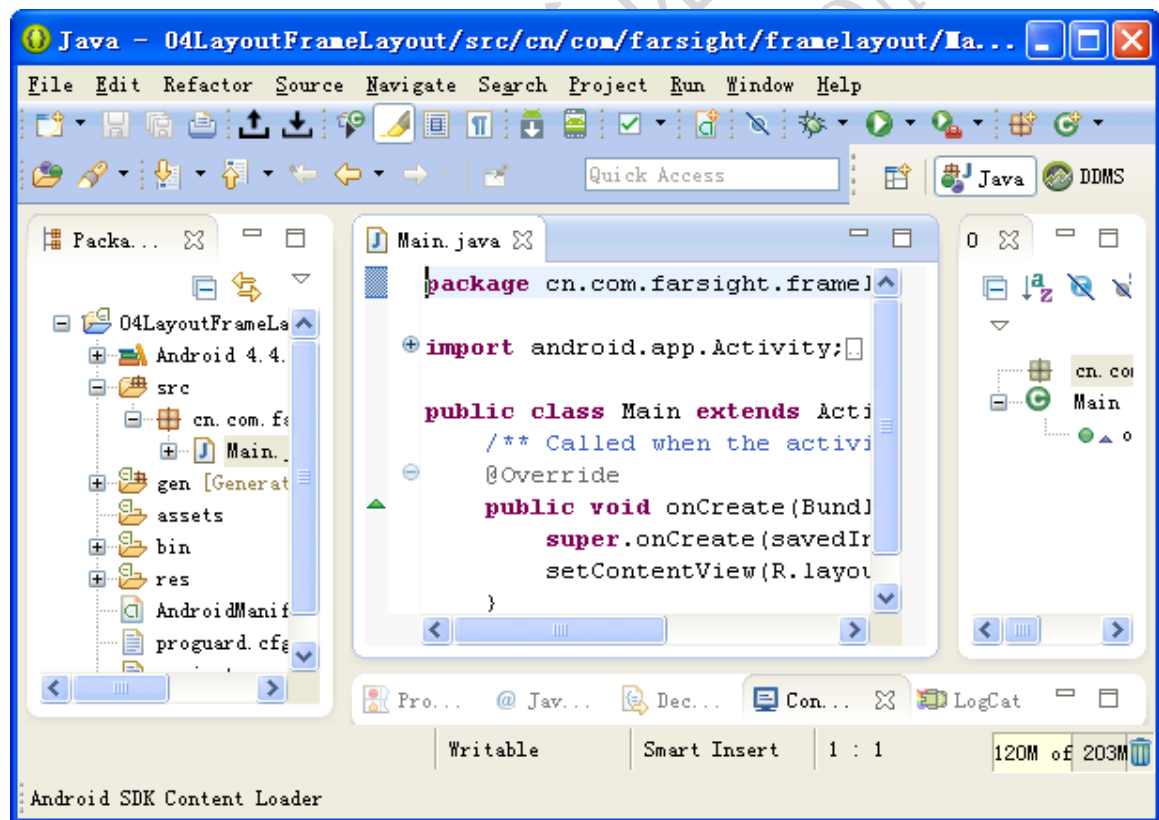
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

22.4.3 实验原理

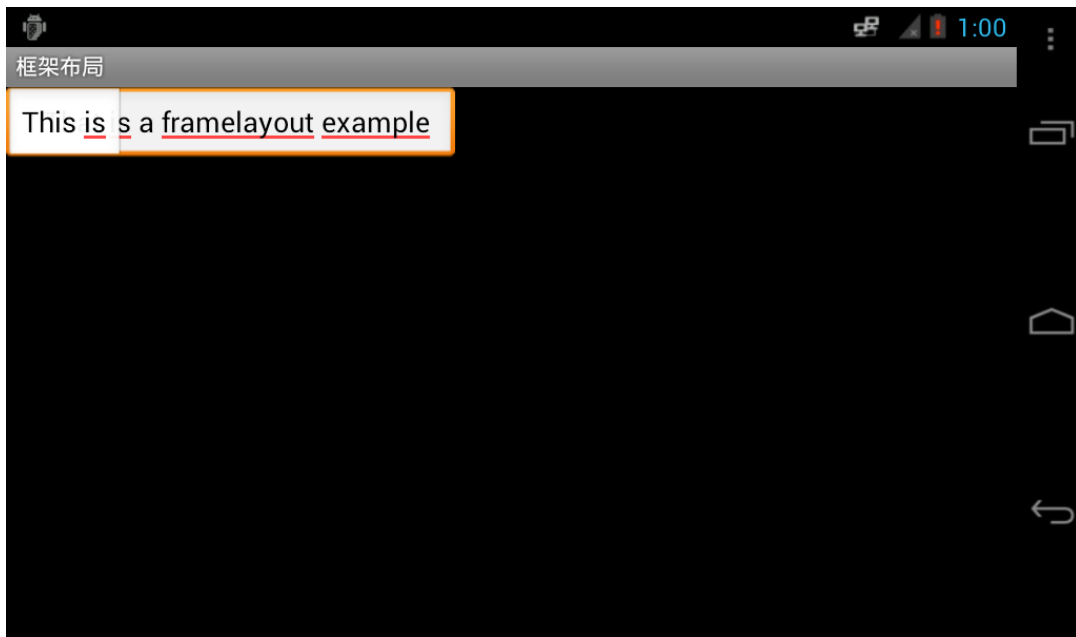
详见“第 15 章用户界面开发-框架布局部分”

22.4.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Android 应用实验源码\界面编程】中“2、 框架布局”目录中下的工程“04LayoutFrameLayout”：



右键点击“04LayoutFrameLayout”，选择“run as ->Android Application”，选择对应试验箱设备，点击 ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



22.4.5 实验代码

Main.java 代码

```
1 package cn.com.farsight.framelayout;
2
3 import android.app.Activity;
4 import android.os.Bundle;
5
6 public class Main extends Activity
7 {
8     /** Called when the activity is first created. */
9     @Override
10    public void onCreate(Bundle savedInstanceState)
11    {
12        super.onCreate(savedInstanceState);
13        setContentView(R.layout.main);
14    }
15 }
```

Main.xml 代码

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android" android:orientation="vertical" android:layout_width="fill_parent" android:layout_height="fill_parent">
    <FrameLayout android:orientation="vertical" android:layout_width="fill_parent" android:layout_height="fill_parent" android:layout_weight="1">
        <!--最底层的组件 -->
    </FrameLayout>
</LinearLayout>
```




```
<EditText android:text=" That is a framelayout example" android:id="@+id/text01" android:layo  
ut_width="wrap_content" android:layout_height="wrap_content" />  
    <!--最上层的组件 -->  
    <EditText android:text="This is" android:id="@+id/text02" android:layout_width="wrap_content"  
android:layout_height="wrap_content" />  
</FrameLayout>  
</LinearLayout>
```

22.5 表格布局实验

22.5.1 实验目的

通过实验学习 android 应用开发中的框架布局方法。

22.5.2 实验平台

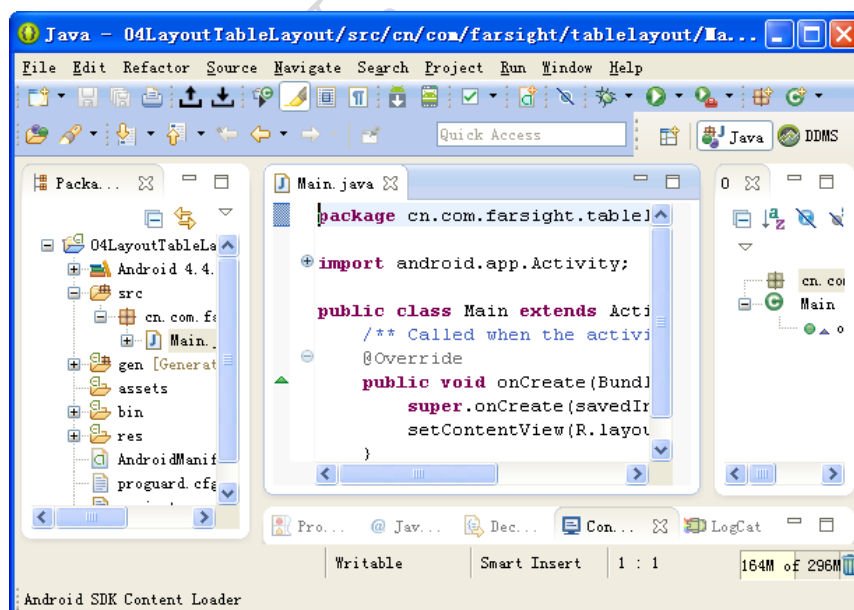
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

22.5.3 实验原理

详见“第 15 章用户界面开发-表格布局部分”。

22.5.4 实验步骤

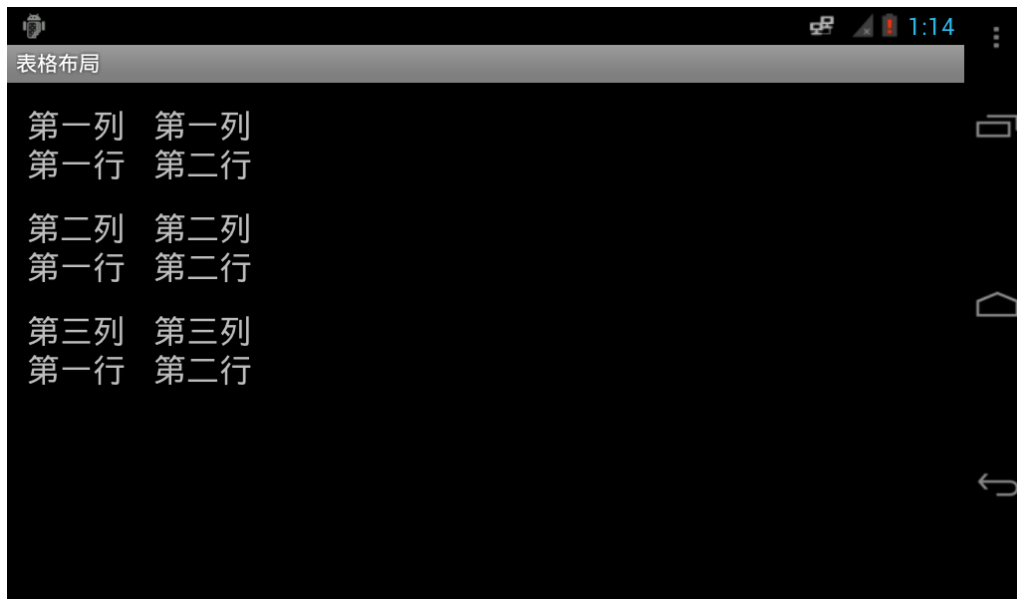
导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Android 应用实验源码\界面编程】
中“4、 表格布局”目录中下的工程“04LayoutTableLayout”：



右键点击“04LayoutFrameLayout”，选择“run as ->Android Application”，选择对应试验箱设备，点



击 ok, apk 文件会安装至 FS6818 开发板的 android 文件系统中, 下面是程序运行效果图:



22.5.5 实验代码

Main.java

```
1 package cn.com.farsight.tablelayout;
2
3 import android.app.Activity;
4 import android.os.Bundle;
5
6 public class Main extends Activity
7 {
8     /** Called when the activity is first created. */
9     @Override
10    public void onCreate(Bundle savedInstanceState)
11    {
12        super.onCreate(savedInstanceState);
13        setContentView(R.layout.main);
14    }
15 }
```

Main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android" android:layout_width="fill_parent" android:layout_height="fill_parent">
    <TableRow android:layout_marginTop="20px">
        <TextView android:layout_column="0" android:text="第一列\n 第一行"
        " android:textSize="22sp" android:layout_marginLeft="20px" />
        <TextView android:layout_column="1" android:text="第一列\n 第二行"
```



```
" android:textSize="22sp" android:layout_marginLeft="20px" />></TableRow>
<TableRow android:layout_marginTop="20px">
<TextView android:layout_column="0" android:text="第二列\n 第一行"
" android:textSize="22sp" android:layout_marginLeft="20px" />
<TextView android:layout_column="1" android:text="第二列\n 第二行"
" android:textSize="22sp" android:layout_marginLeft="20px" />></TableRow>
<TableRow android:layout_marginTop="20px">
<TextView android:layout_column="0" android:text="第三列\n 第一行"
" android:textSize="22sp" android:layout_marginLeft="20px" />
<TextView android:layout_column="1" android:text="第三列\n 第二行"
" android:textSize="22sp" android:layout_marginLeft="20px" />></TableRow>
</TableLayout>
```

22.6 相对布局实验

22.6.1 实验目的

通过实验学习 android 应用开发中的相对布局方法。

22.6.2 实验平台

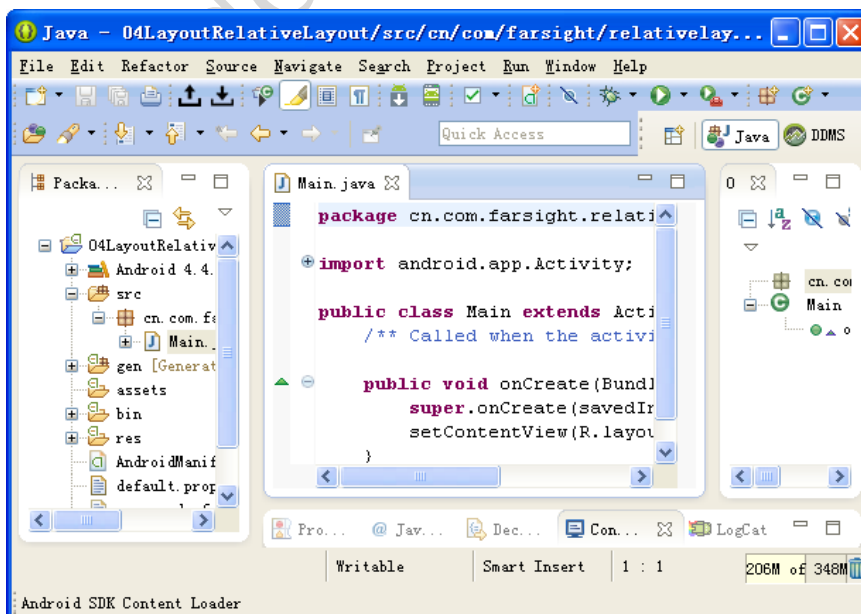
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

22.6.3 实验原理

详见“第 15 章用户界面开发-相对布局部分”。

22.6.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Android 应用实验源码\界面编程】中“3、相对布局”目录中下的工程“04LayoutRelativeLayout”：





右键点击“04LayoutRelativeLayout”，选择“run as ->Android Application”，选择对应试验箱设备，点击 ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



22.6.5 实验代码

Main.java

```
1 package cn.com.farsight.relativelayout;
2
3 import android.app.Activity;
4 import android.os.Bundle;
5
6 public class Main extends Activity
7 {
8     /** Called when the activity is first created. */
9
10    public void onCreate(Bundle savedInstanceState)
11    {
12        super.onCreate(savedInstanceState);
13        setContentView(R.layout.main);
14    }
15 }
```

Main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout android:id="@+id/relativeLayout1" xmlns:android="http://schemas.android.com/apk/res/android" android:layout_width="fill_parent" android:layout_height="wrap_content">
    <TextView android:layout_height="wrap_content" android:layout_width="wrap_content" android:id="
```

```
@+id/textView1" android:padding="8dip" android:layout_marginLeft="10.0dip" android:layout_alignParentLeft="true" android:text="我是左边"></TextView>

<TextView android:layout_height="wrap_content" android:layout_width="wrap_content" android:id="@+id/textView2" android:padding="8dip" android:layout_marginRight="10.0dip" android:layout_alignParentRight="true" android:text="我是右边"></TextView>

<TextView android:id="@+id/textView3" android:layout_width="fill_parent" android:layout_toRightOf="@id/textView1" android:padding="8dip" android:text="我是中间我是中间我是中间" android:layout_toLeftOf="@id/textView2" android:layout_height="wrap_content" android:singleLine="true" android:textColor="#abcdef" android:gravity="left" android:marqueeRepeatLimit="marquee_forever"></TextView>
</RelativeLayout>
```

22.7 TextView 实验

22.7.1 实验目的

通过实验学习 android 应用开发中的 TextView 方法。

22.7.2 实验平台

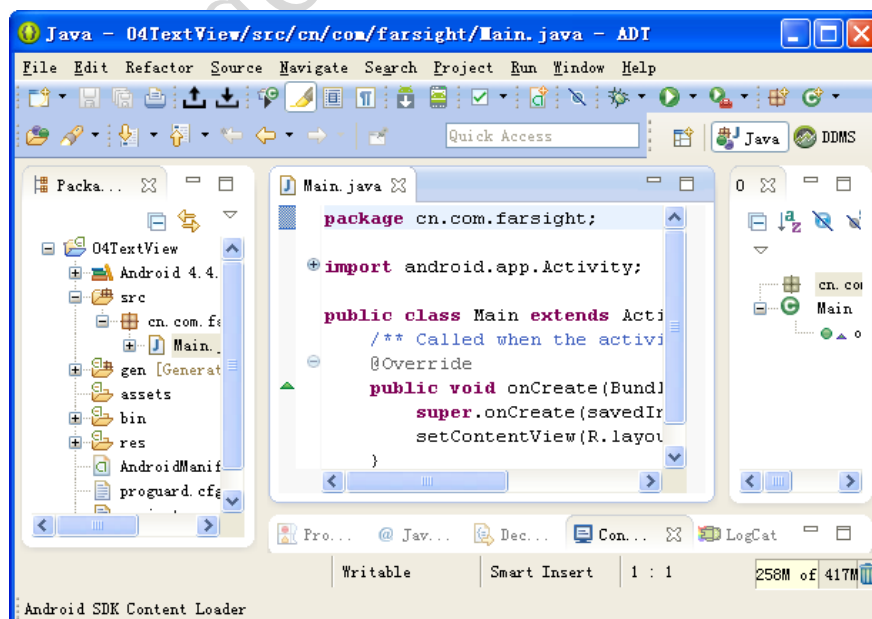
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

22.7.3 实验原理

详见“第 15 章用户界面开发-TextView 部分”。

22.7.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料-I-FS6818: 程序源码\Android 应用实验源码\界面编程】中“5、TextView 的使用”目录中下的工程“04TextView”：





右键点击“O4TextView”，选择“run as ->Android Application”，选择对应试验箱设备，点击 ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



22.7.5 实验代码

Main.java

```
1 package cn.com.farsight;
2
3 import android.app.Activity;
4 import android.os.Bundle;
5
6 public class Main extends Activity
7 {
8     /** Called when the activity is first created. */
9     @Override
10    public void onCreate(Bundle savedInstanceState)
11    {
12        super.onCreate(savedInstanceState);
13        setContentView(R.layout.main);
14    }
15 }
```

main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android" android:orientation="vertical" android:layout_width="fill_parent" android:layout_height="fill_parent">
    <TextView android:layout_width="fill_parent" android:layout_height="wrap_content" android:gravity="center" android:autoLink="web" android:text="Google - www.google.com" />
</LinearLayout>
```



```
<TextView android:layout_width="fill_parent" android:layout_height="wrap_content" android:gravity="center" android:autoLink="web" android:text="Farsight - www.farsight.com.cn" />
</LinearLayout>
```

22.8 EditText 实验

22.8.1 实验目的

通过实验学习 android 应用开发中的 EditText 方法。

22.8.2 实验平台

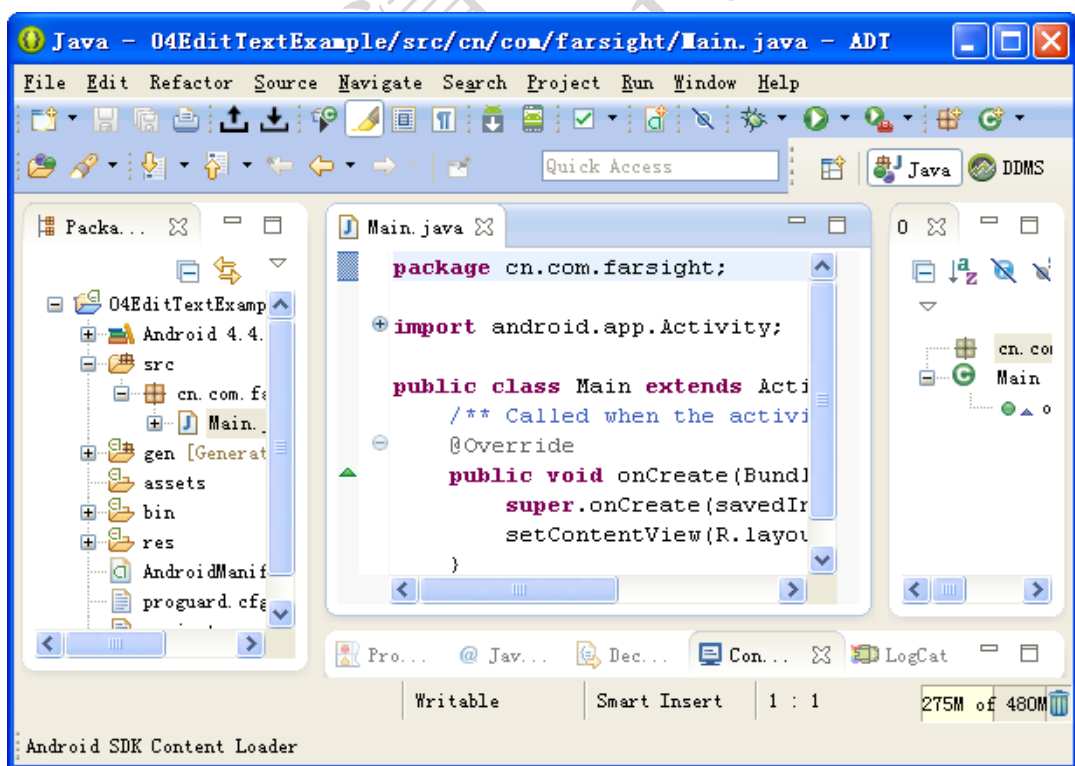
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

22.8.3 实验原理

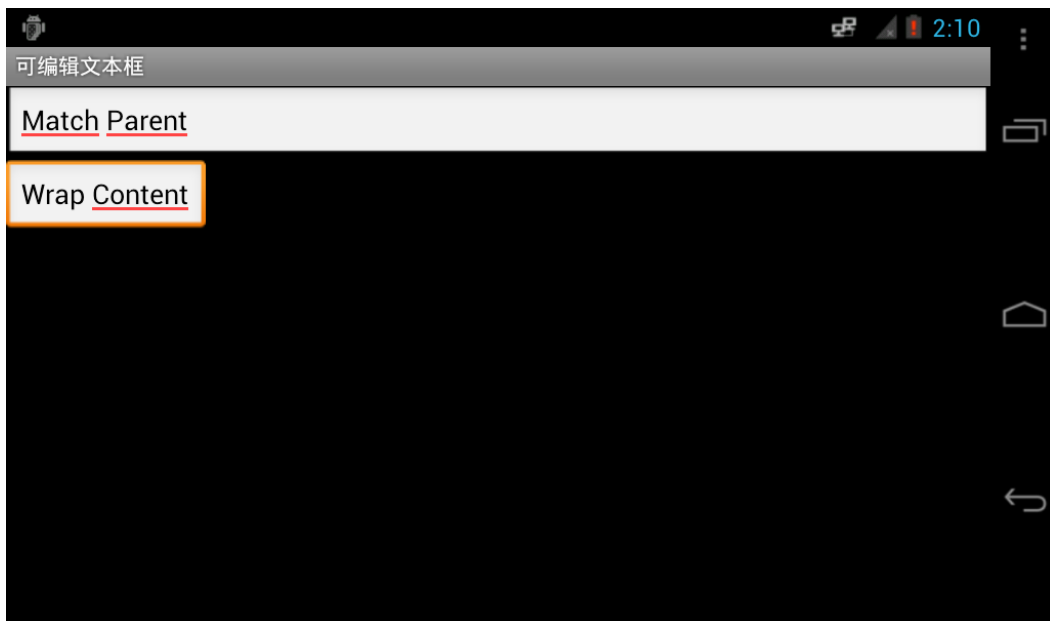
详见“第 15 章用户界面开发-EditText 部分”。

22.8.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料-I-FS6818: 程序源码\Android 应用实验源码\界面编程】中“14、文本编辑框”目录中下的工程“04EditTextExample”：



右键点击“04EditTextExample”，选择“run as ->Android Application”，选择对应试验箱设备，点击ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



22.8.5 实验代码

Main.java

```
1 package cn.com.farsight;
2
3 import android.app.Activity;
4 import android.os.Bundle;
5
6 public class Main extends Activity
7 {
8     /** Called when the activity is first created. */
9     @Override
10    public void onCreate(Bundle savedInstanceState)
11    {
12        super.onCreate(savedInstanceState);
13        setContentView(R.layout.main);
14    }
15 }
```

main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android" android:orientation="vertical" android:layout_width="fill_parent" android:layout_height="fill_parent">
    <EditText android:id="@+id/editText1" android:layout_width="match_parent" android:text="Match Parent" android:layout_height="wrap_content"></EditText>
    <EditText android:id="@+id/editText2" android:text="Wrap Content" android:layout_height="wrap_content" android:layout_width="wrap_content"></EditText>
</LinearLayout>
```




22.9 Button 实验

22.9.1 实验目的

通过实验学习 android 应用开发中的 Button 方法。

22.9.2 实验平台

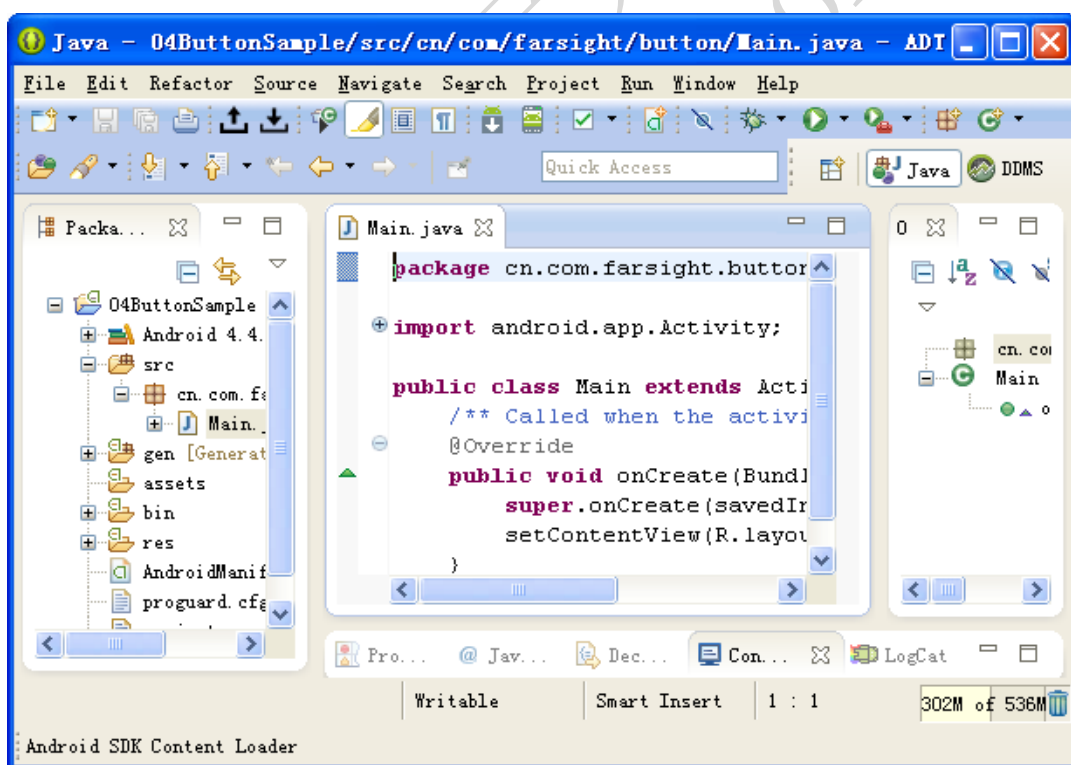
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

22.9.3 实验原理

详见“第 15 章用户界面开发-Button 部分”。

22.9.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Android 应用实验源码\界面编程】中“7、按钮的使用”目录中下的工程“04ButtonSample”：



右键点击“04ButtonSample”，选择“run as ->Android Application”，选择对应试验箱设备，点击 ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



22.9.5 实验代码

Main.java

```
1 package cn.com.farsight.button;
2
3 import android.app.Activity;
4 import android.os.Bundle;
5
6 public class Main extends Activity
7 {
8     /** Called when the activity is first created. */
9     @Override
10    public void onCreate(Bundle savedInstanceState)
11    {
12        super.onCreate(savedInstanceState);
13        setContentView(R.layout.main);
14    }
15 }
```

main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android" android:orientation="vertical" android:layout_width="fill_parent" android:layout_height="fill_parent">
    <Button android:text="确定"
" android:id="@+id/button1" android:layout_width="wrap_content" android:layout_height="wrap_content"></Button>
    <Button android:text="取消"
" android:id="@+id/button2" android:layout_width="wrap_content" android:layout_height="wrap_content"></Button>
```



```
<Button android:text="我是一个长按钮"
" android:id="@+id/button3" android:layout_width="fill_parent" android:layout_height="wrap_content"></Button>
</LinearLayout>
```

22.10 ImageButton 实验

22.10.1 实验目的

通过实验学习 android 应用开发中的 ImageButton 方法。

22.10.2 实验平台

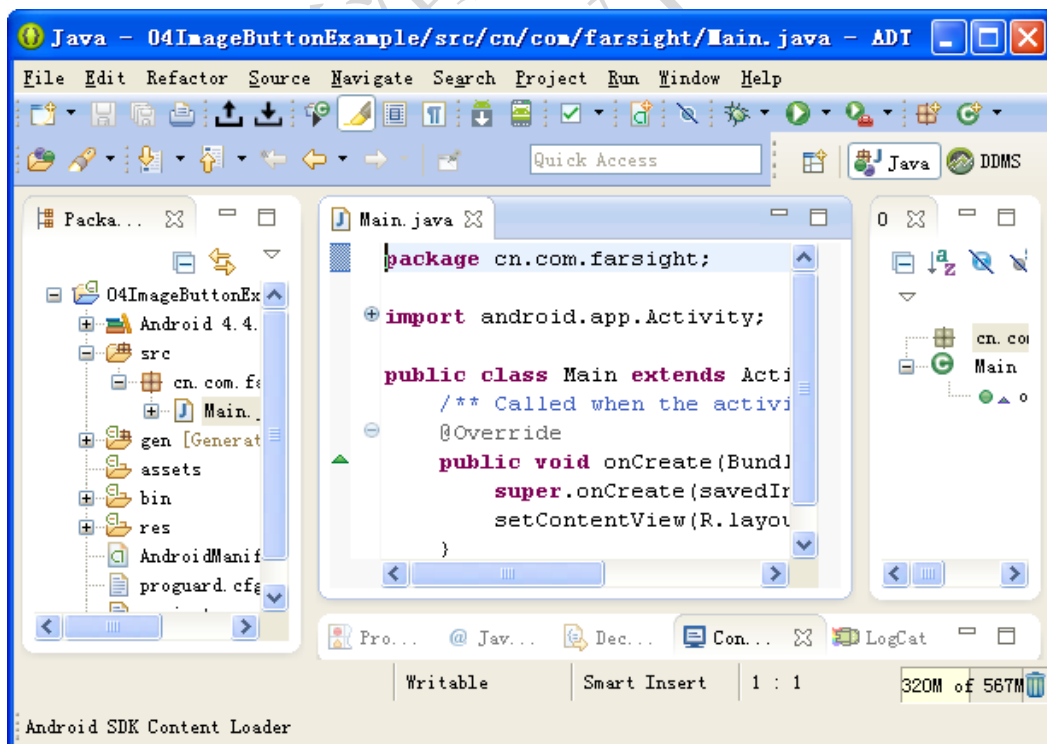
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

22.10.3 实验原理

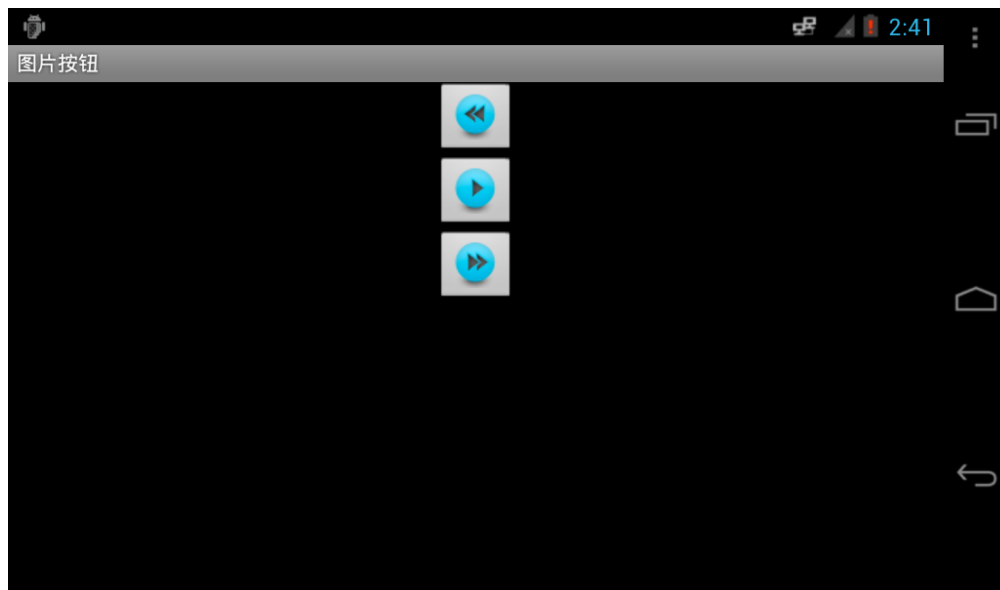
详见“第 15 章用户界面开发-ImageButton 部分”。

22.10.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料-I-FS6818: 程序源码\Android 应用实验源码\界面编程】中“8、图片按钮”目录中下的工程“04ImageButtonExample”：



右键点击“04ImageButtonExample”，选择“run as ->Android Application”，选择对应试验箱设备，点击 ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



22.10.5 实验代码

Main.java

```
1 package cn.com.farsight;
2
3 import android.app.Activity;
4 import android.os.Bundle;
5
6 public class Main extends Activity
7 {
8     /** Called when the activity is first created. */
9     @Override
10    public void onCreate(Bundle savedInstanceState)
11    {
12        super.onCreate(savedInstanceState);
13        setContentView(R.layout.main);
14    }
15 }
```

22.11 日期选择器实验

22.11.1 实验目的

通过实验学习 android 应用开发中的 Button 的使用方法。

22.11.2 实验平台

FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

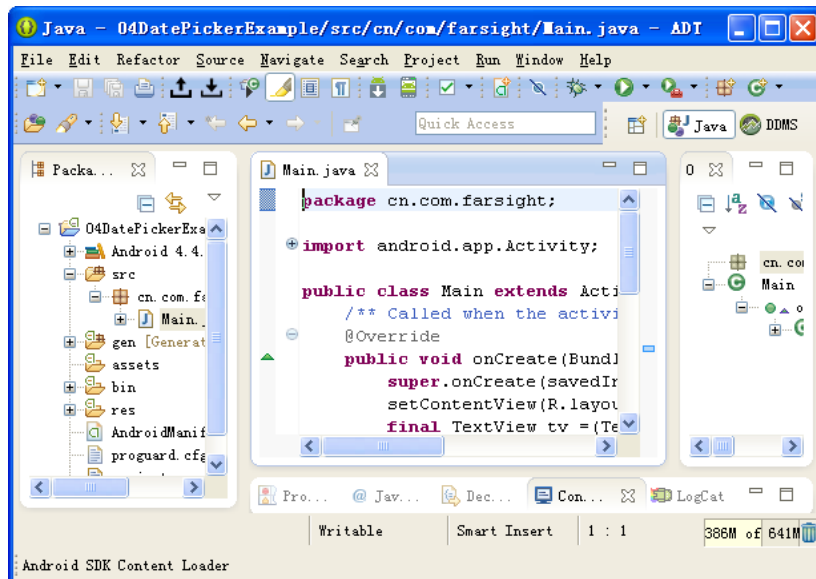


22.11.3 实验原理

详见“第 15 章用户界面开发”。

22.11.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Android 应用实验源码\界面编程】中“12、日期选择器”目录中下的工程“04DatePickerExample”：



右键点击“04DatePickerExample”，选择“run as ->Android Application”，选择对应试验箱设备，点击ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



22.11.5 实验代码

Main.java



```
1 package cn.com.farsight;
2
3 import android.app.Activity;
4 import android.os.Bundle;
5 import android.view.View;
6 import android.widget.Button;
7 import android.widget.DatePicker;
8 import android.widget.TextView;
9
10 public class Main extends Activity
11 {
12     /** Called when the activity is first created. */
13     @Override
14     public void onCreate(Bundle savedInstanceState)
15     {
16         super.onCreate(savedInstanceState);
17         setContentView(R.layout.main);
18         final TextView tv = (TextView)findViewById(R.id.textView1);
19         final Button btn = (Button)findViewById(R.id.button1);
20         final DatePicker dp = (DatePicker)findViewById(R.id.datePicker1);
21         btn.setOnClickListener(new View.OnClickListener()
22         {
23
24
25             public void onClick(View v)
26             {
27                 // TODO Auto-generated method stub
28                 int year = dp.getYear();
29                 int month = dp.getMonth();
30                 int day = dp.getDayOfMonth();
31                 tv.setText(year + "/" + month + "/" + day);
32             }
33         });
34     }
35 }
36
```

main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android" android:orientation="vertical" android:layout_width="fill_parent" android:layout_height="fill_parent">
    <DatePicker android:layout_gravity="center_horizontal" android:layout_width="wrap_content" android:layout_height="wrap_content" android:id="@+id/datePicker1"></DatePicker>
    <Button android:layout_width="wrap_content" android:layout_gravity="center_horizontal" android:text="选择" android:layout_height="wrap_content" android:id="@+id/button1"></Button>
</LinearLayout>
```



```
<TextView android:layout_width="wrap_content" android:text="选择的日期是:"  
" android:id="@+id/textView1" android:layout_height="wrap_content" android:layout_gravity="center  
_horizontal"></TextView>  
</LinearLayout>
```

22.12 CheckBox 实验

22.12.1 实验目的

通过实验学习 android 应用开发中 CheckBox 的使用方法。

22.12.2 实验平台

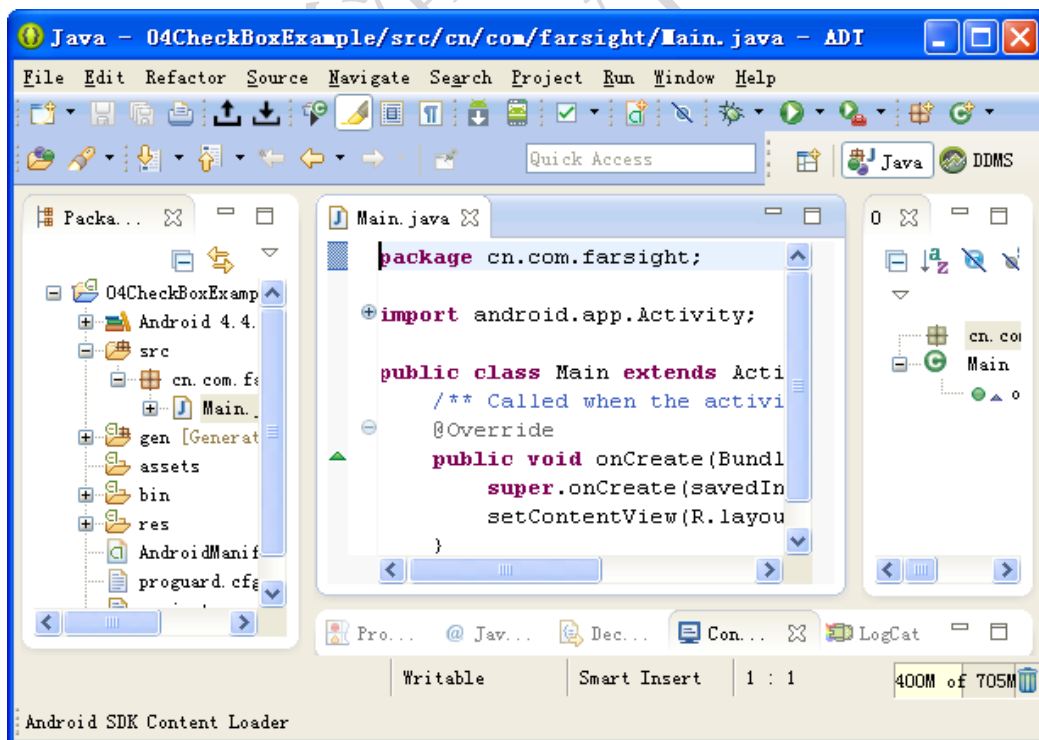
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

22.12.3 实验原理

详见“第 15 章用户界面开发-CheckBox 部分”。

22.12.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料-I-FS6818: 程序源码\Android 应用实验源码\界面编程】中“11、复选框的使用”目录中下的工程“04CheckBoxExample”：



右键点击“04CheckBoxExample”，选择“run as ->Android Application”，选择对应试验箱设备，点击 ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



22.12.5 实验代码

Main.java

```
1 package cn.com.farsight;
2
3 import android.app.Activity;
4 import android.os.Bundle;
5
6 public class Main extends Activity
7 {
8     /** Called when the activity is first created. */
9     @Override
10    public void onCreate(Bundle savedInstanceState)
11    {
12        super.onCreate(savedInstanceState);
13        setContentView(R.layout.main);
14    }
15 }
```

main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android" android:orientation="vertical" android:layout_width="fill_parent" android:layout_height="fill_parent">
    <CheckBox android:text="篮球"
" android:id="@+id/checkbox1" android:layout_width="wrap_content" android:layout_height="wrap_content"></CheckBox>
    <CheckBox android:text="足球"
" android:id="@+id/checkbox2" android:layout_width="wrap_content" android:layout_height="wrap_content"></CheckBox>
```




```
<CheckBox android:text="网球"
" android:id="@+id/checkBox3" android:layout_width="wrap_content" android:layout_height="wrap_con
tent"></CheckBox>
</LinearLayout>
```

22.13 RadioButton 实验

22.13.1 实验目的

通过实验学习 android 应用开发中 RadioButton 的使用方法。

22.13.2 实验平台

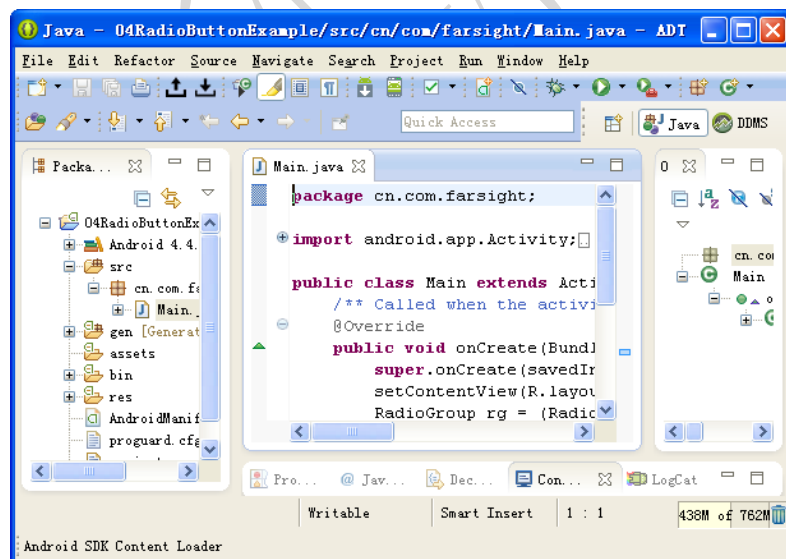
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

22.13.3 实验原理

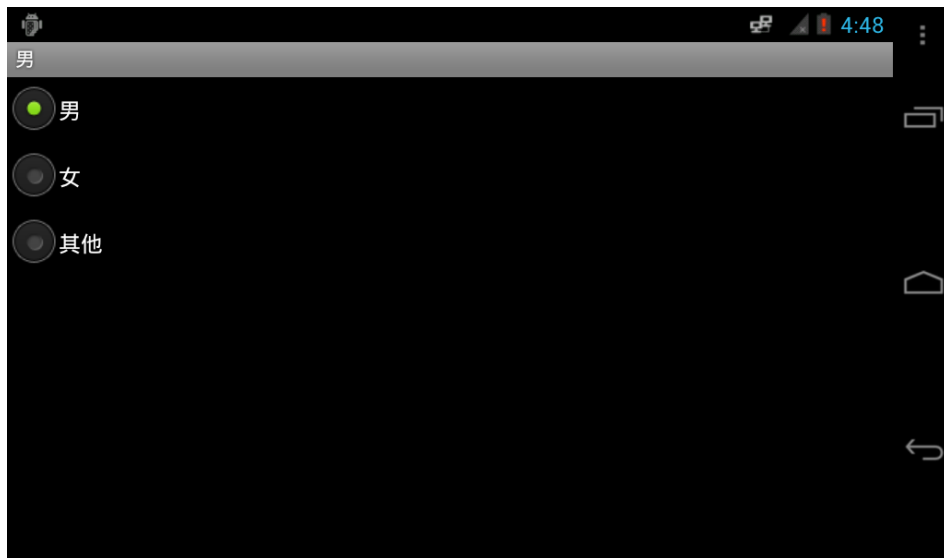
详见“第 15 章用户界面开发-RadioButton 部分”。

22.13.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料-I-FS6818：程序源码\Android 应用实验源码\界面编程】中“9、单选按钮”目录中下的工程“04RadioButtonExample”：



右键点击“04RadioButtonExample”，选择“run as ->Android Application”，选择对应试验箱设备，点击 ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



22.13.5 实验代码

main.xml

Java Code

```
package cn.com.farsight;

import android.app.Activity;
import android.os.Bundle;
import android.widget.RadioButton;
import android.widget.RadioGroup;

public class Main extends Activity
{
    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
        RadioGroup rg = (RadioGroup)findViewById(R.id.radioGroup1);
        rg.setOnCheckedChangeListener(new RadioGroup.OnCheckedChangeListener()
        {

            public void onCheckedChanged(RadioGroup group, int checkedId)
            {
                // TODO Auto-generated method stub
                Main.this.setTitle("" + ((RadioButton)group.findViewById(checkedId)).getText());
            }
        });
    }
}
```



```
    }  
    });  
}  
}
```

main.xml

```
<?xml version="1.0" encoding="utf-8"?>  
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android" android:orientation="vertical" android:layout_width="fill_parent" android:layout_height="fill_parent">  
    <RadioGroup android:id="@+id/radioGroup1" android:layout_width="wrap_content" android:layout_height="wrap_content">  
        <RadioButton android:text="男"  
" android:layout_width="wrap_content" android:id="@+id/radio0" android:layout_height="wrap_content"  
" android:checked="true"></RadioButton>  
        <RadioButton android:text="女"  
" android:layout_width="wrap_content" android:id="@+id/radio1" android:layout_height="wrap_content"  
"></RadioButton>  
        <RadioButton android:text="其他"  
" android:layout_width="wrap_content" android:id="@+id/radio2" android:layout_height="wrap_content"  
"></RadioButton>  
    </RadioGroup>  
</LinearLayout>
```

22.14 Spinner 实验

22.14.1 实验目的

通过实验学习 android 应用开发中 Spinner 的使用方法。

22.14.2 实验平台

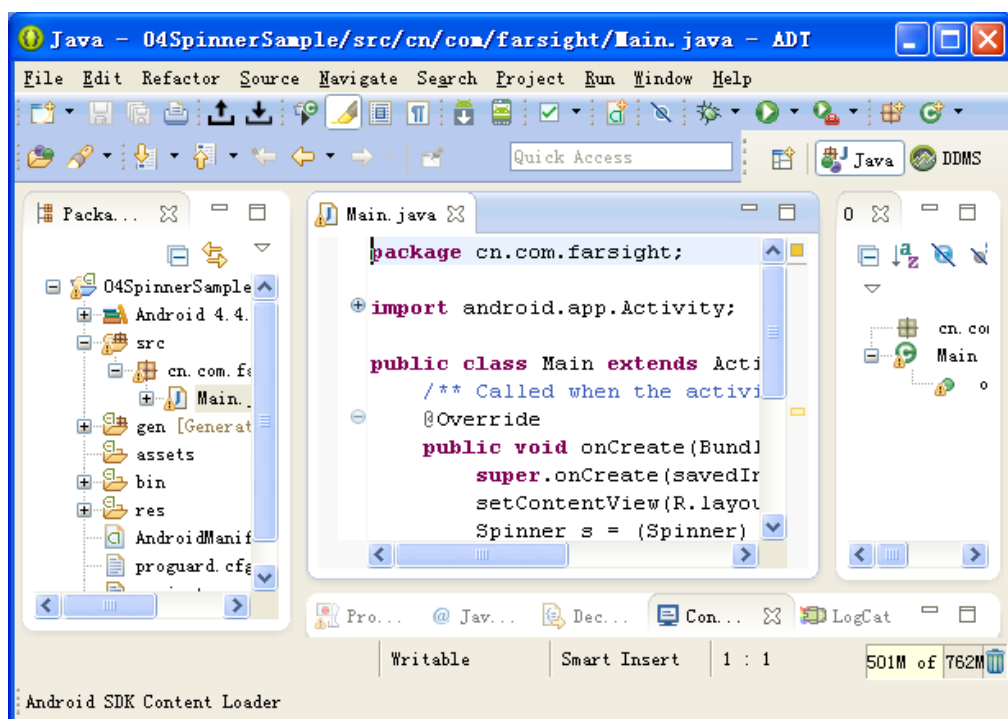
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

22.14.3 实验原理

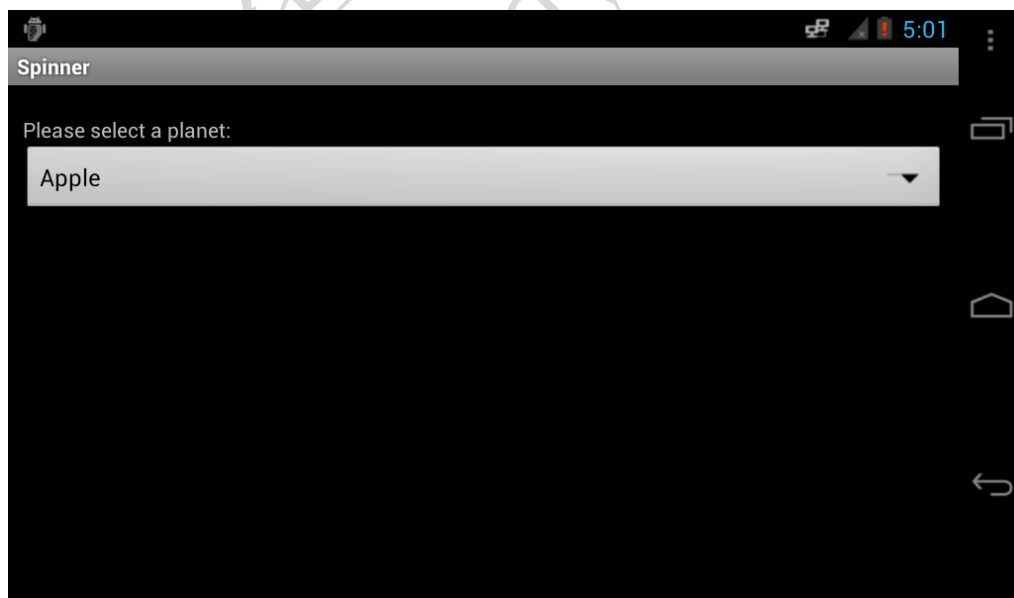
详见“第 15 章用户界面开发-Spinner 部分”。

22.14.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Android 应用实验源码\界面编程】中“26、下拉列表 Spinner 2”目录中下的工程“04SpinnerSample”：



右键点击“04SpinnerSample”，选择“run as -> Android Application”，选择对应试验箱设备，点击ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



22.14.5 实验代码

Main.java



```
1 package cn.com.farsight;
2
3 import android.app.Activity;
4 import android.os.Bundle;
5 import android.widget.AdapterView;
6 import android.widget.Spinner;
7
8 public class Main extends Activity
9 {
10     /** Called when the activity is first created. */
11     @Override
12     public void onCreate(Bundle savedInstanceState)
13     {
14         super.onCreate(savedInstanceState);
15         setContentView(R.layout.main);
16         Spinner s = (Spinner) findViewById(R.id.spinner);
17         ArrayAdapter adapter = ArrayAdapter.createFromResource(this,
18             R.array.mobiles, android.R.layout.simple_spinner_item);
19         /* 设定视图方式 */
20         adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
21
22         /* 设定 Adapter */
23         s.setAdapter(adapter);
24     }
25 }
```

main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android" android:orientation="vertical" android:padding="10dp" android:layout_width="fill_parent" android:layout_height="wrap_content">
    <TextView android:layout_width="fill_parent" android:layout_height="wrap_content" android:layout_marginTop="10dp" android:text="Please select a planet:" />
    <Spinner android:id="@+id/spinner" android:layout_width="fill_parent" android:layout_height="wrap_content" android:drawSelectorOnTop="true" android:prompt="@string/spinner_label" />
</LinearLayout>
```

22.15 ListView 实验

22.15.1 实验目的

通过实验学习 android 应用开发中 ListView 的使用方法。



22.15.2 实验平台

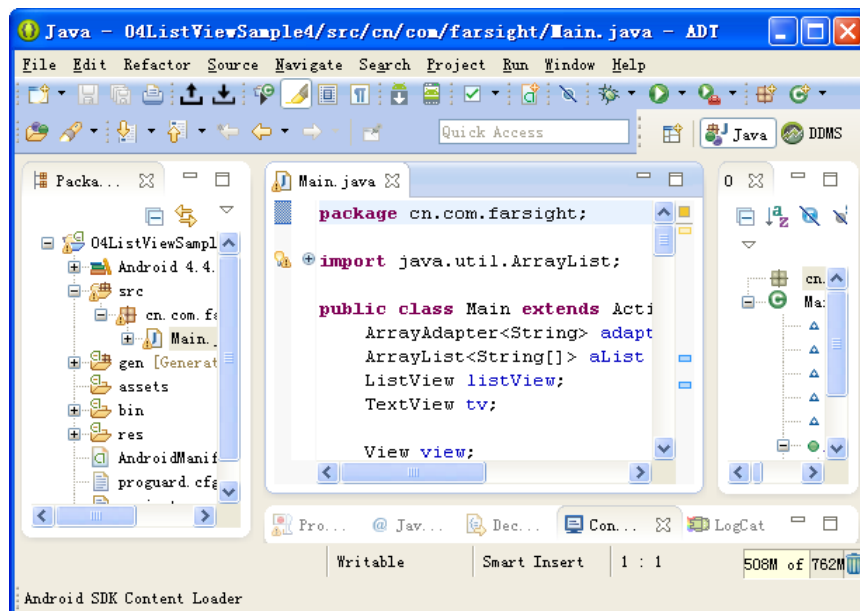
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

22.15.3 实验原理

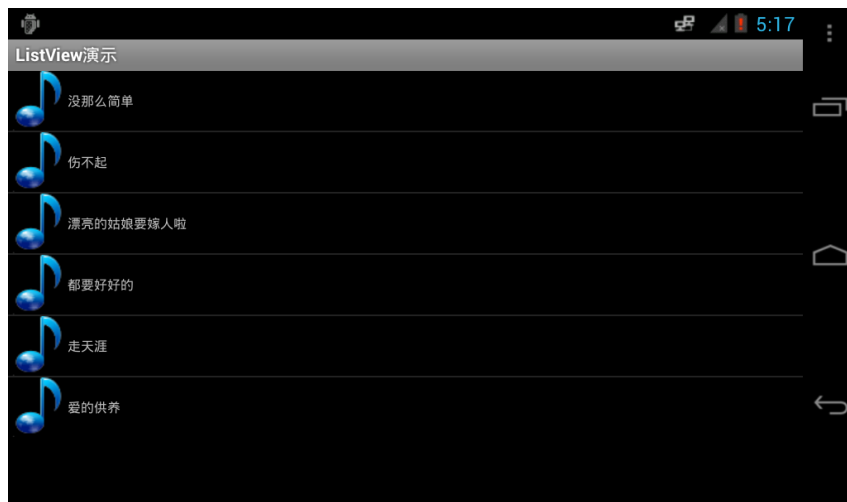
详见“第 15 章用户界面开发-ListView 部分”。

22.15.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Android 应用实验源码\界面编程】中“31、 复杂列表视图”目录中下的工程“04ListViewSample4”：



右键点击“04ListViewSample4”，选择“run as ->Android Application”，选择对应试验箱设备，点击ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



22.15.5 实验代码

main.java

```
1 package cn.com.farsight;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 import android.app.Activity;
7 import android.content.Context;
8 import android.os.Bundle;
9 import android.view.LayoutInflater;
10 import android.view.View;
11 import android.view.ViewGroup;
12 import android.widget.AdapterView;
13 import android.widget.AdapterView.OnItemClickListener;
14 import android.widget.ArrayAdapter;
15 import android.widget.BaseAdapter;
16 import android.widget.ListView;
17 import android.widget.TextView;
18
19 public class Main extends Activity
20 {
21     ArrayAdapter<String> adapter;
22     ArrayList<String[]> aList = new ArrayList<String[]>();
23     ListView listView;
24     TextView tv;
25
26     View view;
27 }
```



```
28  /** Called when the activity is first created. */
29  @Override
30  public void onCreate(Bundle savedInstanceState)
31  {
32      super.onCreate(savedInstanceState);
33      setContentView(R.layout.main);
34      listView = (ListView) findViewById(R.id.listView1);
35      initData();
36      MyAdapter adapter = new MyAdapter(Main.this, 0, aList);
37      listView.setAdapter(adapter);
38      listView.setOnItemClickListener(new AdapterView.OnItemClickListener()
39      {
40          public void onItemClick(AdapterView<?> arg0, View view,
41                                  int position, long id)
42          {
43              Main.this.setTitle("您选中了: " + aList.get(position)[0]);
44          }
45      });
46  }
47
48  private void initData()
49  {
50      aList.add(new String[] { "没那么简单" });
51      aList.add(new String[] { "伤不起" });
52      aList.add(new String[] { "漂亮的姑娘要嫁人啦" });
53      aList.add(new String[] { "都要好好的" });
54      aList.add(new String[] { "走天涯" });
55      aList.add(new String[] { "爱的供养" });
56  }
57
58  private class MyAdapter extends ArrayAdapter<String[]>
59  {
60      private ArrayList<String[]> list;
61
62      public MyAdapter(Context context, int textViewResourceId,
63                      ArrayList<String[]> objects)
64      {
65          super(context, textViewResourceId, objects);
66          this.list = objects;
67          // TODO Auto-generated constructor stub
68      }
69
70      public int getCount()
71      {
```




```
72         return list.size();
73     }
74
75     @Override
76     public long getItemId(int position)
77     {
78         // TODO Auto-generated method stub
79         return position;
80     }
81
82     @Override
83     public View getView(int position, View convertView, ViewGroup parent)
84     {
85         if (convertView == null)
86         {
87             LayoutInflater inflater = (LayoutInflater) getSystemService(Context.LAYOUT_INF
88 LATER_SERVICE);
89             view = inflater.inflate(R.layout.listcontent, null);
90         }
91         else
92         {
93             view = convertView;
94         }
95         tv = (TextView) view.findViewById(R.id.textView1);
96         tv.setText(list.get(position)[0]);
97         return view;
98     }
99 }
100
0
```

22.16 选项菜单实验

22.16.1 实验目的

通过实验学习 android 应用开发中选项菜单的使用方法。

22.16.2 实验平台

FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

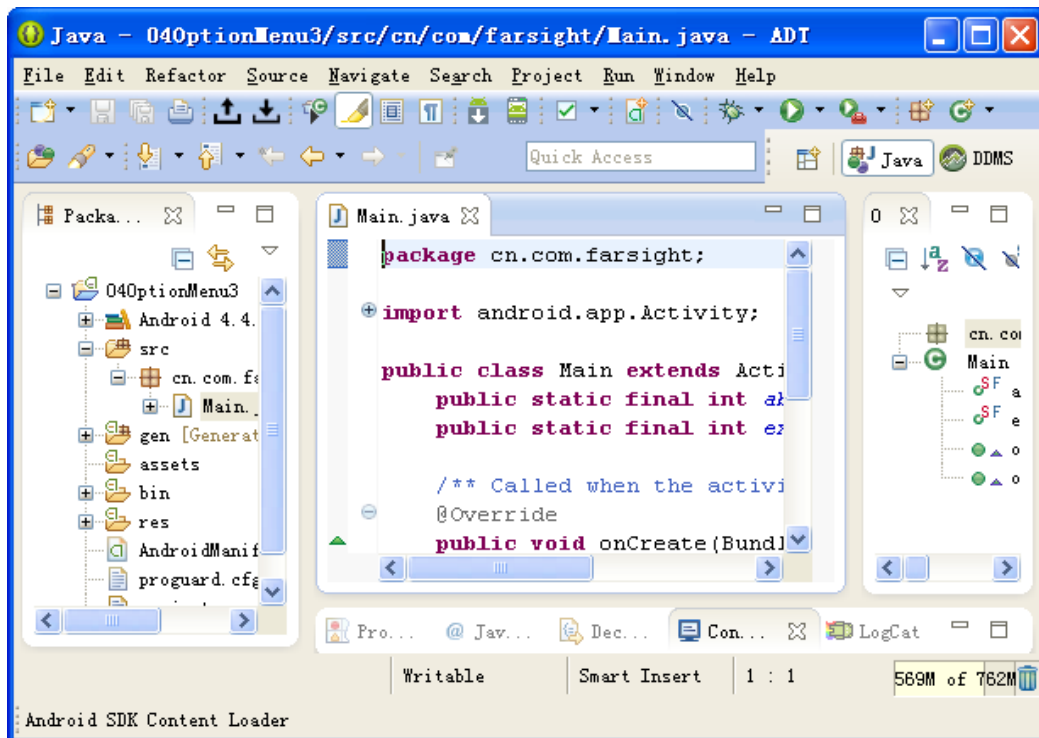
22.16.3 实验原理

详见“第 15 章用户界面开发-选项部分”。

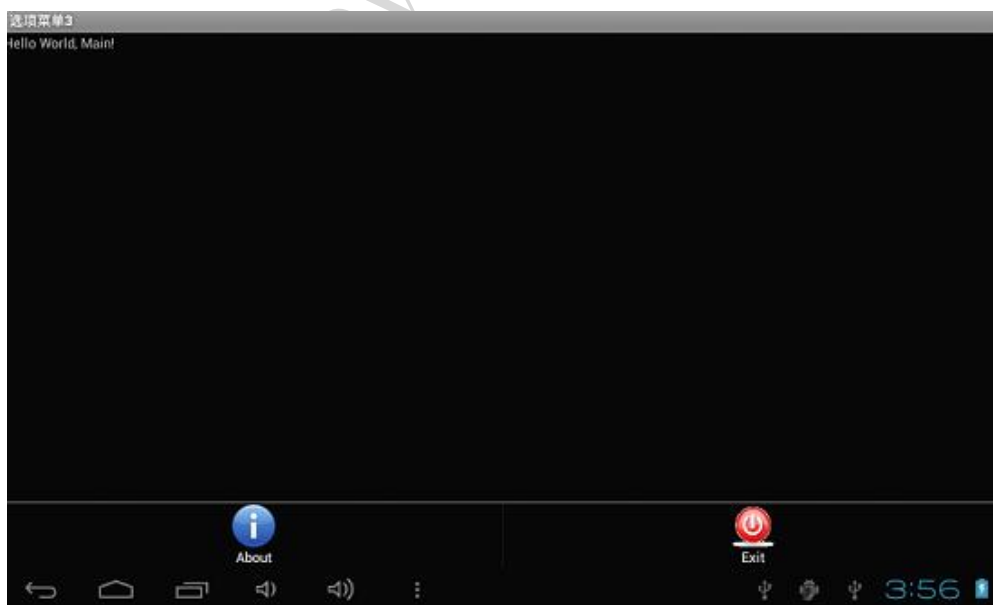


22.16.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Android 应用实验源码\界面编程】中“22、 菜单 3”目录中下的工程“04OptionsMenu3”:



右键点击“04OptionsMenu3”，选择“run as ->Android Application”，选择对应试验箱设备，点击 ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：





22.16.5 实验代码

Main.java

```
1 package cn.com.farsight;
2
3 import android.app.Activity;
4 import android.os.Bundle;
5 import android.view.Menu;
6
7 public class Main extends Activity
8 {
9     public static final int aboutBtnID = Menu.FIRST;
10    public static final int exitBtnID = Menu.FIRST + 1;
11
12    /** Called when the activity is first created. */
13    @Override
14    public void onCreate(Bundle savedInstanceState)
15    {
16        super.onCreate(savedInstanceState);
17        setContentView(R.layout.main);
18    }
19    public boolean onCreateOptionsMenu(Menu menu)
20    {
21        menu.add(0, aboutBtnID, 0, "About").setIcon(R.drawable.about);
22        menu.add(0, exitBtnID, 0, "Exit").setIcon(R.drawable.exit);
23        return true;
24    }
25
26 }
```

22.17 子菜单实验

22.17.1 实验目的

通过实验学习 android 应用开发中子菜单的使用方法。

22.17.2 实验平台

FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

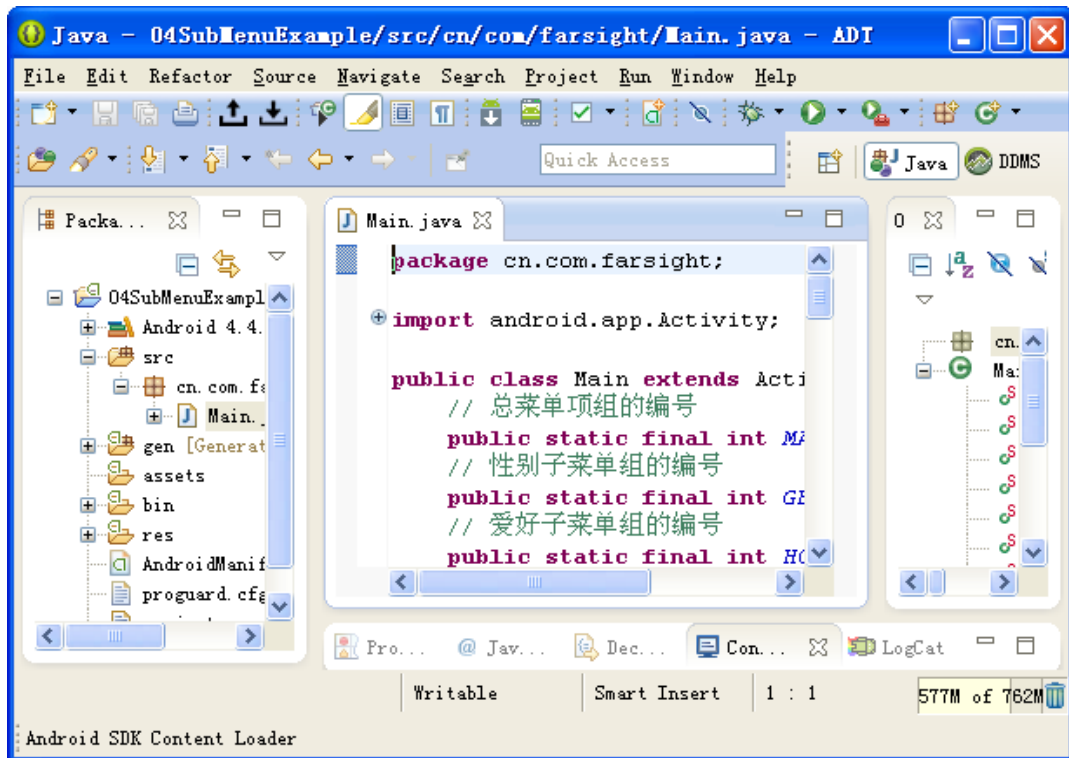
22.17.3 实验原理

详见“第 15 章用户界面开发-子菜单部分”。

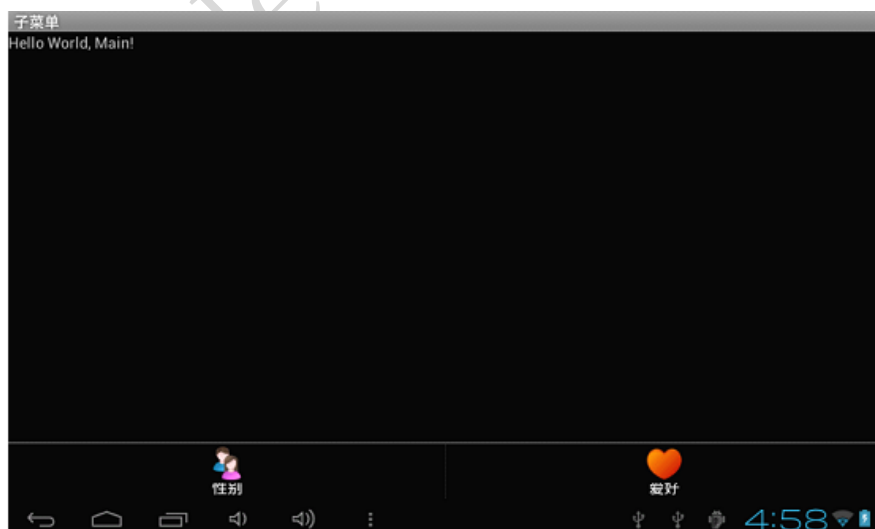


22.17.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Android 应用实验源码\界面编程】中“24、子菜单”目录中下的工程“04SubMenuExample”：



右键点击“04SubMenuExample”，选择“run as ->Android Application”，选择对应试验箱设备，点击ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：





22.17.5 实验代码

Main.java

```
1 import android.app.Activity;
2 import android.os.Bundle;
3 import android.view.Menu;
4 import android.view.MenuItem;
5 import android.view.SubMenu;
6
7 public class Main extends Activity
8 {
9     // 总菜单项组的编号
10    public static final int MAIN_GROUP = 0;
11    // 性别子菜单组的编号
12    public static final int GENDER_GROUP = 1;
13    // 爱好子菜单组的编号
14    public static final int HOBBY_GROUP = 2;
15    // 性别为男的选项编号
16    public static final int MENU_GENDER_MALE = 3;
17    // 性别女的选项编号
18    public static final int MENU_GENDER_FEMALE = 4;
19    // “阅读”编号
20    public static final int MENU_HOBBY_READING = 5;
21    // “旅游”编号
22    public static final int MENU_HOBBY_TRAVELLING = 6;
23    // “游泳”编号
24    public static final int MENU_HOBBY_SWIMMING = 7;
25    // “确定”按钮编号
26    public static final int MENU_OK = 8;
27    // 性别子菜单
28    public static final int MENU_GENDER = 9;
29    // 爱好子菜单
30    public static final int MENU_HOBBY = 10;
31
32    /** Called when the activity is first created. */
33    @Override
34    public void onCreate(Bundle savedInstanceState)
35    {
36        super.onCreate(savedInstanceState);
37        setContentView(R.layout.main);
38    }
39
40    @Override
41    public boolean onCreateOptionsMenu(Menu menu)
```



```
42     {
43         // 构建“性别”子菜单
44         SubMenu subs_gender = menu.addSubMenu(MAIN_GROUP, MENU_GENDER, 0,
45                                                 R.string.gender).setIcon(R.drawable.gender);
46         subs_gender.add(GENDER_GROUP, MENU_GENDER_MALE, 0, R.string.male)
47             .setChecked(true);
48         subs_gender.add(GENDER_GROUP, MENU_GENDER_FEMALE, 0, R.string.female);
49         subs_gender.setGroupCheckable(GENDER_GROUP, true, true);
50         // 构建“爱好”子菜单
51         SubMenu subs_hobbies = menu.addSubMenu(HOBBY_GROUP, MENU_HOBBY, 0,
52                                                 R.string.hobby).setIcon(R.drawable.favorite);
53         subs_hobbies.add(HOBBY_GROUP, MENU_HOBBY_READING, 0, R.string.reading)
54             .setCheckable(true).setChecked(true);
55         subs_hobbies.add(HOBBY_GROUP, MENU_HOBBY_TRAVELLING, 0,
56                         R.string.travelling).setCheckable(true);
57         subs_hobbies
58             .add(HOBBY_GROUP, MENU_HOBBY_SWIMMING, 0, R.string.swimming)
59             .setCheckable(true);
60
61         return super.onCreateOptionsMenu(menu);
62     }
63
64     @Override
65     public boolean onOptionsItemSelected(MenuItem menuItem)
66     {
67         // 子菜单选项点击事件的处理，略
68         return true;
69     }
}
```

22.18 上下文菜单实验

22.18.1 实验目的

通过实验学习 android 应用开发中“上下文菜单”的使用方法。

22.18.2 实验平台

FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

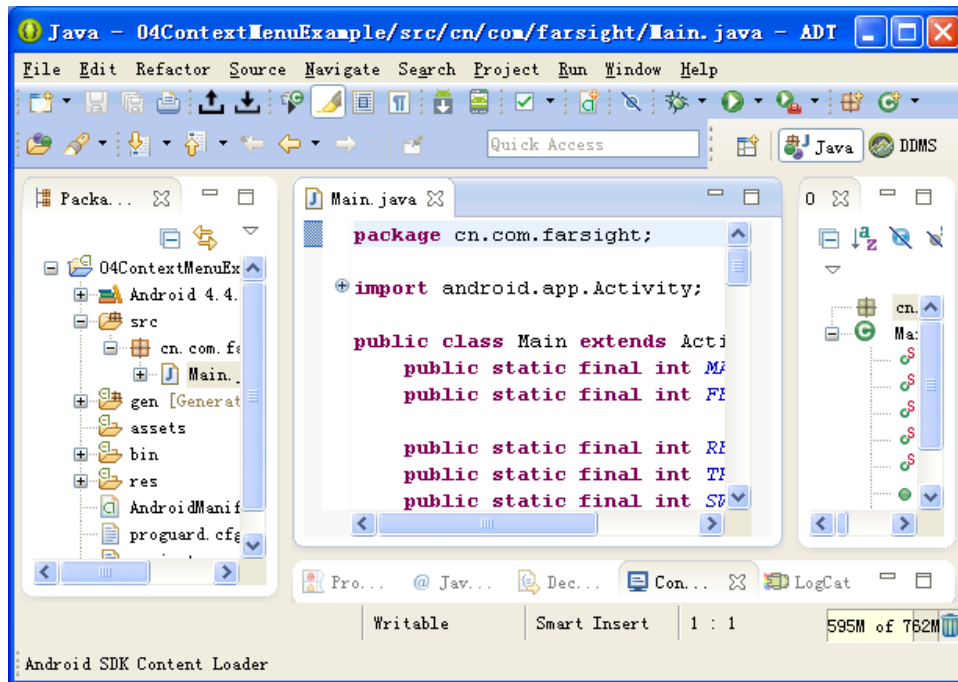
22.18.3 实验原理

详见“第 15 章用户界面开发-上下文菜单部分”。



22.18.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Android 应用实验源码\界面编程】中“23、上下文菜单”目录中下的工程“04ContextMenuExample”:



右键点击“04ContextMenuExample”，选择“run as ->Android Application”，选择对应试验箱设备，点击 ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：





22.18.5 实验代码

Main.java

```
1 Main.java
2 package cn.com.farsight;
3
4 import android.app.Activity;
5 import android.os.Bundle;
6 import android.view.ContextMenu;
7 import android.view.MenuItem;
8 import android.view.View;
9 import android.widget.EditText;
10
11 public class Main extends Activity
12 {
13     public static final int MALE = 1;
14     public static final int FEMALE = 2;
15
16     public static final int READING = 3;
17     public static final int TRAVELLING = 4;
18     public static final int SWIMMING = 5;
19
20     @Override
21     public void onCreate(Bundle savedInstanceState)
22     {
23         super.onCreate(savedInstanceState);
24         setContentView(R.layout.main);
25         // 需为 EditText 注册 ContextMenu, 否则当长按输入框时, 只能弹出系统自带的
26         // Main 中的 onCreateContextMenu 未执行到
27         this.registerForContextMenu(findViewById(R.id.genderEdit));
28         this.registerForContextMenu(findViewById(R.id.hobbyEdit));
29     }
30
31     /*
32      * 只有当组件注册了才调用此方法, 当长按 2 秒后会弹出菜单项 不支持快捷键, 菜单选项不能附带图标
33      */
34     @Override
35     public void onCreateContextMenu(ContextMenu menu, View view,
36                                     ContextMenu.ContextMenuInfo menuInfo)
37     {
38         // 清除系统所有系统自带的菜单项
39         menu.clear();
40         if (view == findViewById(R.id.genderEdit))
41         {
```




```
42         menu.setHeaderIcon(R.drawable.gender);
43         menu.setHeaderTitle(R.string.gender);
44         // 如果该菜单项里面还未添加菜单, 此时菜单项是隐藏的
45         // 添加了 MALE 后会显示出来
46         menu.add(0, MALE, 0, R.string.male);
47         menu.add(0, FEMALE, 0, R.string.female);
48
49     }
50     else if (view == findViewById(R.id.hobbyEdit))
51     {
52         menu.setHeaderIcon(R.drawable.favorite);
53         menu.setHeaderTitle(R.string.hobby);
54         menu.add(0, READING, 0, R.string.reading);
55         menu.add(0, TRAVELLING, 0, R.string.travelling);
56         menu.add(0, SWIMMING, 0, R.string.swimming);
57     }
58 }
59
60 // 上下文菜单选项被选中处理
61 @Override
62 public boolean onOptionsItemSelected(MenuItem menuItem)
63 {
64     EditText editText = null;
65     switch (menuItem.getItemId())
66     {
67     case MALE:
68     case FEMALE:
69         editText = (EditText) this.findViewById(R.id.genderEdit);
70         break;
71     case READING:
72     case SWIMMING:
73     case TRAVELLING:
74         editText = (EditText) this.findViewById(R.id.hobbyEdit);
75         break;
76     }
77     editText.setText(menuItem.getTitle());
78     return true;
79 }
80 }
```



第 23 章 组件间通讯实验

23.1 实验目的

通过实验学习 android 应用开发中“上下文菜单”的使用方法。

23.2 实验平台

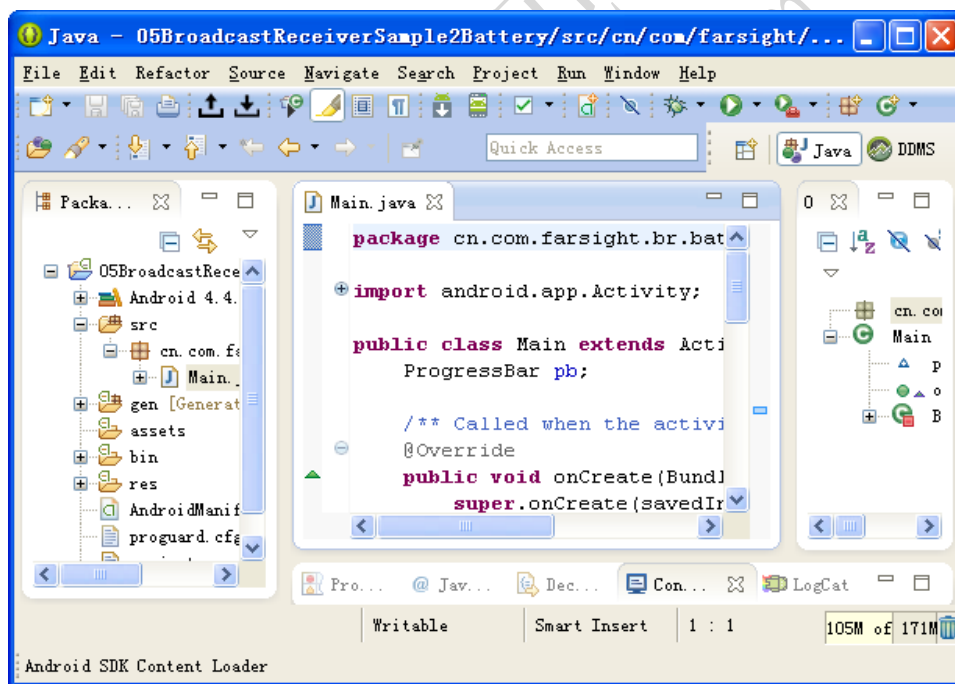
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

23.3 实验原理

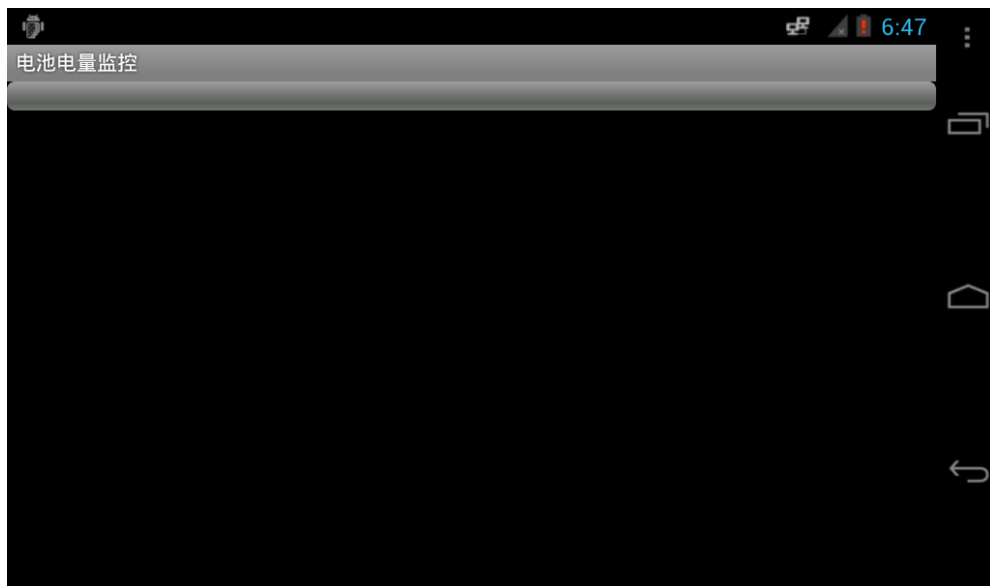
详见“第 16 章组件间通讯部分”。

23.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Android 应用实验源码\常用组件】中“9、接收系统消息”目录中下的工程“05BroadcastReceiverSample2Battery”：



右键点击“05BroadcastReceiverSample2Battery”，选择“run as ->Android Application”，选择对应试验箱设备，点击 ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



23.5 实验代码

Main.java

```
1 package cn.com.farsight.br.battery;
2
3 import android.app.Activity;
4 import android.content.BroadcastReceiver;
5 import android.content.Context;
6 import android.content.Intent;
7 import android.content.IntentFilter;
8 import android.os.Bundle;
9 import android.widget.ProgressBar;
10
11 public class Main extends Activity
12 {
13     ProgressBar pb;
14
15     /** Called when the activity is first created. */
16     @Override
17     public void onCreate(Bundle savedInstanceState)
18     {
19         super.onCreate(savedInstanceState);
20         setContentView(R.layout.main);
21         pb = (ProgressBar) findViewById(R.id.progressBar1);
22         registerReceiver(new BatteryReceiver(), new IntentFilter(
23             Intent.ACTION_BATTERY_CHANGED));
24     }
25 }
```



```
26 private class BatteryReceiver extends BroadcastReceiver
27 {
28     int level = 0;
29     int scale = -1;
30
31     @Override
32     public void onReceive(Context context, Intent intent)
33     {
34         // TODO Auto-generated method stub
35         if (scale == -1)
36         {
37             scale = intent.getIntExtra("scale", 0);
38             pb.setMax(scale);
39         }
40
41         level = intent.getIntExtra("level", 0);
42         pb.setProgress(level);
43     }
44
45 }
```

华清
dev.hayl



第 24 章 数据存储实验

24.1 SharedPreferences 实验

24.1.1 实验目的

通过实验学习 android 应用开发中“SharedPreferences”的使用方法。

24.1.2 实验平台

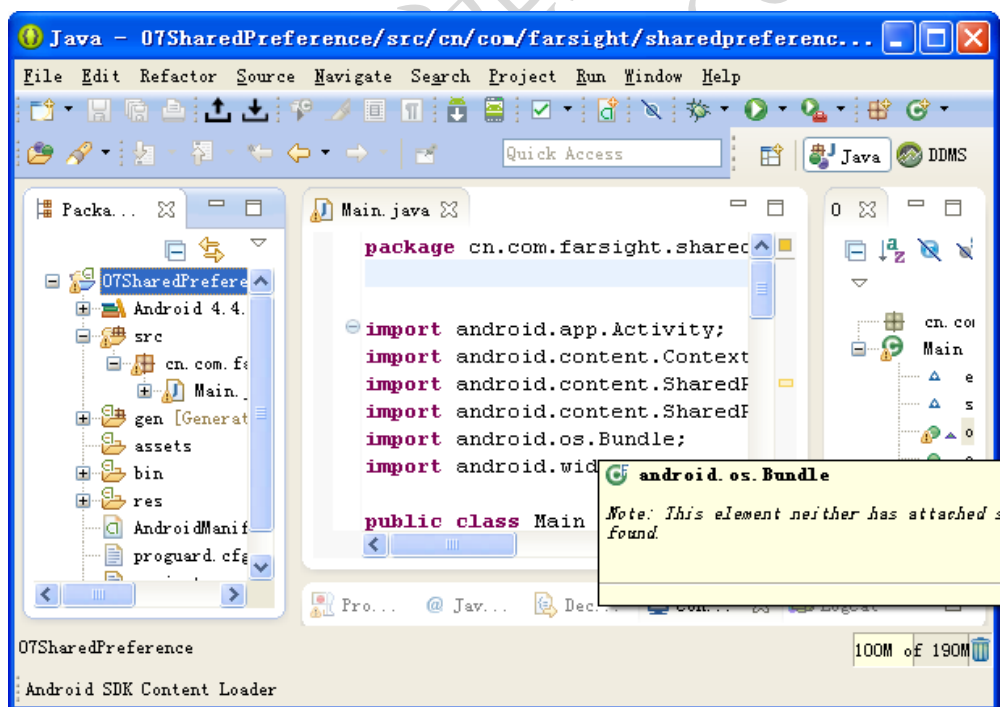
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

24.1.3 实验原理

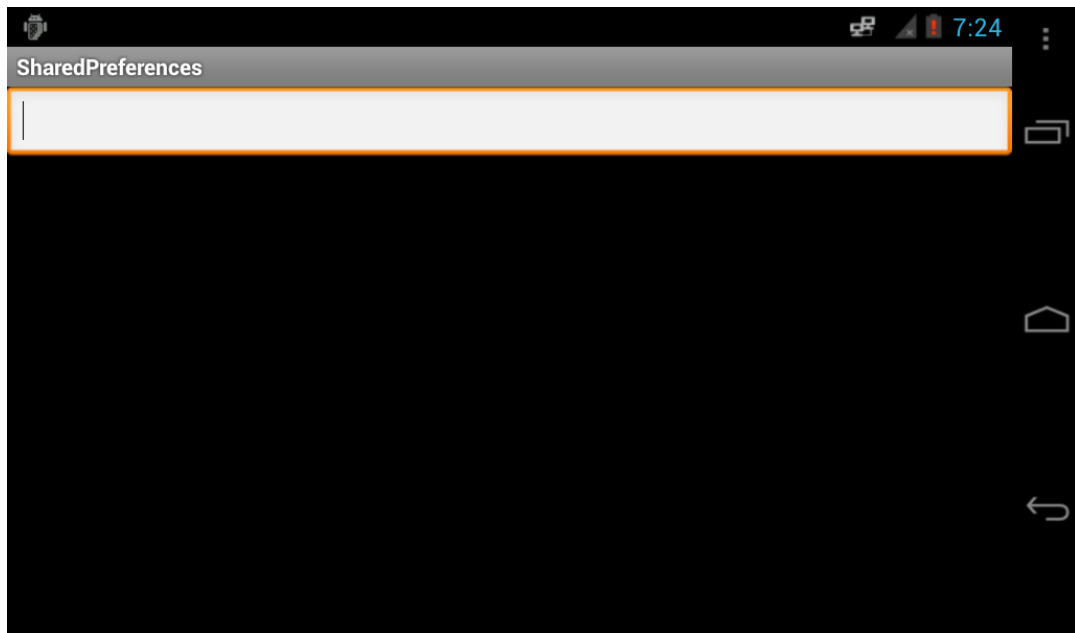
详见“第 17 章数据存储与访问-SharedPreferences”。

24.1.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Android 应用实验源码\数据存取】中“2、 使用 SharedPreferences”目录中下的工程“07SharedPreferences”：



右键点击“07SharedPreferences”，选择“run as ->Android Application”，选择对应试验箱设备，点击ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



24.1.5 实验代码

Main.java

```
1  package cn.com.farsight.sharedpreferences;
2
3
4  import android.app.Activity;
5  import android.content.Context;
6  import android.content.SharedPreferences;
7  import android.content.SharedPreferences.Editor;
8  import android.os.Bundle;
9  import android.widget.EditText;
10
11 public class Main extends Activity
12 {
13     EditText edit;
14     SharedPreferences sp;
15
16     /** Called when the activity is first created. */
17     @Override
18     public void onCreate(Bundle savedInstanceState)
19     {
20         super.onCreate(savedInstanceState);
21         setContentView(R.layout.main);
22         edit = (EditText) findViewById(R.id.editText1);
23     }
24 }
```



```
24 // 传入 Preferences 文件名和打开模式
25 sp = getSharedPreferences("init", Context.MODE_WORLD_READABLE);
26 // 试图从配置文件读取数据
27 String username = sp.getString("username", null);
28 // 如果有数据，则显示到输入框
29 if (username != null)
30 {
31     // 将 EditText 的内容设置为上一次退出时保存在 Preferences 文件中的字符串
32     edit.setText(username);
33 }
34 else
35 {
36     edit.setText("请输入用户名");
37 }
38 }
39
40 @Override
41 public void onDestroy()
42 {
43     // 在 onDestroy()方法中，实现了将当前
44     // EditText 中的字符串存储到 Preferences 文件
45     Editor editor = sp.edit();
46
47     editor.putString("username", String.valueOf(edit.getText()));
48     editor.commit();
49     super.onDestroy();
50 }
51 }
```

24.2 文件存储实验

24.2.1 实验目的

通过实验学习 android 应用开发中“文件存储”的使用方法。

24.2.2 实验平台

FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

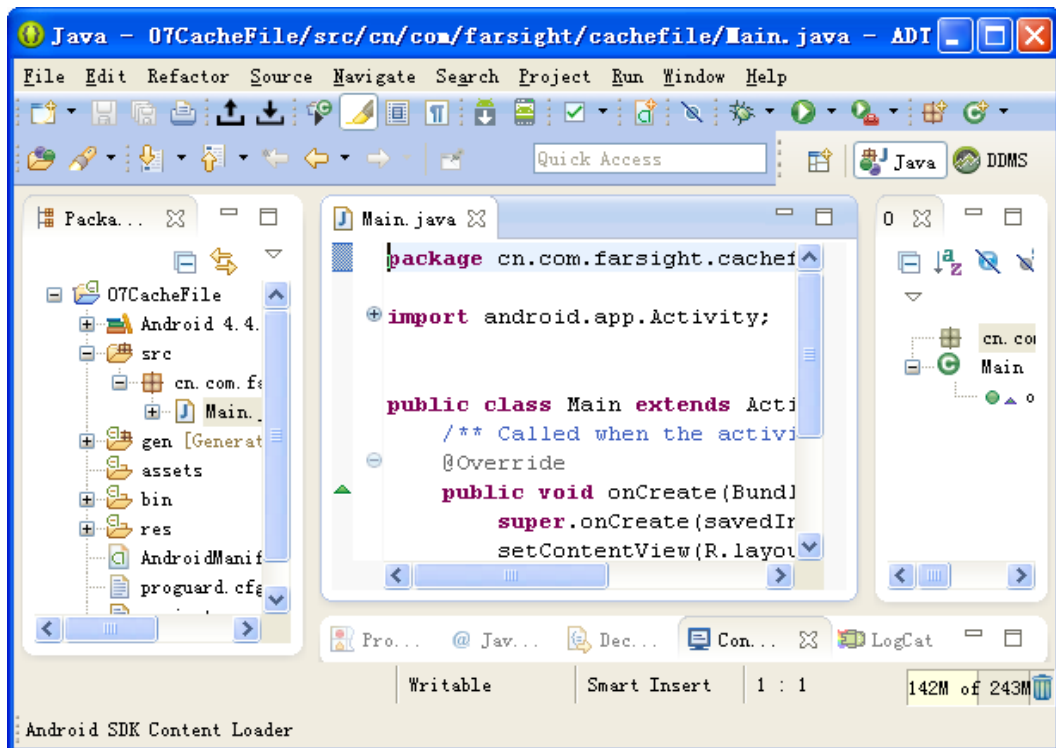
24.2.3 实验原理

详见“第 17 章数据存储与访问-文件存储”。

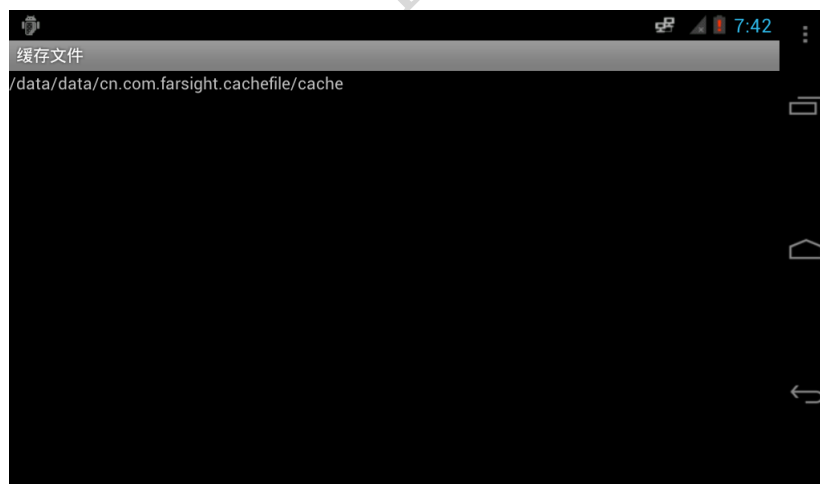
24.2.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Android 应用实验源码\数据存取】

中“1、取得 Android 的缓存文件夹”目录中下的工程“07CacheFile”：



右键点击“07CacheFile”，选择“run as -> Android Application”，选择对应试验箱设备，点击 ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



24.2.5 实验代码

Main.java

```
1 package cn.com.farsight.cachefile;
2
3 import android.app.Activity;
```




```
4 import android.os.Bundle;
5 import android.widget.TextView;
6
7 import java.io.File;
8
9
10 public class Main extends Activity
11 {
12     /** Called when the activity is first created. */
13     @Override
14     public void onCreate(Bundle savedInstanceState)
15     {
16         super.onCreate(savedInstanceState);
17         setContentView(R.layout.main);
18         File f = this.getCacheDir();
19         String path = f.getParent() + java.io.File.separator + f.getName();
20         TextView tv = (TextView) findViewById(R.id.textView1);
21         tv.setText(path);
22     }
23 }
```

24.3 SQLite 数据库实验

24.3.1 实验目的

通过实验学习 android 应用开发中“SQLite 数据库”的使用方法。

24.3.2 实验平台

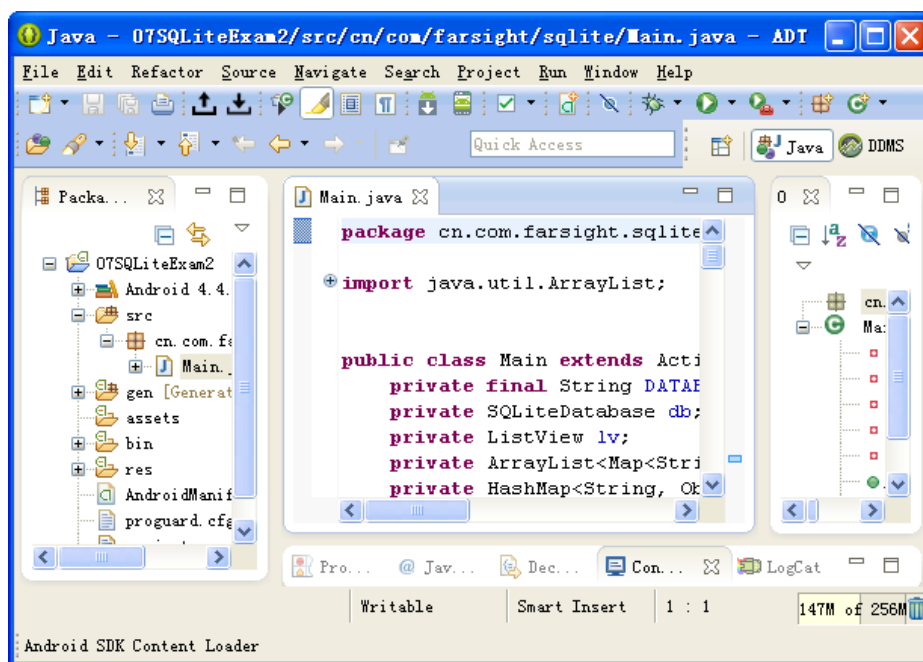
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

24.3.3 实验原理

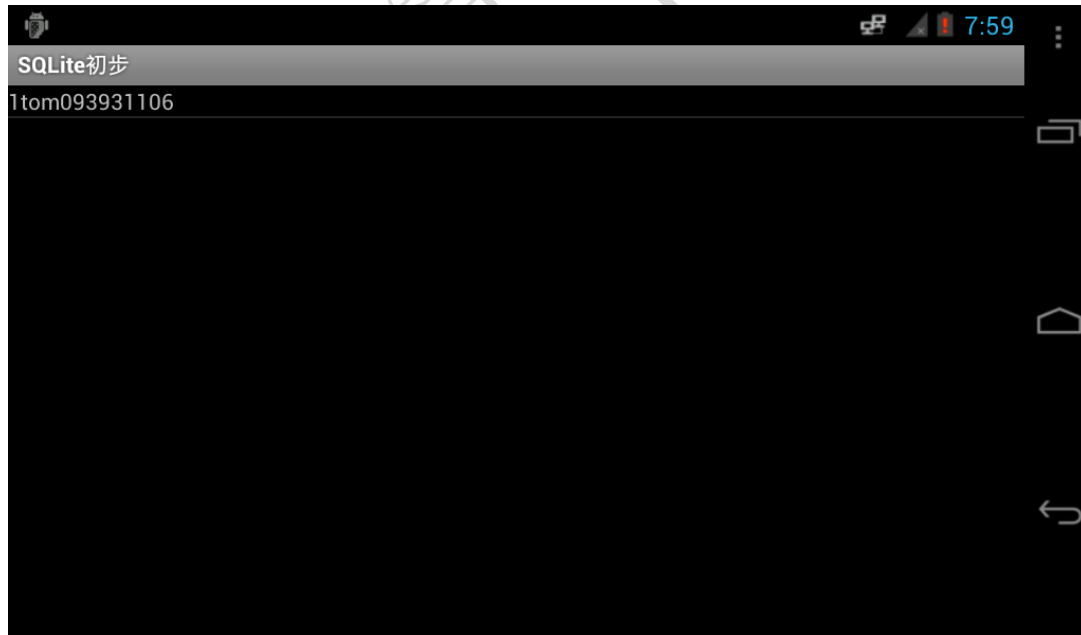
详见“第 17 章数据存储与访问-数据库存储”。

24.3.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Android 应用实验源码\数据存取】中“3、 使用 SQLite 数据库”目录中下的工程“07SQLiteExam2”：



右键点击“07SQLiteExam2”，选择“run as ->Android Application”，选择对应试验箱设备，点击 ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



24.3.5 实验代码

Main.java

```
package cn.com.farsight.sqlite;
```



```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Map;

import android.app.Activity;
import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteDatabase.CursorFactory;
import android.database.sqlite.SQLiteOpenHelper;
import android.os.Bundle;
import android.widget.ListView;
import android.widget.SimpleAdapter;

public class Main extends Activity
{
    private final String DATABASE_NAME = "school";
    private SQLiteDatabase db;
    private ListView lv;
    private ArrayList<Map<String, Object>> data = new ArrayList<Map<String, Object>>();
    private HashMap<String, Object> mymap;

    @Override
    public void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        // setContentView(R.layout.main);
        lv = new ListView(this);

        MyDatabaseHelper myDBHelper = new MyDatabaseHelper(this, DATABASE_NAME,
            null, 3);
        db = myDBHelper.getWritableDatabase();
        // 初始化数据
        initData("tom", "09393", "1106");
        // initData("jerry", "843293", "1107");
        // initData("harry", "232342", "1108");

        // 获取 STUDENTINFO 数据表中的数据
        String sql = "SELECT * FROM STUDENTINFO;";

        Cursor result = db.rawQuery(sql, null);
```



```

data.clear();
//将数据放到 Adapter 中
while (!result.isLast())
{
    result.moveToNext();
    mymap = new HashMap<String, Object>();
    for (int i = 0; i < result.getColumnCount(); i++)
    {
        mymap.put(result.getColumnName(i), result.getString(i));
    }
    data.add(mymap);
}
SimpleAdapter adapter = new SimpleAdapter(this, data,
    R.layout.listcontent, new String[] { "ID", "NAME", "PHONE",
        "CLASS"
        }, new int[] { R.id.textView1, R.id.textView2
        }, new int[] { R.id.textView3, R.id.textView4
        });

/*另一种采用 SimpleCursorAdapter 的方法
//但需要注意，使用 SimpleCursorAdapter 必须有字段_id，且 SimpleCursorAdapter 会把
//所有字段都解释成小写
SimpleCursorAdapter adapter = new SimpleCursorAdapter(this,R.layout.listcontent,result,n
ew String[] { "ID", "NAME", "PHONE",
    "CLASS" }, new int[] { R.id.textView1, R.id.textView2,
        R.id.textView3, R.id.textView4 });
*/
lv.setAdapter(adapter);
setContentView(lv);

result.close();
db.close();
//自己在 ListView 上加上上下文菜单，并且做删除/修改的处理
}

// 初始化表格数据
private void initData(String name, String phone, String sclass)
{
    ContentValues values = new ContentValues();
    values.put("NAME", name);
    values.put("PHONE", phone);

```



```
values.put("CLASS", sclass);
db.insert("STUDENTINFO", "ID", values);
}

class MyDatabaseHelper extends SQLiteOpenHelper
{
    public String DATABASE_TABLE = "STUDENTINFO";
    public final String DB_CREATE_TABLE = "CREATE TABLE " + DATABASE_TABLE
        + "( " + "ID      INTEGER      NOT NULL,"
        + "NAME   CHAR(20)  NOT NULL," + "PHONE  CHAR(20)
        + "CLASS  CHAR(50)," + "PRIMARY KEY(ID) );";

    public MyDatabaseHelper(Context context, String name,
        CursorFactory factory, int version)
    {
        super(context, name, factory, version);
    }

    @Override
    public void onCreate(SQLiteDatabase db)
    {
        db.execSQL(DB_CREATE_TABLE);
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion)
    {
        // TODO Auto-generated method stub
    }
}
}
```

24.4 数据共享实验

24.4.1 实验目的

通过实验学习 android 应用开发中“数据共享”的使用方法。

24.4.2 实验平台

FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

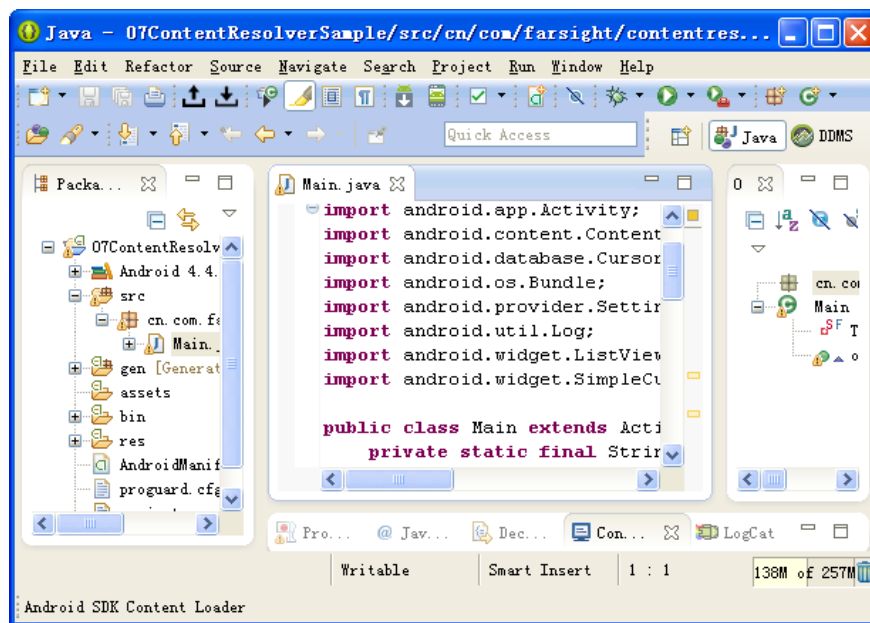


24.4.3 实验原理

详见“第 17 章数据存储与访问-数据共享”。

24.4.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Android 应用实验源码\数据存取】中“4、 使用 ContentResolver 来读取设置”目录中下的工程“07ContentResolverSample”:



右键点击“07ContentResolverSample”，选择“run as -> Android Application”，选择对应试验箱设备，点击 ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：

ContentResolver2	
volume_music	11
volume_ring	5
volume_system	7
volume_voice	4
volume_alarm	6
volume_notification	5
mode_ringer	2
vibrate_on	10
mode_ringer_streams_affected	166



24.4.5 实验代码

Main.java

```
1 package cn.com.farsight.contentresolver;
2
3 import android.app.Activity;
4 import android.content.ContentResolver;
5 import android.database.Cursor;
6 import android.os.Bundle;
7 import android.provider.Settings;
8 import android.util.Log;
9 import android.widget.ListView;
10 import android.widget.SimpleCursorAdapter;
11
12 public class Main extends Activity
13 {
14     private static final String TAG = "ContentUserDemo";
15
16     /** Called when the activity is first created. */
17     @Override
18     public void onCreate(Bundle savedInstanceState)
19     {
20         super.onCreate(savedInstanceState);
21         setContentView(R.layout.main);
22
23         // Get content provider and cursor
24         ContentResolver cr = getContentResolver();
25         Cursor cursor = cr.query(Settings.System.CONTENT_URI, null, null, null,
26                                 null);
27
28         // Let activity manage the cursor
29         startManagingCursor(cursor);
30         Log.d(TAG, "cursor.getCount()=" + cursor.getCount());
31         // Get the list view
32         ListView listView = (ListView) findViewById(R.id.listView);
33         String[] from = { Settings.System.NAME, Settings.System.VALUE };
34         int[] to = { R.id.textName, R.id.textValue };
35         SimpleCursorAdapter adapter = new SimpleCursorAdapter(this,
36             R.layout.row, cursor, from, to);
37         listView.setAdapter(adapter);
38     }
39 }
```



第 25 章 多媒体实验

25.1 实验目的

通过实验学习 android 应用开发中“VideoView”的使用方法。

25.2 实验平台

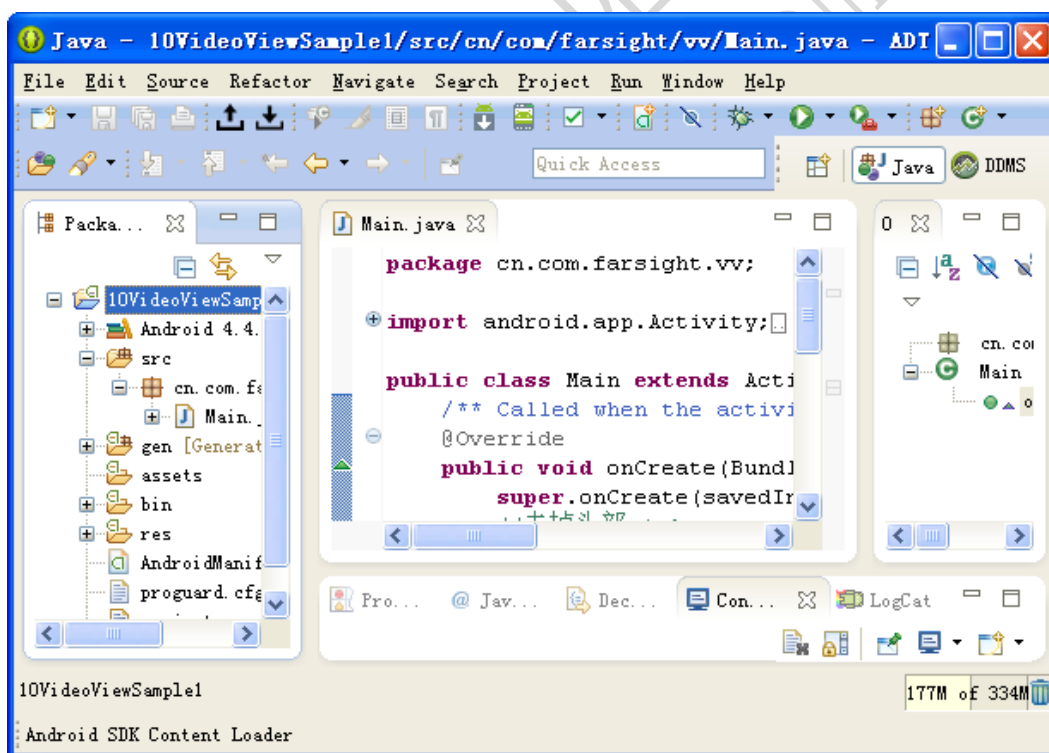
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

25.3 实验原理

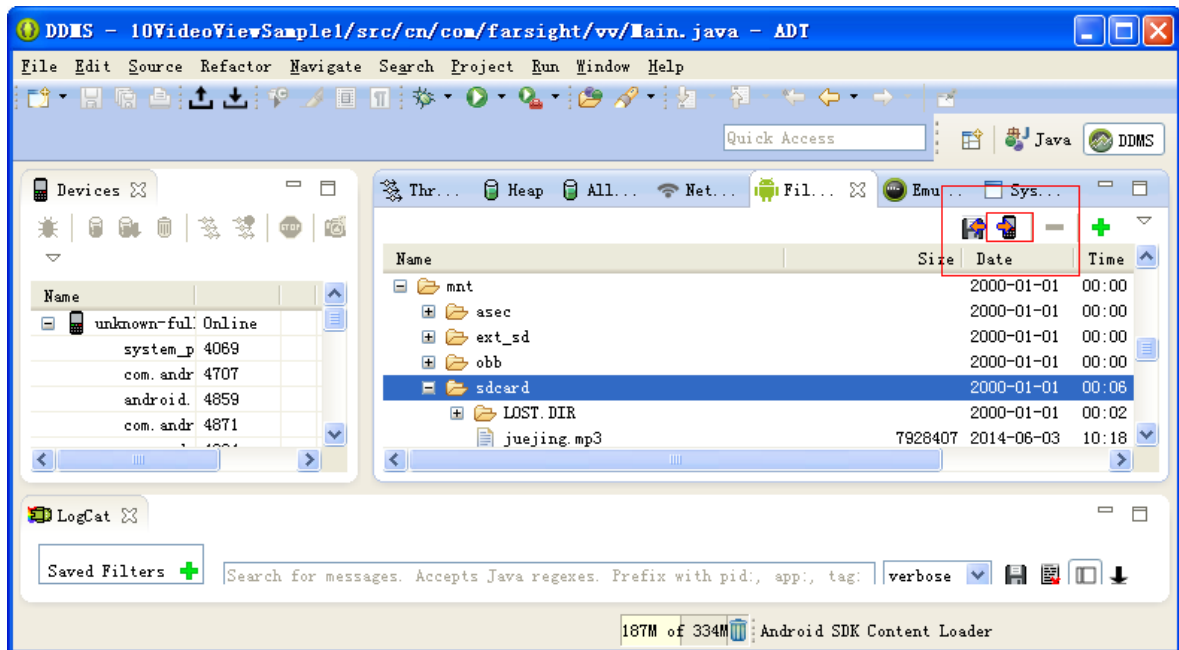
详见“第 18 章多媒体开发部分”。

25.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Android 应用实验源码\多媒体开发】中“7、 视频播放”目录中下的工程“10VideoViewSample1”：



将【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Android 应用实验源码\多媒体开发】中“7、 视频播放”目录中下的“test_mv.mp4”用 DDMS 推送到 android 文件系统中的/mnt/sdcard 目录下：



右键点击“10VideoViewSample1”，选择“run as -> Android Application”，选择对应试验箱设备，点击ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



25.5 实验代码

Main.java

```
1 package cn.com.farsight.vv;
2
```



```
3 import android.app.Activity;
4 import android.content.pm.ActivityInfo;
5 import android.os.Bundle;
6 import android.view.Window;
7 import android.view.WindowManager;
8 import android.widget.MediaController;
9 import android.widget.VideoView;
10
11 public class Main extends Activity
12 {
13     /** Called when the activity is first created. */
14     @Override
15     public void onCreate(Bundle savedInstanceState)
16     {
17         super.onCreate(savedInstanceState);
18         //去掉头部 title
19         requestWindowFeature(Window.FEATURE_NO_TITLE);
20         //设置全屏
21         getWindow().setFlags(WindowManager.LayoutParams.FLAG_FORCE_NOT_FULLSCREEN,
22                               WindowManager.LayoutParams.FLAG_FORCE_NOT_FULLSCREEN);
23         setContentView(R.layout.main);
24         VideoView vv = (VideoView)findViewById(R.id.videoView1);
25         vv.setVideoPath("/mnt/sdcard/test_mv.mp4");
26         //设置横屏
27         setRequestedOrientation(ActivityInfo.SCREEN_ORIENTATION_LANDSCAPE);
28         //设置媒体控制条
29         vv.setMediaController(new MediaController(this));
30         vv.start();
31         vv.requestFocus();
32
33     }
34 }
```



第 26 章 Android 图形图像实验

26.1 实验目的

通过实验学习 android 应用开发中“Matrix”的使用方法。

26.2 实验平台

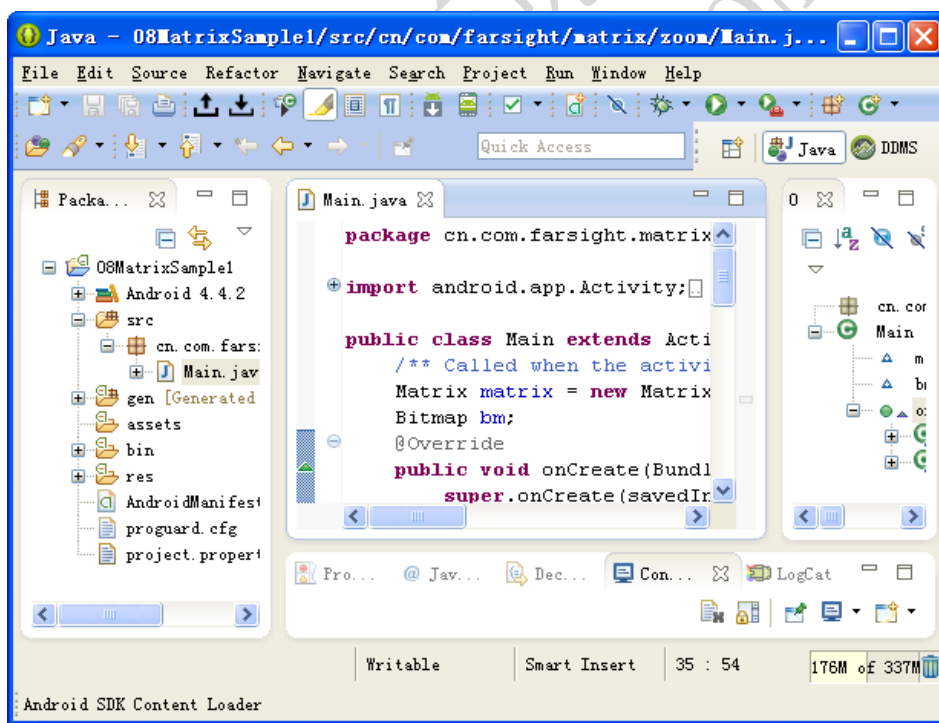
FS6818 平台，Android5.1 系统环境，android eclipse 开发环境。

26.3 实验原理

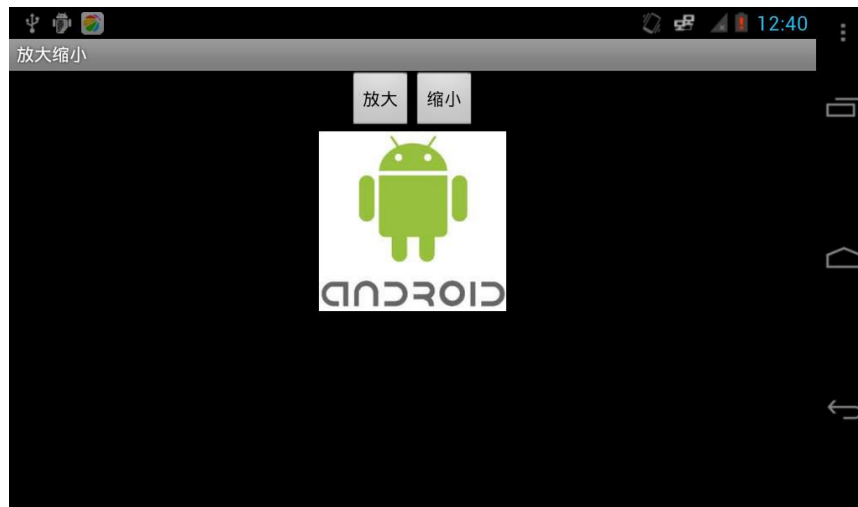
详见“第 20 章 Android 中的图形图像部分”。

26.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Android 应用实验源码\图形图像设计】中“7、使用 Matix 来实现图片缩放”目录中下的工程“08MatrixSample1”：



右键点击“08MatrixSample1”，选择“run as ->Android Application”，选择对应试验箱设备，点击ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



26.5 实验代码

Main.xml

```
package cn.com.farsight.matrix.zoom;

import android.app.Activity;
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.graphics.Matrix;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.ImageView;

public class Main extends Activity
{
    /** Called when the activity is first created. */
    Matrix matrix = new Matrix();
    Bitmap bm;
    @Override
    public void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
        final ImageView iv = (ImageView)findViewById(R.id.imageView1);
        Button btn1 = (Button)findViewById(R.id.button1);
        bm = BitmapFactory.decodeResource(getResources(), R.drawable.android1);
        btn1.setOnClickListener(new View.OnClickListener()
        {
```



```
public void onClick(View v)
{
    //Bitmap bm = BitmapFactory.decodeResource(getResources(), R.drawable.android1);

    matrix.postScale(1.1f, 1.1f);
    Bitmap bitmap = Bitmap.createBitmap(bm, 0, 0,
                                         bm.getWidth(), bm.getHeight(), matrix, true)
;

    iv.setImageBitmap(bitmap);
}
});
Button btn2 = (Button)findViewById(R.id.button2);
btn2.setOnClickListener(new View.OnClickListener()
{

    public void onClick(View v)
    {
        //Bitmap bm = BitmapFactory.decodeResource(getResources(), R.drawable.android1);

        matrix.postScale(0.9f, 0.9f);
        Bitmap bitmap = Bitmap.createBitmap(bm, 0, 0,
                                             bm.getWidth(), bm.getHeight(), matrix, true)
;

        iv.setImageBitmap(bitmap);
    }
});
}
}
```



第 27 章 网络编程实验

27.1 实验目的

通过实验学习 android 应用开发中“网络编程”的使用方法。

27.2 实验平台

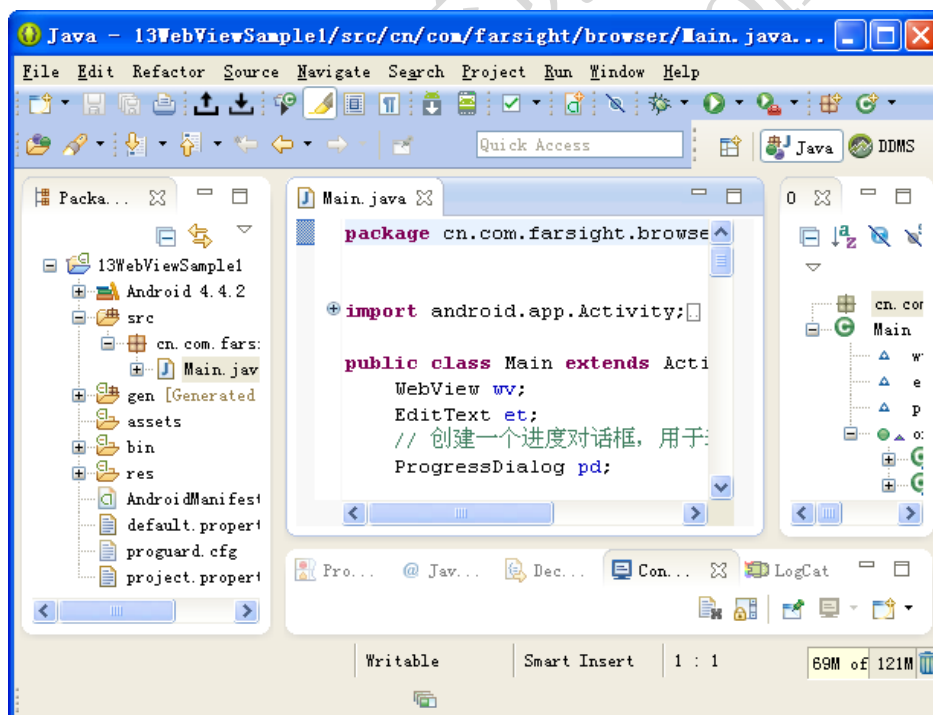
FS6818 平台（插入 wifi 模块联网），Android5.1 系统环境，android eclipse 开发环境。

27.3 实验原理

详见“第 22 章 Android 网络编程部分”。

27.4 实验步骤

导入【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Android 应用实验源码\网络编程】中“2、 访问网页”目录中下的工程“13WebViewSample1”：



右键点击“13WebViewSample1”，选择“run as ->Android Application”，选择对应试验箱设备，点击 ok，apk 文件会安装至 FS6818 开发板的 android 文件系统中，下面是程序运行效果图：



输入完整路径，例如 <http://www.baidu.com>

可获取对应的网页

27.5 实验代码

Main.java

```
1 package cn.com.farsight.browser;
2
3
4 import android.app.Activity;
5 import android.os.Bundle;
6 import android.widget.EditText;
7 import android.widget.Button;
8 import android.webkit.WebView;
9 import android.view.View;
10 import android.webkit.WebViewClient;
11 import android.graphics.Bitmap;
12 import android.app.ProgressDialog;
13 import android.view.inputmethod.InputMethodManager;
14
15 public class Main extends Activity
16 {
17     WebView wv;
18     EditText et;
19     // 创建一个进度对话框，用于表示网页是否已经下载完
20     ProgressDialog pd;
21
22     /** Called when the activity is first created. */
23     @Override
```



```
24 public void onCreate(Bundle savedInstanceState)
25 {
26     super.onCreate(savedInstanceState);
27     setContentView(R.layout.main);
28
29     et = (EditText) findViewById(R.id.editText2);
30     et.setOnClickListener(new View.OnClickListener()
31     {
32
33         public void onClick(View v)
34         {
35             if ("Input URL".endsWith(et.getText().toString()))
36                 et.setText("");
37         }
38     });
39     Button btn = (Button) findViewById(R.id.button4);
40     wv = (WebView) findViewById(R.id.webView1);
41
42     // 按钮点击事件处理
43     btn.setOnClickListener(new View.OnClickListener()
44     {
45
46
47         public void onClick(View v)
48         {
49             // 隐藏软键盘
50             ((InputMethodManager) getSystemService(INPUT_METHOD_SERVICE))
51                 .hideSoftInputFromWindow(Main.this
52                                     .getCurrentFocus().getWindowToken(),
53                                     InputMethodManager.HIDE_NOT_ALWAYS);
54             // 设置 WebView 中的客户端的行为
55             wv.setWebViewClient(new WebViewClient()
56             {
57                 // 让 WebView 对点击网页中的 URL 做出响应
58                 @Override
59                 public boolean shouldOverrideUrlLoading(WebView view,
60                                                         String url)
61                 {
62                     view.loadUrl(url);
63                     et.setText(url);
64                     return true;
65                 }
66
67                 // 网页下载开始
```




```
68         @Override
69         public void onPageStarted(WebView view, String url,
70                                   Bitmap favicon)
71         {
72             if (pd == null)
73                 pd = ProgressDialog.show(Main.this, "网页装载",
74                                         "Page Loading");
75             else
76                 pd.show();
77
78             super.onPageStarted(view, url, favicon);
79         }
80
81         // 网页下载结束
82         @Override
83         public void onPageFinished(WebView view, String url)
84         {
85             if (pd != null)
86                 pd.hide();
87             //让 WebView 获得焦点，这样里面的 EditText 才能获得焦点
88             ww.requestFocus();
89             ww.requestFocusFromTouch();
90
91             super.onPageFinished(view, url);
92         }
93     });
94     ww.loadUrl(et.getText().toString());
95 }
96 });
97 }
```